System Design Patterns — Vol 2: The Patterns

Modules 1-11: moving data, storing data, caching, reliability, scaling, consistency, API and edge, operations, batch and streaming, AI-era systems

11 modules · 60+ patterns · every lesson explained, with course diagrams and links to the originals

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes summarised from the DesignGurus course (Arslan Ahmad). Not for redistribution.

Contents

1. **Module 1 — Thinking in Patterns (Foundations)**
   - Course Introduction
   - Thinking in Patterns
   - The Five Questions
   - Back-of-Envelope Math
2. **Module 2 — Moving Data (Communication)**
   - Module Introduction
   - Request-Response
   - Message Queue
   - Publish-Subscribe
   - Event-Driven Architecture
   - Webhooks
   - Server-Sent Events
   - Bidirectional Streaming
   - Capstone - Designing a Notification System
3. **Module 3 — Storing Data (Storage and Replication)**
   - Module Introduction
   - Primary-Replica
   - Sharding
   - Consistent Hashing
   - Write-Ahead Log
   - Event Sourcing
   - CQRS
   - Capstone - Scaling a Database to 100 Million Users
4. **Module 4 — Serving Data Fast (Caching)**
   - Module Introduction
   - Cache-Aside
   - Read-Through
   - Write-Through
   - Write-Behind
   - Cache Stampede Prevention
   - Capstone - Caching the Product Catalog
5. **Module 5 — Surviving Failure (Reliability)**
   - Module Introduction
   - Timeout
   - Retry with Exponential Backoff
   - Idempotency
   - Circuit Breaker
   - Bulkhead
   - Dead Letter Queue
   - Graceful Degradation
   - Capstone: Hardening a Payment Flow
6. **Module 6 — Growing Under Load (Scaling)**
   - Vertical Scaling
   - Horizontal Scaling
   - Load Balancing
   - Auto-Scaling
   - Database Connection Pooling
   - Multi-Region and Cell-Based Architecture
   - Capstone - From 100 to 1,000,000 Requests per Second
7. **Module 7 — Keeping Data Consistent (Distributed Consistency)**
   - Module Introduction
   - Two-Phase Commit
   - Saga
   - Quorum
   - Vector Clocks
   - Capstone - Order Fulfillment, Built Twice
8. **Module 8 — The Entry Point (API and Edge)**
   - Module Introduction
   - Reverse Proxy
   - CDN
   - API Gateway
   - Backend for Frontend
   - Rate Limiting
   - Cursor-Based Pagination
   - API Versioning
   - Sidecar
   - Service Mesh
   - Authorization Service
   - Capstone - The Entry Point, Assembled
9. **Module 9 — Operating in Production (Observability and Delivery)**
   - Module Introduction
   - Health Check Endpoint
   - Distributed Tracing
   - Blue-Green Deployment
   - Canary Deployment
   - Feature Flags
   - Containers and Kubernetes
   - CI/CD Pipelines and Deployment Strategies
   - Capstone - The Zero-Downtime Migration
10. **Module 10 — Data-Intensive Systems (Batch and Streaming)**
    - Module Introduction
    - MapReduce
    - Stream Processing
    - Lambda and Kappa
    - Change Data Capture
    - Exactly-Once Semantics

- Backpressure
- Partitioned Consumption
- Inside Kafka
- Capstone - Clickstream to Dashboard

11. **Module 11 — AI-Era Systems (LLM and ML Infrastructure)**
    - Module Introduction
    - Feature Store
    - Model Serving (and Batching)
    - GPU Auto-Scaling
    - LLM Gateway
    - Semantic Caching
    - Vector Database Sharding
    - RAG Pipeline
    - Capstone — A Production RAG Chatbot at Scale

Module 1 — Thinking in Patterns (Foundations)

Before we meet any single pattern, this module sets up the *method* that every later module relies on. It covers three things. First, what a pattern actually is. Second, a fixed list of five questions you can use on any design problem. Third, four quick calculations that tell you whether a design is big enough, or bigger than it needs to be. One example runs through the whole course: an online store. It begins on a single database and grows until it serves 100 million users. Every module tackles the next problem that store hits.

Course Introduction

Original lesson: Course Introduction on DesignGurus ↗

What it is

Here is the big idea behind the course. Most large software systems are assembled from the same small collection of solutions that show up again and again. These are called **system design patterns**, and the course covers more than 60 of them. After you learn them, a strange architecture diagram no longer looks like a random pile of boxes. You can point at each part and name it ("this one is a cache, that one is a queue"). You can also build new systems by snapping together pieces you already understand, rather than memorizing entire diagrams.

> **Analogy:** strong chess players don't memorize every game in history. Instead, they learn tactics and positions that keep coming back (forks, pins), and they spot them in games they've never seen. Patterns play the role of forks and pins in system design.

Key terms

- **System design pattern:** a solution you can reuse for a problem that shows up over and over, bundled with the price you pay for using it.
- **Pattern vs technology:** a technology is one particular product, while a pattern is the idea that product carries out. *Redis* is a technology: a data store that keeps its data in memory. *Cache-aside* is a pattern. You hold a quick, temporary copy of data you use often (that copy is a **cache**). You look there first. You only go to the slower database when the copy doesn't have what you need (that situation is a **cache miss**). Products rise and fall over the years, but the pattern keeps its value.
- **Server / database / HTTP request:** these are all you need to know going in. A server is a program that answers requests arriving over the network. A database keeps data safely so it survives restarts. An HTTP request is the usual message a client sends to a web server. (HTTP is the standard language browsers and apps use to talk to web servers, and a **client** is whatever program sends the request: a browser, a phone app, or another server.) If any of these feel shaky, the System Design Fundamentals primer walks through them from scratch.

Who it's for

The course targets software engineers in the early or middle part of their careers. Engineers who build data or AI systems benefit as well: their work is spread over many machines, so they run into the same problems. You don't need any background in **distributed systems** (systems made of many computers that cooperate over a network and must keep working when some of them fail). Each term gets explained the first time it shows up.

The twelve modules

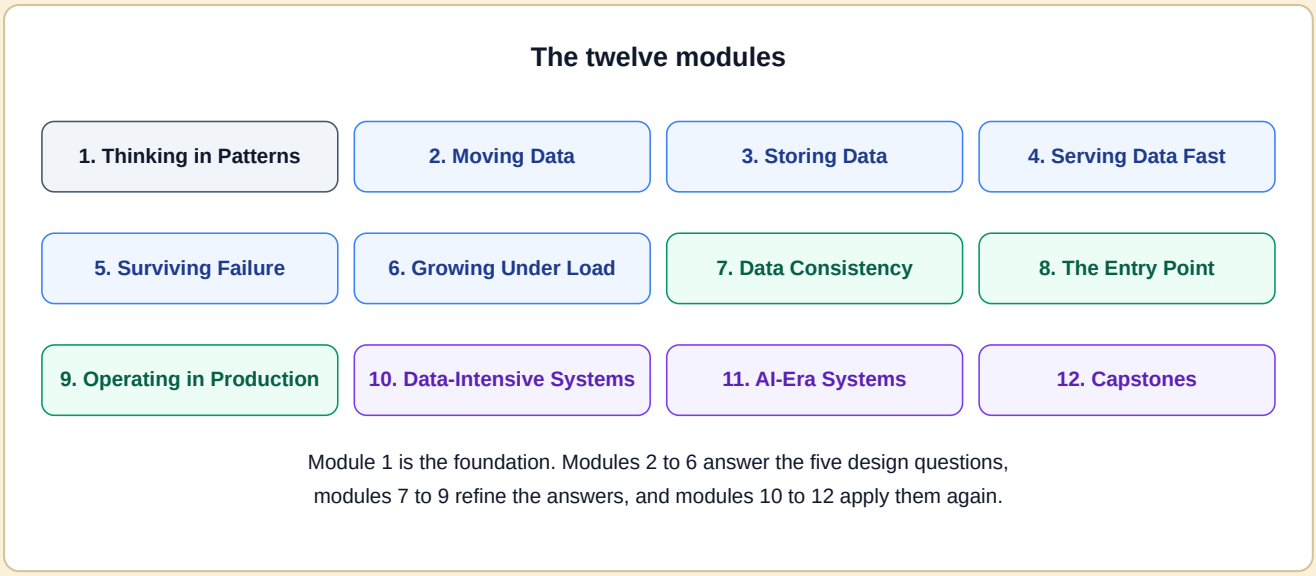


Figure — The twelve modules (course diagram)

The figure above is a map of the course. Module 1 lays the groundwork. Modules 2–6 each tackle one of the five design questions (you'll meet them below). Modules 7–9 sharpen those answers. Modules 10–12 put everything to work again on larger systems.

Module	What it covers
1. Thinking in Patterns	The meaning of a pattern, plus the simple math that shows up everywhere
2. Moving Data	The ways services talk to each other: request-response, queues, pub/sub, streams
3. Storing Data	Replication (keeping copies of data on several machines), sharding (splitting data across machines) and logs: making a database bigger and keeping it running
4. Serving Data Fast	Caching, and the price you pay for it
5. Surviving Failure	Dealing with calls that hang, fail, or arrive two times
6. Growing Under Load	Scaling, load balancing, connection pooling
7. Keeping Data Consistent	How machines reach agreement: transactions (all-or-nothing groups of changes), sagas (multi-step workflows with undo steps), quorums (majority votes)
8. The Entry Point	How traffic gets in: gateways (the front door for all requests), rate limits (caps on requests per second), CDNs (servers near users that hold copies of content), proxies (middlemen that forward requests)
9. Operating in Production	Shipping, watching, and safely undoing changes (deploying, monitoring, rolling back)
10. Data-Intensive Systems	Processing big volumes of events in batches (large chunks at a time) and in streams (one by one as they arrive)
11. AI-Era Systems	The very same patterns, used again inside LLM (large language model, like ChatGPT) and ML (machine learning) systems
12. Capstones	Four full designs that pull everything together

Added in these notes (not part of the course): five extra lessons in modules 6, 8, 9 and 10, and six case-study modules — 13 Building Blocks, 14 Messaging and Media, 15 Location and Real-Time, 16

Infrastructure, 17 Money and Reservations and 18 Client and Mobile — that apply the same patterns to the classic "design X" prompts.

How each lesson works

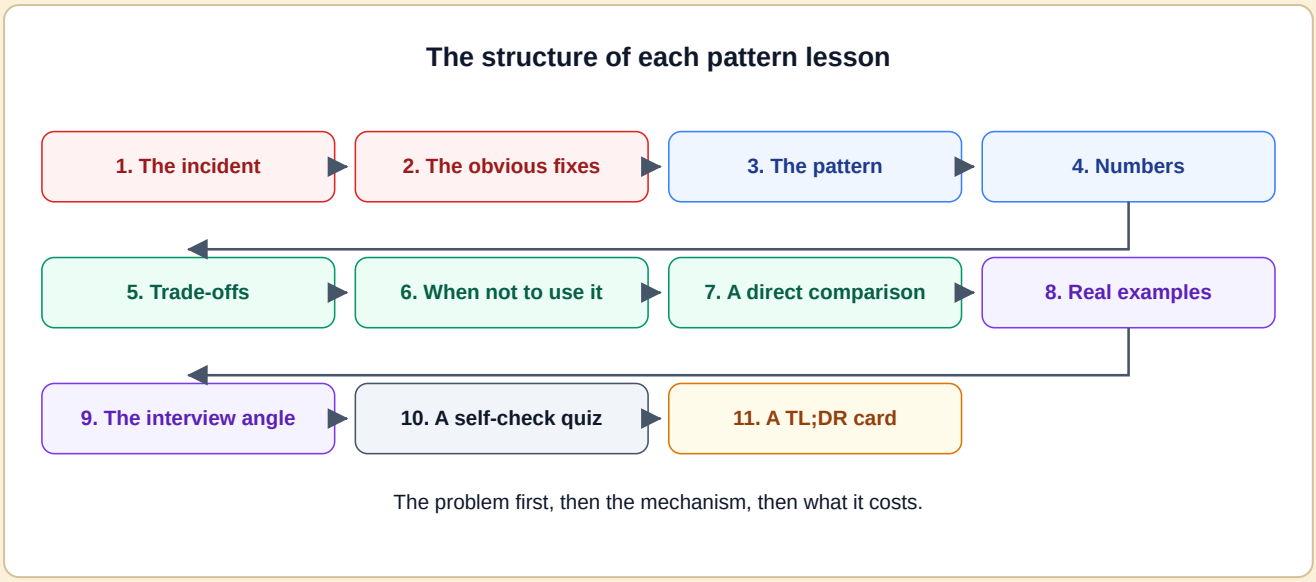


Figure — The structure of each pattern lesson (course diagram)

Every pattern lesson uses the same 11-step layout shown in the figure. The order is always: the problem, then how the fix works, then what it costs.

1. **The incident:** a believable failure in **production** (the live system that real users are using, as opposed to a test setup), with real numbers, so you sense *why* the pattern matters.
2. **The obvious fixes:** the things engineers tend to try first, and the reason they fall short.
3. **The pattern:** the solution, laid out one step at a time.
4. **Walkthrough with numbers:** the pattern applied to that same incident, with the gain worked out.
5. **Trade-offs:** the extra cost, delay or complexity the pattern brings.
6. **When not to use it:** situations where it brings more complexity than benefit.
7. **Direct comparison** against the alternative it's most often confused with.
8. **Real-world examples** taken from systems running in production.
9. **Interview angle:** how interviewers probe it, along with a rehearsed 30-second answer.
10. **Self-check quiz.**
11. **TL;DR card** ("too long; didn't read"): a brief summary. Put all the cards together and you get a review deck for the evening before an interview.

Lessons come in two sizes, full and compact. A simpler pattern just has fewer decisions to cover. At the end of each module there's a **mini-capstone**: a small system design that uses that module's patterns together.

This module

Before any pattern shows up, three short lessons build the method. **Thinking in Patterns** explains what a pattern is and why patterns work better than memorized architectures. **The Five Questions** shows how to turn a fuzzy prompt into a plan. **Back-of-the-Envelope Math** teaches four calculations you'll use again and again. The first place you actually apply the method is the Request-Response lesson that comes next.

Thinking in Patterns

Original lesson: Thinking in Patterns on DesignGurus ↗

What it is

This is a style of reasoning about design. You don't try to recall entire architectures. Instead, you split a system into *problems you already know*, choose the usual solution for each one, and say out loud what each solution costs. **Analogy:** a doctor doesn't remember every single patient. They notice symptoms, link them to known conditions, and know the side effects of each treatment.

The problem: two candidates, one question

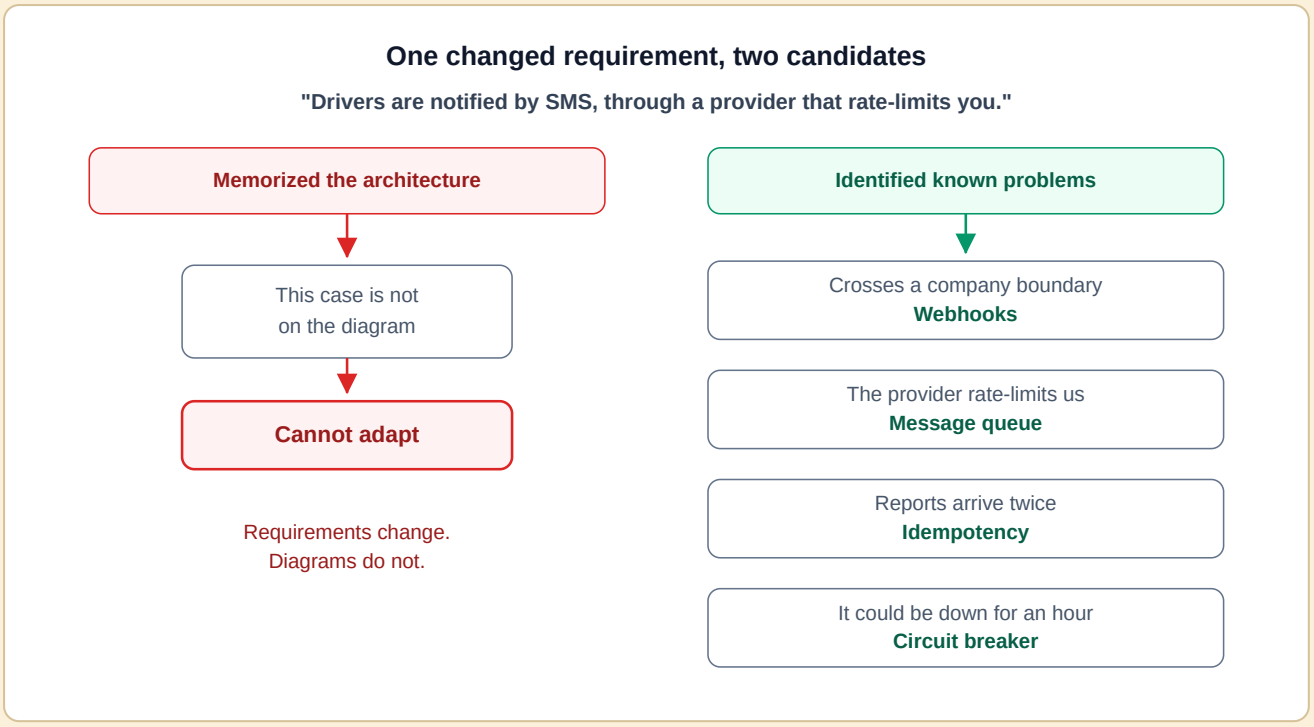


Figure — One changed requirement, two candidates (course diagram)

Picture two candidates who both get the prompt "design a ride-sharing service." The first one has memorized the architecture Uber published and draws it back faithfully. Then the interviewer tweaks a single requirement: *alerts to drivers are now sent as SMS through an outside provider, and that provider limits how fast you can send.* Nothing on the memorized diagram covers this, so the first candidate is stuck.

The second candidate splits the change into four problems they've seen before, just as the figure above shows:

- **The work crosses into another company** (a third party is now involved). The provider will send delivery reports back as **webhooks**, which are HTTP requests the provider makes into *your* server.
- **The provider rate-limits you**, meaning it caps the number of requests you may send each second. So messages should be placed on a **message queue** and sent out at a steady, controlled speed.
- **A report might show up twice**, so the handling has to be **idempotent**. That word means doing the same thing twice leaves the same result as doing it once.
- **The provider might be offline for an hour**, so a **circuit breaker** should pause calls to it for a while, and the app should alert users inside the app instead.

The second candidate puts four patterns together and states the cost of each one. When another requirement changes, they just run the same reasoning again. The figure sums it up: *requirements change; diagrams do not.* This matters on the job as well, since requirements keep shifting after a product launches.

What a pattern is

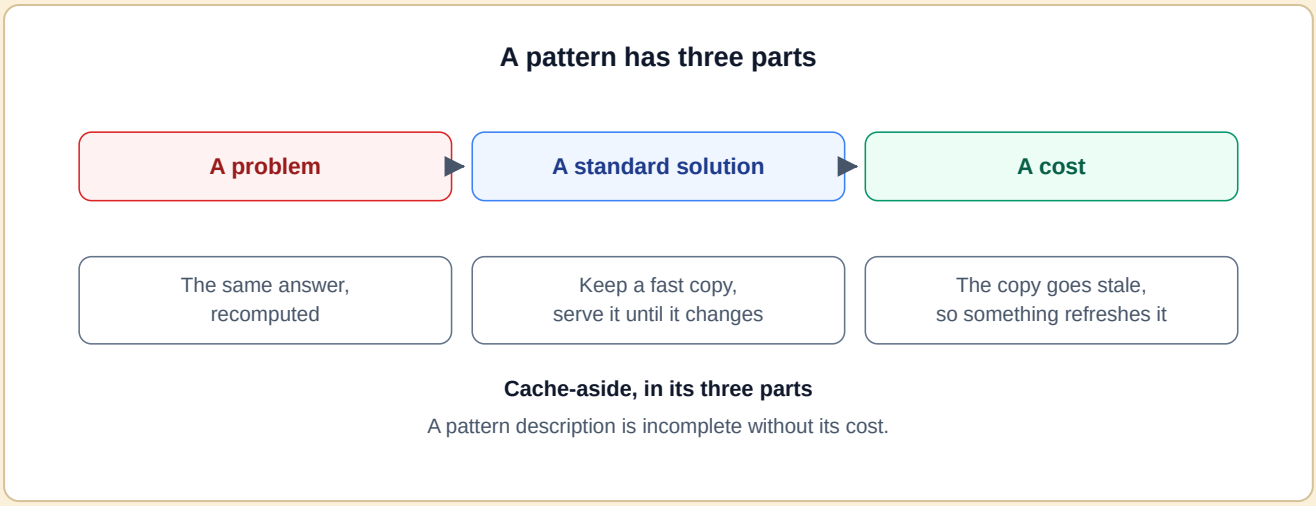


Figure — A pattern has three parts (course diagram)

The figure uses cache-aside to show that every pattern needs three parts. If any one is missing, the description isn't complete:

1. **A problem, described in general terms.** "Instagram is slow" talks about a single product. Put in general terms, it becomes "the same unchanged answer is being worked out again thousands of times." Any product can have that problem.
2. **A standard solution.** This is what seasoned engineers normally do. Here: hold a quick copy of the answer and hand it out until the answer changes.
3. **A cost.** The copy can go **stale**, meaning out of date, so something has to refresh it. This cost belongs to the pattern itself. It isn't a detail you leave for later.

Tools like Redis, Kafka (a popular message-log product) or Postgres (a popular relational database) are not part of what defines a pattern. They are simply products that happen to carry out patterns well right now.

Patterns existed long before software. Banks kept paper ledgers, which is event sourcing: you record every change and work out the balance from them. Ships have bulkheads, walls that keep one flooded compartment from sinking the whole ship. Post offices had a way of handling mail that couldn't be delivered, which is a dead letter queue. Roughly sixty patterns cover so much because every system runs into the same limits. Networks break. Every machine has a maximum capacity. Copies of data slowly stop matching. Load shows up unevenly. Since the problems keep repeating, the solutions keep repeating too.

Patterns combine into architectures

An architecture is just several patterns working together. Take Netflix: it's roughly fifteen patterns cooperating. Caches reduce repeated reads. Queues keep one **service's** work separate from another's (a service is a separately running program that owns one job, such as payments or search). Circuit breakers guard calls to the services Netflix depends on (its **dependencies**: the other systems it needs in order to do its job). Replicas keep extra copies of the data. That means a "design X" question breaks down into three smaller steps:

1. **Find the problems.** A news feed has a **fan-out** problem, where one post must reach many followers. It also has a **hot-key** problem, where a single item draws an enormous share of the traffic. Checkout, on the other hand, has problems around payments and calls to third parties.
2. **Choose a pattern for each problem, then check how the patterns affect each other. Retries** (automatically sending a failed request again) need idempotency, or else a retried payment bills the customer twice. Caches need a rule for when entries expire. **Shards** are pieces of a database spread across machines, and they need a **shard key**: the field that decides which piece stores each row.
3. **Name every cost**, and say no to a pattern when the problem isn't big enough to deserve it.

Each module finishes with a capstone that runs these steps on a small system. The last module runs them on four full designs and names every pattern involved.

Every pattern is a trade

If you keep one habit from this course, make it this one. No pattern makes everything better:

Pattern	What it gives	What it costs
Queue	Soaks up bursts of traffic	Adds waiting time; a message might arrive twice
Cache	Quick reads	Might hand out out-of-date data
Sharding	Room for more writes	Queries that span several shards become difficult
Circuit breaker	Gives up quickly instead of hanging	Might "trip" even though the dependency could still have replied

This is the reason each lesson includes a "When Not to Use It" section. Look at two interview answers side by side. The weak one: "We'll use Kafka here." The strong one: "A queue is a good fit, because no one is waiting on this work. Queues can deliver the same message twice, so the **consumer** (the program that reads messages off the queue) filters out duplicates. They can also change message order, and this operation is fine with that." The strong answer covers the **problem, pattern and cost**. Junior engineers know what patterns do. Senior engineers also know the price of each one and when to turn it down.

How interviews test patterns

Interviewers seldom ask you to define a pattern. They check three things instead:

- **Spotting problems from their symptoms.** For example: every **health check** (an automatic "are you alive?" probe a system sends to each server) is green, but one feature is starved of resources. You should think "bulkhead" (give each feature its own separate pool of resources) before the interviewer says the word. This is why lessons open with an incident.
- **Interactions between patterns.** "Could a retry charge someone twice?" "What happens right when your busiest cache key expires?"
- **Defending your choices.** When they ask "Why not use X everywhere?", they really mean "what does X cost?" Answers that skip the trade-offs sound memorized.

How to study

- **Train yourself to recognize, not recite:** when you hit a lesson's incident, stop and work out the diagnosis before you read the fix.
- **Pair every pattern with its cost** in a single sentence. The TL;DR cards help with this.
- **Practice on systems you know:** run the five questions (coming in the next lesson) on apps you use every day.
- **Say the cost out loud** in the same breath every time you name a pattern.

Interview one-liner

"I don't recall architectures. I split the system into problems I already know, choose one pattern per problem, look at how those patterns affect each other, and name the cost of each. A changed requirement just means rerunning the same reasoning."

TL;DR

- **The claim:** roughly sixty patterns keep showing up under every system design. Knowing them works better than memorizing architectures, because requirements move while a memorized diagram can't.
- **A pattern is:** a problem + a standard solution + a cost. Without all three, it's incomplete. Technologies carry out patterns; they don't take their place.
- **Designing means:** find the problems, pair each with a pattern, check how they interact, and say every cost aloud.
- **The senior habit:** every pattern is a trade-off. Interviews reward you for knowing the price and knowing when to decline.

- **Study by:** diagnosing incidents before you read the fixes, learning costs from the TL;DR cards, and never naming a pattern without naming its cost.

The Five Questions

Original lesson: The Five Questions on DesignGurus ↗

What it is

This is a checklist of five questions, always asked in the same order, that you can point at any system. Each question brings problems to light, and each problem leads you to patterns. **Analogy:** think of a pilot's checklist before takeoff. The pilot already knows how to fly. The checklist just makes sure nothing is forgotten when things get stressful.

The problem

The interviewer says "Design a URL shortener" (a service like bit.ly that turns a long web address into a short code and redirects people who visit it), passes you the marker, and goes quiet. Most candidates freeze here. It's not that they don't know enough; it's that they have no procedure to follow. When your questions come in a fixed order, you always know what to do next. The first lesson was about reasoning with patterns. This one shows *where to look* to find them.

Step 0 — get the numbers first. Before starting the questions, pin down a few numbers: how many users there are, the **RPS** (requests per second), how much data there is, the **read:write ratio** (how many reads happen for every write), and how much an outage costs. A design that suits 100 users won't suit 100 million. (The next lesson teaches you to estimate these quickly.)

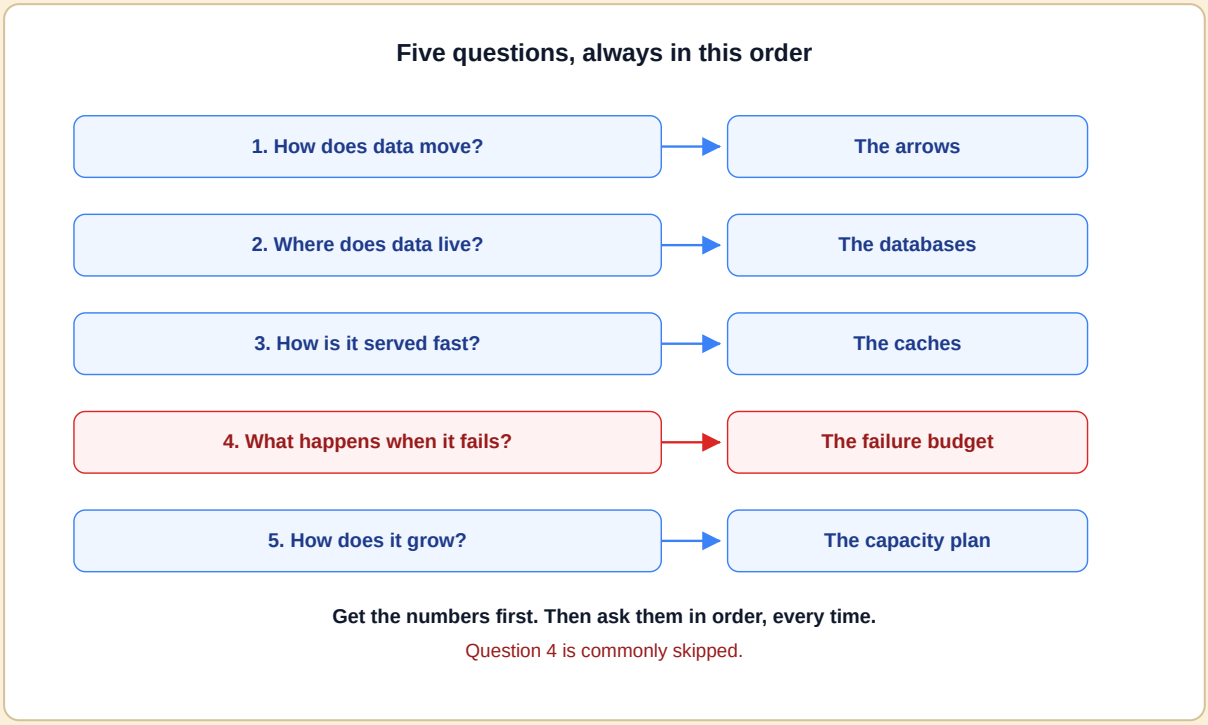


Figure — Five questions, in order (course diagram)

The figure above lists the five questions and which part of the diagram each one fills in: the arrows, the databases, the caches, the failure budget, and the capacity plan. Pay attention to its warning: people skip question 4 more than any other.

The five questions, one by one

1. How does data move? → the arrows. Figure out who talks to whom, and whether the caller needs to wait for an answer. Does the work have to happen exactly once? Do several services care about the same fact?

Does the call go outside your company? Does data need to flow without stopping? Your choices (all covered in Module 2):

- **Request-response** (the default): the caller sends a request and waits until the reply comes back.
- **Message queue**: work that isn't urgent is held until a worker picks it up.
- **Pub/sub** (publish-subscribe): a single event is sent to several services that care about it.
- **Webhooks**: one company's system makes HTTP calls into a different company's system.
- **Streams**: the server keeps pushing data without stopping.

Example: "The order confirmation has to be immediate, so it's a direct call. Fulfillment can start a bit later, so it goes through a queue. Five teams care about every order event, so we publish it to all of them."

2. Where does data live? → the databases. Figure out which store holds the **source of truth** (the official copy everyone trusts), how large the data will get and how quickly, what the read:write ratio is, and whether you need the full history or only the current state. Your choices (Module 3):

- **Replicas**: extra copies of a database, which you add once reads are too much for one machine.
- **Sharding**: splitting the rows over several machines, which you do once writes or total size are too much for one machine.
- **Consistent hashing**: a technique that lets you change how many shards you have without moving most of the data.
- When history matters, you use log-based patterns. A **write-ahead log (WAL)** records each change first and applies it after. **Event sourcing** stores the complete list of events instead of just the final result. **CQRS** keeps one model for writes and a different one for reads.

Example: "Order history matters, since customers dispute charges. So we keep the events, not only the current state."

3. How is it served fast? → the caches. Figure out which reads happen again and again, how out of date each kind of data is allowed to be, what the **hit ratio** is (the share of reads the cache answers), and what breaks when a heavily used entry expires. Begin with **cache-aside**. Switch to write-based variants for data that changes often. Add **stampede prevention** for when many requests miss the same hot entry at the same moment. The key decision in the caching module is this: give each type of data a **staleness budget** (how old it may be), then choose a mechanism that keeps to it.

Example: "Product descriptions are fine if they're an hour old. Prices can be a few seconds old, as long as updates **invalidate** the cache (delete or mark the old cached copy so the next read fetches the new value). At checkout the price must be exact, so that step always reads from the database."

4. What happens when it fails? → the failure budget. Go through *every arrow* from question 1. For each one, ask: what if the other side never answers, fails once, or stays down for ten minutes? Could a retry cause an effect to happen twice, such as a charge? What will the user see? Your toolbox (Module 5):

- **Timeout**: give up waiting once a deadline passes.
- **Retry with backoff**: try again, waiting a bit longer before each new attempt.
- **Idempotency**: sending the same request again can't cause the effect again.
- **Circuit breaker**: stop calling a service while it's down for a long stretch.
- **Bulkhead**: stop one broken feature from eating all the shared resources.
- **Graceful degradation**: send back a smaller result rather than an error.

Example: "If the payment provider slows down, we cut off the call once it takes twice its usual worst-case **latency** (the time one call takes from start to finish). Retries include **idempotency keys** (a unique ID attached to a request so the receiver can spot and ignore a repeat), so a customer can't be charged twice. If failures keep happening, the breaker stops the calls, and we save the cart and email the customer."

5. How does it grow? → the capacity plan. If traffic grows 10x, which part breaks first: reads, writes, storage, or a single hot key? Which parts grow just by adding machines, and which need a redesign? Module 6 covers **load balancing** (spreading incoming requests across many servers), **autoscaling** (adding or removing servers automatically as traffic changes), and **horizontal scaling** (growing by adding machines).

Example: "Replicas and caching take care of reads, but the single **primary** (the one database server that accepts writes; replicas only copy from it) caps how many writes we can do. The next step is sharding by customer ID, and we plan it before the primary hits its limit."

The five questions line up with Modules 2–6 in the same order. Later modules add consistency, the entry point, and operations. The data and AI modules run the same questions on **pipelines** (chains of steps that move and transform data) and ML systems.

Walkthrough: the URL shortener in about three minutes

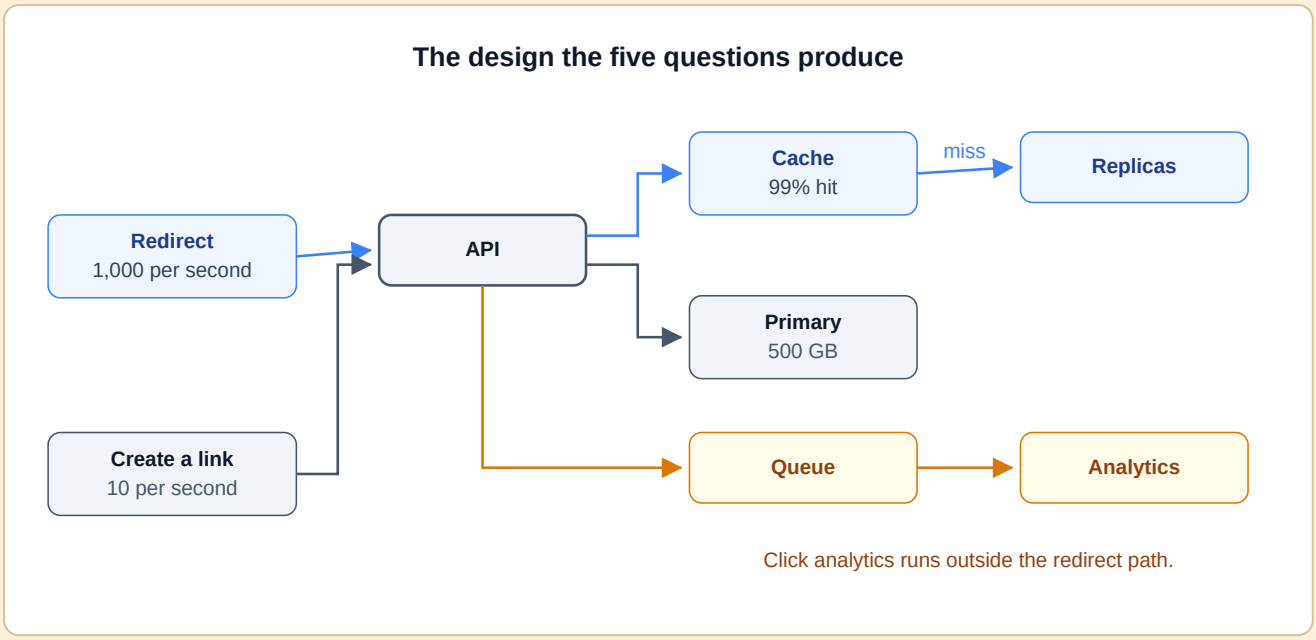


Figure — The design the five questions produce (course diagram)

The figure above shows where the walkthrough ends up. Redirects arrive at 1,000/s and hit a cache with a 99% hit ratio. Misses fall through to replicas that sit behind a 500 GB primary. Creating links (~10/s) goes to the primary. Click analytics travels through a queue that sits *outside* the redirect path.

- **Move:** there's one write path (send a long URL, receive a short code) and one redirect path that's mostly reads, at roughly 100 reads for every write. Both use direct calls. Click analytics is extra work nobody waits on, so it goes onto a queue and never slows a redirect down.
- **Live:** each row is tiny (two URLs and a counter), but there could be billions of them. For now, a primary with replicas is enough. Once rows reach the billions, shard by short code, since every redirect lookup already includes it. There's no need to keep history.
- **Fast:** a given short code always points to the same URL, so use cache-aside with a long **TTL** (time-to-live: how long an entry stays in the cache). A small number of links get most of the clicks, so those hot links get stampede prevention and shouldn't expire during normal operation.
- **Fail:** redirects need very high **availability** (the share of time a service is up and answering correctly), so a cache miss reads from a replica. Link creation can reply "try again shortly," because a short outage there does less harm. Each path gets its own reliability target.
- **Grow:** replicas and cache nodes handle more reads, and there aren't many writes. The real risk is **key generation**. Random codes need a check for **collisions** (two links accidentally getting the same code). Codes based on a counter need coordination so that two servers never hand out the same code. Mark it as future work and offer both options.

After three minutes you've got: separate read and write paths, storage with a shard key picked out, a caching plan, a reliability target for each path, and one growth risk flagged. If a requirement changes, just run the questions again.

Usage tips

- **Announce the questions as your agenda** at the start: "First how data moves, then storage, then caching, then failures, then growth." That way the interviewer knows your plan.

- **Treat them as a checklist** when you're stuck or when you finish with time left.
- **Question 4 gets skipped the most**, and it triggers the most follow-up questions. Ask it for every arrow.
- **Bring them to design reviews** at work too. "What if the other service never answers?" ranks among the most useful questions in distributed systems.

Interview one-liner

"After getting the numbers, I go through five questions in order: how data moves, where it lives, how it's served fast, what happens when it fails, and how it grows. Each surfaces problems that map to patterns, and the fixed order means nothing gets skipped."

TL;DR

- **Procedure:** get the numbers first, then ask five fixed questions (move, live, fast, fail, grow).
- **Why it works:** the questions uncover problems, the problems lead to patterns, and the fixed order makes sure you cover everything.
- **Structure:** the questions line up with Modules 2–6; later modules polish the answers.
- **Most-skipped:** Q4, handling failure. It's where designs become dependable, and where interviewers push hardest.
- **Use it as:** the agenda you say out loud, your checklist when stuck, and your go-to question in reviews at work.

Back-of-Envelope Math

Original lesson: Back-of-Envelope Math on DesignGurus ↗

What it is

These are fast, intentionally rough calculations (the kind you could scribble "on the back of an envelope"). They tell you the *order of magnitude* a system needs. Is it one machine or ten? Are 600 requests in progress at once, or 60,000? Can the data fit in memory? **Analogy:** before a road trip you guess "around 5 hours and two tanks of gas." You don't work out the arrival time to the minute. That rough guess is enough to choose between driving and flying.

Why engineers estimate

A rough answer you get in 30 seconds can still shape the design today. A precise answer that arrives next week can't. Without numbers, you can't really answer the five questions, and interviewers often ask things like "how much storage does this need?" They expect you to state your assumptions, round the numbers, and show each step.

Round hard. There are 86,400 seconds in a day; call it **100,000**. There are about 2.5 million seconds in a month; call it **3 million**. You're estimating **orders of magnitude** (powers of ten), not decimal places. When the direction of rounding matters, round demand up and capacity down. That gives you some spare room (headroom).

Numbers to memorize

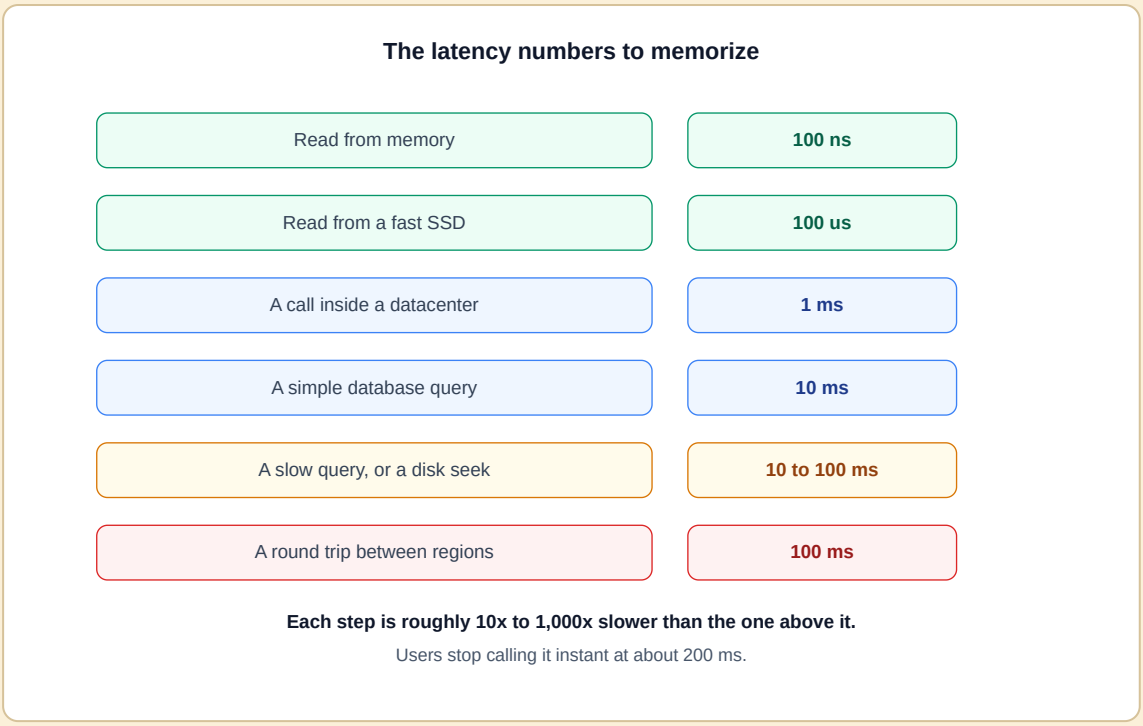


Figure — The latency numbers to memorize (course diagram)

Latency means how long an operation takes. The figure above gives the standard values. Each row is roughly 10x to 1,000x slower than the row above it:

Operation	Time
Reading from memory (RAM, the computer's fast working memory)	~100 ns (nanoseconds)
Reading from a fast SSD (solid-state drive, flash storage)	~100 µs (microseconds)
A network call within one datacenter (a building full of servers), e.g. a cache read	~0.5–1 ms (milliseconds)
A simple database query	~5–15 ms
A seek on a spinning disk, or a slow query	~10–100 ms
A round trip (request there, reply back) from one region (a geographic area with its own datacenters) to another	~100 ms
The longest delay that still feels "instant" to users	~100–200 ms

This table leads to three design facts:

- A cache hit (~1 ms) runs ~10–15x faster than a DB query (~10–15 ms). This is why caching helps.
- Writing to disk in order (sequentially) is much faster than writing to scattered spots on the disk. This is why the write-ahead log pattern works.
- Every *sequential* round trip between regions adds ~100 ms, so just a few of them use up the whole response-time budget.

Sizes: one character ≈ 1 byte; an ID ≈ 16–36 bytes; a typical DB row ≈ 1 KB; an image ≈ 100 KB–1 MB. Each unit is 1,000x bigger than the one before (KB → MB → GB → TB). So 1 million rows of 1 KB ≈ 1 GB, and 1 billion of them ≈ 1 TB.

The four calculations

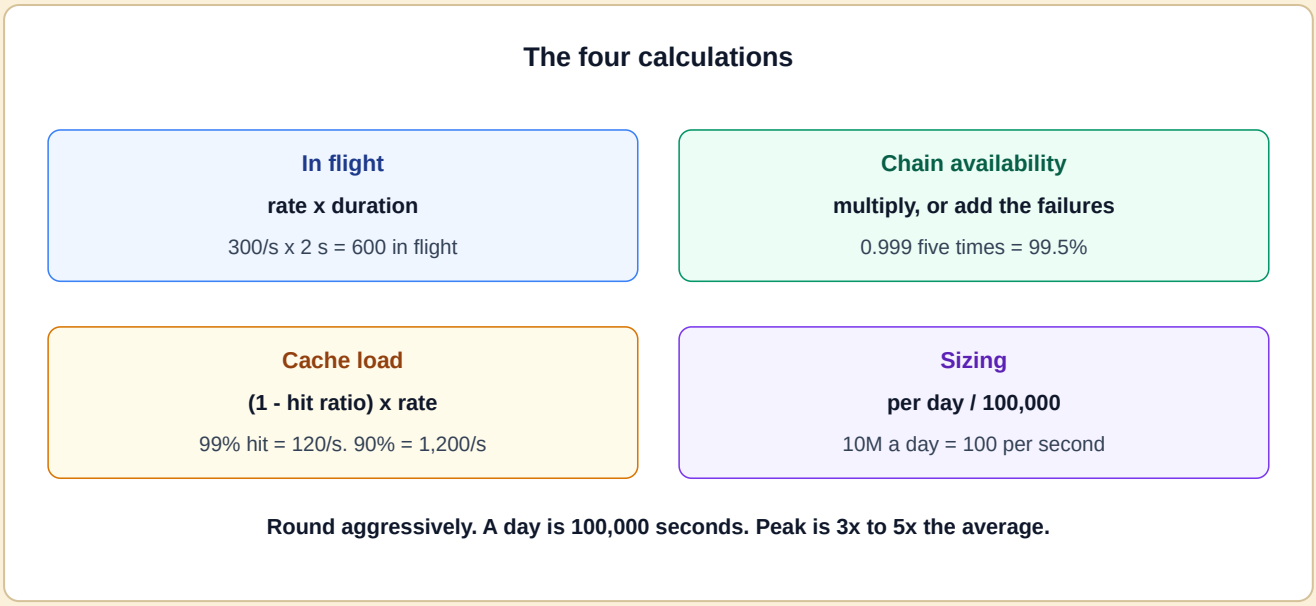


Figure — The four calculations (course diagram)

The figure above sums up the four formulas, with one example for each. Here they are in more detail:

1. Rate × duration = work in progress (Little's law). `things in flight = arrival rate × duration`. Say 300 requests come in every second and each one takes 2 seconds. Then 300 × 2 = **600 requests are active at any moment**, and each one is holding a **thread** (one independent line of execution inside a program; a server runs many at once) or a database connection. A **connection pool** is a fixed set of open DB connections that get reused; one with 200 slots would be too small here. The same formula gives you a queue's **drain time**, meaning how long it takes to clear a backlog. With 552,000 messages processed at 400/s: 552,000 / 400 ≈ 1,400 s ≈ **23 minutes**. You'll use this calculation more than any other in the course. It sizes thread pools, connection pools, bulkhead limits and backlog drain. Its official name, **Little's law**, is a good term to drop in interviews.

2. Availability drops along a chain. Availability is the share of time a service answers correctly. When one request depends on several parts in a row, you multiply their availabilities together. The answer always ends up lower than the weakest part. With five dependencies at 99.9% each: $0.999^5 \approx$ **99.5%**. A shortcut when failure rates are small: just *add* them (5 × 0.1% ≈ 0.5% failure). Turning these into time makes them easier to picture:

Availability	Downtime per year	Per month
99%	~3.7 days	~7 hours
99.9%	~9 hours	~43 minutes
99.99%	~53 minutes	~4 minutes
99.999%	~5 minutes	~26 seconds

Two takeaways. First, every extra "nine" takes about **10x** the engineering work of the one before it. Second, every dependency you require can only *pull the total down*. So the practical fix is to have **fewer required dependencies**, not to make each one perfect. The graceful degradation lesson builds on this idea: when an optional dependency fails, you still return an answer that's smaller but useful.

3. Cache hit ratio. Let *h* be the hit ratio (the share of requests the cache answers). Then:

- `average latency = h × fast + (1 − h) × (fast + slow)` (a miss pays for checking the cache *and* for the DB)
- `load on the database = (1 − h) × request rate`

Example: 12,000 RPS, a 1 ms cache hit, and a 15 ms DB read.

- h = 99% → ~1.15 ms on average, and the DB receives 1% × 12,000 = **120 QPS** (queries per second).
- h = 90% → ~2.5 ms on average, and the DB receives **1,200 QPS**, which is ten times the load.

Dropping from 99% to 90% doesn't mean "9% worse." The *miss* rate jumps from 1% to 10%, so the database gets 10x the traffic. Always judge a cache by its misses. This also explains a **cache stampede**. When a hot entry expires, its hit ratio briefly drops to about zero, and almost every request hits the database at the same moment.

4. Sizing.

- `per second = per day / 100,000`. So 10M orders/day ≈ **100/s average**.
- Traffic isn't steady, so plan for a **peak of 3–5x average**. Here that's 300–500/s.
- `storage = records × size × retention`. 10M/day × 1 KB × 365 days ≈ **3.7 TB/year**. That fits on a single ordinary database server, so this estimate *keeps* you from sharding when you don't need to.

Walkthrough: sizing the URL shortener (~60 seconds, five numbers)

Speak each assumption out loud as you work:

- "Let's assume **100M redirects/day**. Divide by 100,000 and you get **1,000/s** on average, so plan for about **4,000/s** at peak."
- "Each mapping holds two URLs plus some **metadata** (descriptive extras such as creation date and owner); let's say **1 KB**. With **500M links**, that comes to roughly **500 GB**. A single database can handle that today, so sharding can wait."
- "There are roughly 100:1 more reads than writes, and a handful of links get most of the traffic. If we cache the top 1%, that's **5M rows ≈ 5 GB**. At a 99% hit ratio, only around **10 queries/s** make it to the DB."
- "At peak, 4,000/s × 2 ms = **8 requests in flight**. So we need one strong database, one replica, a cache, and careful failure handling."

Now every component has a capacity-based reason to exist, and nothing is built for demand that hasn't shown up yet.

Traps

- **90% is not close to 99%:** it means 10x as many misses.
- **Each extra nine costs ~10x.** Long chains of dependencies can only reduce availability, so keep them short.

Using estimation in interviews

- **Say your assumptions before you calculate** ("assuming 1 KB per record and 10M records a day"). An interviewer can fix an assumption you say aloud, but not one that stays in your head.
- **Round first, then multiply, and check the units at the end.** A rate × seconds gives a count; bytes × rows gives storage. If the units come out wrong, so does the math.
- **Let the numbers remove components.** Saying "At this scale we don't need that yet" is a strong answer. It shows you care about cost, and the arithmetic backs you up.

Interview one-liner

"I round hard (100k seconds per day, a gigabyte per million 1 KB rows, peak at 3–5x average) and use four formulas: rate × duration for concurrency, multiplied availabilities for chains, hit ratio for cache latency and DB load, and records × size × retention for storage. That tells me what the design needs and what it doesn't need yet."

TL;DR

- **Rule:** round hard; a day = 100,000 s; 1M × 1 KB = 1 GB; peak = 3–5x average.
- **Table:** RAM ~100 ns, SSD ~100 µs, a hop within the same datacenter ~1 ms, a database query ~10 ms, across regions ~100 ms, "instant" stops at ~100–200 ms.
- **Four calculations:** in flight = rate × duration; availability of a chain = multiply (or add the failure rates); cache latency and DB load come from the hit ratio; per second = per day / 100,000 and storage = rows × bytes × retention.
- **Traps:** 90% ≠ 99% (10x the misses); each nine ~10x the cost; chains can only lower availability.

- **In interviews:** state your assumptions, check your units, and let the arithmetic rule out over-engineering.

Module 2 — Moving Data (Communication)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module covers

Any system design diagram is made of boxes and arrows. The boxes are services, such as an **API** (application programming interface: the set of requests a program accepts from other programs), a database, or an email sender. The arrows show data traveling between them. Most engineers pay attention to the boxes. This module is about the **arrows**, because each arrow hides a design decision. Any time service A sends something to service B, you pick *how* it gets there. That pick decides three behaviors:

- **Waiting:** does someone have to sit and wait for the answer?
- **Downtime:** what happens if B is offline?
- **Duplicates:** what happens if the message shows up two times?

Getting these choices wrong leads to real outages. Here are two from the lessons coming up. A **flash sale** (a short, heavily advertised sale that makes huge crowds arrive at the same moment) took down a store even though the store had enough capacity overall. And a "real-time" dashboard burned most of its budget asking "anything new?" only to be told "no."

The seven patterns

Situation	Pattern
The caller can't go on until it has the answer	Request-Response
The work can be done later, and no one is waiting on it	Message Queue
Something happened, and more than one service needs to hear about it	Publish-Subscribe
Pack the details into the event so no one needs to call back for them	Event-Driven Architecture
A system at another company needs to be told about something	Webhooks
A steady flow of updates going one way	Server-Sent Events
Both sides keep sending messages to each other	Bidirectional Streaming

Two rules for the whole module

- **Rule 1: request-response is the default.** When someone has to wait for the answer, make a direct call. The other six patterns are for situations where waiting is pointless or can't work. Every alternative comes with costs, so switching to one isn't automatically an improvement. Less experienced engineers often grab a fancy pattern when a direct call would be clearer and safer.
- **Rule 2: anything that retries can deliver twice.** Queues, events and webhooks send a message again whenever they can't be sure it got through. This is on purpose, because it stops data from being lost. But it means your code has to handle receiving the same message twice without doing its effect twice. That property is called **idempotency**: running an operation two times gives the same result as running it once. A later lesson covers it in full. For now, assume duplicates will happen and plan for them from the very first day.

Four terms used throughout

- **Job vs fact.** A *job* is a piece of work that should be done one time ("resize this image"). A *fact* is a record that something happened ("an order was placed"). Jobs go onto **queues**, and facts get **published**. Confusing the two is the most frequent mistake people make with communication.
- **At-least-once delivery.** The system guarantees your message will never be lost, but it might hand it over more than once (this is Rule 2).

- **Push vs pull.** With *pull*, the client keeps asking "anything new?" With *push*, the server sends updates as soon as they happen. Pull costs you requests; push costs you connections that stay open. Neither one is free.
- **Synchronous vs asynchronous.** *Synchronous* means the caller pauses and waits for the answer. *Asynchronous* means the caller passes the message along and carries on with other work.

How to read this module

The lessons track a single online store as it makes one decision after another. At first, everything is synchronous: every step waits for the one before it. A flash sale breaks that setup, which is why we bring in the **queue**. The queue does its job, but then five more teams want its data, which is why we bring in **pub/sub**. The published events hold too little information, though, so every consumer has to call back for the details. That leads to **event-driven architecture**. Next, an outside payment provider needs to send data *into* the store, which brings in **webhooks**. Read those five lessons in order. After that, **Server-Sent Events** and **Bidirectional Streaming** deal with live streams, and the **capstone** uses all seven together in a notification system.

Not much time before an interview? The four that come up most are request-response, message queue, pub/sub and webhooks.

Before you start

Pick a system you've worked on and think about these ("I don't know" is a perfectly good first answer; this module shows you how to find out):

- Which of your API calls is slowest? Is a real person waiting for what it returns?
- Suppose your code puts a message on a queue and it gets delivered twice. What happens?
- Is anything refreshed on a timer? Out of 100 refreshes, how many actually find something new?

Request-Response

Original lesson: Request-Response on DesignGurus ↗

What it is

The caller sends a request and then **waits**. It does nothing else until the answer comes back, either as a result or as an error. This is the normal way services communicate, and the other six patterns are intentional alternatives to it. **Analogy:** you ask a shop assistant something at the counter. You stay put until they reply, and you can't complete your purchase without their answer.

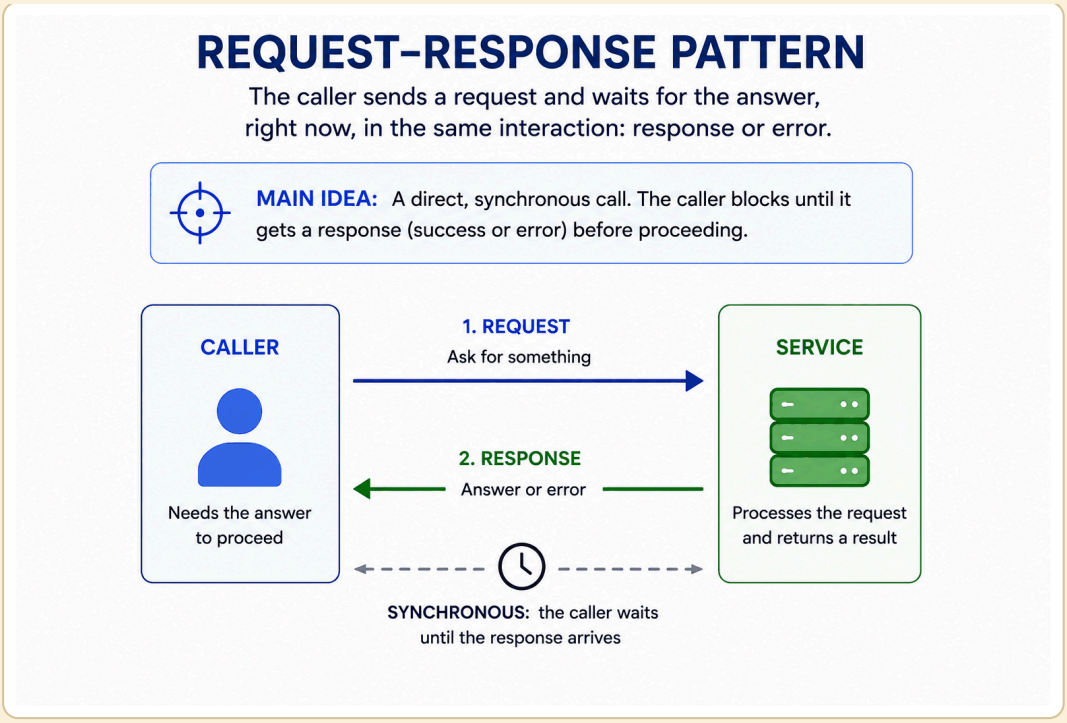


Figure — Request-Response (course diagram)

In the figure above, the caller (the one that "needs the answer to proceed") sends (1) a request. The service works on it, and then (2) a response or an error travels back. The clock drawn between them shows the most important property. The call is **synchronous**, so the caller is stuck for the entire round trip.

Why it needs a lesson

Engineers who have recently discovered queues and events often use them in places that don't need them. In most systems, a plain synchronous call is the right choice for most of the traffic. Think of loading a page, running a search, checking a password, looking up a price, or authorizing a payment. When a user is sitting there waiting for the result, adding a queue doesn't make the wait go away. It only hides the wait behind extra machinery.

What it gives you

- **You get the answer right away.** The very next line of code can use the result. You don't have to design a "we received your request, come back later" state.
- **Errors show up where you can deal with them.** If the call fails, the code that made it can retry, show a message, or fall back to something else. A background job that fails is far harder to track down, because it fails apart from the request that started it.
- **Debugging is easy.** One request travels one path and leaves one **trace** (a record of the path that request took through the services).

One small detail with a big effect on speed: reusing connections. Setting up a new connection is costly. It begins with **TCP and TLS handshakes**, which are setup messages that create the link and encrypt it (TCP is the protocol that gives a reliable, ordered connection between two machines; TLS is the encryption layer behind the padlock in `https`). The System Design Fundamentals primer covers these building blocks in more depth. They take 1–3 network round trips, which is around 100 ms between regions. **Connection pooling** (also known as **keep-alive**) leaves connections open so that later calls can reuse them. Every HTTP client used in production supports this; make sure it's really switched on.

What it costs

Each cost seems minor by itself, but a *chain* of calls piles them up:

- **Both sides have to be running at the same time.** If the receiver is down, the call fails straight away. This is known as **temporal coupling** (being tied together in time).

- **Availability multiplies along a chain.** Availability is the fraction of time a service answers correctly. Say an **endpoint** (one URL/operation a service exposes, such as `POST /checkout`) calls 5 services one after another, and each is 99.9% available. The endpoint only works when all five work: $0.999^5 \approx$ **99.5%**. That looks high, but it adds up to roughly **44 hours of failures a year**, and every extra dependency drags it lower.
- **Latency adds up. Latency** is the time one call takes from start to finish. Five 80 ms calls made one after another take **400 ms**. If the calls don't rely on each other, make them **in parallel**. Then the endpoint only waits for the slowest one (~80 ms). Turning accidental one-after-another work into parallel work is often the quickest way to speed up a slow endpoint.
- **Waiting ties up resources.** A caller that's waiting takes up a thread or a connection. At 300 requests/s, with each taking 2 s, there are 600 requests in progress at the same time, and each one holds a thread (this is Little's law from Module 1).

That's why request-response at scale needs three safeguards, each covered in its own later lesson. A **timeout** stops waiting after a deadline. **Retries** repeat a call after a short-lived failure. A **circuit breaker** stops calling a service that is obviously down.

One question per arrow

Before you draw any arrow, ask: **does the caller's next step need the answer, *and* is someone waiting for it?** If both answers are yes, make a direct call. If not:

Situation	Use
The next step needs the answer, and someone is waiting	Request-response
The work can be done later	Message Queue
Several services need to hear that something happened	Publish-Subscribe
A system at another company needs to be told	Webhooks
A steady flow of updates going one way	Server-Sent Events
Both sides keep sending	Bidirectional Streaming

Each alternative brings its own cost. It might be a **pending state** (work that's been accepted but isn't finished yet), the chance of duplicate delivery, or a connection you have to manage. Change patterns because an instant answer *isn't needed*, not because the other option sounds fancier.

When not to use it

- **Work that runs a long time** (longer than a few seconds). Don't make the caller wait. Accept the job, answer "job started" right away, and hand over the result later through a queue, a status endpoint, or a webhook.
- **Telling many services about one event.** If five services each need the same order update and you call each one separately, you tie them all together. Publish the event instead (see the Pub/Sub lesson).
- **Bursty work that has loose deadlines.** Bursty means it shows up in sudden spikes. If no one is waiting, handle it later through a queue (see the Message Queue lesson).
- **Chains that have become too long.** Once you have six synchronous calls in a row, availability has already fallen and latency has already climbed. Make the chain shorter, run calls in parallel, cache, or move work that can wait onto a queue.
- **The opposite mistake: going async out of habit.** Examples are putting search behind Kafka (a popular high-throughput message log), or making login event-driven. The user still waits, but now the wait goes through a **message broker** (a server that holds messages and passes them on). That adds latency and the risk of duplicates to a path that needs an answer in ~200 ms. Async is the right call when waiting is optional. It isn't "better" across the board.

In the interview

Interviewers seldom ask about it by name. More often they point at a synchronous arrow and ask "why is this synchronous?" A full answer names what depends on it: "Checkout can't move on without the payment result, and the user is waiting. So it stays synchronous, protected by a timeout, a limited number of retries, and a circuit breaker."

30-second answer: "Request-response is my default for anything the caller's next step depends on: auth (authentication, checking who the user is), pricing, payment authorization, interactive reads. I get the answer immediately, errors come back where I can handle them, and tracing is simple. The costs are that both sides must be up at once, availability multiplies down a chain, and latency adds up. So I run independent calls in parallel, reuse pooled connections, and wrap every call in timeout, retry and circuit breaker. Anything nobody waits on moves to a queue or an event."

Likely follow-ups:

- *"Your endpoint calls five services and is slow. Fix it without going async."* Make the independent calls in parallel, so total latency drops from the *sum* of all calls to just the *slowest* one. Cache answers that hardly ever change, check that every call is truly needed, and make sure connections get reused.
- *"When do you switch a call to async?"* When no person or service needs the answer to keep going, or when the work takes longer than a connection should be held open. Going async makes the system tougher, but it also brings pending states, duplicates and questions about order. So it's a trade-off, not an upgrade.

Mistake that fails candidates: turning a user-facing path async (like search through Kafka), or drawing six synchronous calls with nothing protecting them. Both show you don't grasp what synchronous calls cost, or when their instant answer is worth having.

Self-check

- **Q1: Which two questions decide sync vs async, and what costs does each sync call add to a chain?** → Does the next step need the answer, and is someone waiting for it? The costs are temporal coupling, availability that multiplies down, extra latency, and one thread held for each waiting caller. That's why timeout, retry and breaker are a must at scale.
- **Q2: Checkout makes five sequential 80 ms calls to services at 99.9% each. Latency, availability, and how to improve both without async?** → 400 ms and ~99.5% (~44 h/year of failed checkouts). Run the independent calls in parallel (~80–120 ms), cache answers that rarely change such as pricing rules, and drop calls you don't need. For availability, let optional parts of the page show less instead of breaking the whole page (this is graceful degradation).
- **Q3: A teammate wants payment authorization async "for resilience" via a `payment.requested` event. Assess it.** → It breaks the rule, because checkout needs the result and the user is waiting. It creates a pending state the product has to design for, and duplicate delivery could authorize a payment more than once. Protect the sync call instead: a timeout, retries that carry an **idempotency key** (a unique ID that lets the provider spot a repeat so it doesn't charge twice), a circuit breaker, and a plan for outages (save the cart, send an email once payment goes through). The part that should be async is the *receipt email*.

Related patterns

- **Timeout, Retry, Circuit Breaker:** the three safeguards. Request-response at scale isn't safe unless you have all three.
- **Message Queue:** the first alternative, used for work no one is waiting on.
- **Load Balancing:** how one logical service turns into many physical servers behind a single call.

- **API Gateway:** the place where request-response traffic from outside gets authenticated and rate-limited.

TL;DR

> - **Problem:** every arrow needs a default. Teams go wrong in both directions: making things async when someone is waiting, or piling up sync calls until latency and fragility snowball.
> - **Mechanism:** the caller waits for a result or an error. Pool your connections, run independent calls in parallel, and wrap every call in a timeout, retry and breaker.
> - **Costs:** both sides must be up together, availability multiplies down, latency adds up (when calls run in sequence), and every waiting caller holds a thread.
> - **Skip it when:** no one is waiting, the work takes too long to hold a connection open, many services care about the same event, or the chain has grown too long.

Message Queue

Original lesson: Message Queue on DesignGurus ↗

What it is

A **message queue** is a durable to-do list placed between the service that creates work (the **producer**) and the service that carries it out (the **consumer**, also called the **worker**). *Durable* means the list is saved to disk, so a crash won't wipe it out. The producer drops in a task and immediately moves on. Workers pick up tasks at whatever speed they can manage. The queue keeps each task safe until a worker says it's finished. **Analogy:** picture the ticket rail in a restaurant kitchen. Waiters clip up tickets as quickly as customers order. Cooks take them down one by one, as fast as they can cook. When there's a rush, the rail just gets longer; the kitchen doesn't blow up.

The problem: the flash sale

The store puts 10,000 signed editions on sale at 10:00 AM. Usual traffic is 200 orders/s. At 10:00:01 it jumps to **5,000 orders/s for two minutes**. The order pipeline is synchronous. While the buyer waits, the API takes the order, reserves the stock, writes to the database, and schedules fulfillment, and only then says "success." The pipeline can handle **400 orders/s**, and that was plenty for years.

Do the math. Every second, 5,000 orders come in and 400 get finished, so **4,600 per second** are left waiting, timing out, or failing. **92% of buyers get an error page**, and when they hit refresh they pile on even more load.

The postmortem (the review after an incident) uncovers the actual problem. The pipeline could have worked through 600,000 orders from those two minutes in far less than a day. Buyers didn't need their orders fulfilled within two minutes. They needed a *confirmation* right away. But confirmation and fulfillment were fused into a single synchronous call, so every bit of fulfillment work had to beat the confirmation deadline. **That tight coupling is what caused the outage.**

Why the obvious fixes fail

- **"Scale up for the peak."** Handling 5,000/s would mean running **12x** the usual number of servers all year long for an event lasting two minutes. On top of that, the database has its own ceiling, so the extra traffic would simply fail there instead. This shifts the **bottleneck** (the slowest part, which limits how much gets through) rather than getting rid of it.
- **"Buffer orders in memory."** The API holds a list and slowly works through it. That's an unsafe queue, though. If the process crashes, paid orders vanish. Other API servers can't see the list. And monitoring can't see how big it is until memory runs out.
- **"Reject the extra traffic."** Anything above 400/s gets "too busy, try later." That's okay for traffic that doesn't matter much. Here, though, it sends away **9 of 10 customers** during the quarter's biggest sale, for work the pipeline could safely have done an hour later.

All three fixes assume this is a capacity problem. It's really a **coupling** problem. Two things that should run on their own schedules (orders coming in, orders being processed) were forced to go at the same speed every single second.

How it works

There are four mechanics, and each one answers a "what if it fails?" question:

1. **The queue saves the message before it confirms.** The **broker** (the software that runs the queue) writes every message to disk, and *only after that* tells the producer it got it. From then on, a crashed worker, a **deploy** (rolling out a new version of the software, which restarts its servers), or an hour of downtime can't make the work disappear.
2. **Workers confirm when they finish, not when they start.** A worker grabs a message, does the job, and then sends an **ack** (acknowledgment). If the worker crashes in the middle of the job, no ack comes. Once a set **visibility timeout** passes, the broker gives the message to another worker. *Consequence:* a worker could finish the job and then crash right before sending the ack. The broker can't distinguish that from a crash halfway through, so it sends the message again and the job runs twice. That's **at-least-once delivery**: each message shows up one or more times, but never zero times. Protecting against lost work leads to the occasional repeated job, so every consumer needs to be **idempotent**.
3. **Many workers pull from the same queue (competing consumers).** Each message goes to just one worker. Want more **throughput** (messages finished per second)? Add more workers. You don't need to write any special coordination code.
4. **Queue depth tells you how healthy things are.** **Depth** is the number of tasks waiting (what's arrived minus what's been finished over time). If depth climbs during a brief spike, the buffering is doing its job. If depth climbs for a *week*, you have a capacity problem.

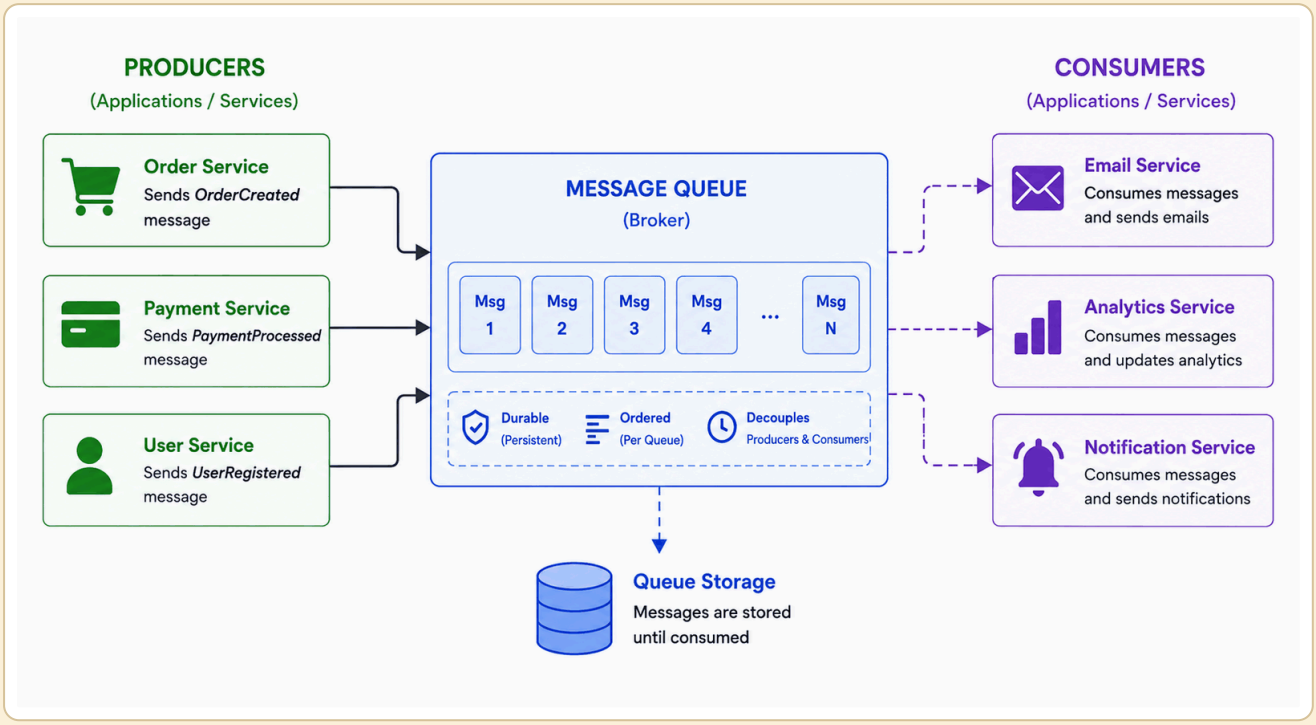


Figure — Message Queue (course diagram)

In the figure above, several producers (the order, payment and user services) drop messages into the broker. The broker stores them durably ("Queue Storage: messages are stored until consumed") and passes them out to consumers at whatever pace the consumers can handle. The three properties in the dashed box, **durable, ordered (per queue), decouples producers and consumers**, are the whole reason for the pattern.

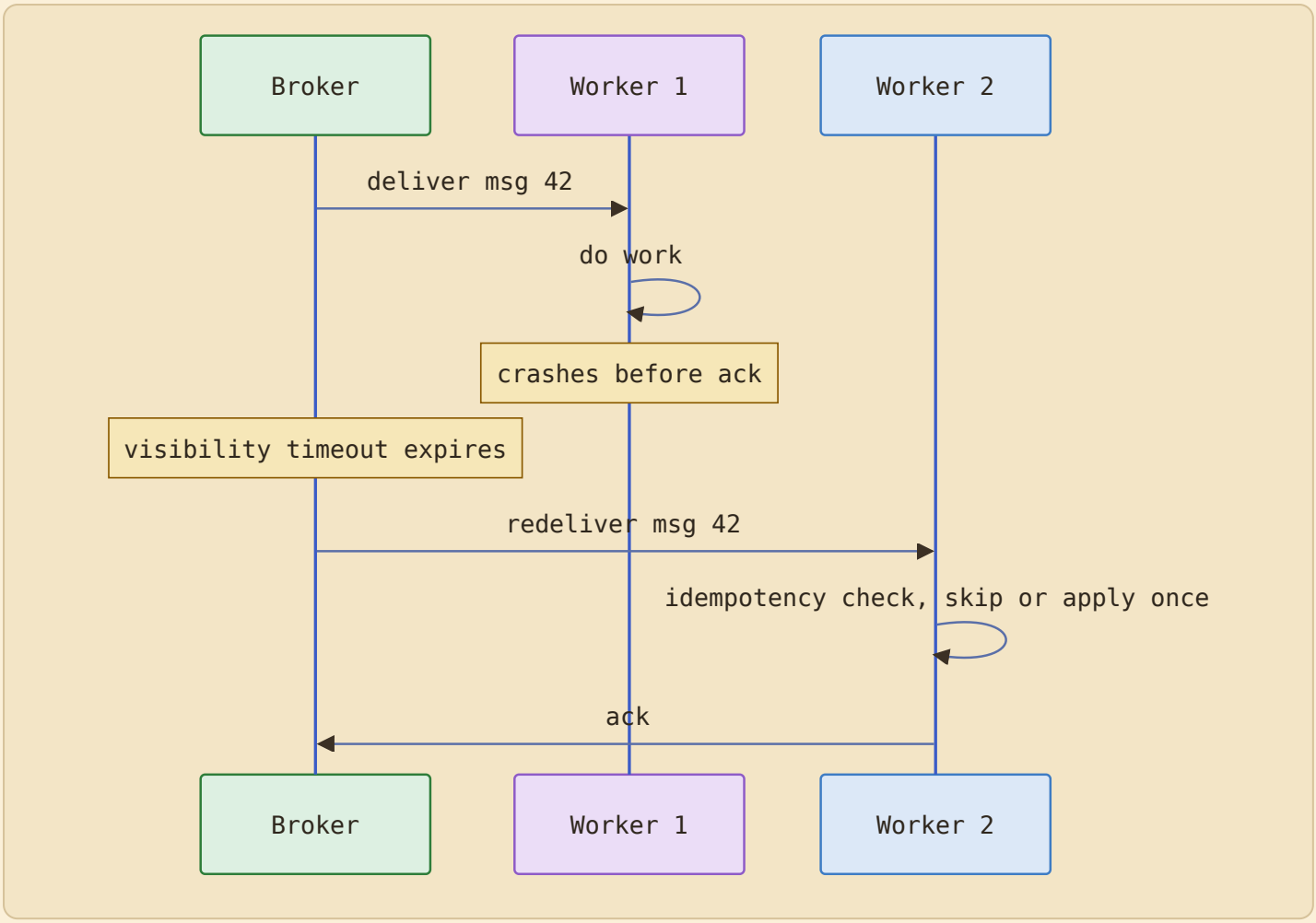
Taken together, these mechanics pull apart three concerns that the synchronous design had stuck together:

- **Rate:** producers can burst up to 5,000/s while workers keep a steady 400/s.
- **Time:** workers can be down, in the middle of a deploy, or not built yet; the work just waits.

- **Failure:** when a worker crashes, the message is redelivered rather than lost, and the producer never even knows.

The sequence below shows exactly where the duplicate comes from:

Ack and redelivery producing at-least-once delivery



Worker 1 finished the work but died before it could ack. The broker gets no ack before the visibility timeout runs out, so it sends message 42 again, this time to Worker 2. Worker 2's idempotency check makes sure the work only takes effect once.

Walkthrough with numbers

Run the flash sale again, this time with a queue between the API and the pipeline:

- **10:00–10:02.** 5,000 orders/s come in. For each one, the API checks it, puts it on the queue, and answers "Order confirmed" in **~50 ms**. The queue grows by 4,600/s. Over 120 s, that builds a **backlog** (work that's accepted but not done yet) of **~552,000 messages**, which sit cheaply on disk.
- **10:02–10:25.** The workers keep going at 400/s. Time to clear it: 552,000 / 400 = **1,380 s ≈ 23 minutes**. Some orders start fulfillment 20 minutes after they were bought, but the confirmation only promised "ships today." The business signed off on this delay ahead of time, so it's a design choice, not a failure.
- **10:07, the failure drill.** A worker dies partway through a message. Its visibility timeout runs out, another worker picks the message up, and the idempotency check keeps the half-finished work from being applied twice. Not a single order is lost.
- **The bill.** A queue service that costs pennies per million messages takes the place of a 12x server fleet.

The big takeaway: the queue added **no** processing power at all. What it did was remove the rule that processing had to keep up with arrivals second by second. The business never actually needed that rule.

Trade-offs

You gain:

- Spikes turn into a backlog instead of errors. The limit becomes disk space, not your slowest service.
- A worker outage turns into a longer to-do list, not an outage. Deploys stop being scary.
- Producers and workers can scale on their own, and queue depth is a ready-made signal for **autoscaling** (adding or removing workers automatically).
- Redelivery gives you retries without writing any retry code.

You pay:

- **Latency, on purpose.** Work gets done seconds or minutes later, so anything a user is actively waiting on shouldn't go on a queue.
- **Duplicates.** Idempotent workers are required, not optional.
- **Lost ordering.** With 20 workers, messages can finish in a different order from how they came in. Ordering by key (explained below) helps, at some cost to throughput.
- **A gap between "accepted" and "done."** The order is confirmed before fulfillment begins, so the product team has to decide what users see in the meantime.
- **More to run.** You need a broker, monitoring of depth and message age, a **dead letter queue (DLQ)** for messages that keep failing (it has its own lesson), and debugging that jumps across systems instead of following one **stack trace** (the list of function calls a program prints when it errors).

Ordering, in more detail: split the work up by a key. Kafka's **partition key** or SQS FIFO's **message group** (SQS is Amazon's managed queue service; FIFO means "first in, first out," i.e. it keeps order) sends all of one customer's messages to the same **partition** (a slice of the queue that keeps its own order). Each customer's messages stay in order, while different customers are still processed in parallel. The costs: one very busy customer can go no faster than a single partition allows, a stuck message blocks everything in its partition, and moving a failed message to the DLQ can break strict order.

Depth that keeps growing for days means more work arrives than you can ever handle. Raising the storage limit only pushes the failure back. Add capacity or fix the bottleneck, drop or reprioritize work, and alert on the **oldest message's age**, because that's the delay users actually experience.

When not to use it

1. **The caller needs the answer.** Login, price lookups, search results: the user is sitting there waiting. No one wants their search results sent by email.
2. **Tight deadlines from start to finish.** If the whole operation has to be done within ~200 ms, a trip through the broker plus a wait for a worker adds risk without any useful separation.
3. **Small, simple systems.** At ~10 requests/s, don't put a broker between two functions. Just call the function until scale, spikes or failure boundaries give you a real reason.
4. **As a database.** A queue moves work around; it isn't a place to store data you can query. If messages need to be searched, changed, or kept forever, use a database.
5. **Covering up a permanent lack of capacity.** Queues soak up short bursts. They don't create capacity.

Message Queue vs Publish-Subscribe

Both rely on a broker and deliver asynchronously, so people mix them up. But they answer different questions:

	Message Queue	Publish-Subscribe
A message is	A job: work that should happen once	A fact: something that happened
Delivered to	Exactly one worker	Every subscriber, each on its own
Adding a consumer	Adds more workers for the same job	Adds another use for the same fact
Question it answers	"Who does this task?"	"Who cares that this happened?"

"Resize this image" is a single job for a single worker, so it goes on a queue. "An order was placed" is a fact that fulfillment, email, analytics and fraud each need on their own, so it's pub/sub. The two fit together nicely: publish the fact once, and feed it into a separate work queue for each team (AWS **SNS → SQS**, where SNS is Amazon's managed pub/sub service; Kafka **consumer groups**, where each group is a named set of readers that together get one full copy of the stream).

Real-world examples

- **AWS SQS:** the pattern in its simplest form. It has visibility timeouts, dead-letter support, almost unlimited depth, and costs pennies per million messages.
- **RabbitMQ:** the classic broker you run yourself. It adds rich routing options on top of the same ack-and-redeliver core.
- **Kafka:** strictly speaking it's a **replicated log** (a record you can only add to, copied across machines), not a queue, but its consumer groups act like a queue. Because it can **replay** old messages, it drives event-driven architecture and stream processing later in the course.
- **Celery, Sidekiq, BullMQ:** the pattern wrapped up as libraries for applications. This is where most engineers first run into it.
- **For AI engineers:** GPU work (on the graphics chips that run AI models) is costly, comes in bursts, and is seldom urgent, which makes it perfect for queues. Examples include embedding pipelines (turning text into lists of numbers called **embeddings** that capture meaning), **batch inference** (running a model over many inputs at once), **fine-tuning** (further training an existing model on your own data), and evaluation runs. Queue depth, or even better **oldest-message age**, is the usual signal for autoscaling GPUs. **RAG** (retrieval-augmented generation: fetching relevant documents and handing them to the model with the question) ingestion pipelines use a queue with a DLQ to catch failures. The **batch APIs** from Anthropic and OpenAI sell exactly this trade-off: it's cheaper if the work is allowed to finish later.

In the interview

It comes up in prompts like "handle a 10x traffic spike" or "design an order pipeline." It also comes up when the interviewer points to a sync arrow and asks "does the user need to wait for this?" If the answer is no, that work might belong on a queue.

30-second answer: "I put a durable queue between the API and the pipeline. The API validates, enqueues, and confirms in milliseconds. Workers process at a sustainable rate, so a 5,000/s spike becomes backlog instead of errors. The broker saves each message before confirming and redelivers if a worker dies. That's at-least-once, so consumers are idempotent and repeatedly failing messages go to a monitored DLQ. Depth drives autoscaling and alerts. The trade is latency, which works only because the business agreed fulfillment can take minutes. Anything the user waits for stays synchronous."

Likely follow-ups:

- *"A worker crashes halfway through a message. Walk me through it."* The visibility timeout runs out, the broker gives the message to a different worker, and idempotency stops the effect from happening twice. If the message fails several times in a row, it gets moved to a DLQ and an alert goes off.
- *"Orders for the same customer must be processed in order. Now what?"* Partition by customer ID (a Kafka partition key or an SQS FIFO message group). Each customer's order is kept, while

different customers are handled in parallel. One catch: a single very busy customer is held to the capacity of one partition.

Mistake that fails candidates: putting a queue on a path the user is waiting on (like search through Kafka), or suggesting a queue without saying how duplicates are handled. The first shows you don't get *why* queues exist. The second ignores at-least-once delivery.

Self-check

- **Q1: Name the three ways a queue decouples producer and consumer, and explain step by step where at-least-once delivery comes from.** → Rate (bursts in, steady processing out), time (work waits while workers are down), and failure (a crash leads to redelivery, not loss). Workers only ack once they've finished. So the broker has to redeliver in two cases: a job that got interrupted, and a job that finished but whose worker died before acking. That second case is what creates the duplicate.
- **Q2: Depth has grown steadily for six days; a teammate proposes raising the storage limit. What is depth telling you, and why is that a trap?** → More work is coming in than you can ever process; the oldest message is already six days late. A larger limit just delays the failure. Add worker capacity or fix the bottleneck, cut down or reprioritize incoming work, send some work elsewhere, and alert on oldest-message age.
- **Q3: Charges and refunds for a customer share a queue with 20 workers; a refund got processed before its charge, leaving a negative balance. Explain and fix without losing parallelism.** → Two different workers picked up the two messages, and the refund happened to finish first (with many workers, overall order is lost). Partition by customer ID so each customer's messages run one after another while different customers still run in parallel. The costs: a hot customer is limited to one partition, a stuck message blocks its partition, and moving messages to the DLQ can break strict order, so some streams need to pause and be repaired instead.

Related patterns

- **Idempotency:** the one thing a queue can't do without, since every consumer can receive a retry.
- **Dead Letter Queue:** where messages end up once redelivery gives up.
- **Retry with Exponential Backoff:** redelivery *is* retrying, just done by the broker. Use that lesson to tune how many attempts and how long the delays are.
- **Publish-Subscribe:** the matching pattern for facts. It's often combined with queues by copying one event into a queue for each team.
- **Auto-Scaling:** queue depth and backlog age are about the clearest scaling signals you can get.

TL;DR

- **Problem:** a synchronous pipeline makes work happen as fast as it arrives, so a spike turns into an outage even when there's plenty of capacity overall.
- **Mechanism:** a durable task list between producer and consumer. Save before confirming, give each message to one worker, redeliver after a crash, and drain at whatever pace the workers can manage.
- **Costs:** latency by design, duplicates (so idempotency is required), no overall ordering, a gap between accepted and done that you have to own, and a broker to run.
- **Skip it when:** the caller needs the answer right now, the deadline from start to finish is tight, the scale doesn't justify a broker, or you'd be covering up a permanent shortage.

Publish-Subscribe

Original lesson: Publish-Subscribe on DesignGurus ↗

What it is

With **publish-subscribe (pub/sub)**, a producer announces a fact one time to a named channel, called a **topic**. It doesn't know or care who is listening. The broker hands a separate copy to every **subscription**. Each subscriber works through its own copy at its own speed and deals with its own failures. **Analogy:** think of a newspaper. It prints each edition once. Every subscriber gets a personal copy delivered and reads it whenever they want. The paper has no need to know what each reader does with it. When a new reader wants in, they just subscribe, and nothing changes in the newsroom.

The problem: checkout delivering to five teams

Once the queue solved the flash sale, other teams started asking for order data. Email wanted it for receipts. Then analytics, fraud and loyalty asked too. Each request added one more "send to this team's queue" call inside checkout. After eighteen months, checkout writes to **five queues** one after another, and this causes three ongoing problems:

1. **Each new consumer requires a checkout deploy.** Checkout is the company's most important money-making service. Yet every team's request for data now means changing it and redeploying it, so its release schedule holds everyone up.
2. **A crash leaves teams with mismatched data.** Last Tuesday, checkout crashed after writing to 3 of 5 queues. Fulfillment shipped the order, but fraud never saw it. Five separate writes can't all succeed or all fail together as one unit. So a crash can leave any mix of teams without the event.
3. **Other teams' troubles turn into checkout's troubles.** When loyalty's queue was slow for an hour, checkout's five back-to-back writes got slow as well.

Next, data science wanted *historical* order events to train models. They had to open a **pull request** (a proposed code change submitted for review) against checkout and wait for a release. When the producer has to hand-deliver to every consumer, each new consumer brings extra work, risk and coordination. That approach doesn't scale across many teams.

Why the obvious fixes fail

- **"It works; keep adding queues."** It does run. But checkout ends up owning every connection, every retry policy and every delivery guarantee. And each extra write raises the odds that a crash leaves consumers out of sync.
- **"Let teams read the orders database directly."** Now five teams rely on the exact shape of the table. A future **schema migration** (changing the table's structure, e.g. renaming or removing columns) could quietly break dashboards and models that checkout doesn't even know about. Their constant **polling** (asking "did anything change?" over and over) also puts load on the production DB. **Change data capture (CDC)** turns the database's change log into a stream of events; it's the careful, controlled way to do this. Uncontrolled table access has none of CDC's safety.
- **"One shared queue that everyone reads."** A queue hands each message to just **one** worker. If fraud, email and analytics all share it, each one gets a random slice of the orders, and no error shows up anywhere. Every team quietly works with incomplete data.

The teams don't need five jobs. They each need their own access to a single *fact*: an order was placed. So publish it once.

How it works

1. **The topic** is a named stream of facts, for example `order.placed`. Checkout publishes one time and moves on. The broker makes **one durable write** (saved to disk before it confirms), no matter how many teams are listening. Since there's just one write, a crash can't leave some destinations with the event and others without it.
2. **A subscription** is one team's own separate copy of the stream. It acts like that team's private queue. It has its own backlog, its own spot in the stream (the **offset**), and its own acks, redeliveries and DLQ. If fraud is offline for an hour, only fraud's backlog grows. Email and checkout keep going.
3. **Joining takes configuration, not code.** Data science sets up a subscription without touching checkout at all. Brokers that keep history (such as Kafka) let a new subscriber **replay** old events. For example, they can train on months of data and then switch over to live traffic.

4. **The event format turns into a public contract.** Consumers you don't know about will treat its fields like an API. So publish **facts, not commands**: "an order was placed, here are the details," not "send a receipt." The publisher says what happened, and each consumer decides what that means for it. Keep changes **additive**. Adding new optional fields is safe. Renaming or removing fields breaks subscribers you can't see. A **schema registry** checks event formats before they go out.

The main idea: the producer states *what happened*, and each consumer decides *what it means*. Checkout doesn't even have to know that receipts exist.

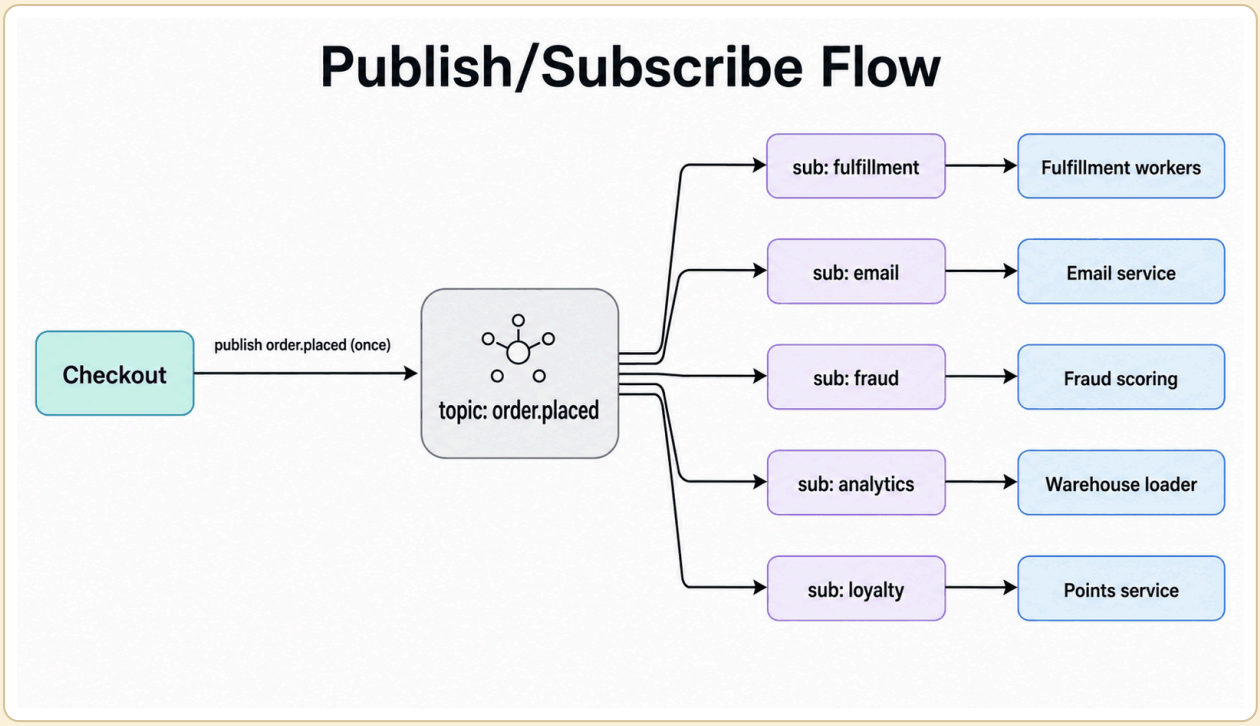


Figure — Publish-Subscribe (course diagram)

In the figure above, checkout publishes `order.placed` **once** to the topic. The broker then fans it out to five separate subscriptions (fulfillment, email, fraud, analytics, loyalty), and each one feeds its own service. To add a sixth team, you add one more subscription box on the right. Nothing on the left has to change.

Walkthrough with numbers (200 orders/s)

- **Cost of publishing:** before, there were five queue writes one after another, taking ≈ **25 ms** and getting longer with every new team. After, there's one durable write taking ≈ **5 ms**, and it stays flat however many subscribers there are.
- **The data-science request:** the subscription is set up in an afternoon with no change to checkout. The topic holds **30 days** of history, so they replay **~500 million** old events for training before going live. Before, the same request needed a PR plus a release cycle, and came with *no* history at all.
- **Failure drill:** fraud is offline for **40 minutes**. Its subscription piles up 200 × 2,400 s = **480,000 events**, which get processed once it's back. Email sends every receipt on time, and checkout isn't affected. In the old design, fraud's outage would have either slowed down checkout or lost 40 minutes of events.
- **The audit passes now:** each subscription can report whether it processed order X, still has it waiting in the backlog, or sent it to the dead letter queue.

Trade-offs

You gain:

- The producer's cost and risk stay the same no matter how many consumers there are.
- Teams release on their own schedules. This separation between teams is the biggest benefit in practice.

- Failures stay inside one subscription. With history kept around, new consumers can start with the whole past.

You pay:

- **The event format has become a public API.** Renaming a field can break consumers you've never heard of. Instead of being tied together at deploy time, teams now need *contract discipline*: a schema registry, changes that only add, and **deprecation windows** (a fixed period during which the old version still works).
- **Duplicates, times N.** Each subscription is at-least-once on its own, so five subscribers means five places that all have to be idempotent.
- **Hidden coupling.** The dependencies are still there, but they no longer show up in the producer's code, so teams lose track of who uses what. Mature organizations keep a **subscriber catalog**. It lists each topic's owner, **schema** (the agreed shape of the event's fields), **retention period** (how long old events are kept), and **data classification** (how sensitive the data is), plus the owner of every subscription. This makes schema changes, incidents and deletion requests manageable, and it reveals consumers nobody is maintaining anymore.
- **No "all done" signal.** Each subscription only knows how far it has gotten. So pub/sub can't tell you "order X is fully processed." When inventory, payment and shipping must all finish before you can call it a success, something has to act as a coordinator and track them (see orchestration below).
- **Slow consumers can lose data.** Brokers only keep history for a **retention window**. A subscriber that falls further behind than that window loses events forever. Alert on backlog age long before retention runs out, so there's time to add capacity or rebuild from another trusted copy.

Rule of thumb: **broadcast across services, compete within a service.** When a schema change truly has to break things, publish v2 next to v1 for a deprecation window.

When not to use it

1. **It's a job, not a fact.** "Resize this image" is one task for one worker, so use a queue.
2. **The producer needs the result.** Checkout can't move on without knowing if the payment was authorized, and publishing `payment.requested` doesn't fix that. Use request-response or an orchestrated workflow.
3. **Two services and little traffic.** One producer and one consumer only need a plain queue between them, which is simpler.
4. **Request-response rebuilt out of events.** Pairing `user.lookup.requested` with `user.lookup.completed` just recreates a synchronous call, but with more latency and harder debugging. Seeing the word "request" in an event name is a warning sign.

Across a big organization, pub/sub without management turns into **topic sprawl**: hundreds of topics with no owners, no schema rules and no visible consumers. If you take away the required coordination between teams, you have to put explicit contracts and a catalog in its place.

Choreography vs Orchestration

Pub/sub makes **choreography** possible. Each service reacts to events on its own, and the overall process *emerges* from how they all behave together. **Orchestration** instead uses a single coordinator that runs the workflow one step at a time.

	Choreography (pub/sub)	Orchestration (coordinator)
Who's in control	No one in the middle; the behavior emerges	The coordinator, openly
Adding a step	A new subscription, no coordination needed	Change the coordinator's workflow
"Is order X fully processed?"	No single place has the answer	The coordinator knows, by design
Failure handling	Each consumer deals with its own	The coordinator retries, undoes, or escalates
Best for	Reactions that don't depend on each other (analytics, email, indexing)	Steps that depend on each other (inventory → payment → shipping)

Decide by asking if the steps are really independent. Email and analytics don't care what order the other runs in, so choreography fits them. Reserving inventory, charging the payment, shipping, and undoing earlier steps when something fails make up a dependent workflow. The **Saga** lesson covers how to orchestrate that. Lots of systems use both: they orchestrate the order workflow and publish facts from each stage for independent consumers.

Real-world examples

- **AWS SNS → SQS:** the pattern built from managed pieces. SNS is the topic, and each team's SQS queue is its subscription. It broadcasts first, and then each team has its own task list.
- **Kafka:** every consumer group is a separate subscription reading a stored log. That gives you broadcast, replay and per-team queue behavior in a single system, which is why Kafka dominates.
- **Google Cloud Pub/Sub:** the same model, fully managed. Subscriptions are objects you create and manage yourself.
- **Redis Pub/Sub, the warning story:** it's fire-and-forget. If a subscriber is offline, the message is lost forever. "Pub/sub" without durable subscriptions is a much weaker promise wearing the same name, so always check which kind you're getting.
- **For AI engineers:** a `user.interacted` topic often powers an entire ML platform. It feeds the analytics warehouse, a **feature store** (a service that holds the input features for models), jobs that refresh embeddings, and real-time features for a recommender. It also keeps replayable history for future models. Agent platforms publish each tool call and decision once, so observability, evaluation and audit systems can read them without slowing the agent down.

In the interview

It can be asked straight out ("how should five teams get order data?") or sideways ("a sixth service needs this too"). If you keep drawing yet another arrow out of the producer each time, that signals coupling that keeps growing. Interviewers may also check whether you wrongly use choreography for a dependent order→payment→shipping flow that really needs orchestration.

30-second answer: "Checkout publishes an `order.placed` fact to a topic, once. Each team gets its own subscription with its own backlog, retries and DLQ, so a slow or crashed consumer affects nobody else. Adding a consumer is configuration, not a checkout deploy, and with Kafka the newcomer can replay history. What I manage: the schema is now a public API, so changes are additive and registry-checked; every subscription is idempotent; and I alert on backlog age so no one falls past retention. If the steps are really a workflow needing completion tracking, I orchestrate that part."

Likely follow-ups:

- *"One subscriber runs at half the speed of the others. What happens?"* Only that subscriber's backlog grows; the rest aren't affected. If its backlog goes beyond retention, events are lost for good. Watch backlog age and scale that subscription with competing consumers (which is the queue pattern used inside a single subscription).

- *"How do you change a schema without breaking consumers you don't know about?"* Only ever add optional fields; never rename or remove anything. A schema registry blocks builds that would break things. For a change that truly breaks compatibility, publish v2 next to v1 for a deprecation window. Keep a consumer catalog so you know who depends on what.

Mistake that fails candidates: hooking several services up to one shared queue and calling that broadcast (each one only gets a sample), or rebuilding request-response out of paired events. Both mix up jobs and facts.

Self-check

- **Q1: Topic vs subscription, and what does adding a sixth consumer cost the producer?** → A topic is a single durable stream of facts. A subscription is one consumer's own feed, with its own position, backlog, acks and DLQ. The cost to the producer is nothing: no code, no deploy, no added latency, and it doesn't even need to know. The new consumer is responsible for idempotency and for watching its backlog, and the system now has one more dependency on the schema.
- **Q2: Argue against "pub/sub decouples services."** → Consumers are still tied to the event schema, so changing a field can break systems the producer can't see. Pub/sub shifts the coupling out of the release schedule and into a data contract. That contract has to be governed with a registry, additive changes, deprecation windows, a consumer catalog, and named owners for subscriptions.
- **Q3: Fraud scores ~20% of orders and email sends ~30% of receipts, yet nothing was lost and publish counts match orders. Diagnose.** → The services are sharing one subscription/queue, which turns them into *competing* consumers, so each event reaches only one of them. Give every service its own subscription, and only use competing consumers among the workers of a single service. *Broadcast across services, compete within a service.*

Related patterns

- **Message Queue:** each subscription basically is one; SNS → SQS puts the two together directly.
- **Idempotency:** N subscribers means N chances to apply something twice.
- **Event-Driven Architecture:** the next lesson. Pub/sub is how events travel; EDA decides what goes inside them.
- **Change Data Capture:** turns a DB's change log into events. It's the careful version of "just read my tables."
- **Webhooks:** the same idea, but between companies, over plain HTTP, with the guarantees worked out again.

TL;DR

- **Problem:** when the producer delivers to every consumer on its own, each new team means producer deploys, slower fan-out, and partial crashes where some teams get the event and some don't.
- **Mechanism:** publish each fact once to a topic. Every subscription gets its own full copy, backlog, retries and DLQ. Subscribing is just configuration, and log-based brokers add replay.
- **Costs:** the schema turns into a public API, duplicates in every subscription, coupling that becomes invisible without a catalog, no signal for "done," and retention as a hard limit.
- **Skip it when:** it's a job (use a queue), the producer needs the result (use request-response/orchestration), or it's really a step-by-step workflow (use a saga).

Event-Driven Architecture

Original lesson: Event-Driven Architecture on DesignGurus ↗

What it is

Event-driven architecture (EDA) means services talk to each other mostly through events. Its core design question is: **how much information should each event hold?** Should it hold enough that consumers can act by themselves, without calling the producer back? **Analogy:** an order ticket that only says "Order #7f3a" makes the cook walk out to the front desk to ask what was ordered. A ticket that says "3 items, $142, ships to Austin" lets the cook get straight to cooking. Pub/sub settled *how* events get delivered. This lesson settles *what goes inside them*.

The problem: the callback storm

Pub/sub did its job: checkout publishes `order.placed` one time, and five teams subscribe. But every event held only `{event: "order.placed", order_id: "ord_7f3a"}`. That's a **notification**. It says an order exists and nothing else. So fraud got the event and then called `GET /orders/ord_7f3a` to find the amount and items. Email, analytics, loyalty and fulfillment did the same (and fulfillment did it twice by accident). **Every event checkout published bounced back at it as five API calls.**

At the next flash sale, 5,000 orders/s turned into 5,000 events/s, which turned into **25,000 GET requests/s** against checkout's API. That happened right during the spike the queue was supposed to soak up. Checkout's read service went down, consumer calls timed out, retries added even more load, and fraud scoring fell minutes behind during the time of year with the most fraud.

The communication was asynchronous now, yet the old load came right back. The reason: events said *that* something happened but not *what* happened. This is **asynchronous delivery wrapped around a synchronous dependency**, not true event-driven design.

Why the obvious fixes fail

- **"Cache the lookups."** Every consumer asks about each new order right away, and that order didn't exist a second before. So all five miss the cache at the same moment and hit the API anyway. They also still need checkout to be up.
- **"Slow the consumers down."** Rate-limiting consumers shields checkout. But then every team downstream can only work as fast as checkout allows. Backlogs grow during the bursts that matter most, and fraud detection becomes limited by checkout's capacity. The dependency is only postponed, not removed.
- **"Could we just put the details in the event?"** Yes, and that's exactly the pattern. Copying data does have real costs (covered below). But the first two ideas only make callbacks cheaper, while complete events make callbacks unnecessary.

How it works: three levels of event content

```
Level 1: Event Notification     "Order ord_7f3a was placed." → consumers call back
Level 2: Event-Carried State    "Order ord_7f3a: 3 items, $142, customer c_91,
         Transfer                ships to Austin." → consumers act without calling
Level 3: Event Sourcing         The events ARE the data; state is rebuilt by replay
```

- **Level 1, notification.** The event says something happened and sends consumers somewhere else for the details. Events stay small and no data gets copied. But this is precisely what caused the callback storm.
- **Level 2, event-carried state transfer.** The event includes the facts that consumers need. Each consumer keeps a **local view**, which is a small table with just the fields it cares about, kept up to date from events. Fraud stores the order fields it uses for scoring and reads them locally in **under a millisecond**, even while checkout is down. Consumers are no longer callers; they own *copies they can throw away*. **This is the sensible default in practice.**
- **Level 3, event sourcing.** The event log *is* the **system of record** (the official data), and tables are rebuilt from it. That's a decision about storage, it has its own lesson, and you don't need it to fix a callback storm.

Three rules for Level 2:

1. **Events are facts, never commands.** Publish "order placed," not "send a receipt." A command ties the publisher back to one particular consumer's job, which defeats the whole point of a topic.

2. **Local views are copies, never the truth.** Any view can be wiped and rebuilt by replaying the topic's history, and checkout stays the official owner. **Each fact belongs to exactly one owning service.** If a consumer spots a wrong value, it must not fix it in its own copy, because that creates a rival version of the truth. Instead, it sends a command to the owner. The owner checks the change and publishes a new fact, so every view gets corrected the same way.
3. **Freshness is about "when," not "if."** A local view is always a little behind the source. Usually that's milliseconds, but during a backlog it can be minutes. This is called **eventual consistency**: all the copies agree *once* the updates finish spreading. Make the delay visible. Store the source version or event timestamp in each view, publish view age as a metric, show an "as of" time on screens like support dashboards, and alert when a view goes past its agreed freshness limit (its **freshness SLO**, where SLO means service-level objective).

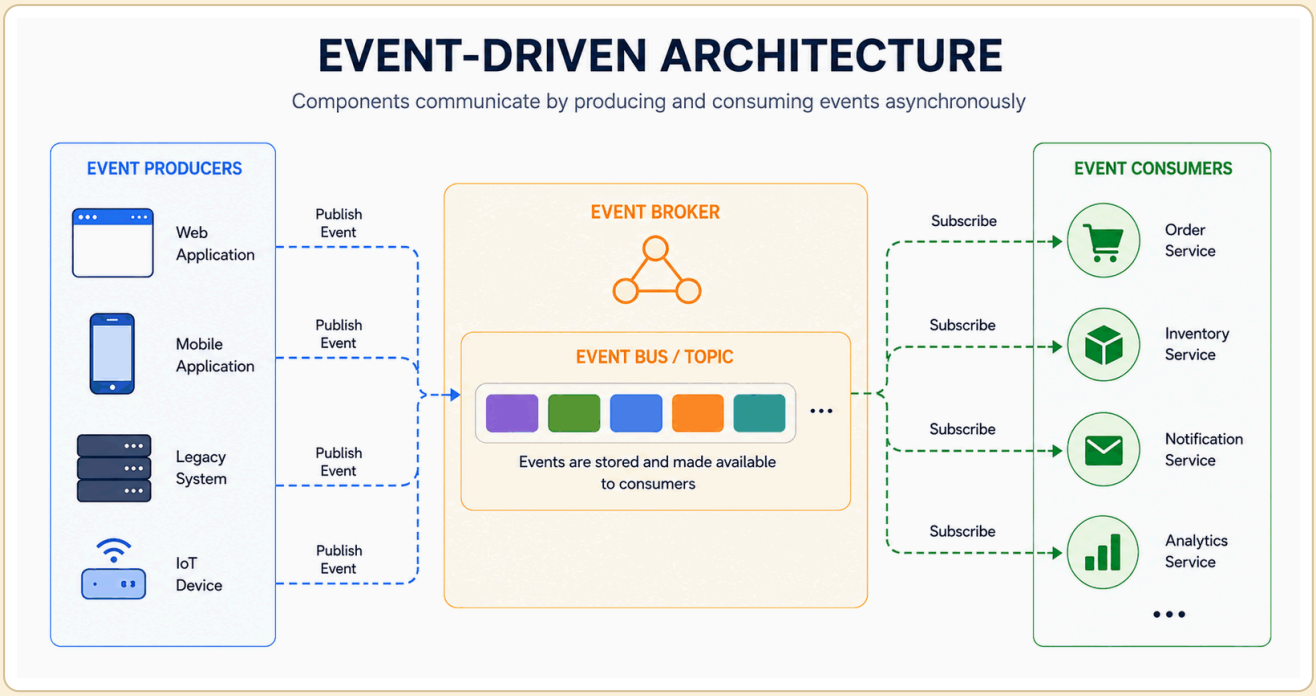
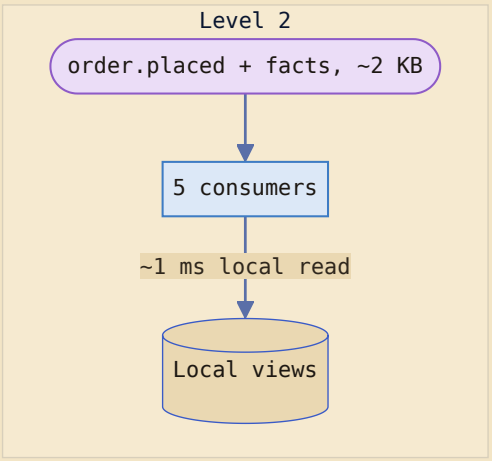


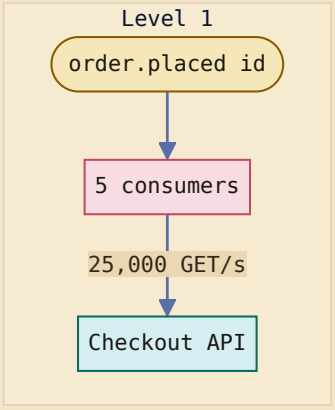
Figure — Event-Driven Architecture (course diagram)

The figure above shows the overall shape. Many producers (a web app, a mobile app, a **legacy system** (an older system still in use), an **IoT** device ("Internet of Things": a small networked gadget such as a sensor)) publish events into a broker's event bus/topic, where they're stored and made available. Consumers (the order, inventory, notification and analytics services) subscribe and react asynchronously. With Level 2 events, no consumer on the right ever has to call back to a producer on the left.

The mermaid diagram below compares the two levels you'll actually use:

Callback storm vs local views





At Level 1, an event with only an ID becomes 25,000 GET/s hitting checkout. At Level 2, a ~2 KB event carrying the facts lets those same five consumers read from their own local views in about 1 ms.

Walkthrough with numbers

- **The event gets bigger**, from ~100 bytes to ~**2 KB**. At 5,000 events/s, the broker moves **10 MB/s**, which is a small load for a production broker. In return, the **25,000 GET/s vanish**. You trade a bit of bandwidth for independence.
- **Checkout's read traffic stays flat during the sale.** With no callbacks, its API can serve real users.
- **Fraud reads locally in ~1 ms** rather than making a 50 ms network call wrapped in a timeout and retry. During the busiest hour, its speed and availability no longer rely on checkout.
- **A new consumer replays the history.** With 30 days of retention, that's ≈ **500 million events**, replayed in a few hours to build its view before it goes live. Without replay, it would have to **backfill** (load all the past data in one go) through checkout's API and add a lot of load.

Trade-offs

You gain:

- Independence. Consumers read locally, keep running when the producer is down, and never flood anyone with callbacks.
- Load on the producer that doesn't depend on how many consumers there are or what data they want.
- Views you can rebuild from history, so recovering a view and onboarding a new one are the same thing: a replay.

You pay:

- **Copies all over the place.** Six services might each hold order data in six different shapes. Storage is cheap, but everyone has to remember the copies can be thrown away and the owner is the authority.
- **Eventual consistency, all the time.** Products have to cope with things like a support dashboard showing data that's 40 seconds old.
- **Personal data spreads out.** If a customer's email goes into an event, it gets copied into every subscriber and into the retained topic. A deletion request then has to clean up **N+1** places. Decide this *before* you publish: include only facts that many consumers need, label sensitive fields, and record the owner of every subscription. Encryption protects **data at rest** (data sitting in storage, as opposed to travelling over the network), but it doesn't remove the need to delete or limit copies you don't need. A field added for convenience can turn into a long-term legal obligation.
- **The schema matters even more.** Bigger events expose more public fields, so a registry, additive-only changes and a consumer catalog become a must.
- **Debugging gets harder.** Every service has its own timestamped version of order X. **Correlation IDs** (one ID that tags the same business request across events and logs) and **distributed tracing** (recording a request's path and timing across every service it touches) become essential.
- **Replay has to be safe to run again.** Rebuilding a view must never resend emails or charge cards again. Keep *state-building* consumers separate from consumers that *cause side effects*, and test replay against production-sized history before you count on it during an incident.

When not to use it

- **Inside a single service.** The functions in a **monolith** (one big application deployed as a single unit) already share **transactions** (groups of changes that succeed or fail together) and consistency. EDA is for the boundaries between services and teams.
- **When a read has to be exactly up to date.** Selling the very last item in stock or approving a withdrawal needs the latest value. Send those reads synchronously to the owning service, and use local views for data that's allowed to lag.
- **Request-response rebuilt from events.** Events full of state don't help a flow that is still a synchronous dependency underneath.
- **Before the team is ready for it.** Without schema enforcement, tracing and backlog alerts, EDA is hard to run. Until those exist, fewer services with direct calls may be the safer choice.
- **As a way to dodge deciding who owns what.** If six services keep views but nobody owns the fact, the copies drift apart and there's no official answer.

Notification events vs Full-state events

	Notification (Level 1)	State transfer (Level 2)
Event contains	Only an ID ("go look it up")	The facts themselves
When a consumer needs data	It calls the producer	It reads its own local view
Producer read load	Rises with how much consumers ask	Zero
Copies / privacy exposure	Minimal	Every subscriber + the retained history
Freshness	The lookup is always current	Views run slightly behind
Best when	Only a few consumers need details; the data is sensitive	Many consumers need the same data; independence matters

A sensible middle path is to decide **one field at a time**. Put fields that many consumers need into the event, and leave sensitive or rarely used fields with the owner, reachable through an ID lookup.

Real-world examples

- **LinkedIn** created Kafka for exactly this. Activity events formed one big stream, and every team read it to keep its own view. Kafka's nickname, "central nervous system," began there.
- **Uber:** a trip exists as a stream of events well before it becomes a database row. Pricing, ETA (estimated time of arrival) prediction, fraud and driver payments each read that stream on their own.
- **Bank ledgers:** a very old form of this pattern. Transactions are facts that never change, and your balance is a view calculated from them.
- **Netflix:** viewing events travel through a central stream into recommendations, artwork personalization and capacity planning.
- **For AI engineers:** full-state events keep model features up to date without API callbacks. A feature store's online values get updated from interaction events. A **vector index** (a search index over embeddings that finds items with similar meaning) re-embeds content when a `content.updated` event arrives carrying that content. Training pipelines use the retained history as a dataset (it's the same replay that sets up a **microservice**, a small service that owns one narrow job). A recommender can see a purchase within **200 ms** without ever calling checkout.

In the interview

They might ask "how does service B get A's data without calling A?" More often, once you draw a topic, they'll ask **"what's in the event?"** That choice decides how much callback load there is, how independent consumers are, and how far personal data spreads.

30-second answer: "The event's contents are the main design decision. Thin notifications make every consumer call back, so coupling and load return. For widely consumed facts I put the state in the event, and each consumer keeps a local view: updated from the stream, rebuildable by replay, fast to read, available when the producer is down. Every view is eventually consistent, so exactly-current reads go synchronously to the owner. Sensitive fields stay out or get encrypted. The larger schema is a registry-governed public API. And one service owns each fact; views are copies, never the truth."

Likely follow-ups:

- *"A support agent updates an order but doesn't see the change on their dashboard. Why, and what do you do?"* The dashboard reads from a local view that hasn't caught up with the write yet. You have options. Show an "as of 12:04:31" timestamp. Give the writer **read-your-writes** behavior, which means sending their next read to the source. Or wait until the view reaches the version of that write. Choose the guarantee screen by screen.
- *"A user invokes their right to be deleted; their data is in a 30-day retained topic and six local views."* Delete it from the owning store and publish a **tombstone**, which is a special event telling every view to wipe that user. If the retained history has to be gone before it would expire naturally, use **crypto-shredding**. You encrypt personal fields with a key unique to each user and then destroy the key. That makes every retained copy, replay and backup unreadable.

Mistake that fails candidates: calling notification events "decoupled" while every consumer still calls the producer, or suggesting full-state events without dealing with staleness, deletion or ownership.

Self-check

- **Q1: Name the three levels and where the truth lives in each.** → Notification: the producer holds all of the truth. State transfer: the producer is still the authority, and consumers hold copies they can throw away. Event sourcing: the log itself is the truth, and tables are rebuilt from it.
- **Q2: Fraud wants `customer.lifetime_value` added to `order.placed` "since it's already flowing." Argue both sides, then decide.** → For it: fraud needs the value, adding it doesn't break anything, and it

saves a callback. Against it: the value describes the *customer*, not the order. It goes out of date fast, and it shows a sensitive business number to every subscriber. Decision: keep `order.placed` focused on facts about the order. Publish lifetime value on its own, or have fraud read it from the feature store, and then combine the two inside fraud's own view.

- **Q3: Eight months in, legal forwards a deletion request. The email exists in checkout's DB, the 30-day topic, and six local views, two with no listed owner. Design the deletion and name the process failure.** → Delete it in checkout, publish a tombstone, and confirm that every consumer wipes it. Check whether the 30-day expiry meets the legal deadline; if it doesn't, crypto-shred. The two views without owners point to the real failure: subscribing to personal data didn't require a named owner, a catalog entry and approval for handling the data. The long-term fix might be taking the email out of the event completely.

Related patterns

- **Publish-Subscribe:** how events travel; this lesson decides what goes inside them.
- **Event Sourcing:** Level 3, where the log *is* the truth.
- **Change Data Capture:** creates full-state events from a DB change log when the producer can't or won't publish them itself.
- **Saga:** for workflows with several steps, which events shouldn't try to mimic.
- **Idempotency:** view updates arrive at-least-once, so applying one twice must still leave the view correct.

TL;DR

- **Problem:** events that are only notifications send every consumer back to the producer's API, so the coupling and load come back, just asynchronously.
- **Mechanism:** put the facts inside the event. Each consumer keeps a local view it can rebuild by replay. Each fact has one owner, and views are copies, never the truth.
- **Costs:** data copied into every subscriber, views that always lag, personal data spread so widely that deletion becomes a project, and a bigger public schema.
- **Skip it when:** you're inside one service, the read has to be exactly current, it's really a sync call or a workflow in disguise, or the team can't yet support schema rules, tracing and alerts.

Webhooks

Original lesson: Webhooks on DesignGurus ↗

What it is

A **webhook** is an event that reaches you as an **HTTP POST sent by another organization's system** (POST is the HTTP method for sending data to a server; GET is the one for reading). For example, your payment provider calls a URL on your server to tell you "payment succeeded." Think of it as pub/sub that crosses a company boundary, using plain HTTP. The rule for receiving one: **confirm it's genuine, save it, answer "received" within milliseconds, and do the actual work later behind your own queue. Analogy:** a courier brings a parcel to your door. You sign for it in a few seconds and open it later. You don't keep the courier standing there while you unpack, put together and test everything. If you do, they give up, mark the delivery "failed," and return later with a second copy.

The problem: the handler that did everything

The payment provider POSTs `payment.succeeded` to `https://api.yourstore.com/webhooks/payments`. The handler marked the order as paid, reserved stock, told the warehouse, sent a receipt, and only *after all that* returned 200 (the HTTP status code for "OK"; status codes are the 3-digit numbers every HTTP reply carries, e.g. 401 = "not authenticated", 202 = "accepted, will process later"). When things got busy, that took **~7.5 s**. The provider only waits **5 s** before it hangs up.

So as far as the provider could tell, **40%** of deliveries failed, and it **retried** them. But your handler usually finished its work anyway, after the provider had stopped listening. So each retry did every action again, leading to **two shipments** for payments that had really gone through. The failure rate kept climbing until it passed the provider's health threshold. Following its published policy, the provider then **disabled your endpoint**.

That failure made no noise. No requests came in, so your logs showed no errors. For **three hours**, customers kept paying while their orders stayed "pending," and support tickets spotted the problem before monitoring did.

Here is the key difference from the earlier lessons: the other side is run by **another company**. Its timeout, retry schedule and disable policy are set in stone, and your system has to work inside them.

Why the obvious fixes fail

- **"Make the handler faster."** Getting it down to 4 s gets you through today. But one slow warehouse call, database hiccup or traffic spike breaks it again. The structure (doing business work inside the request) still allows the same chain: delay → retry → duplicate → disable.
- **"Ask the provider for a longer timeout or fewer retries."** Their short timeout shields *their* service from your slowness. Their retries shield *you* from losing financial events during short failures. Taking either one away pushes risk onto both companies.
- **"Forget webhooks; poll their API."** Polling every minute burns through your **rate limit** (the cap the provider puts on your API calls), adds up to a minute of delay, and usually finds nothing. Still, polling *does* guarantee you'll eventually see everything, while webhooks give you speed. The right answer uses both.

How it works: five receiver rules, in the order a request hits them

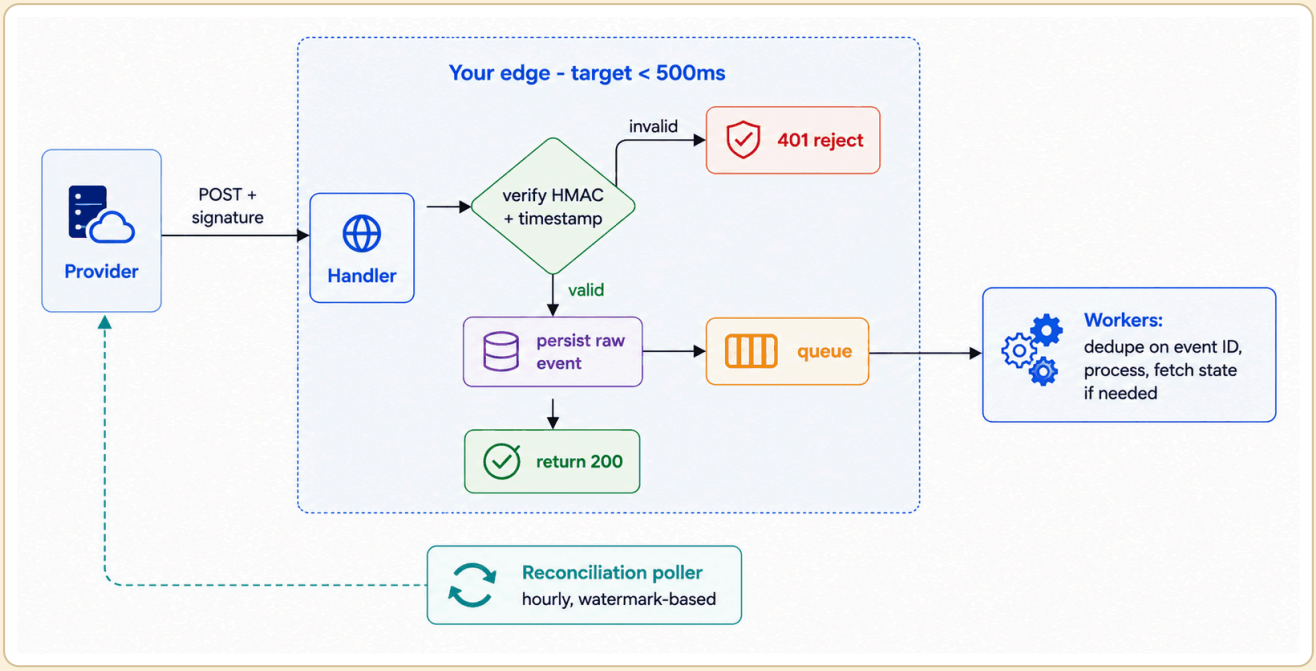


Figure — Webhooks (course diagram)

The figure above follows a request through your edge, with a target of under 500 ms. The provider POSTs with a signature. The handler checks the HMAC and the timestamp; if either is invalid, it sends back 401 and rejects the request. Valid events are saved in raw form, put on a queue, and then 200 is returned. Behind the queue, workers **dedupe** (detect and discard duplicates) by event ID, do the processing, and fetch state when needed. Separately, a reconciliation poller runs every hour, uses a watermark, and asks the provider for anything that was missed. Here are the five rules in detail:

1. **Verify before you believe.** The webhook URL is public, so anyone can POST `{"type": "payment.succeeded"}` to it. Providers attach an **HMAC**: a hash of the message made with a secret key that only you and the provider know. Compute it again over the **raw body** and compare the two. Use a **constant-time comparison**, which takes the same time no matter how close a guess is, so it gives nothing away. Also check the provider's **timestamp** against an allowed time window. That stops an attacker from

replaying an old message that was validly signed. Reject bad requests *before* you parse them. Skip this step, and anyone could mark unpaid orders as paid. (An **IP allowlist**, accepting requests only from the provider's known network addresses, is a nice extra layer, but it isn't authentication.)

2. **Save it and acknowledge. Do nothing else.** Store the raw event, put it on your message queue, and return 200 in **~15 ms**. **200 means "safely received," never "fully processed."** Inventory, the warehouse and receipts all happen later behind your queue, under your own retry and failure rules. The provider's 5 s deadline no longer has anything to do with business processing.
3. **Deduplicate on the event ID.** Every provider says in its docs that delivery is at-least-once: they send again whenever they're unsure it worked. Use each event's unique ID as an idempotency key so repeats only take effect once. This is what prevents double shipments.
4. **Don't trust the order, or how fresh the payload is.** A "refund updated" event might show up before its "charge succeeded." A retried payload could be hours old. For decisions that depend on the current state, treat the webhook as a *heads-up* and **fetch the current state from the provider's API**, which is the source of truth (this is Stripe's own advice). You can skip that fetch for self-contained events that only ever add records, when fetching would go over the API rate limit, or when the action is cheap, idempotent and easy to fix.
5. **Reconcile, because silence doesn't mean success.** Sometimes events just never show up: the endpoint got disabled, an outage lasted longer than the retry window, or a network path broke without anyone noticing. So every so often, ask the provider for "everything that changed since my **watermark**" (a saved bookmark of how far you've already checked). Webhooks give you quick notice; reconciliation makes sure nothing is missing. Only move the watermark forward *after* the recovered events are safely on the queue (if you crash in between, you'd create a new silent gap). Record which time range each run covered, for audits. And dedupe by event ID across both paths, so replaying from an older watermark does no harm.

Also: check the event type and object before you enqueue it, and have the worker confirm that each state change is allowed. A valid signature proves *who* sent the message. It doesn't prove the change it asks for is permitted in your system.

If you're the sender (your product sends webhooks to customers), these rules turn into your duties. Sign payloads using **rotatable secrets** (keys you can swap out). Give every event a unique ID. Publish your retry/backoff schedule. Keep delivery timeouts short. Use a durable queue for each endpoint. Automatically disable endpoints that keep failing. And offer a dashboard with a **delivery log** (event ID, destination, attempt number, response code, latency, next retry, with no secrets shown) plus a **redeliver** button. That way customers can tell "never attempted" apart from "your server rejected it."

Walkthrough with numbers

- **Handler time ~15 ms:** ~1 ms to check the signature, ~10 ms to save and enqueue, then 200. With no slow dependency inside the request, the provider sees a **100% success rate** through spikes and deploys.
- **The actual work** happens behind the queue, just as in the queue lesson. Spikes turn into backlog, crashes turn into redeliveries, and messages that keep failing go to the DLQ.
- **A duplicate** shows up 90 s later because the first 200 was lost on the way back. The worker sees that event ID was already handled and stops. Result: **one shipment**. That's idempotency working across a company boundary.
- **Hourly reconciliation** normally finds nothing. During one delivery incident at the provider, it found **217** missing confirmations and quietly queued them, with no emergency needed.
- **A new alarm for silence:** if an endpoint that's usually busy goes quiet for **15 minutes**, **page** someone (send an urgent alert to the on-call engineer's phone). This catches failures that don't produce any errors on your side.

Trade-offs

You gain:

- Almost real-time updates from other companies without constantly polling them.
- The provider's retry system working *on your behalf*, now that you acknowledge quickly.

- A single, consistent way in for outside events, which feeds the queue-and-worker setup you already run.

You pay:

- **You take on their rules.** Every provider has its own delivery guarantees, ordering, retry schedule, disable policy and payload versioning. Each integration is its own separate contract.
- **A public endpoint to protect.** Signature checks, timestamp windows and secret rotation are now part of your security boundary. If a signing secret leaks, anyone can fake "payment succeeded."
- **Debugging across company lines.** Figuring out whether they never sent it or you dropped it requires their dashboard, your raw-event store, and timestamps you can line up. Keep one **correlation record per event ID** that covers receipt, queue processing, business effects and reconciliation.
- **The polling loop stays forever.** In production, both run permanently. Monitor how far behind reconciliation is, too.

When not to use it

- **Between your own services.** A webhook mimics a broker over HTTP, but without durable subscriptions, replay or managed redelivery. When you own both ends, use pub/sub.
- **When being complete matters more than being fast.** Billing reconciliation, financial close and compliance exports should begin from polling or bulk-export APIs. Webhooks can add freshness on top.
- **Very high-frequency streams.** Thousands of separate POSTs every second is wasteful. Use the provider's streaming or bulk interface.
- **When someone is waiting for the answer.** An interactive question needs request-response. A webhook round trip only adds latency and new ways to fail.

The root mistake in the incident was using the webhook request to *do* the business work, rather than just *accepting* the notification.

Webhooks vs Polling

	Webhooks (push)	Polling (pull)
Freshness	Seconds	As often as you poll
Cost when idle	Close to zero	A steady stream of "anything new?" requests
Completeness	Best effort; gaps can happen	Guaranteed, in the end
Who controls delivery	The sender	You
Worst failure looks like	A quiet day	Stale (but visible) data

The sneaky danger of webhooks sits in that last row: a failed delivery looks *exactly the same* as no activity. That's why the motto is **webhooks for speed, polling for truth**, with a bookmark that lets polling pick up whatever push missed. (For continuous streams to browsers or apps you control, persistent connections such as Server-Sent Events take the place of separate POSTs. Webhooks are specifically server-to-server push across company boundaries.)

Real-world examples

- **Stripe:** the model example. It has signed payloads with timestamp checks, event IDs for dedupe, retries spread out over days, automatic disabling, and official advice to fetch the current state instead of trusting how fresh the payload is.
- **GitHub:** signs every delivery with the `X-Hub-Signature-256` header and provides a delivery log with a redeliver button. In other words, it does the sender's duties well.
- **Slack:** demands a 200 within **3 seconds**. No business logic can fit into 3 s, and that's intentional: the contract forces you to accept first and work later.

- **Twilio, Shopify, PayPal:** every platform built around integrations ends up with the same terms, because they all face the same ways of failing.
- **For AI engineers:** async AI services use webhooks to report when work finishes (batch inference, long fine-tuning jobs). Receivers still verify, store, ack quickly and queue. Agent products get webhook triggers from email, calendars and CRMs (customer relationship management tools such as Salesforce). There, deduplication matters twice as much, because a repeated event can kick off an expensive agent run a second time.

In the interview

It can be asked straight out ("integrate Stripe") or indirectly ("how does payment settlement reach your system?"). Expect questions about work done before returning 200, about duplicates, and about spotting events that never arrive. The completeness question usually reveals the most.

30-second answer: "My webhook handler does three things: verify the signature and timestamp, save the raw event, enqueue it. It returns 200 in about 15 ms, because 200 means received, not processed. Workers dedupe on event ID, since delivery is at-least-once, and fetch current state from the provider's API before state-dependent decisions. Then two protections: an hourly reconciliation poll from a bookmark, because webhooks are best-effort and the worst failure is silence, and an alarm if a normally busy endpoint goes quiet. Speed from push, truth from poll."

Likely follow-ups:

- *"How do you know you missed one?"* The webhook path can't tell you by itself, because a missing event leaves no trace. Reconciliation eventually fills in every gap, and the silence alarm catches big gaps fast. Treat "no traffic" as a state you monitor.
- *"Secure the endpoint."* Check the HMAC over the raw body using the shared secret and a constant-time comparison. Check the timestamp to block replays. Rotate secrets safely. Reject invalid requests before parsing them. IP allowlists can add a layer, but the signatures are what authenticate.

Mistake that fails candidates: doing business processing before replying, or ignoring duplicate delivery. Either design only works as long as every dependency behaves.

Self-check

- **Q1: List the five receiver rules in order and say exactly what 200 means.** → Verify the signature + timestamp; save the raw event durably; ack with 200 within milliseconds; process later through a queue with event-ID dedupe (fetching the current state where needed); reconcile from a bookmark. 200 means the event is safely in your hands, *not* that its effects are finished.
- **Q2: The payload already has everything; why still fetch current state, and when would you skip it?** → The payload shows things as they were when that delivery attempt was created. After retries it can be out of date, or it can arrive out of order, while the API holds the current official state. Skip the fetch for self-contained events that only add records, when the volume would go over the API rate limit, or when the action is idempotent and cheap to fix.
- **Q3: Confirmations quietly stopped; no handler errors for 3 weeks; the dashboard shows the endpoint was auto-disabled 20 days ago after a failure spike during a deploy. Reconstruct and redesign.** → The deploy caused slow responses or errors. Retries ran into the same problem. The failure rate triggered the auto-disable, and deliveries stopped without any local error. Redesign: a 15 ms save-and-ack handler (no business work inside the request), a silence alarm that catches a disable within minutes, hourly reconciliation to limit any gap, and deploy checks that confirm the endpoint is healthy. Never let completeness rest only on another company's retry window.

Related patterns

- **Message Queue:** keeps accepting the news separate from doing the work; the handler's only job is to feed it.

- **Idempotency:** dedupe by event ID is the cross-company version of idempotency keys.
- **Publish-Subscribe:** structurally, this is what a webhook is, carried over HTTP across a company boundary, with the guarantees worked out again as contract terms.
- **Retry with Exponential Backoff:** runs on the sender's side, and can work both for you and against you.

TL;DR

- **Problem:** outside events arrive as POSTs under the sender's timeout, retry and disable rules. Doing the work inline leads to duplicates, a disabled endpoint, and a silent 3-hour gap that support tickets uncover.
- **Mechanism:** verify the signature + timestamp, save the raw event, return 200 within ms, process behind your queue with event-ID dedupe, fetch current state when deciding, and reconcile by polling from a bookmark.
- **Costs:** you take on the sender's rules, defend a public endpoint, debug across companies, and run the polling check forever.
- **Skip it when:** you own both ends (use pub/sub), completeness is the requirement (poll first), or the volume calls for a stream.

Server-Sent Events

Original lesson: Server-Sent Events on DesignGurus ↗

What it is

Server-Sent Events (SSE) is a single HTTP response that never finishes. The client connects one time. The server keeps that connection open and sends a small text message down it whenever there's something new. **Analogy:** rather than phoning the pizza place every 3 minutes to ask "is it ready yet?" (that's polling), you stay on the line, and they tell you the second anything changes.

The problem: polling that mostly says "nothing"

A food-delivery app shows the driver's location, the order status and the ETA. The page **polls**: it calls `GET /orders/{id}/status` every 3 seconds. It's simple, it's reliable, and it's in production.

On a rainy Friday evening, **200,000** customers have that page open. One request every 3 s from each of them = **~66,000 requests/s**. Each is a full round trip with TLS, authentication, a DB read and a response. And **99 of every 100 responses say "no change."** Most of the capacity and cost is spent answering "nothing yet."

Users still grumble that updates feel slow. With a 3 s interval, a new status waits **1.5 s on average** until the next poll picks it up. Polling every 1 s would shrink that, but it would push traffic up to **200,000 req/s**. Meanwhile, an order changes status maybe **12 times in 40 minutes**. The key line of this lesson: **with polling, cost grows with the audience; it should grow with the news.**

Why the obvious fixes fail

- **"Tune the polling interval."** Shorter means fresher but more expensive; longer means cheaper but staler. You're only sliding along the same trade-off curve. The server already *knows* when the data changes; it just has no way to say so.
- **"Long polling."** The server holds each request open until the data changes or a timer runs out, and then the client immediately asks again. That gets rid of most empty responses and nearly all the staleness; it's an older kind of push. But each update still costs a disconnect and a fresh request. So a burst of updates means a burst of reconnections, and proxy timeouts need extra care. It's push faked through repeated pulls.
- **"Just use WebSockets."** That does work, but it gives you *two-way* communication for a page that only listens. The direction you don't use still costs a protocol upgrade, special load-balancer handling, and your own reconnect logic (see the comparison table below).

How it works

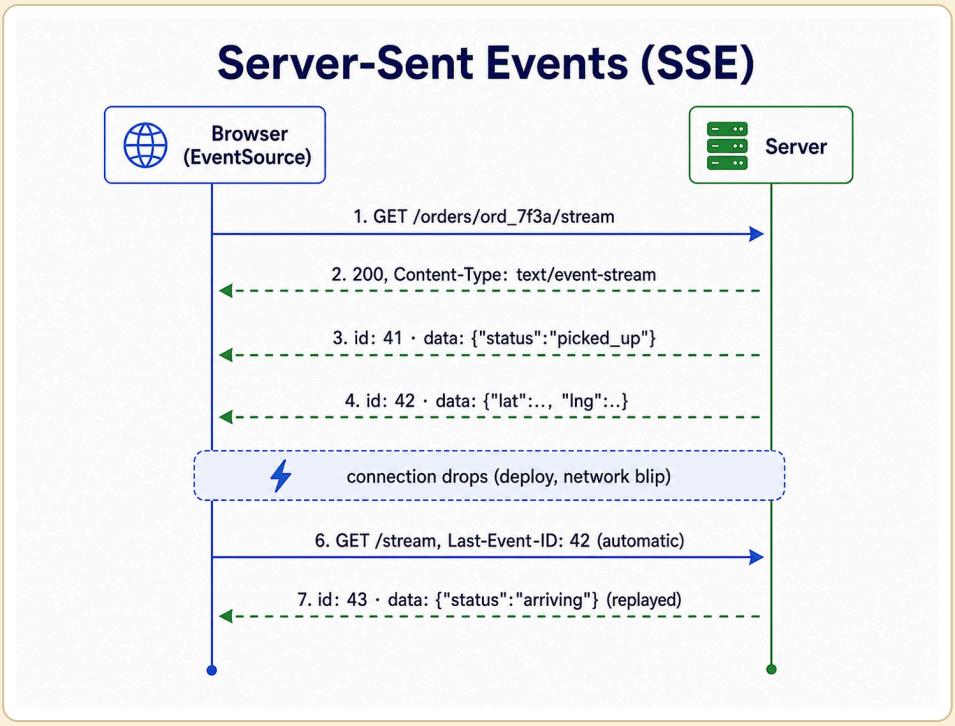


Figure — Server-Sent Events (course diagram)

The figure above shows the entire protocol in seven steps. The browser's `EventSource` opens `GET /orders/ord_7f3a/stream`. The server answers `200` with `Content-Type: text/event-stream` and leaves the response open. It pushes numbered events (`id: 41` status picked_up, `id: 42` a location). The connection drops (because of a deploy or a network blip). The browser reconnects **automatically** and sends `Last-Event-ID: 42`. And the server replays what was missed (`id: 43` "arriving"). Four properties make this work:

1. **It's just HTTP.** An SSE stream is a normal HTTP response that stays open, with the content type `text/event-stream`. There's no protocol upgrade and no special handshake, so your existing load balancers (servers that spread requests across many backends), TLS, authentication **middleware** (shared code that runs on every request before your handler, e.g. to check logins) and proxies all handle it. Most of its operational simplicity comes from this.
2. **The browser handles reconnecting.** Deploys, tunnels and closed laptops break connections constantly. The browser's `EventSource` API reconnects by itself and sends a `Last-Event-ID` header naming the last message it received. The server keeps a short **replay buffer** of recent events and resends everything after that ID. A reconnect could deliver a message twice, so **event IDs also serve as dedupe keys**, and the client skips IDs it has already handled.
3. **Only one direction, on purpose.** The server talks and the client listens. A client action, like cancelling an order, goes through an ordinary request-response call. A lot of real-time UIs have exactly this shape.
4. **Held-open connections change the server math.** 200,000 connections can't each tie up a thread. **Async I/O** (Node, Go, Nginx; I/O means input/output such as network reads, and *async* means the thread doesn't sit blocked waiting for it) lets a single **event-loop** thread keep watch over thousands of idle connections and only do work when data shows up. Each connection costs a few tens of KB of memory, so one server can hold **tens of thousands of connections**.

The SSE server normally gets its updates by **subscribing to internal pub/sub** and passing each event on to the right open streams. **Keep streams tightly scoped:** each stream should only carry events the logged-in user is allowed to see, and routing has to be checked again after account or membership changes. Piping a wide internal topic straight to browsers turns a routing bug into a data leak.

Infrastructure gotchas. Some load balancers, **reverse proxies** (servers that sit in front of your app and forward requests to it) and corporate networks **buffer** responses, meaning they collect the output before passing it on. That's harmless for normal responses but deadly for streams: events show up in batches every 30–60 s. So:

- Turn off buffering on stream routes (for example, the header `X-Accel-Buffering: no`, plus `Cache-Control: no-cache`).
- Raise idle timeouts, and send a **heartbeat** comment roughly every **15 s** so equipment in the middle doesn't close what looks like an idle connection.
- Use **HTTP/2** (the newer version of HTTP that can carry many requests over one connection) at the **edge** (the servers that face the internet) so many streams can share one connection. This gets around browsers' old limit of **6 connections per domain**.

Authentication has to last longer than a normal request. Check the session when the stream opens. Close the stream when the credentials expire or get revoked, because a single connection may stay open for hours. The reconnect then re-authenticates the usual way.

Walkthrough with numbers

- **When nothing happens, nothing gets sent** (except heartbeats). There are 200,000 open connections and almost no requests, so the **66,000 req/s** of "nothing yet" disappear. Work now follows events, not audience size.
- **When there's news,** the customer sees it in **under a second**, compared with up to 3 s of staleness when polling.
- **Capacity:** one async server holds **~50,000** connections, so **4 machines** handle a rainy Friday. The cost moves from CPU (for 66k req/s) to memory (for the held connections).
- **Deploy drill:** restarting a server drops 50,000 connections, and all of them reconnect automatically. If they all come back at the same instant, that burst (a **reconnect stampede**) can overwhelm the fleet. The SSE `retry:` field lets the server set the reconnect delay. Randomize that delay (this is called **jitter**) to spread the load out. Returning clients send `Last-Event-ID`, and the buffer fills in what they missed. So keep **several minutes** of events, which is longer than a typical deploy.

Trade-offs

You gain:

- Push over plain HTTP: middleware, proxies and tools keep working with no changes.
- A client built into the browser, with reconnect-and-resume you didn't have to write.
- Cost that tracks events, not audience size or polling frequency.

You pay:

- **Only one direction.** As soon as the client has to stream typing, cursors or voice back, SSE isn't enough (see Bidirectional Streaming).
- **Running long-lived connections.** Plan capacity by *connections*, not requests. **Drain** servers before deploys: stop taking new connections and let the existing ones move elsewhere. Use **least-connections** load balancing, because two servers can have similar request rates while one holds far more streams and memory. Treat active connections, memory per connection, disconnect and reconnect rates, and replay-buffer misses as top-level metrics. Load-test idle connections and simultaneous broadcasts separately, since they strain different resources.
- **A window for resuming, not a history.** The replay buffer covers short disconnects, not yesterday. If a client comes back after its ID has aged out of the buffer, it has to notice the gap and refetch the current state through the normal API. Spell this out in the protocol. The server sends a named reset event (or closes with a status that means "refetch"). The client replaces its local state with the refetched data *before* reopening the stream, so old and new data never get mixed together. **The stream is for delivery, never for storage.**
- **Text only.** Text and **JSON** (a common text format for structured data) work fine; binary audio/video needs a media protocol.

When not to use it

- **The client talks as well** (typing indicators, collaborative editing, multiplayer games): use Bidirectional Streaming.
- **Updates are rare and viewers are many.** A price that changes once a day doesn't justify a million idle connections; relaxed polling plus a cache is simpler.
- **Binary media where latency is critical** (voice, video): use **WebRTC**, which is designed for media.
- **Between your own backend services:** use pub/sub or **gRPC** streams (typed streaming from server to server).

Anti-patterns: a homemade chat built from SSE going down plus constant POSTs going up (that's a two-way protocol in disguise), and treating the stream as storage with no way to recover once a client falls outside the replay window.

Four levels of push

Choose the lightest method that meets the direction and freshness you need:

	Polling	Long polling	SSE	WebSocket
Staleness	~Half the interval	~Zero	~Zero	~Zero
Cost when idle	A constant flow of requests	One held request per client	One held connection per client	A held connection + the protocol
Direction	Pull	Pull pretending to be push	Server → client	Both ways
Works with existing proxies/infra	Perfectly	Timeout problems	Yes, it's plain HTTP	Needs special handling
Reconnect handling	Not needed	You write it	Built into the browser	You write it
Best for	Rare changes, relaxed deadlines	Old infrastructure limits	Feeds, tickers, progress, LLM (AI chat model) tokens	Conversations

Ask **who has data to send**. Poll when the server speaks now and then, use SSE when it speaks constantly, and use WebSocket when *both* sides speak constantly. Long polling mostly survives in places where old infrastructure can't carry SSE reliably.

Real-world examples

- **Every LLM chat app streams over SSE.** (An **LLM**, large language model, is the kind of AI model behind chat assistants; it produces text in small pieces called **tokens**, roughly word fragments.) Anthropic's and OpenAI's APIs send tokens as SSE events, which is why the words appear one at a time. It's a perfect match: strictly one-way, text, and plain HTTP that every proxy and **SDK** (software development kit: a ready-made client library) understands.
- **Live dashboards and tickers:** log tails, stock and sports feeds, live news.
- **Build and deploy logs:** **CI** (continuous integration: servers that automatically build and test every code change) output that streams into your browser is nearly always SSE.
- **Notification feeds:** one SSE stream per session that carries all in-app events (the capstone builds this).
- **For AI engineers:** streaming tokens makes latency *feel* shorter even when generation takes just as long. A 30 s answer feels snappy if the first token shows up in **300 ms**, so **time-to-first-token** is the metric that counts. A buffering proxy ruins this by holding tokens back. Agent UIs send **named event types** (tool started, result arrived, thinking), and an LLM gateway has to pass streams straight through rather than collecting whole responses first.

In the interview

Live order tracking, notification feeds and token-by-token AI output all check the same skill: can you choose among the four push levels instead of reaching for WebSockets by default or sticking with expensive polling?

30-second answer: "For one-way live updates I use SSE. Each client holds one long-lived HTTP response, and the server pushes events as they happen, so cost tracks news, not audience. Polling here would be hundreds of thousands of requests a second, mostly 'nothing yet.' SSE is plain HTTP, so auth, load balancers and proxies just work. EventSource reconnects automatically with Last-Event-ID, and the server replays from a buffer sized to cover a deploy. Async I/O servers hold tens of thousands of connections each, and randomized reconnect delays prevent stampedes. When the client needs to stream back, for chat or collaboration, I move to WebSockets, not before."

Likely follow-ups:

- *"A server holding 50,000 streams restarts. Walk me through it."* Drain it first: refuse new streams so the load balancer sends them to other servers. Clients that got dropped reconnect after randomized delays, carrying `Last-Event-ID`, and the replay buffer restores the events they missed. Size the buffer to match how long a deploy takes.
- *"Why not WebSockets for everything?"* Two-way communication needs a protocol upgrade, special handling in the infrastructure, and your own resume logic. All of that is wasted when the client only listens. Switch to WebSockets only when you actually need to stream upward.

Mistake that fails candidates: using WebSockets by default for a feed that's read-only; brushing off per-user polling traffic as "cacheable" (it isn't, because each user's status is different); or treating the stream as storage with no refetch path once a client is past the replay window.

Self-check

- **Q1: What single transport fact explains most of SSE's operational advantages, and which two mechanisms make a dropped connection invisible?** → It's plain HTTP with no upgrade, so the TLS, auth, load balancers, HTTP/2 and proxies you already have can carry it. What makes drops invisible: EventSource's automatic, jittered reconnect, plus `Last-Event-ID` checked against a server replay buffer (with event IDs also acting as dedupe keys).
- **Q2: The replay buffer holds 60 s; deploys take 4 minutes. What happens, to whom, and what are your three options?** → Clients that got disconnected early in the deploy come back after their last event has already aged out. They'd quietly miss events unless the server notices. Options: make the buffer bigger; have clients refetch the current state before reopening (you need this no matter what); drain servers so disconnections are shorter.
- **Q3: Corporate-network users get "live" updates in 30–60 s bursts; home users are smooth; server logs show immediate writes. Diagnose and fix.** → A corporate proxy is buffering the response. Turn off buffering on the route (`X-Accel-Buffering: no` or the equivalent), set `Cache-Control: no-cache`, send heartbeats, and prefer end-to-end HTTP/2. Where you can't change their infrastructure, detect the batching and fall back to long polling. The SSE path includes *every* proxy sitting between the server and the browser.

Related patterns

- **Publish-Subscribe:** where the news comes from; the SSE server is a subscriber that forwards events to open streams.
- **Request-Response:** carries the client's actions upward, and is the refetch path after a gap in resuming.
- **Bidirectional Streaming:** the next step up once the client starts talking.
- **Load Balancing:** held connections need least-connections balancing and planned draining.

TL;DR

- **Problem:** polling makes cost grow with the audience and staleness grow with the interval; lots of heavy infrastructure mostly answers "nothing yet."
- **Mechanism:** one HTTP response per client that never ends; the server pushes text events; EventSource reconnects on its own with Last-Event-ID against a replay buffer; async I/O holds the connections; randomized reconnect delays prevent stampedes.
- **Costs:** only one direction, the work of running long-lived connections (draining, heartbeats, proxy buffering), a window for resuming rather than a history, and text only.
- **Skip it when:** the client streams too (WebSockets), changes are rare (polling), the payload is media (WebRTC), or both ends are your own servers (pub/sub).

Bidirectional Streaming

Original lesson: Bidirectional Streaming on DesignGurus ↗

What it is

Bidirectional streaming keeps a single connection open, and *both* sides can send messages over it whenever they want. Think chat, collaborative editing, multiplayer games, or live voice. In browsers this is normally a **WebSocket**, which is an ordinary HTTP connection that gets "upgraded" into an open channel that works both ways. Between backend services it's normally a **gRPC** bidirectional stream. The big insight of this lesson: opening the socket is the easy bit. The hard bit is that a conversation is **shared state spread over many machines**. **Analogy:** placing a phone call is simple. The real challenge is running a conference call where the people dialed into *different* switchboards. Someone has to pass along what each person says to every switchboard that has a listener on it.

The problem: messages that silently vanish

A team launches WebSocket chat. It works flawlessly in the demo and in **staging** (a test copy of the production environment). Production, though, has **three gateway servers** behind a load balancer (a **gateway** is a server that holds client connections). Alice connects to server A. Bob joins the same room through server B. Alice sends "hey." Server A looks at its own local list of connections in that room, sees only Alice, and so Bob gets nothing.

Nothing shows up in the error logs. People on the same server chat just fine. People on different servers miss messages, or see them only one way. Room membership was stored only in each server's own memory. Staging had just one server, which saw everyone, and that hid the flaw.

The socket was never the hard part. SSE usually sends outward from a single central source. With two-way chat, a message *that a client creates* has to reach recipients who are connected to other machines.

Why the obvious fixes fail

- **"Sticky sessions."** (Sticky means the load balancer keeps sending a client to the same server.) A load balancer should keep each client on the same server, because a live connection can't be moved. But stickiness pins *clients*, not *rooms*, so Alice and Bob can still end up on different servers. You need it, but it isn't enough.
- **"Route each room to one server."** This works as long as a room fits on one machine. But large all-hands rooms outgrow it, one server failing takes down every room on it, and moving busy rooms means transferring state. You've accidentally built a stateful, sharded distributed system that can fail over, without ever designing one.
- **"One big server."** Scaling up one machine (vertical scaling) pushes the limit back but can't get rid of it. The biggest machine still has a ceiling, and when it fails, everyone gets disconnected.

All three try to solve **message routing** by controlling **where connections land**. The design that scales keeps those two things separate.

How it works: two tiers

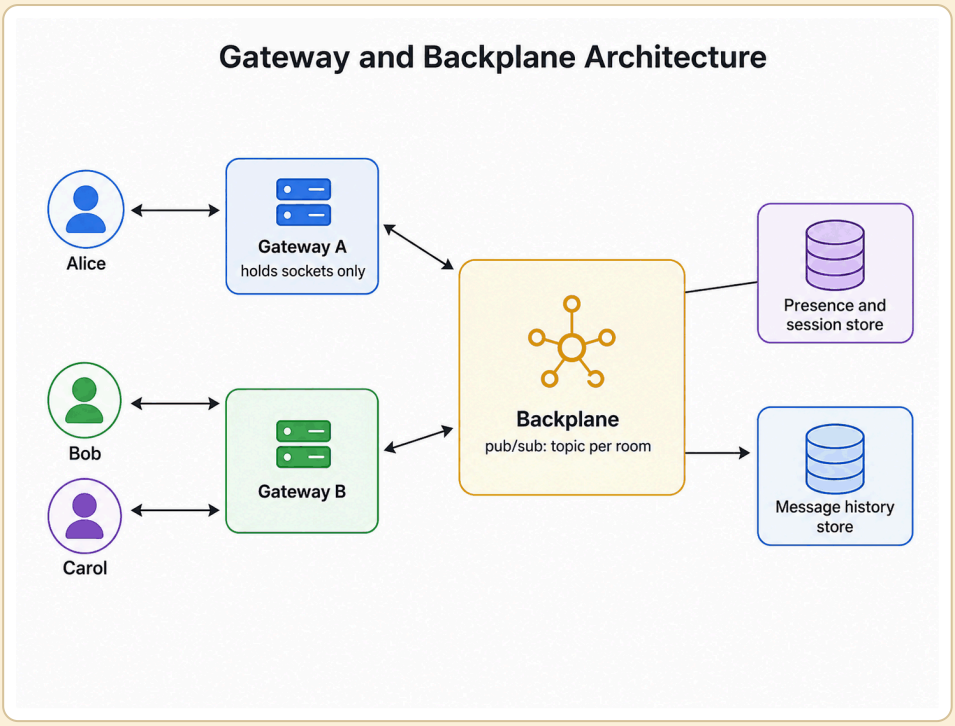


Figure — Bidirectional Streaming (course diagram)

The figure above shows the gateway-and-backplane design. Alice's socket sits on Gateway A ("holds sockets only"), and Bob and Carol are on Gateway B. Both gateways connect to a **backplane** (pub/sub, with one topic per room). Next to it are a **presence and session store** (who's online) and a **message history store** (the durable record of every message).

Here is the reasoning, one step at a time:

1. A connection is a physical fact: it exists on exactly one machine.
2. A room is a group of people whose connections exist on different machines.
3. So machines have to pass messages to each other. But making machines keep track of "who is on which machine" is fragile.
4. Instead, reuse pub/sub. **Each room is a topic.** Every gateway subscribes to the topics of the rooms its connected users are in. To send to a room, you publish to its topic. Each gateway passes what it receives on to its own local sockets. **No gateway ever has to know that any other gateway exists.**

So Alice's "hey" travels socket → Gateway A → room topic → every gateway subscribed to that room (A and B) → local sockets. That removes the launch bug by design. **Presence** is kept in a shared store and **messages** in a history database: **the stream carries data; the history database holds the truth.**

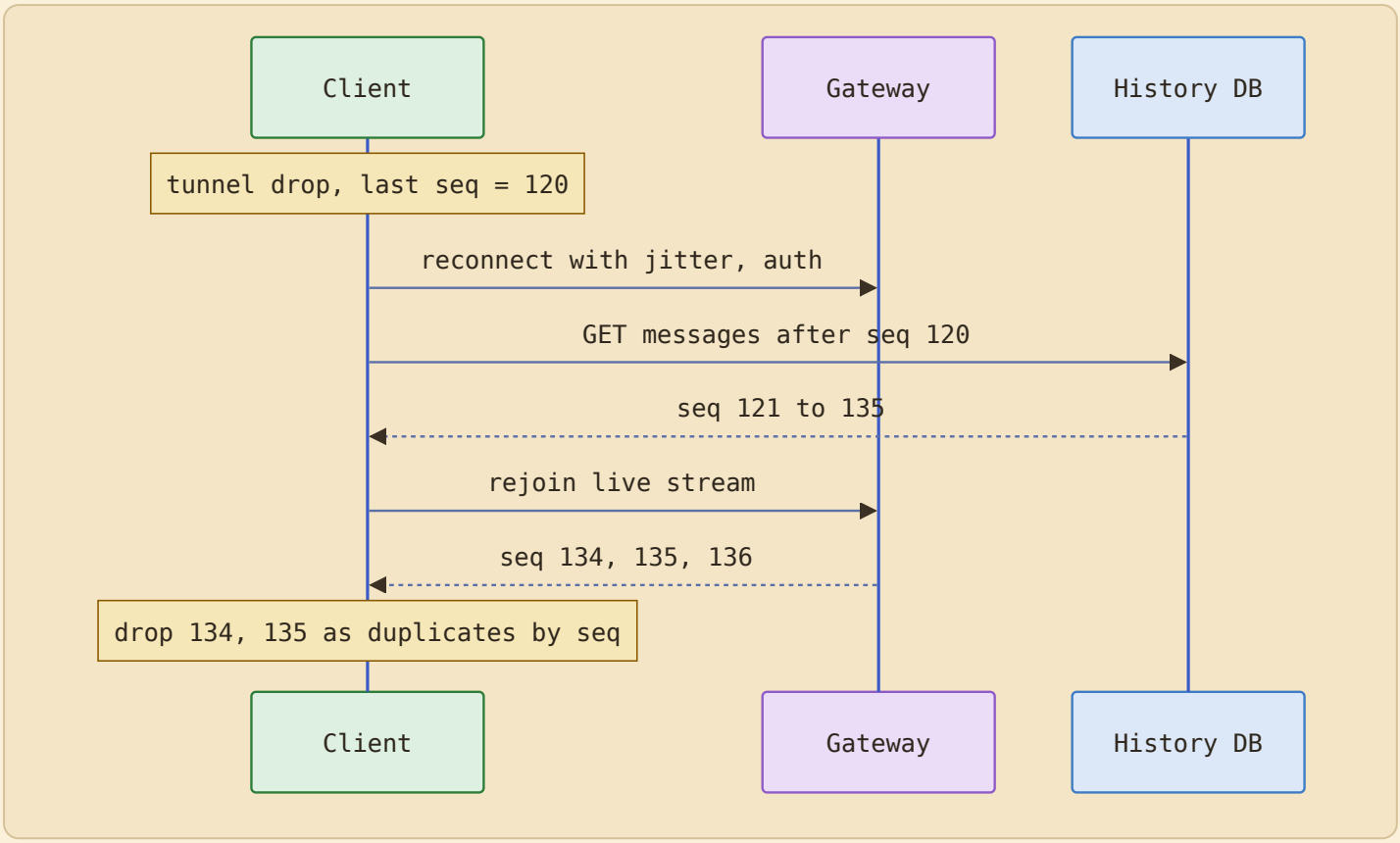
You now own a protocol. A WebSocket or gRPC stream gives you raw transport and nothing else. Everything that SSE's `EventSource` handed you for free, you now have to design yourself:

1. **Message framing and types, with versioning.** Define how each message is structured and how that structure changes over time. Keep **durable messages** apart from **ephemeral signals** (short-lived ones). A chat message needs an ID, sender, room, sequence number, timestamp and version, because it gets stored and may be replayed. A typing indicator is overwritten by the next one, expires by itself, and is never stored. Keeping these two classes clearly separate stops cheap signals from getting costly durability guarantees.
2. **Sequence numbers.** Give each room message a number when it's published, so clients can spot duplicates and gaps.
3. **Heartbeats.** A regular ping/pong lets both ends notice a dead connection instead of sending into nothing.
4. **Auth and acks.** Check the user when the socket opens, and check again from time to time during long sessions (room access or account status can change). Where delivery matters, add **acks** at the application level: a successful write to the socket does *not* mean the client got the message.

5. **Reconnect and resume.** The client saves its last sequence number for each room. After reconnecting, it fetches the messages it missed from the history store (using request-response), then rejoins the live stream and throws away any overlap by sequence number. Resuming makes delivery at-least-once, so sequence numbers act as dedupe keys. **History has to be written before publishing to the backplane, or in the same atomic step** (*atomic* = it either fully happens or doesn't happen at all); otherwise the query for the gap can miss messages.
6. **Backpressure.** What happens when a client receives more slowly than the system sends? With no limit, one weak mobile connection can pile up enough buffered data to use up the gateway's memory. That takes down everyone on that gateway (this is the shared-resource risk that the **bulkhead** pattern deals with). So: put a cap on each connection's outgoing buffer. **Coalesce** updates that replace each other (typing and cursors only need the newest value). And once the cap is hit, **disconnect with a "please resync" hint**. The normal reconnect-and-resume path then brings back the durable messages safely.

The sequence below shows how resuming works after a dropped connection:

Reconnect and resume via history



The last sequence the client saw was 120. After it reconnects (with jitter, and logging in again), it asks history for everything after 120, and it gets back 121–135. Then it rejoins the live stream and receives 134, 135, 136. It throws out 134 and 135 as duplicates, and ends up with a complete conversation with no gaps.

Walkthrough with numbers

- **One million connected users** at **~50,000 sockets per async gateway → 20 gateways**. Adding more gateways adds connection capacity without changing where rooms live.
- **Message path:** two internal hops (gateway → backplane → gateway), which is a few tens of ms from end to end. A 50-member room costs **one publish** (delivered to whichever gateways hold members), rather than 50 separate sends from Alice's server.
- **Typing indicators** can create **50x** more traffic than real messages if every keystroke goes out to every member. Throttle them to **at most 2 updates/s per typer**, with the newest one winning. The gateway handles this, and they're never stored.

- **Deploy drill:** stop the gateway from taking new connections (the load balancer sends them elsewhere), send current clients a "going away" notice, and let them reconnect somewhere else after randomized delays (jitter prevents a reconnect stampede). They resume by sequence number from history.
- **Slow consumer:** a subway rider's buffer reaches its **1 MB cap**. The gateway disconnects them with a resync hint instead of hoarding data. Roughly **30 s** later, the client reconnects and fetches the messages it missed from history. The stream never served as storage.

Trade-offs

You gain:

- A real conversation: both sides can send at any moment over one connection, with almost no overhead per message.
- The lowest interactive latency anywhere in the HTTP family.
- A single channel for an entire session: messages, **presence** (who's online), and signals.

You pay:

- **You own the protocol:** framing, sequencing, heartbeats, resume and versioning, including the failures HTTP and SSE already took care of.
- **A stateful edge.** Gateways hold connection state, so they need socket limits, planned draining, and load balancing that respects held connections (least-connections *and* stickiness).
- **Your HTTP tools can't see messages anymore.** After the upgrade, your usual auth, rate-limiting and tracing middleware can't see individual messages, so you need versions of them that understand sockets.
- **You must design backpressure**, because any buffer without a limit will eventually eat all the memory.
- **The backplane is yet another system**, and its throughput limits **fan-out** (how many recipients one message can reach). Load-test big rooms (one message to 100,000 recipients) apart from total volume (100,000 private messages). Measure backplane delay, gateway write queues and history writes separately, so you can see which stage is the bottleneck.

When not to use it

- **The client hardly ever sends.** Use SSE going down plus ordinary POSTs going up. Sockets are worth it for *frequent, latency-sensitive* two-way data (typing, cursors), not for a Like button.
- **Request-response over a socket.** Sending `{"method": "getUser"}` and waiting for a reply rebuilds HTTP, but without status codes, caching or standard tools. Calls that have responses should stay calls.
- **Between your own backend services.** Native gRPC streams give you typing, **multiplexing** (many streams sharing one connection) and service-mesh support. Use a queue when work has to survive restarts.
- **Media.** WebRTC is designed for low-latency audio/video that can tolerate some loss (its setup may use a WebSocket).

Once a socket is in place, teams get tempted to send forms, uploads and fetches through it too. That creates a private protocol that lacks HTTP's tools. Save the socket for continuous two-way traffic only.

WebSocket vs gRPC bidirectional stream

They have the same communication shape but live in different places:

	WebSocket	gRPC bidirectional stream
Natural home	Browsers, mobile apps, the public internet	One backend service talking to another
Message format	You define it (raw frames)	Typed **protobuf** (Google's compact binary message format) messages, with generated code
Through proxies/firewalls	Excellent (it looks like normal HTTPS)	Unreliable on the public internet
Tooling	Needs special handling everywhere	Fully supported in gRPC and service meshes
Best for	Chat, collaboration, multiplayer	Internal control planes, live links between services

A simple rule: **WebSocket faces users; gRPC streams face services. Envoy's xDS** is a typical gRPC stream. It pushes configuration to proxies in a **service mesh** (a layer of small proxies next to every service that handles networking, retries and security for them) over long-lived, typed connections. **WebRTC** is a third option, used for peer-to-peer media, and it only uses these transports for setup. Failure handling is different too. Mobile and browser clients switch networks, go to sleep, and sit behind infrastructure you don't control, so WebSocket designs need explicit reconnect and history recovery. Backend gRPC clients run on networks you control, but they still need deadlines, **flow control** (the stream-level version of backpressure), and a clear policy for resuming after a service restarts.

Real-world examples

- **Slack:** uses exactly this two-tier design, with WebSocket gateways plus a backplane. Its write-ups on draining and reconnect stampedes read much like this lesson's walkthrough.
- **Figma:** multiplayer editing is WebSocket transport with **CRDTs** on top (conflict-free replicated data types, which are structures that merge edits made at the same time in a predictable way). A line worth using in interviews: *the transport delivers; the data structure resolves concurrent edits.*
- **Discord:** millions of sockets open at once on a small gateway tier, where gateways are treated as completely disposable.
- **Envoy xDS and Kubernetes watch APIs:** long-lived streams used to push **control-plane** updates (configuration and coordination messages, as opposed to user data; Kubernetes is the most common system for running containers across many machines), entirely machine to machine.
- **For AI engineers:** realtime voice sends audio in both directions over a single WebSocket. That makes **barge-in** possible (cutting the model off while it's speaking). Barge-in can't work on a one-way channel, because the client has to send while the server is still streaming. The same concerns apply: number the audio frames, apply backpressure when generation runs ahead of playback, and resume when a phone switches networks. Output that's only tokens stays on SSE; an interactive conversation is what justifies the socket.

In the interview

Questions about Slack, collaborative editors and multiplayer games often begin with "use WebSockets" and then dig into the topology. The key question: **"how does Alice on server A reach Bob on server B?"** Another one: **"what happens to a slow consumer?"** (buffers without limits can take a gateway down).

> **30-second answer:** "Two-way needs two layers. Gateways hold WebSocket connections and nothing authoritative. Behind them a pub/sub backplane, one topic per room, routes each message to whichever gateways hold recipients. Presence is in a shared store and history in a database, so the stream is transport, never the source of truth. On top of the socket I own a small protocol: sequence numbers, heartbeats, and reconnect-and-resume that fetches missed messages from history and dedupes by sequence. Each connection's send buffer is capped, ephemeral signals like typing are merged latest-wins, and past the cap I disconnect with a resync hint rather than burn memory. So gateways are disposable: drain, deploy, reconnect with jitter."

Likely follow-ups:

- *"Walk Alice's message to Bob on a different server."* Alice sends through Gateway A, which publishes to the room's topic. Gateway B is subscribed (because it holds Bob), so it gets the event and writes it to Bob's socket. Gateway A also delivers to its own local members. The message is saved to history at publish time, so resuming has one official source.
- *"A client receives at 1 KB/s in a room producing 50 KB/s. What happens?"* First, coalesce the ephemeral updates. If the capped buffer still fills up, disconnect the client with a resync signal. When the client reconnects, it fetches the durable history and rejoins. The gateway's memory stays safe instead of buffering with no limit.

Mistake that fails candidates: storing room membership only in gateway memory, skipping reconnect-and-resume, or using WebSockets for a stream where the client never sends anything.

Self-check

- **Q1: What must you build above a raw socket that SSE's EventSource gave you free, and where does truth live?** → Randomized reconnect, resume based on sequence numbers, framing and versioning, acks where you need them, heartbeats, auth when the socket opens plus later re-checks, and a backpressure policy. The history store owns the truth about messages, and a shared store owns presence. Sockets and the backplane only move data around.
- **Q2: Why gateways plus a backplane rather than gateways messaging each other directly? Which state lives where?** → Direct routing would need every server to keep a global table, always correct, of where each room's members are, through connects, disconnects and failures. That's the same fragility as the in-memory buffer from the queue lesson. The backplane handles subscriptions in one place, and each gateway knows only its own sockets and topics. Gateways: sockets + capped buffers. Backplane: routing subscriptions. Shared store: presence and sessions. History DB: messages.
- **Q3: Mobile users lose messages after elevators and tunnels; connections do come back, but conversations have gaps. Design the fix end to end.** → Reconnecting without resuming skips everything published during the outage. Number messages when they're published. Store them in history (before publishing, or atomically with it). Have clients save the last sequence number for each room. After reconnecting, fetch everything newer through request-response, apply it, then rejoin live and drop any overlapping sequence numbers. It's SSE's `Last-Event-ID`, rebuilt for a two-way protocol.

Related patterns

- **Publish-Subscribe:** the backplane is one; rooms are topics and gateways are subscribers.
- **Server-Sent Events:** the simpler one-way choice. Only move up from it when the client streams, and borrow its resume design.
- **Load Balancing:** stickiness, least-connections and draining are what make a stateful edge possible to deploy.
- **Idempotency:** resuming makes delivery at-least-once, and sequence numbers are the dedupe key.

TL;DR

- **Problem:** a conversation is shared state across connections sitting on different machines. Once you have more than one server, "send to the room" turns into a routing problem. The socket was never the hard part.
- **Mechanism:** gateways hold only sockets; a pub/sub backplane routes by room; presence lives in a shared store; the truth lives in a history DB; and you write your own protocol of sequences, heartbeats, resume, and backpressure with capped buffers.
- **Costs:** you own the protocol, a stateful edge with draining and stampedes, HTTP middleware that can't see anything after the upgrade, and a backplane that limits fan-out.

- **Skip it when:** the client seldom sends (SSE + POST), it's request-response in disguise, it's service-to-service (gRPC), or it's media (WebRTC).

Capstone - Designing a Notification System

Original lesson: Capstone - Designing a Notification System on DesignGurus ↗

How this capstone works

The last seven lessons gave you seven ways to move data, and this capstone puts them together. The main idea: communication patterns aren't a list of steps to follow in order. Instead, you **pick one for each arrow**. A notification system uses every direction covered in this module. Facts flow in, jobs travel through queues, calls go out to outside providers, webhooks bring delivery status back, streams update browsers, and enterprise deliveries cross company boundaries. At every decision, ask four questions: **who has the information, is anyone waiting, is this a job or a fact, and who controls the destination?**

The brief

The same store has grown to **5 million registered users** and needs a single notification platform.

- **Triggers:** order updates, shipping events, security alerts, marketing campaigns, and anything future teams come up with.
- **Channels:** an in-app feed with an unread badge, mobile push, email, SMS, and, for enterprise customers, delivery directly into their own systems.
- **Scale:** a flash sale creates **600,000 order events in 2 minutes**; a marketing campaign sends one message to **5 million** users; roughly **300,000** users are online at any given moment.
- **Constraints:** the email provider limits you to **10,000 sends/s**; users have preferences and **quiet hours** (times they don't want to be disturbed); and regulators have rules covering all of it.

Decision 1 — Getting events in: pub/sub + full-state events

The tempting design: a `POST /notifications/send` API that each of six product teams calls whenever something happens. For example, checkout would call it inline for every order.

Why not: that's the pub/sub incident all over again, just seen from the consumer's side. Every producer gets tied to the notification service. Its latency ends up inside critical paths like checkout. And changes to its API could force producers to redeploy.

Chosen design: flip the direction. The notification platform becomes just **another subscriber** to topics that already exist: `order.placed`, `shipment.updated`, `login.suspicious`. Those topics carry **full-state events** (event-carried state transfer), so each event already contains everything needed to render a notification, with no callback to the producer. The platform reads additive schemas and ignores fields that don't matter for rendering. It also benefits from sensitive data being kept out of general events, since that means fewer stores and queues to clean up after a deletion request.

A direct API only for intentions that have no event: marketing campaigns and support messages aren't domain events. For these, the API checks and stores the request, then returns `202 Accepted` within milliseconds. That's request-response confirming *acceptance*, not delivery, because no caller should sit waiting for 5 million emails.

Rule: subscribe wherever facts already exist; only offer an API for intentions that have no event.

Decision 2 — The pipeline: message queue

The work: load preferences and quiet hours, pick channels, render templates, **deduplicate** (three events about one order must not turn into three notifications), and optionally group them into digests. Arrivals come in bursts (600k in 2 min). Workers keep up **~2,000/s**. Nobody is waiting for the result.

Chosen design: an ordinary message queue. Each notification is one job, which may finish minutes later. The flash sale turns into a **~5-minute backlog** (600,000 / 2,000 = 300 s) rather than an outage. The queue's must-follow rules come along with it:

- **Dedupe on (event ID, user) before causing any side effect**, because delivery is at-least-once.
- **A monitored DLQ**, so messages that keep failing don't hold up healthy ones.
- **Alert on how *old* the backlog is, not only how deep**, because a notification that's three hours late can do harm ("your table is ready").
- **Dedupe and preference checks happen once, in one central place**, not separately for each channel. Separate decisions lead to inconsistencies, like sending the email but not the push, or contacting a user who turned both off.
- **Quiet hours are a scheduled state, not retries.** Save the rendered notification along with its **earliest allowed send time**, and put it on the channel queue when that time comes.
- **Check preferences again right before the final send**, because the user might unsubscribe while the work is waiting.

Decision 3 — Delivering out (SES, Twilio, APNs): one queue per channel

The problem: there are three outside providers (AWS SES for email, Twilio for SMS, Apple's APNs for push). They have different speeds, limits and ways of failing, and all of them are fed from the same pipeline. How do you keep a Twilio outage from delaying email?

Chosen design: give **every channel its own queue and its own pool of workers**. Then an SMS backlog only affects SMS, while email keeps flowing at its 10,000/s limit. This isolation is the **bulkhead** pattern, and it falls out naturally from having one queue per dependency. Every call to a provider is request-response across a network, so it's wrapped in protections: a **timeout** (a deadline), **retries** of temporary failures that carry a **unique message key** (so the provider can dedupe), and a **circuit breaker** (stop calling a provider that's unavailable).

- **Rate limits apply per channel as a whole**, enforced across all the workers together rather than per process (all the email workers combined stay under 10,000/s; SMS and push follow their own providers' limits).
- **Each channel gets its own backlog-age threshold:** a late security alert is urgent, but a late marketing message isn't.

Decision 4 — Provider status back in: webhooks (receiver side)

Delivery receipts, bounces, complaints and unsubscribes come back from the providers as webhooks. Follow the receiver rules: verify the signature, save the raw event, ack within milliseconds, process behind a queue, dedupe by event ID, and reconcile through the provider's API to catch any that went missing. **Hard bounces** (addresses that reject mail permanently) have to **update the preference store**. If you keep sending to them, you hurt your **sender reputation**, which is how mail providers decide whether to trust your email.

Rule: protect outgoing request-response calls, and apply all five receiver rules to incoming webhooks. A single integration has both directions in it.

Decision 5 — In-app feed and badge: SSE over a database of record

The problem: ~300,000 users are online, and their feed and badge should update without a refresh. The server has the news, and this part of the client only *reads*.

Why not polling: 300,000 users every 5 s = **60,000 requests/s**, almost all of them "no change." **Why not WebSocket:** it's a two-way protocol for a client that has nothing to send upstream.

Chosen design: SSE. Each session opens one `EventSource`, and events update the badge within milliseconds.

- **The database is the truth.** Loading the page fetches the current feed through request-response, and SSE only carries live updates. After a short disconnect, `Last-Event-ID` replays the missed events; longer gaps cause a full refetch.
- **Multiple devices** just hold separate streams for the same user.
- **Mark-as-read** is an ordinary POST. It publishes an event that updates the badges on the user's other devices. Still no WebSocket required.
- **The badge count comes from the database**, not from adding up stream events. Otherwise, a single missed event would leave the number wrong forever. Any reconnect, or any detected gap in the

replay, refetches the official unread count.

Decision 6 — Enterprise delivery: webhooks (sender side)

Enterprise customers want order and security events POSTed to URLs inside their own infrastructure. Now **you're the provider**, and you owe them the sender duties. Sign every payload with **rotatable secrets**. Include unique event IDs so they can dedupe. Retry from a **durable queue per endpoint** on a published backoff schedule (so one dead customer can't hold up the others). Keep timeouts short. Automatically disable endpoints that keep failing. And give them a dashboard showing delivery status, with a redeliver button. The receiver and sender checklists are mirror images: whatever you expect from providers in Decision 4, you should provide to customers here.

Decision 7 — The escalation question: decline the WebSocket

The request: the product team wants users to reply to support notifications right from the feed, and suggests swapping SSE for WebSockets.

Why decline: yes, the client would now send, but only *once in a while*. Send each reply as an ordinary POST, and use SSE to update the user's other devices (just the shape the SSE lesson recommends). Bidirectional streaming is meant for **frequent, latency-sensitive traffic in both directions** (typing, live cursors, voice). A reply box doesn't need sequencing, resume, backpressure (slowing a fast sender down to match a slow receiver), or a stateful gateway tier. If the product later turns into **live support chat** with nonstop activity on both sides, give *just that feature* its own gateway-and-backplane system. Only add complex transport to the feature that actually needs it.

The final architecture

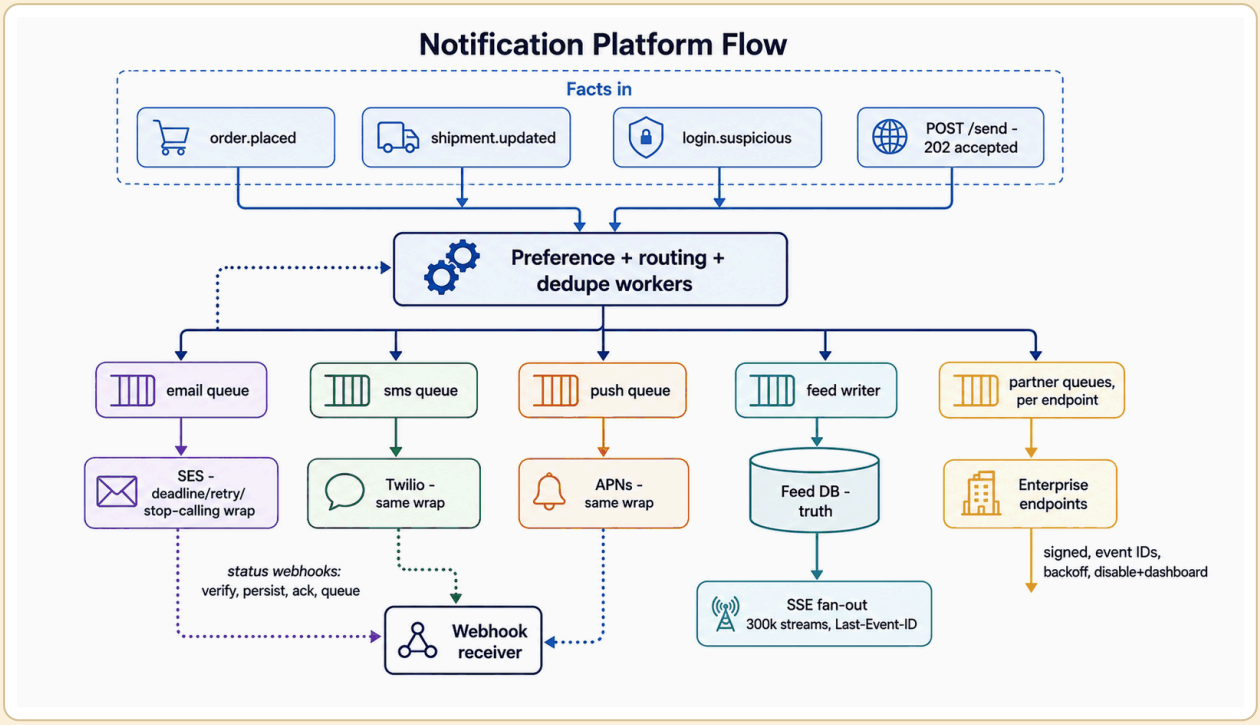


Figure — Notification System Architecture (course diagram)

The figure above shows every arrow with its route. Across the top are the **facts coming in**: `order.placed`, `shipment.updated`, `login.suspicious` (the subscribed topics), plus `POST /send → 202 accepted` for one-off sends. All of them feed the central **preference + routing + dedupe workers**. Those workers fan out into lanes for each channel. There's the **email queue → SES**, the **SMS queue → Twilio** and the **push queue → APNs**, each wrapped in deadline/retry/stop-calling protection. There's a **feed writer → Feed DB (the truth) → SSE fan-out** (300k streams, Last-Event-ID). And there are **partner queues per endpoint → enterprise endpoints** (signed, event IDs, backoff, disable + dashboard). Provider **status webhooks** come back into a **webhook receiver** (verify, persist, ack, queue), which loops back into the workers (for example, hard bounces updating preferences).

Debrief: one question per arrow

Arrow	The routing question answered	Pattern
Product events → platform	Facts that many parties care about; don't make producers call you	Pub/sub + full-state events
Ad-hoc sends → platform	The caller needs acceptance now and delivery later	Request-response (202) into a queue
Events → rendered notifications	Jobs that come in bursts, done once, with nobody waiting	Message queue + dedupe + monitored DLQ
Platform → SES / Twilio / APNs	A call into infrastructure someone else owns	Protected request-response, one queue per channel
Providers → platform (status)	Another company's events arriving at your public URL	Webhooks, receiver rules
Platform → browsers	The server has news; the client only reads	SSE over a database of record
Platform → enterprise systems	Push across the company boundary; your duties	Webhooks, sender rules
"Make it conversational"	The client sends, but rarely	Declined the socket: POST + SSE

If you read the middle column from top to bottom, it's the module's opening question asked eight times over. The result: **no WebSocket, one API, and seven asynchronous relationships**. Each one has a stated reason, and none was picked just for being new.

Two sources of truth: the **notification database** owns feed entries, unread state, preferences and delivery status. The **external providers** own their own delivery records, which are brought in through reconciliation. Queues, topics, SSE streams and webhook attempts only *carry* information; they never take the place of these durable records.

Left for later (Module 5): the defensive controls wrapped around every arrow: timeouts, limited retries, idempotent operations, monitored DLQs, and separate lanes for each dependency (bulkheads). This capstone decides *where data moves*; those patterns make sure each path survives failure.

Extend it yourself

- **Digests** ("bundle my marketing notifications into one daily email"): where does batching go relative to dedupe? What data model holds a digest that's still pending? What happens to digest state when a user changes timezone?
- **Multi-region:** users and their SSE connections now stretch across two continents, but the feed database and topics exist in one region only. Which arrows change? Which replication patterns (primary-replica, to begin with) do you need?
- **For AI engineers:** add an **LLM** stage that sums up each user's unread notifications into a morning brief. Route its arrows: batch inference through your own queue, or a provider batch API that reports completion with webhooks? If you stream the brief token by token in the app, which transport do you use (SSE)? And there's the **semantic cache** question (a cache that reuses an answer when a new request *means* nearly the same thing, not just when it's identical): should two users with almost the same unread lists share one generation?

Interview one-liner

"I route each arrow on its own. Producers publish facts the platform subscribes to; ad-hoc sends get a 202 into a queue. A deduping queue pipeline feeds per-channel, bulkheaded queues calling providers with timeout, retry and breaker. Provider status returns as verified, reconciled webhooks. Browsers get SSE over a database of record, and enterprises get signed, retried webhooks. There's no WebSocket anywhere, because nothing here streams both ways."

Module 3 — Storing Data (Storage and Replication)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. In Module 2 we looked at the *arrows* in an architecture diagram, meaning how the parts talk to each other. Now we focus on the most important box of all: the database. Most other parts of a system are easy to replace at any moment. API servers hold no lasting **state** (data remembered between requests); they are *stateless*. You can add ten of them or lose three, and users won't notice. The database is different. It holds the system's **facts**: payments, who owns what, and the record of what happened. You can't just start a second, independent database next to it. Two separate copies of the truth slowly stop matching. So as a product grows, the database is usually the first part to hit its limit. It is also the most dangerous part to change.

> **Analogy.** Picture the database as a company's official ledger book. Hiring more clerks (app servers) is easy. But you can't buy a second ledger and let each clerk write in whichever book is nearer. Before long, the two books would disagree.

The six patterns and when each applies

The situation	The pattern
Reads are too many for one machine, but every write still has to land in one place	Primary-Replica
There is more data, or more writes per second, than one machine can handle	Sharding
Adding a single extra machine should not make most of the data move	Consistent Hashing
A crash in the middle of a write must never lose or damage data	Write-Ahead Log
You need every change that ever happened, not just today's value	Event Sourcing
Reads and writes each need the data arranged in a different shape	CQRS

Two facts that organize the whole module

1. **Reads and writes scale differently.** You scale reads by *copying*. Place the data on several machines and point each reader at one of them. Now thousands of readers can work at once. Copying does nothing for writes, because every copy must eventually apply every write. You scale writes by *dividing*. Cut the data into pieces and make each machine own one piece. Remember the rule "copy for reads, split for writes". It tells you when the first three patterns fit, and it answers a big share of most database design questions.
2. **An append-only log sits underneath everything.** An append-only log is a file you only add to at the end. Nothing already in it is ever edited. The second half of the module grows out of this one idea. The Write-Ahead Log is how a database recovers from a crash, and it also powers replication. Event Sourcing makes that log the app's own data model. CQRS then separates the read model and the write model that grow out of that log. Module 2 explained *why* services announce facts as events. This module explains *where those facts live*: in an ordered log from which every other view is built.

How the module is told. We stay with the same online store the whole way. It starts with one database box doing every job, which is how most real companies start. Reads overload it first (→ Primary-Replica). Then the number of writes and the size of the data become too much (→ Sharding). Changing the number of shards turns out to hurt (→ Consistent Hashing). Next we ask how a database survives a sudden power cut (→ Write-Ahead Log). Last, we rebuild the order system on top of the log (→ Event Sourcing, CQRS). The capstone replays the whole journey at 100 million users. **Only a little time before an**

interview? Focus on Primary-Replica, Sharding and Consistent Hashing. Nearly every "design X at scale" question relies on them.

Four terms used throughout

- **Read path vs. write path** — a query that *gets* data travels the read path. A request that *changes* data travels the write path. The two paths need different scaling tools and carry different risks. They may even end up on different databases.
- **Replication lag** — a copy stays current by receiving changes from the original. Sending them takes time. So a copy is behind the original by milliseconds, and sometimes by more. This is the database version of **eventual consistency** from the Event-Driven Architecture lesson: the copies match *in the end*, not right away. (Replication and consistency basics are also covered in the System Design Fundamentals primer.)
- **Shard key** — the one field that decides which machine owns a row. This choice decides how evenly the work is spread. A poor shard key causes the main failure in the capstone.
- **Durability** — the promise that once the database says "saved", the data survives a crash, a reboot or a power cut. The write-ahead log is what keeps that promise.

Before you start — three questions about a system you know (this module shows how to answer each one, and the capstone asks them again):

- If your database server died right now, how much data would you lose? How long would it take to recover? Has anyone ever practiced that recovery for real?
- How much of your database traffic is reads, and how much is writes? (Measure it instead of guessing. The answer decides which patterns fit.)
- If you had twice as many users tomorrow, what would run out first: write capacity, read capacity, or disk space?

Primary-Replica

Original lesson: Primary-Replica on DesignGurus ↗

What it is. A single database machine, called the **primary**, takes every write. It keeps sending a stream of its changes to one or more other machines, called **replicas**. Each replica holds a live copy of the data and answers read queries. If the primary dies, a replica can be *promoted* to become the new primary.

Analogy. Imagine a teacher (the primary) who is the only person allowed to write on the main whiteboard. Several assistants (the replicas) copy each change onto their own boards as it happens. Students read from whichever board is nearest. If the teacher gets sick, the assistant whose board is most up to date takes over.

The problem

For years, the store has run on a single PostgreSQL (a popular open-source relational database) machine. One Tuesday, a look at the traffic shows **8,000 reads per second** (product pages, search, order history) and **300 writes per second** (orders, cart changes). The machine can only handle **5,000 queries per second** in total. Product pages need **4 seconds** to load. Checkout gets slow as well, because reads and writes compete for the same CPU, memory and disk. Look at the split: **96%** of the traffic is reads. That split *is* the problem.

A second problem hides behind the first. This overloaded machine is also the **only copy** of the company's data. It has one disk and one power supply. That makes it a **single point of failure**: one part that stops everything when it breaks. Last night's backup can't restore today's orders. If the machine failed at 4 PM, the whole day's orders would be gone. A single pattern solves both problems.

Why the obvious fixes fail

- **"Buy a bigger machine."** This might hold for about a year. But hardware only gets so big, prices rise sharply at the high end, and the upgrade itself means downtime. Worst of all, you still have just one copy. The single point of failure is still there.
- **"Put a cache in front."** A cache does help (Module 4). But a cache is a *performance layer*. It is not a second trusted copy of the data. Cache misses still reach the database, cached data gets out of date, and if the database dies, cached product pages can't rebuild the orders table.
- **"We already copy the database nightly."** A backup and a replica are different things. A backup may be hours old. It serves no traffic. Restoring it takes more hours. It stops you losing everything, but it doesn't protect today's data and doesn't lower today's load.

What you really need: extra copies that stay *current*, *handle traffic*, and are *ready to take over*.

How it works

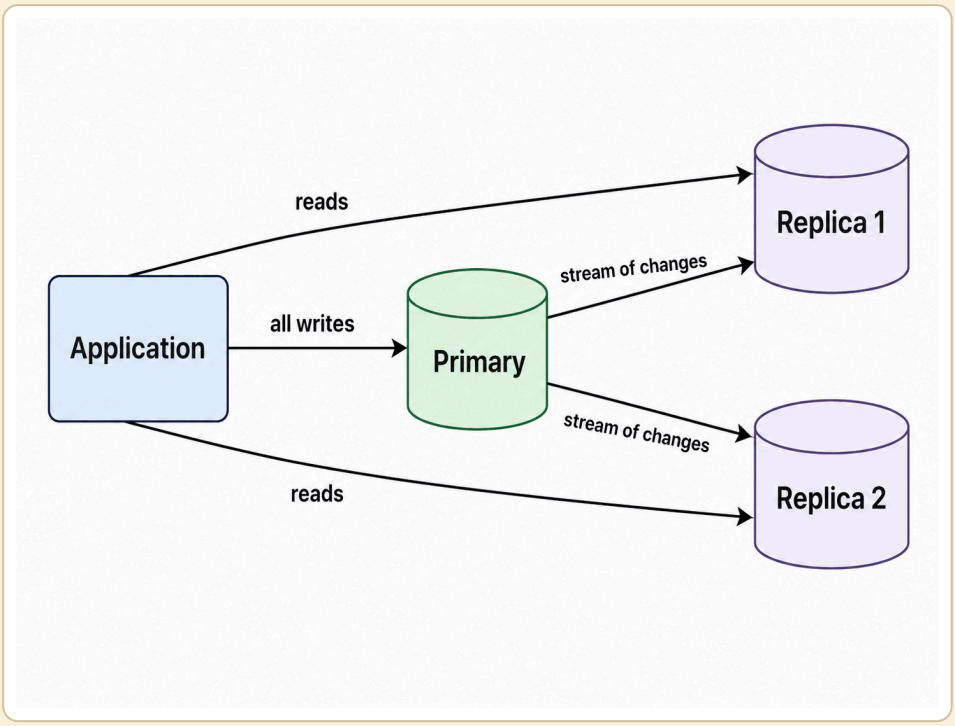


Figure — Primary-Replica (course diagram)

The figure above shows the application sending **all writes** to the green primary. It sends **reads** to Replica 1 and Replica 2. The primary pushes a **stream of changes** out to each replica.

1. **Every write goes to the primary.** If two machines both took writes, they could change the same row in two different ways. Then the copies would disagree. Having one writer and many readers rules out that conflict *by design*. (The next lesson looks at the price of this choice.)
2. **The primary writes each change into an ordered log and sends that log to the replicas.** It is the same log the database uses to survive crashes. (The Write-Ahead Log lesson opens it up.) For now, remember: *replication means shipping the log*.
3. **Every replica replays the changes in exactly the same order.** So its copy ends up the same as the primary's.
4. **Reads are shared out across the replicas.** This is where the extra capacity comes from.
5. **Replication lag is expected and handled.** Sending and replaying changes usually takes milliseconds. Under heavy load it can take seconds. So each replica is a little behind. The most visible symptom is the **read-your-writes problem**. A user saves a change, which goes to the primary. They reload the page, and that read goes to a replica that hasn't caught up. Their edit looks like it disappeared.
6. **Failover when the primary dies.** *Failover* means switching over to a backup. The system notices the failure when **heartbeats** stop arriving. (Heartbeats are small, regular "I'm alive" messages.) It chooses the replica that is most up to date and **promotes** it. Promoting means setting it up to accept writes and sending the application's traffic to it.

7. **Fencing stops split brain.** **Split brain** happens when the old primary comes back, doesn't know it was replaced, and keeps taking writes. Now two machines each believe they are the primary, and the data splits in two directions. So a safe failover **fences** the old primary first. Fencing forcibly blocks it from taking writes before the new primary is promoted.

Walkthrough with numbers

- **Add two replicas and split the reads between them.** Each replica handles roughly **4,000 reads/s**, which fits its capacity. The primary handles the 300 writes/s, plus the small number of reads that must be completely up to date. Its load drops from **8,300 queries/s to a few hundred**. Product pages go from **4 s to 80 ms**.
- **Measure the lag.** Most of the time, replicas are **10–50 ms** behind, and nobody notices. During a bulk import, lag jumps to **8 seconds** for a while, and read-your-writes complaints start. The lesson: lag is a *number you monitor*. It is not a fixed value you can assume.
- **The 2 AM failover drill.** The primary's hardware fails. Spotting the failure takes about **15 s** of missed heartbeats. Promotion takes another **30 s**. During those **45 seconds** writes fail, so checkout is down. Reads still work, because the replicas keep answering.
- **The hidden cost of asynchronous replication.** In async replication the primary says "saved" *before* the change reaches any replica. So a failover can lose the last **~200 ms** of writes that were already confirmed. The comparison below helps you decide when that is acceptable.
- **Bonus.** Backups and heavy analytics can now run at full speed on a replica, without touching production traffic.

Async vs. sync replication (the comparison)

Every replicated setup must answer one key question: *at what moment does the primary tell the client "saved"?*

	Asynchronous	Synchronous
Primary says "saved"...	Right away, and sends the change afterward	Only once a replica confirms it has the change
Write latency (time until "saved")	Fast (only local work)	Slower (one network round trip at minimum)
If the primary dies	The most recent confirmed writes might be lost	No confirmed write is lost
If a replica dies or slows	No one notices	Writes stall or fail, because the replica now sits on the critical path (the steps a write must wait for)
Fits	Most workloads: feeds, catalogs, profiles	Moving money, committing inventory

- **Async is the usual default nearly everywhere.** Its failure (losing ~200 ms of writes during a rare failover) usually costs less than sync's price. With sync, every single write pays for a network round trip, and writes stall whenever a replica struggles.
- **Semi-synchronous sits in between.** The primary waits for exactly *one* replica to confirm. You get failover with zero loss for a modest latency cost. This is what serious systems that handle money usually run.
- **A strong answer picks a mode per table or per operation.** It doesn't force one mode on everything.

Trade-offs

You gain:

- Read capacity that grows as you add machines. This is the standard answer to a workload that is 96% reads.
- Survival: a promoted replica brings service back in under a minute, instead of a half-day restore from a backup.

- A safe spot for backups, analytics and experiments, away from production.

You pay:

- **Lag and the odd behavior it causes.** Every read from a replica is a bit out of date. That is usually harmless. But it shows up as read-your-writes problems. So the application must send reads that *need fresh data* to the primary, and all other reads to replicas.
- **Writes don't scale at all.** Each replica has to apply each write to stay current. So adding replicas adds *zero* write capacity. It even adds work, because the primary now has one more follower to feed. When writes outgrow one machine, the data has to be *divided* (the next lesson).
- **Untested failover is dangerous.** Split brain, data lost to async replication, and promotion scripts that were never run can each break recovery. A failover plan nobody has practiced doesn't really exist.
- **More to run:** extra machines, lag dashboards, heartbeats, **runbooks** (written step-by-step guides for handling incidents).

When not to use it

- **Workloads heavy on writes** (logging, **telemetry** (a constant flow of measurements sent by devices or servers), very frequent updates). When writes are the bottleneck, replicas multiply the work and add no capacity. That problem belongs to sharding.
- **Small scale with good backups.** One properly sized machine, backups you've tested, and an hour of acceptable recovery time is a fully professional setup for a small product. Don't take on distributed-systems problems until scale makes you.
- **When every read must be completely fresh** (uncommon, but it does happen in trading and some inventory cases). Sending every read to the primary cancels the benefit of the pattern. Either pay for synchronous replication, or challenge the requirement.
- **As a promise that writes are always available.** Failover takes seconds to minutes. If writes must *never* pause, this pattern alone can't give you that.
- **Classic misuse:** reacting to a *write* overload by adding three more read replicas. That only gives the primary more work. Always ask "reads or writes?" first.

Real-world examples

- **PostgreSQL streaming replication** and **MySQL replication** — the textbook form of the pattern. For most companies, the first scaling step looks just like the walkthrough.
- **MongoDB replica sets** — MongoDB is a popular document database (it stores JSON-like records instead of table rows). Its replica sets come with automatic failover. The replicas run an *election* and promote a new primary by themselves, usually within seconds.
- **AWS RDS and Aurora** — offer the pattern as checkboxes: "add read replica" and "enable Multi-AZ" (AZ = availability zone, a separate datacenter within a cloud region; this option keeps a standby, replicated synchronously, kept ready for failover). **Aurora** goes a step further and allows up to **15 replicas** that share one storage layer.
- **For AI engineers:** **vector databases** (databases that store **embeddings**, lists of numbers representing meaning, and find the closest ones) use a single writer for embeddings and replicas for **similarity search** (finding the most similar items). The online reads of a **feature store** (a database of the input values ML models use) are exactly the read-heavy traffic replicas are good at. Training and evaluation jobs should read from a replica, so a data scientist's 2-hour scan never slows production. Read-your-writes hits ML as well: right after a fresh feature write, a prediction served from a lagging replica may use yesterday's feature value.

In the interview

Common prompts: "Your database is overloaded — what do you do?" (the interviewer wants you to ask first, *"overloaded by reads or by writes?"*), "What happens when the database dies?", and "A user can't see their new comment right away."

> **30-second answer:** A single primary receives every write and streams its change log to replicas, which serve the reads. This scales reads sideways by adding machines, which matters because most workloads are read-heavy. It also gives failover: if the primary dies, the replica that is most caught up gets promoted, and fencing keeps the old primary from coming back and causing split brain. I manage two costs. The first is replication lag: reads that must be fresh, such as a user reading their own write, go to the primary. The second is that it adds zero write capacity, because every replica applies every write. When writes are the bottleneck, the answer is sharding, not more replicas.

Likely follow-ups (with answers):

- *"A user edits their profile and the edit isn't there after reload. Fix it."* — This is read-your-writes. The write went to the primary; the read went to a replica that was behind. Fixes, from cheapest up: (1) for a short window after a user writes (say ~10 s), send that user's reads to the primary; (2) record the log position of their latest write, and read only from replicas that have reached it; (3) show the change in the UI right away (an optimistic update) while storage catches up.
- *"How automatic should failover be?"* — Detection and promotion should be automatic, because at 2 AM seconds count. But fencing must be in place first. Promoting without fencing risks split brain, and two primaries make for a much worse day than one dead one. Whatever the mechanism, the team must practice it on purpose.

Mistake that fails candidates: suggesting read replicas for a write bottleneck. Or describing failover as instant and free, with no mention of lag, the window for data loss, or split brain. Either one shows you know the name but have never run the pattern.

Self-check

- **Q1. What scales when you add replicas, what doesn't, and why?** → Read capacity scales, since each replica serves more reads. Write capacity doesn't, since every replica must apply every write: a write costs work once *per copy*, and the primary gets one more follower to send to. You also gain failover and a place for backups/analytics.
- **Q2. A platform stores social-feed posts and account balance transfers. Which replication mode for each?** → Feed posts: asynchronous. Users feel write latency on every post, while data is lost only during rare failovers. Balance transfers: synchronous, or at least semi-synchronous. A bank can't lose the last ~200 ms of transfers it already confirmed. Pick per table/operation.
- **Q3. After adding two replicas: "I updated my shipping address, refreshed, the old one was back; a minute later it was fine." Diagnose and give two fixes with costs.** → This is read-your-writes, caused by replication lag. The refresh read from a replica that hadn't yet replayed the update, and it "fixed itself" when the replica caught up. Fix 1: send the user's reads to the primary for ~10 s after a write. It costs **session state** (remembering per-user facts, such as "wrote 5 s ago", between requests), and those reads lose the replica benefit. Fix 2: note the log position of the user's write and read only from replicas past it. The data layer needs extra plumbing for this, but in return those reads keep using the replicas. (For purely cosmetic cases: optimistic UI.) The *wrong* fix is sending every read to the primary. That throws away the reason replicas exist.

Related patterns

- **Sharding** — the next lesson. It covers what to do when *writes* outgrow one machine.
- **Write-Ahead Log** — the stream the primary sends to replicas *is* this log.
- **Load Balancing** — how read traffic is actually spread over the replicas.
- **Cache-Aside** — the other tool for scaling reads. A cache catches the hottest reads before they ever reach a replica.

TL;DR

- **Problem:** one database machine is a read bottleneck (reads are usually ~95% of traffic) and also the company's only copy, which makes it a single point of failure.
- **Mechanism:** every write goes to one primary, which streams its change log to replicas. Replicas serve reads and wait ready for promotion. Fencing prevents split brain.
- **Costs:** replication lag and read-your-writes surprises, zero extra write capacity, and a failover procedure that only counts if it's practiced.
- **Skip it when:** writes are the bottleneck (shard instead), one machine plus tested backups fits your scale, or every read everywhere must be completely fresh.

Sharding

Original lesson: Sharding on DesignGurus ↗

What it is. Sharding divides *the rows of one table* across many machines. Each machine is called a **shard**. A rule based on one field, the **shard key**, decides where each row goes. Every shard completely owns its piece, including its own writes, reads and **indexes** (sorted lookup structures that make finding rows fast). No single machine holds all the data.

Analogy. Think of a phone book that is too big for one volume. You split it into A–F, G–M, N–S and T–Z. Everyone knows the rule, so any name takes one step to find. And four printers can update the four volumes at the same time.

The problem

The replicas did their job. Reads are fast, and the site got through its first failover. But eighteen months on, the *primary* is struggling in a way replicas can't fix. Writes have climbed from **300 to 2,500 per second** (orders, carts, inventory, reviews). Recall the trap: each replica must apply each write. So this volume overloads the primary *and every replica*. Their lag grows from milliseconds to **minutes**. Adding a fourth replica would make things worse, since that is one more follower to feed.

The data has also grown too large. The orders table is **4 TB**. Its **indexes** (sorted lookup structures that make queries fast) don't fit in memory anymore, so every lookup has to read the disk. Backups run all night. A **schema migration** (a change to the table's structure) becomes a weekend project with a rollback plan. And you are already renting the largest machine the cloud provider offers. The two problems, write volume and data volume, look the same: *too much for one machine, however many copies you make*.

Why the obvious fixes fail

- **"More replicas."** Copies help reads. This is a write problem, and copies make write problems worse.
- **"An even bigger machine."** You already have the biggest one. Even if a bigger one existed, prices at the top are steep, upgrading means downtime, and within a year you'd hit the same wall with less money in the bank.
- **"Archive old data."** Moving orders older than two years into **cold storage** (cheap, slow storage for data that is rarely read) is worth doing. It might shrink 4 TB down to 1.5 TB. But it buys quarters, not years. The *write rate* doesn't fall at all, and analytics will soon ask where their history went.
- **"Split by table"** (a **functional or vertical split**: orders on one machine, users on a second, inventory on a third). This is often a sensible first move. But the orders table alone holds most of the writes and bytes. Sooner or later, one table outgrows one machine.

Each of these *moves* the load around. What you need is to *divide* it.

How it works

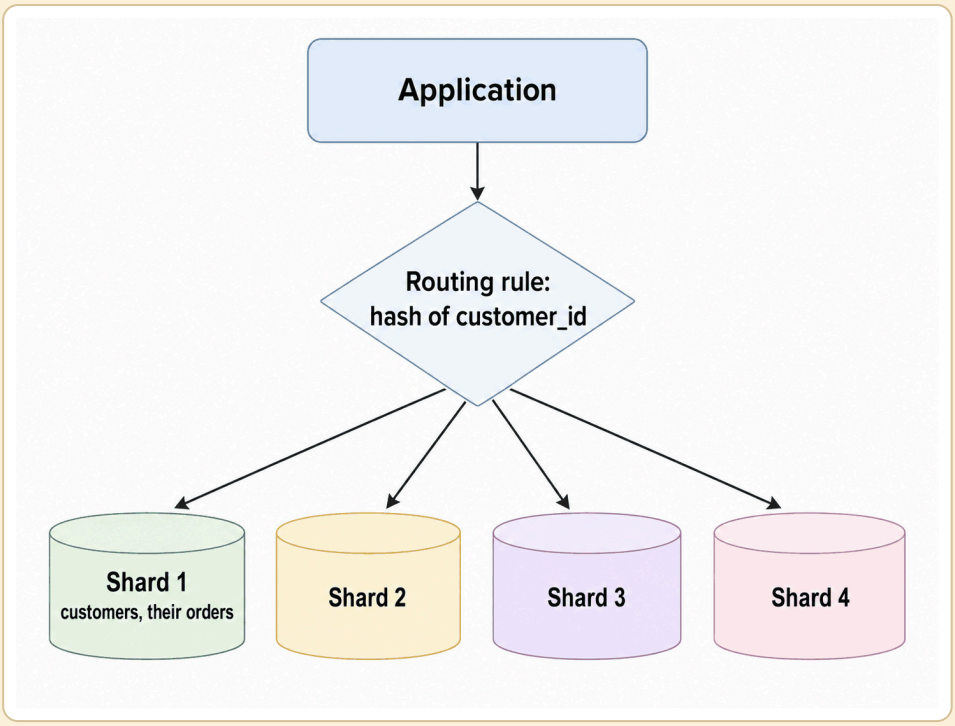


Figure — Sharding (course diagram)

In the figure above, each request from the application goes through a **routing rule** (a hash of `customer_id`). The rule sends it to exactly one of four shards. Shard 1 holds a group of customers *together with their orders*.

Whether sharding works comes down to three decisions:

1. **Pick the shard key. This is the most important decision in the whole module.** A good key has three qualities:
   - It **shows up in almost every query**, so each query can go to a single shard. (If you shard by product, every "my orders" query has to search every shard.)
   - It **spreads the load evenly**, so no single shard becomes the busy one.
   - It **keeps related rows side by side.** With `customer_id`, a customer's orders, addresses and cart all sit on one shard, so checkout never has to cross machines.
2. **Pick the routing rule: how a key turns into a shard number.**
   - **Hash sharding:** pass the key through a **hash function** (something that turns any value into an evenly spread number), then take the remainder: `hash(customer_id) % 4`. This spreads the load well and is the default choice.
   - **Range sharding:** give each shard a range (A–F, G–M, or a date range). Neighboring values stay together, which is great for range queries. It's also risky: if you shard by date, today's shard gets 100% of the writes while the others do nothing.
3. **Decide who does the routing.** It can be the application applying the rule, a **proxy** (a middleman server that forwards each query to the right shard), or the database itself (MongoDB, Vitess, Citus). All three work at scale, as long as every caller applies the exact same rule.

The price, stated honestly. Any question that crosses shards gets harder. "This customer's orders" touches a single shard and is fast. "Top 10 products sold today, across everyone" has to ask every shard and combine the results. That is called **scatter-gather**. **Joins** (queries that combine rows from two tables by matching a shared field, such as `customer_id`) across shards stop being something the database does for you. They become your application's job. In the same way, a **transaction** that touches two shards (say, two customers on different shards swapping money) can't be a single **atomic** database transaction (all-or-nothing) anymore. Patterns such as the **saga** (later in the course) exist for exactly this case. Sharding gives you write capacity, and you pay for it with query flexibility.

Walkthrough with numbers

Four shards, routed by `hash(customer_id)`:

- **Writes:** 2,500/s ÷ 4 comes to roughly **625/s per shard**. That's comfortable and leaves room to grow. Each shard is a primary-replica cluster of its own, so the two patterns *stack*. Sharding doesn't replace replication; it multiplies it.
- **Data:** 4 TB ÷ 4 = **1 TB per shard**. Indexes fit in memory once more, backups finish in hours, and migrations become manageable.
- **Queries, from easy to hard:** "Show customer 8412's orders" → hash the ID and ask one shard. It's as quick as the old single machine. "Top sellers today" → scatter-gather over all four shards and merge in the app. That's slower and clumsy. This is why heavy analytics usually moves into a separate system fed by events (see the change data capture lesson).
- **The problem still waiting:** two years later you need 8 shards. `hash(id) % 4` turns into `hash(id) % 8`. The remainder (what `%`, or **modulo**, keeps) changes for nearly every customer. So almost all the data has to physically move. The next lesson solves this.

Hash vs. range sharding (the comparison)

	Hash sharding	Range sharding
How it routes	`hash(key) % N`	By range: A–F on one shard, G–M on another; Jan on one, Feb on another
Load spread	Excellent, and happens automatically	Only as good as the ranges you pick; time ranges send every new write to one shard
Range queries (dates, prefixes)	Bad: one range ends up spread over every shard	Excellent: neighboring values sit together
Hot key risk	A single huge key still overloads its shard	The same, and hot ranges too
Resharding	Painful with plain modulo (the next lesson fixes this)	You can split a busy range where it sits, which is gentler

Choose **hash** by default. For transactional work, an even spread matters more than range queries. Choose **range** when data really is accessed by range (**time series**, i.e. measurements stored by timestamp; leaderboards; browsing alphabetically). Then manage the ranges with care, so the current month doesn't turn into one overloaded shard.

Trade-offs

You gain:

- Write capacity that finally grows with the number of machines. Replication could never give you this.
- Data size that grows the same way. No table ever has to fit on one machine again.
- A smaller **blast radius** (the share of users one failure affects). If shard 3 dies, only a quarter of customers are hit, and that shard's own replica failover fixes even that.

You pay:

- **Query flexibility:** queries on one shard stay fast. Queries across shards turn into scatter-gather. Joins and transactions across shards become your problem.
- **Hot shard risk:** a **hot shard** gets much more load than the rest. Hashing spreads *keys* evenly, not *load*. A single huge key lands entirely on one shard. It might be a celebrity, or a giant **tenant** (one customer company on a shared product). Adding more shards does nothing for it.
- **Resharding pain:** changing the number of shards means moving a lot of data while the system is live. Plain modulo turns this into a disaster.

- **More to operate:** 4 shards × (primary + 2 replicas) = **12 databases**. That means twelve failovers, twelve backup schedules, and a routing layer every engineer has to understand.

When not to use it

Sharding is a **last resort**. Reach for it only once the cheaper options are used up:

- **Vertical scaling still has room.** A bigger machine brings no new ways to fail.
- **The real problem is reads.** Replication handles that with about a tenth of the complexity.
- **The data can get smaller.** Archiving or expiring rows can delay sharding by years.
- **A functional split is enough.** If no single table outgrows a machine, don't split rows.
- **Your queries don't match the split.** A workload full of joins and transactions across many entities will slow down. Think about changing the data model first.

Misuses: *sharding for prestige*. That means four shards at a scale one Postgres handles easily, paying for the complexity for years before the scale shows up, if it ever does. And the classic key mistake: **auto-incrementing ID with range routing** (an auto-incrementing ID is one the database assigns as 1, 2, 3, …, so new rows always get the highest number). Every new row goes to the newest shard, so one machine does all the writing.

Real-world examples

- **Instagram** sharded Postgres by user ID and stored the shard information inside the IDs themselves. Thanks to a well-chosen key, billions of photos ran on ordinary databases.
- **Slack** shards by workspace. That is a natural tenant key, because nearly every query is "inside this workspace". Its hot shards from giant workspaces are the self-check scenario at huge scale.
- **Vitess** (created at YouTube, now **open source**, meaning its code is public and free to use) and **Citus** add sharding to MySQL and Postgres. **MongoDB** has it built in: you pick a shard key, and routing and rebalancing are handled for you.
- **DynamoDB and Cassandra** (Amazon's managed key-value database and a popular open-source wide-column database) are built around sharding from the start. The *partition key* in your table design *is* the shard key. That's why their docs teach how to choose it first.
- **For AI engineers:** feature stores shard by entity ID (user, item). Training datasets shard by ranges of examples so they can load in parallel. When a vector index grows past one machine, you shard it and scatter-gather every similarity query (covered in the later vector-database sharding lesson).

In the interview

Any "design Twitter / Instagram / Uber at scale" question arrives here within minutes. The interviewer is waiting for your **shard key choice, with reasons**. The follow-ups are easy to predict: the celebrity, resharding, and the cross-shard transaction.

> **30-second answer:** Once writes or data size grow past one machine, I shard. I split the table's rows across machines using a shard key that appears in almost every query, spreads load evenly, and keeps related rows together. For commerce that's customer ID, so checkout never crosses machines. I route by hash by default, and each shard runs as its own primary-replica cluster. The prices I accept: queries across shards become scatter-gather, transactions across shards need patterns at the application level, a single hot key can still overload its shard, and resharding needs a smarter rule than modulo, namely consistent hashing.

Likely follow-ups (with answers):

- *"You sharded by user ID. A celebrity with 40 million followers goes live."* — Their shard is overloaded while the others sit idle, because hashing spreads keys, not load. Fixes, in rising order: (1) cache reads of the hot entity, so the shard hardly sees them; (2) send the few extreme keys to shards of their own; (3) split that entity's data even further (for example,

shard their followers list again by follower ID). Naming the problem before the interviewer does counts for more than any one fix.

- *"Two users on different shards send each other money."* — First, try to avoid it: a good key keeps most transactions on a single shard. When you can't avoid it, it can't be one database transaction. It turns into a coordinated series of steps with failure handling, which is the saga pattern. Seeing the key choice as the first line of defense is what marks a senior candidate.

Mistake that fails candidates: saying "we'll shard from day one" without mentioning cost. Picking a shard key without giving reasons. Or having no answer for what happens when the number of shards has to change.

Self-check

- **Q1. What does sharding scale that replication can't, and what are the three properties of a good shard key?** → Write capacity and total data size. Replication copies *everything* onto every machine, and every copy applies every write. Sharding hands each machine its own slice. A good key shows up in nearly every query, spreads load evenly, and keeps related rows together.
- **Q2. Multi-tenant SaaS (software-as-a-service sold by subscription; multi-tenant means many customer companies share one system; companies sign up, employees use it): shard by `company_id`, `user_id`, or the row's random ID?** → `company_id`. Nearly every B2B (business-to-business) query is "inside this company", so queries stay on one shard, and a company's data stays together for exports and deletion. Known risk: tenant sizes can be very unequal, so plan dedicated shards for giant tenants. `user_id` breaks queries across the whole company ("all documents in this workspace" becomes scatter-gather). A random row ID spreads data perfectly but ruins every real query, because no lookup knows where anything is stored.
- **Q3. Hashed `user_id` across 8 shards; an influencer's live shopping event (40M followers) overloads shard 5 while the others sit at 15%. Diagnose and give three fixes.** → It's a hot key. Hashing spreads keys evenly, but this single key carries the load of 40M users. (1) Cache the storefront reads. They're almost the same for every viewer, so this may be enough by itself. (2) Send celebrity accounts to dedicated shards. (3) Split the influencer's own data by a second key, such as follower ID. The bigger lesson: monitor load per shard, because the business keeps creating unequal keys.

Related patterns

- **Primary-Replica** — every shard is a replicated cluster of its own. Sharding multiplies replication instead of replacing it.
- **Consistent Hashing** — the next lesson. It shows how to change the number of shards without moving almost everything.
- **Saga** — what a transaction across shards turns into once a single atomic transaction can't be used.
- **Cache-Aside** — the first defense against hot keys.

TL;DR

- **Problem:** writes and data size grow past one machine, and replication can't help (every copy applies every write, and no copy makes the table smaller).
- **Mechanism:** split rows across shards using a well-chosen key (present in nearly every query, evenly spread, keeps related rows together). Route by hash by default. Run each shard as a primary-replica cluster.
- **Costs:** scatter-gather queries, cross-shard transactions handled by the application, hot keys, data movement when resharding, and operations work multiplied by the number of shards.
- **Skip it when:** you can still go further with a larger server, more replicas, moving old data out, or splitting tables by feature, or the workload is joins and transactions across everything. Sharding is a last resort.

Consistent Hashing

Original lesson: Consistent Hashing on DesignGurus ↗

What it is. It's a routing rule for spreading keys over servers. With it, adding or removing one server moves only a small slice of the data (about 1/N) instead of nearly all of it. Both servers and keys are placed on a circle by hashing them. Each key belongs to the first server you reach by walking clockwise.

> **Analogy.** Imagine houses (keys) along a circular road. Each house is covered by the next fire station (server) going clockwise. If a new station is built, only the houses between it and the station before it switch over. Every other house keeps its old station.

The problem

The sharded setup has four shards routed by `hash(customer_id) % 4`, and each one handles 625 writes/s. Two years on, growth calls for a **fifth shard**, so `% 4` becomes `% 5`. Before rolling it out, someone runs the numbers. A key stays on its current shard only if `hash % 4` equals `hash % 5`. That's true for only **20%** of random hashes. So one extra shard leaves **80% of the data on the wrong machine**. Around **3.2 of 4 TB** would have to move.

That kind of migration means copying terabytes over a weekend while new orders keep coming in. You'd write to both the old and new locations, keep a tested rollback plan, and live through a slow week. Then it would happen again at the sixth shard and the seventh. Sharding isn't the problem. The `% N` rule is. The shard count N is built into every key's address, so changing N breaks nearly every address. A better rule would move only the data that the new machine is supposed to own.

Why the obvious fixes fail

- **"Accept it: migrate over a weekend."** 3.2 TB has to move while 2,500 writes/s keep changing the source. That needs double-writes and consistency checks. And every later resize repeats it. Teams end up putting it off and running too few, overloaded shards for years.
- **"Only ever double: 4 → 8."** With doubling, each key either stays put or moves to one particular new shard. Still, **50%** of the data moves. A need for 25% more capacity turns into buying 100% more capacity, plus a **2 TB** migration. The move is a bit smaller, but far more expensive.
- **"Skip the formula; keep a lookup table."** A **directory** that notes "customer 8412 lives on shard 3" is a valid design (compared below). But a simple table with one row per key has a hundred million rows and must answer every query. That makes it the part of the system whose availability matters most. Grouping keys into ranges lightens this load.

How it works

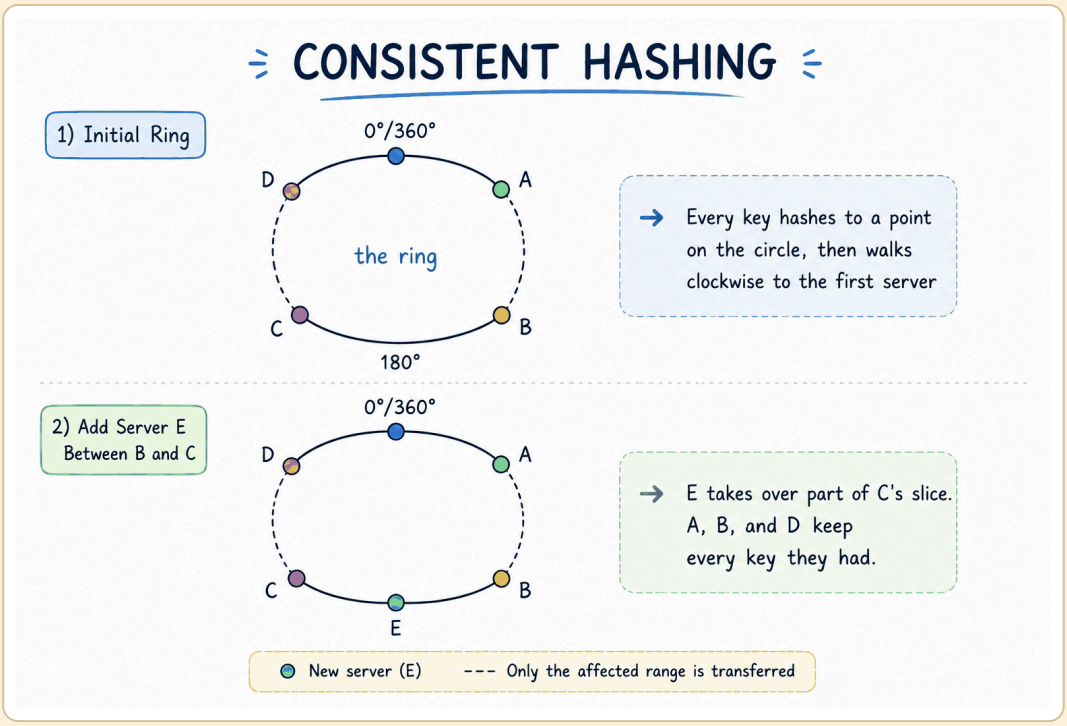


Figure — Consistent Hashing (course diagram)

In the figure above, panel 1 shows servers A–D placed around the ring. Each key hashes to a point, then walks clockwise to the first server. Panel 2 adds server E between B and C. E takes over part of C's slice. A, B and D keep every key they had before. Only the dashed range that is affected gets transferred.

1. **Think of the hash space as a circle.** Hash values go from 0 up to a large maximum, then wrap around to 0.
2. **Hash every server name and every key onto that circle.**
3. **Each key belongs to the first server clockwise from it.** Keeping the server positions in a sorted list lets you find that server in microseconds.
4. **Adding a server moves just one slice.** When E lands between B and C, only the keys in E's new slice get a different "next server". All of them used to belong to C. So E copies that slice from C, and nothing else changes. With N servers, adding one moves about **1/(N+1)** of the data: **20%** from 4→5, and about **2%** from 50→51.
5. **Removing a server works the other way round.** The next server clockwise takes over its slice, along with all of its load. This reveals two weak spots. With only a few servers, random placement makes slices very uneven. And a failure dumps everything onto a single neighbor.
6. **Virtual nodes solve both.** Put each server at **100–200 points** on the ring instead of just one. Each point is called a **virtual node (vnode)**. Now every server owns many small, scattered slices, and the random differences even out. When a server dies, its ~150 slices go to ~150 different neighbors, so the load is shared. You can also give a stronger server more vnodes, which weights placement by capacity.

What it does *not* fix. Consistent hashing handles **membership churn** (servers joining and leaving). It does *not* handle **hot keys** (a single key with unusually heavy load). The celebrity from the sharding lesson still hashes to one point, and so to one server. Hot keys still need caching and special placement.

Walkthrough with numbers

- **The resize, done again.** 4 TB across 4 shards, and a fifth is added. Modulo moves 3.2 TB and touches every shard. The ring moves about **0.8 TB (20%)**, and only from the new shard's neighbors. The copying happens in the background over several days. Each old owner keeps serving its slice until the handoff.

- **The failure, done again.** Without vnodes, if shard 3 dies, shard 4 takes all of its load and may fall over as well. That chain reaction is a **cascade**. With **150 vnodes** per server, every surviving shard picks up a small piece and handles a manageable ~25% increase.
- **Where it started: caches.** Take a 50-node cache cluster (see Cache-Aside). With modulo routing, adding or removing one node changes N and remaps roughly **98%** of keys. So ~98% of lookups suddenly miss and fall through to the database. That's a **miss storm**. A ring remaps only **~2%**. This is why cache clients have used consistent hashing since the late 1990s: cache clusters change size all the time.

The ring vs. the directory (the comparison)

The simple lookup table from earlier, when built properly, is the ring's real rival. It becomes a **directory service** that maps *ranges* of data (not single keys) to servers. Clients ask it and cache the answers.

	Consistent hashing (ring)	Directory (range-to-server map)
Data moved on resize	About 1/N, with no manual work	Whatever you decide; completely flexible
Placement control	None: the hash decides	Complete: you pin, split and move ranges yourself
Hot range/tenant	Can't be treated as a special case	Split the hot range, move it, finished
Extra infrastructure	None: it's just math inside the client	A metadata service (a small service holding the "what lives where" map) that has to stay highly available
Range queries	Spread around	Natural: ranges stay together
Typical home	Caches, clusters that change size, peer-to-peer (machines that talk directly with no central server)	Databases that want operational control

The modern hybrid combines the best of each. Hash keys into many small, fixed **buckets**. Then keep a small directory that maps buckets to servers. **Redis Cluster** does exactly this with **16,384 "hash slots"**. The hash spreads keys evenly, which is the ring's strength. Moving a slot is a one-row change in the directory, which is the directory's strength. If an interviewer asks "what comes after the ring?", this hybrid is the answer.

Trade-offs

You gain:

- Changes in membership that move as little data as possible: about 1/N for each server added or removed.
- The freedom to grow by exactly one machine, instead of doubling just to be safe.
- Failures that spread out thinly (with vnodes) instead of cascading.
- No central directory: every client that has the server list works out the same answer.

You pay:

- **Virtual nodes are required.** Without them, slices are uneven and a failure overloads a single neighbor. Mature libraries turn them on by default. Home-made versions often leave them out.
- **Routing is a little more complex:** a sorted lookup instead of one modulo. It takes microseconds, but every client must implement it identically and correctly.
- **It's still hashing:** range queries ("all orders from March") stay spread across servers.
- **Moving less data still means moving data.** A cache can shrug off a miss. Storage handoffs, though, need a careful order: stream the slice → keep serving it from the old owner → switch

ownership → delete the old copy.

When not to use it

- **The cluster size really never changes.** A fixed 4-shard setup that will still have 4 shards in five years can keep the simplicity of modulo. (But be honest about "never".)
- **Range queries dominate.** "Scan March's orders" calls for range sharding with a directory. No kind of hashing suits it.
- **You want to place data by hand.** "Pin this giant tenant to a shard of its own" is clumsy on a ring and easy in a directory.
- **A tiny, fixed system.** Two cache nodes behind one app don't need ring math.
- **Common misuse:** using the ring for database shards but plain modulo for the cache layer. Caches change membership *more* often, and every change can trigger a miss storm. So the cache may need this pattern first.

Real-world examples

- **Akamai** — consistent hashing was invented there (Karger et al., **1997**) to route web requests over cache fleets that kept changing.
- **Amazon's Dynamo paper (2007)** made "ring plus virtual nodes" the trademark technique of distributed databases. **Cassandra** and **Riak** took it over directly, and **DynamoDB** is descended from it.
- **Memcached** (a simple in-memory cache server) **clients** have included it for two decades, and the miss-storm math is the reason.
- **Redis Cluster** uses the hybrid: 16,384 hash slots, mapped to nodes through a directory.
- **For AI engineers:** in LLM **inference** (running a trained model to produce answers), a conversation's attention cache (the **KV cache**, the model's saved intermediate results for earlier tokens) stays on the GPU that last served it. Routing by conversation ID with consistent hashing preserves that match even as autoscaling changes the number of workers. Each scaling event moves only ~1/N of conversations, and the rest keep their warm caches. The same approach assigns partitions of the embedding space to workers in a sharded vector index.

In the interview

Typical ways in: "design a distributed cache", then "now add a node". Or the interviewer lets you shard with modulo and then asks what happens going from 4→5 shards. Be ready to work out the 80%.

> **30-second answer:** Modulo routing builds the number of servers into every key's address. So changing that number remaps nearly everything: going 4→5 shards moves 80% of the data. Consistent hashing places servers and keys on a ring. Each key belongs to the next server clockwise. Adding a server takes over one slice, about 1/N, and nothing else moves. Each server shows up as 100+ virtual nodes, so slices stay even and a dead server's load is spread over the cluster. It solves churn, not hot keys. When I need to place data by hand or run range queries, I'd use the hybrid: hash slots plus a small directory, as Redis Cluster does.

Likely follow-ups (with answers):

- *"Why virtual nodes?"* — Give both reasons. With one point per server, random placement makes slices very uneven. And a failed server dumps its entire load on one neighbor. 100+ points per server even out the slices and spread the load of a failure across the cluster.
- *"Does the ring fix your celebrity problem?"* — No, and saying that plainly earns points. A single key still hashes to a single point and overloads a single server. The ring fixes *churn*. *Skew* (keys with unequal load) still needs caching and special handling.

Mistake that fails candidates: using modulo routing in any design that autoscales. Or saying "consistent hashing" without being able to explain what moves when a node joins. The entire pattern is the answer to that one question.

Self-check

- **Q1. Going from N to N+1 servers, what fraction of keys move under modulo vs. consistent hashing, and why?** → Modulo: about N/(N+1) (80% for 4→5), because the divisor is part of every key's address. Ring: about 1/(N+1) (20% for 4→5), because only the new server's slice changes hands. For every other key, the "next server clockwise" stays the same.
- **Q2. A teammate places each server at exactly one point "to keep it simple". What two problems arrive, and when?** → Uneven load, with some slices several times larger than others. That shows up immediately. And when a server fails, its whole slice lands on one neighbor, which may then fail as well. That shows up at the first failure. Virtual nodes fix both.
- **Q3. The cache cluster autoscales up every evening; ten minutes later the database spikes and alerts, though nothing is failing. Connect them, fix it, and quantify for 20 nodes.** → The cache client routes with modulo. Every scale-up changes N and remaps most keys. The cluster then misses on almost everything, and all those misses reach the database together. It's a stampede the team caused itself, and it appears only as database load (no error alert). Fix: use consistent hashing with vnodes in the cache client. At 20 nodes, adding one remaps ~1/21 ≈ **5%** instead of ~**95%**, so the spike gets about **19×** smaller. For caches, resizing is the normal case.

Related patterns

- **Sharding** — the ring is sharding's routing layer done properly. All the advice about shard keys still holds.
- **Cache-Aside** — where the pattern was born. Cache clusters change membership all the time, and without the ring each change is a miss storm.
- **Load Balancing** — sticky routing (the same session always goes to the same server) that survives autoscaling is this pattern doing a different job.

TL;DR

- **Problem:** modulo routing builds the server count into every address. So adding one shard moves ~80% of the data, and every resize is a painful migration.
- **Mechanism:** hash servers and keys onto a ring. Each key goes to the next server clockwise. Each server appears as 100+ virtual nodes, so slices stay even and failures spread thinly. A resize moves only ~1/N.
- **Costs:** virtual nodes are a must, routing code has to match everywhere, range queries stay spread out, and storage handoffs still need careful orchestration.
- **Skip it when:** the cluster size truly never changes, range queries dominate (use a directory), or you need to place data by hand (use the slot-directory hybrid).

Write-Ahead Log

Original lesson: Write-Ahead Log on DesignGurus ↗

What it is. Before the database changes any of its real data files, it adds one record to the end of an append-only log file. That record describes the entire change. The database forces the record onto the physical disk, and only after that does it tell the client "saved". The actual tables and indexes get updated later, in the background. If the machine crashes, the database replays the log to rebuild them.

Analogy. A careful cook writes every order on a ticket and pins it up *before* cooking anything. If the kitchen loses power halfway through a dish, the tickets show exactly what was promised and

what is left to finish.

The problem

It's Tuesday, 3:07 AM. A power failure knocks out the **rack** (a tall cabinet of stacked servers) that holds shard 2. The machine reboots 90 seconds later, and one of two bad mornings follows:

- **Bad morning A — corruption.** The database won't start. It reports "data files corrupted". How? A single `UPDATE` isn't a single disk operation. Changing an order's status touches the table row *and* two indexes: three separate spots on disk. The power died after the first write but before the other two. Now an index points at a row that says something different, and the database is right to trust none of it.
- **Bad morning B — lost writes.** The database starts, but the last two seconds have vanished. **40 orders** that customers were told were "saved" never happened. The database confirmed them from memory before writing them to disk.

The physics seem impossible to beat. A disk can lose power partway through a write, even partway through a single **4 KB page** (this is a **torn write**). And your data is spread across thousands of pages that can't all change in one atomic instant. So how can any database promise "once I say saved, it's saved"? Every serious database gives the same answer: *don't do the work first. First write down what you're about to do, in one safe place. Then do it.*

Why the obvious fixes fail

- **"Write everything to disk immediately, no memory buffering."** This is very slow. A single `UPDATE` hits three scattered locations. **Random writes** (jumping from place to place on the disk) cost much more than one continuous stream; on spinning disks, about **100×** more. And it still doesn't stop corruption, because the power can fail between the first location and the second.
- **"Keep everything in memory and snapshot to disk every few minutes."** It's fast. But now a crash loses *minutes* instead of seconds. And copying a dataset while it is changing is a hard problem of its own. Still, snapshots end up being half of the real answer (see the comparison).
- **"Buy batteries."** A **UPS** (battery backup) makes the window smaller but doesn't close it. Kernels panic (the operating system's core crashes), hardware fails, someone unplugs the wrong cable. A durability plan that relies on nothing going wrong isn't a durability plan.

All three miss the same need: to record one *logical* change (the whole `UPDATE`) as a single cheap, atomic disk operation. Scattered data pages can't do that. A log can.

How it works

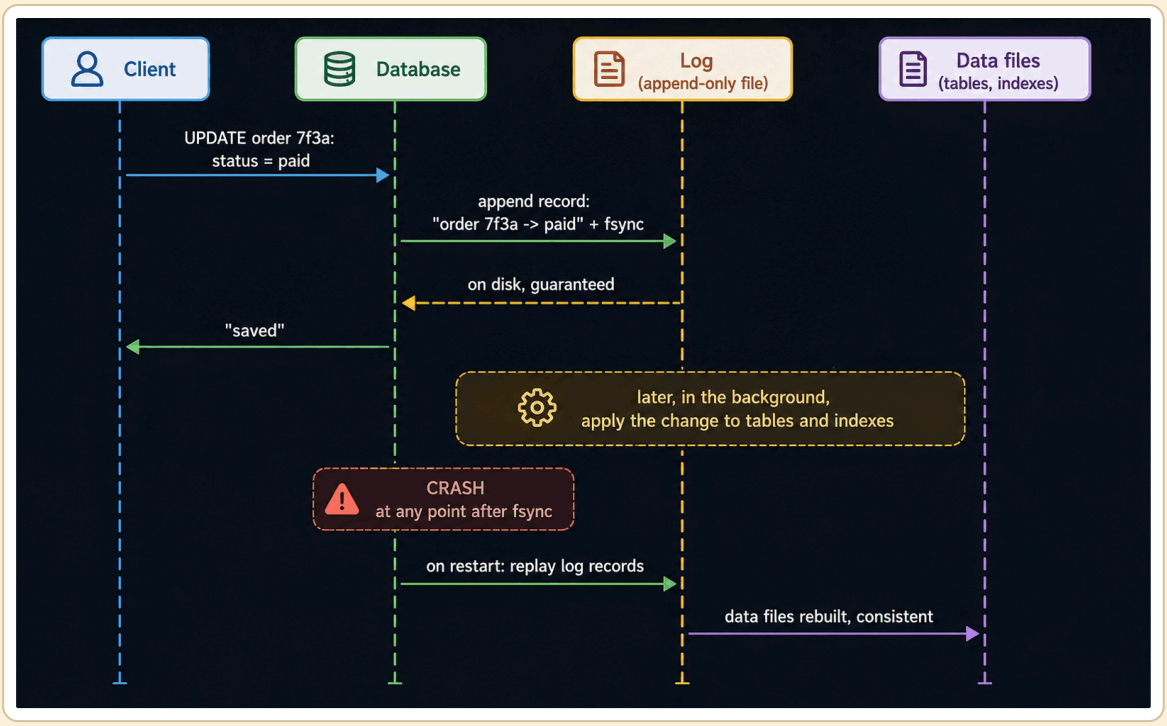


Figure — Write-Ahead Log (course diagram)

In the figure above, the client sends "UPDATE order 7f3a: status = paid". The database appends a record to the log and runs **fsync** on it. It gets back "on disk, guaranteed", and only then answers "saved". Applying the change to the tables and indexes happens later, in the background. If a crash happens at any moment after the fsync, the database replays the log when it restarts, and the data files are rebuilt in a consistent state.

1. **Append one record that describes the whole change** (the row plus both indexes) to the end of the log. Writing to one file in one continuous stream is the *easiest* job a disk can do, so this is cheap. The record includes a **checksum**: a small number calculated from the record's bytes that exposes any damage.
2. **Run `fsync` on the log.** Normally, operating systems hold writes in memory and flush them when it suits them. `fsync` is the system call that means "don't come back until this is physically on disk". This step is what makes the promise real.
3. **Tell the client it's done. The one rule the whole pattern depends on: never acknowledge before the log record has been fsynced.** Everything the client was told is safely written down. Anything not yet written down was never confirmed to anyone.
4. **Apply the change to the tables and indexes later**, in background batches. This keeps it off the **critical path** (the work a client has to wait for).
5. **After a crash, replay the log on restart.** The database goes through the log and re-applies each change. This rebuilds the data files to match every promise it made. Both bad mornings go away. A half-written, torn record at the very end (the **tail**) fails its checksum and gets thrown away. That's safe: a record that never finished its fsync was never confirmed to anyone.
6. **Checkpoints keep replay short.** Replaying a month of log would take hours. So every so often, the database flushes all pending changes into the data files and writes down a marker: "everything before this point is applied". That marker is a **checkpoint**. Recovery replays only what comes after it, and the older log can be archived or deleted.

The connection to replication. In the Primary-Replica lesson, the primary shipped a "stream of changes, replayed in order" to its replicas. That stream *is* the write-ahead log. A replica is simply a machine doing crash recovery nonstop, using another machine's log. One mechanism does two jobs: durability and replication. The log is turning into the most important structure in the database, and the next lesson pushes that idea as far as it goes.

Walkthrough with numbers

- **Timing the write path.** Appending an order's ~**100-byte** record and running fsync takes about **1 ms**. Updating the row and two indexes in place while the client waits would take several times longer, because of the scattered writes. And it still wouldn't be atomic. With a log, the fast path and the safe path are the same path.
- **Group commit multiplies throughput** (*throughput* = the number of writes finished per second). Say **50 writes** arrive in the same millisecond. The database appends all 50 records and runs fsync *once*. That's 50 durable commits for the cost of a single disk flush. It's how databases handle thousands of durable writes per second on ordinary disks.
- **The crash drill, done again.** The power fails at 3:07 AM. Since the last checkpoint (30 s ago), the log has collected about **75,000 records**. Replaying them takes a few seconds. Every confirmed order is there, including all 40 from the incident. The single torn record at the tail fails its checksum and is dropped. Its client was never told "saved", so it retried. The retry carried an **idempotency key** (a unique request ID), so it was applied only once.
- **The knob behind the drill.** A checkpoint every 30 s means fast recovery but frequent flushing. A checkpoint every hour means less overhead but minutes of replay. You make that choice (see the self-check).

The log vs. the snapshot (the comparison)

Redis makes this easy to see. It offers **RDB** (snapshots taken periodically) and **AOF** (an append-only log), and lets you pick.

	Snapshot (RDB-style)	**Write-ahead log (AOF-style)**
Cost per write	Nothing	One append to the log (plus whatever fsync policy you set)
Crash loses	Everything since the most recent snapshot (minutes)	At most the final moment not yet fsynced (~zero to seconds)
Recovery speed	Fast: load a single compact file	Slower: every change must be replayed, unless a recent snapshot gives you a starting point
Disk footprint	Small and compact	Keeps growing, unless history that's already absorbed can be thrown away

Each one is strong exactly where the other is weak. So real systems **use both together** through checkpointing: a snapshot, plus the log written after it. Production Redis turns on both. PostgreSQL does the same thing with a **base backup + WAL archive**. The practical answer to "snapshot or log?" is "both, working together".

Trade-offs

You gain:

- A durability promise that actually holds: "acknowledged" always means "on disk".
- Fast writes all the same, because one sequential append replaces many scattered in-place updates.
- Replication nearly for free: just ship the log you're already writing.
- **Point-in-time recovery (PITR):** keep the old logs plus regular snapshots, and you can restore the database to *any moment you choose*. Every serious operation runs this backup strategy.

You pay:

- **Everything gets written twice**: once into the log, and later into the data files. This **write amplification** is the pattern's honest cost. It's disk bandwidth spent on safety.
- **A minimum latency for every commit.** You can't confirm a write faster than the disk can confirm an fsync. Group commit gives up a little latency to gain a lot of throughput, but the minimum is

still there.

- **Log housekeeping.** Checkpoints and retention need tuning. Old dependencies can also stop the log from being deleted. A dead replica that is still "subscribed", or an archive job that is stuck, can slowly fill the disk without any immediate error.
- **Recovery time is a setting you have to choose on purpose.** The checkpoint interval sets the upper limit on how long a restart takes.

When not to use it (really: when to turn it down)

You almost never *pick* a WAL. Your database picked it long ago. What you do pick is when to *weaken* it:

- **Data that truly doesn't matter.** A pure cache can do without durability. If it dies, it fills up again from the source. That's why Memcached has no persistence at all and Redis lets you switch it off. This is correct, not careless.
- **Bulk loads you can run again.** Reloading **500 million rows** from a file you still have? Relax durability during the load, then turn it back on. This is a normal, honest technique.
- **Learn the polite phrasings.** `fsync = off` or "asynchronous commit" both mean the database might say "saved" before that's true. That can be fine, for example for a metrics store that can afford to lose a second. But make that decision deliberately and write it down. Don't just copy a config from a blog post.
- **Don't build your own.** If your app needs a durable append-only log, use a database or a message queue. **Kafka** is this very pattern packaged as a product. Almost everyone gets hand-written fsync handling wrong at least once.
- **Benchmark trap:** a storage engine that says it's "10× faster than Postgres" often has durability turned off. Take away the persistence guarantee and any database gets a lot faster.

Real-world examples

- **PostgreSQL's WAL** — the reference implementation. It does three jobs at the same time: crash recovery, replication (streaming the WAL to replicas), and point-in-time recovery (archived WAL + base backups).
- **MySQL/InnoDB** keeps a *redo log* for exactly this pattern. **SQLite's WAL mode** means most phones use it too.
- **Kafka** — the pattern turned into a product. It is a system that is *nothing but* the log, with consumers doing the replaying. That's why it kept showing up in Module 2.
- **DynamoDB, Aurora, Spanner** — every cloud database has this at the base of its stack.
- **For AI engineers:** vector databases (**Qdrant, Milvus, Weaviate**) put their ingestion through a WAL, so an embedding they confirmed is never lost. Long training runs use the same checkpoint-plus-log thinking: regular snapshots of the model's state let a crash resume the run instead of restarting it. An agent can durably record the action it intends *before* it performs a side effect. Then a crashed agent can recover and reconcile, as long as the action is also idempotent.

In the interview

The direct version: "How does a database survive a power cut without losing acknowledged writes?" Related ones: design a **key-value store** (a database that just maps a key to a value, like a giant dictionary), explain why sequential writes are fast, or describe what a restarted database does before it accepts queries.

> **30-second answer:** Before it touches any data structure, the database appends one record describing the whole change to an append-only log and fsyncs it. Only after that does it acknowledge the client. Tables and indexes are updated later, in the background. On restart it replays from the last checkpoint and uses the checksum to throw away a torn record at the tail, so every acknowledged write survives. The append is sequential, so the safe path is also the fast path, and group commit packs many commits into each fsync. The same log provides

replication (send it to replicas) and point-in-time recovery (archive it along with snapshots).

Likely follow-ups (with answers):

- *"Why is the log faster than just updating the data files?"* — It's one sequential append to one file, compared with scattered writes to many pages. Disks, SSDs included, strongly prefer streams. Also, one log record can stand for a change that touches many pages. That gives you atomicity, which in-place writes physically can't provide.
- *"What bounds recovery time, and what if the crash tore the last record in half?"* — The checkpoint interval limits replay, since only what's newer than the last checkpoint gets replayed. The torn tail record fails its checksum and is discarded. That's safe by design: it can't have been fsynced, so its client never heard "saved". Linking this back to the acknowledgment rule is what makes the answer complete.

Mistake that fails candidates: saying "the database writes it to disk" without explaining the order of steps. Or quoting benchmark numbers that turn out to have durability switched off.

Self-check

- **Q1. Put these in order — apply to tables, fsync, acknowledge client, append to log — and state the rule that makes it safe.** → Append to log → fsync → acknowledge → apply to tables (later, in the background). The rule: never acknowledge before the log record is fsynced. Then no promise that was acknowledged can be lost, whenever the crash happens.
- **Q2. One team checkpoints every 15 s, another every 2 h. Describe their good and bad mornings.** → The 15-second team pays all the time: frequent page flushing, more background I/O, and a little less spare throughput. But they recover in seconds. The 2-hour team has little overhead day to day. But after a crash they may have to replay up to two hours of log, which means minutes to tens of minutes of downtime once the machine is back up. Neither choice is wrong. Pick based on the **recovery-time objective** (how long a restart is allowed to take), not on the defaults.
- **Q3. A teammate says a new engine is "11× faster on writes" than Postgres; its config says `durability: relaxed`. Explain the 11× and make it fair.** → Most of the 11× is the cost of the durability promise, handed back. The engine acknowledges before the data is on disk. So you're comparing the speed of memory with the speed of a disk promise: "saved means saved" versus "saved means probably". For a fair test, set `synchronous_commit = off` in Postgres, or make the new engine fsync on every commit, and run it again. Relaxed durability is fine if the workload can afford to lose a second. The mistake is not knowing you chose it.

Related patterns

- **Primary-Replica** — the replication stream *is* this log. A replica is crash recovery that never stops.
- **Event Sourcing** — the next lesson. The log stops being a hidden internal technique and becomes the application's data model.
- **Change Data Capture** — tools that read the WAL directly and turn database changes into an event stream.
- **Message Queue** — Kafka is this pattern as a product of its own.

TL;DR

- **Problem:** one logical write touches many scattered spots on disk, and the power can fail between any two of them. So writing in place can't deliver "saved means saved" at any speed.
- **Mechanism:** append one checksummed record for the whole change, fsync it, then acknowledge. Apply it to the real structures later. After a crash, replay from the last checkpoint. Use group commit to cover many writes with one fsync.

- **Costs:** every change is written twice, fsync sets a minimum latency, the log needs housekeeping, and recovery time is capped by your checkpoint interval.
- **Skip/relax it when:** the data is truly disposable (caches), or a bulk load you can redo justifies it for a while. Always do it on purpose, never by config accident.

Event Sourcing

Original lesson: Event Sourcing on DesignGurus ↗

What it is. Normally you store the *current state* of things and overwrite it whenever something changes. Event sourcing flips that: you store *what happened*. Each change is appended as an immutable event with a business meaning (`OrderPlaced`, `ItemRemoved`, `OrderCancelled`). **Immutable** means it never changes once written. The current state is simply the result of replaying an entity's events from the start. The log isn't a record of the truth. The log *is* the truth.

Analogy. Your bank doesn't keep your balance and overwrite it. It keeps your transactions, and your balance is their running total. Ask "why is my balance $412?" and the bank can explain it line by line, forever. A mistake gets fixed by adding a correcting entry, never by erasing an old one. Accountants have worked this way for about five hundred years.

The problem

In one week, the team gets three questions. The database can't answer any of them:

- **Monday, a dispute.** A customer says they cancelled order 7f3a before it shipped, and they want their return postage refunded. Support sees `status = shipped`. Was the order ever `cancelled`, maybe for a few minutes at 14:02 before the shipping job overwrote it? The row has a single status column, and each `UPDATE` replaced the value before it. The history isn't hidden. It's **destroyed**.
- **Wednesday, a bug.** A pricing bug went out two weeks ago, got some totals wrong, and was later fixed. Finance wants to know which of the roughly **90,000** orders from that period were affected. But the wrong totals were updated in place, some were fixed by hand, and the fix wiped out the evidence.
- **Friday, product.** "How often do customers add an item and then remove it before checkout?" Nobody ever stored cart removals. The data wasn't lost. It was simply never kept.

There's one root cause. The database keeps only the present, and `UPDATE` destroys what came before. Think back to the last lesson. Beneath its tables, the database *already* keeps an append-only log of every change. But it keeps that log privately, for its own survival, and in **row-diffs** ("column X changed to value Y"), not in business terms. What if the application kept a similar log, in language the business understands, and treated that log as the truth?

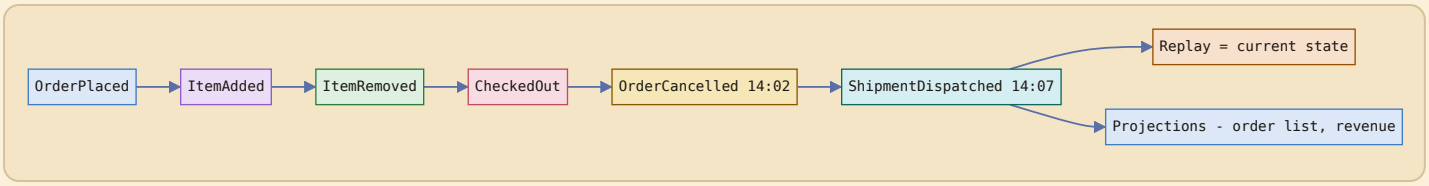
Why the obvious fixes fail

- **"Add an audit table"** (**triggers**, i.e. database rules that run automatically on every change, or app code copy every change into `order_history`). It half works. But it's a copy added as an afterthought. Some code path always forgets to write to it, backfills skip it, and on the day it disagrees with the orders table, which one is right? An audit log that sits beside the truth is only as reliable as its sloppiest writer. It also still records diffs ("status: shipped") rather than intent ("customer clicked cancel").
- **"Add columns: `updated_at`, `previous_status` …"** This clutters the schema and captures just one step of history. It answers Monday's question only if you're lucky, and can't answer Wednesday's or Friday's at all.
- **"We have point-in-time backups."** That's true (from the WAL lesson). But restoring a database from two weeks ago just to answer one question is slow and clumsy. And a backup shows *that* a row changed, never *why*. No backup contains the fact "the customer cancelled".

All three attach history to a design whose source of truth forgets. Event sourcing turns that relationship around.

How it works

Order 7f3a's stream: replay reveals the cancel-then-ship race the old design overwrote.



The diagram above shows order 7f3a as a chain of six events. It was placed ($49, item A, 13:41). Item B was added ($89). Item A was removed ($40). It was checked out ($40, 13:51). The customer cancelled it at 14:02. UPS dispatched it at 14:07. Replaying all six gives the current state: "shipped but cancelled". The old design quietly overwrote that conflict. Now it's out in the open. The same stream also feeds projections, such as an order list and revenue totals.

1. **The event store.** This is an append-only home for events. It is organized as **one stream per entity** (for example, all of order 7f3a's events, in order). Appending is the *only* kind of write. It's the write-ahead log again, one level higher, using business verbs instead of row-diffs.
2. **Current state = replay.** Load 7f3a's six events and apply them in order, starting from nothing: place, add, remove, check out, cancel, ship. What you end up with is the state. For long streams, a **snapshot** taken every N events saves the computed state, so replay only has to cover the tail. That's the checkpoint idea, showing up for the third time in this module.
3. **Projections handle queries separately.** You can't ask an event log for "all orders over $100 this month" without replaying everything. So you keep **projections**. These are ordinary tables optimized for reading, such as a `current_orders` table or a daily-revenue table. Each one is built by reading the event stream and updating itself. This is the same mechanism as the local views in the Event-Driven Architecture lesson. Projections are **derived, disposable, and rebuildable** from the log.
4. **Mistakes are corrected, never erased.** The fix for the pricing bug isn't an `UPDATE`. It's a new `OrderTotalCorrected` event with a reason attached. The error and the correction both stay visible forever, which is exactly what finance and auditors want. Immutability isn't a limitation you work around. It's the feature.

Walkthrough with numbers

Here are the week's three questions, answered from the event store:

- **The dispute.** Load stream 7f3a. Event 5 is `OrderCancelled` at **14:02**, and event 6 is `ShipmentDispatched` at **14:07**. The customer is right, and the refund goes out within minutes. As a bonus, you've found a real **race condition** (two steps running without checking on each other): the shipping job doesn't look for cancellations. The evidence of the bug *is* the data.
- **The pricing bug.** Scan two weeks of `CheckedOut` events and pick the ones whose replayed total matches what the buggy formula produced. One query gives you an exact list of affected orders. Then append a traceable series of `OrderTotalCorrected` events. Finance gets a spreadsheet plus a complete record.
- **The product question.** Across all streams, count the `ItemRemoved` events that come after an `ItemAdded` for the same item. Nobody set out to track this question, yet it gets answered after the fact, because everything was kept. This is the pattern's most underrated strength: *history answers questions you haven't asked yet.*
- **The bill.** A typical order stream is **~12 events × 300 bytes ≈ 4 KB**, compared with a single **400-byte** row. That's about **10–15× the storage**. It's a real cost, but small next to what you get. Replaying one order takes microseconds. But **rebuilding a projection over 100 million orders takes hours**. That's the number that actually limits you, and it's why projections are kept up to date continuously instead of being computed on demand.

Event sourcing vs. change data capture (the comparison)

If what you need is history, there's a cheaper option that leaves the application's structure alone. **Change data capture (CDC)** reads the database's own WAL and publishes every row change as an event (a full lesson comes later).

	Event sourcing	CDC-based history
Source of truth	The event log	Your current tables (left as they are)
What is captured	What the business meant: `OrderCancelled`, by customer	Row diffs: status 'active' → 'cancelled'
Adoption cost	You rebuild the write path	You add a connector; the app doesn't change
Can it miss changes?	No, because events *are* the writes	No, because the WAL sees every change (audit tables can miss some)
Answers "why?"	Yes, since intent is stored	Partly: you see what changed and guess why

Routing rule: use CDC when you need *history*. Use event sourcing when the *events are the product*. CDC gives you a complete change history that can't miss anything, for the cost of a connector. It's the mainstream choice for analytics, caches and search indexes. Event sourcing is worth its cost only where the business meaning of every change has to be stored as well: money, disputes, audits.

Trade-offs

You gain:

- A complete, trustworthy history *by design*, not by discipline. Audit is the storage model itself, not an add-on.
- Debugging with the full history: the state of anything at any moment, plus the exact steps that led there.
- Analytics after the fact: new questions answered from old data.
- A natural match with Module 2. The events you'd want to publish already exist, so the log also serves as the **outbox** (the store of events waiting to be sent).

You pay:

- **Queries need projections.** The log answers "what happened to this order?" perfectly and "which orders exceed $100?" very badly. So you'll maintain read models. (The next lesson, CQRS, is all about this tension.)
- **Events last forever, and so do their old formats.** Code written years from now may replay today's `OrderPlaced`. So event schemas need **versioning from day one**: only additive changes, plus "upcast-on-read" logic that upgrades old formats as they are read.
- **Eventual consistency, once more.** Projections trail the log by a moment. Everything about reading your own writes applies here too.
- **The cost of the model itself.** Uniqueness rules ("this email can register only once") are awkward across append-only streams. Teams new to the approach move slowly at first. And data-deletion laws clash with an immutable log. The answer for deletion is **crypto-shredding**: encrypt each person's personal fields with a key of their own, and destroy that key to make the data unreadable everywhere.

When not to use it

This is the course's clearest "when not": **the default answer is no.** Most applications manage current state (profiles, settings, content pages), where history really doesn't matter. A plain table plus an audit log covers the occasional "who changed this?". Use event sourcing where the domain is a **ledger**:

- **Money** — balances, payments, wallets. The bank statement isn't just an analogy here; it's the requirement.
- **Inventory and orders** — contested, audited and prone to disputes, which is exactly the world of the incident.
- **Anything audited for compliance**, where "show me the complete, provably untampered history" is required by law.

Skip it for high-volume data where only the current value matters (sensor readings where only the latest counts), for CRUD-style admin tools (**CRUD** = create, read, update, delete), and for any team whose unfamiliarity would cost more than the history is worth. Watch for two misuses. **Ideological event sourcing** means converting the whole company because the payments team loved it. **CRUD disguised as events** is a log full of `OrderUpdated {field: status, new_value: shipped}`. That's just row-diffs with event names. Real events are business verbs (`OrderCancelled`, `PaymentCaptured`).

Real-world examples

- **Banking and accounting** — the oldest example, roughly 500 years old. A double-entry ledger is really an event log kept by hand.
- **Git** (the standard version-control tool for code) — commits are immutable events, your working directory (the files you currently see) is a projection, and `git log` is the audit trail.
- **Kafka-centric architectures** — the topic is the durable log, and every database is a projection of it. This is event sourcing at the scale of a whole company.
- **EventStoreDB, Axon, Marten** — event stores built for the purpose.
- **For AI engineers:** agents are event-sourced whether you planned it or not. A conversation is an append-only stream of messages, tool calls and results, and the agent's state is a replay of that stream. That's why you can re-run an **eval** (an automated test of a model's or agent's quality) from history, debug by replaying a session, and audit what an agent did and why. So store agent traces as real event streams. Feature stores need the same idea for **point-in-time correctness**. Training a model "as of March 1st" means replaying feature history up to that moment, and that only works if the history was stored as events.

In the interview

The classic prompt: "Design a banking ledger or digital wallet". Say that the transactions are the authority and balances are derived from them. Other prompts: "we need a complete audit history", "how would you find out what happened to one order?"

> **30-second answer:** For domains that truly are ledgers, like money, orders and inventory, I store events rather than state. Each entity has an append-only stream of business facts such as OrderPlaced and OrderCancelled. Current state comes from replaying that stream, with snapshots to keep replays short. Queries run against projections: read tables built from the events, which can be thrown away and rebuilt. Mistakes get compensating events, never edits, so the audit trail is the storage model. The costs I manage are eventually consistent projections, event schemas that can only evolve additively forever, and crypto-shredding for deletion. I pay this price only where history is the product. For plain history, CDC on a normal database is cheaper.

Likely follow-ups (with answers):

- *"An account has 2 million events — do you replay them all for the balance?"* — No. Snapshots save the state every N events, and only the tail since the last snapshot gets replayed. It's the same idea as database checkpoints. For accounts read often, a projection that is kept continuously up to date means no replay at all at request time.
- *"Legal says delete this user, but your log is immutable."* — Crypto-shredding. From day one, personal fields in events are encrypted with a key per user. Destroying that key makes every copy unreadable: the log, snapshots, projections *and backups*. The structure is immutable, but the content can be erased. Mentioning backups without being asked is a sign of a senior candidate.

Mistake that fails candidates: using event sourcing for everything. Writing row-diff events with event names (`OrderUpdated`). Or having no plan for projections ("we'll replay on every request"

falls apart at the first entity with a long history).

Self-check

- **Q1. Where does truth live, how is current state obtained, and what happens when a past event was wrong?** → The truth is the append-only event log. Everything else (tables, dashboards, caches) is a projection. Current state = replaying the entity's events in order, made shorter with snapshots. A wrong event is never edited or deleted. Instead a compensating event is appended (for example `OrderTotalCorrected`, with a reason), so both the mistake and its fix stay visible.
- **Q2. A digital wallet, user profile settings, warehouse inventory, a marketing CMS — which two deserve event sourcing?** → The wallet and the inventory. Both are ledgers: contested, audited, and "how did we reach this number?" is critical to the business. Profile settings and the CMS are current-state domains. Use plain tables, plus an audit log or CDC if "who changed this?" matters now and then.
- **Q3. The revenue dashboard (a projection) disagrees with the event store's total. Which is right, what happened, what's the fix?** → The event store is right: it's the authority, and the projection is derived from it. Most likely the projection's consumer crashed and missed events, applied a redelivered event twice without **idempotency** (handling repeats safely), or has a bug. Fix the consumer. Then rebuild the dashboard table by replaying the log. That may take hours for a big store, but derived data can always be regenerated instead of staying corrupted.

Related patterns

- **Write-Ahead Log** — the same technique, one level lower. Event sourcing is the WAL promoted into your data model, with business verbs.
- **Publish-Subscribe** — the events you'd announce already exist, so the log also acts as the outbox.
- **CQRS** — the next lesson. Keep the write side shaped as events, and give the read side completely different storage.
- **Change Data Capture** — the cheaper choice when you only need plain history.

TL;DR

- **Problem:** `UPDATE` wipes out history. So disputes can't be settled, the reach of a bug can't be reconstructed, and questions nobody asked yet can't be answered later.
- **Mechanism:** append immutable business events (one stream per entity) and treat them as the source of truth. Get state by replaying them (with snapshots). Serve queries from projections you can rebuild. Fix mistakes with compensating events.
- **Costs:** projections and their lag, ~10–15× the storage, schemas that can only grow additively forever, crypto-shredding for deletion, awkward uniqueness rules, and a learning curve.
- **Skip it when:** the domain is current-state CRUD, which covers most domains. Default to no. Use CDC or an audit log for plain history. Save event sourcing for ledgers: money, orders, inventory, compliance.

CQRS

Original lesson: CQRS on DesignGurus ↗

What it is. CQRS (Command Query Responsibility Segregation) breaks one data model into two. **Commands** are writes, such as "place order 7f3a". They go to a **write model** designed for correctness: normalized (each fact stored once), validated and transactional. **Queries** are reads, such as "orders for customer 8412". They go to one or more **read models**, and each one is shaped to fit its question exactly: pre-

joined tables, search indexes, aggregates (pre-computed totals and counts). A stream of changes from the write side keeps every read model current.

> **Analogy.** At a bank, one desk takes deposits, and a separate system prints statements and reports. Each is fast because it only has one job.

The problem

The orders database serves every team, and every team wants something different:

- **Checkout** writes orders. It wants a **normalized** schema (each fact stored in exactly one place), strict validation and transactions, so nothing invalid slips in.
- **The orders page** has to join six tables (order, items, customer, addresses, shipments, payments) to draw a single screen. That takes **300 ms**.
- **Support** wants to search orders by customer name and by product. That's text search, and the database is bad at it.
- **Analytics** wants daily **aggregates** (totals and counts across many rows).

Each read problem was "fixed" by adding another **index**. But an index is really a copy of part of the table, sorted a different way, and it has to be updated on *every* insert. The orders table now carries **14 indexes**, so each new order does **15 writes**. Write latency has slowly risen from **5 ms to 40 ms**. Last month, one extra index (for support search) made checkout a measured **15%** slower. The write side wants a lean schema. The read side wants it pre-joined, duplicated and indexed sixteen different ways. Because both sides share **one model**, every improvement for one side hurts the other. Replicas fixed reads and writes that differed in *volume*. Here they differ in **shape**, and adding more copies of the same shape can't fix a shape problem.

Why the obvious fixes fail

- **"Add the index anyway."** Each index speeds up one query and slows down every write, permanently. At 14 indexes, this approach has run out of road.
- **"Denormalize the main table."** **Denormalizing** means copying one fact into several places, so a query can avoid a join. Here, that means flattening everything into one wide orders table. Reads get faster. But every write now has to keep all the duplicates in step. If a customer changes their name, you either update a million old rows or live with wrong data. You've blended the read model into the write model and made both worse.
- **"Read replicas."** Replicas deal with volume, not shape. A replica has the same schema, so the six-way join takes 300 ms there as well.
- **"Cache the hot queries."** This helps (Module 4). But the shape mismatch is still there, and now cache invalidation is your problem too. It's a workaround, not a fix.

The real fix: stop asking one model to do two jobs.

How it works

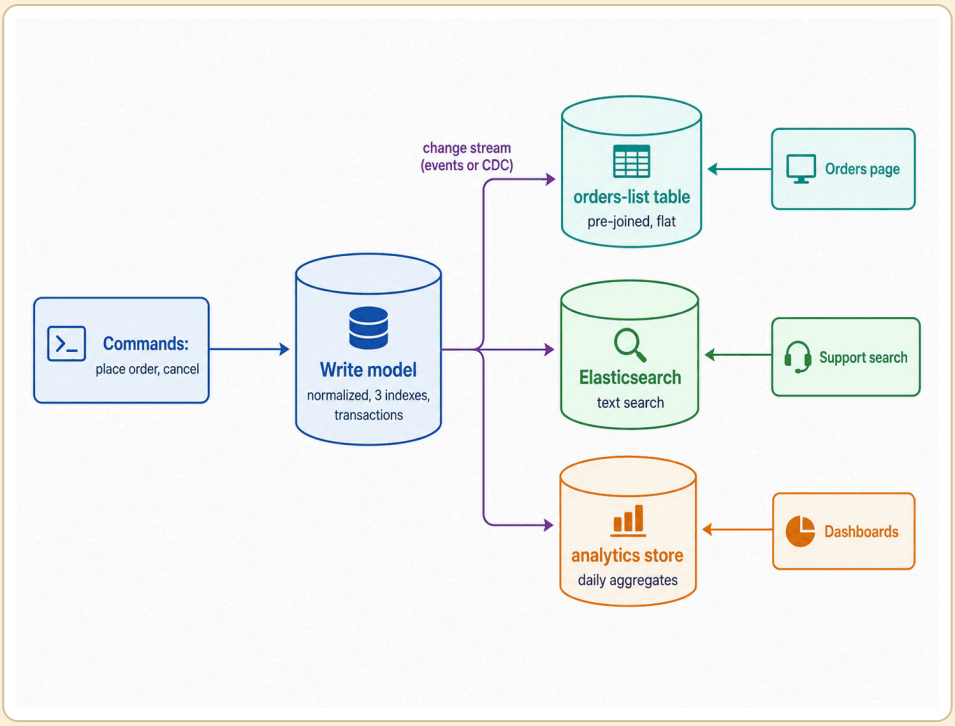


Figure — CQRS (course diagram)

In the figure above, commands (place order, cancel) go into a write model. It is normalized, has only 3 indexes and uses transactions. A **change stream** (events or CDC) flows out of it into three read models. A flat, pre-joined `orders-list` table serves the orders page. Elasticsearch (a dedicated search engine) serves support search. An analytics store of daily aggregates serves dashboards.

1. **Commands and queries travel separate routes.** A command goes to the write model, which checks it and commits it. A query goes to whichever read model has the right shape for it, and it **never touches the write model**.
2. **Read models are projections.** They're the same derived, disposable, rebuildable copies from the Event Sourcing lesson (and the local views from the EDA lesson). CQRS is the name for splitting the model. The machinery behind it is something you already know.
3. **Sync from the log, never with dual writes.** This is the first thing everyone gets wrong. The tempting approach has app code write to Postgres and *then* write to **Elasticsearch** (a search engine) as well. That's a **dual write**, and it's a trap. The two writes aren't one transaction. The app can crash in between. Then the two models disagree forever, and nobody knows. The safe approach reads one stream of truth. Either consume the write model's **domain events** (business facts such as `OrderPlaced`), or tap the database's own change log using **change data capture (CDC)**, which reads each committed change out of the WAL and passes it along. Projection updates must be **idempotent**: applying one twice has the same result as applying it once. That makes redeliveries harmless. If you use event sourcing, your projections are already CQRS: one writer, one log, many followers.
4. **CQRS is a spectrum, not a single step.** The second thing everyone gets wrong is assuming it always means "two databases". It has three levels:
   - **Level 1:** the same database, but separate code paths and query models for reads and for writes. It's cheap, and it's often enough.
   - **Level 2:** read models as separate tables or replicas with a different shape (for example, a flat orders-list table).
   - **Level 3:** separate systems, such as Elasticsearch for search and a **data warehouse** (a database for analytics) for reports. Move along the spectrum only as far as your measured shape conflict forces you to.

CQRS and event sourcing are related, but they aren't the same thing. Event sourcing can exist without CQRS (a log with a single state view). CQRS without event sourcing is everywhere (read models fed by CDC over a plain **relational** write model, i.e. ordinary SQL tables linked by keys). You can adopt either one by itself.

Walkthrough with numbers

- **The write model drops 11 of its 14 indexes.** It keeps the 3 that support writing and validation. Insert latency drops from **40 ms to 8 ms**, and checkout feels it right away.
- **The orders page** reads from a flat, pre-joined `orders_list` projection. The **300 ms** six-way join turns into an **8 ms** read from one table. The join now happens once, at write time, inside the projection updater, rather than on every page view.
- **Support search** uses Elasticsearch, fed from the same stream. Full-text queries take about **40 ms**. **Analytics** reads from a warehouse fed the same way, so its heavy queries no longer touch production.
- **The lag.** Read models trail by about **200 ms**. A user places an order, and their orders page (a projection) hasn't caught up, so the order "isn't there". This is read-your-writes for the third time (after replica lag and EDA view lag), and the remedies are the same. Send that one read to the write side for a short time, or show the new order in the UI optimistically.
- **Adding features.** Six months later, the mobile team wants a compact order summary. It's simply another projection on the same stream. The write side sees no changes and no risk, much like adding a subscriber in pub/sub.

CQRS vs. read replicas (the comparison)

	Read replicas	CQRS read models
Solves	How *many* reads	What *shape* reads need
The copy is	The same schema, identical byte for byte	Shaped around each query: flattened, indexed and pre-summed
A 300 ms six-way join becomes	Still 300 ms, just on another machine	8 ms, because it was pre-joined at write time
Cost	Low: one checkbox on managed databases	A pipeline, projections, and managing lag
Lags the primary?	Yes	Yes; this trade-off never goes away

Diagnostic question: would more copies of the same shape help? If there are simply too many reads, use replicas (cheaper and quicker to set up). If reads have the wrong shape (joins at read time, text search, aggregates), no number of same-shaped copies will help. Reshape the data with CQRS. Real systems combine the two: a CQRS read model that is itself replicated to handle volume.

Trade-offs

You gain:

- Each side tuned without slowing down the other: lean, fast writes and reads shaped for their purpose.
- Read scaling per model: each read model can be cached, replicated or sharded to fit its own load.
- New read needs handled by adding projections, not by risky edits to the core schema.
- The right tool for each question: full-text search in a search engine, aggregates in a warehouse, all from one source of truth.

You pay:

- **Eventual consistency between the two sides.** The read side catches up after a delay, and read-your-writes has to be handled. This can't be negotiated away, so plan for it.
- **A pipeline to run.** The change stream, the consumers, the lag metrics and the alerts are real infrastructure. That's the price of avoiding the dual-write trap.
- **Duplicated data.** It costs storage, and you need the discipline to remember which copy is the truth (always the write model).

- **Complexity that has to be justified.** CQRS is one of the most over-used patterns in the industry. Whole CRUD apps get split in two because a conference talk sounded exciting. It's only worth it for a real shape conflict you have actually measured.

When not to use it

1. **Most applications, to be honest.** A CRUD app at moderate scale, where one schema serves reads and writes well enough, needs indexes and maybe a replica. It doesn't need two models and a pipeline. The incident's 14-index conflict was *measured*. Wait until you have one of your own.
2. **Reads must always be instantly consistent.** If every read everywhere has to see the latest write, the lag rules out the split. Otherwise you end up sending so many reads to the write side that the split achieves nothing.
3. **The team can't operate the pipeline.** Without lag monitoring, backfills and projection rebuilds, read models can drift apart without anyone noticing. Users trusting stale results is worse than users getting correct results slowly.
4. **Level 1 is enough.** Often, "CQRS done right" simply means this: stop letting the **ORM**'s single model (the library that maps tables to code objects) shape both paths, and write focused query code. If that fixes it, you never needed an architecture change.

Misuses: turning on CQRS by default in every microservice template. And the dual-write implementation (app code writing to both models separately), which gives you all of the pattern's operating cost and none of its correctness.

Real-world examples

- **Every large e-commerce (online shopping) product page** — a normalized write side for catalog (the product listings) and inventory, plus a denormalized, pre-joined, heavily cached product-view read model, with a search index next to it.
- **Twitter timelines** — the famous extreme case. The write side stores tweets. The read side builds each user's home timeline in advance at tweet time (**fan-out**), so reading is just fetching a list. The "tweets from everyone I follow" join runs once at write time instead of billions of times at read time.
- **YouTube, LinkedIn, Uber** — view counts, profiles and trip histories are read from projections shaped for the purpose, not from the transactional core.
- **For AI engineers:** a **feature store** is CQRS. A compute/write path builds features, and a separate low-latency online path serves them, with pipelines keeping the two in sync. Every **RAG** system (retrieval-augmented generation: a model answers using documents fetched for the question) is CQRS as well. Documents live in a source-of-truth store (the write model). The **vector index is a read model** shaped for "what is similar to this?". It's fed by the document change stream and rebuilt when the embedding model changes, which is the projection-rebuild property from Event Sourcing.

In the interview

The everyday version: "this page joins six tables and it's slow — fix it". The famous version: "design Twitter's timeline" (precomputing at write time is a CQRS read model, whether or not anyone uses the acronym). Another: "add full-text search over orders" (CQRS, with Elasticsearch as the projection).

> **30-second answer:** Reads and writes want different shapes. Writes want normalized, validated data. Reads want data pre-joined and indexed for each question. One model slows both down. So I split them. Commands go to a lean write model. Queries go to read models shaped for each question: a flat orders table, a search index, an aggregates store. Every read model stays in sync by consuming the write side's change stream, either events or CDC. Never dual writes from app code, because two unsynchronized writes drift apart. Read models lag slightly, so a read that must show the user's own write goes to the write side or updates optimistically. And it's a spectrum: separate query code, then reshaped tables, then separate systems. I only go as far as the measured conflict forces me.

Likely follow-ups (with answers):

- *"How exactly do the read models stay in sync?"* — They consume one authoritative change stream. That's the event log if you use event sourcing, and otherwise CDC reading the database's WAL. Bring up the dual-write trap without being asked: app code writing to two stores can crash in between, and log-based sync exists to make that impossible. Projection updates are idempotent, so redeliveries are safe.
- *"A user places an order and their order list doesn't show it."* — Read-your-writes, the third time it appears in the course. The list is a projection that lags. For a few seconds after the write, send that user's list read to the write side, or show the new order optimistically. What the interviewer is scoring is whether you see this as the same problem as replica lag, with the same remedies.

Mistake that fails candidates: separate read and write stores kept in sync by dual writes from the application. CQRS for a settings page. Or claiming that CQRS requires event sourcing.

Self-check

- **Q1. What does the write model optimize for, what do read models optimize for, and what single rule governs sync?** → The write model optimizes for correctness: normalized, validated, transactional, with as few indexes as possible. Each read model optimizes for the shape of one question: pre-joined, aggregated, or text-indexed. The rule: every read model is fed from **one authoritative change stream** (events or CDC). Never have app code write to several stores, because dual writes can partly fail and drift apart silently.
- **Q2. Team A: checkout and orders page are fine, but the new free-form order search is painfully slow. Team B: dashboards are fast but write latency tripled as indexes piled up. Prescribe a level for each.** → Team A has a shape problem limited to one feature (full-text search is poor in a relational store no matter how many indexes you add). Give them **Level 3 for that one path only**: an Elasticsearch projection fed by CDC, with everything else left as is. Team B has the incident's problem (read indexes slowing down writes). Give them **Level 2**: move dashboard queries to a reshaped read table (or a replica + projection) and drop the extra indexes on the write side. Neither team needs event sourcing or a rewrite. The spectrum exists so you can stop early.
- **Q3. After a deploy, some orders in Postgres are missing from search. The app writes to Postgres, then calls Elasticsearch, and some ES calls timed out during the deploy. Root flaw, immediate remediation, permanent fix?** → The root flaw is the **dual write**: two separate calls with no shared transaction. A failure between them leaves the models different permanently and invisibly. Immediate step: backfill. Compare recent Postgres orders with the index and re-index the missing ones. Permanent fix: remove the second write. Feed Elasticsearch from the change stream (CDC, or the outbox/event stream), applying updates idempotently. After that, a sync failure turns into *lag* (visible, monitored, heals itself) instead of *loss* (silent, permanent). Read models have to follow the log, not the application.

Related patterns

- **Event Sourcing** — the natural partner. Its projections are ready-made read models, and its log is the ideal change stream (though each works on its own).
- **Change Data Capture** — how plain relational write models feed read models without dual writes. It's the mainstream route to CQRS.
- **Primary-Replica** — handles read volume, while CQRS handles read shape. The two are often stacked as replicated projections.
- **Cache-Aside** — a cache is a small read model that doesn't live long. The next module turns that idea into five patterns.

TL;DR

- **Problem:** one model serving both sides makes both worse. Every index added for reads slows every write, while reads still pay for joins and searches the write schema was never designed for.
- **Mechanism:** commands go to a lean, validated write model. Queries go to read models shaped for their purpose, all fed from one change stream (events or CDC), never from dual writes in the app.
- **Costs:** eventual consistency and handling read-your-writes, a sync pipeline to run, duplicated data, and complexity that only a measured shape conflict can justify.
- **Skip it when:** one model serves both sides well enough (most apps), reads must always be instantly consistent, or the team can't run the pipeline. Move along the spectrum only as far as you need.

Capstone - Scaling a Database to 100 Million Users

Original lesson: Capstone - Scaling a Database to 100 Million Users on DesignGurus ↗

What this capstone is. This is the story the module introduction promised: **one database machine grown to 100 million users, adding one pattern at a time.** The first half is a *scaling* story: each stage is a volume limit that a pattern removes. The second half is a *quality* story: limits around trust, history and shape. At every stage, the course stops and asks you to name the pattern and its key decision before you read on. Do it. The order matters. Real companies often run into exactly this sequence, and interviewers love to ask "which limit comes next?".

The brief

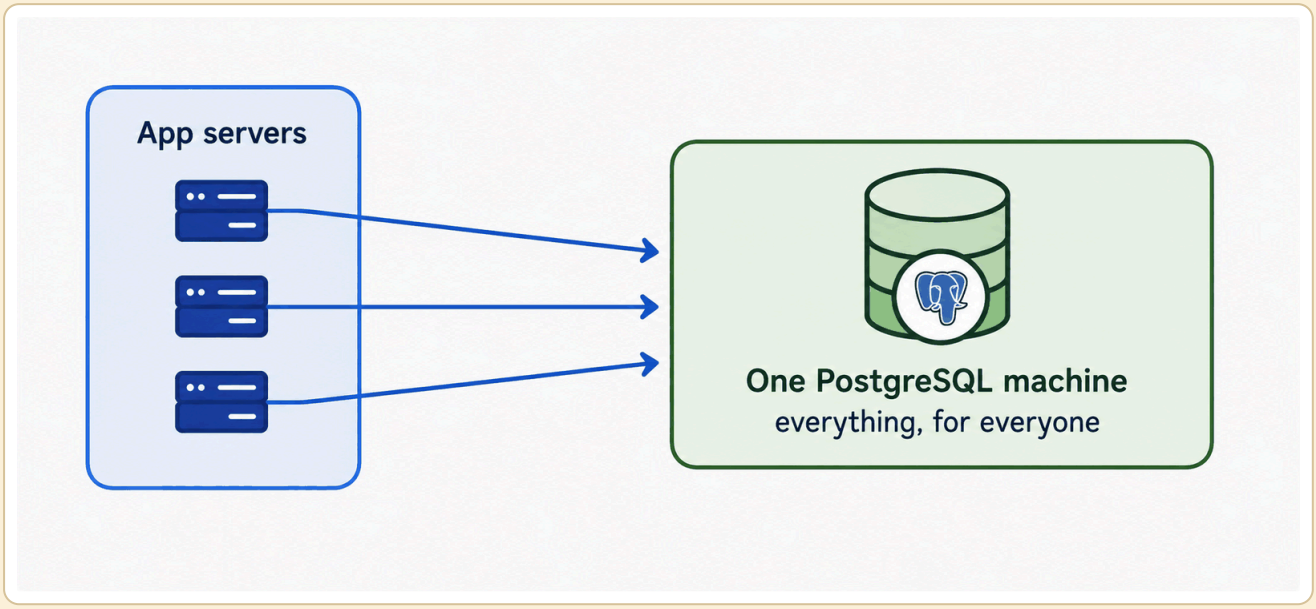


Figure — Scaling the DB: the brief (course diagram)

In the figure above, a few app servers all point to **one PostgreSQL machine** that does "everything, for everyone". That's where we start: a commerce startup with **50,000 users** and a few hundred queries per second. A single machine handles orders, customers, inventory, search and dashboards. Life is easy. Transactions cost nothing, and any query can join any table. It won't stay that way.

Stage 1 — Running out of read capacity (500,000 users)

Incident. The company has grown 10×. There are **8,000 reads/s + 300 writes/s**, but the machine can handle only **5,000 queries/s**, and product pages take four seconds. During the incident review, someone

points out a second problem. This machine is also the company's *only copy* of the data, and last night's backup is the whole disaster plan.

Pause and decide: 96% of traffic is reads. Which pattern fits? Where are writes allowed to go? And what new problem do copies bring?

Decision: Primary-Replica. Every write goes to the primary. Two replicas receive its change stream and serve the reads (**~4,000 reads/s each**, which is comfortable). Why this instead of a bigger machine or a cache? Because it fixes read capacity and the single point of failure at the same time. Failover is **rehearsed**, not just configured. The team runs a practice promotion and learns its **fencing** procedure (blocking the old primary from taking writes) before a real 2 AM failure forces the lesson.

New problem, just as predicted: replication lag. A user edits their address, the next page reads from a replica, and the old address reappears. **Reads that need fresh data are sent to the primary.** Everything else can live with a few milliseconds of delay.

What this sets up: reads are handled, the business keeps growing, and writes become the problem. Every replica applies every write, so copies can't help.

Stage 2 — Running out of write capacity (5 million users)

Incident. Writes hit **2,500/s**. The primary is maxed out, and so is every replica, since they all apply the same stream. Lag stretches to **minutes**. The orders table is **4 TB**. Its indexes don't fit in memory anymore, backups run all night, and there's no bigger machine left to rent.

Pause and decide: copies can't help, so the data has to be divided. Which pattern is it, and what is its single most important decision?

Decision: Sharding on `customer_id`. Split the rows over **4 shards**. The shard key matters more than any other decision. `customer_id` wins for three reasons. It appears in almost every query. It spreads load evenly. And it keeps each customer's orders, cart and addresses on one shard, so checkout never crosses machines. **Hash routing** provides the even spread. Every shard is its own primary-replica cluster, and the operations bill is stated plainly: **4 shards × 3 machines = 12 databases**. The costs are accepted openly. "Top products today" turns into **scatter-gather** across 4 shards. Transactions across shards are *avoided through the key choice*. And when one influencer's storefront goes viral (hashing spreads keys, not load), **a cache takes that hot key's reads**.

What this sets up: the routing rule is `hash(customer_id) % 4`, and that `% 4` is a problem that will show up two years later.

Stage 3 — Changing the shard count (20 million users)

Incident. Four shards need to become five. Someone checks the math first. Changing `% 4` → `% 5` gives about **80% of all rows** a new address, because a key keeps its home only when both remainders match, which happens **one time in five**. Terabytes would have to move during business hours while 2,500 writes/s keep arriving. It would be a weekend-long migration, repeated at every future resize.

Pause and decide: the formula is the problem, not sharding. What replaces modulo, and what share of the data *should* a resize move?

Decision: Consistent Hashing. Shards and keys are hashed onto a ring. Each key belongs to the next shard clockwise. Each shard shows up as about **150 virtual nodes**, so slices stay even and a dead shard's load gets spread around. Adding shard five now moves about **1/5** of the data, which is only the slice the new shard takes over. It streams in the background over several days, and the old owner keeps serving until the handoff. **Bonus fix:** the same math also cures the cache layer. Its nightly autoscaling had been causing **miss storms** (bursts of cache misses that all hit the database together), and nobody had linked them to routing.

What this sets up: for the first time, volume is handled (shards for writes, replicas per shard for reads, the ring for growth). The next limits are about **trust**.

Stage 4 — Crash and corruption (a bad Tuesday)

Incident — two scares in one quarter, taken step by step:

1. *Power loss.* A rack loses power in the middle of a write, and shard 2 restarts. The team waits, not sure what "recovery" means or whether the **40 orders** confirmed in the last seconds made it. They did, but nobody can explain why.
2. *Bad deploy.* A buggy deploy writes corrupted totals into thousands of orders for **two hours** before anyone spots it. The corruption is committed and **replicated**: the replicas faithfully copied the bad data. Restoring last night's backup would throw away a whole day.

Pause and decide: a single mechanism explains both the survival and the rescue. What is it, and which two abilities does it give you besides crash safety?

Decision: understand (and use) the Write-Ahead Log.

- *Why the 40 orders survived:* the database **fsyncs** a log record (forces it onto the disk) *before* confirming anything. Replay after the crash rebuilt every promise.
- *How the corruption gets fixed:* the log's third job, **point-in-time recovery**. Restore the most recent base backup, replay the archived WAL up to **13:59** (one minute before the bad deploy), and get back the clean state. Then re-apply the legitimate orders from the corrupted window. Replicas couldn't help, because they copy bad writes as well. Only history can.
- *Operational follow-up:* checkpoint intervals are set from recovery-time targets, WAL archiving is checked, and there's a new rule: durability settings (for example, for benchmarks) are decided in writing.

What this sets up: all along, the database has kept a perfect history for its own use, in row-diffs. Meanwhile, support handles disputes with a table that only stores the present.

Stage 5 — History destroyed by UPDATE (60 million users)

Incident. A customer says "I cancelled before it shipped", but the table only shows `status = shipped`. Finance asks which orders a pricing bug from two weeks ago affected. There's no way to know, because the fix overwrote the error. Compliance wants an audit trail for payments that can be proven. Three questions, one cause: `UPDATE` **destroys history.**

Pause and decide: why not an audit table? Which pattern fits, where should it be used, and where shouldn't it?

Decision: Event Sourcing, applied selectively. Orders and payments are the ledger-shaped domains (contested, audited, prone to disputes). They move to append-only event streams (`OrderPlaced`, `ItemRemoved`, `OrderCancelled`). State comes from replay, with **snapshots** for long streams. The dispute is settled in minutes (`OrderCancelled` 14:02, then `ShipmentDispatched` 14:07 — so the customer was right), and a race-condition bug turns up as a bonus. Corrections turn into **compensating events** with reasons attached, just what finance asked for. **Where not:** profiles, sessions and the product catalog stay as plain tables. Current-state domains shouldn't pay the cost. **Why not an audit table:** a copy kept next to the truth drifts, because some code path forgets to write to it. Here the log *is* the truth, so it can't miss anything.

What this sets up: the orders write side is now shaped as events. That's great for history and useless for "all orders over $100". Queries need projections, and the rest of the system has a related problem anyway.

Stage 6 — One model, two jobs (100 million users)

Incident. Loading the orders page takes **300 ms**, because it joins six tables. Support wants full-text search, which the database handles badly. Dashboards run heavy aggregates against production. At the

same time, the write path carries **14 indexes**, so every insert does fifteen writes, and support's newest index measurably slowed checkout. Every fix for reads slows down writes.

Pause and decide: why won't replicas help? Which pattern fits, what feeds it, and which implementation trap must you refuse?

Decision: CQRS. Replicas won't help, because they're copies with the same shape: the 300 ms join costs 300 ms on each one. So the model is split. The write side keeps **3 indexes** and its validation. Reads move to projections shaped for their purpose: a flat `orders_list` table (**300 ms → 8 ms**), Elasticsearch for search, and a warehouse for analytics. All of it is fed from **one change stream**: the event streams where they exist, and **CDC** reading the WAL everywhere else. **Dual writes from app code are refused**, because two unsynchronized writes silently drift apart. Lag (visible, heals itself) is better than loss (silent, permanent). Read-your-writes comes back one final time with its usual remedies. And the mobile team's new order-summary feature ships as just another projection, with no risk to the write side.

The final architecture

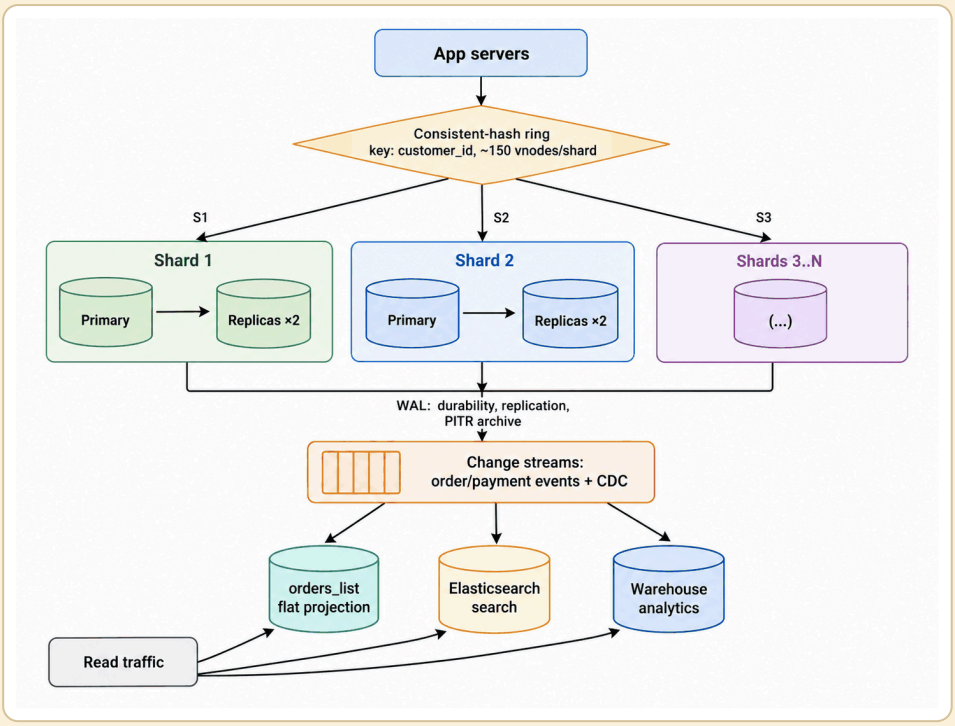


Figure — Scaling the DB: final architecture (course diagram)

In the figure above, app servers route through a **consistent-hash ring** (key `customer_id`, ~150 vnodes per shard) to Shard 1, Shard 2 and Shards 3..N. Each shard is a primary with two replicas. Their **WAL** gives durability, replication and a PITR archive. It also feeds the **change streams** (order/payment events + CDC), which fill three read models: the `orders_list` flat projection, Elasticsearch search, and the warehouse. Those read models serve all read traffic. Count how many jobs the append-only log is doing here: durability, replication, point-in-time recovery, *and* feeding every read model. The introduction said the log was the key idea. Here it *is* the architecture.

Debrief: the limits, in order

Stage	The limit	Pattern	The decision that mattered
1	Too many reads for one machine	Primary-replica	Send reads that need fresh data to the primary; practice failover
2	Too many writes and too much data for one machine	Sharding	Shard key = customer_id, with reasons given
3	Changing how many shards there are	Consistent hashing	Ring + virtual nodes; a resize moves ~1/N
4	Crash and corruption	Write-ahead log	Understood fsync-before-ack; PITR undid the bad deploy
5	History wiped out by UPDATE	Event sourcing	Used only for ledger-shaped domains
6	One model doing two jobs	CQRS	Projections fed from the log; dual writes rejected

- Read down the "limit" column: **reads → writes → growth → trust → history → shape**. That sequence is the honest answer to "how does a database scale?". Each pattern was added because of a *measured* limit, never because it was fashionable.
- Stages 4, 5 and 6 are all the append-only log playing different roles: the database's own WAL, the application's event streams, and the change streams feeding projections. Six patterns, one big idea: **copies for reads, splits for writes, and a log underneath everything.**

Extend it yourself (open questions to practice)

- **Multi-region:** users on two continents want low local latency, and regulators want EU data to stay in the EU. Which patterns stretch across regions cleanly (the ring? replicas?), where does the primary live, and what happens to writes that cross regions?
- **The hot tenant:** your largest enterprise customer is now **8% of all traffic** on a ring keyed by `customer_id`. Design an alternative for it using this module's tools (for example, a dedicated shard through a directory/slot override), and explain what breaks first if you don't.
- **For AI engineers:** add semantic search over orders ("find orders like this complaint"). Design the vector index as a CQRS read model. What feeds it? What happens when you switch embedding models? And why does "projections are disposable and rebuildable" become the feature you care about most?

Interview one-liner

> Scale a database in the order its limits show up: replicas for reads, sharding with a well-argued key for writes and size, consistent hashing for resizing, the WAL for durability and point-in-time recovery, event sourcing only for ledgers, and CQRS projections fed from the log once read and write shapes diverge.

Module 4 — Serving Data Fast (Caching)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. The storage module finished with one remark: *a cache is a small, short-lived read model*. This module builds on that remark. Caches are some of the most widely used pieces of infrastructure, and also some of the most misused. They also come up in system design interviews more often than almost any other storage technique. A **cache** is a small, fast copy of some data. The authoritative ("true") version of that data lives in a slower system, such as a database. (New to caches, databases and latency? The System Design Fundamentals primer introduces them.)

> **Analogy.** Using a cache is like keeping the few books you need every day on your desk, instead of walking to the library each time. It's quick. But if the library's copy gets revised, your desk copy is now out of date, unless someone tells you.

The key metric: hit ratio. A **hit** means the data you asked for was in the cache. A **miss** means it wasn't, so the whole request has to go to the source. The **hit ratio** is the share of lookups that are hits. The math is simple. Say a hit costs 1 ms, and a miss costs that 1 ms *plus* a 100 ms database call. At a **99%** hit ratio the average is about **2 ms**. At **50%** it's about **51 ms**, and on top of that you're now running an extra system. So the real question isn't "should I cache?". It's two linked questions: **how does data get into the cache, and how does it stay correct?** The five patterns give five different answers.

The situation	The pattern
On a miss, the application fetches the data and saves a copy for next time	Cache-Aside
The same idea, except the cache does the fetching itself	Read-Through
Update the copy and the original together before confirming	Write-Through
Update the copy now and write the original later, in batches	Write-Behind
A popular entry expires, and only one request should fetch it again	Cache Stampede Prevention

Two facts that organize the module

1. **A cache buys speed by giving up some accuracy.** Any cached copy can disagree with its source. You have to decide how much staleness you can live with, and which mechanism keeps the copy correct: expiry timers, updating at write time, or deleting on purpose. (Interviewers enjoy the old joke: *"There are only two hard things in computer science: cache invalidation and naming things."* Invalidation takes up half this module.)
2. **Caches fail as crowds, not one request at a time.** A single miss does no harm: it's one short database query. The serious failures happen when a popular key expires and ten thousand requests reach the database at the same moment, or when a cache cluster restarts empty in the middle of peak traffic. The stampede lesson covers these events. They mark the difference between *using* a cache and *running* one reliably.

How the module is told. Every lesson uses the same product page. It's read millions of times a day, and it sits on top of the database we scaled in Module 3. In Cache-Aside (the default and the longest lesson), the app fetches missing data. Read-Through moves that fetching into the cache. Write-Through and Write-Behind are the two write-side patterns (safety versus speed). Stampede Prevention deals with reliability when the flash sale comes back. The capstone chooses a strategy for each kind of data on the page and tests it under load. **Short on time?** For interviews, Cache-Aside and Cache Stampede Prevention matter most. The write pair usually shows up as a comparison question.

Four terms used throughout

- **Hit / miss / hit ratio** — defined above. You should know this number for every cache you run.
- **TTL (time to live)** — an expiry timer on each cached entry ("trust this copy for 5 minutes, then delete it"). It's the simplest and most common tool for keeping data correct.
- **Eviction** — a cache is kept smaller than the full data set on purpose. So when it fills up, it has to throw something away. The usual rule is **LRU (least recently used)**: remove the item nobody has touched for the longest time, on the assumption that recent use predicts future use.
- **Invalidation** — actively removing or updating a cached entry when its source changes. It's more precise than waiting for a TTL, but harder to get right (which is why the joke exists).

Before you start — three questions about a system you know:

- What is your cache's hit ratio today? (If no one knows, the cache's value is a guess.)
- When the underlying data changes, how does the cache learn about it: a timer, an update, a delete, or not at all?
- At peak traffic, what happens the instant your most popular key expires? Who goes to the database, and how many of them go?

Cache-Aside

Original lesson: Cache-Aside on DesignGurus ↗

What it is. The application looks in the cache first. On a **hit**, it returns the cached copy. On a **miss**, the application reads the database, saves a copy in the cache with an expiry timer (TTL), and returns it. That way the next reader gets the fast copy. On a **write**, the application updates the database and then *deletes* the cached copy, so the next reader pulls fresh data. The cache sits "to the side": it never talks to the database on its own.

Analogy. Picture a receptionist who keeps sticky notes with answers to common questions. When someone asks something, they check the notes first. If the answer isn't there, they call the right department, write the answer on a fresh sticky note (marked "throw away after 1 hour"), and reply. When a department announces a change, the receptionist rips up that sticky note.

The problem

In Module 3 we scaled the database with replicas, shards and projections, and it works. But look at what all that machinery spends its day doing. **Item 8412** is this month's bestseller. It's read **4,000 times per minute**. Its row hasn't changed in **three weeks**, yet the database works out the same **15 ms** answer every single time. Across the whole catalog, reads peak at **12,000/s**, and well over **99%** of them return exactly what the request before got. The replica fleet (read-only copies of the database) keeps up, but it needs **eight** database machines that mostly repeat work. **Latency** (the time each request takes) climbs above 15 ms when requests queue up at peak. And every marketing campaign sets off another capacity meeting. The waste is paying database prices for answers that haven't changed. A better idea: keep the last answer in fast storage and serve it until the data changes.

Why the obvious fixes fail

- **"More replicas."** They really do scale read volume, but the cost grows just as fast. You're running full database queries only to reproduce answers that haven't changed. The problem is *repetition*, not a shortage of database capacity.
- **"A dictionary in each app server's memory."** It's cheap and tempting. But with **40 app servers**, each one keeps its own private copy. That means 40 cold misses per key instead of one, 40 copies going stale on their own, and every **deploy** (rollout of a new code version, which restarts the servers)

wiping all of them and dumping the full load back on the database. It only works for data that is tiny, used all the time, and unaffected by staleness.

- **"Cache it forever."** The hit ratio is excellent. But customers never see Friday's price change. A cache with no way to stay correct doesn't give you fast current data. It gives you *fast old data*.

What you need: one shared, fast copy that fills up when data is requested, plus a plan for keeping it correct.

How it works

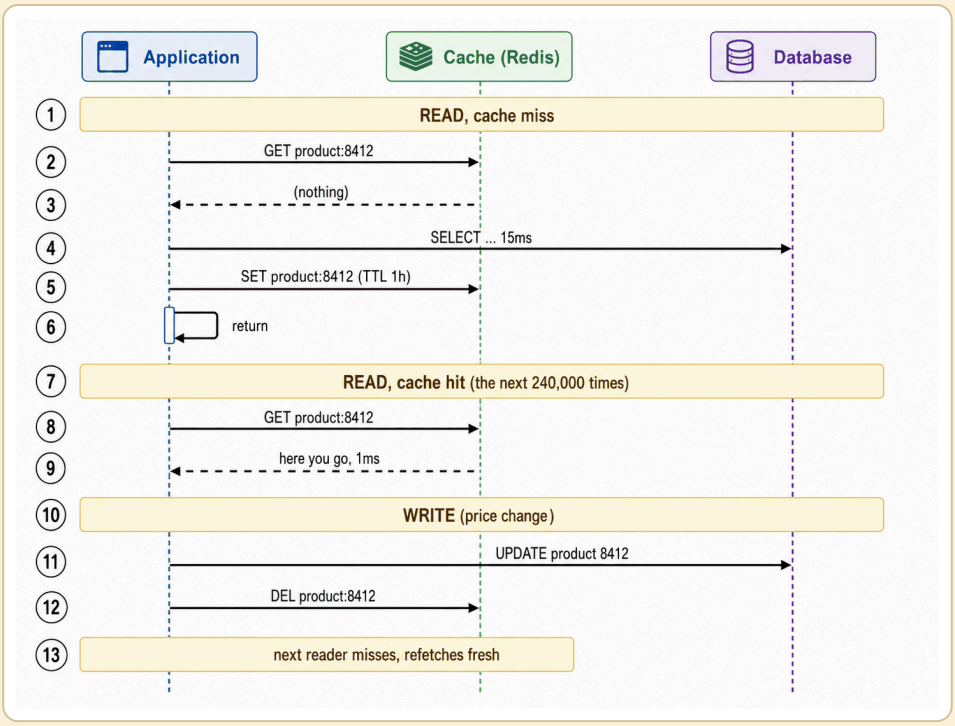


Figure — Cache-Aside (course diagram)

The figure above shows three phases as a numbered sequence between the Application, the Cache (Redis) and the Database. **(1–6) A read that misses:** `GET product:8412` comes back empty, the app runs a `SELECT` (15 ms; `SELECT` is the SQL command that reads rows, SQL being the query language of relational databases, and `UPDATE` is its command for changing rows), then does `SET product:8412` with a 1-hour TTL, and returns. **(7–9) A read that hits:** for the next ~240,000 reads, `GET` returns the value in 1 ms. **(10–13) A write:** a price change runs `UPDATE product 8412` on the database, then `DEL product:8412`. So the next reader misses and fetches fresh data.

The cache itself is kept simple on purpose. **Redis** and **Memcached** (both in-memory key-value servers) are the standard tools: a fast store that replies "here it is" or "not here". Every decision lives in the application code. That gives the pattern its flexibility, and also its maintenance cost.

1. **Read: look in the cache.** If it's a hit, return straight away.
2. **Read miss: get it from the database**, since the cache doesn't have it.
3. **Save a copy with a TTL, then return it.** This is called **lazy loading**: only data that someone actually asked for gets cached. Item 8412 ends up in the cache *because* it's read 4,000 times a minute. The **long tail** that nobody requests (the many items that are rarely read) uses no memory. So the cache naturally fills up with **hot data** (the data read most often). The downside: the first reader of any item takes the slow path. And when many readers ask for the same missing item at the same moment, you get the stampede problem (a later lesson).
4. **Write: update the database, then delete the cached key.** Don't update the cached copy in place. Deleting is safe and lazy, because the next reader fetches the true value. Updating the copy directly opens up races where an older value can overwrite a newer one (see the comparison below).
5. **Put a TTL on everything, even when you also invalidate.** Delete-on-write only keeps data correct if *every* code path remembers to delete. Sooner or later, a bulk import, a manual database fix or a rushed admin tool won't. The TTL caps how long that kind of mistake can last. **Invalidation gives**

you freshness; the TTL gives you insurance. Important data should never depend on a TTL alone, and no data should depend on invalidation alone.

6. **Eviction deals with overflow.** When the cache is full, LRU removes the item used least recently. An evicted key just turns into a miss later, which is harmless by design.

Walkthrough with numbers

Redis cache-aside on the product page, with a 1-hour TTL and a delete on every write:

- **The first read of 8412 is a miss:** 1 ms cache check + 15 ms database + 1 ms SET = **17 ms**. That's a little *worse* than having no cache. It's the honest cost: misses pay for the cache check as well.
- **The next ~240,000 reads that hour: 1 ms** each. The catalog's hit ratio settles at about **99.2%**. So the average latency = 0.992 × 1 ms + 0.008 × 17 ms ≈ **1.1 ms** (down from 15 ms).
- **Read load on the database:** it drops from **12,000/s to about 150/s**, which are the real misses. **Six of the eight replicas** are shut down by the end of the quarter. RAM (fast working memory, where caches keep their data) is cheap; database machines are not.
- **A price change on 8412:** `UPDATE` the database, then `DEL product:8412`. The next reader misses and fills the cache again. The window of staleness is a few **milliseconds**. With a TTL alone, the old price could have stayed for up to a whole **hour**.
- **The forgotten-delete drill:** three months later, a bulk import updates **2,000 products** without invalidating. Those stale copies are served for up to one TTL, then fix themselves. That's annoying, but limited and survivable. It's exactly what the TTL is there for.

Delete vs. update on write (the comparison)

You've changed a price in the database. What should happen to the cached entry? Interviewers ask this a lot.

	DEL the key	**SET the new value**
Next read	One extra miss (fetch again from the DB)	An immediate hit that is already fresh
Race safety	Safe: the true value is fetched after all writes have settled	Risky: two writes at once can SET in the wrong order and leave the older value cached until the TTL
Wasted work	None, if nobody reads it again	Stores values that might never be read
Complexity	One line that's hard to get wrong	You must serialize (force one-at-a-time) or version updates to stay correct

The race, step by step: write A sets the price to $45 and write B sets it to $40 at almost the same moment. Suppose A is the *last* to commit to the database, so the truth is $45 — but the two cache updates can arrive in the reverse order, so B's `SET 40` lands after A's `SET 45`. Now the cache says $40 and the database says $45, and it stays wrong for up to a full TTL. With `DEL`, the worst possible ordering is just two deletes, and the next reader fetches the true $45 from the database. A single extra miss wipes out a whole category of wrong answers.

DEL still has one uncommon race. A slow reader that fetched the *old* value before the write can put it back into the cache right after the delete. The short TTL limits that case as well, which is one more reason to keep it.

Trade-offs

You gain:

- Big relief on reads: about **two orders of magnitude** (roughly 100×) less read load on the database, in exchange for one small service and a few GB of RAM.
- Latency that users can feel: **15 ms → 1 ms** on the hottest pages.
- Full control: your code chooses, for each data type, what to cache (rows, whole pages, computed results), how to key it, and how long to trust it.

- Resilience: if Redis dies, every read becomes a miss. That's slow but *correct*, because the database is still the truth. (Whether the database survives the sudden full load is the topic of the stampede lesson.)

You pay:

- **Staleness you have to manage.** Between a change and its delete or expiry, readers see the old value. It's limited, but it's a real property you choose for each data type.
- **The consistency chore.** Every write path you have now *and every future one* has to invalidate. If one forgets, you get admin-panel edits that show up instantly while bulk imports stay stale (see self-check Q3).
- **Boilerplate.** Every data-access function repeats check → fetch → set → return. So teams pull it into helpers, and the next lesson turns that helper into a pattern.
- **Misses cost more.** Each miss pays for the cache check *and* the database. That's fine at 99% hits. At **~60%**, you add a hop while saving very little work.
- **Cold starts and hot expiries.** A cache that restarts empty, or a popular key that expires at peak, sends crowds to the database (solved in the stampede lesson).

When not to use it

- **Little repetition** — data that's read about once for each write can't produce useful hits (per-user notifications, one-time tokens).
- **No tolerance for staleness at all** — the last unit of stock at checkout, or the execution price of a trade. Read these from the source of truth. (Browsing beforehand can usually tolerate seconds.)
- **The database's own memory already has it** — small data sets often already live in the database's built-in **buffer cache**. If a query takes 15 ms against data that's in memory, it's probably missing an **index**. Fix the query first. (A cache in front of a bad query still pays the full cost on every miss.)
- **Data that is state, not a copy** — sessions and shopping carts need a durable primary home. If they live only in a cache that evicts, carts can disappear.
- **Common misuse:** caching everything out of habit with a 24-hour TTL, no invalidation and no hit-ratio monitoring. The result is stale data and mediocre performance, and nobody measures either.

Real-world examples

- **Redis and Memcached** — the industry-standard tools. "**Look-aside caching**" is another name for this pattern.
- **Facebook's memcache paper** — cache-aside at huge scale, including fixes for the races and request surges covered in this module.
- **Framework helpers** — Rails' `Rails.cache.fetch`, Django's cache API and Spring's `@Cacheable` (Rails, Django and Spring are popular web frameworks for Ruby, Python and Java) all implement cache-aside, so you don't have to repeat the code.
- **Your own hardware** — the database buffer pool, the operating system's **page cache** (recently read disk blocks kept in RAM), and the CPU's **L1/L2/L3** caches (tiny, very fast memories built into the processor) all do the same thing. They check a fast copy, fetch from slower storage on a miss, and keep the result. It's one of the most common patterns in computing.
- **For AI engineers:** exact-match caching works well for embeddings of strings that come up again and again (search queries, product titles), for **moderation** decisions (checks for harmful content), and for **feature vectors** (the lists of input numbers a model receives) at serving time (the online layer of a feature store often uses it). Every cached embedding saves a paid API call. LLM answers are harder. Users rarely repeat a question byte for byte, so exact keys miss. A **semantic cache** (later in the course) matches by meaning instead.

In the interview

When a product page is slow or a database is buried in reads, cache-aside is usually your first move. The real conversation is about where its correctness ends: how it stays consistent, which TTL to use, and what happens after a restart.

> **30-second answer:** With cache-aside, the app checks Redis first and serves hits in a millisecond. On a miss, it reads the database and fills the cache with a TTL. On writes, I update the database and *delete* the cached key instead of updating it. Concurrent updates can race and leave an older value stuck in the cache, while a delete makes the next reader fetch the truth. Every entry keeps a TTL as a safety net for any write path that forgets to invalidate. The TTL is sized per data type: an hour for descriptions, minutes for prices, seconds or no cache at all for inventory. And I track the hit ratio. Below ninety-something percent, the cache barely earns its keep.

Likely follow-ups (with answers):

- *"Why delete instead of updating the cache with the new value?"* — Two writes at almost the same time can reach the cache in a different order than they committed to the database. That leaves the older value stuck until it expires. Deleting shrinks the worst case to a single extra miss. The one rare race left, where a reader puts an old value back, is limited by the TTL.
- *"What TTL would you pick?"* — No single global value makes sense, because the cost of staleness depends on the data type. Descriptions: ~1 hour. Prices: minutes, with invalidation doing the real work. Inventory: seconds, or no cache at all, and the purchase itself checks the database.

Mistake that fails candidates: updating the cache on write without knowing about the race. Caching without *both* a TTL and an invalidation plan. Or putting a cache in front of a **900 ms** query instead of looking for the missing index.

Self-check

- **Q1. Walk the miss path and the write path in order, and say who does each step.** → Miss: the *application* asks the cache, queries the database, SETs the result with a TTL, and returns it. Write: the *application* updates the database, then deletes the key. The next reader then follows the miss path. The cache never talks to the database, and that's what "aside" means.
- **Q2. Pick TTLs (and invalidation) for product descriptions, prices and live inventory, justified by the cost of being wrong.** → Descriptions: a 1-hour TTL + invalidation on edit (staleness costs little). Prices: invalidation as the main mechanism + a 5-minute TTL as a backstop (wrong prices cost revenue and trust). Displayed inventory: a 5–30 s TTL with no invalidation, but the actual purchase reads the database. Each TTL reflects what it costs when that data type is wrong.
- **Q3. Admin-panel edits show up immediately, but nightly bulk-import changes take "up to an hour, randomly". No errors. Diagnose, fix, and name what prevented worse.** → The admin panel deletes cache keys. The bulk import writes straight to the database and doesn't invalidate. So every stale copy lasts until its own TTL runs out, which is why it's "random, up to an hour". Fix: make the import delete every key it affects (or bump a **version** built into the keys). Even better, send all writes through one data-access layer, so no future path can forget. The TTL is what kept the 2,000 products from staying stale forever.

Related patterns

- **Read-Through** — the same steps, but the fetch-and-fill moves out of the app and into the cache layer.
- **Cache Stampede Prevention** — protects against many requests missing the same popular key at once.

- **Consistent Hashing** — spreads keys over a cache cluster and prevents miss storms when the cluster changes size.
- **CQRS** — a cache is a small, short-lived read model, with the same questions about truth, derived copies and lag.

TL;DR

- **Problem:** the database answers the same unchanged questions thousands of times a minute, paying database cost and latency for nothing but repetition.
- **Mechanism:** check the cache. On a miss, read the DB and store the result with a TTL. On a write, update the DB and delete the key. Use TTLs everywhere as insurance, and LRU for overflow.
- **Costs:** limited staleness, a permanent invalidation chore on every write path, boilerplate, misses that cost double, and crowd behavior on cold starts and hot expiries.
- **Skip it when:** data hardly repeats, staleness can't be tolerated (read the truth), or the slow query really just needs an index.

Read-Through

Original lesson: Read-Through on DesignGurus ↗

What it is. The application only ever talks to the cache, by calling `get(key)`. On a miss, the **cache itself** gets the data from the database, saves a copy, and returns it. It does this through a configured **loader**: a function that knows how to fetch something, for example a product by its key. In short, it's cache-aside with the job of fetching moved out of your code and into the cache layer.

Analogy. With cache-aside, every employee phones the warehouse whenever the shelf is empty. With read-through, there's one shelf-stocker instead. You simply ask the shelf. If it's empty, the stocker goes and gets the item. And if twenty people ask at the same time, the stocker makes just *one* trip.

The problem it solves

Cache-aside left every data-access function repeating the same check → fetch → set → return steps. Across many functions and services, some skip a step (for example the SET), some behave a little differently, and some use keys that don't match. Also, each caller runs its own miss logic, so callers can't coordinate with each other. When a popular key is missing, every caller goes to the database on its own.

How it works

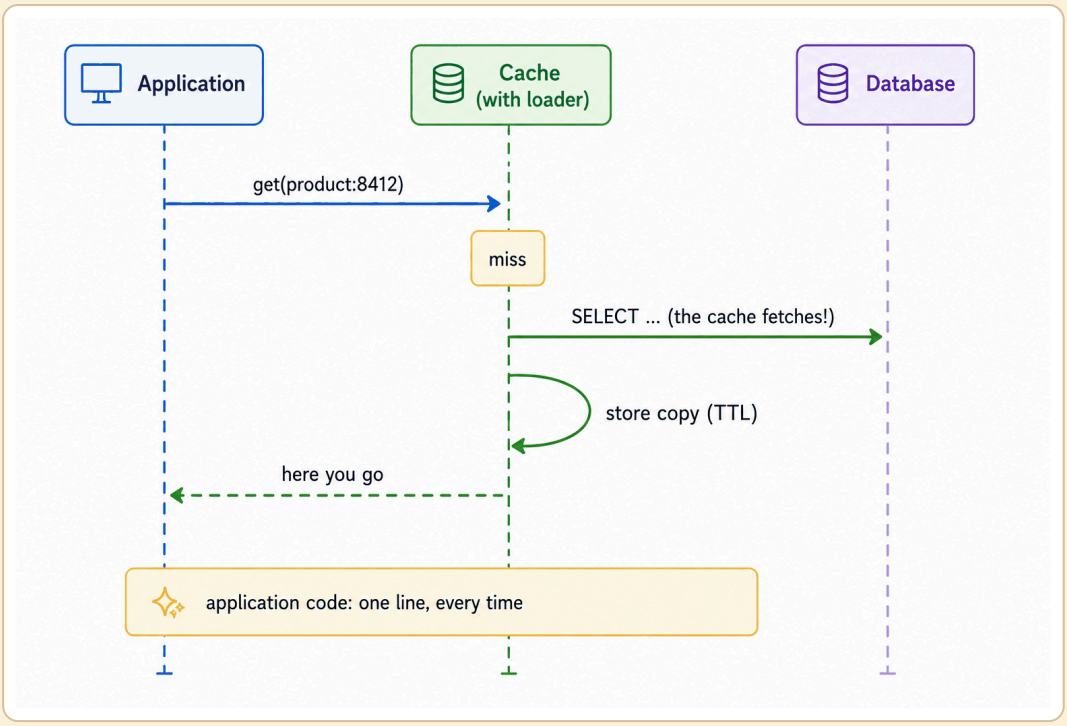


Figure — Read-Through (course diagram)

In the figure above, the application calls `get(product:8412)`. The cache (which has a loader) finds a **miss**, runs the `SELECT` by itself ("the cache fetches!"), saves a copy with a TTL, and replies "here you go". The note at the bottom makes the key point: the application code is **one line, every time**.

1. **The app calls `get(key)` on the cache.** There's no cache-checking or database logic in the application code.
2. **On a hit, the cache returns the value.**
3. **On a miss, the cache runs its loader.** It fetches from the database, saves the result with a TTL, and returns it. These are the same miss steps as cache-aside, just with a different owner.
4. **Request coalescing comes for free**, because the cache sees every reader. With cache-aside, **5,000** misses at the same moment on one key mean 5,000 copies of app logic, each querying the database. A loading cache lets the *first* caller do the fetch while the other **4,999** wait a moment for the same answer. Many simultaneous misses on one key → **one** database fetch. This is the first defense against the module's second fact (caches fail as crowds), which the stampede lesson develops fully.
5. **Writes aren't handled.** Read-through only deals with reads. So changes still need delete-on-write invalidation plus TTLs. The other option is to pair it with write-through (next lesson), which moves writes into the cache layer too.

Where you'll run into it: loading-cache libraries like **Caffeine** and **Guava's** `LoadingCache` (both Java libraries), **ORM** second-level caches like **Hibernate**'s (an ORM, object-relational mapper, is a library that maps database tables to objects in your code), and managed front ends like **DynamoDB DAX**, a caching service that knows how to reach the table behind it.

Read-through vs. cache-aside (the comparison)

	Cache-Aside	Read-Through
Who fetches on a miss	Your application code	The cache layer, using its loader
Miss logic lives	Inside every data-access function	In one configured place
Concurrent misses on one key	N separate database queries	Can be merged into 1 (the main benefit)
Can cache computed, multi-source results	Yes, since the app can cache anything	Awkward, since a loader fetches one kind of thing by key
Cache infrastructure	Any simple cache works (Memcached)	Needs a smart cache: a library, an ORM layer, or a service
Consistency across teams/services	Every team rebuilds the procedure	One implementation, used the same way everywhere

Trade-offs

You gain:

- A single implementation of the miss path. No forgotten SETs, and no forty slightly different versions.
- Request coalescing. Simultaneous misses on a **hot key** (one that many readers want at the same time) shrink to one database query.
- Cleaner application code. Data access shows what you want (`get`), not how it's done.

You pay:

- **Flexibility.** A loader fetches one kind of object by key. A dashboard or page fragment built from three sources doesn't fit, while cache-aside can cache anything the app computes.
- **Smarter infrastructure.** The cache layer has to run fetch logic with access to the database. That means an **in-process** library (one running inside your application's own program), an ORM or a caching service. Plain Memcached can't do it.
- **Coupling.** Queries and access rules now live in the cache layer too, so **schema** changes (changes to the database's table structure) can spill over into it.
- **Writes are still your job.** You have the same invalidation duties and TTL insurance as with cache-aside.

When not to use it

- **Computed and composite results.** The app knows how to put several sources together; a per-key loader doesn't. Use cache-aside.
- **Your cache is meant to be a simple shared store.** A shared Memcached fleet has no place to run a loader. Adding that intelligence might cost more than the boilerplate it saves.
- **One small codebase.** A single service with a well-built `fetch_with_cache()` helper already gets most of the benefit.
- **Where it shines:** many services, teams and languages all reading the same data. At that scale, separate implementations regularly lead to forgotten SETs, TTLs that don't match, and keys that don't line up. One shared read-through layer prevents all of that by design.

In the interview

It's often asked directly: "What's the difference between cache-aside and read-through?" A strong answer explains *who owns what* and what follows from that, not just two diagrams.

30-second answer: It's the same miss path with a different owner. In cache-aside, my application checks the cache and does the fetch-and-fill itself. That's as flexible as it gets, but it's repeated everywhere, and someone always forgets a step. In read-through, the app only talks to the cache, and the cache fetches through a configured loader on a miss. There's one implementation. And because the cache sees every reader, it can merge simultaneous misses on a hot key into one database query, which is real protection when a popular key expires. The trade-off: loaders fetch one kind of thing by key, so computed results still call for cache-aside. And in both cases, writes need their own invalidation plan.

Likely follow-ups (with answers):

- *"Which for a product catalog read by forty **microservices** (small, separately deployed services)?"* — Read-through, as a shared caching service or layer. Every consumer gets one loader, one TTL policy, the same behavior and coalescing. The alternative is forty teams building forty slightly different procedures.
- *"Does read-through fix the stale-after-bulk-import bug from cache-aside?"* — No. That failure happened on the *write* path, and read-through only controls reads. It still needs careful invalidation or a pattern on the write side.

Mistake that fails candidates: treating the two as rivals that give different results, or claiming read-through solves invalidation. When they succeed, their read behavior is exactly the same. Only the ownership, and what follows from it, is different.

Self-check

- **Q1. Who fetches after a miss in each pattern, and what new capability does centralizing it enable?** → In cache-aside, the application fetches. In read-through, the loader inside the cache layer does. Putting it in one place makes coordination possible (merging simultaneous requests for one key into one database query), and gives you one uniform implementation.
- **Q2. Cache (a) product records read by 40 services in 6 languages, and (b) a per-seller dashboard computed from three databases. Which pattern for each?** → (a) Read-through as a shared layer: one loader, one consistent policy, and coalescing that protects the shared database. (b) Cache-aside: the application has to combine three sources before saving the finished result with a TTL and invalidation rules.
- **Q3. A popular key expires at peak and 5,000 requests miss within 50 ms under plain cache-aside. Database load? With coalescing? What lesson does this lead to?** → Cache-aside sends **5,000 identical queries**. A loading cache that coalesces sends **one**, while 4,999 requests wait a moment for its result. A spike becomes one normal query. This shows that caches fail as crowds, which Cache Stampede Prevention handles at a larger scale.

Related patterns

- **Cache-Aside** — the same behavior with the work handed out differently. Its rules about TTLs, delete-on-write and hit ratio all still hold.
- **Write-Through** — the matching pattern on the write side. It sends writes through the cache as well.
- **Cache Stampede Prevention** — extends request coalescing and adds other defenses against misses that happen together.

TL;DR

- **Problem:** cache-aside's miss steps get repeated in every function and service. That leads to forgotten steps and inconsistent behavior, and readers who don't coordinate stampede the database on shared misses.

- **Mechanism:** the app only calls the cache. On a miss, the cache fetches through a configured loader, saves the result and returns it. That's one implementation, and it can merge simultaneous misses into one fetch.
- **Costs:** loaders fetch one thing by key (computed results stay with cache-aside), the cache needs intelligence and database access, there's coupling, and writes still need invalidation.
- **Skip it when:** you're caching composite/computed results, your cache is a simple shared store, or a helper in one small codebase already does the job.

Write-Through

Original lesson: Write-Through on DesignGurus ↗

What it is. Writes pass *through* the cache. The application sends a write to the cache layer. The cache layer writes to the database **synchronously** (the caller waits for it), then updates its own copy. The caller hears "done" only after both the original and the copy are updated. So for data written this way, the cache is never stale, because it was part of the write.

Analogy. Think of a secretary. When you dictate a change to the official register, they write it in the register first. Then they update the quick-reference card on the front desk. Only then do they say "done". Anyone who looks at the card afterward sees the new value.

The problem it solves

Cache-aside and read-through make the *application* responsible for deleting keys after writes, with TTLs as insurance. That depends on every write path remembering the chore (the forgotten-delete type of bug). It also costs an extra miss at exactly the moment a user reads back what they just saved. Write-through makes cache upkeep **structural**. The cache *is* the write path, so it can't forget the update. It's the natural partner of read-through. With both, reads and writes all pass through the cache, which becomes the single way into the data rather than a copy kept up beside the application. Both need the same smart infrastructure: a loading cache that can also write, an ORM layer, or a service like DAX.

How it works

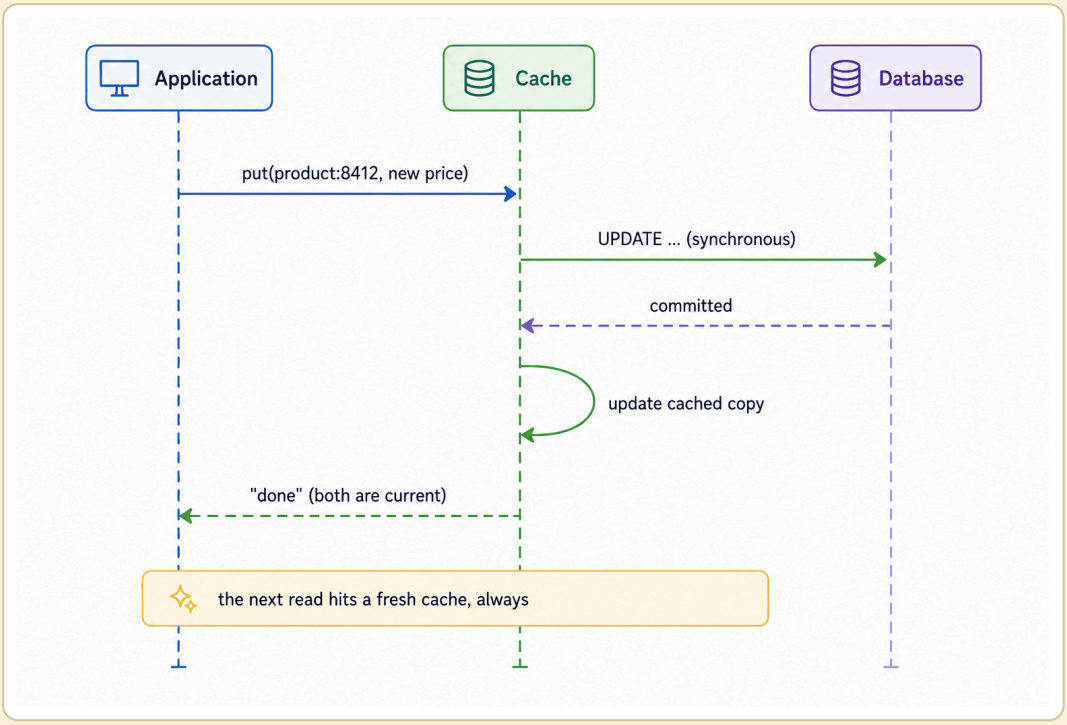


Figure — Write-Through Cache (course diagram)

In the figure above, the application calls `put(product:8412, new price)` on the cache. The cache sends a **synchronous** `UPDATE` to the database and waits for "committed". Then it updates its cached copy, and only after that does it return "done" (both copies are current). The note at the bottom says the next read always hits a fresh cache.

1. **The app sends the write to the cache layer**, not to the database.
2. **The cache writes to the database first, synchronously.** The order matters. The database is the truth, so it must succeed before the copy claims anything.
3. **If the database write fails, the cache stores nothing and passes the error back.** The copy never shows a value the original doesn't have.
4. **If the database write succeeds, the cache updates its own copy, then confirms.**
5. **If the database write succeeds but the cache update fails, invalidate (delete) the entry and still report success.** The commit is safe. The next read just misses and fetches the true value. That's one slower read, and it's still correct.
6. **The cache stays fresh and warm.** Cache-aside deletes after a write. Write-through *keeps* the new value instead, so the read that usually comes right after a user's edit is an immediate hit.
7. **The guarantee has a limit.** The cache is fresh only for writes that go *through* it. A direct database write from another service, a bulk import or a manual fix skips the cache and brings back the forgotten-delete problem. So either every writer uses the cache, or every entry keeps TTL insurance.

Write-through vs. delete-on-write (the comparison)

	Cache-aside + DEL	Write-Through
After a write, the cache is	Empty for that key (the next read fills it)	Already fresh
Read-after-write	One extra miss	An immediate hit
Write latency	Only the database write	Database write + cache update (a little slower)
Caches data nobody reads?	Never	Yes, since every write goes into the cache whether it's read or not
Requires	Any simple cache	A smart cache layer sitting in the write path

Trade-offs

You gain:

- **Structural freshness.** There's no invalidation chore and no forgotten-DEL bugs for anything written through the cache.
- **Read-after-write that hits.** Save-then-view (most user edits) is served from the cache right away.
- Together with read-through, **one coherent data layer**, instead of a cache maintained alongside the code.

You pay:

- **Write latency.** Every write makes two updates, one after the other, before confirming. The extra millisecond or two matters on paths where latency is critical.
- **Cache churn.** Every key that gets written enters the cache, including values nobody will read. In a write-heavy workload, they can push truly hot entries out of limited memory.
- **Discipline at the boundary.** Writes that skip the cache quietly bring staleness back.
- **Smart infrastructure.** A basic key-value store can't run this sequence.

When not to use it

- **Data that's written a lot and read rarely** — logs, metrics, audit records, click activity. Caching every record gains nothing and pushes out useful entries. Write these straight to storage.
- **Writers that bypass the cache are a fact of life** — if three other systems update the same tables directly, the freshness promise doesn't really exist. Cache-aside + TTL describes the real guarantee more honestly.
- **Writes where latency is critical** — some operations can't afford the extra synchronous step. (Write-behind might fit, but only after you weigh its durability cost.)
- **Common misuse:** adding write-through on top of the old invalidation code and then removing every TTL. Nobody can tell which mechanism actually keeps things correct. Pick **one** main mechanism, document it, and keep TTLs as insurance.

Real-world examples

The course doesn't give a separate list for this lesson. The implementations are the same smart-cache infrastructure as for read-through: a **loading cache with a writer**, an **ORM layer**, or a **DAX-style** caching service sitting in front of the database.

In the interview

It usually comes up when comparing cache-aside, write-through and write-behind. It also comes up with "users edit a profile and must immediately see the new value", because read-after-write is its biggest strength.

> **30-second answer:** With write-through, writes go through the cache. The cache updates the database synchronously, then its own copy, and only then confirms. So the cache is never stale for anything written through it, and the read-after-write that follows most user edits hits a warm, fresh cache. The costs: writes pay for both updates, and every write goes into the cache whether or not anyone reads it. The guarantee only covers writes that pass through the cache, so writers that bypass it still need TTL insurance. It fits naturally with read-through, which makes the cache the one entry point to the data layer.

Likely follow-ups (with answers):

- *"The database write succeeds but the cache update fails. Now what?"* — Invalidate the entry and return success. The database commit is safe, and the next read fetches the true value. You accept one slower read rather than serving a wrong copy.
- *"Why not always use write-through instead of cache-aside's delete?"* — It needs a smart cache in the write path, it stores values that may never be read, and it loses its guarantee whenever any writer goes around it. Delete-on-write is one simple operation on basic infrastructure. The choice is stronger structure versus more simplicity and flexibility.

Mistake that fails candidates: claiming write-through means you don't need TTLs even though writers that bypass it exist. Or being unable to say what happens when either of the two updates fails.

Self-check

- **Q1. Describe the write sequence including both failure cases, and state exactly which writes get the freshness guarantee.** → App → cache → database updated synchronously → cached copy updated → confirm. If the DB fails: cache nothing and return the error. If the DB commits but the cache update fails: invalidate the entry and confirm, so the next read fetches the truth. Only writes that go through this whole sequence are guaranteed fresh.

- **Q2. Choose behavior for (a) users editing profiles they view right away, and (b) an activity log written on every click but almost never read.** → (a) Write-through (with read-through): structural freshness, and the read-back hits a warm cache. (b) Don't cache the write at all. A record nobody reads only wastes cache memory. If recent logs become popular later, cache-aside can add them when they're requested.
- **Q3. Six months after adopting write-through and removing TTLs, customers see stale prices only for products changed by a new bulk-pricing tool. Diagnose and fix.** → The tool writes to the database directly and skips the cache. So write-through never refreshes those copies. With the TTLs gone there's no backstop, so they stay stale until they're evicted. Fix: send the tool through the cache's write path *and* bring the TTLs back. Every correctness guarantee has a boundary, and forgotten writers live outside it.

Related patterns

- **Read-Through** — the natural partner. Together they make the cache the single entry point for reads and writes.
- **Write-Behind** — the next lesson. Same entry point, but the database update is delayed, trading durability for speed.
- **Cache-Aside** — the simpler option (delete-on-write + TTLs) for basic infrastructure and for systems with independent writers.

TL;DR

- **Problem:** delete-on-write relies on every write path remembering the chore, and it costs an extra miss right when users read back what they saved.
- **Mechanism:** writes go through the cache. The database is updated synchronously first, then the copy, then the confirmation. If only part of it fails, invalidate and stay correct.
- **Costs:** extra write latency, churn from writes nobody reads, smart infrastructure in the write path, and a guarantee that ends at the cache's boundary.
- **Skip it when:** data is written much more than it's read, writers that bypass the cache can't be avoided, or write latency is precious.

Write-Behind

Original lesson: Write-Behind on DesignGurus ↗

What it is. Writes land in the cache, and the caller gets an **immediate acknowledgment** (it's told "done"). The database finds out *later*. A background **flusher** sweeps up the changed entries every few seconds and writes them to the database in efficient batches, folding many small changes into a few big ones. Another name for it is *write-back*.

Analogy. Picture a tally counter in a shop that clicks up for every customer. Nobody phones head office on each click. Every couple of seconds, someone reads the counter and sends a single message: "+24,000". If the shop loses power between messages, the clicks since the last message are gone.

The problem

The product team wants to show "12,483 people viewed this today" on every product page. The straightforward way is to run `UPDATE products SET views = views + 1` on every view. At peak, the bestseller gets **12,000 views/s**, so **12,000 UPDATEs per second hit one row**. A database guards a row that's being changed with a **lock**. So these updates line up in a queue, each one waiting for the one in front of it. Update latency goes past **one second**. Waiting connections use up capacity that important work needs. And checkout slows down, because counting views is hogging the database.

Now look at what's really required. 12,000 times a second, the system durably records a fact before confirming it, and the whole fact is "+1". Nobody needs each increment on permanent storage right away. The displayed number changes all the time anyway, and a 2-second delay is invisible. Still, each +1 pays for a lock, a write, and an entry in the **WAL** (write-ahead log). The database could get one summary fact every two seconds instead: "+24,000 for the last two seconds".

Why the obvious fixes fail

- **"Write-through."** It keeps the cache fresh, but it still sends *every* write synchronously to the database. That's still 12,000 row locks per second, plus a cache update on top. Write-through fixes staleness. This incident is about write *volume*.
- **"Sample it: count every 10th view."** That's fine for rough analytics, but it's only an estimate. Small counts come out wrong (7 views might show as 0 or 10). And it stops being acceptable once badges, rankings or payouts depend on the number.
- **"Put the writes on a queue."** Put each view on a queue and have a worker batch the database updates. That's a valid Module 2 answer and the closest alternative. But the page has to *show* the current count, and a queue can't serve reads. Add a cache next to the queue, and you've built most of this pattern (see the comparison).

How it works

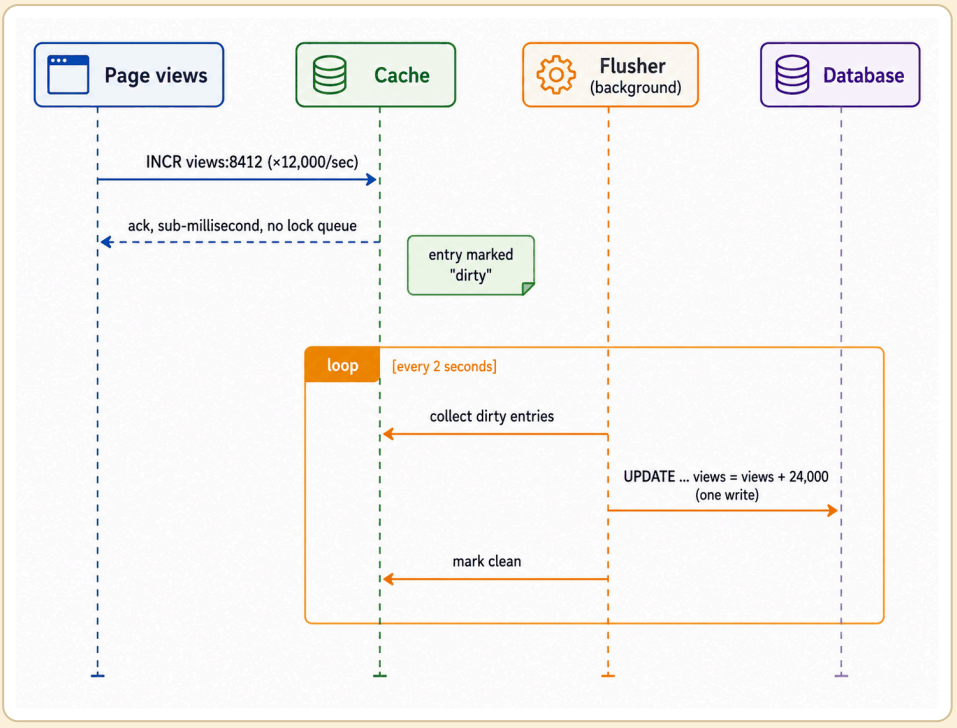


Figure — Write-Behind Cache (course diagram)

In the figure above, page views send `INCR views:8412` to the cache 12,000 times per second. Each one gets an ack in under a millisecond, with no lock queue, and the entry is marked **"dirty"**. In a loop every 2 seconds, the background flusher gathers the dirty entries, sends one `UPDATE … views = views + 24,000` to the database, and marks the entry clean.

1. **The write lands in the cache and is confirmed right away.**
2. **Dirty tracking.** The cache keeps track of which entries have changes that haven't reached the database yet. These entries are **dirty**. A dirty entry must **never be evicted** before it's flushed. Evicting it wouldn't just free memory; it would delete data that was already confirmed. So implementations **pin** dirty entries (mark them as not allowed to be evicted) until they're clean.
3. **The flusher coalesces. Coalescing** = merging many changes to one key into a single database write. Counter increments get added up (12,000 × "+1" → one "+24,000" per flush). Repeated updates to a field keep only the last value (five edits to a product within two seconds → one write of the final state). Database work now grows with the number of **distinct keys changed**, not with the total write rate. That's a much better growth curve.

4. **Flush policy:** flush every N seconds *or* every N dirty entries, whichever happens first. A shorter interval puts less data at risk but batches less. A longer one is more efficient but puts more at risk.
5. **Flush failures need a plan.** If the database is down for a moment, the flusher holds on to the dirty batch and tries again. An entry that fails over and over goes into a **quarantine** (a side area, like a message queue's dead-letter queue), so one bad record can't block every flush after it.

The cost, said plainly — this is the most dangerous pattern in the module:

- **"Acknowledged" doesn't mean "saved" anymore.** Until the next flush, confirmed changes exist only in cache memory. The WAL lesson's promise (fsync — force to disk — *before* acknowledging) is given up on purpose, in exchange for speed and less database load. If a cache node fails, every dirty change from one flush interval can be lost. So every use of this pattern has to finish this sentence, in writing: **"if we lose the last N seconds of these writes, then ___"**. And the answer has to be truly acceptable.
- **The truth flips.** During the dirty interval, the *cache* has the current value and the *database* has an older one. Anyone who needs the live value must read the cache. Reports or services that read the database directly see the past. For that stretch, the cache is the authoritative source, not a disposable copy.

Walkthrough with numbers

The view counter, rebuilt with a **2-second** flush:

- **Write path:** `INCR views:8412` in Redis completes in **under 1 ms**. There's no database lock and no queue. The page reads the same key, so it always shows the current count. 12,000 in-memory increments per second are easy work for Redis.
- **Database load:** the hot key now produces one `UPDATE … + 24,000` every 2 seconds. That's a drop from **12,000 writes/s to 0.5/s**, a **24,000×** reduction. Across the catalog, each flush folds thousands of dirty keys into a handful of bulk statements, which hands database capacity back to checkout.
- **Crash drill:** a cache node dies **1.8 s** after its last flush and loses about **21,600** views that weren't flushed. The loss sentence ("if we lose 2 seconds of view counts, the numbers are imperceptibly lower") was written and accepted *before* launch, so this is fine.
- **Making the window smaller, if you need to:** a **Redis replica** keeps the dirty data if one node fails. Redis **AOF** (append-only file, which is basically a write-ahead log owned by the cache) saves it to disk. A shorter flush interval moves you closer to write-through. Each option buys back durability by spending some speed.
- **The proposal to reject in review:** reusing this design for payment records. "If we lose the last 2 seconds of payments, then…" has no safe way to end. Money keeps the WAL's durability promise.

Write-behind vs. queue + batching worker (the comparison)

	Write-Behind	Queue + batching worker
Ack comes from	Cache memory (fast, but volatile: lost if power or the process dies)	The broker, once it has persisted the message
Durability of acknowledged writes	Whatever is in the flush window is at risk	Durable in the broker: nothing confirmed is lost
Coalescing (+1 merging)	Happens naturally, in place	Can be done in the worker, but it's clumsier
Live reads of the current value	The cache serves them, which is the whole point	Not possible; readers see the database's lag
Machinery	Dirty tracking inside the cache	A topic, a consumer and offsets (each reader's saved position in the stream; the usual Module 2 kit)

The question that decides it: does any reader need the hot, current value? The product page shows the live count, so write-behind wins. The cache acts as both the write buffer and the read layer. For a

pure ingestion stream that nobody reads right away (analytics events, telemetry), use a queue and a worker. The broker keeps every confirmed write, and nothing useful is lost. Adding a read cache to the queue, or a persistent log to the cache, pushes each design toward the other. Choose the one whose built-in strengths fit the main workload.

Trade-offs

You gain:

- Write confirmations at cache speed. Row **contention** (many writers fighting over the same row) disappears at its source.
- Orders of magnitude fewer database writes, thanks to batching and coalescing. Load grows with distinct keys, not with the write rate.
- Natural absorption of bursts. Spikes pile up as dirty entries, and the flusher drains them at whatever pace the database can handle (the same shape as the queue lesson).

You pay:

- **The durability window.** Confirmed writes sit in memory until they're flushed, and a crash loses them. This is the main cost, and every deployment needs its consequence written down.
- **Truth inversion.** The cache is the authority during the dirty interval. Anyone reading the database directly quietly sees older values.
- **Real complexity.** Dirty tracking, eviction pinning, retries, quarantine and per-key ordering add up to the heaviest machinery in the module.
- **Harder debugging.** "What was the value at 14:02?" now has two answers (the cache and the database), and incident response has to reconcile them.

When not to use it

- **Money, ledgers, payments, orders, anything related to compliance.** Any chance of a loss window rules it out, no matter the **throughput** (operations handled per second). Writes that matter one by one keep the WAL's promise.
- **Other systems read the database directly** and need current values. Truth inversion breaks them *without any error*, which makes it especially hard to spot.
- **Writes that don't coalesce.** Unique records (new orders, messages) can be batched, but they don't get the huge reduction a hot key gets. A durable queue can batch them without putting confirmed data at risk.
- **Modest volume.** A counter at **50 updates/s** doesn't justify dirty tracking and flushers. An ordinary database handles it fine.
- **Judgment calls:** shopping carts → a durable store (losing a user's choices is harm they can see). Inventory → the displayed badge can be buffered, but the decrement that accepts a sale must be durable. Token-usage metering → buffer the live dashboards, but bill from a durable, reconciled event stream.
- **Risk of adopting it by accident:** some cache products and ORMs offer write-behind as a simple switch. Turning it on for speed without a documented loss window only postpones the decision until a failure makes it for you.

Real-world examples

- **CPU write-back caches** — the name comes from hardware. Processors have made this trade between cache and RAM for decades.
- **Social counters at scale** — views, likes and impressions are buffered in caches and flushed as totals. Locking a row for every reaction would be absurdly expensive.
- **Rate limiters and leaderboards** — Redis counters that serve live reads and reach permanent storage later.

- **EhCache, Hazelcast** (Java caching products) — offer write-behind as a built-in feature. The rule about documenting the loss window still applies, even though it's easy to turn on.
- **For AI engineers:** training metrics and telemetry fit well (buffer, batch, flush; losing ~2 s of **loss-curve** points (the training-error measurements plotted over time) is fine). Online feature stores can buffer updates the same way. Token metering needs care. It's shaped like a counter (which suits write-behind), but it's billing (money). So buffer dashboards aggressively, and compute invoices from a durable, reconciled event stream. Dashboards can lose a moment; invoices can't.

In the interview

A typical prompt: a like button or view counter at large scale. Diagnose **row contention** first, then propose this. Another: the interviewer accepts write-behind, then asks "what happens when a cache node dies?"

> **30-second answer:** Twelve thousand increments per second on one row is a lock-contention problem, so I buffer in the cache. Increments land in Redis in under a millisecond. The page reads the same key, so it's always current. A background flusher merges them into one bulk update every couple of seconds, which cuts database writes by orders of magnitude. The price: confirmed writes live in cache memory until the flush, so a crash loses that window. That's acceptable, and documented, for view counts, but never for money. Also, during the window the cache is the truth, so anything reading the database directly sees the past. If nobody needed the live value, I'd use a durable queue with a batching worker instead.

Likely follow-ups (with answers):

- *"The cache node dies. What exactly is lost, and how would you shrink it?"* — Up to one flush interval of confirmed dirty changes. You can shrink it with a cache replica (survives one node failing), AOF persistence (which has its own fsync policy), or a shorter interval. Each one gives up some performance for durability and moves the design toward write-through.
- *"Would you use this for inventory during the flash sale?"* — Separate the display from the decision. "Only 3 left" can be seconds stale or buffered. But the decrement that accepts a sale matters financially and needs a durable, authoritative path.

Mistake that fails candidates: using write-behind for payments or orders. Ignoring the risk of evicting dirty entries. Or claiming that Redis persistence removes all loss, when AOF's default fsync policy `everysec` still allows about **one second** of loss.

Self-check

- **Q1. Describe the full write path, define "dirty", and say exactly what the acknowledgment promises.** → The cache accepts the write and confirms it right away. It marks the entry dirty (changed, but not in the database yet) and pins it so it can't be evicted. Every so often, the flusher merges the dirty changes, writes them to the database and marks them clean. The ack only promises that the value is in cache memory and *should* reach the database later. It does not promise the value is durably saved.
- **Q2. For view counts, cart contents, payment records and a "trending now" score: write-behind or not, and complete the loss sentence.** → View counts: yes, "…the numbers are imperceptibly lower." Trending score: yes, since it's approximate, derived and can be recomputed. Cart contents: no, use a durable store. Losing user choices is visible harm, and the volume rarely calls for it. Payments: never, because no loss window is acceptable. Whether it fits depends on what losing the dirty window would cause.
- **Q3. A nightly analytics job reading the database reports view counts ~2% below the cache-backed dashboard, and counts dip slightly after every deploy. Explain and fix proportionately.** → The 2%

gap is truth inversion doing what it was designed to do (the database trails the cache's dirty counts). Have analytics read post-flush data labeled "as of" a time, or accept the difference and label it. The dips after deploys happen because restarting cache nodes drop changes that weren't flushed. **Flush dirty entries on graceful shutdown** (a planned stop where the process finishes its work before exiting), and add a replica if the remaining gap matters. Persisting every write would be overkill for views and would basically rebuild write-through.

Related patterns

- **Write-Through** — the safe alternative: update the database synchronously before confirming.
- **Message Queue** — the durable alternative for batching write streams that nobody needs to read live.
- **Write-Ahead Log** — the durability promise that write-behind weakens. It can partly rebuild that promise inside the cache using AOF.
- **Read-Through** — the read side of the smart cache layer. Combined with write-behind, the cache becomes the single entry point.

TL;DR

- **Problem:** frequent writes to hot keys (counters, scores) turn database rows into lock queues. That's thousands of facts per second that don't matter one by one, each paying the full price of a durable write.
- **Mechanism:** writes land in the cache and are confirmed instantly. Dirty entries are pinned and flushed to the database in merged batches every few seconds, so load grows with distinct keys, not with the write rate.
- **Costs:** a durability window (acknowledged ≠ saved), truth inversion (direct DB readers see the past), and the heaviest machinery in the module.
- **Skip it when:** writes involve money or matter one by one, other systems read the DB directly, writes don't coalesce, or volume is modest. Always finish "if we lose the last N seconds, then ___", and mean it.

Cache Stampede Prevention

Original lesson: Cache Stampede Prevention on DesignGurus ↗

What it is. This is a group of techniques for one situation: a popular ("hot") cache entry needs to be rebuilt. They make sure **exactly one request does the rebuilding**, while everyone else waits a moment or receives the slightly old copy. Without them, thousands of requests hit the database at the same time. That crowd of simultaneous misses is called a **cache stampede** (also known as **dog-piling**).

Analogy. When the office coffee pot runs dry, you don't want forty people each brewing a new pot at once. One person makes coffee. Everyone else waits a minute, or drinks the last lukewarm cup while the fresh pot brews.

The problem

During a flash sale, the bestseller's page serves **12,000 requests/s** from the cache with a **99.9%** hit ratio. The database only sees **150 queries/s**. Everything is fine. At **8:00:00 PM**, that product's entry reaches its **one-hour TTL** and expires. This is routine; every key expires eventually. Now watch the next 50 milliseconds:

1. Rebuilding the value takes one database query of about **50 ms**.
2. Every request that arrives during those 50 ms misses as well: **12,000 × 0.05 = 600 simultaneous misses**.
3. With plain cache-aside, each of them runs its own miss path. That's **600 identical queries** against a database that's used to 150/s.

4. The burst slows the query from 50 ms to **500 ms**. That *stretches* the miss window, while traffic keeps coming at 12,000/s. Now **6,000** requests are missing.
5. The database's **connection pool** (its limited set of open connections) fills up. More queries queue behind it. Checkout, which shares the database, stops working. By **8:00:03** a single ordinary expiry has taken the site down.

The damage equals **request rate × rebuild time**. And overload makes the rebuild slower, which makes the crowd bigger. That's a **feedback loop**. It's the module's second fact in action: caches fail as crowds. A cache cluster that restarts **empty** at peak runs the same math on *every* key at the same time.

Why the obvious fixes fail

- **"Longer TTLs."** A 6-hour TTL makes stampedes happen less often, but not smaller. And data stays stale for longer.
- **"Size the database for the burst."** Provisioning for 6,000 simultaneous queries is a huge permanent cost for a **50 ms** event that happens a few times a day. The burst also grows with traffic, so success keeps outgrowing whatever reserve you bought.
- **"Never expire hot keys — refresh everything from a background job."** That works for the hottest keys. But recomputing the whole catalog wastes resources on millions of entries nobody reads. **Refresh-ahead** only pays off for the small set of keys whose traffic justifies it.

Misses aren't the issue. The issue is **600 requests repeating the same work**. The fix is coordination. It's the **request coalescing** idea from read-through (many identical requests → one operation), fully developed here.

How it works

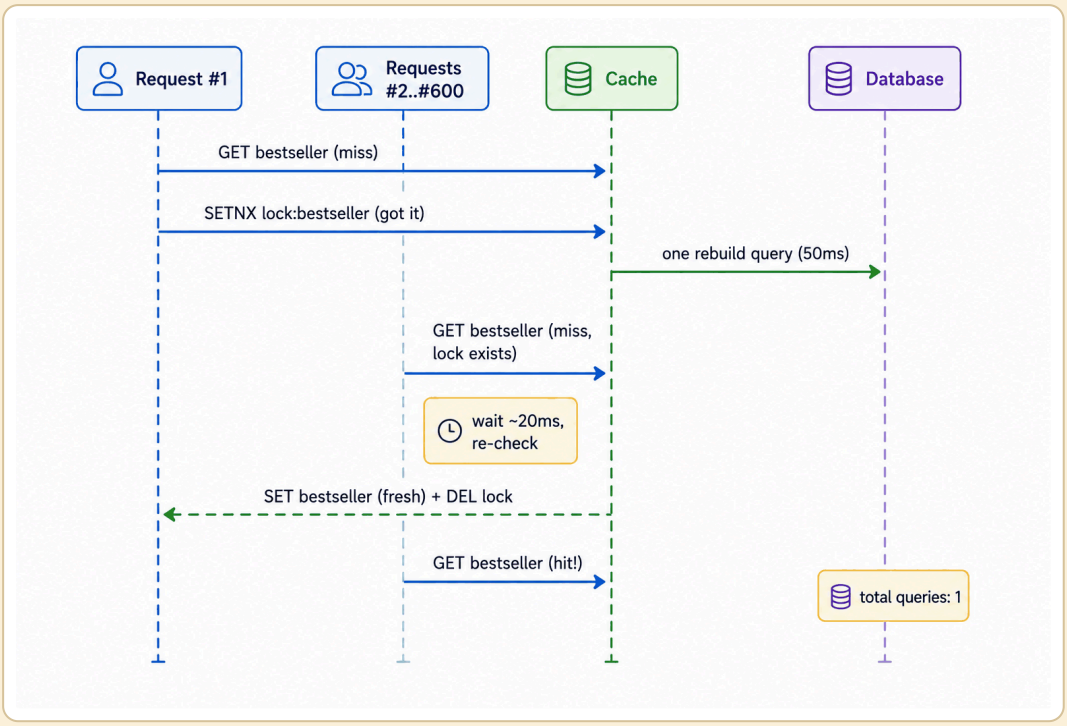


Figure — Cache Stampede Prevention (course diagram)

The figure above shows the lock tool. Request #1 misses the bestseller and takes `SETNX lock:bestseller` ("got it"). The single rebuild query (50 ms) goes to the database. Requests #2..#600 also miss, but they see that the lock exists, wait ~20 ms and check again. Once the fresh value is SET and the lock is deleted, their next `GET` is a hit. Total database queries: **1**.

Three tools protect a single hot key. Two more protect the whole cache fleet. Real production systems combine them.

Tool 1 — The lock ("single flight"). Only one request at a time may rebuild a given key.

1. The first request that misses grabs a short lock for that key. In Redis this is atomic using `SETNX` ("set if not exists"), so only one caller can win.

2. It fetches the value from the database and fills the cache.
3. The other requests see the lock, wait a moment, and check the cache again once the fresh value is there. Result: 600 misses → **1 query + 599 short waits**. The lock needs a **short TTL of its own**, because the request holding it might crash. A lock that never expires turns into an outage by itself.

Tool 2 — Serve stale, refresh behind. If slightly old data is fine, nobody has to wait. Keep entries a little past their *logical* expiry. When an entry has expired, serve the stale copy right away and let **one** background refresh, protected by a lock, fetch the new value. Users get data a few seconds old at full speed, and the database gets one query. (A product page that's 50 ms stale looks exactly like a fresh one.) HTTP makes this standard with the `stale-while-revalidate` **header** (a name-value line of metadata sent with an HTTP request or response).

Tool 3 — Probabilistic early refresh. Stop the hottest keys from ever reaching expiry. As a key gets close to its TTL, each request has a small random chance of refreshing it early, and that chance grows as expiry gets nearer. A key with thousands of readers nearly always gets refreshed before it expires. A cold key with no readers just expires quietly, with no crowd. The randomness automatically points refresh work at the keys whose traffic makes it worth doing.

Fleet tool 1 — TTL jitter. Mass expiry is a stampede spread over *many different* keys. For example, a deploy warms **10,000** entries within a minute, all with TTL **3600**. So all of them expire together an hour later. Add a small random variation, such as **3600 ± 10%**, so the expirations spread out over many minutes. That randomness is called **jitter**.

Fleet tool 2 — A cold-start plan. An empty cluster at peak means a 100% miss ratio at full traffic. Before you let traffic in:

1. **Warm** (pre-load) the most popular keys from an **access log** (a record of which keys were requested) or a snapshot (fewer misses right away).
2. **Ramp traffic up slowly** (the load that's left stays within the database's capacity).
3. Count on **single flight** to limit database work to **one query per distinct key**. That gives operators a measurable upper limit instead of an uncontrolled crowd. Also use **consistent hashing** for the cache cluster, so a normal resize doesn't turn into mass misses the way modulo routing does.

Walkthrough with numbers

Back to 8:00:00 PM, now with serve-stale + a per-key lock turned on:

- The bestseller reaches its logical expiry. The next request finds it stale, takes the lock, starts the rebuild, and serves the stale copy in **1 ms**. The other **599** requests in that 50 ms window also get the stale copy in 1 ms. Nobody waits.
- **Database queries: 1** (instead of 600 spiraling up to 6,000). **Page latency: a flat 1 ms** (instead of jumping to seconds). **Staleness served: 50 ms** worth, which nobody can notice.
- At **8:00:00.050** the fresh value arrives, and the next hour goes just like the last one.
- **Cold-start drill:** a bad deploy restarts the cache cluster at **7:50 PM**. Warming brings back the top **10,000** keys in about **40 s**. The load balancer ramps traffic up over **2 minutes**, and single flight caps the work at one query per key. The database briefly peaks under **2,000 queries/s**. That's unpleasant but survivable, and it recovers within minutes. Without the plan, the same event could snowball into an outage.

The three hot-key tools compared

	Lock (single flight)	Serve stale + refresh	Probabilistic early refresh
During rebuild, others get	A few ms of waiting	The stale copy, right away	Nothing, because expiry rarely comes
Staleness served	None: everyone gets fresh data	One rebuild's worth	Practically none
Complexity	Lock TTLs and logic for waiters	Tracking the stale window	Probability math in the client
Wasted work	None	None	A few refreshes that happen early
Best for	Keys where staleness isn't allowed	The default for data that tolerates staleness	The very hottest keys at the largest scale

Big systems often run **all five** together. They serve stale for ordinary tolerant data, guard every rebuild with a lock, refresh the known hottest keys early, jitter TTLs everywhere, and keep a written cold-start runbook. Each one covers a different gap, and none of them replaces the others.

Trade-offs

You gain:

- **Flat latency through expiries.** The **p99** (the latency that 99% of requests beat) stops spiking every TTL cycle.
- **Database load that follows the number of distinct keys needing a rebuild**, never the request rate. The crowd is capped at 1 by design.
- **Cold starts and deploys you can survive.** That's the difference between a cache that helps and a cache your architecture depends on.

You pay:

- **Limited staleness, served on purpose.** Serve-stale returns old data for one rebuild interval. For many data types that's negligible, but it's a decision per key, not a universal default.
- **Lock complexity and its own class of bugs.** Expiry timers, handling a crashed lock holder, and fallback for waiters. Use proven code (Go's `singleflight`, from the Go programming language's libraries; documented Redis lock recipes) instead of writing your own.
- **Waiting requests tie up resources.** Single flight *without* stale serving makes requests wait a few ms, and that can matter at extreme concurrency.
- **A little wasted work.** Early refresh sometimes refreshes an entry that could have expired without harm, and the cache client gets more complex.
- **Justify it with measurements:** simultaneous misses per key, database queries per rebuild, time spent waiting on locks, and stale responses served. These show whether the protections cut real load and which keys deserve early refresh.

When not to use it

- **Cold and long-tail keys.** A key read once a minute can't have many simultaneous misses, so locks and stale tracking add nothing. Measure simultaneous misses first.
- **Low-traffic systems.** At **50 requests/s**, the biggest possible crowd is too small for the database to notice. **Distributed locking** (a lock shared by many servers, usually kept in Redis) at **~100 req/s** guards against a crowd that never shows up. (The opposite mistake is more common: nobody thinks about simultaneous expiry until a flash sale exposes it.)
- **Staleness truly not allowed.** Then serving stale data is out. Use the single-flight lock so callers wait briefly for fresh data, or don't cache that key at all. (The cache-aside advice to read

critical values from the source still holds.)

Real-world examples

- **Facebook's memcache "leases"** (from the same paper mentioned in cache-aside). The cache gives the first request that misses a token that allows it to rebuild, while the others wait. This is single flight at huge scale.
- `stale-while-revalidate` — made standard in **RFC 5861** (RFCs are the numbered documents that define internet standards). **CDNs** (content delivery networks: servers around the world that keep cached copies near users) and browsers use it to answer right away while refreshing in the background.
- **Go's** `singleflight` **package** — single flight within one process, offered as a small standard building block.
- **The XFetch paper**, "Optimal Probabilistic Cache Stampede Prevention" — the math behind probabilistic early refresh.
- **For AI engineers:** a stampede costs more when the rebuild is a **billable, multi-second LLM generation** rather than a 50 ms query. One popular answer that expires can set off dozens of identical paid generations. Single flight for identical prompts is one of the most valuable protections in an inference stack. It works alongside the later semantic cache (which also catches *similar* requests). Inference cold starts likewise need **model weights** (the learned numbers that make up a trained model) and caches warmed up before traffic is let in gradually.

In the interview

Once you propose caching, expect this: "What happens when your most popular key expires at peak?" Work through the **concurrency** math (how many requests are in progress at the same moment) out loud, then explain how coordinated rebuilds put a ceiling on it.

> **30-second answer:** A hot key that expires under load causes a stampede. At 12,000 requests a second with a 50 ms rebuild, 600 requests miss together and send identical queries, and the slowdown feeds on itself. I stack several defenses. Every rebuild is guarded by a lock, so exactly one request does the work (single flight). Keys that tolerate staleness serve the stale copy while one background refresh runs, so latency stays flat. The hottest keys refresh probabilistically before expiry, so that moment never arrives. Across the fleet, TTL jitter keeps keys that were warmed together from expiring together. And there's a cold-start plan: warm the top keys, ramp traffic, and let single flight cap the database at one query per distinct key.

Likely follow-ups (with answers):

- *"The request holding the rebuild lock crashes. Now what?"* — The lock has a short TTL, so it's released automatically. Waiting requests serve stale data or retry once. A lock with no expiry would turn the protection itself into an outage.
- *"Your whole cache cluster restarts empty at peak. Walk me through it."* — First, avoid cold restarts where you can (replicas, consistent hashing). During recovery, warm the popular keys from an access log, ramp traffic up slowly, and use single flight so the database sees at most one query per distinct key. That limit is the key operational guarantee.

Mistake that fails candidates: saying "on expiry it just goes back to the database" and ignoring simultaneous misses. Suggesting client **retries** (they make the crowd bigger). Or locking every key without a measured traffic threshold.

Self-check

- **Q1. Do the incident's math: why can one key expiring at 12,000 req/s endanger the whole database, and what creates the feedback loop?** → The rebuild takes 50 ms. During that time 12,000 × 0.05 = 600

requests miss and send identical queries. That burst slows the database and stretches the rebuild. At 500 ms there are 6,000 simultaneous queries, which slow things down even more until the shared connections run out. Damage = request rate × rebuild time, and overload makes the rebuild time itself longer.

- **Q2. Pick a tool (or none) for a news site's headline block, a stock-price display, and a retailer's top-100 product pages.** → Headlines: serve stale while refreshing. A few seconds of older news does no harm, and latency stays flat. Stock price: staleness makes the product wrong, so use a lock-guarded refresh with a short wait (if you cache it at all). Top 100 products: probabilistic (or scheduled) early refresh, since with their traffic, expiry should almost never happen. The choice depends on each key's traffic and how much staleness it can tolerate.
- **Q3. Every day at exactly 4:00 AM the database has a ten-minute load spike; the team deploys daily at 4:00 PM. No errors. Diagnose and fix with one line.** → The 4 PM deploy warms thousands of entries at once, all with the same **12-hour** TTL. So they all expire together 12 hours later, and the database rebuilds them at the same time. Fix: TTL jitter, `ttl = 43200 ± rand(10%)`, which spreads the expirations over about an hour. Moving the deploy time would only move the spike. Jitter removes the synchronization.

Related patterns

- **Cache-Aside** — this lesson provides the protection for its problem of misses happening together.
- **Read-Through** — its request coalescing is single flight on a smaller scale (loading caches often offer it per process).
- **Consistent Hashing** — keeps cache-cluster resizes from causing mass misses and needless stampedes.
- **Graceful Degradation** — serving stale data belongs to a wider family of ideas: "give a reduced answer instead of failing".

TL;DR

- **Problem:** a hot key that expires under load sends every simultaneous request to rebuild it (request rate × rebuild time identical queries), in a spiral that feeds itself. A cold cache runs that math on every key at once.
- **Mechanism:** one rebuilder per key (lock / single flight). Serve stale while refreshing in the background. Refresh the hottest keys probabilistically before expiry. Jitter TTLs. Warm cold caches and ramp traffic up.
- **Costs:** limited staleness served on purpose, lock TTLs and waiter logic (use libraries), waiters held for a moment, and some wasted early refreshes.
- **Skip it when:** keys are cold or traffic is low. Measure simultaneous misses before adding machinery.

Capstone - Caching the Product Catalog

Original lesson: Capstone - Caching the Product Catalog on DesignGurus ↗

What this capstone is. A product page *looks* like a single resource. But behind its URL sit **five different kinds of data**. Each has its own read/write pattern and its own cost of being wrong, so each needs its own caching decision. After that, a flash sale stress-tests all the choices together. For every decision, name both the **pattern** and its **correctness mechanism**. That mechanism is a TTL (a timer that expires an entry), invalidation (deleting the entry when its source changes), or both. Then finish the sentence **"if this data is N seconds old, then ___."** Answering that separately for each value is the entire module in one exercise.

The brief

- **Traffic:** 12,000 requests/s at peak, **99%+ reads**.
- **Five kinds of data on the page:** the product **description** (name, images, specs), the **price**, live **inventory** ("only 3 left!"), the **view counter** ("12,483 viewed today"), and **"customers also bought"**

(a strip of computed recommendations).

- **Existing infrastructure:** the sharded, replicated database from Module 3; a Redis cluster routed with consistent hashing; and a merchandising team with a **bulk-import tool** that writes straight to the database (it has already caused two incidents in this course, so keep it in mind).

Decision 1 — The description

The data: read all the time, edited every few weeks. Stale values cost almost nothing (nobody can tell that a description is three minutes old).

Decision: cache-aside, the default. The app checks Redis, fetches misses from the database, and saves them with a **1-hour TTL**. An edit updates the database and then **deletes** the key. It doesn't update the copy, because of the concurrent-write race from the Cache-Aside lesson. **Why keep the TTL?** The type of bug to guard against is the bulk importer, which writes straight to the database without invalidating. The TTL limits its stale data to one hour instead of forever.

Staleness sentence: "If the description is an hour old, then a typo fix took an hour to appear." That's acceptable.

Decision 2 — The price

The data: read the same way as the description. But a wrong price has financial, legal and trust costs.

Decision: the same pattern, with stricter correctness. Cache-aside still suits the *read profile*. But **invalidation on every write becomes the main way to stay fresh**: a price change deletes its key within milliseconds. The backup TTL shrinks from 1 hour to **5 minutes**, so the bulk-pricing tool can leave a price wrong for five minutes at most. **Checkout keeps a direct line to the truth:** the price saved on an order is read from the database. Only *displayed* prices can be briefly stale.

The central principle of the module: **the read profile chooses the cache pattern; the cost of being wrong chooses the correctness rules.**

Decision 3 — Live inventory

The data: "Only 3 left!" changes with every sale, possibly hundreds of times a second. It shows on every page view, and it's close to money.

Decision: split the reads, and keep the writes on the truth.

- **The display badge** is treated as a marketing number. It's cached with a **10-second TTL** and **no invalidation per sale**. Why not invalidate? Hundreds of invalidations per second would bring back the very database load the cache removes, with no real benefit for users.
- **The checkout decision** that reserves the last unit always runs a **conditional database decrement** (it only succeeds if stock is left). This purchase path never relies on the cache.

Staleness sentences: for the badge, "if the count is 10 seconds old, a customer sometimes sees 'only 3 left' when it's 2; urgency marketing survives." For checkout, no staleness is acceptable, so there's no cache. It's one number with two designs, because the two reads have different consequences.

Decision 4 — The view counter

The data: 12,000 increments/s on the bestseller, which is exactly the Write-Behind incident. This is the one kind of data on the page where *writes* are the problem.

Decision: write-behind. Each view is a Redis `INCR` (under 1 ms, no row locks), and the page reads the same key. Every **2 seconds**, a background flusher merges the batch into one `+24,000` instead of 24,000 separate `+1`s. The loss sentence is written down: "if we lose the last 2 seconds of view counts, the numbers are imperceptibly lower." A **flush-on-shutdown** step in the deploy process keeps routine restarts from losing even that. **Truth inversion** applies. Redis holds the current value and the database trails by ~2 s, so analytics that read the database must label their results "as of" a time.

Decision 5 — "Customers also bought"

The data: takes about **400 ms** to compute, by combining order history from several tables. It's refreshed by a **nightly model run**. Yesterday's result is just as good as this morning's (staleness doesn't hurt it).

Decision: cache-aside, and definitely *not* read-through. This is the limit from the Read-Through lesson. A read-through loader fetches one object by key, but this strip needs application logic to put several sources together. So the app builds it, caches it with a **12-hour TTL**, and **invalidates the affected entries when the nightly model publishes** new recommendations. Note the warning sign for the next stage: a 400 ms rebuild makes this key especially exposed to misses happening at the same time.

Decision 6 — The flash sale (every decision stress-tested)

The event: at 8 PM, traffic **triples**. Every failure from the stampede lesson shows up. Hot keys expire under 12,000 req/s, keys that were warmed together expire together, and the cache cluster changes size in the middle of the sale.

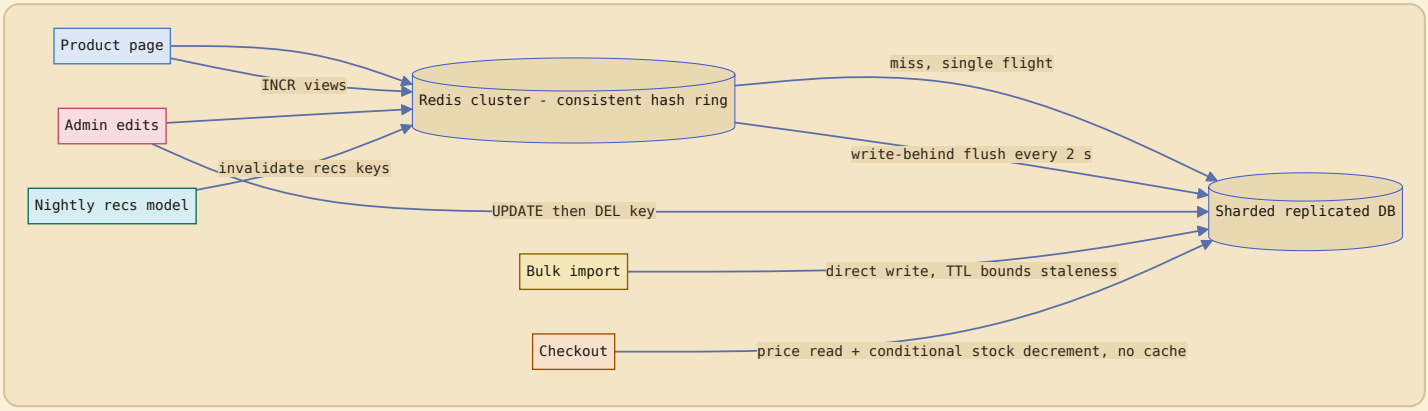
The layered defenses, each matched to the failure it prevents:

1. **Serve-stale as the default policy** (hot-key expiry). Descriptions, display prices and recommendations return stale values in 1 ms while one protected refresh runs. The 400 ms recommendation rebuild runs **once instead of 600 times**. (A display price is stale only for the length of the rebuild, not its whole TTL, and checkout still reads the database.)
2. **Single flight on every rebuild** (the limit on the crowd). Each lock has a short TTL. Other requests get stale data or wait briefly. The database sees **at most one query per distinct expired key**, and that limit is what makes capacity planning possible.
3. **Early refresh for the top 100 products** (the hottest keys). Probabilistic early refresh means the busiest products almost always refresh *before* they expire. So the bestseller's 8 PM stampede never happens.
4. **Jitter on every TTL** (mass expiry). All TTLs vary by **±10%**. So the thousands of values warmed by the **4 PM deploy** expire over several minutes instead of all together at 5.
5. **Consistent hashing for the Redis fleet** (the cluster change). Adding one node remaps about **1/N** of keys instead of about **95%**, so there's no miss storm.
6. **A written cold-start runbook** (the empty-cache case): warm popular keys, ramp traffic up slowly, and let single flight cap the database at one query per key. It's documented *before* anyone needs it.

Result: the sale finishes with database reads **under 400 queries/s**, even though traffic tripled. That's because caching was handled as a set of separate correctness and load decisions, not as one global habit.

The final architecture

Final architecture: each data type gets its own path, with checkout decisions always on the source of truth.



In the diagram above, the product page reads through the consistent-hash Redis cluster. The cluster falls through to the sharded database only on a miss, using single flight. View increments go to Redis and reach the database through the 2-second write-behind flush. Admin edits update the database and then delete the key. The bulk import writes to the database directly, so only TTLs limit its staleness. The nightly recommendations model invalidates the recs keys. And checkout reads the price and runs the conditional stock decrement directly on the database, never on the cache.

Data on the page	Staleness budget	Pattern + correctness	Stampede treatment
Description	~1 hour	Cache-aside; DEL on edit + 1 h TTL (insurance against bypassing writers)	Serve-stale + single flight
Price (display)	Seconds	Cache-aside; DEL on write + 5 min TTL	Serve-stale (limited to the rebuild) + single flight
Price (on the order)	Zero	No cache; read the source	n/a
Inventory badge	~10 seconds	Cache-aside; TTL only (writes happen too often to invalidate each one)	TTL jitter
Inventory decision	Zero	No cache; conditional write on the source	n/a
View counter	Live from cache	Write-behind; 2 s flush, loss sentence written down, flush-on-shutdown	n/a (it never expires; while dirty, it is the source)
Recommendations	Hours	Cache-aside (computed, so a loader can't build it); nightly invalidation	Early refresh (top 100) + single flight

How to read it: go down the "staleness budget" column. Every decision starts from the oldest the data is allowed to be. The patterns enforce those budgets. The **hit ratio** is the financial reason to cache. **Concurrent misses** are the reliability risk. Those three questions sum up the module.

Extend it yourself (open questions to practice)

- **Multi-region:** the catalog now serves two continents, and each has its own Redis cluster. A price change made in one region has to stop being wrong in the other within seconds. Design cross-region invalidation. What gets sent (deletes or values?), over what channel (hint: a Module 2 pub/sub-style channel), and what goes wrong when that channel lags (the other region serves stale prices)?
- **Personalized prices:** enterprise customers see negotiated prices. So the key becomes `price:{product}:{customer}`, and the **key space** (the number of possible distinct keys) jumps from **100K to 500 million**. What's left of the price design? Think about hit ratios on long-tail keys (rarely read, so they rarely hit) and memory budgets. Also ask whether the *inputs* to the price calculation are easier to cache than the result.
- **For AI engineers:** add semantic product search ("cozy reading chair") using an LLM-embedded vector query that takes ~**200 ms** and costs real money each time. Exact-match caching misses all the time, because nobody words a query the same way twice. Sketch a design, then compare it with the semantic cache lesson (merging *similar* requests is a pattern of its own).

Interview one-liner

> Don't cache "the page". Cache each data type according to its staleness budget: cache-aside with TTL insurance for descriptions and recommendations, invalidation first for prices, a TTL-only badge plus an uncached purchase decision for inventory, write-behind for counters, and layered stampede protection so the database sees at most one query per expired key during the sale.

Module 5 — Surviving Failure (Reliability)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. On a whiteboard, an arrow from service A to service B looks innocent. In a live system, that single arrow can break in four different ways. B may never reply at all (we call this a *hang*). B may reply with an error. B may finish only part of the job. Or B may reply so late that nobody can use the answer. This module gives you seven well-known patterns. Each one lets you handle these outcomes deliberately, rather than finding out by accident.

Why "rare" failures are not rare. A failure rate can sound tiny and still hit you all day long once traffic is high. Picture a checkout that processes 100,000 orders a day, where 1 call in 200 fails. That works out to **500 failures every single day**. A reliable design plans for that real, measured number instead of wishing it away. This is also why interviewers keep coming back to failure. Each A-calls-B arrow you sketch carries a hidden question: "and what if B doesn't answer?"

Pattern picker — first name the failure you see, then choose the tool:

What the failure looks like	Pattern
Silence; waiting callers keep stacking up	Timeout
Breaks right now, then works again a few seconds later	Retry with Backoff
You can't tell whether your payment actually went through	Idempotency
Stays broken for minutes, not just moments	Circuit Breaker
No errors anywhere, yet one workload is swallowing all the capacity	Bulkhead
A single bad message holds up every message behind it	Dead Letter Queue
All the mechanisms did their job, yet the user still landed on an error page	Graceful Degradation

Two kinds of pattern. Six of the seven are **mechanisms**. Their job is to decide *when to stop*: stop waiting, stop retrying, stop calling, or stop redelivering. The seventh, Graceful Degradation, is a **policy**. Its job is to decide *what the user gets* while a part stays broken. You need both kinds. With mechanisms only, you just produce errors more quickly. With a policy only, nothing actually enforces it.

Why the order matters. Every lesson solves a problem that the lesson before it either causes or reveals. Timeouts make silent hangs show up as visible failures → retrying those failures might perform a write twice → idempotency makes repeated attempts harmless → yet it is useless when a **dependency** (another service or system that yours calls) stays down for minutes, so circuit breakers cut off the calls → bulkheads deal with floods of *successful* requests, which breakers cannot detect → queues need somewhere to put messages that will never succeed → and once all of that is in place, someone still has to choose what the user sees. The capstone then uses all seven fixes on seven incidents inside a single payment system. Short on time? Timeout, Retry, Idempotency, and Circuit Breaker are the "critical four". They build on each other in a fixed order, so study them as a group.

Four terms used everywhere in this module

- **Transient vs. sustained failure.** A *transient* failure goes away on its own within seconds, like a lost network **packet** (one small chunk of data travelling over the network). A *sustained* failure sticks around because the service truly is down or swamped. Retries fit transient failures. Circuit breakers fit sustained ones. Choose the wrong tool and you pile extra load onto an outage.
- **Fail open vs. fail closed.** Sometimes a check cannot run. *Failing open* means you let the operation go ahead anyway. *Failing closed* means you block it. Nice-to-have features are allowed

to fail open. Security and money must always fail closed — if the login service is unavailable, it must never let everyone in.

- **At-least-once delivery.** Queues and retry loops guarantee that no work gets lost. The price is that the same item can show up more than once. That duplication is expected behavior, not a bug, so whatever code receives the items has to cope with repeats safely.
- **Little's law.** *Work in progress = arrival rate × time each item takes.* Suppose 300 requests/second arrive and each takes 2 seconds. Then 600 requests are in flight at the same moment. If every request ties up a thread (a unit of execution that a server uses to handle one request), you need 600 threads. Three of the lessons lean on this formula to predict the exact point where a server runs out of resources.

Before you start — think about a system you know and try to answer these. The capstone repeats them, and "I don't know" is a perfectly good place to begin:

1. Suppose something your service calls goes silent — no error, just nothing. How long does your service keep waiting, and who picked that number?
2. Suppose your service retries a payment or a write that failed. What prevents the retry from charging the customer twice?
3. Take your most important page. When its *least* important dependency goes down, what exactly shows up on the user's screen?

Timeout

Original lesson: Timeout on DesignGurus ↗

What it is. A timeout is a deadline you put on every network call. If the reply has not come back by the deadline, the caller gives up waiting. It marks the call as failed and releases anything it was holding for that request. *Analogy:* you phone a restaurant to reserve a table. If no one answers after ten rings, you hang up and call somewhere else. You don't sit there holding the phone all evening.

The problem: waiting is not free. Plenty of network clients have no real deadline until you give them one. Take Python's widely used `requests` library: by default it waits forever. Or take a raw TCP connection (TCP is the basic network connection that most protocols run on top of; the System Design Fundamentals primer explains TCP, HTTP and other networking basics). If the machine on the other end has quietly died, that connection can stay open for minutes before the operating system finally gives up.

A waiting call is not idle. It holds a thread, a connection, and often a lock (a marker that keeps other work away from the same data). Let's run the numbers. A service handles 300 requests per second. Its dependency starts hanging for 30 seconds on every call. New requests keep arriving at 300 per second, and each one now sits for 30 seconds. To keep up, you would need 300 × 30 = **9,000 threads**. A normal service has only a few hundred. So the entire service freezes in roughly **one second**. Errors did not kill it. *Waiting* did.

Timeouts come first for a second reason too. A hung call is invisible. It raises no error, moves no metric, and sends no signal. A call that times out, on the other hand, is an *event*. The rest of this module can respond to that event: retries can try again, and circuit breakers can count it. Without timeouts, those later patterns have nothing to see.

The figure below has two halves. The top half is the failure case. The client starts a timer (for example, 300 ms) as it sends the request. The timer runs out before any response arrives, so the client deliberately fails the call. The bottom half is a normal call that comes back before the deadline. The side panel explains why this matters: nothing hangs forever, threads and connections get released, "slow" turns into a failure you can count, and that countable failure is the foundation for retries, circuit breakers and SLOs (service-level objectives — the **latency** (response time) and **availability** (share of time the service works) targets a team commits to).

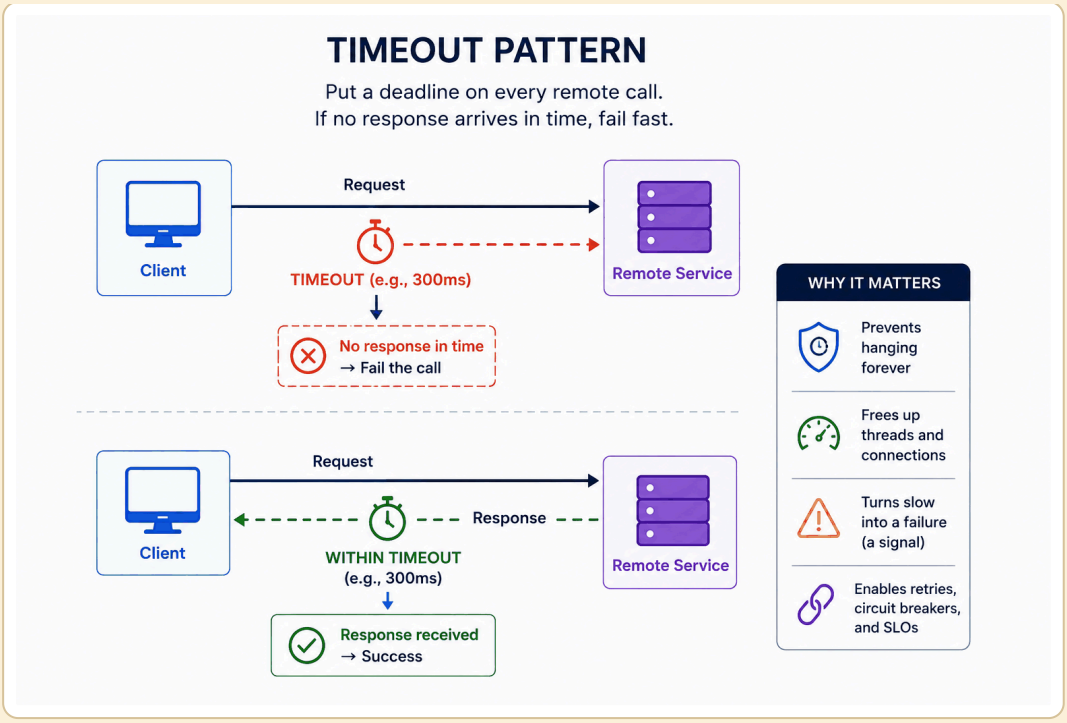


Figure — Timeout Pattern (course diagram)

How it works — three rules

1. **Base the number on measurement, not copy-paste.** Across the industry, the most common timeout is whatever value somebody copied from elsewhere — usually 30 seconds. That value protects nothing, since no user is still waiting at second 29. Measure instead how the dependency performs while it is *healthy*. The usual yardstick is the **p99**: the latency that 99 out of 100 calls come in under. Say a service's healthy p99 is 120 ms. Then set its timeout around **2–3× that, roughly 300–400 ms**. This gives slow-but-normal calls enough room, yet still cuts off genuine hangs fast. Pick **one value per dependency**. A database that answers in 8 ms and a machine-learning model that needs 900 ms can't sensibly share a single number.
2. **Keep the connection timeout much tighter than the request timeout.** They answer two separate questions. The *connection timeout* asks "can I reach you at all?" An unreachable host reveals itself within milliseconds, so waiting longer buys you nothing. The *request timeout* asks "can you give me an answer?" That involves real work, and its duration can legitimately vary.
3. **Hand the leftover time down the chain (deadline propagation).** Imagine your API promises a reply within 2 seconds and makes three calls one after another. If every call gets a brand-new 2-second timeout, the worst case becomes 3 × 2 = **6 seconds** — three times what you promised. The fix is for each hop to inherit only the time that is *left*:
   - The **API gateway** (the front-door server that receives every outside request and passes it inward) begins with a budget of 2,000 ms.
   - The auth service (which checks who the user is) spends 100 ms → 1,900 ms remain.
   - The database spends 400 ms → 1,500 ms remain.
   - The search service's timeout = min(its own limit, 1,500 ms).

   gRPC (a widely used framework for remote calls) handles this for you: the caller's deadline rides along with the request, and each hop respects whatever is left. Plain HTTP mostly doesn't. Without propagation, services deep in the chain keep working on requests that have already failed further up, producing answers that no one will ever read.

Trade-offs

- *You gain:* a firm limit on the damage (a failing dependency can tie up your threads for the length of the timeout and no longer); failures that turn into signals, which feed retries,

breakers, alerts and dashboards; and a predictable worst-case response time, which is the only thing that makes latency promises possible.

- *You pay:*
  - **False alarms.** Now and then the timer goes off on a call that was just about to succeed. For a *write*, this leaves genuine uncertainty: did it happen or not? That exact question is why the Idempotency lesson exists.
  - **Ongoing tuning.** Set it too long and it protects nothing. Set it too short and you create errors on ordinary slow days. Whenever a dependency's latency changes, go back and check its timeout.
  - **The work does not stop.** This one is easy to miss. A timeout ends *your waiting*, not *the server's working*. The database may still be crunching the query you walked away from. Under heavy load, that is the worst outcome: capacity burned on answers no one will read. Some systems pass the *cancellation* along (gRPC does); plain HTTP mostly does not.

When not to use it. Never go without timeouts. Every misuse is really about picking the wrong *value*:

- **Genuinely long jobs** (building reports, **transcoding** video, i.e. converting it to other formats or sizes, large LLM jobs, where an LLM is a large language model such as the one behind a chat assistant) should not be given a 10-minute timeout on a live call. Reshape them as asynchronous work instead: take the job, reply right away, and hand back the result later through a message queue, **polling** (the client asking "done yet?" every so often), or a webhook (a callback the server makes to you once it's finished).
- **A single global number** for all dependencies ends up being the right number for none of them.
- **Relying on library defaults.** Check every client you use. Defaults run anywhere from 30 seconds to infinity, and both ends are wrong.

In the interview. You'll rarely get a direct question about it. Yet each arrow in your diagram silently asks "how long do you wait if this hangs?" Strong candidates answer that before anyone asks. When the question does come up, it usually sounds like "what happens if this service just… doesn't respond?"

> **30-second answer:** I give every network call a deadline derived from the dependency's measured healthy p99 — usually 2–3× it — and I make connection timeouts much tighter than request timeouts. Timeouts convert silent hangs into failures I can count, and those counted failures are what my retries and circuit breakers react to. Along a chain of calls I pass the remaining budget downward, the way gRPC does, so inner calls never run longer than the request that kicked them off.

- *Follow-up: "How do you pick the number?"* — By measuring. Take the dependency's healthy p99, multiply by a small factor, set it separately for each dependency, and revisit it whenever that dependency's latency shifts. "30 seconds" with no reasoning behind it fails in both directions: it's useless during an incident and wasteful during recovery.
- *Follow-up: "The timeout fired, but the write actually went through — now what?"* — That uncertainty is genuine and you can't avoid it. The fix is to make the write safe to repeat, and that is the entire topic of Idempotency.
- *Mistake that fails candidates:* designing a system with several services and never mentioning what happens when a call hangs — or saying "30 seconds" without any reasoning. Either one suggests a person who has only watched systems on their good days.

Self-check:

- Q1: Why should connection timeouts be tighter than request timeouts, and where should the request timeout come from? → A host you can't reach shows itself within milliseconds, so extra

waiting gains you nothing. The request timeout comes from the dependency's measured healthy p99 × 2–3, chosen per dependency.

- Q2: What does a timeout *not* stop, and why does that matter most during overload? → It doesn't stop the work on the server. Queries you abandoned keep consuming a struggling server's capacity, which makes the overload deeper. Where it's available, passing cancellation along (as gRPC does) fixes this.
- Q3: An API promises 2 s, makes three sequential calls each with a 2 s timeout, and users see 6-second errors. Why, and how do you fix it? → Every hop started with a fresh budget, so the worst case is 3 × 2 = 6 s. Switch to deadline budgeting: each call's timeout = min(its own limit, time remaining). Then the request always fails by the 2 s mark.

Related patterns: Retry (reacts to the failures that timeouts create); Circuit Breaker (treats timeouts as evidence — without them it can't notice hangs); Idempotency (settles the "did my write go through?" question that every expired timeout raises); Bulkhead (limits how many threads can be stuck waiting, even while still inside the timeout window).

TL;DR

- **Problem:** calls with no deadline can hang forever, quietly holding threads and connections until one slow dependency has eaten the entire caller.
- **Mechanism:** put a deadline on every call at 2–3× the dependency's healthy p99, use a tighter connection timeout, and pass the remaining budget down the call chain.
- **Costs:** occasional false failures (which cause write uncertainty), ongoing tuning, and abandoned work that keeps using the server unless cancellation is passed along.
- **Skip it when:** never — instead, move long work to async and swap copied values for measured ones.

Retry with Exponential Backoff

Original lesson: Retry with Exponential Backoff on DesignGurus ↗

What it is. Some failures fix themselves. When a call fails for that kind of reason, you try again. But you wait a bit longer after each failure. You add randomness to each wait so that thousands of clients don't all retry in the same instant. You limit how many times you try. And you only retry errors that could actually come out differently next time. *Analogy:* when a phone line is busy, you don't hit redial every second forever. You wait a little, then a little longer, and at some point you give up. And if everyone in town redialed at exactly the same second, the line would stay jammed.

The problem. A checkout calls a payment provider. The provider is healthy, and both sides are coded correctly. Still, roughly 1 call in 200 fails. Networks drop packets, reset connections, and once in a while send a request to a bad server. Overall the provider is "up", yet single calls still fail. With 100,000 orders per day, that means **500 customers a day** typed in their card details and got an error — on a request that would most likely have worked 100 ms later. Brief failures that clear up on their own like this are called **transient** failures. If you treat each one as final, you throw away operations that a second try would have finished.

Why the obvious fix fails. The obvious fix is "just retry right away, up to 3 times." At scale, this goes wrong in two ways:

- **You show up at the worst possible moment.** Retrying immediately assumes the failure is already gone. But a provider that is overloaded is still overloaded a microsecond later. Take a service sending 20,000 requests/second. During a one-second failure, three immediate attempts can add up to **60,000 requests/second** landing on the provider exactly when it has the least room to spare. This traffic surge that you caused yourself is called a **retry storm**.
- **Everyone retries in the same instant.** Requests that failed at the same time will also retry at the same time if they follow a fixed schedule. That produces repeated, synchronized spikes. Such a synchronized crowd is called a **thundering herd**, and it can knock a service that is just recovering back down again.

- There is a third problem, which the next lesson covers: if the payment *succeeded* but its response got lost, a retry can charge the customer twice.

How it works. The figure below follows a single payment call. Attempt 1 times out. The client waits a random time between 0 and 100 ms. Attempt 2 gets a 500 error (an HTTP status code meaning "the server hit an internal error"). The client waits a random time between 0 and 200 ms. Attempt 3 succeeds, and the customer never noticed anything. The box on the right shows the formula and the wait windows, which double each time (0–100, 0–200, 0–400, 0–800 ms…, capped at 5 s).

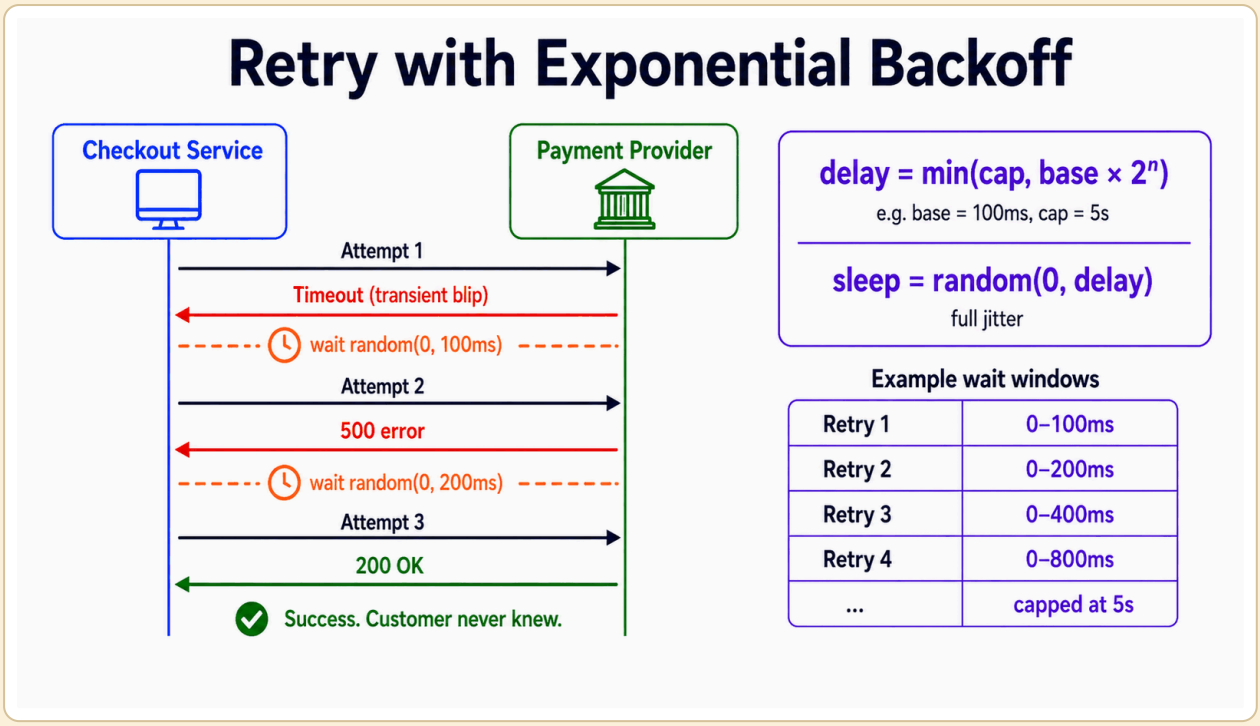


Figure — Retry with Exponential Backoff (course diagram)

The formula ("full jitter"):

```
window = base × 2^attempt          (e.g. base 100 ms → 100, 200, 400 ms…)
wait   = random(0, min(cap, window))   (cap e.g. 5 s)
```

Every part of it has a single job:

1. **Base delay (~100 ms).** This gives the most frequent transient problems — lost packets, connection resets — a moment to clear before the next try.
2. **Doubling (the "exponential backoff").** Each extra failure is a hint that the problem is bigger than you assumed, so you back off more strongly. Doubling is the difference between a patient client and one that makes the outage worse.
3. **Randomness, called jitter — not optional.** Jitter scatters the thundering herd. Rather than a thousand clients retrying at the same instant, they spread out randomly across the window. Backoff *without* jitter merely moves the same synchronized wave to 100 ms later, then 200, then 400.
4. **Caps and budgets.** Set a longest allowed wait, so you never back off into minutes. Set a maximum number of attempts, too (3–5 for anything a user is waiting on). Ideally, add a **retry budget** as well: a rule for the whole service, like "retries may add at most 10% extra traffic." Once that budget is used up, stop retrying — by then the failures are obviously not transient.
5. **Only retry errors that can change.** This rule is just as important as the formula. Worth retrying: timeouts, connection failures, HTTP 500 (server error), 503 (service unavailable), and 429 (too many requests — follow its `Retry-After` header, which says how long to wait). Never retry: 400 (bad request), 401 (bad credentials), 404 (not found). Those will fail in exactly the same way every time.

Walkthrough with numbers

- *One request.* A payment call runs into a short 200 ms network problem. Attempt 1 fails, and the client waits ~60 ms (random within 100). Attempt 2 fails, and it waits ~150 ms (random within 200). Attempt 3 succeeds. Checkout took roughly **400 ms longer** than normal — but it finished, rather than showing an error. Now multiply that by the 500 customers a day who quietly succeed.
- *The whole fleet.* The provider struggles for one full second while receiving 20,000 requests/second. With immediate retries, ~20,000 failed requests each fire 3 attempts back-to-back straight into the outage. Then everything that piled up retries at the very instant the outage ends — about **3× normal load** pointed at the weakest moment. With backoff and jitter, the first round of retries spreads over 100 ms, the second over 200 ms, the third over 400 ms. The provider sees only **a small bump a few percent above normal**, and traffic eases back in. The rule for each request is identical, but the system behaves completely differently: synchronized spikes turn into smooth load.

Trade-offs

- *You gain:* transient failures vanish from user screens, dashboards, and support queues; services that are recovering receive smooth traffic rather than waves; and there are fewer manual re-runs of **batch jobs** (programs that process a large pile of work in one scheduled run) at 2 AM.
- *You pay:*
  - **Slower worst cases.** Each retry adds its wait to somebody's response time (≈400 ms in the example). Your slowest requests now contain your retry policy.
  - **Extra load when the failure is real.** If a dependency is completely down, every retry is wasted traffic. Backoff softens this. Only a circuit breaker or a retry budget truly stops it.
  - **Multiplication across layers.** Say the mobile app retries 3 times, the API gateway retries 3 times, and your service retries 3 times. A single tap can then turn into 3 × 3 × 3 = **27 requests** at the bottom of the stack. Choose one layer on purpose (two at most) to retry in — usually the one nearest the failure. Never retry "everywhere, just in case."
  - **Hidden chronic problems.** Quiet retries can make a dependency with a 5% failure rate look healthy, while secretly adding latency and load. Always record how many retries happen. A climbing retry rate is an early warning.
  - **One hard prerequisite.** Retrying a *write* is only safe when doing it twice causes no harm. That property is called idempotency. It doesn't come for free, and it's the next lesson. Until a write has it, don't retry that write.

When not to use it

- **Writes that aren't safe to repeat.** The risk of charging twice is real. Add an idempotency key first, and retries after that.
- **Errors that can't change** (validation, permission, not-found). Sort errors into kinds before you start looping.
- **When the user has already waited long enough.** Someone staring at a spinner is usually better off with a quick failure and a Retry button than with four silent attempts over 3 seconds. Paths where a person is waiting get small budgets (1–2 retries). Background jobs can afford generous ones.
- **When the failure is sustained.** An outage that lasts minutes is the circuit breaker's job.
- *Common misuse:* "retry 3 times" at every layer, even on 400s, with no jitter and no metrics — that extra traffic becomes painfully obvious during the next partial outage.

Retry now vs. queue for later. Retry is the optimistic tool and the circuit breaker is the pessimistic one. Real systems stack both (the next lessons compare them). A less obvious alternative is a message

queue, since both answer the same question: "this call failed — now what?"

Situation	Reach for
The caller needs the answer during this request (payment authorization, a user is waiting)	Synchronous retry with backoff
The caller needs the work to happen, but not the answer right now (send a receipt, sync the warehouse)	Put it on a queue; the consumer retries over minutes or hours
The retry has to survive a process crash or a long outage	Queue, with a dead letter queue as the final safety net
The entire retry window fits inside one request's timeout	Synchronous retry

The question to ask out loud: *does anyone need this result right now?* If nobody does, the sturdiest retry is a durable one. On a queue, attempts can stretch over hours, survive restarts, and be kept safely once they run out.

Real-world examples

- **AWS SDKs** use exponential backoff with jitter out of the box. AWS's "Exponential Backoff and Jitter" blog post is the classic explanation, and it grew out of real thundering herds hitting DynamoDB (Amazon's managed key-value database).
- **gRPC and Envoy** (a popular network proxy) support retry budgets: retries are limited to a percentage of live traffic, so they can never take over the load.
- **Stripe** combines retries with idempotency keys on every endpoint that moves money — the combination that makes retrying a charge safe.
- **Kafka producers** (programs that write messages into Kafka, a widely used message log) automatically retry sends with backoff. Turn on idempotence and you get this pattern plus the next one, built into the client.
- **For AI engineers:** LLM APIs regularly shed load by returning 429 and "overloaded" errors, and the official Anthropic and OpenAI **SDKs** (ready-made client libraries for calling their APIs) include backoff-with-jitter by default. Good production habits: follow `Retry-After` exactly, keep a separate retry budget for each provider, and when one provider's budget runs out, switch to another model at your **LLM gateway** (a proxy your apps call instead of calling model providers directly) rather than retrying into a sustained outage.

In the interview. "What happens if this call fails?" is the follow-up you'll hear most often in system design interviews. The moment you answer "retry," the trap springs: "Could your retries make an outage worse?" Strong candidates raise retries without being asked and mention idempotency in the same breath.

> **30-second answer:** I retry failed calls using exponential backoff with jitter — about 100, then 200, then 400 ms, each one randomized so thousands of clients don't retry together and flatten a service that's recovering. I limit the number of attempts, and I only retry errors that can change, such as timeouts and 500s. Before any of that, I make the operation idempotent, so a retried write can't take effect twice. AWS SDKs do exactly this by default. And when failures are sustained instead of transient, I stop retrying and hand over to a circuit breaker.

- *Follow-up: "Why jitter? What breaks without it?"* — Without it, all the clients that failed together retry together. Backoff alone just books the thundering herd for 100, 200, 400 ms. Randomizing each wait spreads the clients out and turns the spikes into a smooth plateau.
- *Follow-up: "How do you stop retries from making an outage worse?"* — Three controls, from weakest to strongest: backoff spaces the attempts apart; a retry budget limits retries to a fraction of

traffic; a circuit breaker drops retries to zero once the failure is clearly sustained. On top of that, a structural rule: retry at one layer, not at every layer.

- *Mistake that fails candidates:* "I'd just retry 3 times." No backoff, no jitter, no idempotency, no sorting of errors — you've just handed the interviewer four follow-ups you'll miss.

Self-check:

- Q1: Write the full-jitter formula and say what the cap protects you from. → `wait = random(0, min(cap, base × 2^attempt))`. The cap puts a ceiling on the worst-case wait, so a call that keeps failing can't back off into minutes, blowing through response budgets and holding resources. It also keeps timing predictable for capacity planning.
- Q2: App retries 3×, gateway 3×, backend 3× on DB calls; the DB has a 10-second brownout (slow but not fully down). What does it experience and where should retries live? → As many as 27 attempts for each tap — an overload you caused yourself, with every layer's queue stuffed with duplicate work. Retry in one layer, usually the one closest to the failure (the backend's DB call, with a budget). The outer layers fail fast and show the user a Retry button.
- Q3: A dependency recovers from a 30-second outage, collapses 100 ms later, recovers, collapses again, in perfect rhythm; your config is 3 attempts, fixed 100 ms delay, no jitter. → Your clients have become a thundering herd on a fixed timetable: the entire backlog arrives in one wave the instant the dependency recovers. Fix it with doubling delays, jitter, and a circuit breaker, so a sustained failure stops generating retry pressure. The other team is right — their service is fine.

Related patterns: Idempotency (what you need before retrying writes); Timeout (decides what "failed" means to begin with); Circuit Breaker (steps in once a failure turns out to be sustained); Dead Letter Queue (where a queue's used-up retries land rather than being thrown away); Rate Limiting (the other side of the conversation — well-behaved retriers respect 429 and `Retry-After`, and good limiters send them).

TL;DR

- **Problem:** network calls fail briefly all the time, and naive instant retries turn those failures into synchronized spikes aimed at a dependency that is already struggling.
- **Mechanism:** wait longer after each failure (100, 200, 400 ms…), randomize every wait, cap attempts and delays, retry only errors that can change, and stay within a service-wide retry budget.
- **Costs:** slower worst cases, extra load during real outages, 27× multiplication if layers are stacked carelessly, hidden chronic failures, and a hard requirement for idempotency on writes.
- **Skip it when:** the write isn't safe to repeat, the error can't change (400s), the user's patience is used up, or the failure is sustained.

Idempotency

Original lesson: Idempotency on DesignGurus ↗

What it is. We call an operation **idempotent** when running it many times has exactly the same effect as running it once. Some operations don't behave that way on their own — "charge this card" is one. For those, the client attaches a unique **idempotency key**. The server remembers every key it has handled, along with the result. When a repeat arrives, it gets the saved answer rather than being executed a second time. *Analogy:* think of an elevator call button. Pressing it five times does not bring five elevators. An idempotency key is how you give a card charge that same behavior.

The problem. Retries rescued 500 customers a day, but they brought a correctness risk with them. A customer clicks Pay on a **$499** course bundle. The provider charges the card, but the success response

gets lost on its way back. After ~100 ms the retry policy sends an identical request. To the provider, it looks completely new — so it charges again. The customer sees **$998**.

The root cause is this: a timeout means "no response came back," *not* "the operation failed." Maybe the request never got there. Or maybe it finished and only the reply went missing. Repeating a read does no harm. Repeating an uncertain *write* can apply it twice. Duplicate writes come from three sources: retry policies, users double-clicking, and message queues with at-least-once delivery.

The numbers: ~500 uncertain timeouts a day. Suppose a third of those had in fact charged the card → **~165 double charges per day**. Every one of them costs a refund, support time, and roughly a **$15 chargeback fee** (a chargeback is when the customer's bank reverses a charge; it also puts a mark on your record with the payment processor). Call it **~$4,000 a day** — plus customers watching you bill them twice.

Why the obvious fixes fail

- **"Check before you write."** Before charging, ask "is there already a recent charge for this order?" The catch is the gap in time between the check and the write. Two retries running at once can both find nothing and both charge. This is a *race condition* — the result depends on timing you can't control — and bursts of retries make it far more frequent than tests would suggest.
- **"Don't retry writes."** That brings the 500 failed checkouts a day right back and gives up the reliability you gained.
- **"Detect duplicates by similarity."** Say you treat equal charges from the same user within 60 seconds as duplicates. That would block someone who legitimately buys two $499 gifts. And a real duplicate that shows up at 61 seconds would still get through.
- What all three share: they're missing information that *only the client has*. From the server's side, a retry and a genuinely new identical order look the same. Only the client knows whether two requests came from a single click of Pay, so the client has to send that identity along.

How it works. The figure below replays the $998 incident with idempotency switched on. The checkout service sends "charge $499, Idempotency-Key: ord_7f3a". The provider atomically claims the key in its key store ("first time seen"), runs the charge, and stores the result under that key. Then the response is lost. The retry carries the *same* key. This time the claim finds the key already exists, so the stored result is sent back. The customer gets 200 OK and no second charge. Same key + stored result = same effect as one execution.

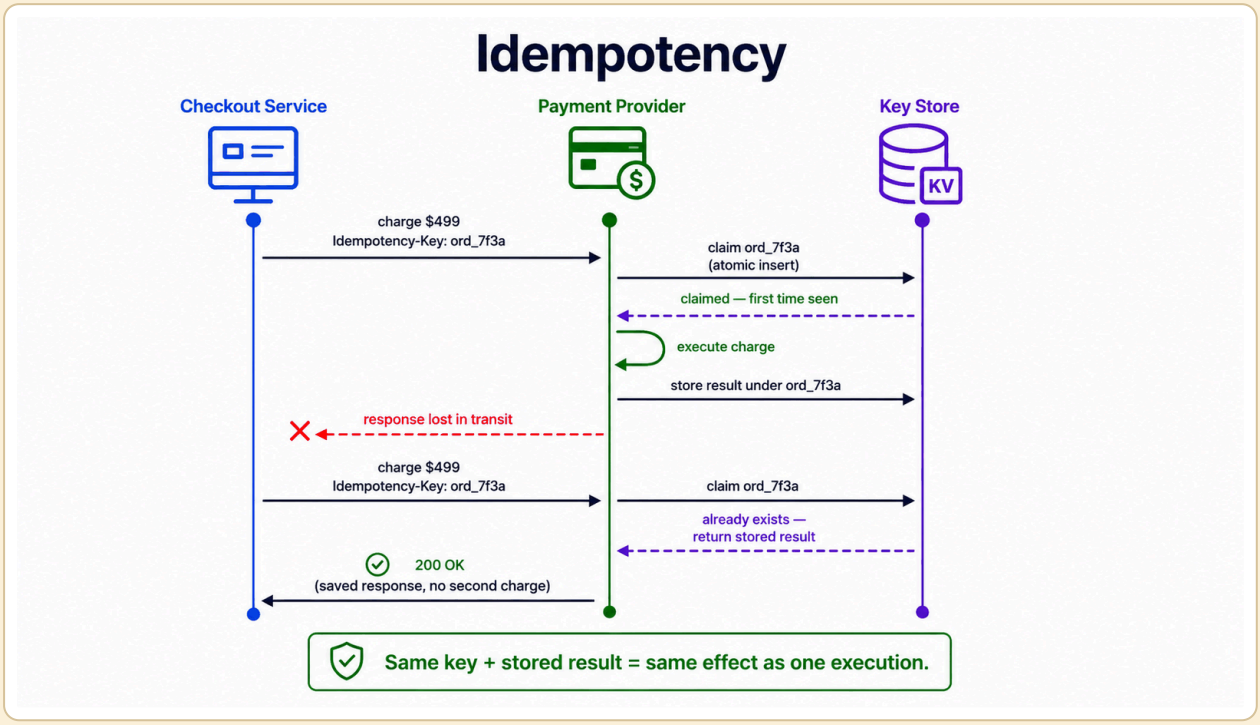


Figure — Idempotency (course diagram)

1. **The client creates the key before the first attempt.** Use one key per *logical* operation. This order's payment gets `ord_7f3a`, and every retry sends that same key. Create it when the user clicks Pay and store it with the pending order. That way, even if the app crashes and reloads, the retry still carries the same key. A client that invents a new key on every retry only *looks* idempotent.
2. **Claiming the key is atomic.** *Atomic* means it happens in one step that can't be split, with no gap in the middle. The server inserts the key under a uniqueness rule: insert it if it's missing, fail if it's already there. The database guarantees that only one insert can win — and that's what really closes the check-then-write race. If two requests arrive at the same moment, exactly one claims the key and executes. The other receives the winner's result.
3. **Save the result, not just the key.** A caller that is retrying needs the original answer (charge ID, status), not an error. And if a duplicate shows up while the first attempt is still running, the server replies "in progress — wait" (or HTTP 409 Conflict) rather than running it again.
4. **Keys expire.** Stripe holds on to them for **24 hours**. The store only has to remember keys for longer than your longest realistic retry window. This expiry timer is called a **TTL (time to live)**.

The cheaper option first: make the operation naturally idempotent. "Set status to shipped" can run ten times and the status is still "shipped." "Increment login count" can't — ten runs give ten increments. So favor *set* semantics ("balance = 90") over *increment* semantics ("subtract 10"). Use **conditional writes** ("update only if version is still 6"). And use **upserts** (insert the row if it's missing, update it if it's there). Any operation built this way never needs a key store.

Walkthrough with numbers (100,000 orders/day, 1 in 200 payment calls times out, retries on)

- *Without idempotency:* ~500 uncertain outcomes/day → ~165 double charges → ~$4,000/day, plus damage to your reputation.
- *With keys:* the retry shows `ord_7f3a`, the store recognizes it in about **5 ms**, and the saved response goes back. Double charges: **zero** — not "rare," but zero by design.
- *Cost:* one key lookup for each write (**~1 ms**), plus storage for 100,000 keys/day × ~1 KB saved responses ≈ **100 MB** with a 24-hour TTL. A few megabytes and a millisecond to remove your scariest failure mode — the cheapest protection in the course.

Trade-offs

- *You gain:* retries, queue redeliveries and double-clicks become safe for writes, by design; **the effect of exactly-once on top of at-least-once delivery** (the only "exactly once" that truly exists — see below); and calmer incidents, since "did the retry double anything?" is no longer a question.
- *You pay:*
  - **A key store on your busiest write path.** It needs a uniqueness guarantee, result storage and expiry — modest infrastructure, but real.
  - **Client discipline.** Keys have to be made before attempt one and survive crashes. That's an agreement with every client team, and it needs testing.
  - **The in-progress case.** The same key can arrive while the first attempt is still running. So you need a small *state machine* (a set of states plus rules for moving between them): claimed → in progress → completed.
  - **The boundary problem** (the trickiest one). The key only protects what happens inside the claim-and-execute step. Side effects outside that step — the confirmation email, a webhook to the warehouse, a third-party call — can still happen twice. Each of them needs its own key (for example, key the email on the order ID).

When not to use it

- **Operations that are already naturally idempotent** — reads, deletes, sets of a full value. Key machinery there is pure overhead.

- **When you can redesign instead.** The best key is one you never had to use: choose "set balance to 90" or a conditional write over "subtract 10."
- **Streams where duplicates are acceptable** — for example, high-volume analytics events that get deduplicated later, or cases where a 0.5% duplicate rate changes no decision.
- **As business-level duplicate detection.** A key means "the same operation, retried." It does not mean "the user probably didn't intend to do this twice." A customer who deliberately places two identical orders is making two operations, with two keys. Blocking that is a product decision.
- *Common misuse:* adding a key store to every endpoint while leaving the operations themselves as unsafe increments and appends — more complexity, no more clarity.

Idempotency vs. "exactly-once delivery." Interviewers bring this up on purpose, since exactly-once *delivery* can't be done over an unreliable network. Any acknowledgment (the receiver saying "got it") might be lost. When that happens, the sender has to choose. It can resend and risk a duplicate (**at-least-once**), or not resend and risk losing the message (**at-most-once**). No third choice exists. What systems *can* reach is exactly-once **processing**: deliver at-least-once, and make the handler idempotent so duplicates only take effect once.

You need	Build
No lost work	At-least-once delivery (retries, queues with acknowledgments)
No doubled effects	An idempotent consumer (keys, or natural idempotency)
"Exactly once"	Both of the rows above — there's no third ingredient

Kafka's advertised "exactly-once semantics" is exactly this under the hood: producer IDs plus sequence numbers make the broker's (the Kafka server's) writes idempotent, combined with transactional consumers. In an interview, "I'd use exactly-once delivery" only names the problem. "At-least-once delivery plus an idempotent consumer" actually solves it.

Real-world examples

- **Stripe:** an `Idempotency-Key` header on every endpoint that moves money, with saved responses replayed for 24 hours. Their docs turned the term into standard industry vocabulary.
- **Kafka's idempotent producer:** tags each batch with a producer ID and a sequence number, so the broker can throw away duplicates — idempotency keys under a different name.
- **AWS SQS FIFO** removes duplicates based on a message ID within a **5-minute** window. **DynamoDB conditional writes** give you the atomic "claim" step without any extra infrastructure.
- **Webhook consumers:** providers openly admit they deliver webhooks at-least-once, so every handler deduplicates on the event ID.
- **For AI engineers:** an agent framework might retry a `send_email` or `create_ticket` tool call that timed out, or the agent loop might ask for it again. Every tool that changes something outside the system needs an idempotency key built from the tool-call ID. Writes that record usage for billing need the same protection, or retries will quietly overcharge customers.

In the interview. You'll rarely be asked to "define idempotency." It shows up as a follow-up instead. You say "retry" or draw a queue, and the interviewer asks "what if the payment succeeded but the response was lost?" or "your consumer crashes before acknowledging — what happens on redelivery?" Answer it before they ask, in the same sentence where you bring up retries.

> **30-second answer:** A timeout is uncertain — the write might have succeeded with only the response lost — so a blind retry can apply it twice. The client creates one unique key per logical operation and sends it with every retry. The server atomically claims that key, runs the operation once, saves the result, and replays that result for any duplicate. This is

> Stripe's Idempotency-Key model: an exactly-once effect built on at-least-once delivery, which is the only exactly-once there is.

- *Follow-up: "Two requests with the same key arrive at the same moment — walk me through it."* — The claim is an atomic insert-if-absent, so exactly one request wins and runs. The other sees that the key is already claimed, waits or is told "in progress," and then gets the saved result. If your answer involves a check followed by a write, the interviewer has just found your race.
- *Follow-up: "Why can't the server generate the key?"* — Only the client knows that two physical requests are really one logical operation. A key made by the server would be different for each request, which defeats the whole point. The client creates it before the first attempt and holds on to it across retries.
- *Mistake that fails candidates:* proposing "check if it exists, then write" as the defense, or claiming that a queue with exactly-once delivery will take care of it.

Self-check:

- Q1: Define idempotent; why must the client generate the key, and when? → N executions have the same effect as one. Only the client knows that two requests are a single intent. The key must exist before the first attempt and must survive crashes (store it with the pending order).
- Q2: With standard idempotency keys on checkout, what duplicates can still happen? → Side effects outside the claim-and-execute boundary (emails, webhooks, third-party calls); intentional business duplicates (two keys); a buggy client that makes a new key for every retry; and duplicates that arrive after the key's TTL has run out.
- Q3: After enabling retries, customers get duplicate welcome emails, yet orders have no duplicate rows and the key store correctly replays results. Diagnose. → The email is sent outside the idempotent boundary. Attempt one charged the card, sent the email, and then lost its response. The retry correctly replayed the result, but the email code ran a second time. Move the email inside the boundary or, better, key it on the order ID (or send it through a queue that deduplicates). Each side effect needs its own idempotency plan.

Related patterns: Retry (the reason this lesson exists); Message Queue (at-least-once delivery turns every consumer into a retry target); Webhooks (deduplicate on event ID); Saga (every step and every undo step of a workflow spanning several services must be idempotent, because the coordinator retries them); Exactly-Once Semantics (how Kafka assembles exactly-once processing from these parts).

TL;DR

- **Problem:** timeouts are uncertain, so retries, redeliveries and double-clicks can apply a write twice — double charges, duplicate orders.
- **Mechanism:** the client makes one key per logical operation and reuses it; the server atomically claims it, executes once, saves the result and replays it for duplicates. Favor operations that are naturally idempotent (set-style).
- **Costs:** a key store on the busy write path, client discipline about keys, an in-progress state, and side effects outside the boundary that need keys of their own.
- **Skip it when:** the operation is already safe to repeat, duplicates do no harm downstream, or the "duplicate" is really a question of business rules.

Circuit Breaker

Original lesson: Circuit Breaker on DesignGurus ↗

What it is. A circuit breaker sits around every call to a dependency and keeps a tally of recent failures. Once failures pass a threshold, it "trips." For a while after that, it rejects every call immediately and doesn't even try the dependency. When a cooldown ends, it lets a few test calls through to check whether the dependency has recovered. *Analogy:* the breaker panel in your home cuts the power

on its own before an overloaded wire can catch fire. Nobody has to make a decision in the heat of the moment.

The problem. It's 2:15 PM on Black Friday. The checkout page calls a recommendations service to display "customers also bought." That widget is *optional* — no purchase depends on it. Then the recommendations database begins to fail. Queries that usually take 20 ms now take 30 seconds or never come back. And checkout goes down.

Why? Checkout has **200 threads** and receives **300 requests/second**, and each request now waits up to 30 s on recommendations. By Little's law, 300 × 30 = **9,000 threads** would be needed. Within about a second, all 200 available threads are stuck on the widget. Checkout already had short timeouts and retries. But even a 2-second timeout needs 300 × 2 = **600 threads** — three times the pool. On top of that, each failure's retries triple the number of calls hitting the dying database (a *retry storm*).

The deeper issue: both timeouts and retries assume *a later call might work*. During a sustained outage, that assumption does harm. The overloaded dependency needs **zero** new calls, and nothing we have so far can make that happen.

Why the obvious fixes fail

- **"Tighten the timeout more."** Dropping from 2 s to 500 ms cuts checkout's thread usage. But every request still reaches the failing database, which spends its remaining capacity on 300 hopeless queries/second. It shields the caller without helping the dependency recover.
- **"Retry harder."** Retries are meant for transient failures. This one is sustained, so extra retries only add load.
- **"Have someone flip a switch."** A flag such as `recommendations_enabled: false` lets **on-call** (the engineer whose turn it is to respond to alerts) stop the calls. The policy is correct, but it moves at human speed: the outage begins at 2:15:00, the alert fires at 2:19, and the flag gets flipped at 2:26 — that's **11 minutes** of broken checkout, when an automatic decision could take roughly a second.

How it works. The first figure shows where the breaker lives: between Service A (the caller) and Service B (the dependency). In the closed state (and during successful half-open tests), requests pass through and responses come back as usual. In the open state, the request is never sent at all. The caller gets a fast failure or a fallback (for example, a cached response) in about 1 ms.

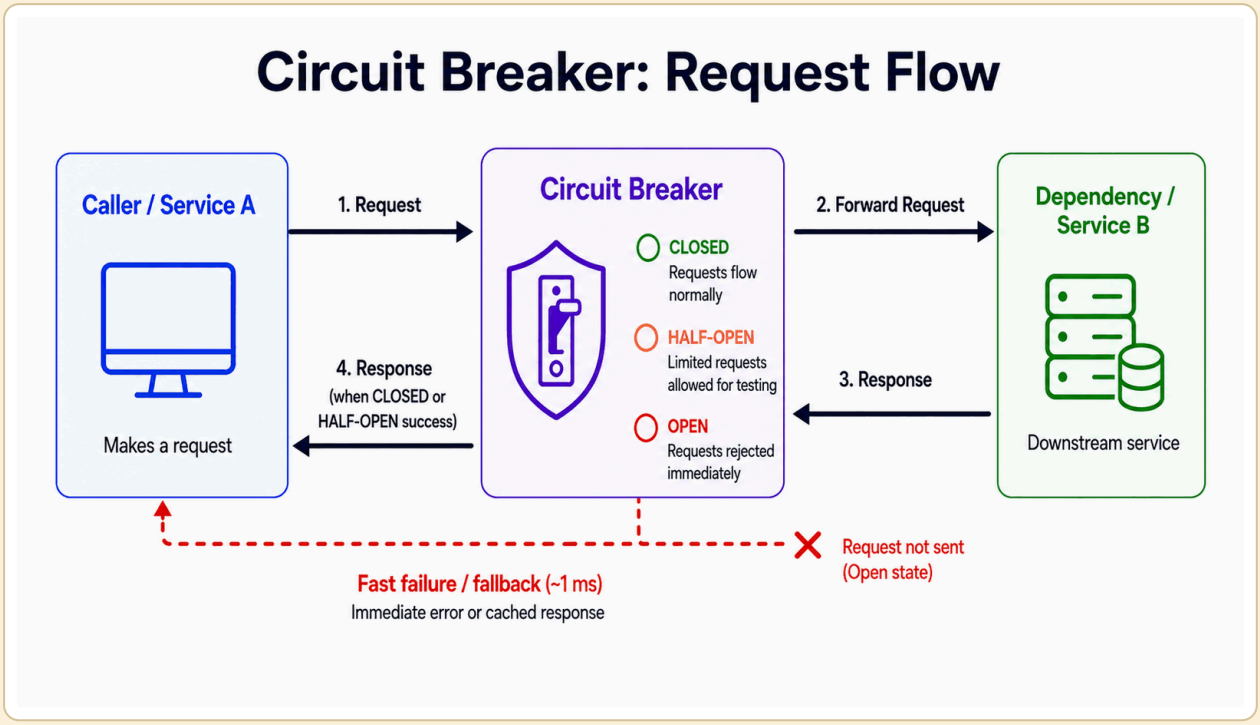


Figure — Circuit Breaker: request flow (course diagram)

The breaker is a small **state machine** with three states. The second figure shows how it moves between them, with example numbers. Closed → Open when more than 50% of the last 20 calls fail. Open → Half-Open when a 30-second cooldown runs out. Half-Open → Closed if all 5 test calls succeed, or back to Open if any test fails. Notice that each arrow points one way only. Open never jumps directly back to Closed — recovery is only ever *proved* by test calls.

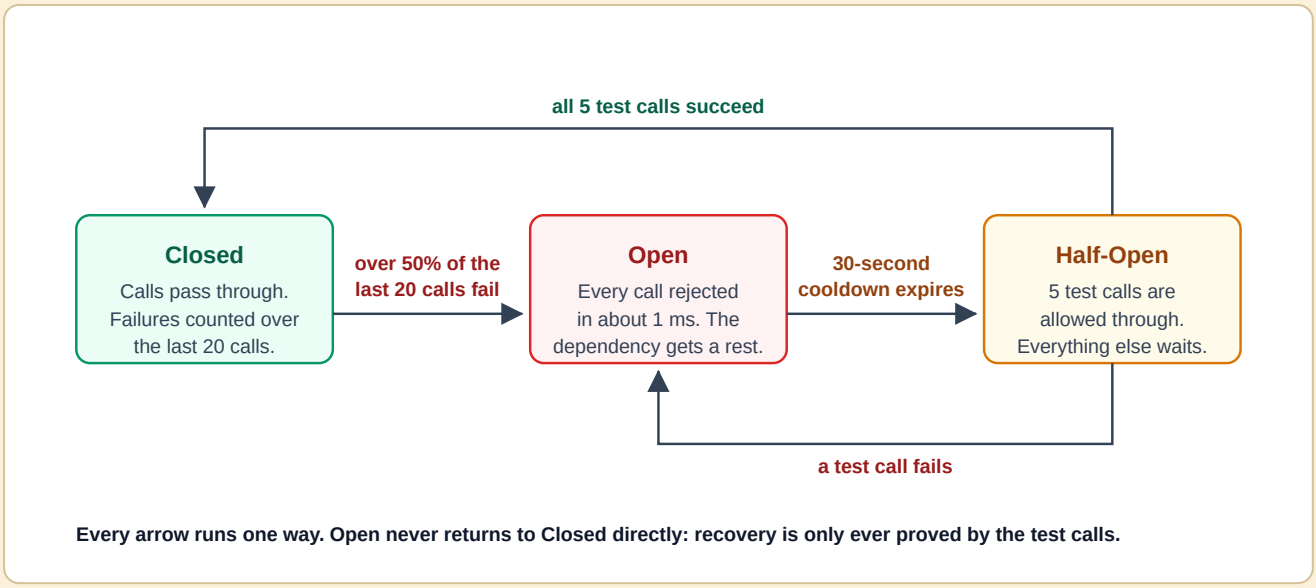


Figure — Circuit Breaker: state transitions (course diagram)

1. **Closed (normal).** Calls go through, and the breaker counts failures over a recent window — say, the last 20 calls. Failures include errors *and timeouts*. That's why the Timeout lesson had to come first: without deadlines, a hung call never counts as a failure, and the breaker can't see anything.
2. **Open (tripped).** The failure rate has passed the threshold (for example, more than half of the last 20). From now on, every call is rejected right away, in about a millisecond, without touching the dependency. Two good things happen together. Your threads stop getting used up. And the failing service finally gets what it needs most — a break from incoming traffic.
3. **Half-open (testing).** After a cooldown (for example, 30 s), a handful of trial calls are allowed through. If they succeed, the breaker closes and normal traffic starts again. If they fail, it opens again for another cooldown. This spots recovery automatically, without dumping full traffic onto a service that has only just come back.

One sentence to remember: *a breaker swaps slow, costly failures for fast, cheap ones, and gives the dependency space to recover.*

Walkthrough with numbers (config: window 20 calls, trip at 50%, 2 s timeout, 30 s cooldown, 5 half-open test calls)

- **2:15:00** — the recommendations DB begins failing; calls time out at 2 s.
- **2:15:01** — roughly 10 of the last 20 calls have failed, so the breaker trips. Damage so far: ten calls tied up threads for 2 s each.
- **2:15:01 onward** — all 300 req/s receive an instant "breaker open" in ~1 ms. Thread math: 300 × 0.001 s = **0.3 threads** busy on average, compared with the 9,000 the unprotected version needed. The page renders without the widget, and checkout carries on as normal.
- **2:15:31** — the cooldown ends and 5 test calls go through. The DB is still failing, all 5 fail, and the breaker opens again. The price of checking: five calls.
- **2:19:31** — a few cycles later the tests succeed. The breaker closes and recommendations come back on their own.
- Result: the dependency was broken for ~4 minutes, while checkout availability didn't change at all. You gave up an optional feature for a few minutes to protect everything around it.

Trade-offs

- *You gain:* failure isolation (one bad dependency can't swallow your threads, connections or memory); fast failure (a millisecond rather than the full timeout); automatic recovery, without a sudden burst of full traffic and without anyone flipping flags at 2 AM; and genuine relief for the dependency — the quickest way to help an overloaded service is to stop calling it.
- *You pay:*
  - **False trips.** A short blip can trip the breaker — for example a 5-second *garbage-collection pause* (the runtime freezing for a moment to reclaim memory) or a *database failover* (switching over to a standby copy). Good traffic then gets rejected for the entire cooldown. Tuning the threshold is ongoing work.
  - **More settings to manage.** Window size, threshold, cooldown, number of test calls. Poor values give you a breaker that *flaps* (flips open and closed over and over) or one that never trips.
  - **Each server decides on its own.** Breaker state usually lives per *instance* (per running copy of your service). With 50 instances, each one spends ~10 failed calls discovering the same outage. Sharing state through something like Redis solves that, but adds one more thing that can break. Most teams simply accept the per-instance cost.
  - **It only decides *whether* to call.** It has nothing to say about what the user sees. An open breaker without a fallback is just a quicker error page — that's the subject of the Graceful Degradation lesson.

When not to use it

- **Local calls inside the same process.** Breakers guard against remote things that can hang or fail for a long time. Wrapping an ordinary function call in one is cargo-culting.
- **Calls where failing fast is worse than waiting**, such as **capturing** a payment (actually taking the money from the card) or sending a legal notice. Those should wait, retry carefully, or go onto a queue.
- **Paths with very little traffic.** For a call made 4 times an hour, "half of the last 20 calls" covers days. The breaker either never trips or trips on random noise. A plain timeout is enough.
- **As a stand-in for capacity.** If a DB breaker opens every day at peak time, the database doesn't have enough capacity. Don't hide that.
- *Common misuse:* breakers on every call, with default settings, no fallbacks and no dashboards — healthy traffic gets rejected and nobody can tell why. Every breaker needs an **owner, a fallback, and an alert** for when it opens.

Circuit breaker vs. retry — they make opposite diagnoses, and interviewers love to make you pick one:

Situation	Reach for
Transient failures: a dropped packet, one bad server, a brief failover	Retry with backoff
Sustained failures: the dependency is down or overloaded	Circuit breaker
Another try would probably succeed	Retry
Another try only piles load onto a struggling service	Circuit breaker

In production you use them together. Put the **breaker outside the retries**, and have it count each request's *final* result after all retries are done. Retries soak up transient blips. The breaker cuts off their extra load when the failure keeps going. (A breaker with no retries rejects blips that would have recovered too quickly.) The Bulkhead, coming next, answers a different question. The breaker decides *whether to call at all* (avoidance). The bulkhead limits *how much capacity* a dependency can take up (containment). You can stack them.

Real-world examples

- **Netflix** created **Hystrix**, the library that made this pattern famous, after slow dependencies kept *cascading* (spreading failure from one service to the next) into complete streaming outages. Its successor, **Resilience4j**, is now the standard on the **JVM** (Java Virtual Machine, the runtime that runs Java and similar languages).
- **Envoy and Istio** (a *service mesh*: proxies that run next to every service) do circuit breaking under the name "outlier detection" — servers that keep failing get removed from the pool automatically, with no application code needed.
- **Polly** (.NET, Microsoft's programming platform) and **PyBreaker** (Python) implement the same three-state machine. **AWS SDKs** include adaptive versions for *throttling* (when a service caps how many calls it will accept).
- **For AI engineers:** model providers routinely rate-limit and slow down under load. Giving each provider its own breaker lets an LLM gateway switch to another model in milliseconds, rather than hanging on an endpoint that's failing.

In the interview. It's rarely asked about by name. Instead you'll hear "what happens to your service if this dependency goes down?", "how do you prevent cascading failures?", or "this search service you call — what if it hangs?" The signal they're looking for: do you understand that **a slow dependency is more dangerous than a dead one**?

> **30-second answer:** A circuit breaker wraps the call and tracks failures over a rolling window. Once they pass a threshold, it opens and rejects calls instantly, instead of letting them stack up waiting on timeouts. That protects my thread pool and takes pressure off the struggling service. After a cooldown it sends a few test calls, and it closes again once they succeed. Netflix built Hystrix for exactly this, after slow dependencies kept cascading into full outages.

- *Follow-up: "How do you pick the threshold and cooldown?"* — By measuring, the same as the timeout. The window needs enough traffic to be meaningful (20+ calls). The threshold should sit well above the dependency's normal error rate. The cooldown should roughly match how long that dependency usually takes to recover. After that, tune it from production data and alert every time the breaker opens.
- *Follow-up: "The breaker is open — what does the user see?"* — The breaker is only half of the pattern. The other half is a fallback — cached data, a default, or hiding the optional widget (Graceful Degradation).
- *Mistake that fails candidates:* suggesting retries for a failing dependency without any mechanism that ever *stops* the retrying.

Self-check:

- Q1: Name the three states and what causes each transition. → Closed (failures counted over a rolling window) → Open when the failure rate passes the threshold; Open (every call rejected instantly) → Half-open when the cooldown runs out; Half-open (a few test calls) → Closed if they succeed, → Open if they fail.
- Q2: Your team adds a breaker to primary-database calls with library defaults. What new failure modes did you introduce? → False trips (an ordinary 10 s failover could reject good queries for an entire cooldown, turning a blip into minutes of partial outage); defaults that flap on quiet endpoints; and, with no fallback, instant errors on your primary data — probably a case where waiting is better than failing fast. It needs tuned thresholds, a fallback (cached reads or read-only mode) and an alert.
- Q3: Worst-case latency spikes to exactly 10 s (your timeout) whenever optional search degrades; soon after the whole API errors. Explain and fix. → Every request holds a thread for the full 10

s. Requests/second × 10 s is more than the pool holds, so healthy endpoints starve behind the stuck threads — a cascading failure caused by running out of resources. Fix it with the whole module: a timeout just above search's healthy p99, a breaker so a sustained failure costs 1 ms, a bulkhead that caps search's threads, and rendering the page without search results.

Related patterns: Timeout (where the breaker's evidence comes from); Retry (handles transient failures on the inside; the breaker keeps them from turning into a storm); Graceful Degradation (what runs while the breaker is open); Bulkhead (limits how much is occupied while the breaker is still collecting evidence); Health Check Endpoint (a URL that a **load balancer**, the server spreading requests across copies of a service, polls to ask "are you healthy?"; the same idea at the machine level — removing failing servers from a pool).

TL;DR

- **Problem:** a slow or failing dependency eats your threads through waits as long as the timeout and pulls your healthy service into its outage; retries make it worse, and nothing can say "stop calling altogether."
- **Mechanism:** a three-state machine — count failures (closed), reject instantly after a threshold (open), and probe with a few test calls after a cooldown (half-open).
- **Costs:** threshold tuning, false trips on short blips, per-instance state, and you still owe the user a fallback.
- **Skip it when:** the call is local, traffic is too thin to give a signal, or failing fast costs more than waiting.

Bulkhead

Original lesson: Bulkhead on DesignGurus ↗

What it is. A bulkhead divides a shared resource (threads, connections, machines) into separate pools with walls between them — one pool per feature, dependency, or **tenant** (one customer organization sharing the system with others). That way, when one pool fills up, it can't overflow and starve the rest. *Analogy:* a ship's hull is split into watertight compartments (bulkheads). A hole floods one compartment, not the entire ship. The total capacity stays the same. What changes is that no single workload can grab all of it.

The problem. A single API **deployment** (one running installation of a service, often many identical copies) serves checkout, search, and order history. All three share **200 threads** and **100 database connections**. (A *pool* is a fixed set of workers or connections that requests borrow for a short time.) Marketing sends out **2 million emails** with a link to the order-history page, and that page's traffic leaps from **50 to 1,500 requests/second**. Order history also runs the service's most expensive query, **~500 ms** each.

By Little's law: 1,500 × 0.5 = **750 threads** wanted. There are only 200, and order history takes every one of them within about a fifth of a second. Checkout is left with zero threads and stops selling.

And yet *nothing fails*. Every order-history query succeeds. The database is up. The error rate is **0.0%**. Error dashboards stay green, and the circuit breaker stays closed (it counts failures, and there aren't any). The problem is **occupancy** — how much shared capacity a single workload is holding — not outcomes. The same shape appears whenever workloads share limited resources: one big tenant starving 500 others, a batch job holding every connection, or one expensive endpoint eating the whole service.

Why the obvious fixes fail

- **"Add more threads."** Raising the pool to 1,000 just moves the limit; it doesn't control who gets the threads. The next surge needs 1,200, and in the meantime 1,000 threads are fighting over the same 100 DB connections. Growing every resource for one temporary workload is costly, and it still isn't isolation.

- **"The circuit breaker will handle it."** It can't. These calls complete before their timeouts, so nothing counts as a failure and the breaker sees a 0% failure rate. This situation calls for a mechanism that watches occupancy, not outcomes.
- **"Rate-limit the users."** Rate limiting controls individual clients and is helpful at the edge. But here, thousands of legitimate users each send one request, so nobody goes over a per-user limit. What you need is a cap on the *feature's share* of the common pool.

How it works. The figure below puts the incident and the fix side by side. Before: a single shared 200-thread pool. Checkout needs 300 × 250 ms = 75 threads, search needs 200 × 150 ms = 30, and order history needs 1,500 × 500 ms = 750 — so order history holds all 200, checkout and search get 0, and the error rate is still 0.0%. After: walled pools. Checkout has 80 (75 in use, still serving). Search has 60 (30 in use). Order history has 40 (FULL: it serves 80 req/s and refuses 1,420 req/s in under 1 ms). And 20 are kept in reserve. Each wall is sized by rate × duration, and it kicks in on *fullness*, not on errors.

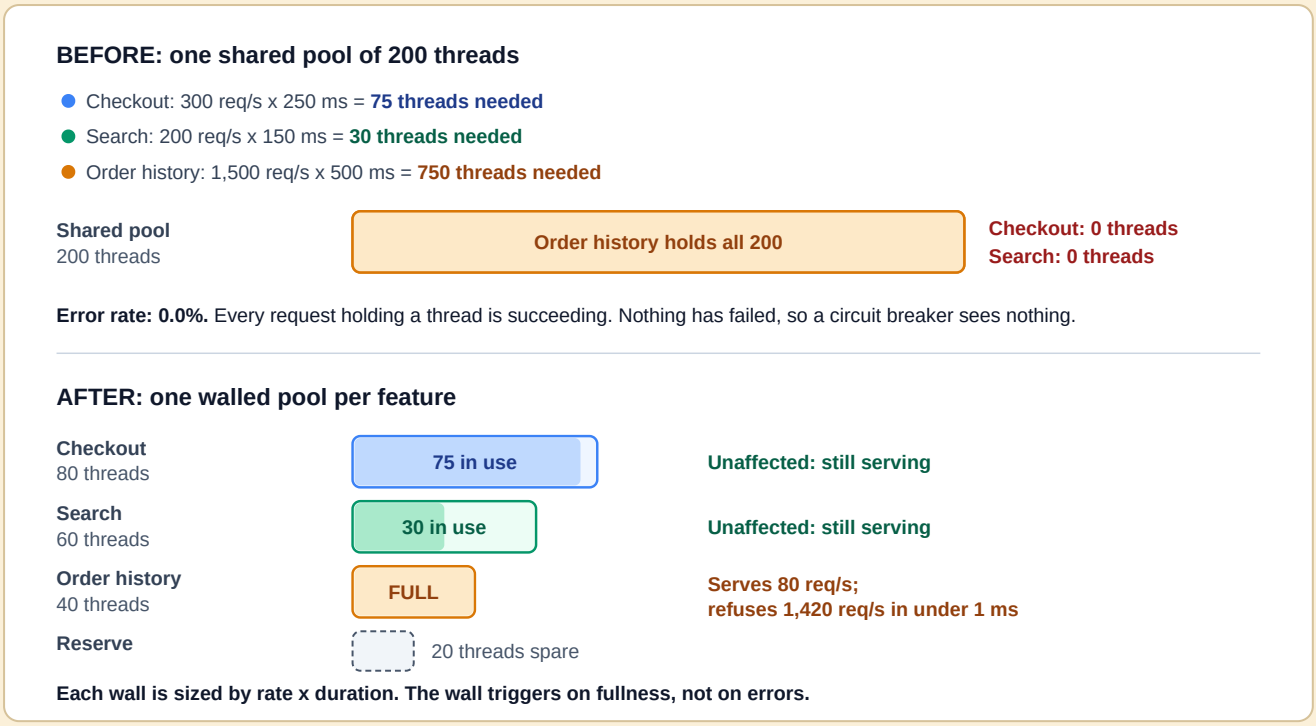


Figure — Bulkhead (course diagram)

You can put the walls up at three levels, each costing more than the last:

1. **Counters (semaphores).** A *semaphore* is a counter that allows at most N things at the same time: "at most 40 order-history requests may run at once." The 41st gets turned away immediately. This is the cheapest option, and it's the default in Resilience4j.
2. **Separate pools.** Each feature or dependency gets its own thread pool or connection pool. There's more overhead, but even a compartment that is completely stuck can't touch anyone else's threads. This was Hystrix's signature technique.
3. **Physical separation.** Different processes, **containers** (lightweight, isolated packages that run an app with its own slice of CPU and memory), or machines for each workload or customer tier. It's the heaviest option, and the only level that also isolates CPU, memory, and crashes.

The rule is the same at every level. A full compartment **rejects new requests quickly** (or queues them for a moment), and it can never borrow capacity from another compartment. A rejection means the wall is doing its job. What the rejected user gets is Graceful Degradation's job.

Sizing each compartment: run the incident's arithmetic backwards — *needed* = *rate* × *duration*, plus *headroom* (extra room for bursts). Checkout: 300 req/s × 0.25 s = 75 threads → give it **80**.

Walkthrough with numbers (checkout 80, search 60, order history 40, reserve 20)

- **The flood arrives.** Order history gets 1,500 req/s and fills its 40 threads. At 500 ms per query, those threads serve 40 / 0.5 = **80 req/s**. The other **1,420 req/s** get "busy right now, try again shortly" within ~1 ms.
- **Checkout, at the same moment.** It needs 75 threads and has its own 80. Latency and **conversion** (the share of visitors who complete a purchase) don't change — the marketing email never touches it.
- **The database is protected as well.** It sees at most 40 concurrent order-history queries instead of 200. A cap upstream also works as a ceiling downstream.
- **Total damage:** one feature degrades honestly under a load it was never sized for. The alternative was the whole product going down in a fifth of a second while every dashboard stayed green.
- The promise shifts from "the service survives only if *total* demand fits" to "**each feature survives if its own demand fits**" — several small promises you can keep, instead of one big fragile one.

Trade-offs

- *You gain:* protection even when *nothing is failing* (it's the only tool in this module that looks at occupancy instead of outcomes); a *blast radius* (how far a single failure can spread) that you choose ahead of time instead of discovering during an incident; honest capacity numbers for each feature (how full each pool is tells you who needs scaling); and a ceiling that protects downstream systems.
- *You pay:*
  - **Idle capacity you can't lend out.** Overnight, checkout's 80 threads might sit unused while order history is rejecting requests right next door. Walls block sharing in both directions, so every compartment needs its own spare room.
  - **Sizing goes out of date.** The mix of traffic drifts. An outdated split rejects requests in one full pool while capacity sits idle nearby — an outage you caused yourself that looks fine in the totals. Redo the numbers on a schedule.
  - **More to watch.** N pools means N fullness metrics, N rejection alerts, and N configs to review whenever latency changes. Alert on **per-compartment fullness** — error-rate alerts can't see this kind of failure at all.
  - **One more concept** that every on-call engineer has to keep in their head.

When not to use it

- **Services that do only one thing**, which are already isolated at their service boundary.
- **Tiny pools.** Splitting 20 threads into four pools of 5 means ordinary ups and downs cause rejections. Below a few dozen threads, isolation can cost more than it protects.
- **Uniform, elastic workloads.** If all the traffic has similar cost and priority, and the platform adds servers within seconds (*elastic*), a shared **autoscaled** pool (one that grows and shrinks automatically with load) may use capacity better. Bulkheads earn their keep when workloads differ in cost, importance, or how much you trust them.
- **As a stand-in for capacity.** If every compartment is almost full every day, you need more total capacity.
- *Common misuse:* a compartment for every endpoint and every client, each too small for normal variation — a hundred undersized pools give you rejections, not resilience.

Bulkhead vs. circuit breaker

	Circuit Breaker	Bulkhead
Watches	Failures (errors, timeouts)	Occupancy (who is holding how much)
Triggers when	The dependency is failing	Something is hogging the capacity
Action	Stop calling completely; check again later	Limit the share; reject the overflow
Sees "everything succeeds, slowly, everywhere"?	No — there's nothing to count	Yes — fullness shows up
Sees "dependency is hard down"?	Yes — drops calls to zero quickly	Partly — limits the stuck threads

Pick the breaker when the dependency is failing, and the bulkhead when sharing is the danger. In practice, stack them. The bulkhead limits how many threads can get stuck in a bad dependency while the breaker is still collecting evidence. Once the breaker trips, that compartment drains. Hystrix shipped the two together for exactly this reason. Where it meets rate limiting: rate limiting controls *outside clients by who they are*, at the front door. Bulkheads control *internal sharing by compartment*.

Real-world examples

- **Netflix Hystrix** made thread pools per dependency famous — a single hung dependency could trap only its own small pool. **Resilience4j** carries the pattern on, defaulting to the lighter counter approach.
- **Kubernetes** is bulkheads built into a platform: CPU/memory limits per *pod* (a unit of deployed containers), separate node pools for batch versus interactive work, and quotas for each team.
- **Envoy** enforces connection caps per upstream inside the mesh — bulkheads with no application code at all.
- **Database connection pools split by service** (through PgBouncer, a connection-pooling proxy for PostgreSQL, or per-deployment limits) keep one busy service from grabbing every PostgreSQL connection.
- **For AI engineers:** multi-tenant GPU serving is today's biggest *noisy-neighbor* problem (where one tenant's load hurts everyone on shared hardware) — a single customer's batch embedding job can starve every interactive chat. Put interactive and batch work in separate GPU pools (or give batch a limited number of quota slots). Give each pricing tier its own compartment, so free-tier load can't push out paying customers. And add concurrency caps per tenant at the LLM gateway. It's the same rate × duration arithmetic, just counted in GPU slots.

In the interview. It shows up as "one endpoint suddenly gets 100× traffic — what happens to the others?", "how do you handle a noisy neighbor in a multi-tenant system?", or "what does one slow feature do to a shared thread pool?"

> **30-second answer:** I split thread, connection, or node pools by feature or by tenant, and I size each one with rate × duration plus some spare capacity. A full compartment rejects only its own overflow. That protects important work even when the competing requests all *succeed* — something a circuit breaker can't detect, because it counts failures. Hystrix's thread pools per dependency are the best-known implementation.

- *Follow-up: "How do you size the compartments?"* — Rate × duration for each compartment, plus headroom for bursts. Then treat "one pool keeps rejecting while overall capacity sits idle" as the signal to re-split. Admitting that the split drifts out of date shows you've actually operated one.
- *Follow-up: "Bulkhead or circuit breaker here?"* — Diagnose it out loud. If the dependency is returning errors or timing out, use a breaker. If legitimate, successful load is taking over shared

capacity, use a bulkhead. Production systems stack both.

- *Mistake that fails candidates:* prescribing a circuit breaker for a hogging problem, or treating green error dashboards as proof that important features are getting capacity.

Self-check:

- Q1: What does a bulkhead partition, what triggers it, and how does that differ from a breaker? → It divides shared capacity (threads, connections, memory, machines, GPU slots) into walled pools and reacts to *fullness*. A breaker reacts to evidence of *failure*. So bulkheads catch hogging that looks like success, which breakers can't see.
- Q2: Overnight, checkout's pool is 90% idle while order history rejects; a teammate wants to merge pools. What's the real choice? → Efficiency versus worst-case protection. Merging gets back the idle capacity but brings the original failure mode back too. The teammate is right only if the workloads are equal in importance and shape, capacity autoscales quickly, or the pool is too small to split. With revenue-critical checkout in the mix, idle threads are the price of protection.
- Q3: A multi-tenant **SaaS** (software-as-a-service: one hosted product shared by many paying customers) with 500 customers; one ships a buggy integration opening 2,000 concurrent requests; everyone slows down; error rate near zero. Diagnose and fix at two layers. → It's hogging that looks like success, starving 499 tenants. At the front: rate limits and concurrency caps per tenant, answering the excess with 429. On the inside: bulkhead the shared pools by tenant tier (or per tenant for the biggest ones), sized by rate × duration. And alert on per-compartment fullness, not only on errors.

Related patterns: Circuit Breaker (cuts off failing dependencies while the bulkhead limits the damage in the meantime — stack them); Timeout (limits how long each slot is held — bulkheads without timeouts fill up and stay full); Rate Limiting (limits clients by who they are, at the boundary); Graceful Degradation (decides what a rejected request turns into).

TL;DR

- **Problem:** shared pools let any single feature, tenant, or dependency legally take over everything, starving the rest while error rates stay at zero.
- **Mechanism:** divide capacity into walled pools per consumer (counters, separate pools, or physical separation), sized by rate × duration; a full compartment rejects its own overflow quickly.
- **Costs:** idle capacity that can't be lent out, sizing that goes stale, and more pools to monitor.
- **Skip it when:** the service does one thing, the pool is too small to split, or the workloads are uniform and autoscale elastically.

Dead Letter Queue

Original lesson: Dead Letter Queue on DesignGurus ↗

What it is. A **dead letter queue (DLQ)** is a separate queue that holds messages which keep failing. Once a message has failed processing a set number of times, the queue system pulls it out of the main line, together with notes about how it failed. The main line keeps flowing. Meanwhile the failed message is kept safe, so someone can inspect it, fix the cause, and replay it. *Analogy:* the postal service's "dead letter office" keeps undeliverable mail safe, rather than throwing it out or letting it jam the sorting line.

Terms first. A *message queue* stores messages from a *producer* until a *consumer* processes them. When the consumer finishes, it sends an *acknowledgment (ack)*, which tells the queue to delete the message. The *broker* is the queue system itself (SQS, RabbitMQ, Kafka). Under at-least-once delivery, any message that isn't acked gets delivered again.

The problem. Picture an order-fulfillment pipeline. Checkout publishes an event, and a consumer reserves inventory and notifies the warehouse. It handles **200 messages/second** and has run flawlessly

for months. Then at **2:47 AM** a partner publishes an order whose shipping address is **null**. The consumer throws an **exception** (the error signal code raises when it can't continue) and doesn't ack, so the broker delivers it again — forever. This is not a transient failure. Malformed input fails *deterministically*: attempt 4,000 fails for the same reason as attempt 1. A message like this is called a **poison message**. Since processing is ordered, everything behind it waits. At 200 msg/s, that's **720,000 stuck orders per hour**. A single null field halted every shipment in the company.

A teammate's emergency fix — catch the exception, ack it, and move on — gets things flowing again but actually makes matters worse. The next malformed message simply gets deleted, leaving no record that can be replayed. The following weekend, **40 paid orders vanish**: they never ship, and there's no payload left to investigate. So a poison message forces an awful choice. Either block ordered processing forever, or quietly delete business data. The DLQ is the third option.

Why the obvious fixes fail

- **"Retry harder."** Backoff and jitter help with failures that might clear. A message that can't be parsed fails every single time, and each extra retry takes capacity away from healthy messages.
- **"Catch, log, and skip."** Better than deleting silently, but logs may be sampled, rotated, cut short, or not allowed to store full payloads. And you can't replay them through the pipeline once a fix is in. The order is still lost — you just have a record of losing it.
- **"Validate everything at the producer."** Worth doing, but it can't stop consumer bugs, valid-but-unusual edge cases, or rejections further downstream. Some messages will always fail during processing, and the design has to say where they go.

How it works. In the figure below, the producer writes to the main queue and the consumer processes each message. Success leads to an ack. A failure with attempts < 5 sends the message back to the main queue to be delivered again. On the 5th failure it goes to the dead letter queue. When the DLQ's depth is > 0 an alert fires. An engineer looks at the payload and the error. Once a fix is deployed, the message is redriven back to the main queue.

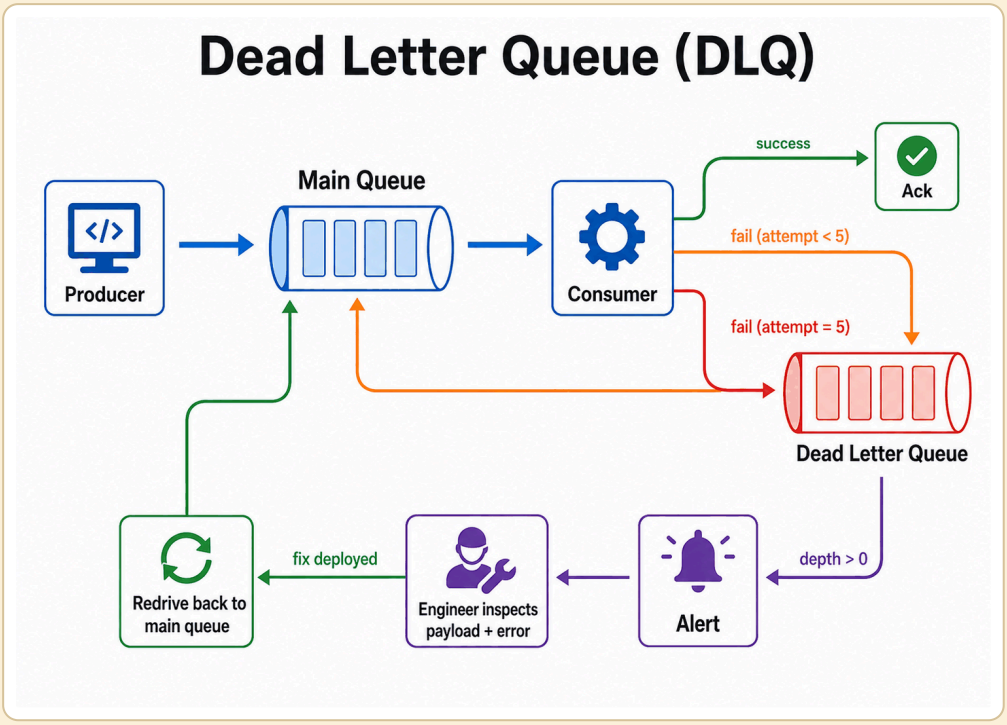


Figure — Dead Letter Queue (course diagram)

Four pieces make it work (only the first one is really code):

1. **A limit on delivery attempts, enforced by the broker.** The broker counts how many times each message has been delivered. At the limit (SQS calls it `maxReceiveCount`, usually **3–5**), the broker itself moves the message to the DLQ. This has to be the broker's job, not the consumer's. The failure you're handling may be the consumer crashing on this very message — and code that can't survive a payload can't be trusted to route it.

2. **The failure notes travel with the message:** source queue, exception, attempt count, and the first and last failure times. Without context, a DLQ entry is a puzzle. With context, it's a bug report.
3. **An alarm on the DLQ.** A DLQ that nobody watches only *postpones* deletion. DLQs have retention limits (**14 days on SQS**), and after that, messages are deleted automatically. Alert the moment an important DLQ stops being empty.
4. **A way back (redrive).** Once the fix ships, the messages are *redriven* — sent back to the main queue to be processed for real. Logging could never give you this. And since a redriven message counts as a redelivery, consumers must be idempotent.

The DLQ pulls apart two jobs that the emergency fix had jammed together. The queue has to keep healthy messages moving within milliseconds, while a person may need hours to make sense of strange input. The DLQ holds that input in the meantime.

Walkthrough with numbers (5 attempts with backoff, then dead-letter)

- **2:47:00** — the null-address event fails attempt 1; attempts 2–5 fail over about 30 seconds.
- **2:47:30** — the broker moves it to the DLQ, along with its exception and attempt history. The consumer moves on straight away.
- **Backlog math:** 30 s × 200 msg/s ≈ **6,000 messages** pile up behind it. They clear at normal speed in well under a minute — compared with 720,000 per hour with no end.
- **2:47:31** — the DLQ alarm fires. On-call reads the **payload** (the message's actual contents) and **stack trace** (the list of function calls active when the error happened) and spots the partner's null field. The fix (validation at the edge plus a consumer patch) ships in the morning.
- **9:15 AM** — one click redrives the message. The order ships a few hours late rather than never.
- **Total impact:** one order delayed, zero lost, and a 30-second slowdown instead of an all-night outage — all for the cost of one extra queue and one alarm.

Trade-offs

- *You gain:* the poison message no longer blocks processing and remains available; a proper structured, searchable record (payload, error, history), which logs can't give you; replay — fix the bug, redrive, and the missed work really happens; and DLQ depth, one of the most honest health signals a pipeline has.
- *You pay:*
  - **One more queue to own.** If nobody watches it, it turns immediate, visible loss into delayed, silent deletion.
  - **Broken ordering for the parked message.** Message 47 now runs after 48 through 6,000 — that's fine for independent events (orders, emails), but not for streams where order carries meaning.
  - **A judgment the counter can't make.** An attempt count can't tell bad input apart from a healthy message caught by a ten-minute downstream outage. A poorly chosen threshold can dump thousands of valid messages into the DLQ (see self-check Q2).
  - **Redrive tooling, plus a data-retention question:** payloads containing personal data (PII) now live in a second place.

When not to use it

- **Strictly ordered streams** — for example, per-customer sequences where event 48 must never run before 47 (account ledgers, state machines). Skipping a message corrupts the data. The right move is to **pause that stream** (stop the **partition**, the ordered slice of the queue that holds that key's messages), alert loudly, and **fix forward** (fix the code and resume without skipping anything). Other streams keep going. A DLQ in that situation becomes a data-corruption feature.
- **Throwaway data** — high-volume telemetry where losing one reading changes nothing. Drop it and count it.

- **Before retries are tuned.** Dead-lettering on the very first failure turns every transient blip into an engineering ticket. The DLQ is where messages go *after* the retries have run out.
- **As a route for business logic.** "Payment declined" is a normal, expected result, and it deserves its own flow. Save the DLQ for the unexpected, so that its alarm always means something is truly wrong.

DLQ vs. retry-queue tiers. Redelivering in place with backoff works when the gaps are short. But if a downstream dependency has outages lasting an hour, tying up a consumer slot for that hour is wasteful. Some systems (especially around Kafka) use **retry tiers** instead. A failed message goes to a `retry-5m` queue, whose consumer waits five minutes and then retries. If it fails again, it moves up to `retry-1h`. Only after that does it get dead-lettered.

Situation	Reach for
Failures clear within seconds (transient, brief contention)	In-place redelivery with backoff, then DLQ
Failures need minutes to hours (downstream outages, rate limits)	Retry tiers, with the DLQ as the last stop
Deterministic failure (malformed payload, code bug)	Straight to the DLQ — waiting won't help
Ordering per key must be kept	Neither one can skip safely — pause the stream

These aren't competing choices; they're steps in a single sequence: quick redelivery → spaced-out tiers → DLQ. Most systems need the first and the last. Add the middle step only when outage-length failures happen often.

Real-world examples

- **AWS SQS** (Amazon's managed queue service) brought the pattern into the mainstream: a redrive policy per queue, automatic moves done by the broker, and a redrive button in the console (AWS's web control panel).
- **RabbitMQ** generalizes it as *dead letter exchanges*. Combined with delays per queue, that's also how RabbitMQ builds retry tiers.
- **Kafka** has no DLQ built in. Its ecosystem (Kafka Connect, Spring Kafka) provides dead-letter topics, and the warning about strict ordering applies fully.
- **Google Cloud Pub/Sub** includes dead letter topics, with tracking of delivery attempts built in.
- **For AI engineers:** LLM pipelines generate poison messages all the time — invalid **JSON** (the common text format for structured data), documents that break embedding, moderation rejects — and each of them fails deterministically. During RAG ingestion (loading documents for retrieval-augmented generation), one corrupt PDF in a 100,000-document crawl should go to a DLQ for a person to review, while the other 99,999 get indexed. The DLQ also serves as the review queue.

In the interview. As soon as you draw a queue, expect: "what happens when a message can't be processed?" or "one malformed message is crashing your consumer — walk me through the next ten minutes." Recognizing the phrase *poison message* immediately marks you as someone who has operated a queue.

> **30-second answer:** Under at-least-once delivery, a malformed message gets delivered again forever and blocks everything behind it, while dropping it is silent data loss. So after a few failed attempts, the broker moves it to a dead letter queue together with its error details. The main line keeps flowing, nothing gets lost, and an alarm fires as soon as the DLQ is non-empty. Once we fix the bug, we redrive the messages. SQS's redrive policy is the standard example. The one rule to remember: a DLQ nobody watches is just data loss on a timer.

- *Follow-up: "What's your process for what lands in the DLQ?"* — Alarm at depth ≥ 1 for critical flows; inspect the payload and the exception; fix; redrive. Treat any steady flow into it as a defect, never as normal operation. And mention the retention limit — "parked" doesn't mean "safe forever."
- *Follow-up: "Your events are ordered per customer — does dead-lettering still work?"* — No. Skipping a message breaks the sequence. Pause the affected stream and fix forward. Only dead-letter where events are independent.
- *Mistake that fails candidates:* "I'd retry until it succeeds" (a deterministic failure never succeeds), or describing a fancy DLQ without saying who watches it.

Self-check:

- Q1: What moves a message to the DLQ, why must the broker do it, and what should travel with it? → The count of delivery attempts hitting its limit (say 5). The consumer might be the thing that's crashing, so it can't be trusted to do the routing. Carry the source queue, exception, attempt count, and first/last failure times.
- Q2: A downstream dependency is down for ten minutes; the queue is set to 5 attempts over 30 s, then DLQ. What happens and how do you fix it? → Every message that arrives during the outage uses up its attempts within 30 s and gets dead-lettered. Thousands of healthy messages flood the DLQ, hiding any real poison and forcing a bulk redrive. Fix: stretch the retry window past realistic outage lengths (retry tiers), and put a circuit breaker on the consumer's downstream call so consumption pauses during a hard outage.
- Q3: A payments DLQ set up eight months ago: 12,000 messages passed through, 300 remain, no incident was ever raised. What happened? → Nobody was watching it. The 14-day retention quietly deleted ~11,700 payment events, and an auditor found it instead of an alarm. What was missing: an alarm on DLQ depth/inflow, a named owner with an inspect-fix-redrive *runbook* (written steps), and a retention window long enough for the response process.

Related patterns: Message Queue (at-least-once delivery is what makes endless redelivery — and so DLQs — necessary); Retry (handles failures that pass; the attempt limit is where the two patterns meet); Idempotency (redrive is a redelivery — consumers must deduplicate); Circuit Breaker (placed on the consumer's downstream calls, it keeps ordinary outages from looking like thousands of poison messages).

TL;DR

- **Problem:** under at-least-once delivery, a poison message forces an awful choice — redeliver forever and block everything, or drop it and silently lose data.
- **Mechanism:** after N failed attempts, the broker moves the message, with its failure details, to a separate queue; an alarm fires when that queue isn't empty; messages are redriven once the fix is in.
- **Costs:** one more queue to own, broken ordering for parked messages, an attempt counter that can't tell poison from an outage, and the trap that an unwatched DLQ is just loss that happens later.
- **Skip it when:** ordering is strict (pause instead), the data is throwaway (drop it and count it), or the "failure" is a normal business outcome.

Graceful Degradation

Original lesson: Graceful Degradation on DesignGurus ↗

What it is. Graceful degradation means that when part of a system fails, the system gives back a *lesser* answer rather than an error. Optional features vanish or show older data. Meanwhile, the few critical operations that must never be wrong refuse honestly. You get there by ranking every feature by how critical it is, and giving each one a sequence of fallbacks planned in advance. That way, failures cost you *quality* rather than *availability*. *Analogy:* when a restaurant's grill breaks, a good kitchen

doesn't shut down. It takes the grilled dishes off the menu and keeps serving everything else. But it will never serve undercooked chicken just to fill an order.

The problem. A homepage makes **12 service calls** — catalog, cart badge, recommendations, reviews, pricing, promotions, recently viewed, and more. Each one is guarded by a timeout, controlled retries, and a circuit breaker. Recommendations goes down. Its breaker opens within a second and rejects calls instantly, exactly as intended. And yet the homepage still shows an error page. Why? The calling code has no handling for an "open breaker" result. So it throws an exception, and that exception kills the whole page **render** (the step that assembles the final page sent to the browser). The breaker did its job. The application just had no alternative answer ready.

The math makes it worse. Each dependency is **99.9%** available, but the page only renders if all twelve succeed: $0.999^{12} \approx$ **98.8%** page availability. That missing 1.2% comes to about **104 hours of homepage downtime a year**, mostly caused by optional widgets. The rule: *every dependency without a fallback turns into a required one*, and page availability becomes all of their availabilities multiplied together. The earlier patterns decide when to stop calling. None of them decides what the user gets after that.

Why the obvious fixes fail

- **"Make every dependency five-nines."** (*Five-nines* = 99.999% uptime.) Even twelve services at 99.99% still multiply down, and one bad deployment wrecks the estimate. Better targets don't change the "all twelve must succeed" structure.
- **"Wrap every call in try/catch and return null."** (*try/catch* is the code construct that intercepts an exception; *null* means "no value".) This treats the optional reviews widget and the essential shopping cart exactly the same, and tells the user nothing. Worse, it can hide critical failures — for example, a checkout that quietly skips reserving inventory and ends up with inconsistent data. It sidesteps the central decision of this pattern: which features are required?
- **"Cache everything."** A cache can be one step in a fallback sequence. But caches serve reads, not writes, and they need explicit *staleness limits* (how old an answer is allowed to be). A cache miss during an outage still needs some other response.
- What's really needed: engineering **and product** must rank each feature and pick its fallback *before* the incident happens.

How it works — two parts, and the first isn't code

1. **Criticality tiers, decided in a meeting with product.** Put every feature into one of three buckets. *Critical* — the page means nothing without it (catalog, checkout, login). *Important* — clearly worse without it (pricing detail, search filters). *Optional* — nobody would open a ticket about it (recommendations, recently viewed). That meeting *is* the pattern. The code only enforces what the meeting decided.
2. **A fallback sequence for each feature** — an ordered list of answers, each a step lower than the last. The figure below shows the standard ladder. (1) A fresh, personalized answer at full quality → if the dependency fails, (2) a cached copy that's seconds to minutes old → on a miss or if it's too stale, (3) a static default such as "popular this week" or the list price → if no sensible default exists, (4) hide the feature and render the page without it → and, only for critical features, (5) refuse clearly and loudly ("nothing was charged, nothing was faked"). The lower half of the figure shows how far each tier is allowed to fall. Optional features may go all the way down to "hidden." Important ones stop at a static default. Critical ones (checkout, login, payment capture) never serve a lesser answer — they go straight to refusing. How far a feature may drop is agreed with product *before* the incident, not decided by an exception handler during it.

Figure — Graceful Degradation (course diagram)

Examples: *recommendations* → a personalized list, otherwise a precomputed "popular this week" list, otherwise hide the row. *Pricing* → the live price, otherwise a cached price within a staleness limit the business signed off on, otherwise fail — because a wrong price isn't just cosmetic.

Whole-system versions of the same idea

- **Read-only mode:** people can browse, while writes are put on hold.
- **Queue-writes-for-later:** accept the action, place it durably on a message queue, and confirm by email once it's done.
- **Priority shedding:** under overload, drop optional traffic first so that critical traffic survives.

Two operating rules that separate real implementations from decorative ones

- **Alarm on the inside, degrade smoothly on the outside.** Degraded mode has to page the on-call engineer even when users see a perfectly acceptable page. Otherwise the root cause never gets fixed, and the "temporary" fallback quietly becomes permanent.
- **Fallbacks rot unless you exercise them.** An else-branch that never runs will be broken by the time you need it. Netflix runs **Chaos Monkey** (a tool that breaks things in production on purpose) to make fallbacks run for real. **Game days** (scheduled practice outages) are the gentler version.

Walkthrough with numbers (catalog critical, 99.95% available on its own; the other 11 fragments each get a ladder ending in "hide")

- **The recommendations outage, again.** The breaker opens, and the fallback serves the hourly "popular this week" list in **~2 ms**. The page looks complete. Internally, a `degraded_mode` metric switches on and on-call sees it. Conversion drops a measured **~1%** instead of falling to zero.
- **Availability, rebuilt.** Now the page hard-fails only if catalog fails: 99.95% ≈ **4.4 hours/year** of downtime, down from **104**. None of the dependencies became more reliable — 11 of them simply lost the power to take the page down. Availability went from twelve numbers multiplied together to just one.
- **The write side.** During a 20-minute incident on the payments database, checkout switches to queue-writes mode. Orders are accepted, given idempotency keys, queued durably (so they survive a crash), and confirmed by email minutes later. Revenue keeps coming in through what used to be 20 minutes of zero sales.

Trade-offs

- *You gain:* whole-system availability stops depending on optional dependencies; incidents get gentler (quality drops gradually, which buys responders time); and you end up with an honest written map of what matters, which sharpens later decisions about capacity and spending.
- *You pay:*
  - **Fallback code is real code.** It has to be written, reviewed, and above all exercised — or it's a second bug hiding behind the first.
  - **Staleness with a name attached.** Cached prices and yesterday's inventory each need a business owner who has approved how wrong they're allowed to be.
  - **The masking hazard.** A fallback can hide a failure so well that nobody looks into it. The site could serve "popular this week" for a whole quarter unless degraded mode raises an internal alert.
  - **Many states to test.** N features × M fallback steps. Testing them takes deliberate *fault injection* (causing controlled failures and checking each response).

When not to use it — things that must fail closed (an honest, immediate "no," never a lesser answer)

- **Writes where correctness is critical:** capturing a payment, committing inventory, calculating a dosage, executing a trade. A wrong or half-done answer is strictly worse than a refusal. Queuing the request *without executing it* for later can be fine. Pretending it ran never is.
- **Authentication and authorization.** An auth outage never justifies "let them through" or "trust the old answer" — degraded security has a name, and it's a breach. Auth degrades in only one direction: refuse.
- **Compliance flows:** audit trails, consent checks, regulatory reports.
- **When stale costs more than down:** cached prices in a fast-moving market can lose more money than an honest maintenance page.
- *Common misuse:* degrading without any metric, alert, owner, or exit plan — the fallback quietly becomes permanent product behavior.

Degradation vs. failover. *Failover* means switching over to a standby copy of the failing dependency. Both approaches answer "the dependency is failing — now what?", but in opposite ways:

	Failover	Graceful Degradation
The move	Swap out the dependency (replica, standby, another region)	Swap out the answer (cached, default, hidden)
Quality afterward	Full	Lower, on purpose
Cost paid in advance	Extra capacity sitting idle	Fallback code and product decisions
Fails when	The whole class is down (a bug in every replica, the provider itself)	No sensible lesser answer exists

They're steps in one sequence. Fail over first (users never notice), then degrade once failover runs out (users barely notice). A primary-replica database promoting a follower is failover. This lesson adds the plan for the day when promotion isn't enough.

Real-world examples

- **Netflix:** when personalization fails, members see popular titles that aren't personalized instead of an error. Chaos Monkey deliberately triggers those fallbacks in production to check them.
- **Amazon** degrades recommendations, reviews, and even search, rather than let anything touch checkout. "Add to cart must always work" has shaped its architecture for two decades.

- **Cloudflare Always Online** serves cached copies of pages when their **origin servers** (the site's own servers, behind the CDN) are down — degradation sold as a product.
- **Facebook** has turned to read-only mode several times during major incidents.
- **For AI engineers:** an LLM gateway can fall back from the main model to a smaller or older one, then to a **semantic cache** (which matches cached answers by meaning rather than exact text), before finally saying "try again shortly." When a RAG system's retrieval is down, it can answer from the base model with a visible "sources unavailable" label. The label matters: quietly dropping the sources degrades *correctness*, which may belong in your fail-closed tier. Under GPU pressure, shed quality first (smaller **token** limits, where tokens are the word-pieces models read and write; trimmed **context**, the text sent to the model with each request) before you start dropping requests.

In the interview. The direct version: "Service X is down — what does the user see?", usually right after you mention a breaker or bulkhead. Most candidates stop at "fail fast" and never ask *into what?* The version that signals seniority: **"Which parts of your design are allowed to fail?"** A candidate with tiers has an answer. One without them discovers, live, that they accidentally made everything critical.

> **30-second answer:** Together with product, I rank features by how critical they are, then give each dependency a fallback sequence: fresh, otherwise cached, otherwise a static default, otherwise hide the feature. Only truly critical things are allowed to fail, and they fail loudly. The system gives up quality rather than availability — recommendations fall back to popular items while checkout stays untouchable, the way Netflix does it. Degraded mode raises an alarm internally even while things look smooth externally, and we exercise fallbacks with game days, because an else-branch that never runs is already broken. Auth and payment capture fail closed instead: a lesser answer there means a breach or a double charge.

- *Follow-up: "Who decides what's critical, and how?"* — Product and engineering together, before the incident. Features on the revenue path and features bound by correctness are critical. Enhancements get ladders. Say clearly that the meeting is the pattern, not the try/catch.
- *Follow-up: "How do you know your fallbacks still work?"* — Scheduled fault injection: game days, chaos tooling, or continually sending a small share of real traffic down the fallback paths. Plus a metric and an alert for each degraded state, so degradation is always tracked and has an exit.
- *Mistake that fails candidates:* a design in which every dependency is secretly critical — or the reverse, degrading something that must fail closed ("if auth is slow we can skip the check" has ended interviews).

Self-check:

- Q1: Name the fallback steps in order; who assigns criticality, and why not engineering alone? → Fresh → cached/stale → static default → hide → (critical only) an honest, loud failure. Product and engineering together, because criticality is a business judgment about revenue, trust, and legal weight. Engineering on its own tends to rank by how hard things are to build, or to make everything critical.
- Q2: Degradation works so well that a recommendations outage causes zero complaints and no visible revenue dip. Why is that dangerous and what keeps it safe? → It hides the failure, so the root cause can sit there for a quarter. Keep it safe with a degraded-mode metric that alerts internally, plus an owner and an exit plan for every degraded state.
- Q3: During an auth brownout a teammate proposes caching each user's last auth answer for an hour and letting known session cookies through if auth stays down — "like recommendations." Evaluate. → The two halves land on opposite sides of the fail-closed line. Briefly caching auth answers under a limit owned by security (including the "no"s) can be defended. Letting requests through while auth is down is failing open on a security boundary — a breach. The real mistake is

treating auth like recommendations. For some features the worst case is "missing"; for others it's "wrong." Auth only ever degrades toward refusing.

Related patterns: Circuit Breaker (decides when to stop calling; the fallback is what runs while it's open); Bulkhead (rejected overflow lands on a fallback step rather than an error page); Cache-Aside (the stale-copy step, promoted from a performance trick to an availability mechanism); Message Queue (drives queue-writes-for-later).

TL;DR

- **Problem:** without fallbacks, every optional widget quietly becomes load-bearing, and availability becomes a dozen other teams' numbers multiplied together.
- **Mechanism:** rank features with product, plan a fallback sequence for each feature ahead of time (fresh, cached, default, hidden, fail), and alarm internally while degrading externally.
- **Costs:** fallbacks that rot unless exercised, staleness that someone has to own, a masking hazard, and many states to test.
- **Skip it when:** the operation must fail closed — payment capture, auth, compliance, or anywhere a lesser answer is worse than no answer.

Capstone: Hardening a Payment Flow

Original lesson: Capstone: Hardening a Payment Flow on DesignGurus ↗

What this capstone is. Seven patterns, one payment system, and seven incidents, in the order teams usually run into them. Each fix either causes or uncovers the next problem. That is the real lesson here: reliability patterns interact as a system, not as a checklist of separate items. At every stage, stop at the question and decide on the pattern and its key numbers yourself before you read on. That decision-making is exactly what interviews test.

The brief. You've just taken over checkout at a mid-size commerce company, one week before Black Friday. The figure below shows the starting design. The user clicks Pay. The Checkout API (a **200-thread pool**) then calls, synchronously, the Payment Provider (external, healthy **p99 = 400 ms**), the Orders DB (**100-connection pool**), the Email Service, and the Warehouse API. All four happen inside one call, in that order, while the user waits. Traffic: **100,000 orders/day**, peaking at **300 checkout requests/second**

during promotions. The promise: answer within **2 seconds, 99.9% of the time**. Reliability tooling: none — every call uses its HTTP client's defaults.

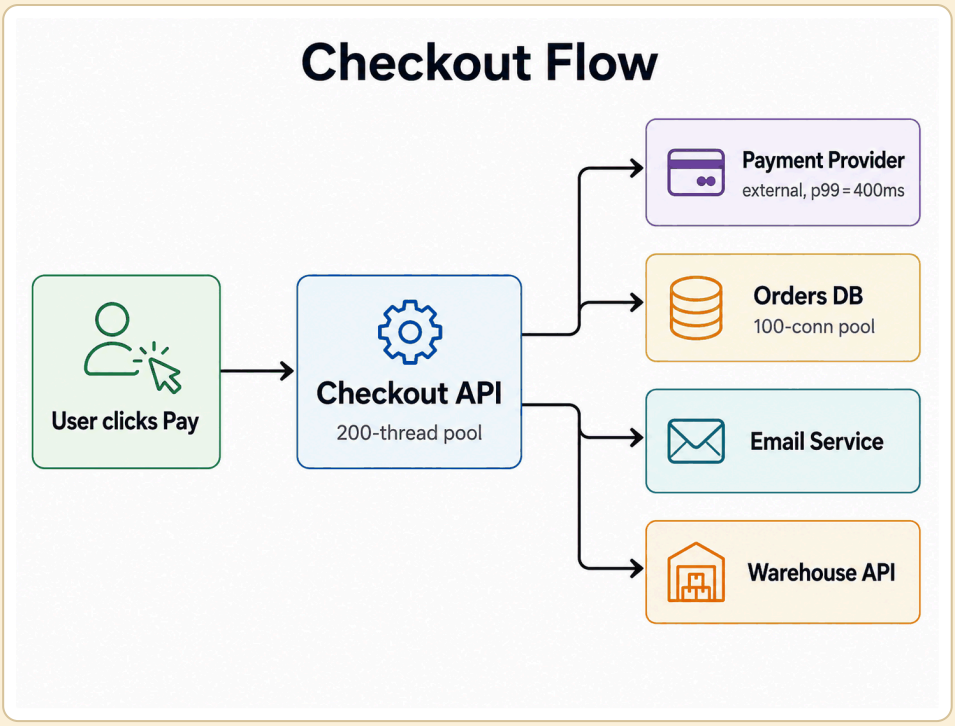


Figure — Payment flow: the brief (course diagram)

Stage 1 — The hang → Timeout

- **Incident.** Monday, 11:40 AM. One of the provider's servers receives a bad deploy and returns *nothing* — connections just sit open. Within **90 seconds**, all 200 checkout threads are tied up, health checks fail, and the load balancer takes your **instances** (running copies of your service) out of rotation. The provider's real error rate: **0.4%**. Your outage: **total**.
- **Pause and decide:** how did a 0.4% problem turn into a 100% outage? Which pattern, and what number?
- **Why it happened.** The HTTP client's default was to wait forever. At 300 req/s, every hung call holds a thread with no end in sight, so a 200-thread pool runs dry in under a second of hung traffic.
- **Decision.** Set deadlines based on measurement: a provider request timeout of **1,200 ms** (3× its 400 ms healthy p99) with a **200 ms** connection timeout; the database at **250 ms**; email and warehouse at **500 ms** each. *Why measured values rather than "30 s":* they cut off genuine hangs fast while still giving slow-but-normal calls room.
- **Budget check — the first design problem.** 1,200 + 250 + 500 + 500 = **2,450 ms**, which doesn't fit inside the 2 s promise. The honest conclusion: email and warehouse don't belong in the synchronous path at all (that turns into Stage 6). For now, cut them to **300 ms** each so everything fits.
- **What it creates.** Hangs turn into visible failures. About 1 in 200 payment calls now fails because of a transient problem that would have worked 100 ms later. You've swapped an invisible problem for a visible one — a good swap.

Stage 2 — Transient failures → Retry with backoff

- **Incident.** **~500 checkouts/day** fail because of dropped packets, a slow provider server, or short connection resets. Support tickets say: "I clicked Pay, got an error, tried again and it worked."
- **Pause and decide:** the failures are transient and random — which pattern, and which three configuration properties?
- **Decision.** Retry failed payment calls **up to 3 times**, with waits that double starting from **~100 ms**, plus **jitter**. Only errors that can change are eligible: timeouts, connection failures, 5xx (any server-error status in the 500s), and 429 (following `Retry-After`). A declined card is an *answer*, not a failure, so it's never retried. Retries happen in **exactly one layer** — the Checkout API — so the app and the gateway can't add their own on top (which avoids 3 × 3 × 3 multiplication).

- **What it creates.** A timeout is uncertain. Three days after retries go live, finance forwards a chargeback: a customer was charged **$499 twice**.

Stage 3 — The double charge → Idempotency

- **Incident.** Roughly a third of payment timeouts happen *after* the provider has already taken the money. Each of those, when retried, becomes a double charge — dozens a day, and each one costs a refund, support time, and a **$15** chargeback fee. The retries are working exactly as designed, and that *is* the problem.
- **Pause and decide:** the retry has no way to know whether attempt one landed. Who can know, and what has to travel with the request?
- **Decision.** The Checkout API creates **one idempotency key at the moment the user clicks Pay** — one per logical payment, stored with the pending order, so even a client that crashed and reloaded retries with the same key. Every attempt sends it. The provider atomically claims the key, charges once, saves the result, and replays it for any duplicates. So the retry after a lost response now gets the original confirmation. The same discipline applies to your own writes: the **order insert is keyed as well**, so work that gets redelivered can't create duplicate rows. *Why on the client side:* only the client knows two requests are the same intent.
- **What it creates.** For once, nothing bad — idempotency is the rare fix in this module that brings no new failure mode. But there's still one case the retry policy can't cope with.

Stage 4 — The sustained outage → Circuit Breaker

- **Incident.** Black Friday, 2:15 PM. The provider slips into a sustained *brownout* (it's up, but too slow to be useful). Every call times out at 1,200 ms, and the retries faithfully fire, tripling traffic against a dependency that's already struggling. Each checkout thread spends about **4 seconds** slowly failing through a full retry sequence. Your API isn't down, but it isn't working either — and you've become part of the provider's problem.
- **Pause and decide:** retries assume the problem is transient, and this one is sustained. Which pattern, what are its three states, and what exactly should it count?
- **Decision.** Put a breaker around the provider call: a rolling window of **20 calls**, a **50%** failure threshold, a **30 s** cooldown, and **5 half-open test calls**. It counts **only final outcomes, after retries**, so the two mechanisms don't skew each other's signals (breaker on the outside, retries on the inside).
- **Result.** About **90 seconds** into the brownout, the breaker opens. Checkout rejects provider calls in **1 ms instead of 4 s**, payment-thread usage drops close to zero, and the provider receives no new traffic. About **10 minutes** later, successful test calls close it automatically.
- **What it creates.** Two questions you can't avoid. (1) While the breaker was open, users got an *instant* error rather than a slow one — but it's still an error (→ Stage 7). (2) The **postmortem** (the written review after an incident) notices that everything slowed down during the brownout, even order-status lookups that never touch the provider, because every feature shares the same thread and connection pools (→ Stage 5).

Stage 5 — The success-shaped flood → Bulkhead

- **Incident.** Cyber Monday. Finance runs its end-of-day reconciliation batch against the refunds endpoint, which lives in the same service: **800 req/s at ~600 ms each**, and every one succeeds. Error rate **0.0%**. Breakers are correctly closed. Checkout is starving for threads. The arithmetic: 800 × 0.6 = **480 threads** wanted, but the service has **200** shared. Every dashboard looks healthy while sales drop.
- **Pause and decide:** nothing is failing anywhere, so Stages 1–4 can't see this. Which pattern, and what arithmetic sizes it?
- **Decision.** Split the pools by feature and priority. **Checkout 100** (300 req/s × 250 ms = 75 needed, plus headroom). **Refunds 40**, which limits reconciliation to about **65 req/s** — the batch now takes **3 hours instead of 1**, a trade-off *agreed with finance* rather than one they stumble on. **Order-status 40**, **reserve 20**. Apply the same split to the DB connections. When reconciliation fills its 40, its overflow waits, and checkout doesn't notice a thing. New alert: **per-compartment fullness**, since error-rate alerts can't see floods that look like success.

- **What it creates.** With heavy work capped, the pressure to move slow work out of the synchronous path can no longer be ignored. Email (300 ms) and warehouse (300 ms) still sit inside the user's 2 s, and they can still fail a checkout *after* the card has been charged.

Stage 6 — The poison message → Dead Letter Queue

- **Change.** Work after payment moves to async. Once the charge succeeds and the order is committed, the API publishes an `order.paid` event. Consumers send the receipt, notify the warehouse, and award loyalty points. The synchronous path shrinks down to **charge + order insert** — fast, and a failing email can no longer kill an order that's been paid for.
- **Incident.** Three weeks later, a partner order comes in with a **null shipping address**. The warehouse consumer throws, and the queue redelivers it, forever. Every order behind it waits. Meanwhile checkout at the front keeps running, with no idea that fulfillment has stopped across the whole company.
- **Pause and decide:** the failure is deterministic. Redelivery can't fix it, and dropping it loses a paid order. Which pattern, and which *non-code* part makes it real?
- **Decision.** Every consumer queue gets a redrive policy: **5 delivery attempts with backoff**, after which the broker moves the message to a DLQ along with its exception and attempt history. The line unblocks in about **30 seconds** instead of never. The non-code part: **an alarm on the DLQ, sent to the owning team, with a runbook** (inspect, fix, redrive). Without that, the DLQ is data loss on a **14-day** retention timer. Consumers are idempotent using Stage 3's keys — for example, the receipt is keyed on the order ID — so redriven messages can't send twice or ship twice.
- **What it creates.** A changed business rule: an order can now be paid for but not yet reported to the warehouse for several minutes. Product and engineering have to accept that delay explicitly — and the final stage turns decisions like that into a system.

Stage 7 — The question nobody answered → Graceful Degradation

- **Incident.** The provider goes down for **20 minutes**, and every mechanism works perfectly. Timeouts expire, retries stop, the breaker opens, bulkheads hold, and queues drain. Yet every click on Pay returns an instant error page, losing **$40,000** of purchase intent. The review asks what users *should* have seen — and it turns out nobody had ever decided.
- **Pause and decide:** this isn't a missing mechanism. What's missing, which parts of checkout are allowed to degrade, and which must fail closed?
- **Decision (made with product).** Payment capture is **fail-closed** — never fake a charge, and never assume it worked. But failing closed doesn't mean failing rudely, or failing everything:
  - **The rest of the site stays up.** Cart, browsing, and order history never touch the provider. Bulkheads plus fallbacks keep them running.
  - **The cart is protected.** The Pay button turns into a message: "Payments are briefly delayed; your cart is saved, and we'll email you when we're back." The cart's state is kept on the server, and one click signs the user up for the notification.
  - **Where the business accepts the risk, capture is postponed.** For trusted repeat customers below a spending threshold, checkout switches to **authorize-later**. (*Authorizing* asks the bank to approve and hold the amount now; the capture comes later.) The order is accepted, queued durably with its idempotency key, and captured once the breaker closes. It carries an explicit "payment processing" status, and an email goes out either way. Finance approved that risk budget *before* the incident — that meeting was the pattern.
  - **Alarm internally, degrade smoothly externally.** A degraded-mode metric pages on-call as soon as it starts, and the state has an owner and an exit plan.
- **Result.** In the next 20-minute outage, about **70% of would-be buyers convert within the hour**, thanks to saved carts and postponed capture.

The final architecture. The figure below brings it all together. The Checkout API is split into bulkheaded pools (checkout 100, refunds 40, status 40 + 20 reserve). The provider call is wrapped with a 1,200 ms timeout, ×3 jittered retries, an idempotency key and a circuit breaker, plus a dashed path to degraded mode (saved cart, notify-me, postponed capture) for when the breaker is open. The Orders DB

has a 250 ms timeout, keyed inserts and partitioned connections. And an `order.paid` queue feeds the email, warehouse and loyalty consumers, each with 5 attempts and then a DLQ with a depth alarm.

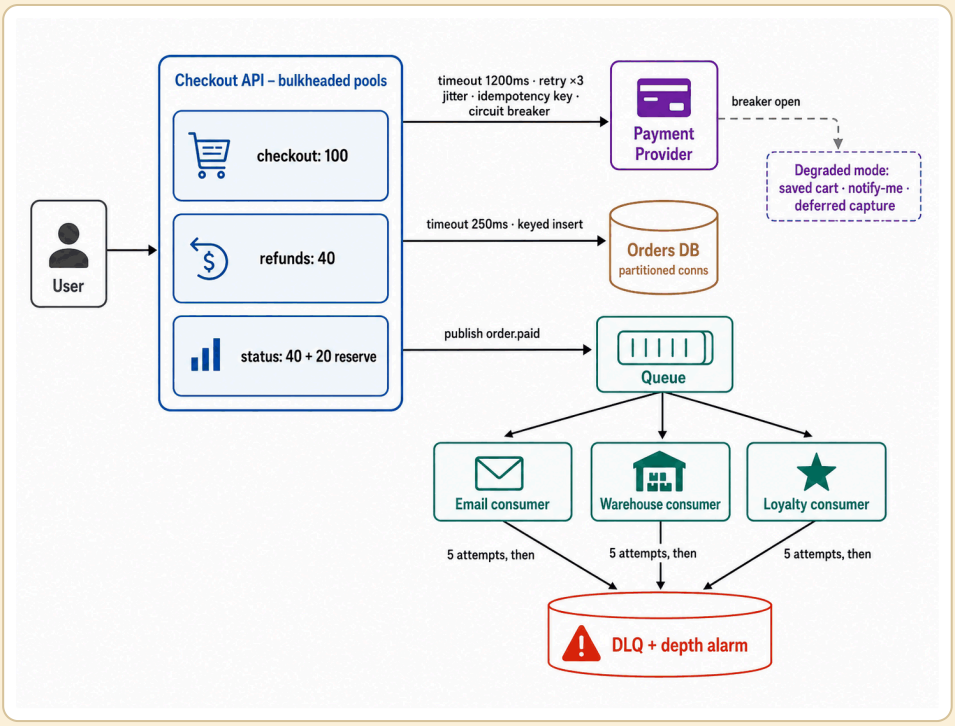


Figure — Payment flow: final architecture (course diagram)

Debrief: the chain, named

Stage	Symptom	Pattern	The number or decision that mattered
1	Hangs eat up every thread	Timeout	3× healthy p99, budgeted along the chain
2	Transient failures kill real checkouts	Retry + backoff	Jitter; only retryable errors; one layer
3	Retries charge twice	Idempotency	Key made on the client, before attempt one
4	A sustained outage wastes threads on hopeless calls	Circuit breaker	Counts outcomes after retries; 30 s cooldown
5	Green dashboards, starving checkout	Bulkhead	Rate × duration per compartment; fullness alerts
6	A poison message halts fulfillment	Dead letter queue	The alarm and runbook, with an owner
7	Flawless machinery, error page anyway	Graceful degradation	Fail-closed capture; tiers signed off by product

Read the Pattern column from top to bottom — that's your interview answer to "how do you make a payment system reliable?" And every row exists because the row above it shipped. Timeouts created visible failures → retries created duplicates → idempotency made retries safe → sustained failure beat retries, hence the breaker → the breaker couldn't see a flood that looked like success, hence the bulkhead → the bulkhead pushed slow work onto queues → queues brought the poison message, hence the DLQ → and once every mechanism worked, one human question was left: *what should the user see?* Take that question into every design you do. Don't stop at "what happens when it fails?" Ask **"what happens instead?"**

> **Interview one-liner:** Timeouts make hangs visible, jittered retries soak up blips, idempotency keys make those retries safe, a breaker stops storms caused by sustained failure, bulkheads contain floods that look like success, async work gets a DLQ that someone watches, and a degradation policy agreed with product decides what users see — with payment capture failing closed.

Extend it yourself

- **Multi-provider:** add a second payment provider. When should the breaker trigger a switch of provider, and what does idempotency mean across two providers that don't share a key store?
- **The webhook side:** the provider confirms captures through webhooks, delivered at-least-once and out of order. Harden that inbound path using this module's patterns.
- **For AI engineers:** swap the provider for an LLM API inside an agent-powered checkout assistant. Which stage changes the most? Hint: Stage 7's fallback sequence gains a semantic cache, and Stage 4's breaker can switch models at the LLM gateway.

Module 6 — Growing Under Load (Scaling)

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Earlier modules scaled the database (storage) and lifted read traffic off it (caching). This module scales everything that sits above the data. That means the application servers, the way traffic gets shared among them, how many of them run at 4 AM compared with peak time, and how to keep a bigger fleet from crushing the database behind it. It finishes the answer to the course's fifth question — *how does it grow?* — and presents scaling as everyday engineering, not a heroic event. In practice it comes down to four decisions. Choose a direction to grow in. Make servers interchangeable, so that extra capacity really helps. Spread requests across that capacity. And protect the systems that sit behind the application tier.

Pattern picker

The situation	The pattern
The machine isn't big enough	Vertical Scaling
No single machine could ever be big enough	Horizontal Scaling
Traffic has to be steered across many servers	Load Balancing
Demand rises and falls hour by hour	Auto-Scaling
Opening each database connection is costly	Connection Pooling
One region, one bad deploy or one noisy customer can take everyone down	Multi-Region and Cell-Based Architecture *(added lesson)*

Two facts that organize the module

1. **Horizontal scaling has a prerequisite, and that prerequisite is what this module is really about.** Extra servers only help if *any* server can handle *any* request. So no server is allowed to remember anything about a client between requests — no sessions kept in local memory, no files saved to local disk. This property is called **statelessness**. You have to work for it, because every value that needs to last must be given a shared home (a database, a cache, object storage). Engineers sum up the result as **"cattle, not pets"**: servers are numbered, identical, and replaceable, rather than individually named and cared for. A lot of what people call "scaling work" is really the job of pulling state out of application servers.
2. **Scaling one tier multiplies the load on the next one.** Double the application servers and you may double their database connections, their queries, and their downstream traffic. So capacity work has to *follow the bottleneck* — the one part that's currently limiting **throughput** (how much work the system completes per second, e.g. requests per second). And once you widen it, something else becomes the limit. The Connection Pooling lesson exists because this multiplication is easy to overlook: the database's connection limit is often the very first thing a fast-growing app tier breaks.

How to read it. Vertical Scaling (the simple bigger-machine option: no new ways to fail, but a hard ceiling) → Horizontal Scaling (the central lesson: the statelessness changes you need before more servers can help) → Load Balancing (making many servers look like one address) → Auto-Scaling (changing the number of servers as demand changes, including wrong metrics, flapping, and startup delay) → Connection Pooling (how app growth multiplies DB connections until Postgres — PostgreSQL, a popular open-source relational database — turns clients away) → **Multi-Region and Cell-Based Architecture** (an added lesson, not in the original course: surviving the loss of a whole **region** (one geographic location of a cloud provider's datacenters), and splitting each region into independent cells so a bad deploy or a noisy customer hurts only a slice of users — plus a short map of cloud building blocks) → the capstone (the closing, end-to-end worked example), which follows one service from 100 to 1,000,000 requests/second and, at every stage, names what to scale *and what to leave alone*. Short on time? Read

Horizontal Scaling and Load Balancing first. "How would you scale this?" is the interview question you can count on hearing, and those two lessons are the heart of the answer.

Four terms used throughout (new to load balancers, caches, latency or DNS? The System Design Fundamentals primer covers the basics this module builds on.)

- **Stateless vs. stateful.** A *stateless* server keeps nothing from one request to the next. Everything it needs either comes with the request or lives in shared storage. A *stateful* server holds something that makes that particular machine important (a session, an open socket, a local file). Stateless tiers grow by adding identical servers. Stateful ones need the harder storage patterns.
- **Bottleneck.** The one component that's currently limiting throughput. Scaling anything else adds no capacity, so measure to find it before you change anything.
- **Elasticity.** Capacity that tracks demand in *both* directions. Growing under load protects users. Shrinking when traffic drops saves money.
- **Headroom.** The gap between normal load and total capacity. It soaks up surprises while auto-scaling catches up. The course's arithmetic rule plans peak traffic at **3–5× the average**. Headroom covers the time between a spike beginning and new servers being ready.

Before you start — think about a system you run:

1. Could *any* of your app servers handle *any* request? Or does something make one server special (a session, a file, an in-memory cache with no fallback)?
2. If you doubled your app servers this afternoon, what would break first further downstream?
3. What's your ratio of peak to average traffic — and who pays for the gap at 4 AM: idle servers, or an auto-scaler?

Vertical Scaling

Original lesson: Vertical Scaling on DesignGurus ↗

What it is. Vertical scaling ("scaling up") means giving the machine more muscle — more CPU cores, more memory, faster disks — while leaving the architecture exactly as it is. *Analogy:* when your kitchen can't keep up, the first thing you buy is a bigger stove, not a second restaurant.

Why it matters (the problem it solves). This is the least fashionable pattern in the course, and it deserves more respect than it gets. Teams often leap straight to fleets of servers and **microservices** (an app split into many small, separately deployed services that call each other over the network), when one bigger machine would have done the job for years. The real problem vertical scaling solves is "the machine is too small." The industry habit of skipping this simple fix burns years on distributed complexity that nobody needed yet.

How it works, and why it's valuable

1. **It brings no new ways to fail.** No other scaling pattern can say this. There's no statelessness work, no load balancer (a component that spreads incoming requests across several servers), no keeping caches consistent across machines, no distributed coordination. Transactions (groups of database changes that succeed or fail as one unit), joins (queries that combine rows from several tables), and in-memory state carry on working exactly as they did. Every hard problem in the next four lessons begins the moment a *second* machine shows up, and vertical scaling puts all of them off until you really need to face them.
2. **There's more room to grow than people think.** Cloud instances run from about **2 cores / 8 GB RAM** (a core is one independent processor inside the CPU; RAM is the machine's fast working memory) all the way up to **128+ cores and several terabytes of RAM** — roughly **64× capacity** from a single configuration setting. Plenty of successful products spend their entire life inside that range. *Stack Overflow* is the famous example: it answered a huge share of the internet's programming

questions from a handful of large machines, while much of the industry was recommending fleets of hundreds of nodes.

Its two hard limits

- **There is a biggest machine.** Every provider's catalog runs out eventually, and the price per unit of capacity usually rises steeply near the top. Software can also stop benefiting before the hardware runs out. A database held back by *lock contention* (queries waiting their turn to update the same rows) gains almost nothing from core #96. And single-threaded code can't use extra cores at all.
- **It is still one machine.** Doubling the CPU gives you no redundancy. Putting more work on one host actually *raises* the damage when that host fails (a bigger *blast radius*). The machine is still a *single point of failure* (SPOF — one component whose failure takes everything down). And resizing it usually needs a maintenance window (planned downtime).

Combining directions. The practical skill is knowing when to switch direction and how to blend the two. Real fleets tune both the size and the count — for example, 24 small vs. 12 medium vs. 6 large servers. And after a database is sharded (split across machines), every shard still needs its own decision about vertical size.

Trade-offs

You gain	You pay
More capacity with no change to the architecture — often done in an afternoon, with a maintenance window as the entire cost	A ceiling, a steep price curve leading up to it, and shrinking software gains near the top
Single-machine guarantees (transactions, consistency, locality) that distributed designs spend entire modules winning back	No gain in availability (the share of time the system is up and answering): one bigger machine means one bigger outage; redundancy needs at least two of something
Time — really, a year to do the statelessness work properly instead of in a panic	No elasticity: you can't resize every hour, so the big box sits idle and expensive at 4 AM
	Downtime to resize, and long waits to get hardware at the top of the catalog

When not to use it

- **When availability is what you actually need.** Two medium machines still have half their capacity after one fails. One large machine has none left.
- **When load is spiky.** If the daily peak is 5× the average, a machine sized for the peak sits roughly **80% idle** for most of the day. Lots of smaller elastic units fit better.
- **When you're already close to the top of the catalog.** Paying the highest unit price for the last 2× only postpones a wall you can already see. Spend that money on moving to horizontal scaling.
- **When the software can't use the hardware** (single-threaded code, lock contention). Profile before you buy.
- *The more common misuse is the reverse:* sharding the database and running a fleet at **50 requests/second** because big companies blog about doing it. The arithmetic says that scale doesn't need it yet.

In the interview. It's rarely asked about by name, but interviewers watch whether you think of it before jumping to distributed options. When a service is overloaded: measure the bottleneck, and check whether a bigger instance buys enough time. If it does, use that time to strip state out of the app, so horizontal scaling is ready before the big box reaches its ceiling.

> **30-second answer:** Vertical scaling is the bigger machine: more cores and RAM, no change to the architecture, and no new ways to fail — transactions and single-machine simplicity keep working. Cloud catalogs offer about 64× of capacity through that one setting, and it's often

> the right first step. Its limits: there's a biggest machine, and the price curve gets steep before you reach it. It also does nothing for availability or elasticity — it's still one machine, now with a bigger blast radius. So I scale up to buy time, and I use that time to make the service stateless, so scaling out is ready when it's needed.

- *Follow-up: "Why not just keep buying bigger forever?"* — Three kinds of limits. Physical (the catalog ends). Economic (unit prices climb near the top while software gets less out of each extra core). And architectural (one machine can't give you redundancy or hour-by-hour elasticity).
- *Follow-up: "One big machine or three small ones, same total capacity?"* — For a stateless service, three small ones: a single failure costs a third of capacity rather than all of it. Pick one large machine when the workload depends on single-machine guarantees or memory locality, like a primary database. Equal capacity doesn't mean equal availability, or equal effort managing state.
- *Mistake that fails candidates:* proposing distributed infrastructure for a problem one machine can handle — or proposing a bigger machine after an *availability* incident. More resources don't stop a host from dying.

Self-check:

- Q1: What does vertical scaling preserve that horizontal designs must rebuild, and what are its two hard limits? → Single-machine transactions, strong consistency, joins, locality, simple in-memory state, and freedom from distributed coordination. Limits: the hardware is finite, with prices rising and software benefit falling at the top; and it's still one machine — no redundancy, no elasticity, and downtime to resize.
- Q2: A primary DB runs at 80% CPU growing ~5%/month; half the team wants to shard now. What do you do? → First scale up one or two instance sizes. That gives 2–4× headroom within a week, which lasts years at 5%/month, rather than rushing into the course's most expensive storage pattern. Use that time well: choose a shard key, fix queries that won't survive partitioning, and set a trigger for migrating, such as sustained 70% CPU on the largest reasonable instance (sharding as a last resort).
- Q3: A payments service runs on one large instance at 30% utilization; its host died last month (40 min downtime), and leadership wants an even bigger instance. Evaluate. → Wrong fix. 30% utilization shows capacity was fine; the outage happened because there was only one machine. A bigger box keeps the same failure mode and costs more. What it needs is 2+ stateless instances behind a load balancer (after working through the statelessness checklist), plus a primary-replica failover plan for the parts that hold state. Vertical scaling fixes too little capacity; this was too few *machines*.

Related patterns: Horizontal Scaling (what comes next, and a complement — scale up to buy the time needed to make scaling out possible); Sharding (each shard still needs a decision about vertical size); Primary-Replica (the answer when the incident was about availability, not capacity); Auto-Scaling (capacity that tracks demand — something vertical scaling can't offer).

TL;DR

- **Problem:** the machine is too small — and the habit of skipping the simple fix wastes years on distributed complexity.
- **Mechanism:** buy a bigger machine: ~64× of room on one setting, zero change to the architecture, and single-machine guarantees still in place.
- **Costs:** a real ceiling with a steep price curve, zero redundancy (a bigger SPOF), no elasticity, and downtime to resize.
- **Skip it when:** availability is what you need, load is spiky, or you're already near the top of the catalog.

Horizontal Scaling

Original lesson: Horizontal Scaling on DesignGurus ↗

What it is. Horizontal scaling ("scaling out") means running many identical copies of your application server and adding more copies as load grows. It only works when every server is **stateless** — it remembers nothing between requests — so that any server can handle any request and the servers are interchangeable. *Analogy:* a supermarket opens more checkout lanes at rush hour. That only helps because any cashier can serve any customer. If your groceries were locked in lane 3's back room, opening lane 7 wouldn't do you any good.

The problem. For years the store has run its app tier on a single server that keeps getting bigger. It finally runs into the limits from the last lesson: the next size up costs too much, and a host failure recently stopped checkout for **40 minutes**. The long-overdue fix: run **two servers** behind a simple traffic splitter that sends half the requests to each. It goes live on Tuesday morning. By the afternoon, three strange bugs have shown up:

- **Users get logged out at random.** Alice logs in, and her *session* (the record saying she's logged in) is kept in server A's memory. Her next request lands on server B, which has never heard of her. So the site forgets who she is on every other request.
- **Uploaded images return 404 (HTTP "not found") half the time.** Product photos were saved to the local disk of whichever server happened to receive the upload. Requests handled by the other server can't find them.
- **The UI keeps flipping back and forth.** Each server caches *feature flags* (switches that turn features on or off) and refreshes them on its own timetable. So after a flag changes, the page layout depends on which server answers.

Two servers aren't hard in themselves. What the failure shows is that the code is full of hidden assumptions that there is exactly *one* machine — one process holding sessions, one disk holding files, one process running the nightly job. Horizontal scaling means hunting down every one of those assumptions.

Why the obvious fixes fail

- **"Sticky sessions."** The splitter uses a cookie (a small piece of data the browser stores and sends back with every request) or an IP address (the numeric network address of a machine) to keep sending each user to the same server. The logouts stop right away, and it works fine as a *migration bridge*. But if server A dies, everyone pinned to it is logged out. Heavy users can overload a single server. Deploys have to wait for pinned users to leave. And it does nothing about the local images or the flag caches.
- **"Synchronize state between servers."** Getting A and B to copy sessions and caches to each other builds an accidental distributed database inside the app tier. That re-creates the hardest consistency problems from the storage module. Session clustering and multicast sync (one network message delivered to many servers at once) used to be common, but shared stores give clearer, more reliable ownership.
- **"Deploy carefully and hope."** Random session loss comes from the architecture, not from how you release. Being careful can't fix it.
- All three try to manage *disagreement between servers that hold state*. The pattern gets rid of the cause instead: app servers must not own any lasting state.

How it works. The figure below shows the target architecture. A load balancer sits in front of Server 1, Server 2, … Server N, and every server talks to the same shared back-ends — shared sessions in Redis (a fast in-memory key-value store, often used as a shared cache), object storage (a service

such as S3 that stores whole files and serves them over HTTP) for files, a shared cache, and the database. No server holds anything that the others don't have.

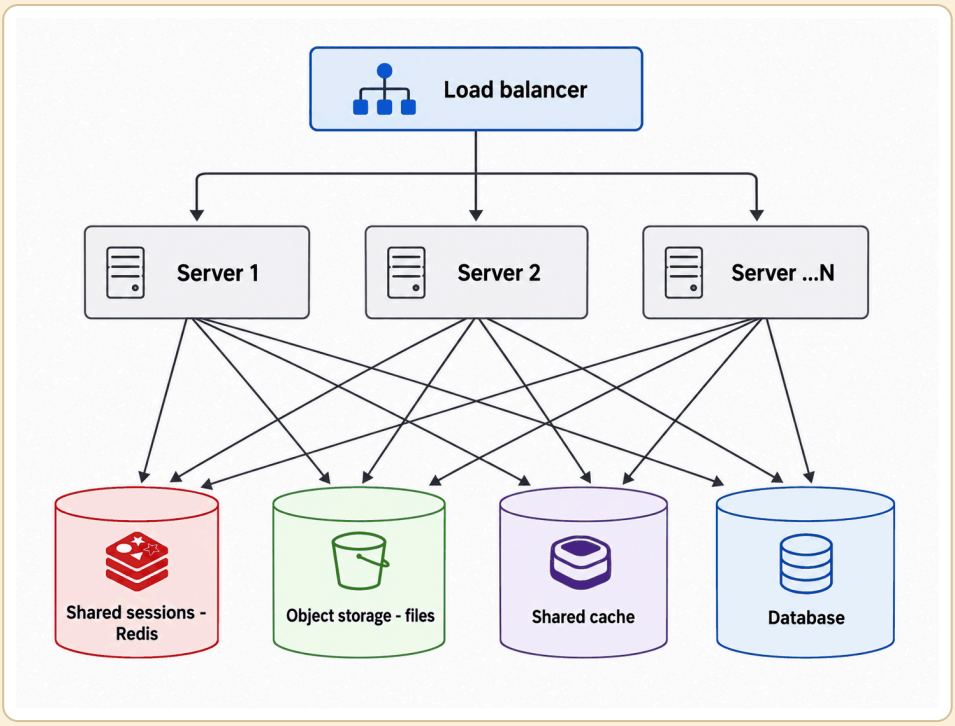


Figure — Horizontal Scaling (course diagram)

Step 1 — the statelessness checklist. Track down every value an app server holds on to, and move it to a shared home:

State hiding in the server	Where it moves	Why
Sessions ("who is logged in")	A shared store (Redis), or into the request itself as a signed token the browser carries	Then any server can recognize the user
Uploaded files	Object storage (S3 and similar) — never local disk	Every server can read every file
In-process caches	A shared cache (Module 4), or a local one with a short TTL (time-to-live: how long an entry may be kept) where brief disagreement does no harm	Servers stop disagreeing about data
Scheduled jobs ("the nightly report")	A job queue or a *distributed lock* (a lock that all servers honor)	Otherwise every server runs the cron (scheduled job) — N times
Counters, rate limits, locks	A shared store	A counter on one server only sees 1/N of what's really happening

Step 2 — collect the three payoffs.

1. **Capacity becomes a matter of addition.** Each server takes a slice of the traffic. More capacity just means more servers, with the load balancer (next lesson) spreading the requests.
2. **Availability becomes simple arithmetic.** Run **N+1**: enough servers for the peak, plus one spare. Then any single failure only uses up the spare, instead of causing an outage.
3. **Deploys stop being big events.** A *rolling deploy* swaps servers out one at a time, with no downtime and no special treatment for any machine.

Step 3 — expect the load to multiply. Each server you add brings more downstream connections and requests. Databases, caches, and other services now carry the combined load of N servers. Connection Pooling and the storage module take care of that.

Walkthrough with numbers

- **Sizing.** One server comfortably handles **800 req/s**, and the peak is **4,000 req/s** → 5 servers needed → run **6** (N+1). The peak estimate already includes headroom for traffic; the sixth server is headroom for *failure*.
- **The cost of keeping state elsewhere.** Every request now fetches its session from Redis, which adds about **1 ms** — the measurable price of letting any server serve any user.
- **A server dies at peak.** Health checks (automatic probes asking each server "are you OK?") take it out of the load balancer within seconds, and the remaining 5 carry all 4,000 req/s. Requests that were in flight on the dead host fail once, and their retries land on healthy servers. That's safe because writes are idempotent (doing them again has the same effect as doing them once).
- **Growth.** Traffic grows 10× over two years → about **50 servers**, with the same architecture and cost that grows almost in a straight line. New capacity is something you buy, not something you redesign.
- **The multiplied load shows up right on time.** 50 servers × 20 DB connections each = **1,000 connections**, and the database starts turning clients away. Scaling the app worked — and pushed the bottleneck downstream. Connection Pooling deals with that.

Horizontal vs. vertical. The figure below compares them row by row. Vertical's ceiling is the end of the catalog (~64× the starting machine), while horizontal has none in the app tier (the ceiling moves downstream). Vertical's cost curve gets sharply steeper at the top, while horizontal's is roughly linear (10× traffic, 10× servers). Vertical adds no availability, while horizontal gives you N+1. Only horizontal is elastic. Vertical needs nothing in advance, while horizontal needs the statelessness overhaul *finished*, not mostly finished. Vertical keeps single-machine guarantees, while horizontal keeps nothing automatically. The figure's footnotes: horizontal's cost is paid once up front (the overhaul) plus ~1 ms per request, and six servers for 50 req/s is imitation, not engineering — two servers and a failover are often enough.

	Vertical one bigger machine	**Horizontal** more machines
Ceiling	The catalog runs out. About 64 times the machine you start on.	None in the application tier. The ceiling moves downstream.
Cost curve	**Steepens sharply at the top.** The last doubling costs the most.	**Roughly linear.** 10 times the traffic, 10 times the servers.
Availability	None. It is one bigger single point of failure.	N plus one. A lost server becomes routine.
Elasticity	No. A machine cannot be resized by the hour.	Yes. The fleet grows and shrinks with the traffic.
Prerequisite	None. It works on the code you already have.	**The statelessness overhaul, finished, not mostly finished.**
Preserves	Single-machine guarantees, and simplicity.	Nothing automatically. Sessions, files, caches and crons all move out.

Vertical scaling is the one that gets expensive at the top, not horizontal. What horizontal costs is paid once and up front, in the overhaul, plus about a millisecond per request to fetch state that used to sit in local memory.

Six servers for 50 requests per second is imitation, not engineering. Two servers and a failover are often enough.

Figure — Vertical vs. Horizontal Scaling (course diagram)

	Vertical (bigger machine)	Horizontal (more machines)
Ceiling	The catalog runs out (~64× of room)	None in the app tier; it moves downstream
Cost curve	Gets sharply steeper at the top	Roughly linear
Availability	None — one bigger SPOF	N+1 — losing a server is routine
Elasticity	No — can't resize every hour	Yes — the fleet grows and shrinks
Prerequisite	None — works on any code	The statelessness overhaul
Preserves	Single-machine guarantees, simplicity	Nothing automatically — everything shared is explicit

They work together rather than compete. Scale up to buy time, use that time to remove state, and scale out when the measurements justify it. And keep tuning both size and count (6 large vs. 24 small is a tuning choice, not a principle).

Trade-offs

- *You gain:* capacity by addition, with no ceiling in the app tier (it moves downstream); availability through redundancy (a machine dying becomes routine); elasticity as an option (fleets can grow and shrink — two lessons from now); and quiet rolling deploys with zero downtime.
- *You pay:*
  - **The statelessness overhaul.** Code changes and testing for sessions, files, caches, scheduled work, and counters — all paid *before* the first extra server works correctly. This is the main cost.
  - **A cost on every request.** About 1 ms to fetch external state for each request.
  - **Shared stores take on the criticality.** Moving sessions out of forty servers into one Redis makes that Redis a possible single point of failure, so it needs replication and care. State gets *relocated*, not eliminated.
  - **Coordination problems.** Work that naturally ran once on one machine (crons, global counters) now needs explicit coordination.
  - **Multiplied downstream load**, every time.

When not to use it

- **Workloads that hold state by nature.** A database's whole job *is* owning state, so it can't scale by adding memoryless copies (use the storage patterns instead). WebSocket gateways (servers that keep a two-way connection open with each client) hold live connections and need a two-tier design (local sockets + global routing). Adding stateless copies is for compute.
- **Small scale.** Two servers with failover may be enough. Even so, working through the checklist early is smart, because hidden assumptions are cheapest to remove before they spread.
- **When the bottleneck is downstream.** More app servers can't help a system held back by its database; they just pile more connections and queries onto it. Measure first.
- **Microsecond latency budgets** (latency = the time one request takes, start to finish) that can't afford a hop to external state may deliberately keep state in the process.
- *The classic failure:* a migration left half-done — one forgotten cron, local counter, or in-memory rate limiter that quietly runs N times or sees only 1/N of what's really happening.

Real-world examples

- **The Twelve-Factor App** (a manifesto from Heroku, a cloud hosting platform): factor VI, "execute the app as stateless processes," is this whole pattern in one line.
- **Kubernetes ReplicaSets and AWS Auto Scaling Groups** (Kubernetes is the standard open-source system for running containers on a cluster of machines; AWS is Amazon's cloud) turn horizontal

scaling into a configuration setting: `replicas: 50` assumes that every *pod* (a deployed unit) is interchangeable.

- **Every modern web tier** looks like this. The interesting decisions are about where the sessions, files and other state ended up.
- **For AI engineers:** inference services (servers that run a trained model to answer requests) run identical copies of a model behind a load balancer, with two twists. First, each copy may need to load tens of GB of model *weights* before it can serve — minutes of startup, which changes how auto-scaling works. Second, a conversation's *KV cache* (the model's working memory of earlier tokens — a token is a word or word-piece the model reads and writes) works best if it stays on the same copy, so requests aren't fully interchangeable. *Consistent-hash* routing (a hashing scheme that maps each key to a server and moves few keys when servers change) by session keeps a conversation on the copy that holds its cache. Batch embeddings (numeric vectors that represent text meaning) and evaluations fit the simple pattern: independent workers running in parallel.

In the interview. "How would you scale this?" is one of the most common questions. After you say "add servers," expect "what stops you from doing that today?" List every form of state that lives on a server, rather than assuming that deploying more copies creates capacity.

> **30-second answer:** Horizontal scaling means identical, stateless servers behind a load balancer, so any request can land on any server. The real work is the statelessness checklist: sessions go to Redis or signed tokens, files go to object storage, caches become shared, crons sit behind a lock or queue, and counters live in a shared store — because state makes a server special, and special servers can't be swapped out freely. After that, capacity is addition, availability is N+1, and deploys are rolling. I call out two costs up front: shared stores take on the criticality (the session Redis needs replication), and every server I add multiplies database load, which connection pooling and the cache tier soak up.

- *Follow-up: "A server dies mid-request — what do users see?"* — Health checks pull it out within seconds. Its in-flight requests fail once, and retries safely reach healthy servers because writes are idempotent. N+1 sizing lets the rest carry the full load.
- *Follow-up: "You doubled the fleet and latency got worse. Why?"* — The app tier wasn't the bottleneck. More servers multiplied DB connections, queries and lock contention. Follow the constraint and fix the data path (storage, caching, a connection pooler).
- *Mistake that fails candidates:* saying "add servers" without checking for state, presenting sticky sessions as the permanent design, or scaling a tier that wasn't what limited throughput.

Self-check:

- Q1: List the five kinds of state commonly left in app servers and where each goes. → Sessions → Redis or signed client tokens; uploaded files → object storage; in-process caches → a shared cache (or a short local TTL where brief disagreement is fine); scheduled jobs → a queue or distributed lock so only one worker runs each one; counters/rate limits/locks → a shared store, since each server only sees 1/N.
- Q2: Make the case for and against sticky sessions; when should they be removed? → For: a quick bridge needing little code that stops random logouts while shared session storage gets built — and genuinely needed for connections that really hold state (WebSocket gateways). Against: a dead server drops the users pinned to it, heavy users throw load out of balance, deploys drain slowly, and new servers start out empty. Remove them as soon as sessions live in shared storage or tokens — before more code starts relying on the pinning.
- Q3: A team grows from 6 to 30 servers; the DB refuses connections at peak and customers get the nightly summary email several times. Fix both. → 5× the servers multiplied each server's DB connections past the DB's limit: add a connection pooler and shrink each server's pool. The email cron still runs locally on every server: move it to a queue with a single consumer, or

behind a distributed lock, and make it idempotent, keyed on customer + date. Statelessness means finishing *every* item on the checklist.

Related patterns: Vertical Scaling (buys time for this migration and stays available as a sizing option); Load Balancing (makes N servers look like one endpoint); Auto-Scaling (a stateless fleet can grow and shrink with demand); Connection Pooling (keeps the combined DB connections of N servers under control); Idempotency (makes retries safe after a server dies).

TL;DR

- **Problem:** one machine has a ceiling and is a SPOF — but naively adding a second one exposes every hidden one-machine assumption (sessions, files, caches, crons, counters).
- **Mechanism:** the statelessness checklist moves all server memory to shared homes; after that, identical servers behind a balancer give capacity by addition, availability by N+1, and rolling deploys.
- **Costs:** the overhaul, ~1 ms per request for external state, shared stores that take on the criticality, new coordination problems, and multiplied downstream load.
- **Skip it when:** the workload's state *is* its job (databases, socket gateways), the scale is small, or the bottleneck is downstream.

Load Balancing

Original lesson: Load Balancing on DesignGurus ↗

What it is. A load balancer is a dedicated layer that stands at the front door of a server fleet. It gives clients **one stable address** that covers many servers, even as those servers change. It checks which servers are alive (**health checks**). It spreads requests using an algorithm that reflects the real load. And it **drains** servers (lets their current work finish) before they get replaced. *Analogy:* a restaurant host at the door who knows which tables are free, which waiter is swamped, and which section is closing for cleaning. Diners just come in through the one front door.

The problem. The stateless fleet now has six identical servers sitting behind Tuesday's simple splitter. The splitter uses **round-robin**: request 1 goes to server 1, request 2 to server 2, and so on around the list. That looks fine until **2:14 AM**, when a runaway log fills up server 4's disk. The server doesn't crash. It just gets slower and slower, taking **20 seconds** per request before failing. The splitter knows nothing about health, so it keeps sending **one-sixth** of all traffic there. For **11 minutes**, users see failures or long waits, until an engineer notices and pulls the server out by hand.

Two weeks later, a deploy exposes a second gap. The splitter can't stop sending new work to a server while letting its current requests finish. So replacing server 2 kills **several hundred in-flight requests**. Every deploy causes a small, predictable spike in errors.

The underlying flaw: the splitter treats servers as interchangeable *and permanent*. Statelessness made them interchangeable. But real servers still slow down, die, get deployed, and get replaced — and whatever sits in front of them has to notice within seconds.

Why the obvious fixes fail

- **"DNS round-robin."** List all six IP addresses under one domain name and let DNS (the Domain Name System, which turns names like `shop.com` into IP addresses) rotate through them. The trouble is that clients and resolvers (the DNS servers that look names up on their behalf) cache answers for minutes or hours. So after server 4 is removed, clients that still hold its address keep using it. DNS publishes addresses; it doesn't steer individual requests. That means it can't measure health or take back a choice that's been cached.
- **"Let clients pick a server."** Hand clients the list of servers and let them choose. Internal service meshes (an infrastructure layer of small proxies that handles calls between a company's own services) legitimately do a more capable version of this. But public browsers and apps would need health checks, retries, and list management — and you can't reliably update those.

- **"A runbook: on-call removes bad servers."** That *is* the 11-minute response. Like the manual kill switch in the Circuit Breaker lesson (a circuit breaker stops calling a failing dependency for a while), it's the right decision made at human speed. This pattern makes it at machine speed.

How it works — four jobs. The figure below catches a load balancer in the middle of its work. Clients see one address. It health-checks every 2 s, routes by least-connections, and drains before replacing. Server 1 (41 in flight), Server 3 (38), Server 5 (44) and Server 6 (39) are serving. Server 2 is *draining* for a deploy (6 requests finishing). Server 4 was *removed* after 3 failed probes. The next request goes to Server 3, which has the fewest requests in flight. So a slow server automatically gets fed less, whereas round-robin would keep giving Server 4 its full share.

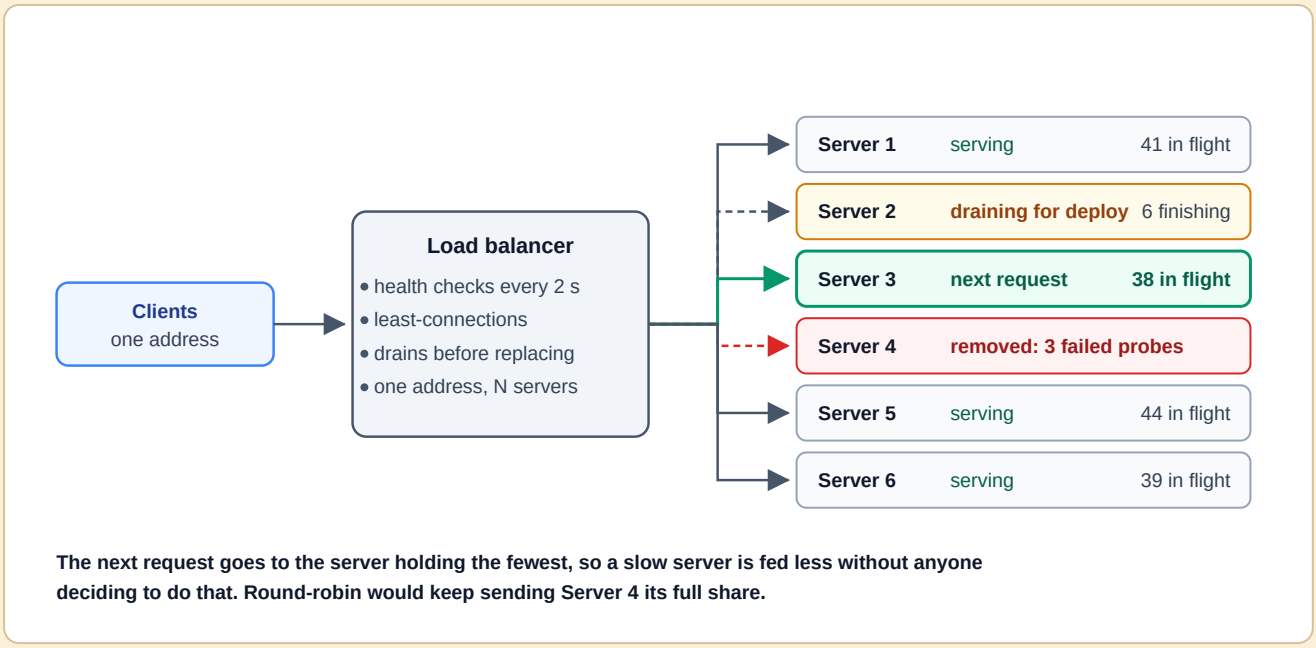


Figure — Load Balancing (course diagram)

1. **Health checks — how failure gets noticed.** *Active* checks call each server's health endpoint (a special URL that reports whether the server is fine) every few seconds (the Operations module designs that endpoint). Several failures in a row remove a server; several successes in a row bring it back. *Passive* checks watch real traffic and can react to a sudden jump in errors between probes. Together, they shrink the 11-minute manual response to about **6 seconds**.
2. **The algorithm — how traffic gets spread.**
   - *Round-robin* works when all requests and all servers are alike.
   - *Least-connections* sends each new request to the server with the fewest requests currently being worked on. It's the better general default. A struggling server builds up in-flight requests, and so it automatically gets less new work — which directly fixes what happened at 2:14 AM.
   - *Weighted* algorithms handle servers of different sizes.
   - *Hash-based* routing (consistent hashing, "the ring") sends the same user to the same server when *affinity* matters — warm per-user caches, long-lived connections.
3. **Draining — the deploy story.** Before a server gets replaced, the balancer stops handing it new requests and lets the current ones finish (seconds for ordinary API calls; streams held open get a notice before being disconnected). The replacement is let in only after it passes health checks. The error spike during deploys goes away.
4. **One address — and what it makes possible.** Clients use a single stable endpoint, while the fleet behind it changes size and membership (which is exactly what makes auto-scaling possible). The balancer often *terminates TLS* — it handles the encryption for each connection (TLS is the encryption layer behind `https://`). **L4** balancers route by IP address and port (a number that picks which program on a machine gets the traffic) — at the connection level. **L7** balancers read HTTP (the request/response protocol of the web) paths, headers, and cookies (at the request level), which allows routing by path and retries that understand the request. Public entry points usually use L7; internal tiers may prefer the simpler L4.

Who balances the balancer? It sits on the path of every request, so it can't be allowed to be a single point of failure. Standard setups run a **redundant pair sharing a floating IP** — an address that moves on its own to the surviving machine if one dies. At internet scale, **anycast** lets many balancers announce the same IP, and the network sends each client to a nearby one. That ends the "who balances that?" chain after one level.

Walkthrough with numbers

- **2:14 AM, replayed.** Server 4's disk fills up. Active probes fail at 0, 2 and 4 seconds, and it's removed at ~6 s (passive error monitoring may react even sooner). One-sixth of traffic sees a short blip, and the system recovers with no human involved.
- **The GC pause.** Server 5 hits a long garbage-collection pause (the runtime freezing to reclaim memory). It's slow, not failed. Round-robin would keep feeding it an equal share. Least-connections sees its in-flight count rise from **40 to 300** and sends new work elsewhere until it recovers.
- **The deploy.** Six servers are replaced one at a time: drain each for **~30 s**, start the replacement, and let it in once probes pass. The SSE (server-sent events — long-lived streaming connections) fleet uses longer drains plus *jittered reconnects* (every client waits a slightly different random time before reconnecting, so they don't all stampede at once). Either way, zero deploy errors.
- **The math check.** The balancer adds **well under 1 ms** per request and easily handles the full **4,000 req/s**. Its value isn't speed. It's replacing slow human detection.

Trade-offs

- *You gain:* failure handling at machine speed (seconds, rather than minutes waiting for a paged human); distribution that matches reality (least-connections corrects itself for slow servers and uneven requests); zero-downtime deploys through draining; and one stable address that hides all the churn in the fleet — which is what makes auto-scaling possible.
- *You pay:*
  - **A critical hop on everything.** The added latency is under a millisecond, but *all* traffic depends on this tier. Its availability therefore matters more than any single server's — hence redundant pairs or bigger distributed deployments.
  - **Health checks can be wrong in both directions.** A *shallow* check (is the process running?) might pass a server that can't reach its database — it looks healthy but can't do any work. A *deep* check that tests a shared dependency is even more dangerous. If that dependency blips, *every* server reports unhealthy, and the balancer removes the entire fleet. The rule: a health check answers "should *this server* receive traffic?", not "is the whole architecture OK?"
  - **Settings with big consequences.** Probe intervals, failure thresholds, drain times, algorithms: small settings that can cause large outages.
  - **L7 costs.** Inspecting every request is more work than routing connections. At very large scale, the balancer tier needs horizontal capacity of its own.

When not to use it

- **One server.** A balancer in front of a single instance adds latency and configuration with no redundancy. (A proxy might still handle TLS or routing — a separate role covered in the API & Edge module.)
- **Internal service-to-service traffic inside a mesh.** Sidecar proxies (small proxies running beside each service) balance on the client side using rich health data — the same jobs, spread out instead of centralized.
- **Geographic routing.** Sending European users to an EU region happens at the DNS/anycast layer, *before* any regional balancer. It complements the balancer; it doesn't replace it.

- *Common misuse:* buying a load-balancer product and leaving round-robin defaults, shallow checks, and no drain integration — the original splitter's failures, with extra infrastructure on top. The pattern is four active jobs, not a box you purchase.

Round-robin vs. least-connections

	Round-robin	Least-connections
Assumes	All requests and servers are alike	Nothing — it measures
A slow/failing server gets	Its full equal share (the 2:14 AM failure)	Less and less, automatically, as its backlog grows
Mixed light/heavy requests	Lucky servers sit idle, unlucky ones get overloaded	Corrects itself
Long-lived connections (SSE, sockets)	Bad — it balances when connections *start*, not the load	The right default — it counts what's actually being held
Cost	Trivial	Trivial (one counter per server)

Least-connections is the practical default because it measures rather than assumes. Round-robin still suits short, uniform requests, and DNS (which can't see active work). Hash-based routing is the third option when cache locality or session affinity matters. In one line: *round-robin assumes traffic is uniform; least-connections measures it.*

Real-world examples

- **nginx and HAProxy:** long-established software balancers, with L4 and L7 modes and all of the algorithms above.
- **AWS maps the types onto products: ALB** is L7 (it reads paths, headers, and WebSocket upgrades); **NLB** is L4, built for millions of connections with microsecond overhead. Other clouds offer similar pairs.
- **Envoy:** places a small L7 proxy next to every service (a service mesh), giving every caller balancing that understands requests, plus local health data.
- **Anycast at the edge:** Cloudflare and other CDNs (content delivery networks: servers spread around the world that serve content from close to users) announce one IP around the world, and internet routing does the first layer of balancing to the closest entry point.
- **For AI engineers:** counting connections gives a false picture of inference load — two LLM (large language model) requests can differ by **100×** (10 tokens vs. 2,000). A GPU (graphics processor — the chip that runs AI models) running three long generations can be busier than one running ten short ones. So inference routers balance on **queue depth and outstanding tokens** per replica. They also use consistent-hash routing by conversation ID, so later turns of a conversation reach the replica holding its KV cache. It's still "measure the work" — just measured by how big the work is, not how many requests there are.

In the interview. Everyone draws a load balancer. Interviewers check whether you can explain it: how it chooses a server, how it spots failure, how it handles deploys, and whether it's a single point of failure itself.

> **30-second answer:** The balancer does four jobs. Health checks — active probes plus watching real errors — remove a failing server within seconds. Distribution uses least-connections, which corrects itself for slow servers, where round-robin would keep feeding them. It drains servers during deploys so in-flight work can finish. And it offers one stable address for a fleet that keeps changing, usually terminating TLS at L7. It runs as a redundant pair with a floating IP, or with anycast at bigger scale. I keep health checks deliberately shallow: a deep check on a shared dependency can pull the entire fleet when that dependency blips. Each

server reports only its own health, and it handles dependency failures by degrading, not by dying.

- *Follow-up: "What exactly should the health check test?"* — The process, the listening port, and the critical *local* resources that decide whether this server can take traffic. Not every downstream dependency — a single brief shared failure would throw out every healthy server. Handle dependency failures inside the service, with breakers and degraded responses.
- *Follow-up: "Who balances the balancer?"* — A redundant pair sharing a floating address that fails over in seconds. At larger scale, anycast, so that many balancers share one advertised address. Naming the mechanism completes the answer.
- *Mistake that fails candidates:* accepting round-robin without thinking about slow servers, restarting machines without draining, or acting as if the balancer can't fail.

Self-check:

- Q1: List the four jobs and the two opposite ways a health check can be wrong. → Health checking, distribution that's aware of load, draining before replacement, and one stable endpoint (often with L7 TLS termination). A shallow check can keep a server that responds but can't do any useful work. A deep check can throw out healthy servers when a shared dependency fails — removing the whole fleet over a problem outside it.
- Q2: Pick an algorithm for (a) a uniform internal API under 10 ms, (b) a public API with wildly varying request cost, (c) an SSE fleet holding 50k streams per server. → (a) Round-robin — its assumptions hold. (b) Least-connections — it steers away from servers that are piling up expensive requests. (c) Least-connections when placing streams (the held connections *are* the load), plus long drains and randomized reconnect delays.
- Q3: At 3:07 AM the whole site goes down; all six servers have low CPU and no errors; the balancer marks every server unhealthy at once, then restores them all two minutes later. → The health endpoint tests a shared dependency (Redis or the DB). A two-minute blip failed all six probes, and the balancer dutifully removed a healthy fleet. Restrict the check to the process, the port, and local conditions, and handle shared-dependency failures inside the app with breakers, fallbacks, and reduced responses.

Related patterns: Horizontal Scaling (statelessness is what lets the balancer send any request anywhere); Auto-Scaling (adds and removes servers behind this stable entry point); Consistent Hashing (affinity routing for warm caches, conversations, sockets); Health Check Endpoint (designs what the probe hits); Graceful Degradation (servers return reduced answers during dependency failures instead of dropping out of rotation).

TL;DR

- **Problem:** a simple splitter treats servers as interchangeable and permanent — it feeds failing machines their full share for minutes at human speed, and kills in-flight work on every deploy.
- **Mechanism:** a front-door layer with four jobs — active + passive health checks, least-connections distribution, draining, and one stable (L4 or L7, TLS-terminating) address — made redundant itself with pairs or anycast.
- **Costs:** a critical hop on all traffic, health checks that can mislead in both directions (especially the deep-check trap that removes the fleet), and settings where small values cause big outages.
- **Skip it when:** there's only one server, the traffic stays inside a mesh (sidecars balance it), or the decision is geographic (DNS/anycast).

Auto-Scaling

Original lesson: Auto-Scaling on DesignGurus ↗

What it is. Auto-scaling is a **control loop**. It keeps measuring a load signal, compares it with a target, and adds or removes servers to close the gap. The load balancer lets the new servers in and drains the ones that are leaving. The fleet size stops being a one-off guess and becomes an answer that's corrected all the time. *Analogy:* a thermostat. It measures the temperature, compares it with the setting, and switches the heating on or off — instead of someone guessing in January how much heat the whole year will need.

The problem. The stateless fleet runs **8 servers**, sized for an evening peak of **4,000 req/s**. Two incidents in a single month show what's wrong with a fixed size:

- **The 4 AM bill.** Average traffic is about **900 req/s**, and evenings climb to 4–5× that (in line with the course's peak rule). So five or six of the eight servers sit idle for most hours, at full price. Finance works out that roughly **60% of the app tier's spend pays for idle time**.
- **The Tuesday the TV mentioned you.** A morning TV segment features one of the products, and traffic leaps from 900 to **12,000 req/s in twenty minutes** — three times the planned peak. All eight servers max out. An engineer has to wake up, figure out what's wrong, provision instances, and wait for them to boot. Much of the spike is over before the new capacity makes any difference.

Fixed sizing is wrong in both directions — too much at 4 AM, too little during a surprise — because it's an estimate made at design time of something that changes every hour.

Why the obvious fixes fail

- **"Size for the worst case ever."** Running 24 servers all the time would have handled the TV spike, but it triples the 4 AM bill. And a growing business produces bigger surprises, so you'd be paying every day for a once-a-year event and still be exposed.
- **"On-call scales by hand."** Noticing, diagnosing, acting and booting used up most of a 20-minute spike. It's even worse the other way round: nobody gets up at 4 AM to *remove* idle servers. People are slow going up and absent going down.
- **"Schedule it: more servers at 6 PM, fewer at midnight."** Handy for known evenings, weekends and big sales (it'll be part of the final design). But a calendar can't see a TV segment coming, or an evening that turns out quiet.

How it works. The figure below shows the loop. Measure the signal every ~1 minute → compare it with a target (for example, 65% of capacity) → if it's over, **add servers fast**; if it's under, **remove servers**

slowly, draining them first → the load balancer lets new servers in and drains the ones leaving → measure again.

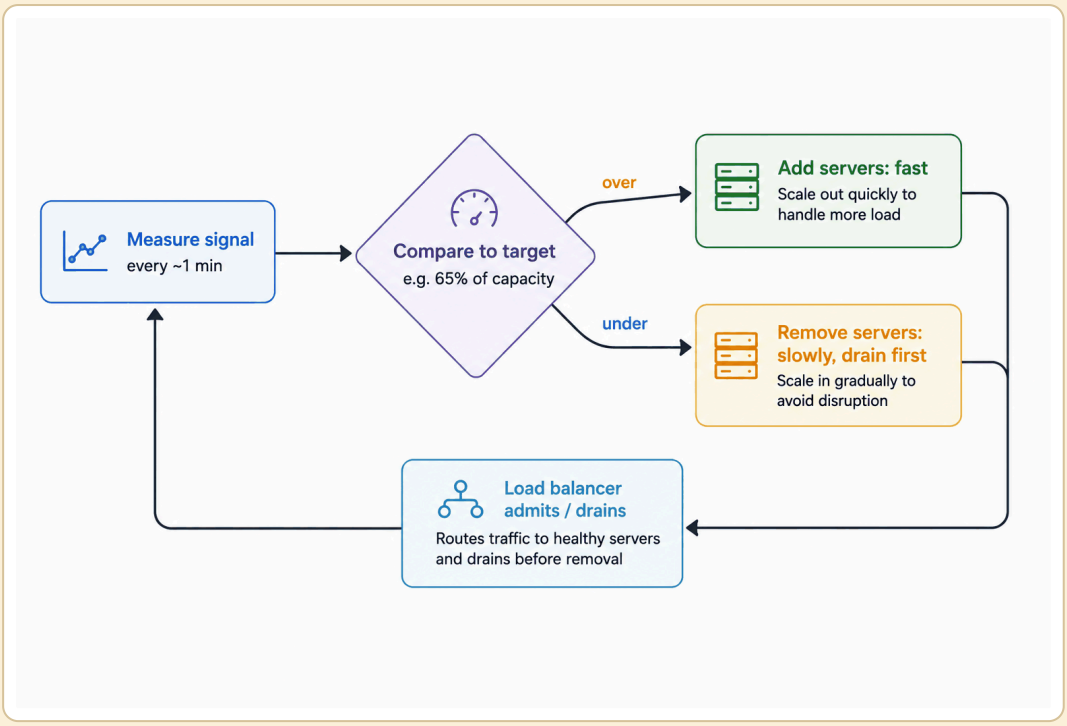


Figure — Auto-Scaling (course diagram)

Each part of the loop is a deliberate choice:

1. **Pick the signal — this is *the* design decision.** The *signal* is the number that stands in for current load. CPU utilization works for *compute-bound* services (ones limited by processing). It misleads you for *IO-bound* services, whose requests spend most of their time waiting on disks, networks, or databases. There, hundreds of blocked threads can make users suffer while CPU sits at **15%**, and a scaler driven by CPU does nothing. Better choices: **requests per instance** compared with each server's tested capacity, or **latency** (for example, p95 — the time within which 95% of requests finish) compared with the service target. For queue workers, use **backlog age** — how long the oldest unprocessed job has been waiting. If the oldest job has waited **90 seconds**, you clearly have too few workers.
2. **Set the target with headroom on purpose — about 60–70%, not 90%.** The unused slice is headroom. It protects you while traffic keeps climbing and the new servers aren't ready yet.
3. **Respect the reaction lag — it's physics.** Metrics usually get averaged over ~1 minute before a rise is even noticed. Then the instance boots, the app starts, and its cache warms up. Total: **3–7 minutes with no new capacity** after a spike begins. Headroom covers that stretch. A fleet that's already at 90% is overloaded before help arrives.
4. **Scale up fast, scale down slowly.** Add several servers at once, since an unneeded addition costs little. Remove them one at a time, and only after a *cooldown* (a waiting period after any scaling action) and a load-balancer drain. Removing too aggressively can instantly trigger another addition. That's **flapping**: adding and removing servers over and over, rebuilding caches and DB connections each time. The cure is asymmetry plus **hysteresis**: separate thresholds for scaling up and for scaling down, together with cooldowns.
5. **Set both a minimum and a maximum. Min = N+1**, so cost-cutting never takes away your failure headroom. **Max = measured downstream capacity**, not some arbitrary big number. A scaler with no cap can keep adding connections and queries until it flattens the database. The max works as a **bulkhead** (a hard limit that stops one part's trouble from flooding another, named after a ship's watertight walls) that protects downstream systems. Traffic beyond it has to be deliberately shed (rejected on purpose so the rest can be served), queued, or rate-limited (capped at a set number of requests per client per second) at the edge.
6. **Combine the triggers.** *Scheduled* scaling for known cycles (start before the 6 PM ramp — zero reaction lag). *Reactive* scaling for surprises. *Predictive* scaling (a forecast based on history) for

large fleets or slow boots. Use the calendar for what you know and measurements for what you don't.

Walkthrough with numbers (each server handles 800 req/s; a 65% target treats ~**520** as effective capacity; min 3, max 20)

- **4 AM, 900 req/s:** 900 / 520 ≈ 2 servers, raised to the N+1 floor → **3 servers**. The five idle evening servers — and their bill — are gone.
- **6 PM, scheduled:** the calendar adds capacity ahead of the ramp, so **8 servers** are ready when 4,000 req/s shows up. Reaction lag: zero, because nothing had to react.
- **TV Tuesday, replayed (900 → 12,000 in 20 min):** headroom soaks up the first ~**90 seconds**. The reactive loop adds servers in waves (~60 s to notice, ~3 min to boot). By minute six the fleet is at **16** and rising, and then it caps at **20**. That cap was chosen because the database (with its replicas and cache) is known to cope with 20 servers' worth of traffic and no more. Overflow beyond that gets shed at the edge. Bad minutes: about **four**. Human actions: **zero**.
- **The bill:** the average fleet size over a day drops from a fixed 8 to about **4.2** — roughly **half the compute spend**, with the same evenings and better Tuesdays. Scaling up protected users. Scaling down paid for the project.

Trade-offs

- *You gain:* a bill that tracks demand; surprises absorbed at machine speed (limited only by boot time and your max); self-healing for free (the loop replaces crashed instances to keep the minimum); and sleep for on-call — the 2 AM capacity page mostly goes away.
- *You pay:*
  - **Reaction lag, always.** 3–7 minutes, however good the policy. Scheduling and headroom soften it, but they can't remove boot time. Spikes shorter than the lag need traffic controls at the front.
  - **Disasters from the wrong signal.** A CPU-based scaler can miss an IO-bound incident. A policy based on memory might keep adding servers to hide a memory leak (a bug where a program keeps grabbing memory and never releases it) instead of getting it fixed. Choosing a signal is a diagnosis, and automating the wrong diagnosis makes it bigger.
  - **Flapping and churn.** Up/down thresholds set close together, with no cooldowns, cause nonstop fleet changes. Each unnecessary replacement brings cold caches and a burst of new connections.
  - **An automated flood downstream.** Every scale-up adds downstream traffic without any human reviewing it. The max, connection pooling (next lesson), and caching have to protect the database from its own scaler.
  - **Policies are code that runs overnight.** They need review, metrics you can watch, and load tests that actually trigger them — otherwise production traffic is their first real test.

When not to use it

- **Flat, predictable traffic.** An internal service at a steady 200 req/s may just need a fixed N+1 fleet. A control loop you don't need is one more way to fail.
- **Tiers that hold state.** Databases and socket gateways aren't interchangeable compute. Adding DB capacity takes the storage module's careful changes, not a fast loop (managed DB auto-scaling does something similar internally, slowly and within explicit limits).
- **When downstream can't take it.** If the database is already the bottleneck, automatically adding app servers makes the overload worse. Fix the bottleneck first — that rule holds whether a person or a loop is adding the capacity.
- **Boot times longer than the spike.** A server that takes 15 minutes to start can't respond to a 10-minute event. Workloads like AI serving need scheduled or predictive capacity and warm pools.
- *Common misuse:* using auto-scaling instead of actually measuring capacity — no known throughput per instance, no max, CPU only. It seems fine until an unusual day comes along. Then diagnosis

gets harder, because the fleet keeps changing size in the middle of the incident.

Reactive vs. scheduled vs. predictive

	Reactive	Scheduled	Predictive
Trigger	The signal, right now	The calendar	A model's forecast
Known daily cycle	Late by the lag, every single time	Perfect — capacity arrives before the load	Well
The TV moment	Yes — limited by lag + headroom	No — it isn't on the calendar	Only if it looks like history
Complexity	Moderate	Trivial	High — a model you have to trust
Failure mode	Wrong signal, flapping	Wrong guess, quiet drift	Wrong with confidence

Stack them instead of picking one: schedule what you know, react to what's happening, and add prediction once boot delay or fleet size makes forecasting worthwhile.

Real-world examples

- **AWS Auto Scaling Groups** with *target tracking* implement the whole loop: a signal, a target, a min and a max.
- **Kubernetes HPA** (Horizontal Pod Autoscaler) runs the loop on pods with custom signals. A separate **cluster autoscaler** adds or removes the machines those pods run on — two loops stacked on top of each other.
- **Queue-driven workers:** SQS (Amazon's managed message queue) queue depth can drive AWS Lambda (Amazon's serverless functions: code that runs on demand without servers you manage) concurrency — the backlog is the signal, startup is close to zero, and there are zero workers when things are idle.
- **Netflix Scryer** is a predictive scaler that forecasts demand and provisions ahead of time, because reaction lag is extremely costly at Netflix's scale.
- **For AI engineers:** GPU auto-scaling is the same pattern with tougher constraints. The signal should be queue depth and outstanding tokens, not CPU. Startup is expensive — pulling a container (a packaged app plus everything it needs to run) and loading tens of GB of weights takes minutes — so purely reactive scaling fails. You need warm pools, predictive changes, and a realistic decision about scaling to zero (idle GPUs are expensive, but cold starts make users wait directly). The max may be a **provider quota** rather than a budget choice, so shedding and queueing at the front are a must.

In the interview. A common setup: predictable evening peaks, plus the occasional 10× marketing event. Schedule the known cycle, react to surprises, keep headroom to cover boot time, and work out the max from downstream limits. After you say "it auto-scales," expect "on what signal?"

30-second answer: Auto-scaling is a control loop: measure a signal, keep it near a ~65% target, add servers fast and remove them slowly through the load balancer's drain. The signal is the real decision — CPU for compute-bound work; requests per instance or latency for IO-bound work, where CPU misleads you because threads are just waiting; backlog age for queue workers. I keep headroom because reaction lag is 3–7 minutes of physics. I schedule the known cycles so capacity arrives before the 6 PM ramp. I set the min at N+1 and the max at what the database can survive — an uncapped scaler is an automated stampede — and beyond the max, the front of the system sheds load.

- *Follow-up: "Your API is IO-bound — what do you scale on, and what if you'd chosen CPU?"* — Requests per instance against tested capacity, or p95 latency (the time within which 95% of requests finish). With CPU, a slow database leaves blocked threads piling up at ~15% CPU, so the scaler does nothing while users suffer.
- *Follow-up: "Flash sale: the scaler hits max and the site is still slow. Walk me through it."* — Find the active bottleneck before you touch the cap. If the database is saturated, the max is doing its job, and more app servers can't help. Improve caching, replicas, and pooling, and reject or delay excess work at the edge. Only raise the max after you've measured more downstream capacity.
- *Mistake that fails candidates:* CPU for every workload, no max ("it scales infinitely"), ignoring reaction lag and headroom, or treating a database like an interchangeable stateless tier.

Self-check:

- Q1: Name the parts of the loop and why scale-up and scale-down follow different policies. → Measure regularly, compare with a ~65% target, add/remove servers, let the balancer admit/drain them, and measure again. Scale up fast because a temporary surplus is cheap compared with an outage. Scale down slowly, with cooldowns and one removal at a time, because aggressive cuts cause instant re-adds — flapping, cold caches, bursts of connections.
- Q2: Pick a signal for (a) an IO-bound web API, (b) a queue-consuming image processor, (c) a GPU inference fleet. → (a) Requests per instance or p95 latency — users can suffer while CPU is low. (b) Backlog age — the oldest waiting job shows directly whether the workers are keeping up. (c) Queue depth or outstanding tokens per replica — the load is generation work, not CPU; plus warm/predictive capacity to cover startup that takes minutes.
- Q3: Policy: min 3, max 30, CPU target 65%, scale-down threshold 60%, no cooldown. The fleet bounces between 6 and 9 all evening, then *shrinks* during a database slowdown. Explain and fix. → The up and down thresholds are only 5 points apart, with no cooldown, so normal noise keeps undoing the last action. Widen them (for example, up at 70%, down at 40%), add cooldowns, and remove one server at a time. During the DB slowdown, threads were waiting instead of burning CPU, so the signal read "spare capacity" and the fleet shrank. Switch to requests per instance or latency.

Related patterns: Horizontal Scaling (only interchangeable stateless servers can be added and removed automatically); Load Balancing (health checks let new servers in, draining takes old ones out); Connection Pooling (keeps the surge of connections from each scale-up in check); Message Queue (backlog age is the clearest scaling signal in the course); Bulkhead (the fleet max is a deliberate boundary that protects downstream).

TL;DR

- **Problem:** fixed fleets are wrong both ways — paying for idle servers at 4 AM, and swamped when the TV mentions you — because size is a design-time guess about something that changes hourly.
- **Mechanism:** a control loop that keeps a load signal near ~65%, adding fast and removing slowly through draining; scheduled for known cycles, reactive for surprises; min N+1, max from downstream capacity.
- **Costs:** 3–7 min of reaction lag (headroom bridges it), disasters from the wrong signal (CPU misleads for IO-bound work), flapping without hysteresis/cooldowns, and an automated flood downstream that only your max holds back.
- **Skip it when:** traffic is flat, the tier holds state, downstream is already the bottleneck, or boot time dwarfs the spikes (schedule, predict, pre-warm).

Database Connection Pooling

Original lesson: Database Connection Pooling on DesignGurus ↗

What it is. Opening a database connection is expensive. Running a query over a connection that's already open is cheap. Connection pooling opens a small number of connections once, keeps them alive,

and lets requests *borrow* a connection for a query and *give it back* afterwards. At fleet scale, a shared **pooler** sits in front of the database, so that the connections from many servers get funneled into a small number of real ones. *Analogy:* a company car pool. Instead of every employee buying a car for a ten-minute errand (slow and wasteful), a few cars sit in the lot. Each errand borrows one and brings it back.

The problem — part 1: setup cost. A new database connection needs a TCP handshake (the back-and-forth that opens a reliable network connection), TLS negotiation (setting up the encryption), and authentication — about **20–50 ms** before the first query can even begin. The database also sets aside real resources for each connection: Postgres runs a dedicated *process* (with its own memory) for every one. Spending 20–50 ms of setup on a **5 ms** query — ten times the actual work — makes no sense.

The problem — part 2: multiplication. Most frameworks already handle part 1 with a pool on the app side. The less obvious problem shows up as the fleet grows. A sensible 20-connection pool on each of 50 servers = **1,000 connections**, all aimed at a database built for a few hundred (Postgres's default limit is around **100**), and every extra process eats memory and scheduling time. Then, when the auto-scaler adds five servers during a spike, those servers immediately ask for another 100 connections. The app tier gains capacity while the unchanged database starts turning clients away. (This is the module intro's "scaling one tier multiplies load on the next" fact, in its most common form.)

How it works. The figure below shows the basic app-side pool in four steps. (1) The application asks the in-memory pool for a connection; the pool holds a set of *idle* (available) connections. (2) It uses one, and that connection moves into the *in-use* (checked-out) set. (3) The query runs against the database server (MySQL, PostgreSQL, SQL Server, Oracle…). (4) The connection goes back into the idle set, ready for the next request. Nothing gets opened or closed for each query.

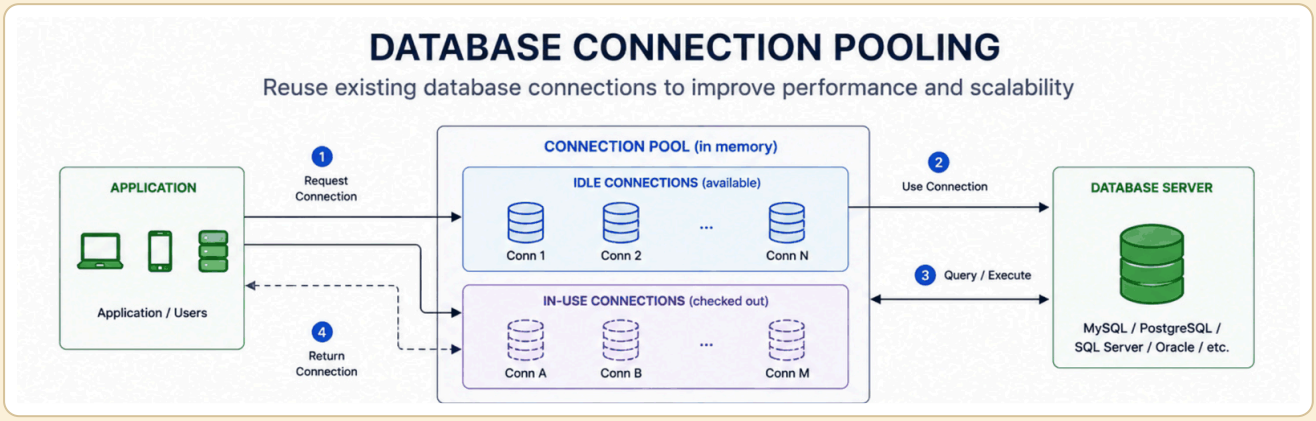


Figure — Database Connection Pooling (course diagram)

1. **Layer 1 — an app-side pool on each server.** At startup, each server opens perhaps **10–20** connections and keeps them warm. Each query borrows one and returns it within microseconds. So the 20–50 ms handshake is paid once per connection *lifetime*, not once per query.
2. **Layer 2 — a shared pooler in the middle.** A proxy such as **PgBouncer** (or the managed **AWS RDS Proxy**) sits between the fleet and the database. It accepts, say, 1,000 cheap *client* connections from the servers and **multiplexes** them (shares a few real connections among many clients, taking turns) over about **50 real database connections**. It lends a real connection to whichever client is running a transaction right now, and takes it back when the transaction ends.

```
Without pooler:  50 app servers × 20 conns ──▶ 1,000 conns ──▶ DB (overloaded)
With pooler:     50 app servers × 20 conns ──▶ PgBouncer ──▶ ~50 conns ──▶ DB (calm)
                 (cheap client connections)   (multiplexes)   (sized to cores)
```

3. **Why so few real connections are enough (Little's law).** Active connections = query rate × query duration. At **400 queries/second** averaging **5 ms**: 400 × 0.005 = **2** connections actually busy at any instant. The fleet can hold a thousand *logical* connections while only about two are doing anything.
4. **Keep pools small — small pools beat big ones.** A database can only run roughly one query per CPU core at a time. Connections beyond that limit on parallel work add no processors. Their queries just wait, while adding context switches (the CPU juggling between processes) and lock contention. So throughput peaks at a modest pool size and can actually *fall* beyond it. HikariCP's (a popular

Java pool) well-known guidance: about **cores × 2** — tens of connections, not thousands. If that feels too few, fix slow queries or add a cache or read replicas (read-only copies of the database that take query load), rather than adding connections.

5. **Read the transaction-mode fine print.** Aggressive poolers multiplex *per transaction*, so a client owns a real connection only for the length of one transaction. Features that assume a private, lasting session — **session variables, temporary tables, some prepared-statement setups** (settings, scratch tables and pre-parsed queries that live only as long as one connection) — can break. Check the pooler's compatibility notes before you migrate.

Trade-offs

- *You gain:* the 20–50 ms handshake paid once per connection lifetime instead of on every query; a database that sees tens of warm connections instead of a flood the size of the fleet (scale-ups stop turning into connection storms); and a **built-in bulkhead** — the pool limits how many queries your service can send at once, which gives you *backpressure* (push-back that slows callers down when downstream is full) by design.
- *You pay:*
  - **Pool exhaustion — a new way to be down.** If every connection stays borrowed — because slow queries hang on to them, or code *leaks* them (borrows without returning) — new requests have to wait for the pool. The queue has moved out of the database and into the app or the proxy. Guardrails: a **borrow timeout** (fail fast rather than hang forever), alerts on **pool utilization and wait time**, and **leak detection**.
  - **Housekeeping.** The pool has to spot and replace dead connections, using *validation queries* (a cheap test query) and a *maximum connection lifetime*. A shared pooler is yet another critical service, so keep it simple and redundant.
  - **The transaction-mode limits** described above.

When not to use it — pooling is nearly universal; the real questions are *where* it runs and *how big* it is:

- **Serverless functions** (for example, AWS Lambda — the provider starts and stops short-lived instances of your code per request) can't keep reliable local pools — each instance lives only briefly, and thousands may start at the same moment. Put the pool in a shared proxy outside the functions. **RDS Proxy** exists precisely to absorb Lambda-style bursts of connections.
- **Small scripts and one-off jobs** that open one connection and then finish get nothing out of a pool.
- **Not a valid exception:** "the pool keeps running out, so make it 500." That just moves the queue into the database, where connections past the parallel-work limit *lower* throughput through contention.
- A related sizing habit: use **separate pools for the primary (writes) and replicas (reads)**, so report traffic can't eat up checkout's connections.

In the interview. It usually shows up as a diagnosis question: "the app tier grew from 10 to 50 servers and now the database refuses connections — why?" Or as "how big should the pool be?", which tests whether you grab for a huge number.

> **30-second answer:** Each connection costs a 20–50 ms handshake and real memory on the database, so every server keeps a warm pool and borrows a connection per query. The trap is multiplication — 50 servers × 20 connections is 1,000 — so at fleet scale a pooler like PgBouncer multiplexes those down to tens of real connections. That's safe because rate × query duration shows only a handful are in use at any instant. Pools should be small, close to the database's core count. Connections beyond the parallel-work limit just queue inside the

> database and add contention. I alert on pool wait time, because exhaustion is an outage mode of its own.

- *Follow-up: "Why does a bigger pool make it slower?"* — The database runs about one query per core. Extra connections don't add processors, so their queries wait while piling on context switches and lock contention. Throughput peaks at a modest pool size and can drop beyond it.
- *Follow-up: "Where should the pool live when auto-scaling is on?"* — At both levels. A local pool on each server for reuse, plus a shared proxy in front of the database that caps the *total* no matter how many servers the auto-scaler spins up. The app tier can change size while the DB's connection count stays fixed.
- *Mistake that fails candidates:* raising the database's connection limit to 5,000 as though connections were capacity (each one costs process memory and adds contention), or leaving out a borrow timeout, which turns slow queries into hung threads across the entire fleet.

Self-check:

- Q1: Why is opening a DB connection expensive, and which two pooling layers does a scaled fleet need? → TCP + TLS + authentication ≈ 20–50 ms, plus a dedicated DB process and its memory. Layer 1: a local pool on each app server for reuse. Layer 2: a shared proxy (PgBouncer) that multiplexes all the logical connections onto a set of real ones sized to the cores, so sensible per-server settings can't multiply into an unsafe total.
- Q2: 400 queries/s at 5 ms average; a teammate wants a pool of 200 "for safety." What's the real need and the cost of 200? → 400 × 0.005 = **2** active; a pool of 10–20 leaves lots of room for bursts. A pool of 200 wastes DB processes and memory even when idle, and under load it lets 200 queries fight over a few dozen cores — moving the queue into the DB and lowering throughput. Safer: a small pool, a short borrow timeout, and wait time that's monitored.
- Q3: After a deploy, p99 latency (the time within which 99% of requests finish) jumps; requests log "pool borrow timeout" while the database is nearly idle; rolling back fixes it. Diagnose and name two guardrails. → A connection **leak**. Some code path (an early return or an exception) borrows a connection without releasing it — for example, the release is missing from a `finally` block — so the pool empties while the DB sits idle. Guardrails: leak detection that reports connections held past a threshold, along with the borrower's stack trace; and alerts on pool utilization and wait time before exhaustion hits (the existing borrow timeout already keeps threads from hanging forever).

Related patterns: Horizontal Scaling (N sensible pools still multiply into too many connections); Auto-Scaling (each new server is a burst of connections unless a shared proxy caps the total); Bulkhead (a pool is bounded concurrency and backpressure toward the database); Primary-Replica (separate read and write pools, so reports can't starve checkout).

TL;DR

- **Problem:** each connection costs a 20–50 ms handshake and a process's worth of DB memory, and per-server pools multiply into a flood the size of the fleet (50 × 20 = 1,000) that auto-scaling grows on its own.
- **Mechanism:** warm app-side pools for reuse, plus a shared pooler (PgBouncer / RDS Proxy) multiplexing the fleet onto a set of real connections sized to the cores — backed by rate × duration, which shows that only a handful are ever busy.
- **Costs:** pool exhaustion as a new outage mode (borrow timeouts, wait-time alerts, leak detection), connection housekeeping, and transaction-mode limits on session features.
- **Skip it when:** practically never — decide where it goes (a shared proxy for serverless and big fleets) and how big it is (small, near the DB's cores; never "500 for safety").

Multi-Region and Cell-Based Architecture

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. Two ways of cutting one big system into pieces that fail separately. **Multi-region** runs the system in two or more geographic **regions** (a region is a cloud location such as "US East" or "EU West", made of several **availability zones**, or AZs — separate datacenters a few kilometres apart). **Cell-based architecture** splits the fleet *inside* each region into **cells**: complete, independent copies of the whole stack (app servers, cache, database shard, queues), each serving a fixed slice of the customers. A thin **cell router** in front sends each customer to their cell. *Analogy:* a ship's hull is divided into watertight compartments. A hole floods one compartment, not the ship (cells). And a shipping company that owns ships in two oceans keeps trading when a storm closes one of them (regions).

The problem. The module capstone (right after this lesson) grows the store to about a million requests per second in **one region, three AZs**. Three AZs protect against a datacenter fire. They do not protect against the next three incidents, all of which happen within a year:

- **The regional dependency.** A provider problem in the region's shared control systems (identity, DNS, the load-balancer API — an API is the interface programs use to ask a service to do something) breaks all three AZs at once: **4 hours** down.
- **The global config push.** A malformed rate-limit rule reaches every server in **90 seconds**, as designed, and checkout fails for **100% of customers**. Three AZs did nothing — the fault came from *inside*.
- **The poison tenant.** One business customer's export script saturates the shared order database; **20 million** other customers get slow pages for an hour.
- And Sydney customers wait **~200 ms per round trip** to the US region on every write.

What connects them: the **blast radius** (the share of users one failure can hurt) of every one of these faults was *the whole company*, because everything was one shared system.

Why the obvious fixes fail

- **"Add a fourth AZ."** AZs protect against one building failing. All AZs of a region still share the regional control plane (the management systems that create, configure and route resources, as opposed to the data plane that serves user traffic) and, above all, *your* code and config.
- **"Back up to another region."** Necessary, but restoring 50 TB onto an untested stack takes **a day or more** — recovery from losing the region forever, not from a 4-hour outage.
- **"Every region active, one globally consistent database."** Every commit (the moment a write becomes permanent) would wait **~75 ms** (US East ↔ EU West) to **~200 ms** (US ↔ Sydney) for another continent — the cost of linearizability (every reader sees the latest write, as if there were one copy of the data) from DDIA Chapter 9. Right for a few datasets, far too high for all.
- **"Test config changes better."** Worth doing, but some faults appear only under production traffic. What's missing is a way to let a bad change hurt *5%* of customers before it hurts *100%*.

How it works — failure domains first. A **failure domain** is the set of things that fail together. List them from small to large, and note what protects against each:

Failure domain	Typical cause	What contains it
Process / host	Crash, disk full, GC pause	Horizontal Scaling + Load Balancing (N+1)
Availability zone	Power, cooling, network in one building	Spreading every tier across 3 AZs
Region	Regional control plane, large network event	**A second (and third) region**
Your own change	Bad deploy, bad config, schema (table layout) mistake	**Cells**, plus deploying one cell at a time
One noisy customer	Pathological workload	**Cells** (or shuffle sharding, below)

The two lower rows are the new ones. Regions answer "what if a whole location goes away?" Cells answer "what if *we* break something, or one customer does?"

Step 1 — pick a multi-region strategy by RPO and RTO. RPO (recovery point objective) is how much recent data you can afford to lose; **RTO** (recovery time objective) is how long you can afford to be down. (The SQL book's High availability section defines both.) The cloud providers describe four standard strategies, from cheapest to strongest:

Strategy	What runs in the second region	RPO	RTO	Extra cost
Backup and restore	Only backups	Hours (last backup)	Hours to a day	Lowest
Pilot light	Data replicated continuously; servers off or minimal	Seconds to minutes	Tens of minutes (start and scale the servers)	Low
Warm standby	A small, working copy of everything, taking no users	Seconds	Minutes (scale up, shift traffic)	Medium
Active-active (multi-site)	Full stack, serving users all the time	Seconds, or zero for synchronously replicated data	Near zero (traffic simply moves)	Highest

Choose per system, not per company: the checkout path may need active-active while internal reporting is fine with backup and restore.

Step 2 — decide where each dataset lives. Servers are easy to copy between regions; data is not. Sort every dataset into one of three kinds:

1. **Home-region data** — each customer's orders, cart and profile live in one **home region** (nearest, or the one the law requires: EU data stays in the EU). All their writes go there; it is copied asynchronously (in the background, without the write waiting for it) elsewhere only for disaster recovery (DR — getting a system back after a major loss). Writes stay local *and* conflict-free, because each record has exactly one region that may change it. Most data should be this kind.
2. **Read-mostly shared data** — the product catalog, prices, feature flags. One region writes; every region holds a read-only copy that lags by a second or so. This is leader-follower replication across continents.
3. **Truly global, strongly consistent data** — unique usernames, the last unit of a limited item, balances spendable from anywhere. Give each such dataset *one* global home region, or use a cross-region consensus database (Spanner, CockroachDB — databases whose replicas vote so that a majority agrees on every write) and pay a cross-region round trip per write. Keep this category small. If two regions must both accept writes to the same record, you are in multi-leader replication and need a conflict rule (last-write-wins, where the newest timestamp wins; CRDTs, data types designed to merge concurrent edits automatically; or the Vector Clocks lesson's siblings).

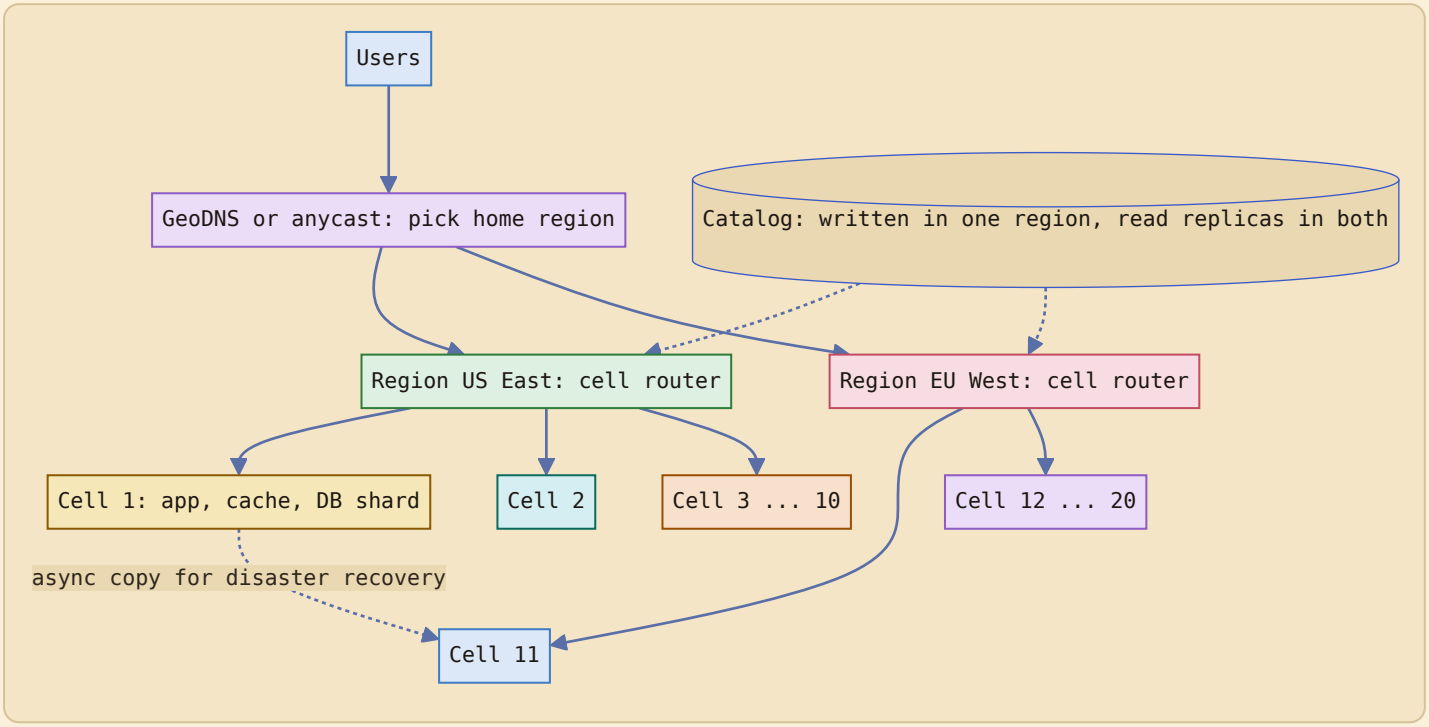
Step 3 — route users to a region. Above the regional load balancer sits **GeoDNS** (the DNS answer depends on where the user is) or **anycast** (many sites announce one IP; the internet routes each user to the nearest). A **global health check** removes a failed region. DNS answers are cached, so DNS failover takes about *TTL + detection* (e.g. 60 s + 30 s); anycast reroutes without waiting for caches.

Step 4 — cells inside each region. A cell is the whole stack, sized to a fixed maximum, serving a fixed set of customers:

- **Fixed size, grown by count.** Pick a cell's maximum (say 1 million customers or 10,000 req/s), load-test it once, and grow by adding cells — never past the tested size. "Cattle, not pets", one level up.
- **The partition key.** Customers are assigned to cells by a key that every request carries (customer ID, tenant ID, account ID). The router looks the key up in a **mapping table**, not a `hash % N` formula, so a customer can be moved to a new cell without reshuffling everyone (the modulo trap from Consistent Hashing).
- **A router as dumb as possible.** It is the one shared component: a cached lookup and a forward, no business logic, and *fail-static* (keep routing with the last known table if the mapping store is down).
- **Deploy in waves.** Code and config go to **one cell first** (a canary *cell*), then a few, then the rest, with a bake time (a waiting period to watch metrics before going further) and automatic checks between waves — the Canary Deployment idea applied to whole stacks. The global config push from the problem becomes a one-cell incident.
- **No cross-cell calls on the request path** — otherwise a failure in cell 3 hurts cell 7's customers. What must be global (sign-in, billing) is a separate, small, carefully built global service.

Shuffle sharding — cells for shared workers. Sometimes giving every customer a whole private cell is too expensive. **Shuffle sharding** gives each customer a small, random *combination* of shared workers instead. With 8 workers and 2 per customer there are 8 × 7 / 2 = **28** possible pairs. A poison customer takes down its own 2 workers; another customer is fully cut off only if it drew *exactly the same pair* — a 1 in 28 (≈ 3.6%) chance — and one sharing a single worker still gets answers from its other one (clients retry). Route 53 (Amazon's DNS service) uses the idea at scale: 2,048 virtual name servers, 4 per customer domain, about **730 billion** possible combinations, assigned so that no two domains share more than two name servers.

Two regions, cells inside each, customers routed by home region and then by cell



GeoDNS/anycast picks the home region, its router picks the cell, and cells never call each other. Dotted lines are background replication: each cell's data to a partner cell for DR, and the catalog to

both regions.

Walkthrough with numbers. The store: **20 million customers**; about **200,000 req/s** reach the app tier at peak (the CDN absorbs the rest — the capstone's Stage 4 numbers).

- **Cells.** A cell is load-tested at **10,000 req/s** and **1 million customers**. 200,000 / 10,000 = **20 cells**, 10 per region. The poison-tenant hour now slows **1 cell = 5%** of customers instead of all of them — and that tenant can be moved to a cell of its own.
- **Config push, replayed.** The rule goes to one cell first. Its error rate jumps within a minute, the pipeline halts and rolls back automatically. Impact: **5% of customers for ~5 minutes**, instead of 100% until someone notices.
- **Static stability.** If US East is lost, EU West must carry all 200,000 req/s *with capacity already running* — scaling up mid-disaster bets on scaling APIs that may be part of the outage. So each of two regions is sized for the full peak and runs at **~50%**: **2.0×** peak paid. With **three** regions, any two must carry the peak, so each needs 100,000 req/s of capacity and runs at ~67%: total **1.5×** peak. A third region is often *cheaper* than a second.
- **Latency.** Sydney customers get a home region in Asia-Pacific: writes drop from **~200 ms** to **~10 ms** round trips. Only the small "truly global" category still crosses oceans.
- **Recovery targets.** **~1 s** replication lag → RPO ≈ 1 s (unless synchronously replicated). Failover ≈ detection (~30 s) + DNS TTL (60 s) + replica promotion (~1 min) → **RTO of a few minutes**.

Trade-offs

- *You gain:* survival of whole-region loss; a bounded blast radius for your own mistakes (1/N per wave) and for noisy customers; lower latency for far-away users; growth by copying a tested unit.
- *You pay:* **money** (static stability = 1.5–2× compute, plus billed cross-region transfer); **data complexity** (every dataset needs a home, a replication mode and a conflict story; global uniqueness and counters get hard); **operational weight** (N databases to migrate, N dashboards, wave-aware deploys); **a new shared failure domain** in the router and global services, which must stay tiny and fail-static; and **drills** — failover that has never been tested does not work.

When not to use it

- **Small scale or low stakes:** one region, three AZs, good backups and a tested restore beat a multi-region design that roughly doubles cost.
- **No clean partition key:** a social graph where everyone interacts with everyone doesn't fit neat cells (partition what you can; keep the rest global).
- **Active-active by default:** many systems need only pilot light or warm standby — decide from RPO/RTO.
- *Common misuse:* a "second region" that depends on the first — a shared global database, config store or sign-in service. The first regional outage takes both down.

Real-world examples

- **AWS** recommends cell-based architecture to limit blast radius, and describes shuffle sharding in Route 53 (the 2,048 name-server example above).
- **Slack** moved to a cellular design in which each availability zone is a cell that can be drained of traffic in minutes, after partial-AZ network failures caused incidents.
- **Netflix** runs active-active across several AWS regions and practises evacuating a region.
- **Google Spanner** productises the "truly global" option: Paxos (a classic consensus algorithm) replication across regions plus tightly bounded clocks (TrueTime), paying the round trip on writes.

- **For AI engineers:** GPU capacity differs by region, so inference fleets go multi-region for *capacity* as much as failure; prompts and documents may have data-residency rules (laws saying which country the data must be stored in), and model rollouts should go cell by cell.

Cloud building blocks (a short map). The same building blocks exist on every major cloud under different names:

Need	AWS	Azure	Google Cloud
Virtual machines	EC2	Virtual Machines	Compute Engine
Managed Kubernetes	EKS	AKS	GKE
Functions (serverless)	Lambda	Azure Functions	Cloud Run functions
Object storage	S3	Blob Storage	Cloud Storage
Managed relational DB	RDS, Aurora	Azure SQL Database	Cloud SQL, Spanner
Key-value / document DB	DynamoDB	Cosmos DB	Firestore, Bigtable
Queue / stream	SQS, Kinesis	Service Bus, Event Hubs	Pub/Sub
CDN / global entry	CloudFront	Front Door	Cloud CDN
DNS	Route 53	Azure DNS	Cloud DNS

- **How serverless functions run.** AWS Lambda runs each instance inside a **Firecracker microVM** — a minimal virtual machine that boots in a fraction of a second and isolates tenants more strongly than a container. A **cold start** (no warm instance available) means creating or restoring a microVM, starting the runtime and running your initialisation code: ~100 ms to several seconds, worst for large JVM (Java Virtual Machine, the runtime that runs Java programs) apps. Remedies: small packages, less start-up work, provisioned concurrency (pre-warmed instances), or snapshot restore (Lambda SnapStart).
- **Cost levers, biggest first:** delete idle resources; right-size to measured use; scale down at night; commit baseline usage for a discount (savings plans, committed-use discounts); put interruptible batch work on **spot** capacity (much cheaper, reclaimable at short notice); tier cold data with lifecycle rules; and watch **data transfer** — cross-AZ, cross-region and internet egress (data sent out of a cloud network) are billed and surprise new multi-region teams.

In the interview. "How would you make this survive a region outage?" and "how do you limit the blast radius of a bad deploy?" are the two prompts. Expect follow-ups on data: "what happens to writes during failover?"

> **30-second answer:** Three AZs survive a datacenter, not a regional control-plane failure or our own bad config. For regions I'd pick the strategy from RPO/RTO and give each dataset a home: customer data in one home region, copied asynchronously for DR; read-mostly data replicated one-way; only a small truly-global set pays for cross-region consensus. Regions are statically stable — each can absorb a failed one without scaling. Inside each region, cells: fixed-size full stacks, each serving a slice of customers behind a tiny fail-static router, deployed one cell at a time, so a bad change or noisy customer hurts at most one cell.

- *Follow-up: "A region dies mid-write — what is lost?"* — With async replication, the writes in the lag window (~1 s) are lost or delayed until the region returns; say so, and replicate synchronously only where that is unacceptable (payments).
- *Follow-up: "Why not grow cells as you grow?"* — A cell's value is its *tested* size; past it, it's an untested system with a bigger blast radius. Add cells.
- *Mistake that fails candidates:* drawing two regions but one shared global database, config store or login service; or claiming active-active without saying how write conflicts are avoided.

Self-check:

- Q1: Failure domains from process to "your own change", and what contains each? → Host: N+1 behind a load balancer. AZ: every tier over three AZs. Region: a second/third region, strategy by RPO/RTO. Your own change and noisy customers: cells deployed in waves (or shuffle sharding).
- Q2: Two regions or three for 300,000 req/s peak with static stability? → Two: each carries 300k → 600k (2×). Three: any two carry 300k → 150k each → 450k (1.5×). Three is cheaper, at the cost of one more region to operate.
- Q3: "We're active-active": both regions serve traffic, but all writes go to a primary in region A, and sign-in lives only in A. Evaluate. → Active-active for reads only: losing A stops all writes and sign-ins everywhere. Fix: home-region writes per customer (or tested automatic promotion in B), sign-in in each region, and regular drills that evacuate A.

Related patterns: Horizontal Scaling (cells are "cattle, not pets" one level up); Load Balancing (DNS and anycast move above the regional balancer); Consistent Hashing (why the router uses a mapping table, not `% N`); Bulkhead (a cell is a bulkhead around a whole stack); Quorum (majorities for the few globally consistent datasets); Canary Deployment (deploying one cell at a time).

TL;DR

- **Problem:** three AZs don't survive a regional failure, and nothing in a single shared system contains your own bad change or one noisy customer — every blast radius is 100%.
- **Mechanism:** regions (strategy chosen by RPO/RTO; data sorted into home-region, read-mostly and truly global; statically stable capacity) plus cells inside each region (fixed-size full stacks, a tiny fail-static router keyed by customer, deploys in waves; shuffle sharding for shared workers).
- **Costs:** 1.5–2× capacity, cross-region data and conflict design, N stacks to operate, a router and global services that must stay tiny, and failover that only works if drilled.
- **Skip it when:** the scale or stakes are small (one region, three AZs, tested backups), or the data has no clean partition key.

Sources: AWS Well-Architected Framework, *Reliability Pillar* and the whitepaper *Disaster Recovery of Workloads on AWS* (the four DR strategies); AWS whitepaper *Reducing the Scope of Impact with Cell-Based Architecture* (2023); C. MacCárthaigh, "Workload isolation using shuffle-sharding", *Amazon Builders' Library*; B. Weiss and M. Furr, "Static stability using Availability Zones", *Amazon Builders' Library*; Slack Engineering, "Slack's Migration to a Cellular Architecture" (2023); Netflix Tech Blog, "Active-Active for Multi-Regional Resiliency" (2013); J. Corbett et al., "Spanner: Google's Globally-Distributed Database", OSDI 2012; A. Agache et al., "Firecracker: Lightweight Virtualization for Serverless Applications", NSDI 2020; Kleppmann, *DDIA* ch. 5 and 9 (Replication, Consistency and Consensus).

Capstone - From 100 to 1,000,000 Requests per Second

Original lesson: Capstone - From 100 to 1,000,000 Requests per Second on DesignGurus ↗

What this capstone is. All five patterns in this module share one habit: **capacity follows the bottleneck**. This capstone grows a single service across four orders of magnitude (four factors of 10) (100 → 1,000,000 requests/second) and asks the same three questions at every stage:

1. **What is limiting throughput right now?**
2. **What is the cheapest tool that moves that limit?**
3. **What should stay unchanged?**

At each "pause and decide," name the constraint *before* you choose a pattern. Interviewers run exactly this exercise — "now traffic is 10× — what breaks next?" Strong answers track the moving bottleneck, rather than drawing the full planet-scale architecture on day one.

The brief. The store's public API (browsing, search, product details) starts out simple:

- **One modest 4-core server** that runs the app, keeps sessions in memory and files on local disk, and runs a cron job at midnight.
- **100 requests/second** on average, and one database primary (the main database copy that accepts writes; the storage module's patterns haven't been applied yet).
- The business grows about **3× a year**, and marketing has big plans.

Stage 1 — 100 → 1,000 req/s: the single point of failure

- **Incident.** As peaks reach several hundred req/s, the server goes past **70% CPU**. Moving up **two instance sizes** solves capacity in an afternoon — the right move (Vertical Scaling). Eight months later, close to 1,000 req/s, that bigger host dies at lunchtime: **40 minutes of total outage**. Capacity isn't the problem anymore. The problem is that there's only *one* server.
- **Pause and decide:** the postmortem (the written review after an incident) says "single point of failure." What's the fix, and what hidden work does it uncover?
- **Decision: at least 2 servers behind a load balancer with health checks — plus the statelessness checklist.** Adding the second server immediately brings local state to the surface. Users lose their in-memory sessions, images go missing across local disks, and the midnight job runs twice. So: sessions → Redis, files → object storage, and the cron → behind a distributed lock. Sticky sessions serve as a **two-week migration bridge** and are removed once shared state is working.
- **Why this instead of an even bigger box:** a bigger box keeps the same failure mode. The real win isn't "two processes running." It's that *any* server can take *any* request, so losing one lowers capacity rather than wiping out sessions or stopping the service. That property makes every later change to the fleet routine.
- **Not scaled:** the database — it's mostly idle. At ~800 req/s per server, two servers (1,600 req/s together) cover the 1,000 req/s peak. Be precise about what that buys: it removes the single point of failure, but it is not strict N+1 — if one server dies, the survivor's 800 req/s is below the 1,000 req/s peak, so peak traffic would degrade. Strict N+1 here would be 3 servers (2 for peak + 1 spare).

Stage 2 — 1,000 → 10,000 req/s: the fleet, and the first downstream flood

- **Incident.** Growth pushes both servers past 80% at every peak. Because they're stateless, adding a third server and then a fifth is routine. But at **~10 servers** the next limit shows up downstream: **database read latency doubles at peak**, because growing the app tier also multiplied the queries about fivefold.
- **Pause and decide:** the app tier is healthy, and the database is drowning in reads. Which earlier modules own this problem, and what app-tier discipline goes along with them?
- **Decision.** The fleet grows up: **13 servers (12 for peak + 1 spare)** (check the math: 12 × 800 = 9,600 req/s, enough for a peak just under 10,000; a full 10,000 req/s peak would need 13 + 1 spare) behind **least-connections**, with rolling deploys and drain hooks. The read pressure gets routed cleverly instead of being dumped on the DB. **Cache-aside** (the app checks the cache first and, on a miss, reads the database and fills the cache) reaches a **>98% hit ratio** (share of reads answered from the cache) on product reads, cutting database reads by about **40×**, and **read replicas** handle the misses that remain. Also audit the app-side pools: 13 servers × 20 connections = **260 connections**, which is already close to a common database limit.
- **Why this instead of a bigger database:** reads that never reach the DB are cheaper than a bigger DB answering them, and the caching and storage modules already have the tools.
- **Not scaled:** no auto-scaler yet. Traffic is predictable enough for a fixed N+1 fleet, and the team's attention is better spent on cache correctness than on a control loop that comes too early.

Stage 3 — 10,000 → 100,000 req/s: the fleet becomes elastic

- **Incident.** The fleet of **~65 servers** sized for peak sits **below 30% utilization 18 hours a day** (a big 4 AM bill). Then a successful campaign brings **3× normal peak within 20 minutes**, which swamps the fixed fleet. The size is wrong in both directions — the Auto-Scaling incident all over again.

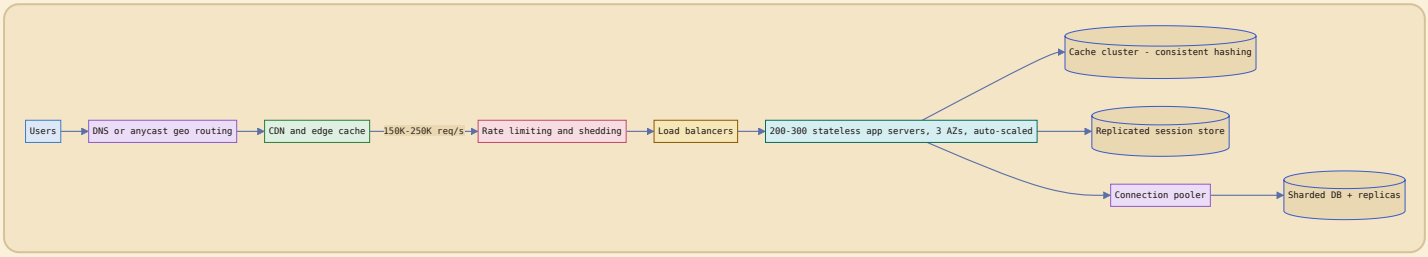
- **Pause and decide:** design the loop — signal, target, min, max. Where does the max come from, and what does each scale-up do downstream?
- **Decision: auto-scaling, with the homework done.**
  - **Signal:** requests per instance against measured capacity for the IO-bound API (CPU gives a false picture of blocked IO work); **backlog age** for async workers.
  - **Scheduled + reactive:** add capacity ahead of the known 6 PM ramp; a reactive **65%** target for surprises; scale up fast and down slowly, with cooldowns to stop flapping.
  - **Min 8, max 120:** the max comes from the measured capacity of the database, cache and replicas — a **bulkhead** that protects them, not a hopeful guess.
  - **The pooler:** up to 120 servers × 20 connections = **2,400 clients** hitting the database. Put **PgBouncer** in front to multiplex them onto about **60 real connections**. Rate × duration shows that only a few are ever active at once, so scale-ups stop causing connection storms.
- **Not scaled — avoided instead:** move images and JavaScript to a **CDN** (covered in the API & Edge module). About **one-third of requests** never reach the app tier at all anymore. Getting rid of traffic beats serving it more efficiently.

Stage 4 — 100,000 → 1,000,000 req/s: putting it all together

- **No single incident** — earlier stages have already removed or contained the ways things fail. The useful question now is how a nominal million requests per second actually spreads across the architecture.
- **Pause and decide:** what share of requests do you think ever reaches an app server, and which tier is the hard one now?
- **The honest anatomy of a million:**
  - **Most requests never get to your servers.** CDNs and edge caches serve static and public cacheable content, and the cache tier soaks up most of the remaining reads. Perhaps **150,000–250,000 req/s** make it to the application fleet. Avoiding a request is cheaper than serving it.
  - **The app fleet is big and boring: 200–300 stateless servers across three availability zones** (separate data-center locations), auto-scaled, drained, and rolled. Boring is the *outcome* of every pattern in this module.
  - **The hard tiers are the ones that hold state,** and each has its own specialized design: a **sharded** database (storage module), a cache cluster using **consistent hashing** ("the ring"), and a **replicated session store**. State now lives in systems built specifically to own it.
  - **The front door gets a module of its own.** At this scale, rate limiting, shedding beyond the fleet max, and geographic routing become first-class concerns — which is where the next module starts.

In the diagram below, follow a request from left to right. Users are routed by geography (DNS or anycast) to a CDN/edge cache, which absorbs most of the traffic. The remaining 150K–250K req/s go through rate limiting and shedding, then load balancers, into the stateless app fleet. The app servers talk to the tiers that hold state — the consistent-hash cache cluster, the replicated session store, and (through the connection pooler) the sharded database with replicas.

Architecture at 1M req/s: most traffic is absorbed before the app fleet; state lives in specialized tiers.



Debrief: the bottleneck's journey

Stage	The bottleneck was	The move	Where it went next
100 → 1K	One machine's capacity, then the fact that there was only one	Bigger machine, then 2+ servers + statelessness + LB	Into growing the fleet
1K → 10K	App capacity (routine), then database reads	Fleet + N+1; cache + replicas	Toward connections and cost
10K → 100K	Cost at 4 AM, surprises at 7 PM, connections all the time	Auto-scaling (signal, max-as-bulkhead) + pooler + CDN	Toward the tiers that hold state
100K → 1M	The tiers that hold state, and the front of the system	Work from earlier modules + the next module's patterns	Where it always ends up: the data

Takeaways

- Look at the last column. Every stage *moves* the bottleneck instead of getting rid of it, each time putting the limiting work in the tier with the best tools for it. Scaling means applying the **cheapest pattern that's enough for the current constraint**, while leaving unrelated tiers alone.
- **Order matters.** Using a later-stage pattern too early (an auto-scaler at 1K req/s, sharding at 100 req/s) adds cost and complexity without moving the current limit. Putting off an earlier pattern you need (statelessness, pooling) makes the next 10× unsafe.
- Many of the answers come from other modules — caches, replicas, shards, idempotent retries — which is why scaling questions test the whole course.
- **Beyond a million:** the next bottleneck is no longer throughput but *blast radius* — one region, one deploy or one tenant can still hurt everyone. Regions and cells (the added lesson before this capstone) cap that: each region is statically stable, and each cell is a tested, fixed-size copy of the stack that fails alone.

> **Interview one-liner:** Scale by following the bottleneck: a bigger box, then stateless N+1 behind a load balancer; then a cache and replicas for database reads; then auto-scaling capped by downstream capacity, plus a connection pooler and a CDN; and at a million req/s most traffic never reaches the app tier — the hard problems are the tiers that hold state and the front door.

Extend it yourself

- **Multi-region (active-active, two regions):** which tiers replicate cleanly? The stateless fleet, trivially. Which become truly hard? The database (the Consistency module). Where does balancing move to? Up to DNS/anycast, above your regional balancers. (The Multi-Region and Cell-Based Architecture lesson works this through: home regions for data, static stability, and cells deployed one at a time.)
- **The stateful edge:** add **500,000 concurrent WebSocket users**. Which assumptions still hold — any-request-anywhere, drain-in-30-seconds, least-connections — and which need the WebSocket lesson's two-tier redesign?
- **For AI engineers:** scale an inference endpoint from **10 to 10,000 req/s**. The "servers" cost **~100×** more and take minutes to boot, the cloud may refuse to sell you more (quotas), and the work per request varies by **~100×**. Which stages carry over directly, and which have to be rethought, starting with the scaling signal?

Module 7 — Keeping Data Consistent (Distributed Consistency)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Picture one database handling "move $50 from checking to savings." Either both halves happen, or neither does. That all-or-nothing promise has a name: **atomicity**. A single database gets it almost for free from its **write-ahead log (WAL)**. The WAL is a file where the database first writes down, durably (saved to disk so it survives a crash or power loss), what it plans to do. Only then does it do it. If it crashes halfway, it reads the log and either finishes or reverses the work. Now split the data across several machines (shards — each machine holds one slice of the data) or several services (separately deployed programs that each own their own database). No one WAL sees both halves anymore. Say one change lands on shard 2 and its partner change lands on shard 5. Nothing ties them together on its own. This module shows how to get all-or-nothing back across machines. It also shows what each approach costs you in speed, availability, or complexity.

> **Analogy.** Think of a single database as one bank teller with one ledger. Moving money between your two accounts takes a single pen stroke. A distributed system is more like two separate banks. Each must update its own ledger for the same transfer. Somebody has to coordinate them, and every coordination method comes with a price.

Only four patterns live here, so it is the smallest module. It is also the toughest material.

The four patterns

The situation	The pattern
Several parties must either all commit (make their changes permanent) or all back out	Two-Phase Commit (2PC)
A workflow has many steps, and any one of them might fail midway	Saga
Several copies of the same data need to agree, with no single leader (main copy that takes all writes)	Quorum
We need to know who saw which update, and in what order	Vector Clocks

Two facts that organize the module

Fact 1 — when the network splits, you have to pick a side. A **network partition** happens when groups of machines keep running but lose contact with each other. At that moment a system has two options. Option (a): stop serving requests until the link comes back. Data stays **consistent**, but you lose **availability**. Option (b): keep serving on both sides and sort out the differences afterwards. You stay available, but the data is briefly inconsistent. This unavoidable choice is known as the **CAP trade-off** (Consistency, Availability, Partition tolerance). (The System Design Fundamentals primer introduces CAP, replication and sharding from scratch.) Smarter engineering cannot make it go away. It comes from how networks physically behave — much like how availability shrinks as you chain components together. Each pattern here makes its own choice. Interviewers also like to ask about CAP directly.

Fact 2 — the cheapest distributed transaction is one you never need. (A *distributed transaction* is one all-or-nothing change that spans several machines or services.) Coordinating many machines is costly. So reach for cheaper tools first:

- **Shard-key discipline** (covered in the sharding lesson): choose the shard key (the field that decides which shard a record lives on) so that a single customer's checkout sits on a single shard. Then a normal local transaction (one that runs inside a single database) still does the job.

- **Idempotent operations** and **commutative operations** shrink what truly needs agreement. Idempotent means repeating it is safe: running it twice has the same effect as running it once. Commutative means the order doesn't matter: any order gives the same result.

In most systems, only a few flows truly need distributed-transaction machinery. Typical examples are moving money between different owners, or promising stock across services. Save the heavy machinery for those.

How to read the module

The store's order workflow now touches three services: inventory, payment, and shipping. We need all-or-nothing behavior across all of them again.

- **Two-Phase Commit** relies on a coordinator that gathers a unanimous "yes". The catch: every participant is stuck waiting until the coordinator announces its decision.
- **Saga** is the usual answer between services. You run the steps in order. If a later step fails, you run "undo" steps (called compensations) for the earlier ones that already finished. It reuses the pub/sub (publish-subscribe: a sender broadcasts events and any interested service receives them) ideas from earlier.
- **Quorum** deals with several leaderless replicas (copies of the same data kept on different machines) of the same data. You pick how many replicas must confirm a write (W) and how many a read must ask (R). Choose them so that **W + R > N**. Then any read and any write always have at least one replica in common.
- **Vector Clocks** keep track of which updates were aware of which other updates. That lets a leaderless system tell edits made at the same time apart from edits made one after another.
- The **capstone** builds order fulfillment two ways — with 2PC and with a saga — and compares how each one breaks.

> Short on time? Start with **Saga** and **Quorum**. Sagas answer "how do services do transactions?" Quorums answer "how do replicas agree?" Together they cover most interview questions on this topic.

Four terms used throughout

- **Atomicity** — all-or-nothing. A set of changes happens entirely or not at all. One machine gets this for free from its WAL. Across machines it is costly to build.
- **Partition** — a network split. Both sides are still alive, but they can't reach each other. Partitions *will* happen, and they are what forces Fact 1's choice.
- **Strong vs. eventual consistency** — with *strong* consistency, every read returns the latest write, just as if only one machine existed. With *eventual* consistency, the copies do become identical eventually, but a read may see slightly old data for a moment. You saw this earlier as **replication lag** — the gap between a write reaching the primary and showing up on a replica — and as stale views.
- **Compensation** — an "undo" carried out as a *brand-new* action: cancel the booking, refund the charge, release the hold. It differs from a rollback because nothing gets erased. Businesses have corrected mistakes this way on paper for hundreds of years. Sagas are built from this idea.

Before you start — questions to ask about your own system

1. Do you have an operation that writes to two databases (or two services) and just trusts that both writes succeed? What does your data look like on the day one of them fails?
2. When you read from a replicated store, how many copies back up the answer you got? Do you *know* this, or are you hoping?
3. Two users change the same record in the same second. Which change survives? Does the other user ever learn their change was lost?

Two-Phase Commit

Original lesson: Two-Phase Commit on DesignGurus ↗

What it is. Two-Phase Commit (2PC) is a protocol that gives true all-or-nothing behavior to one operation spread over several machines. There are two roles. The **coordinator** runs the show. Each **participant** is one of the databases involved. First, the coordinator asks every participant, "Can you commit this?" Each answer is a binding yes or no. Only when every answer is yes does the coordinator tell everyone to commit. In short, no one commits until all of them are able to.

> **Analogy.** Think of a wedding. The officiant asks each partner, "Do you?" Only after both answer "I do" are they declared married. After saying "I do", a partner can't just leave on their own; they have to wait for the officiant's announcement. And if the officiant faints right then, both partners are left standing at the altar.

The problem: $50 that vanished

The store now lets customers send gift-card balances to each other. Customer A's data sits on shard 2 and A sends $50 to customer B, whose data sits on shard 5. So a single logical operation touches two databases, and no one transaction spans both.

Version one simply performs two writes, one after the other: take $50 off on shard 2, then add $50 on shard 5. Thousands of transfers go through fine. Then, at **2:37 AM**, the application server (the machine running the store's own code) crashes. The subtraction had already committed. The addition had not been sent yet. No retry happens, because the process that knew "shard 5 still needs its addition" no longer exists. No error gets logged either. The nightly reconciliation job (a batch check that compares records and flags mismatches) later notices $50 is missing. (Had the writes run in the reverse order, the very same crash would have *invented* $50 from nothing.) Both shards behaved perfectly on their own. The problem is that the *operation as a whole* was never atomic. All-or-nothing across machines doesn't come for free; you have to build it.

Why the obvious fixes fail

- **"Write A, then B, and keep retrying B until it works."** Idempotency keys help here. An idempotency key is a unique ID per operation, so a repeated request is applied just once. That makes retries safe. Still, two holes are left. First, if shard 5 is down for an hour, users see money missing for that whole hour. Second, if the application itself dies between the two writes, the retry loop in its memory dies too. The plan to finish the job has to be saved durably, outside the process. (The saga lesson builds directly on this idea.)
- **"Do both writes in parallel."** Either write can still fail by itself. So you don't get rid of the half-done state; you now have two possible half-done states instead of one. Being faster is not the same as being atomic.
- **"Let the nightly reconciliation fix it."** Reconciliation only *spots* a mismatch, and only after users may have seen it. It doesn't *stop* the mismatch. That works if the business can live with hours of inconsistency. It doesn't work for operations where a half-done state must never be seen.

How it works

Look at the course figure below. The coordinator speaks straight to Shard 2 (subtract $50) and to Shard 5 (add $50). The two shards never speak to each other. That is precisely why a silent coordinator can freeze both of them.

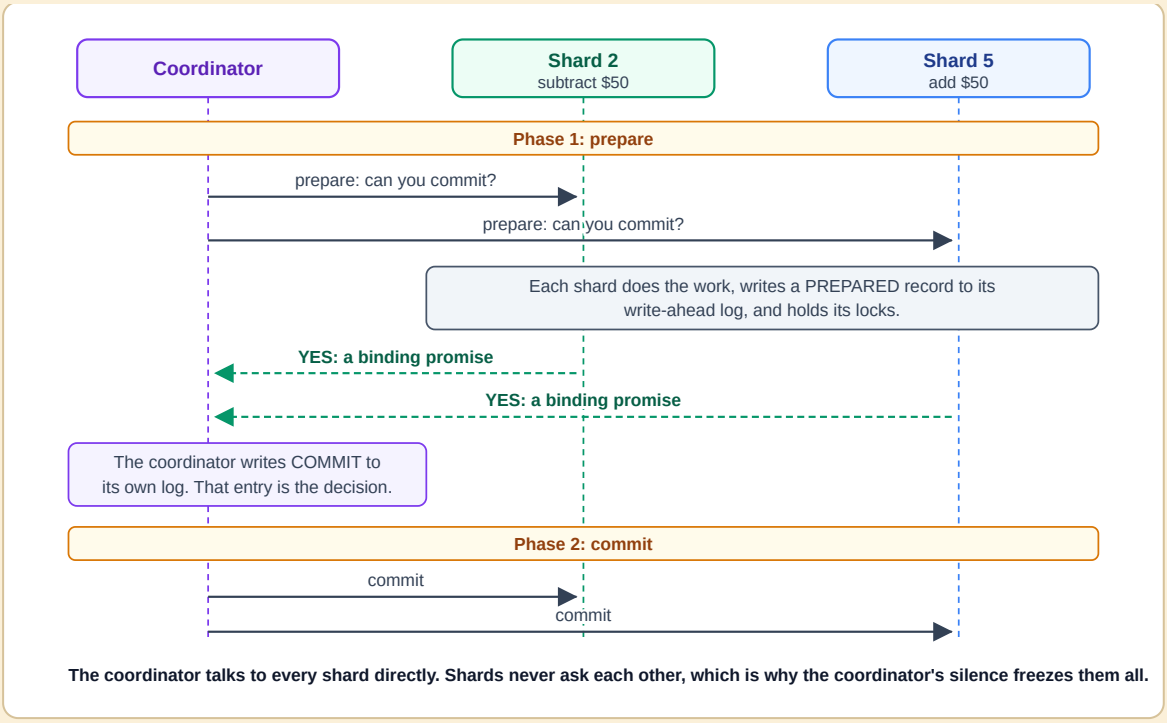


Figure — Two-Phase Commit (course diagram)

1. **Phase 1 — Prepare.** The coordinator asks each participant, "Can you commit this — yes or no?" Before answering, each participant does *all* of the work. It writes a durable **PREPARED** entry into its own write-ahead log. It **keeps its row locks** (a lock stops other transactions from changing — or sometimes reading — those rows until it is released). Only then does it vote. *Why:* because the work is done and logged first, the participant can still carry out a "commit" even after crashing and restarting.
2. **A yes vote is a binding promise.** It means: "The work is done and safely on disk. I can commit even if I crash. I'll keep my locks until you tell me the result."
3. **The coordinator writes down its decision.** If every vote is yes, it logs COMMIT in its own log. If any vote is no, or any participant stays silent, it logs ABORT. *Why:* that one log entry *is* the transaction's final outcome. Recovery can always look it up.
4. **Phase 2 — Commit or abort.** The coordinator sends the decision to everyone. Each participant commits (or aborts) and then lets go of its locks.

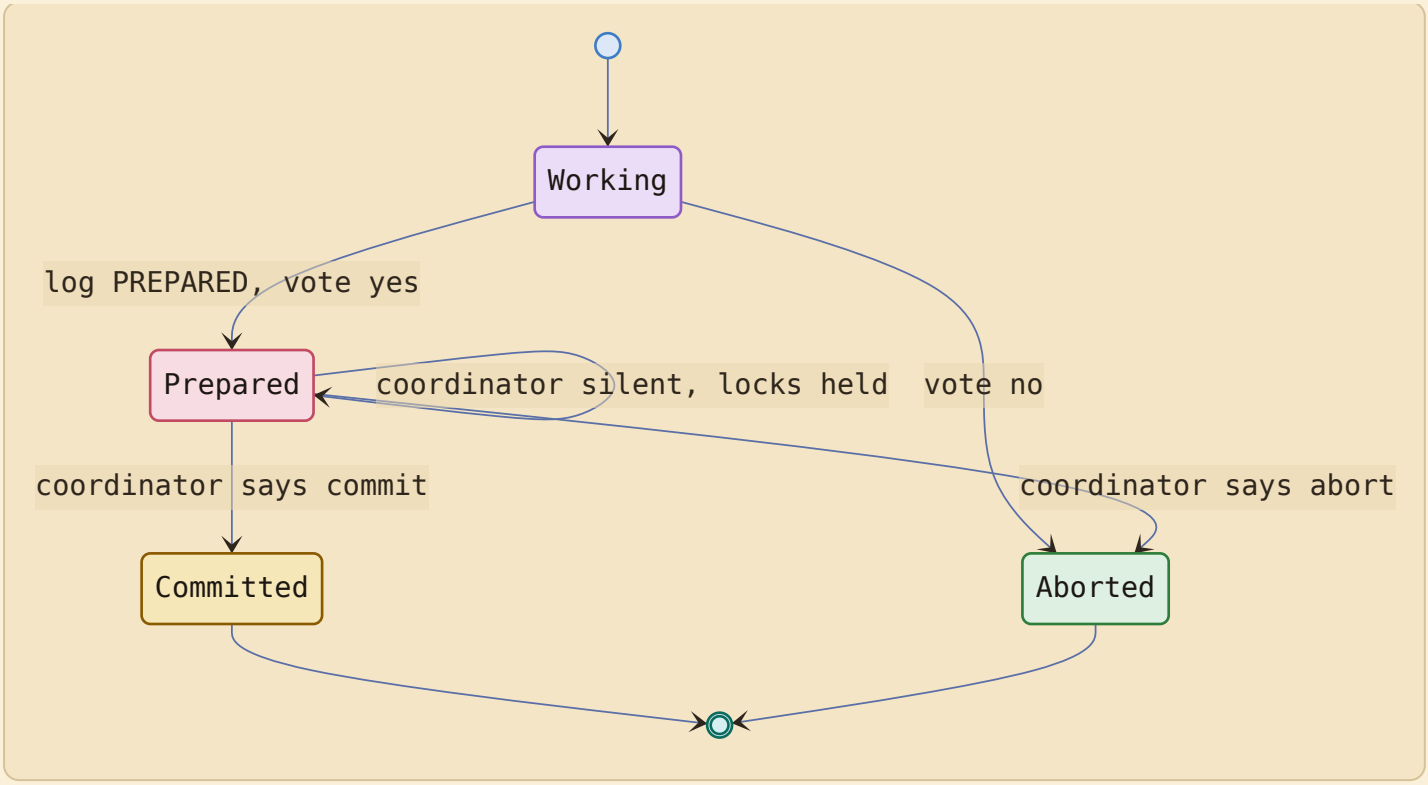
Ordering rules that make recovery safe. Two rules matter. A participant must log PREPARED *before* it votes yes. The coordinator must log its decision *before* it announces it. Thanks to these rules, recovery never has to guess. A participant that restarts reads its own log, then asks the coordinator for the decision. It does not re-run any business logic.

The in-doubt state (the pattern's big risk). Once a participant votes yes, it is **in doubt**. It can't commit by itself, since another participant may have voted no. It can't abort by itself either, since the coordinator may have already chosen commit. So all it can do is wait, still holding its locks.

This is why a silent coordinator freezes everyone. Suppose the coordinator crashes right after collecting the yes votes. Every participant is now in doubt with its locks held. Any other transaction that needs those rows is blocked until the coordinator comes back. 2PC achieves atomicity by placing the decision in one spot. In exchange, it takes on that one spot's weakness.

The state diagram below traces one participant's life. Once it reaches `Prepared`, only the coordinator can move it on. If the coordinator says nothing, the participant keeps looping in `Prepared` with its locks still held.

Participant states: after voting yes, only the coordinator can move it



Walkthrough with numbers (the $50 transfer under 2PC)

- **Cost when all goes well.** You pay for two network round trips (a round trip is one message sent plus its reply) (one for prepare, one for commit). You also pay for a durable disk write at every participant and at the coordinator. That disk write is an **fsync**, which forces data onto disk before moving on. Altogether the transfer takes about **4x** as long as one write on a local shard. You'll notice it, but it's often acceptable.
- **The hidden cost: time spent holding locks.** Each shard keeps the gift-card row locks from prepare all the way to commit. That stretches lock time from roughly **1 ms to 10–20 ms**. Recall the rule *rate x duration = concurrency*. If locks are held **10x longer**, you get roughly **10x more collisions** on busy rows. So 2PC slows down its neighbours, not only itself.
- **Failure drill.** The coordinator crashes **5 ms** after both shards vote yes. Both shards stay in doubt with locks held. Eventually a replacement coordinator replays the log and sends commit — **40 seconds** later. During those 40 s, any transaction that touches either customer's rows waits or fails.
- **Replicating the coordinator** makes recovery faster, because the decision survives the loss of one machine. It does *not* get rid of prepare-time locks, the extra round trips, or the need for every participant to be online. It eases the single worst failure, but the protocol's basic cost stays the same.
- **Availability math.** The coordinator and both shards all have to be up at once. If each of the three is up **99.9%** of the time, the combination is up only about **99.7%**. Every extra participant pushes that number lower.

Trade-offs

You gain:

- **True atomicity across machines.** The "vanished $50" kind of bug simply can't happen by design, so no reconciliation is needed.
- **Strong consistency.** Nobody ever catches the half-done moment where money has left A but not yet reached B.
- **It looks normal to application code.** With driver (the library an app uses to talk to a database) support it feels like an everyday transaction. The **XA standard** is a common interface for 2PC, and Java's **JTA** is built on top of it.

You pay:

- **Blocking — its trademark cost.** In-doubt participants keep their locks even while the coordinator might be unreachable. For *making progress*, the coordinator is a single point of failure, since no participant can finish by itself. Prepared transactions that get stuck are a genuine emergency for database administrators.
- **Latency and throughput.** Two round trips, several fsyncs, and longer lock holds all add delay and create contention (many requests competing for the same resource and waiting on each other) on popular rows.
- **Availability shrinks as you multiply.** No one can commit unless every participant is up.
- **It can't cross trust boundaries.** A **trust boundary** is the line between your systems and another company's. No outside company will allow your coordinator to hold locks in its database. So 2PC stays inside your own walls.

When not to use it

- **Between microservices.** When locks are held across service boundaries, every participant's speed and uptime become tied to the slowest one. A checkout might take seconds or even minutes — much too long to keep locks. The coordinator also turns into a bottleneck, both technically and between teams. Sagas were made to solve exactly this.
- **Hot rows (rows that a large share of traffic reads or writes), operations across companies, or flows where "fixed by morning" reconciliation is good enough.** In these cases, 2PC's atomicity costs more than the inconsistency it would prevent.

Where it *does* fit: operations that are short, internal, rare, and high-value — and most of all, **inside distributed databases**. Google Spanner (Google's globally distributed SQL database) runs 2PC between groups of shards. But in Spanner, each participant and the coordinator is not one machine. Each is a **quorum-replicated group**, meaning a majority of its copies must agree. A majority can keep working when individual machines die, and that removes most of the risk of a blocked coordinator. So 2PC isn't obsolete, and it isn't for everything either. Its best home is inside a database whose participants are replicated.

Operational safeguards (even there): set strict deadlines on transactions. Make the age of prepared transactions visible. Alert *before* held locks start slowing unrelated work. Build recovery tools that display the coordinator's decision and each participant's state. This matters because people guessing during recovery can cause the very inconsistency 2PC is meant to prevent.

2PC vs. Saga (preview)

	Two-Phase Commit	Saga
Atomicity	Genuine: all-or-nothing, guaranteed by locks	Imitated: each step commits separately; if something fails, compensations undo earlier steps
Intermediate states	Hidden from everyone	Visible — the outside world can see "payment taken, shipping pending"
Blocking	Yes — in-doubt participants keep their locks	No — every step is an ordinary local transaction
Coordinator dies	Participants are frozen	The workflow picks up again from its durable log
Fits	Short, internal, rare work (or work inside a database)	Business workflows that span services

The whole contrast in one line: **2PC hides the in-between state by holding locks; a saga lets that state show and manages it.** All the other differences come from that one choice.

Real-world examples

- **XA standard and Java's JTA** — 2PC plumbing for committing databases and message brokers (servers that receive, store and hand out messages between programs) together. Banks and airlines still run on these systems.
- **PostgreSQL** `PREPARE TRANSACTION` (finished with `COMMIT PREPARED` or `ROLLBACK PREPARED`) — lets you play the participant role directly. The PostgreSQL docs warn you not to use it casually. A prepared transaction that gets abandoned keeps its locks forever, until an operator steps in.
- **Google Spanner** — runs 2PC across shard groups whose participants are replicated with **Paxos**. (Paxos is a consensus protocol that lets several copies settle on one value.) Replication soaks up single-machine failures, but you pay for the consensus overhead.
- **Kafka transactions** — a protocol shaped like 2PC for atomic consume-process-produce work (Kafka is a widely used distributed log that stores streams of messages). One broker plays the coordinator.
- **For data engineers:** writing exactly-once (each record's effect lands one time, never zero or twice) into external systems often relies on this pattern. **Flink** (a stream-processing engine) transactional sinks (the output stage that writes results to another system) treat a checkpoint (a saved snapshot of the job's progress) as the "prepare" step, where every operator (one processing step in the pipeline) durably records that it's ready. Finishing the checkpoint acts as the "commit" into Kafka or a database.

In the interview

You may be asked, "How can two services update their databases atomically?" Walk through the full reasoning: 2PC gives real atomicity; it blocks when things fail; today it mostly lives inside distributed databases; and service workflows use sagas instead. Answering "use 2PC everywhere" or "2PC is dead" both show you don't know its real limits.

> **30-second answer:** "Two-phase commit delivers genuine atomicity across machines. A coordinator gathers binding yes votes. To vote yes, each participant does the work, logs it durably, and keeps its locks. Then the coordinator tells all of them to commit. The downside is blocking. A participant that voted yes can't commit or abort by itself. So if the coordinator dies at a bad moment, everyone freezes with locks held. On top of that you get two round trips, and availability that drops with every participant. That's why I use sagas between microservices — local transactions plus compensating undo steps. And I respect where 2PC lives today: inside distributed databases like Spanner. There, every party is quorum-replicated, so no single crash can block the protocol."

Likely follow-ups (with answers):

- *"The coordinator crashes just after both participants vote yes — walk me through it."* Both participants are in doubt. They are bound to obey the decision but can't make it themselves. They hold their locks until the coordinator recovers and replays its log. If the tooling is weak, a person may need to resolve the prepared transactions manually. The word to use is **blocking**.
- *"Why is 2PC fine inside Spanner but wrong between your services?"* Inside the database, participants are replicated groups, transactions last milliseconds, there is one trust boundary, and one operations team runs it all. Between microservices, participants are usually single machines, business workflows last several seconds, and separate teams own each piece. Every weakness of 2PC shows up at full force, and none of the protections apply.

Mistake that fails candidates: suggesting 2PC for a checkout that runs for several minutes; never covering what happens when the coordinator fails; or calling 2PC obsolete without realizing databases still use it internally.

Self-check

- **Q1: Name the two phases, state precisely what a yes vote promises, and explain how that promise leads to blocking.** → First comes prepare: participants do the work, write PREPARED durably, keep their locks, and vote. Then comes commit/abort: if every vote is yes, everyone commits; otherwise, everyone aborts. A yes means "I can commit even after a crash, and I'll keep my state and locks until you decide." The participant can't commit alone, because someone might have voted no. It can't abort alone, because commit might already be decided. So if the coordinator goes silent, the participant is blocked with no end in sight.
- **Q2: Would you use 2PC for (a) the gift-card transfer — two shards, same cluster, ~50 ms — and (b) checkout across inventory, payment, and shipping services, 2–30 s?** → (a) It's defensible: internal, rare, one trust domain, and short. But first ask whether both balances could sit on one shard (Fact 2). (b) Use a saga. Holding locks across services for seconds or minutes would destroy throughput and chain three teams' availability together.
- **Q3: You get paged at 3 AM. Queries on the orders table have been hanging for four minutes. The DBA finds several transactions, hours old, sitting in `PREPARED` and holding locks. What happened, what do you do now, and what should the postmortem ask?** → Those participants voted yes, but the coordinator never sent its decision. Maybe it crashed, lost its log, or failed between the two phases. PostgreSQL keeps their locks until someone tracks down the coordinator's decision and runs `COMMIT PREPARED` or `ROLLBACK PREPARED`. If no decision survived, the business must choose by hand. For the postmortem, ask: does this flow truly need 2PC? And if it does, why was the coordinator not replicated? The fix is either a saga or a replicated coordinator.

Related patterns

- **Write-Ahead Log** — holds the PREPARED records and the coordinator's decision. The promise can only be as durable as this log.
- **Saga** — the replacement for workflows between services (next lesson).
- **Sharding** — the reason 2PC became necessary. Choosing a good shard key is still the cheapest way to avoid it.
- **Quorum** — how modern systems make participants and coordinators survive crashes (the Spanner approach).

TL;DR

Problem	When one logical operation spans two machines, there's a gap where a crash can leave it half-committed (money disappears or doubles). All-or-nothing across machines has to be built; you can't just assume it.
Mechanism	Prepare: each participant does the work, logs that it's ready, keeps its locks, and votes — a binding promise. Commit: if every vote is yes, all commit; otherwise, all abort. The coordinator's log stores the final decision.
Costs	Blocking (in-doubt participants freeze with locks if the coordinator goes quiet), two round trips plus fsyncs, contention from long-held locks, and availability that drops with each participant.
Skip it when	You're between microservices, across companies, on hot rows, or running long flows (those belong to sagas). Where it truly fits: short internal transactions, and inside quorum-replicated databases.

Saga

Original lesson: Saga on DesignGurus ↗

What it is. A saga breaks a workflow that spans several services into a chain of plain **local transactions**. Each one commits right away inside its own service. No locks are held across services. Before each step, the saga writes its progress to a durable log. If a later step fails, the saga runs

compensations in reverse order. A compensation is a new action that cancels out the business effect of a step that already finished.

> **Analogy.** Think about booking a trip. First the flight, then the hotel, then a rental car — each from a different company. If the hotel booking falls apart, you can't "turn back time". Instead, you *cancel the flight*. The companies' cancellation policies play the role of compensations. (The word "saga" comes from a database paper about long-running transactions, and travel booking is where the idea started.)

The problem: 340 orders stuck in limbo

Checkout now involves three services. It reserves inventory, then charges the card via an **external payment provider**, then creates a shipment. The code makes these three calls in order, and most orders go through.

Then a bad deploy (a release of new code to production) knocks the shipping service offline for **20 busy minutes**. Reservations still succeed. Charges still succeed. Only the third call fails. The catch block (the code that runs when an error is thrown) writes an error to the log and does nothing more. The outcome: **340 orders** where the customer paid, stock was reserved, and nothing shipped. Each service did its own part correctly, so none of them saw a problem. Support found out from angry customer emails. Then **three engineers spent a weekend** working from a spreadsheet, refunding charges and releasing reservations by hand.

2PC was already off the table. An external provider won't hold locks for twenty seconds, never mind twenty minutes. This workflow crosses a trust boundary, takes real time to run, and can fail after earlier steps have already committed. It needs two things: (1) a durable record of how far each order has gotten, and (2) a clear plan for reversing steps that already finished. Locks can't safely give you either one.

Why the obvious fixes fail

- **"Put the undo logic in the catch block."** That's an improvement, but still unsafe. If the app crashes in the middle of checkout or in the middle of undoing, the plan held in memory is lost and nothing picks it up again. The undo calls can also fail or time out. A catch block has no durable retries and no way to escalate. Recovery has to outlive the process that began the work.
- **"Just use two-phase commit."** Locks would be held across three services for anywhere from seconds to hours. And the external payment provider can't take part at all.
- **"Combine everything into one service and one database."** If every step *can* share a single transaction, this is the cheapest fix in the whole module (Fact 2). But payment is external. Inventory and shipping may also be split for good organizational reasons. Some workflows simply can't be made local.

How it works

The course figure below shows the full picture. Across the top runs the forward (happy) path: T1 Reserve inventory → T2 Charge payment → T3 Create shipment → Order complete. Each of these is a local transaction. When T3 still fails after retries (the dashed red arrow), compensations run in *reverse* order: first C2 Refund payment, then C1 Release inventory. The flow ends at "Order failed — customer notified, nothing owed".

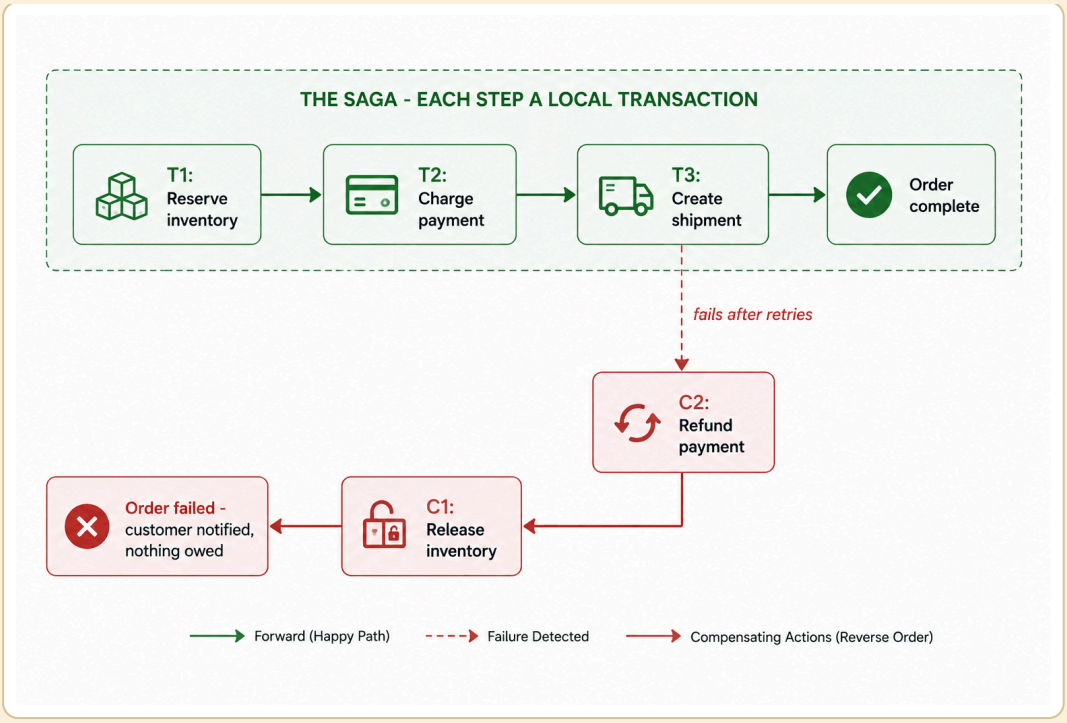


Figure — Saga (course diagram)

1. **Break the workflow into local transactions.** Reserve, charge, and ship each commit at once inside their own service. *Why:* no service sits holding locks while it waits for another. Hot rows stay free, and a slow service can't freeze the rest.
2. **Record your intent durably before every step — this is the saga log.** Before doing anything, save a note like "order 7f3a: T1 done, starting T2". It's the WAL's "write it down before you do it" rule, applied to a whole workflow. The **orchestrator** is the process that drives the workflow. It may crash; if so, another instance reads the log and carries on from the saved state. Progress is never kept only in memory. A good log records: the saga ID, the current state, which steps are done, how many attempts were made, idempotency keys, deadlines, and which action is allowed next.
3. **Make each state change conditional.** Before triggering the next external effect, take a lease (a lock that expires automatically after a set time) or do a **compare-and-set** update. Compare-and-set means "write only if the state is still what I expect". *Why:* if two orchestrators are both recovering, they must never both move the same order forward.
4. **If failure persists, compensate in reverse order.** Undo the most recent finished step first (the refund), then the earlier ones (the release). *Why:* undoing in reverse puts the world back in the opposite order from how it was changed.
5. **Make every step and every compensation idempotent.** The orchestrator retries whenever something times out, so any request may show up more than once. Each step gets its own stable **idempotency key**. Also, "refund order 7f3a" must be safe to run twice. If it isn't, the saga will create duplicates.
6. **Hand off what can't be fixed automatically.** Suppose a refund has been failing for ten minutes. It stays recorded as a durable *pending obligation*, and the saga is not marked done. After a limited number of retries, the orchestrator creates a task for a human. The task includes the order, the provider's references, the attempts so far, and the safe next steps. The unresolved state stays visible the whole time.

Compensations are a business undo, not a rollback. A local transaction that has committed can't be wiped out. So each step gets a new, visible opposite action: refund, release, or cancel. This resembles the corrective entries used in event sourcing (storing every change as an append-only event instead of overwriting the current state). It has three consequences:

- Compensations are *new actions that others can see*, not erasures.
- **Business owners have to define them**, since not every effect can be perfectly reversed. You can't un-send a confirmation email. Its "compensation" is a follow-up message that apologizes and explains.

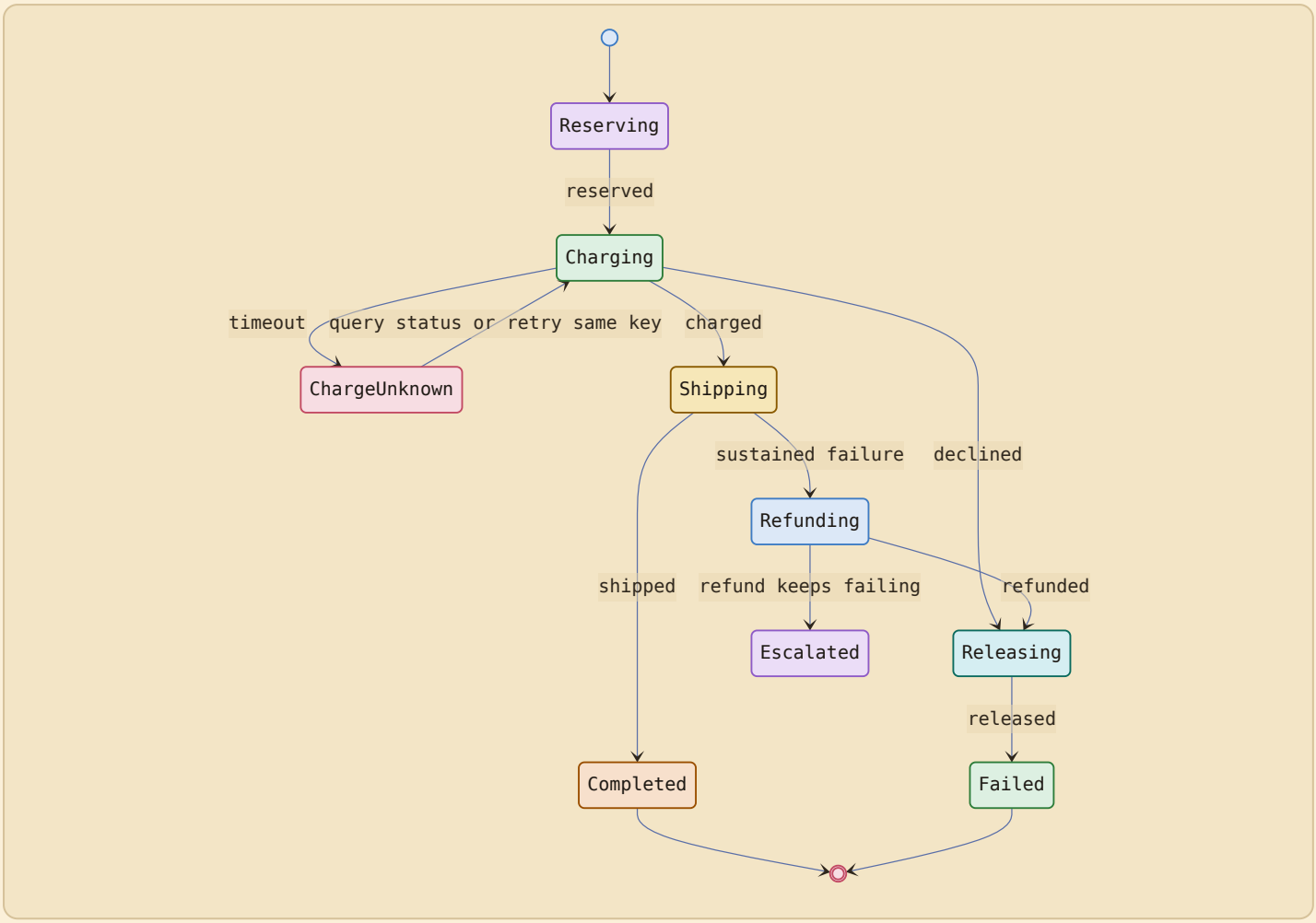
- Some effects can't be compensated at all. Once a package has physically shipped, you need **forward recovery** instead — such as a return label or a support case. That point where there's no going back is called the **pivot**. Place pivot steps *last*, and recover forward from anything after them.

Who drives it: orchestration. Workflows whose steps are ordered and high-stakes need an **orchestrator**. It's an explicit **state machine**: a definition of the possible states and the allowed moves between them. It owns the saga log. Ask it "where is order 7f3a?" and one query answers. (The other option, **choreography**, is compared further down.)

The honest price: no isolation. Isolation means other code can't see a transaction's half-finished state. 2PC hides "charged, not shipped". A saga reveals it in the gap between T2 and T3. Other services can read that state and act on it. So the product team must *define* these in-between states deliberately. An order status of "processing" is real workflow state, not filler text. Some systems also use **semantic locks**. For example, a field like `order_status=PROCESSING` signals to other workflows that this order has temporary limits — e.g., it can't be cancelled and shipped at the same time. Normal reads still work, and no database lock is held across services.

The state diagram below shows the orchestrator's point of view. Look at the `ChargeUnknown` state. It covers a timeout where you can't tell whether the card was charged; you settle it by checking the status or retrying with the same key. Also notice that a refund that keeps failing moves to `Escalated` instead of quietly ending.

Orchestrator persists state before every step so a crash resumes instead of stranding



Walkthrough with numbers (reserve 50 ms, charge 200 ms, ship 100 ms)

- **Happy path.** The whole thing takes roughly **400 ms**, and no locks are held between steps. Every service commits a local transaction that lasts about a millisecond, so busy inventory rows stay open.
- **Replaying the shipping outage.** T3 fails. The orchestrator retries, waiting longer each time (exponential backoff), for around **30 seconds**. Shipping is still down, so this is a *sustained* failure, not a passing blip. Compensation starts in reverse: refund first, then release, and both

are idempotent. In about **90 seconds**, the customer learns the order failed and the charge was reversed. No orders stuck in limbo and no spreadsheet.

- **Crash drill.** The orchestrator dies just after T2 commits. A different instance reads "T2 done, T3 not started" from the log and goes ahead and creates the shipment. Even a crash *in the middle of* T2 can't charge the customer twice. The retry reuses the same idempotency key, and the provider hands back the original result.
- **Ambiguity drill.** T2 times out. Did the provider charge the card? There's no way to know yet. Don't guess, and don't refund a charge that might not exist. Instead, **check the charge status, or retry with the same key**. Write "outcome unknown" into the saga log *before* you start checking. Then, if the orchestrator dies during the check, its replacement repeats the *status check* (with the same identifiers) rather than starting a fresh charge. A recorded "we don't know" is better than an assumption nobody wrote down.

Trade-offs

You gain:

- **Workflows across services and across companies with no distributed locks** — every step is a quick local transaction.
- **Progress that survives crashes** — thanks to the saga log, a workflow *picks up where it left off* instead of starting over.
- **Long-running flows cost nothing extra** — a saga can wait for hours (say, for a fraud review) without locking anything.
- **Failure handling that's designed in** — compensations run on their own, at machine speed, with retries and escalation.

You pay:

- **No isolation.** In-between states are visible, and others can act on them. These *anomalies* are how sagas typically go wrong, so product must spell out every state.
- **Compensation code.** Every step needs an undo that's tested and idempotent. Some need a business decision, not just a technical reverse.
- **Workflow machinery.** You have to run an orchestrator that keeps durable state (or live with choreography's poor visibility).
- **Eventual consistency.** Orders really are "in progress" for a while. That affects reads, reports, and what users expect.

Running it: build dashboards that show workflows by state, age, last error, and next retry. Alert on any workflow stuck in one state for longer than its **SLO** (service-level objective). Total throughput can look fine while a handful of valuable orders sit stuck.

When not to use it

- **When a single database transaction would work.** Always check Fact 2. A saga inside one database is just extra complexity for nothing.
- **When in-between states must never be seen.** Some financial flows can't let "charged but not shipped" be visible at all. A saga can't hide it. That's 2PC's single real advantage (or a hint that the design needs rethinking).
- **Simple two-step internal flows.** A retry queue plus idempotency already handles the ways it can fail. Not every pair of writes needs a state machine.
- **When nobody will run the machinery.** A workflow engine that no one watches is just the 340-order incident with more config files.

Common misuses: wrapping simple create/read/update/delete operations in a saga. Also, **incomplete compensations**: for example, a refund that forgets to take back loyalty points, or a release that forgets to reset a flash-sale counter. Those don't truly restore the business state. Engineering and the business owners should sign off on the complete reversal together.

Orchestration vs. choreography

With **orchestration**, a single coordinator chooses the next step. With **choreography**, every service responds to events that other services publish, and nobody is in charge.

	Orchestration	Choreography
Who decides the next step	One coordinator that runs a state machine	Every service, responding to events
"Where is order 7f3a?"	A single query to the orchestrator	A hunt through the logs of N services
The saga log	Explicit, in one place, and resumable	Spread across services, if it exists at all
Adding a step	Change the workflow definition	Add new subscriptions — which create hidden new coupling
Failure handling	The coordinator retries, compensates, and escalates	Every service has to build its own part
Fits	Workflows where order and stakes matter	Independent reactions such as analytics or notifications

Rule: orchestrate the workflow itself; use events only for independent reactions. The order saga is run by an orchestrator. Its final `order.completed` fact is then published over pub/sub to analytics, email, and anyone else interested. If you choreograph reserve → charge → ship → compensate purely through events, control gets spread so thin that nobody can say what state an order is in.

Real-world examples

- **Travel industry** — where it began: flight, hotel, and car are booked one after another, and cancellation policies act as compensations.
- **Temporal** (and **Cadence**, which came before it) — turn the saga log into **durable execution** infrastructure, so workflow code automatically resumes after a crash. **AWS Step Functions** provides a managed state machine.
- **Every serious order or payment system** — the "processing" status you see there is an intentional saga in-between state.
- **For AI engineers:** an agent whose tools have side effects *is* a saga. Say it books a meeting, emails the attendees, and updates a CRM (customer relationship management system — the database of customers and contacts). If a later tool call fails, you need a designed way to reverse the earlier ones. A durable workflow engine lets an interrupted agent pick up again instead of charging or messaging twice. Idempotency keys on tool calls, plus compensations, are what separate a production agent from a demo.

In the interview

A checkout spanning inventory, payment, and shipping is a test of saga thinking. Point out that one local transaction can't cover the whole workflow anymore. Rule out 2PC (it blocks, and the external provider can't join). Then lay out the steps, the durable state, the compensations, idempotency, and orchestration.

> **30-second answer:** "I'd make checkout a saga: reserve, charge, ship. Each is a local transaction, with no locks across services. Before every step, progress goes into a durable saga log, so if the orchestrator crashes it resumes rather than leaving orders stranded. If a step still fails after retries, compensations run in reverse — refund, then release. Each one is idempotent, because everything here gets retried. In-between states like 'charged, not shipped' are visible on purpose, so product defines them and every reader handles them. Steps that can't be reversed, like physically shipping, go last. Failures after that pivot recover

> forward, for example with a return label. I orchestrate the workflow and use events only for reactions like analytics and email. 'Where is this order?' has to be a single query."

Likely follow-ups (with answers):

- *"The payment call timed out and you don't know if it charged. Do you compensate?"* No. Find out what actually happened first. Either retry with the **same idempotency key** (the provider sends back the original result) or check the charge status. Only continue or compensate once you know the true state.
- *"What if the refund itself fails?"* Compensations need to be just as reliable as forward steps. Retry the idempotent refund with backoff. If it keeps failing, send it to a **dead-letter** escalation queue (a side queue for messages that keep failing, so they're parked instead of retried forever) that alerts a person and gives them the full context.

Mistake that fails candidates: suggesting 2PC; saying "compensations" without explaining what they are; or storing orchestration state only in process memory. Each of these brings back a failure this lesson solves.

Self-check

- **Q1: In one sentence, say what a saga trades away compared with 2PC. Then name the two mechanisms that take the place of locks and reliance on a coordinator.** → A saga gives up isolation (in-between states can be seen) so that it avoids 2PC's locks and blocking. The two replacements are a **durable saga log**, so workflows resume after a crash, and **designed, idempotent compensations**, which reverse finished effects when a later step fails.
- **Q2: Design compensations for these steps: reserve inventory, charge card, send confirmation email, hand the package to the courier. Which one has no true compensation?** → Release; refund; an approved follow-up email that explains (not perfect — so send the confirmation *late*). Handing off to the courier is the **pivot** and can't be reversed, so recover forward (return label, support case, redelivery). Put the steps you can compensate first and the irreversible ones last.
- **Q3: A customer receives "Thanks for your purchase!", yet the order was refunded 90 s later because shipping failed. The email service insists it did nothing wrong. Diagnose it, and give two fixes of different depth.** → This is a no-isolation anomaly. The email service correctly responded to the in-between "payment succeeded" state, before the saga compensated. Shallow fix: only send confirmations from the final `order.completed` state. Deeper fix: every consumer treats in-between states as provisional, and any outward action based on them must have its own compensation. Only tell customers things the saga can no longer undo.

Related patterns

- **Idempotency** — needed on every step and every compensation, because the orchestrator's retries make duplicates a normal event.
- **Two-Phase Commit** — the contrast that defines sagas: locking and hiding versus showing and compensating.
- **Message Queue** — steps are often sent through queues, so the saga picks up Module 2's delivery rules.
- **Publish-Subscribe** — for reactions: final events are broadcast to everyone who just wants to be informed.
- **Dead Letter Queue** — where compensations that keep failing get escalated to people, along with context.

TL;DR

Problem	Workflows across services can't use locks (so 2PC is out). A failure in the middle leaves half-finished business state — customers charged but nothing shipped — that someone cleans up by spreadsheet.
Mechanism	Local transactions, with progress written durably before each step. A failure triggers designed, idempotent compensations in reverse. Irreversible steps (the pivot) go last and recover forward. An orchestrator owns the state machine. Events are broadcast only from final states.
Costs	No isolation (in-between states and anomalies are visible), a designed and tested undo for every step, workflow machinery to run, and eventual consistency while a saga is in progress.
Skip it when	One DB transaction is enough, visible in-between states are unacceptable (the one case where 2PC wins), or the flow is too simple to need a state machine.

Quorum

Original lesson: Quorum on DesignGurus ↗

What it is. Quorum is the way a **leaderless** replicated store keeps its reads reliable. "Leaderless" means there are several copies of the data, any copy can take reads and writes, and there's no single primary. Suppose there are **N** replicas. A write counts as done once **W** replicas confirm it. A read asks **R** replicas. Pick numbers so that **W + R > N**. Then every group you read from must overlap every group you wrote to. So at least one replica you ask holds the newest completed write.

> **Analogy.** Three friends each keep a copy of a shared shopping list. Every time you add an item, you tell at least two of them. Every time you check the list, you ask at least two. Since 2 + 2 is bigger than 3, at least one friend you ask is someone you told. That means you'll always hear about your latest addition.

The problem: the flickering cart

The last two lessons dealt with *different services doing different jobs*. This lesson, and the next one, deal with *copies of the same data* agreeing on the current value.

The cart ran on a primary-replica setup (one primary copy takes all writes and copies them to read-only replicas). Its weak spot was **failover**: when the primary dies, a replica gets promoted to be the new primary. For the cart, failover took about **30 seconds**, and no customer could add items during that time. Plenty of datasets can live with a 30-second pause on writes. Carts can't, because they feed straight into revenue. The business wants the cart to keep accepting writes through the failure of any single machine.

So the team moves to a **leaderless** store: **3 replicas**, no primary, and any replica can take reads and writes. With no leader, there's never a promotion.

Version one writes to a single replica and reads from a single replica. A customer adds sneakers through replica A. Then they reload the page. The read lands on replica C, which hasn't received the update yet. The sneakers **disappear**. On a later read that reaches A, they **reappear**. All the replicas are healthy; the copies just disagree for a moment. Writing anywhere got rid of failover, but it also got rid of predictable reads.

Why the obvious fixes fail

- **"Write to all three, read from any one."** Now every read sees every finished write. But a single slow or dead replica now holds up *every write*. That brings back the exact availability problem you left the leader to avoid.
- **"Write to one, read from all three, and pick the newest."** This flips the problem. Every read now relies on every replica. One slow answer delays the read, and one dead replica makes it fail.

- **"Always send each user to the same replica" (sticky routing).** Alice always goes to A. That works until A fails, or her device connects through a different entry point. Then her cart vanishes — during precisely the failure you were planning for. A pinned replica is really just a leader with weaker promises.

All three ideas need *every* replica, either on the read side or on the write side. What you really need is much less: a guaranteed **overlap** between the replicas that took a write and the replicas a later read asks.

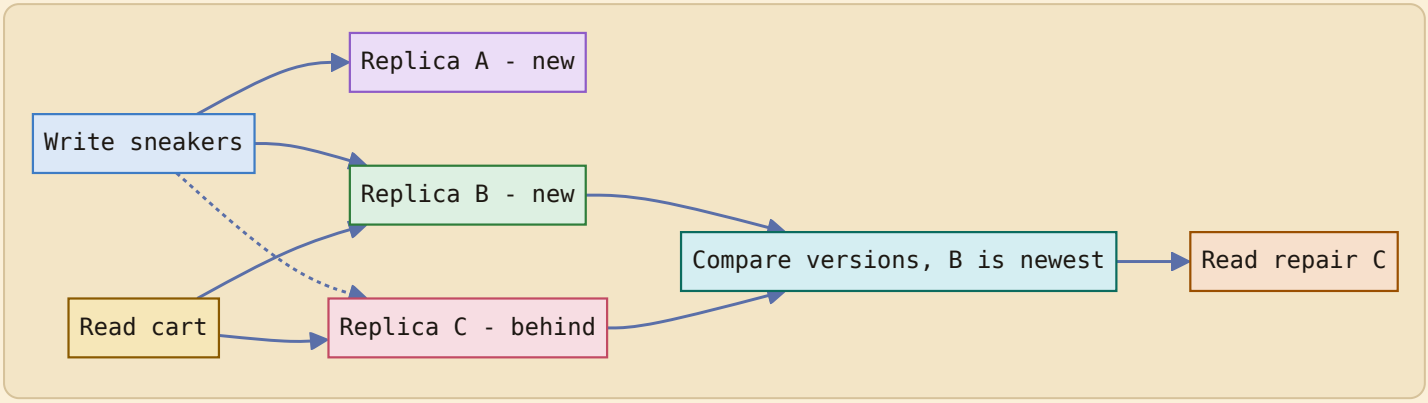
How it works

With N = 3, W = 2, R = 2 (W + R = 4 > 3):

```
WRITE "sneakers" ──► A ✓   B ✓   C (behind)
READ the cart    ──►       B ✓   C ✓
                           ▲
            B is in both groups — it must be, because 2 + 2 > 3.
            The read compares versions; B's is newer, sneakers stay.
```

Two groups of 2 picked from only 3 replicas add up to 4 "seats". With just 3 replicas to fill them, at least one replica has to sit in both groups. The guarantee is nothing more than counting. The flowchart below shows the same idea, including how the stale replica C gets fixed by read repair.

N=3, W=2, R=2: the read and write groups must share a replica



Supporting mechanisms:

1. **Versions break ties.** The R answers might not match, because one replica missed the latest write. Each value carries a version, and the reader chooses the newest *by version*. It never just takes the quickest reply, since speed tells you nothing about freshness. Simple systems can use timestamps; the next lesson explains why distributed systems often need richer versions. The coordinator also sends the chosen version details back to the client. That way, the client's next write can state which history it saw. Without this, the system couldn't tell "replace what I just read" apart from an unrelated edit happening at the same time.
2. **Reads fix the cluster (read repair).** When a read sees that C returned an old value, it immediately writes the newer value back to C.
3. **Anti-entropy handles everything else.** A background process keeps comparing replicas and syncing them. It fixes keys that are rarely read, which read repair would never reach. Watch three things: how far versions have drifted apart, the repair backlog, and the oldest difference not yet repaired. Requests can keep succeeding while one replica quietly stops catching up.
4. **W and R are settings, chosen per operation.** W=3/R=1: slow writes but instant, fresh reads (good for configuration that is read far more than written). W=1/R=3 is the opposite. W=1/R=1 is fast but gives **no overlap guarantee**, so reads can fall behind (eventual consistency). That's fine for like counts. It's also exactly what caused the cart bug when it was picked by mistake.
5. **Tolerating stragglers (a bonus you don't see at first).** With W=2 of 3, the coordinator moves on after the two quickest replies. So the slowest replica (the **straggler**) doesn't decide your **p99** latency — the time within which 99% of requests complete.

Three qualifications on the guarantee:

- **Sloppy quorums and hinted handoff.** Some Dynamo-style systems (stores modeled on Amazon's Dynamo design, such as Cassandra and Riak) act differently during a partition. If the intended replicas can't be reached, temporary stand-in nodes (machines in the cluster) are allowed to accept writes. Later, **hinted handoff** hands that data back to the proper replicas. This boosts availability. But while stand-ins are in use, read and write groups may not overlap. So reads can miss writes even when W + R > N.
- **Partial writes.** A coordinator might collect fewer than W acknowledgements and report a failure — even though one replica actually saved the value. A later read or retry might then see it anyway. So writes still need stable IDs or merge behavior, even when the response said "failed".
- **Quorum promises visibility, not ordering.** Two devices might update the same cart at the same moment. Each could reach W=2 through a *different* pair of replicas. That leaves two "newest" versions, called **siblings**. Vector clocks (next lesson) can tell whether writes were concurrent or sequential. If you need a single, ordered winner, you need **consensus** protocols such as **Raft** or **Paxos**. They add stricter rules on top of simple majority counting.

Walkthrough with numbers (N=3, W=2, R=2)

- **Write.** Send "add sneakers" to all three replicas. Report success after **2** acknowledgements — roughly **5 ms**, even if C is slow.
- **Read.** Ask all three, use the first **2** answers, compare versions, and return the newest. Every pair you read overlaps every pair you wrote, so one answer includes the sneakers. Read repair then fixes C.
- **Failure drill (the reason for all this).** Replica B dies. A and C can still meet W=2 and R=2. **Zero failover, zero pause, zero promotion.** Traffic keeps flowing while B's replacement catches up through anti-entropy.
- **The honest cost — amplification.** Every logical read or write now takes 2–3 network calls instead of 1. A service handling **100,000 logical reads/s** might produce **200,000–300,000 replica requests**, before counting repair, anti-entropy, and retries. Size your capacity for replica traffic, not client traffic. Also, simultaneous edits can split versions, so you must have conflict resolution.

Trade-offs

You gain:

- **No leader and no failover** — reads and writes carry on through the loss of one replica without pausing, so the revenue-critical cart stays writable.
- **Consistency you can tune per operation** — W and R let you trade speed against visibility. One cluster can serve carts, telemetry (automatically collected measurements such as metrics and events), and configuration, each with its own guarantees.
- **Tolerance of stragglers** — the slowest replica no longer sets p99.

You pay:

- **Writes at the same moment conflict** — quorum doesn't put simultaneous updates in order, so you need version tracking and a way to resolve conflicts. (A leader avoids this by handling writes one at a time.)
- **Amplification** — R or W network calls for each operation, plus background repair traffic that doesn't appear in client request counts.
- **Edge cases where the guarantee weakens** — sloppy quorums and partial writes can return old data even when W + R > N. Learn exactly what your store promises.
- **Only one key at a time** — quorum settles the value of a single key. It won't make a stock decrement and a card charge atomic together (that's a job for 2PC or sagas). This shapes how you model data. Values that need a single consistency decision may need to live under one key — as long as that key stays small and doesn't turn into a hot partition (one shard or key that gets a disproportionate share of traffic).

Client guarantees also need thought. A user can still read stale data right after their own write, if the next request uses a weaker level. Sessions that need **read-your-writes** should carry along the version they saw and reject any older response. Make consistency levels part of the API (the interface other code uses to call the service) design and service ownership, not something buried in database defaults. For example, a checkout read might need quorum, while a recommendation counter on the same cluster reads just one replica. Track strict and sloppy responses separately. An overall success rate hides the requests that quietly got weaker guarantees during a partition.

When not to use it

- **You need exactly one winner** — unique usernames, choosing a leader, "sell the last item exactly once". These need ordered consensus (Raft/Paxos) or a single leader. Quorum makes sure the winner is *seen*; it doesn't make sure there's only one.
- **Transactions across several keys** — wrong tool. Look at 2PC and sagas.
- **Mostly-read data that can survive a failover** — primary-replica is simpler, and a 30-second failover is fine for a product catalog. The cart's need to always accept writes was a specific reason; don't apply it everywhere.
- **Small systems** — three replicas and quorum math for a service running at **50 requests/s** is complexity no one needs.

Common failure: choosing a Dynamo-style store "for scale" without looking at its W/R defaults, then discovering its real guarantees through flickering data in production. W and R should be deliberate product decisions.

Leaderless quorum vs. primary-replica

	Primary-Replica (leader)	Leaderless Quorum
Write conflicts	Can't happen — all writes pass through one leader	Can happen — you need versioning plus resolution
Failover	Real: a pause of seconds on writes, plus promotion and fencing (blocking the old leader from writing)	None: a single replica dying goes unnoticed
Consistency story	Easy to reason about (apart from read-your-writes)	Adjustable, subtle, and full of fine print
Latency profile	Your p99 is the leader's p99	Tolerates stragglers (waits only for the fastest W)
Complexity lives in	The failover machinery	The version and conflict machinery
Fits	Most data: catalogs, orders, accounts	Keys that must always accept writes: carts, sessions, presence (online/offline status)

A leader gives you simpler behavior but pauses during failover. Leaderless stays available but shows you the conflicts. Mature systems make this choice dataset by dataset.

Real-world examples

- **Amazon's Dynamo paper** — introduced this pattern for a shopping cart that had to accept writes even during failures. It also defined sloppy quorums and hinted handoff.
- **Cassandra** (an open-source, Dynamo-inspired wide-column database) — its per-query consistency levels `ONE`, `QUORUM`, and `ALL` turn W and R into API parameters.
- **DynamoDB and Riak** — both descend from Dynamo.
- **etcd and ZooKeeper** (small, highly reliable stores for configuration and coordination) — rely on *consensus* instead (Raft and the Paxos family). That's why Kubernetes (the standard system for running containers on a cluster) can trust their ordered control-plane data.
- **For data engineers:** feature stores (databases that serve the input values an ML model needs) and time-series systems (stores optimized for timestamped measurements) built on Cassandra face these

operational decisions. They use `QUORUM` reads where stale features could change how a model behaves, and `ONE` for high-volume telemetry where throughput counts more. The choice can differ from query to query.

In the interview

A classic question is: "With three replicas, how many write acknowledgements and read responses guarantee you see the newest write?" Follow-up questions dig into W=1/R=1 or simultaneous writes. Show that you understand both what quorum promises and what it doesn't.

> **30-second answer:** "With N replicas, I confirm writes at W and read from R, keeping W + R > N. That way every read group overlaps every write group, so someone I ask has the newest version, and versioning picks it out. N=3, W=2, R=2 is the standard setup. It survives any one replica dying with zero failover — which is the whole point of going leaderless. The catch: quorum promises visibility, not ordering. Two simultaneous writes can each reach W=2 and split the value. So you need version machinery — vector clocks or last-write-wins. Sloppy quorums give up the overlap guarantee to stay available during partitions. And for problems where exactly one winner is required, you need consensus like Raft, not quorum reads."

Likely follow-ups (with answers):

- *"What happens with W=2, R=1 on N=3?"* W + R = 3, which is *not bigger* than 3. So a read can land on the one replica the write skipped and return old data. You get faster reads but only eventual consistency. That's acceptable only when you choose it on purpose for data that can handle it.
- *"Is W=2/R=2 strong consistency, then?"* It guarantees you'll see *completed* writes, thanks to overlap. But simultaneous writes can split, sloppy quorums weaken overlap during partitions, and partial writes cause edge cases. Call it "overlap visibility plus conflict resolution", not full strong consistency.

Mistake that fails candidates: treating quorum as if it were a transaction across payment and inventory; forgetting the overlap inequality; or running on storage defaults without knowing what they are.

Self-check

- **Q1: Give the formula and, in one sentence, why it works.** → W + R > N. Because the total is larger than the number of replicas, every successful write group and every read group must share at least one replica, and that replica holds the newest completed write.
- **Q2: One cluster holds three datasets — (a) feature flags, read all the time and written weekly; (b) the cart, which must accept writes no matter what; (c) raw clickstream (a log of every click and page view), with heavy writes that are batch-read later. Choose W and R.** → (a) W=3/R=1. Writes are rare, so they can afford to wait for every replica, and the constant reads need only one reply and still stay current. (b) W=2/R=2 — overlap holds even through one failure. (c) W=1, and let the batch reader choose R. A few late events cost less than slower writes. Set these per dataset or per query.
- **Q3: The flickering cart is back — but only during last week's partition between datacenters, and only for some users. Settings are N=3/W=2/R=2, and the docs mention "hinted handoff." Explain it and lay out the real choice.** → During the partition, some coordinators couldn't reach the intended replicas B and C. So a **sloppy quorum** counted stand-in nodes toward W=2 and kept hints to hand off later. Those stand-ins weren't part of the normal read groups, so reads through A+B or A+C missed writes that had succeeded. The real choice is between a **strict quorum** (reject writes when the true replicas can't reach W — consistent, but unavailable) and a **sloppy quorum** (keep accepting writes, but temporarily miss data). For a cart, availability plus later reconciliation may well be right. But it has to be a deliberate CAP decision. Vector clocks then reconcile the versions.

Related patterns

- **Primary-Replica** — the leader-based alternative: simpler behavior but with failover pauses. Most systems use both, chosen per dataset.
- **Vector Clocks** — the version machinery that tells split concurrent writes apart from sequential ones (next lesson).
- **Consistent Hashing** — chooses *which* N replicas store each key (the ring from the storage module).
- **Two-Phase Commit** — what databases put on top of quorum-replicated groups to get transactions across shards (the Spanner stack).

TL;DR

Problem	Leaderless replicas always accept writes but can fall behind each other. Read a random one and the cart flickers. Requiring every replica on either side brings back the fragility you left the leader to escape.
Mechanism	N replicas. Writes are confirmed at W and reads ask R, with W + R > N so the groups must overlap. Versions pick the newest value. Read repair and anti-entropy fix lagging replicas. W/R are tuned per dataset.
Costs	Amplified reads and writes; simultaneous writes that split and need version machinery; fine-print edge cases (sloppy quorums, partial writes); and it covers a single key only — it is not a transaction.
Skip it when	You need a single ordered winner (consensus or a leader), atomic changes across keys (2PC/saga), or you have a simple mostly-read dataset where a failover pause is fine.

Vector Clocks

Original lesson: Vector Clocks on DesignGurus ↗

What it is. A vector clock is a small version label attached to each key. It's a table with **one counter for each writer**. Every time a writer writes, it increases its own counter. Compare two labels and you can tell one of two things. Either one version *knew about* the other (so it replaces it), or the two were written **concurrently** — meaning neither saw the other. That's a real conflict. Both versions should be kept and merged, not quietly discarded.

> **Analogy.** Think of Git (the version-control tool that tracks changes to code). Branches are edits made concurrently, and the commit graph records who built on whose work. When Git reports a "merge conflict", it's saying "neither branch comes from the other — you choose." A vector clock is the same idea, squeezed down to one number per writer instead of a full commit history.

The problem: which "newest" version wins?

The quorum lesson finished with two devices updating one cart at the same moment. Each write reached W=2 through a different pair of replicas. That left two "newest" versions and no clear winner.

"Just take the later timestamp" sounds simple, but it breaks in two ways:

1. **Clocks don't agree.** Machine clocks normally differ by milliseconds. When **NTP** (Network Time Protocol, the service that keeps clocks in sync) breaks, they can differ by seconds. Timestamps are never fully trustworthy.
2. **"Which came later?" is the wrong question.** Say Alice's phone writes {sneakers}. Three seconds later her *synced* laptop writes {sneakers, socks}. The laptop saw the phone's write and built on it, so it correctly replaces it. Now picture a different case: an *offline* laptop writes {hat} without ever seeing the sneakers. These are two separate, real changes. Whichever you pick, you lose the other one — regardless of which timestamp is later.

The question that actually matters is **causality**: did one write know about the other? That's precisely what vector clocks record.

Why the obvious fix fails

- **"Just keep the clocks very tightly in sync."** Even perfect clocks answer the wrong question. Two writes 5 ms apart can still be concurrent if neither one saw the other. "The later one wins" would still throw away a valid change. (Google Spanner's precise clocks help *order transactions inside a consensus system that has a leader* — which is a different problem.)

How it works

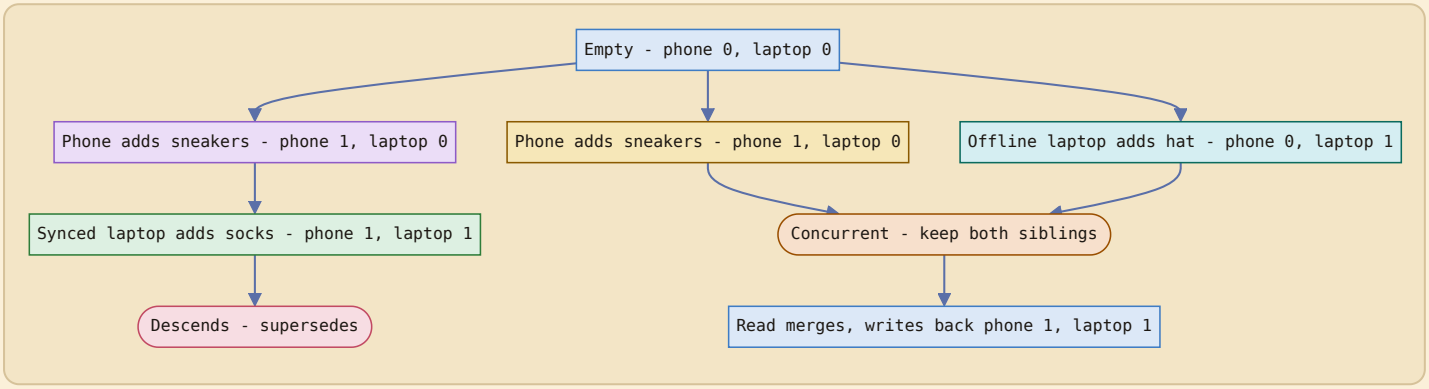
Here there are two writers, the phone and the laptop:

```
cart starts empty                          {phone:0, laptop:0}
phone adds sneakers                        {phone:1, laptop:0}
laptop (synced, saw sneakers) adds socks   {phone:1, laptop:1}
  compare {1,1} vs {1,0}: >= in every slot → DESCENDS: supersedes. One story.

...or instead, from the empty cart:
phone adds sneakers                        {phone:1, laptop:0}
laptop (offline!) adds hat                 {phone:0, laptop:1}
  compare: each is ahead in one slot → CONCURRENT: a true conflict.
  Keep BOTH as siblings; resolve at the next read.
```

1. **On every write, the writer bumps its own counter.** *Why:* the counter tracks "how many of my writes are included in this version".
2. **Compare two versions one slot at a time.** If version A is ≥ version B in *every* slot, A **descends** from B. That means A knew everything B knew, so A cleanly replaces B.
3. **If each version leads in some slot, they're concurrent.** Neither one saw the other. The store keeps **both** as **siblings** rather than choosing one.
4. **Vector clocks spot conflicts; they don't settle them.** On the next read, the application merges the siblings using one of three approaches:
   - **Merge by meaning** — for carts, take the union: {sneakers} ∪ {hat}. This was Amazon Dynamo's original approach (it has a well-known trap — see self-check Q3).
   - **CRDTs (conflict-free replicated data types)** — counters, sets, and documents built so that concurrent edits combine on their own. This is the technology behind Figma-style collaboration (several people editing the same design or document live).
   - **Ask the user** — show both versions, like a git merge conflict.
5. **Write back one merged version whose clock descends from every sibling.** Take the larger value in each slot — like a git merge commit. *Why:* if the merged version kept only one sibling's clock, another replica would see it as concurrent again, and the conflict would start all over.

The diagram below draws both histories starting from the empty cart. On the left, the laptop saw the phone's write, so its version descends and replaces it. On the right, the two writes are concurrent. They become siblings, and a read merges them into `{phone 1, laptop 1}`.

Sequential vs. concurrent histories from an empty cart



Tombstones. Union merging makes deletes risky. Say the phone removes the sneakers, but an offline laptop's concurrent version still has them. The union brings the sneakers right back. The fix is to store deletions as **tombstones**: explicit records such as "sneakers: removed, version X". Tombstones take

part in the merge and override the earlier "present" state. Clean them up after a while. A deletion is a *fact*, not just missing data.

Trade-offs

You gain:

- **Nothing lost silently** — concurrent writes stay around as siblings until a clear merge policy deals with them.
- **Immunity to clock skew** (the gap between different machines' clocks) — causality is tracked with counters, not wall-clock time, so clock drift can't choose the wrong winner.
- **Precise language** — "descended" versus "concurrent" makes leaderless updates something you can actually reason about.

You pay:

- **You have to resolve the conflict** — every data type needs a merge policy that someone owns and tests.
- **Space and growth** — one counter per writer per key can grow large when there are many clients. Real systems usually key the counters by **replica/server**, not by end-user device, to keep the vectors small.
- **Pruning is risky** — deleting a retired writer's entry is only safe if you can prove it won't come back (an old replica, an offline client, a backup, or a delayed message). If it does, you've wiped out causal history, and a known descendant turns into a fake conflict.
- **Mental effort** — applications have to compare versions, deal with siblings, and maintain merge code, instead of just accepting whatever value the database chose.

When not to use it

- **Anywhere there's a leader.** Primary-replica systems already send every write through one leader in order, so causality comes free. Vector clocks would be answering a question that never comes up. This pattern belongs in the leaderless world.
- **When losing a concurrent write truly doesn't matter.** Think presence indicators, metric samples, or "last seen" fields — short-lived data where "roughly the newest" is enough. Here, **last-write-wins (LWW)** — the later timestamp overwrites the other — is a valid, *intentional* choice. The mistake is using LWW by default and only finding the data loss during an incident.
- **When a CRDT fits.** If the data type can carry its own merge rules (a counter, a set, a shared document), build the resolution into the type rather than handling siblings by hand.

Vector clocks vs. last-write-wins

	Last-write-wins (timestamps)	Vector clocks
Concurrent writes	One is quietly thrown away	Both are kept as siblings
Clock skew	Picks the winner (incorrectly)	Doesn't matter
Application burden	None — that's why people like it	A merge policy for each data type
Honest description	A data-loss policy you sign up for	A conflict-detection service you have to maintain

Cassandra chooses LWW by default, on purpose, for simplicity. **Dynamo** and **Riak** keep siblings using vector clocks. LWW quietly throws a conflict away; vector clocks bring it to the application's attention. Decide based on how much it costs to lose a real write.

In the interview

This typically comes after a quorum design: "Two clients update the same key at once — who wins?" Answering with timestamps ignores clock skew, concurrency, and the question of who owns the merge.

> **30-second answer:** "Timestamps can't tell 'B overwrote A on purpose' from 'B never saw A'. And clock skew makes 'later' unreliable anyway. Vector clocks track causality instead: one counter per writer, bumped on every write. Any two versions either descend one from the other, so one cleanly replaces the other, or they're concurrent. In that case both are kept as siblings and the application merges them — union for carts, CRDTs where the type can merge itself, or show it like a git conflict. Vector clocks detect conflicts; they don't resolve them. And last-write-wins is a data-loss policy — fine only when you pick it on purpose."

Likely follow-ups (with answers):

- *"Why not just keep the clocks tightly synchronized?"* Perfect clocks still answer the wrong question. Writes 5 ms apart can be concurrent if neither saw the other, so the later timestamp still throws away a valid change. Spanner's precise clocks serve a consensus system with a leader — a different need.
- *"Who resolves the siblings, and when?"* The next reader gets all the siblings. It applies that data type's merge policy and writes back a single combined version whose clock descends from all of them. That folds the branches together, like a git merge commit.

Mistake that fails candidates: answering with timestamps and never mentioning causality or skew; or saying vector clocks *pick* a winner, when they only detect concurrency.

Self-check

- **Q1: Compare {phone:2, laptop:1} with (a) {phone:1, laptop:1}, (b) {phone:2, laptop:2}, (c) {phone:1, laptop:2}.** → (a) {2,1} ≥ {1,1} in every slot, so ours descends and replaces it. (b) {2,2} ≥ {2,1}, so the other one replaces ours. (c) Each leads in one slot, so they're concurrent siblings that need merging.
- **Q2: Would you pick LWW, vector clocks, or a CRDT for (a) "user last active at", (b) the shopping cart, (c) the text of a shared document?** → (a) LWW. The goal is rough recency, and losing one concurrent sample costs nothing. (b) Vector clocks + union + tombstones. Losing an added item matters. (c) A CRDT. The data type itself has to combine complicated concurrent edits automatically.
- **Q3: A Dynamo-style cart uses union merge. A customer removes sneakers on her phone, then adds a hat on her offline laptop. After syncing, the sneakers are back. Why, and how do you fix the merge?** → Union treats "missing" as "never there". Meanwhile the laptop's concurrent version still includes the sneakers, so the union puts them back. The deletion left no trace. Fix: store deletions as **tombstones** ("sneakers: removed, version X"). They take part in the merge and override the earlier "present" state, and they get cleaned up later. A deletion is a fact, not missing data.

Related patterns

- **Quorum** — produces the siblings. Vector clocks are its version machinery, and together they make leaderless stores hang together.
- **Event Sourcing** — treats causality as first-class history. An event stream is a fully ordered relative of the same idea.
- **Bidirectional Streaming** — the CRDTs used in collaborative editing merge by design: vector-clock thinking baked into data types.

TL;DR

Problem	In a leaderless store, concurrent writes split a key's value. Timestamps "fix" the split by quietly dropping a real write — often the wrong one, because of clock skew.
Mechanism	One counter per writer, bumped on each write. Versions compare as "descends" (replaces) or "concurrent" (keep both as siblings). Reads merge siblings by policy (union, CRDT, ask the user) and write back a version that descends from all of them.
Costs	It only detects (merging is up to you), per-writer storage with risky pruning, and real mental overhead compared with "the database picks."
Skip it when	A leader already orders the writes, the data is short-lived enough for intentional LWW, or a CRDT can handle the merge.

Capstone - Order Fulfillment, Built Twice

Original lesson: Capstone - Order Fulfillment, Built Twice on DesignGurus ↗

What this capstone does. We build the same order-fulfillment system two times. First with two-phase commit, then with a saga. We hit each version with the same three failures and compare the damage. The point is not to crown the "better" pattern. The point is to ask **which way of failing the business can live with**. Whenever you reach a "pause and decide" point, predict the damage before you read further.

> **Analogy.** Picture two ways to handle a relay runner who stumbles. In the first, *every* runner freezes in place until the referee rules (2PC — it fails by freezing). In the second, the race keeps going; the crowd sees the stumble, but a rehearsed recovery plan kicks in (saga — it fails by showing).

The brief

- **Three steps:** reserve inventory → charge payment (**external provider**) → create shipment. The business rule: a customer must never be *charged while nothing ships*, and never *shipped without being charged*.
- **Load: 200 orders/s**, with spikes during popular releases. A single **SKU** (stock-keeping unit — one product code) shows up in **30%** of active checkouts. This is the **hot SKU**.
- **Latency:** checkout must confirm in roughly **2 s**. Fulfillment is allowed to trail by minutes.
- **The cart:** brought over from the quorum lesson. It sits on a leaderless store because it has to accept writes no matter what fails.

Build One: Two-Phase Commit — fails by freezing

Design. One coordinator, with inventory, payment, and shipping as participants in prepare/commit.

A flaw that shows up before anything even breaks: the payment provider has no prepare phase. An outside company won't hand control of its state to your coordinator (2PC doesn't cross trust boundaries). You could add an internal **payment-intent record** to stand in as a participant. But the *actual* charge still happens outside the transaction:

- If you commit the intent and then the provider's charge fails, your system reports success even though no money moved.
- If you charge first and then the internal commit fails, money moved but nothing recorded it.

This workaround quietly swaps one explicit distributed workflow for an *undocumented* one. That hidden workflow still needs durable retries and compensation — in other words, it's a hidden saga.

Drill 1 — the coordinator crashes after gathering yes votes. *Pause: inventory voted yes on the hot SKU. What can it do, and what happens to everyone else who wants that SKU?*

1. Inventory is in doubt. It made a promise, so it can't commit or abort without the coordinator.
2. It keeps locks on the hot-SKU rows until the coordinator comes back roughly **40 s** later.

3. For those 40 s, the **30%** of checkouts that include the hot SKU queue behind those locks or time out.
4. A single failed coordinator turns one small transaction into a wide slowdown in revenue.

Drill 2 — shipping is slow, not down. During its deploy, shipping's p99 rises from **100 ms to 4 s**. *Pause: who feels this, and how?*

1. Every transaction includes shipping, so every one of them waits for shipping's prepare reply.
2. All that time, the inventory and payment-intent locks stay held for as long as shipping's p99.
3. Apply rate x duration: **200 orders/s x 4 s = 800 transactions' worth of locks** in flight at once, many of them on the same popular rows.
4. The speed of one participant now decides the contention in all three services.

Drill 3 — a normal Tuesday (nothing is broken).

1. Every order still pays for two round trips and several fsyncs.
2. Hot-SKU locks last tens of milliseconds rather than ~1 ms. That forces requests into single file, right when release-day demand peaks.
3. At 200 orders/s, a small rise in lock time multiplies the number of requests fighting over the hot row → queueing → longer locks → even more queueing.
4. Capacity can collapse before any service is officially "down".

Verdict: 2PC fails by freezing. It keeps partial state hidden by *halting progress* whenever things are uncertain. Unrelated checkouts that happen to share a row feel the stall.

Build Two: Saga — fails by showing

Design, and the reason behind each choice:

- **Orchestrator + durable saga log** — a crash leads to resuming, not stranded orders.
- **Order: reserve → charge → ship** — steps that can be compensated go first; the **pivot** (shipping, which can't be undone) goes last.
- **Compensations: release inventory, refund payment** — signed off by the business.
- **Every step and every compensation is idempotent** — so retrying is safe.
- **Confirmation email sent only at a final state** — customers hear only about things that can't be reversed.
- **Each service runs short local transactions** — no distributed locks.

The saga log has now become **consistency-critical infrastructure**. Each state change has to be durable *before* its external effect begins. Competing orchestrators need conditional updates so that only one of them moves an order forward. Operators need to search by state, age, and last failure so they can find workflows that keep retrying without making progress.

Drill 1 — the orchestrator crashes after the charge succeeds. *Pause: this is the same moment as 2PC's Drill 1 — money committed, coordinator dead. What's frozen?*

1. Nothing is locked.
2. The log reads "order 7f3a: reserved, charged, shipping not started". Another orchestrator picks up at shipping.
3. The stable payment idempotency key stops a second charge after an unclear crash.
4. Inventory holds a *committed reservation*, not an open lock. Other orders can keep buying the remaining stock. What would have been a sitewide slowdown is now just a workflow record that can be resumed.

Drill 2 — shipping goes down for 20 minutes with 340 orders mid-saga. *Pause: trace what happens to them and what the customer sees.*

1. Each order's shipping step fails. The orchestrator retries with backoff for ~**30 s** to tell a brief hiccup from a real outage.
2. It's a sustained outage, so it runs the idempotent **refund**, then the **release**, followed by an approved "we couldn't complete your order" email.
3. Roughly **90 s** per order, and no spreadsheet.
4. The cost: 340 temporary charges that customers could see, and 340 refunds in the ledger. But the damage is bounded and handled automatically. (2PC would have *turned away* new orders, because every

participant must be up, and frozen the in-flight ones with their locks held.)

5. Customer communication gets better as well. "Processing" is shown once the order is safely recorded in the workflow. "Completed" appears only after shipping. If compensation begins, the state machine provides an exact explanation, so support doesn't have to piece it together from three services' logs.

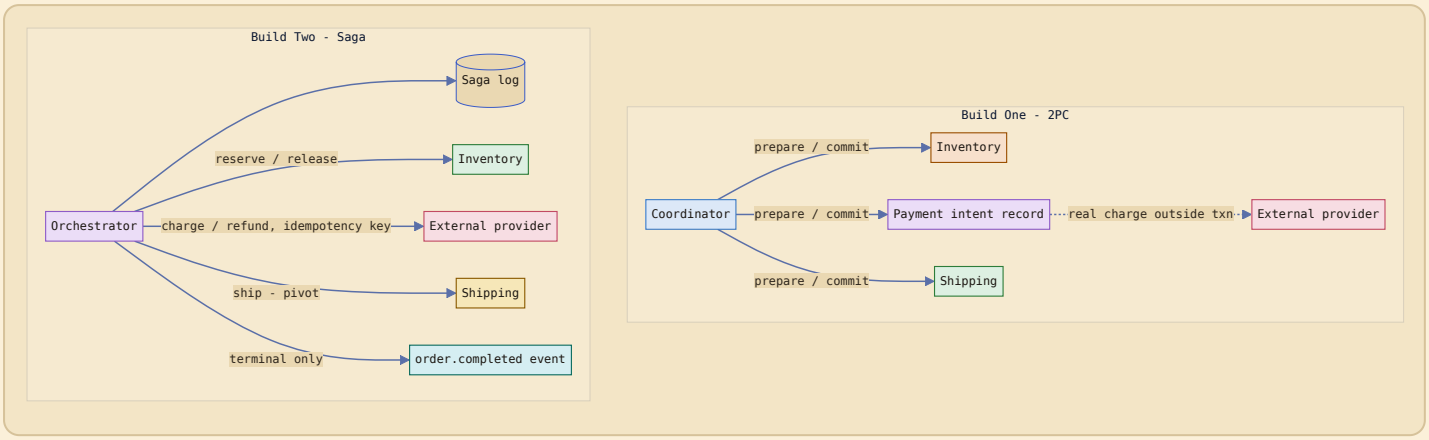
Drill 3 — the anomaly cost (nothing is down). *Pause: what's quietly off on the analytics dashboard, and who else is acting on it?*

1. In-between states are visible. A revenue dashboard that counts "charged" orders includes some that will be refunded minutes later. So it reads a bit higher than the final finance ledger.
2. A marketing system that sent purchase confirmations from the same provisional state would cause errors that customers see.
3. The rule: messages to the outside world come from final states, unless they have their own compensation. Every consumer must know which states are provisional.

Verdict: sagas fail by showing. They reveal in-between truth (refunds, provisional states), but they handle it on purpose instead of hiding it behind locks.

The diagram below puts the two builds side by side. In Build One, the coordinator drives three participants, but the real charge to the external provider (dotted line) happens *outside* the transaction. In Build Two, the orchestrator writes to the saga log, uses idempotency keys with the provider, places shipping last as the pivot, and emits `order.completed` only at the final state.

Two builds of the same checkout



The comparison

Failure injected	2PC damage	Saga damage
Coordinator/orchestrator crash	In-doubt participants frozen with their locks; the hot SKU slows down for as long as recovery takes	The workflow resumes from the log; nothing is held and no one waits
One service slow	Everyone's locks are held for the slowest p99; a storm of contention	Only that step retries; other orders aren't affected; steps still take milliseconds
One service down 20 min	New orders turned away (every participant is required); in-flight ones frozen	Orders finish and then compensate: 340 visible, automatic refunds
Nothing failing at all	Locks force hot rows into single file; 2 round trips + fsyncs for every order	Full speed — plus provisional states that every consumer has to handle
The half-state ("charged, not shipped")	Nobody ever sees it — 2PC's one clean win	Visible for seconds to minutes: designed, labeled, and compensated

That last row is the heart of the trade-off. **2PC fails by freezing; sagas fail by showing — pick the failure the business can live with.** Customer-facing workflows that span services usually pick sagas.

The **gift-card transfer** from the 2PC lesson is still a fair use of 2PC. It involves two internal shards, takes ~50 ms, stays inside one trust domain, and must never show a partial state. In Spanner-class databases, every participant is quorum-replicated, so one crash can't block the protocol. Use 2PC for short internal database work, and sagas for workflows across services.

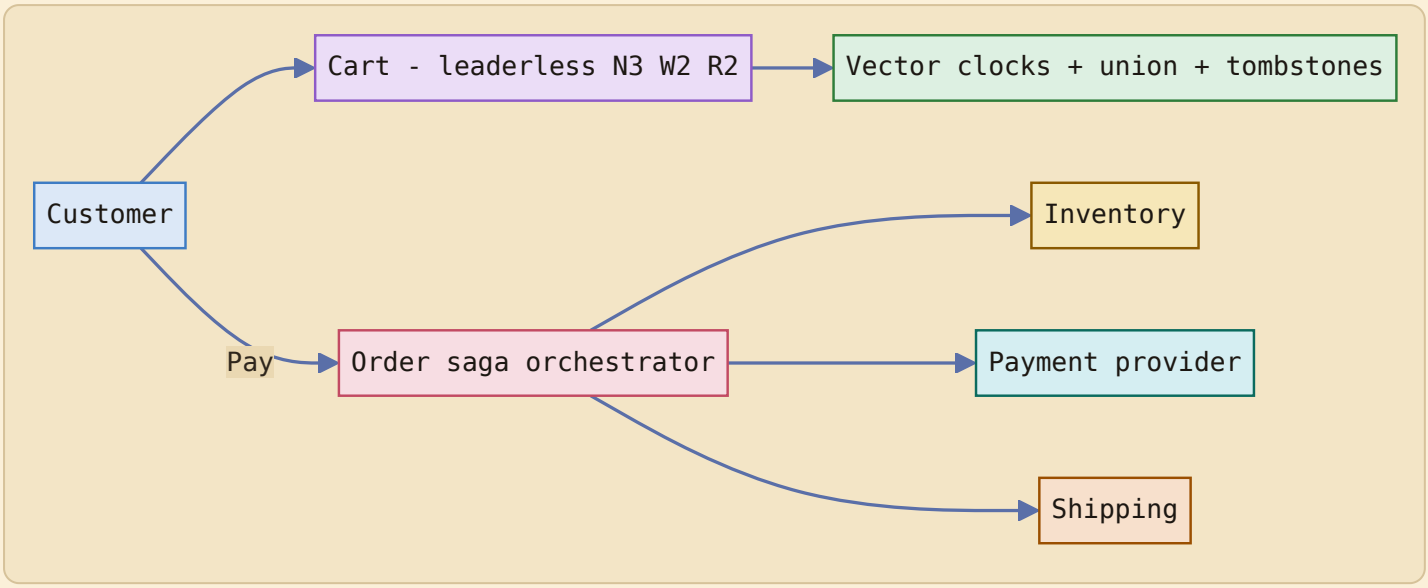
Postscript: the cart's own consistency

A single checkout actually contains **two separate consistency problems**:

1. **Agreement between replicas for one key (the cart).** The cart runs leaderless with N=3, W=2, R=2. During a release, a phone and a laptop may update the same cart at the same time through different replica pairs. Vector clocks notice that neither version descends from the other. The read returns siblings and merges them with a union. Tombstones keep an intentional sneaker deletion from being undone. The customer just sees one correct cart.
2. **A workflow across services for one order (fulfillment).** Once the customer taps Pay, the saga's per-order log and its compensations take over.

Good design begins by working out *which* consistency problem each failure belongs to. The diagram below shows that split.

Where each consistency pattern applies



Extend it yourself

- **Checkout across multiple regions.** Now the orchestrator, its log, and the three services span two regions. Which parts are easy to replicate? Where should the saga log live? It's the new consistency-critical state, so what W and R does it need? What happens to a saga that's mid-flight when the regions are partitioned? (This is Fact 1 applied to the machinery itself.)
- **The provider's webhook.** The payment provider confirms charges later, through **webhooks** (HTTP callbacks). They arrive at-least-once (every message is delivered, but some may arrive more than once) and possibly out of order. Connect that incoming path to the saga. Which step turns into "wait for external confirmation"? What's the timeout rule? How do duplicate confirmations interact with the saga log?
- **For AI engineers.** Swap checkout for an agent doing a three-step job — book a venue, email the attendees, update the CRM — on a durable-execution engine. Design the compensations, choose where the pivot goes, and decide which tool calls are allowed to run before the workflow hits a state it can't undo.

Interview summary

> If you build checkout with 2PC, it can't even include the external payment provider. On top of that, a coordinator crash or one slow participant freezes the hot-SKU locks for everybody. If you build it as a saga, crashes resume from the log, and outages turn into automatic refunds that

> customers can see. 2PC fails by freezing; sagas fail by showing. For customer workflows across services, showing is the failure the business can live with.

Module 8 — The Entry Point (API and Edge)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. The scaling capstone ended at the **system boundary**. That's the spot where clients — browsers, phones, partner systems — first make contact with your system. This module lives right at that boundary. It decides three things: which requests are let in, which are turned away, and which can be answered without ever bothering an application server. Nine patterns deal with accepting, controlling, and shaping incoming traffic. The final two take the same ideas and apply them to calls *between* your own services.

Analogy. Picture a big office building. The lobby has a single front door with a security desk that checks your ID once. There's a reception desk that answers common questions without phoning anyone upstairs. Turnstiles stop anyone who tries to come in too often. Signs point visitors to the right floor. Further inside, every office has its own assistant who handles phone manners for the person they work for. All of these do the same basic job — "stand between the caller and the person being called" — just in different spots, with different rules.

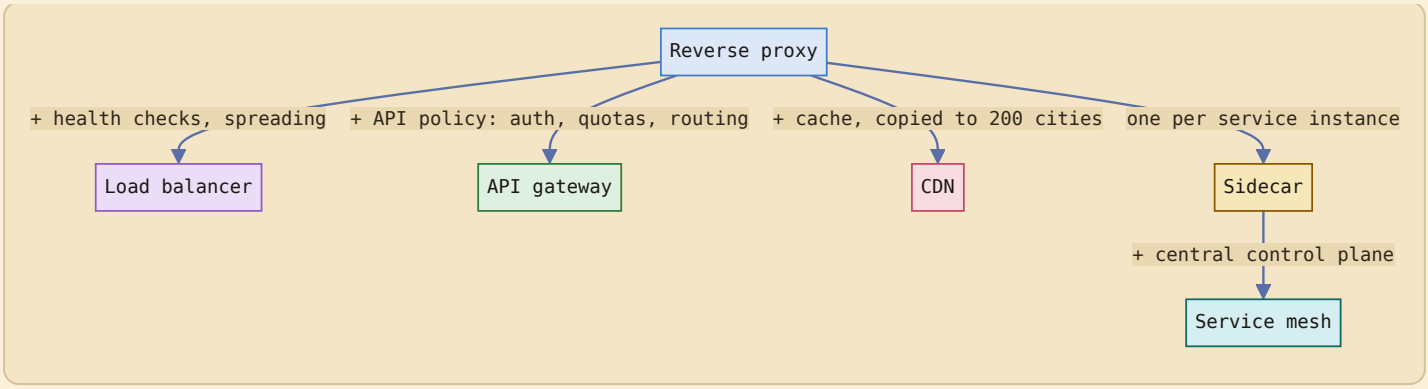
The nine patterns

The situation	The pattern
No one outside should talk to internal servers directly	Reverse Proxy
Popular content should sit close to the people reading it	CDN
A single entrance that authenticates (checks who the caller is), routes, and enforces policy	API Gateway
Different clients need the API shaped differently	Backend for Frontend
Fair use, enforced right at the entrance	Rate Limiting
Pick up a listing exactly where you stopped	Cursor Pagination
Change the API without breaking clients that already use it	API Versioning
Every service gets the same helper for shared chores	Sidecar
All those helpers, configured from one central place	Service Mesh
Every service must answer "may this user do this to this object?" the same way	Authorization Service *(added lesson)*

Two facts that organize the module

Fact 1 — nearly everything here is one building block playing different roles. A **primitive** means a basic building block. Here, that block is the **proxy**: a server that takes in requests and passes them on somewhere else. Place it in front of your servers and it's a **reverse proxy**. Give it health checks (regular "are you alive?" probes of each server) and have it spread traffic, and it's a **load balancer**. Give it API policies (who may call, and how often) and it's an **API gateway**. Copy it, with a cache (a store of recent answers that can be reused without redoing the work), into 200 cities and it's a **CDN**. (New to proxies, caches or CDNs? See the System Design Fundamentals primer.) Put a small one next to every service and wire them together, and you get a **sidecar**, then a **service mesh**. The diagram below shows this family tree.

One primitive, many roles



Fact 2 — the edge is where you're cheapest and strongest. The cheapest request is the one that never gets to your application tier. Turning away bad logins or too much usage at the edge uses up no service capacity. Serving cached pages or static files (files that are identical for every user, such as images, scripts and stylesheets) from there skips application work completely. The edge is also the *single* place every request passes through. So rules like authentication, rate limits, and version policies can be applied once there, instead of being rebuilt (each a little differently) in every service.

North-south vs. east-west traffic

- **North-south** traffic crosses the system boundary: outside clients, browsers, and partners calling in. It's controlled by gateways, CDNs, rate limiters, and version policies.
- **East-west** traffic flows *between* your internal services — for example, checkout calling inventory and then pricing. It needs services to trust each other, plus retries and tracing (recording the path and timing of one request as it hops between services). Sidecars and meshes supply those.

Both rely on the proxy primitive, but they trust different things. That difference splits the module into two halves, and it comes up a lot in interviews.

How to read the module

Begin with **Reverse Proxy**. It's short, and it defines the primitive. Next, move inward from the edge. **CDN** answers lots of requests before they cross an ocean. **API Gateway** applies public policy. **BFF** deals with clients that want responses shaped differently. After that come three key gateway policies: **Rate Limiting**, **Cursor Pagination**, and **API Versioning**. Then switch to east-west traffic. **Sidecar** gives every service the same helper, and **Service Mesh** configures and links those helpers. **Authorization Service** is an added lesson (not in the original course): the gateway authenticates, but deciding whether *this user* may touch *this object* belongs to a shared decision service — roles, attributes and Zanzibar-style relationship graphs (Zanzibar is Google's internal permission system). The **capstone** pulls it all together into a public API layer for web and mobile.

> Short on time? Read **API Gateway**, **Rate Limiting**, and **CDN** first. Every "design X" interview draws this boundary, and those three are what the follow-up questions dig into.

Terms used throughout

- **Edge vs. origin** — the *edge* is the spot near the client where traffic enters your system. The *origin* is where the authoritative servers and data live. Answer from the edge whenever you can; go to the origin only when you have to.
- **Forward vs. reverse proxy** — a *forward* proxy acts on behalf of clients and hides *who is asking* (for example, a corporate web filter). A *reverse* proxy acts on behalf of servers and hides *who is answering*. Everything in this module grows out of the reverse kind.
- **North-south / east-west** — traffic crossing the outer boundary vs. traffic between services (described above).
- **Policy enforcement point** — a single place that applies a rule, instead of N separate copies of it. Authentication, usage limits, and quotas usually live here.

Before you start — questions about your own system

1. How many separate places in your code check authentication? (Most teams are surprised by how many there are. That number is the case for a gateway.)
2. What share of requests could be served without ever reaching your servers — and are they actually served that way today?
3. When your API's response shape changes, what happens to the year-old mobile app that you can't force people to update?

Reverse Proxy

Original lesson: Reverse Proxy on DesignGurus ↗

What it is. A reverse proxy is a server placed in front of your real application servers. Every client request lands on the proxy first. The proxy takes care of the "boundary chores" and then passes the request along to an app server. Clients never speak to your application servers directly.

> **Analogy.** Think of the host at the front of a restaurant. Guests never wander into the kitchen. The host greets them, handles the slow or talkative ones, hands out menus (the static content), and passes finished orders on to the kitchen. The kitchen can then cook without being interrupted. It can even rearrange its stations, and guests never notice.

The problem it solves

When an application server faces the internet directly, it must do lots of jobs it isn't good at:

- Handle **TLS certificates**. (TLS is the encryption behind HTTPS. The certificate proves the server is who it says it is.)
- Serve images and JavaScript on top of running business logic.
- Reveal its own address and internal layout (its **topology**) to everyone.
- The least obvious one: **cope with slow clients**. A phone might dribble in one request over **4 seconds**. The app keeps a thread open that whole time, just waiting for bytes.

App servers shine at quick, predictable internal traffic. A reverse proxy soaks up the slow, hostile, messy public internet first. Then it hands the work to the app in a tidy, controlled burst.

How it works — the six jobs

In the course figure below, clients send their requests to the reverse proxy, which is the one public entry point. The proxy routes and load-balances them across App Server 1–3, keeps those internal servers hidden, and sends back the responses.

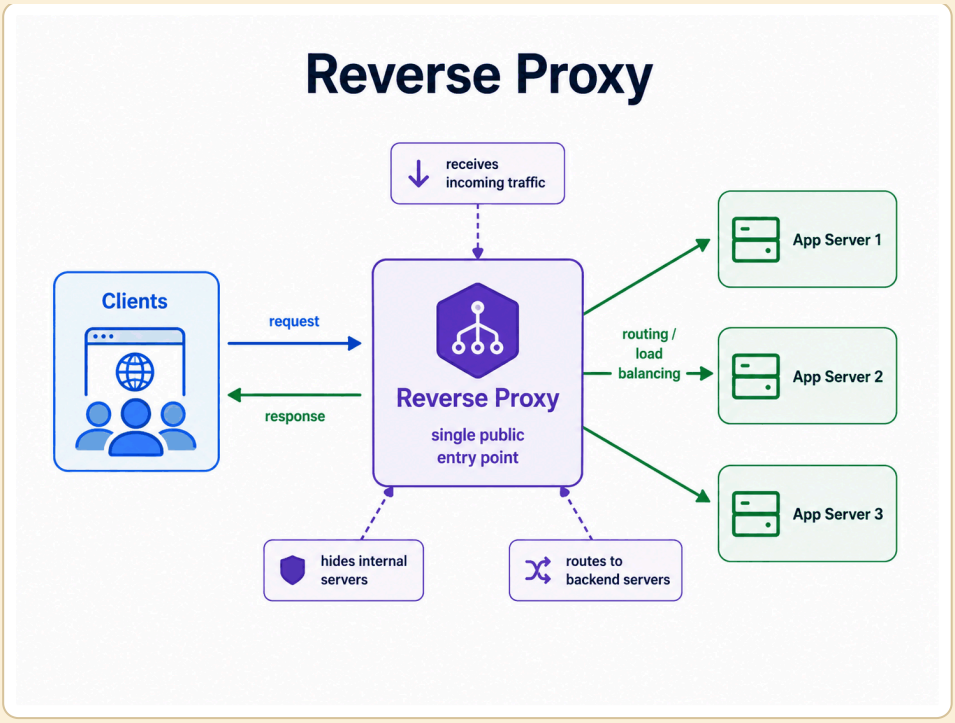


Figure — Reverse Proxy (course diagram)

1. **Terminates TLS.** It takes care of the HTTPS handshake and the encryption. Certificates, renewals, and protocol settings then live in one layer rather than in every app. *Why:* there's just one place to configure and patch.
2. **Buffers slow clients.** An **event-loop proxy** uses one thread (a single sequence of execution inside a program) to juggle many connections — the async I/O model (waiting on many network operations without blocking a thread for each) from the SSE (server-sent events, a long-lived server-to-browser stream) lesson. It can hold thousands of idle connections for a few kilobytes each. It waits until a slow upload is *completely* received, then sends a single fast request inward. *Why:* app threads are costly, and waiting on a slow phone shouldn't tie one up.
3. **Serves static files itself.** Images, JavaScript, and CSS (the stylesheet language that controls how pages look) never reach app code.
4. **Reuses connections.** **Keep-alive** lets a single client connection carry many requests. **Connection pooling** keeps ready-to-use connections open to the backends (the servers behind the proxy that do the real work). *Why:* you pay the handshake cost once, not on every request.
5. **Hides the topology.** Clients only see one public address. Behind it, internal servers can be renumbered, swapped, or added whenever you like.
6. **Serves as the enforcement point.** Every request passes through this one layer. That makes it the obvious home for authentication, limits, and the other policies later in this module.

Forward vs. reverse. It's the same mechanism, facing the other way. A forward proxy works for clients and hides who is asking (like a corporate web filter). A reverse proxy works for servers and hides who is answering.

The family. Add health checks and traffic spreading → load balancer (Module 6). Add auth, quotas, and routing → API gateway. Put cached copies in 200 cities → CDN. Place a small proxy next to every service → sidecar; connect them → mesh. One primitive, five names.

Walkthrough with numbers (buffering)

- Suppose **300 requests/s** come in, and each takes **3 s** to fully arrive from slow clients. With no buffering, app threads stay busy during the entire transfer: 300 x 3 = **900 threads** just waiting for bytes.
- With buffering, the app only receives complete requests, and each takes **50 ms** to process: 300 x 0.05 = **15 threads**. That frees up about **98%** of the thread pool.
- The rule: *threads in use = request rate x how long each request holds a thread*. Buffering cuts "how long" down from transfer time to processing time.

Trade-offs

You gain:

- The internet's messiness (slow clients, TLS, static files, junk requests) gets absorbed before your business logic ever sees it.
- One address, one place for certificates, and one policy enforcement point. The fleet behind it can change freely.
- It's cheap. A modern proxy forwards a request in **well under 1 ms** and handles enormous numbers of connections on modest hardware.

You pay:

- **An extra hop.** It adds a real delay, though under a millisecond, plus one more system to patch and watch. Since every request depends on it, it has to be **redundant** across failure domains (groups of machines that can fail together, such as one rack or one datacenter). That means paired instances, or **anycast** — one IP address advertised from several places, so the network picks a nearby one that works.
- **A trust boundary you have to set up correctly.** The app now sees the *proxy's* IP address instead of the client's. So the proxy writes the real client address into the `X-Forwarded-For` header (a header is a named key-value line sent along with an HTTP request or response). But clients can send that header themselves and lie in it. The rules: the proxy must **overwrite** any value the client sent (never append to it). And the app must trust the header **only** when the request came from its own proxy's address.
- **A false sense of safety.** A proxy on its own isn't a security control. It's just the place where you'll *put* your controls.

When not to use it

Almost everyone uses one, so the real question is whether you need *another* proxy layer:

- **Your platform already gives you one.** A cloud load balancer or hosted front end may already handle these jobs. Adding nginx (a widely used open-source web server and reverse proxy) behind it adds delay and one more header-trust boundary, with no separate owner or purpose.
- **Small, purely internal services on a trusted network** might not need one. That holds until they need encryption, retries, or **observability** (logs, metrics, traces). At that point, the sidecar brings this primitive to east-west traffic.
- **Don't stack proxies without thinking.** CDN + cloud load balancer + nginx + app means four sets of timeouts, body-size limits, and header-trust rules to check in every incident. When a **12 MB** upload is rejected, you might have to dig through all four.

In the interview

This is normally a box everyone assumes you'll draw first, not a question on its own. Expect "why not expose the app servers directly?" or a scenario involving an `X-Forwarded-For` bug.

> **30-second answer:** "A reverse proxy sits in front of the fleet. It terminates TLS and buffers slow clients, so a 4-second mobile upload doesn't tie up an app thread. It serves static files, pools connections, and hides the topology behind a single address. It's the primitive underneath half of system design. Add health checks and traffic spreading, and you have a load balancer. A gateway is this plus API policy. A CDN is this spread out geographically with a cache. The config detail I always check: the app trusts client-address headers only when they come from the proxy, and the proxy overwrites whatever outsiders send. Otherwise rate limits and audit logs run on data anyone can fake."

Likely follow-ups (with answers):

- *"What does buffering really save?"* With no proxy, busy app threads = request rate x the slow *transfer* time. With buffering, it's rate x the app's *processing* time. That's because the proxy holds the slow sockets cheaply on its event loop and forwards only finished requests.
- *"Forward vs. reverse?"* Forward acts for clients and hides who is asking. Reverse acts for servers and hides who is answering.

Mistake that fails candidates: trusting `X-Forwarded-For` without checking where the request came from; not knowing why the app sees the proxy's IP; or stacking proxies without giving each one a clear job.

Self-check

- **Q1: Explain forward vs. reverse in one sentence, then list the reverse proxy's four main jobs.** → Forward acts for clients and hides who's asking; reverse acts for servers and hides who's answering. Its main jobs are TLS termination, buffering slow clients, serving static files, and reusing connections. On top of that is its strategic job: being the single enforcement point.
- **Q2: Phones upload 200 photos/s, ~2 MB each, taking ~3 s to upload. How many app threads are busy with and without buffering (0.1 s processing)?** → Without: 200 x 3 = **600** threads waiting for bytes. With: 200 x 0.1 = **20** threads. That's a **30x** reduction.
- **Q3: After launch, the rate limiter reports that 99% of traffic comes from one client and blocks it, taking the site down. The audit log also blames `10.0.0.5` — the proxy's address — for a suspicious admin action. Diagnose it and give the header rules.** → Both systems used the proxy's address instead of a trusted client address. The limiter saw every user as `10.0.0.5` and blocked all of them, and the audit log pinned the action on the proxy. Fix: the proxy fills in `X-Forwarded-For` from the real connection source and overwrites any value that came in. The app trusts it only from approved proxy addresses. And everything that uses client identity — limits, logs, geo rules, bans — reads the same verified value.

Related patterns

- **Load Balancing** — adds health checks and traffic distribution on top of the primitive.
- **API Gateway** — adds API authentication, quotas, and routing.
- **CDN** — spreads cached proxies across geography.
- **Sidecar** — a small reverse proxy for each service instance, for east-west traffic.

TL;DR

Problem	App servers exposed directly take on all the internet's problems: TLS inside app code, threads stuck on slow clients, topology visible to all, and policy scattered everywhere.
Mechanism	A front server receives every request. It terminates TLS, buffers slow clients, serves static files, pools backend connections, and forwards complete requests. Outside, there's one address; inside, an invisible fleet; and one enforcement point in between.
Costs	A hop of under a millisecond, one more component that has to be redundant, and header-trust discipline (the client address is set by exactly one party and trusted from exactly one source).
Skip it when	Your platform already provides one (don't stack them), or the service is small and internal — until the sidecar brings the primitive east-west.

CDN

Original lesson: CDN on DesignGurus ↗

What it is. A **Content Delivery Network (CDN)** takes the reverse-proxy primitive, copies it into 200+ cities, and gives every copy a cache. Each user is sent to the closest copy, called an **edge** location. The edge answers from its local cache. Only on a miss does it fetch from your **origin** (the authoritative servers) — and that happens once per edge, not once per user.

> **Analogy.** Imagine every reader in Australia had to mail a request to one library in Virginia. Instead, the publisher sends copies of popular books to a local branch in Sydney. Readers borrow them locally. The branch only orders from Virginia when it doesn't have a title yet.

The problem: six-second pages in Sydney

The store opens in Australia. Customers in Sydney see **6-second** page loads, while the same pages open in **400 ms** at the Virginia office. The origin is perfectly healthy. It replies in **20 ms**, with no errors and no capacity problems. The time is going into *distance*:

- A single Sydney↔Virginia **round trip** takes around **200 ms**.
- The **TLS handshake** (setting up HTTPS encryption) needs **2–3 round trips**. That's **400–600 ms** before any content shows up.
- Then the page loads **60** images, scripts, and styles. Their dependencies pile on more round trips, even with connection reuse and parallel fetching.

Adding servers in Virginia can't speed up light. Even an origin that answered instantly would leave Sydney waiting 200 ms per round trip. The content has to be close to the reader already.

Why the obvious fixes fail

- **"Optimize the backend."** The origin adds 20 ms. The trip across the world adds 200 ms, several times over. Speeding up the smallest ~**3%** of the delay can't fix the ~97% that comes from geography. Always split server time from network time first.
- **"Open a datacenter in Sydney."** A full region would fix distance eventually. But you'd have to run the entire stack there and replicate the database between regions. You'd take on the consistency module's hardest trade-offs just to move identical images and JavaScript closer.
- **"Use HTTP/2 and buy more bandwidth."** (HTTP/2 is a newer version of the web protocol; bandwidth is how many bytes per second a link can carry.) **Multiplexing** (sending several requests over one connection at the same time) cuts the number of connections and improves parallel loading. More bandwidth moves more bytes at once. Neither one removes the round-trip wait before those bytes begin to arrive.

Most of the slow content is identical, popular, and static for every Sydney user. So this is a **distribution** problem, not a computation problem.

How it works

The course figure below shows the Sydney user reaching the Sydney edge cache in 15 ms (and a London user reaching their London edge in 12 ms). Only when there's a miss does an edge ask the **origin shield** — a single mid-tier cache. The shield fetches from the Virginia origin **once** and caches the result.

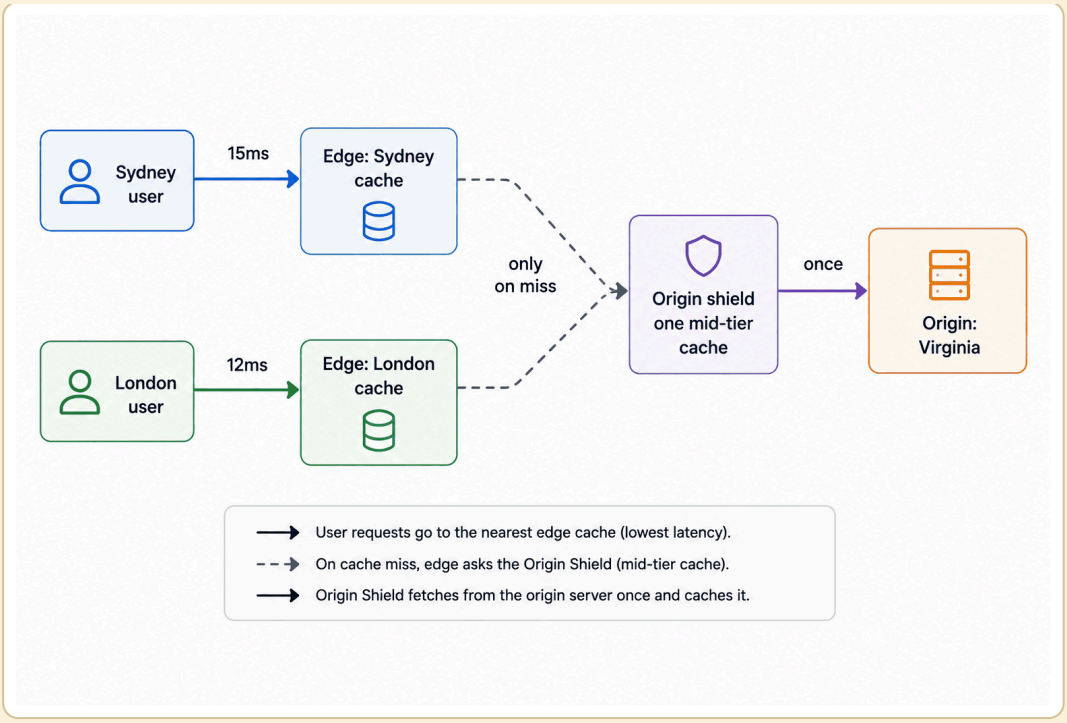


Figure — Users reach the nearest edge cache; only a miss travels to the origin (course diagram)

1. **Send each user to the closest edge.** This is done through DNS or **anycast** (one IP address advertised from many places, where internet routing picks the nearest one). *Why:* the handshake and the cached bytes are now ~15 ms away, not ~200.
2. **The rules travel in a header.** The `Cache-Control` header tells 200+ edges what they're allowed to store and for how long:
   - `max-age` — how many seconds the copy lives; browsers follow it too.
   - `s-maxage` — how long it lives in **shared caches** (CDNs) only.
   - `stale-while-revalidate` — keep serving the old copy while one refresh runs in the background.
   - `stale-if-error` — serve the old copy if the origin is failing. Every caching rule from Module 4 still holds. That includes **hit ratio** (the share of requests answered from cache), **TTL** (time to live — how long an entry counts as valid, which is your staleness budget), and misses that arrive all at once.
3. **Beat invalidation by never invalidating — use fingerprinted URLs.** **Invalidation** means removing a stale cached copy, and across 200 cities it takes seconds to minutes. So instead, the build tool (the program that packages source code for release) puts a hash of the content (a short fingerprint computed from the bytes, which changes whenever they change) into the filename, such as `app.3f9a2c.js`, and marks it as cacheable forever with `immutable`. When the content changes, it gets a *new* URL. That way, a stale response can never live at the new address.
4. **Get the cache key right.** The **cache key** is how an edge looks up a stored response. It's the URL plus whatever headers you declare. Cookies are the classic trap. If you vary on every cookie, every user's request becomes unique (hit ratio → zero). If you accidentally cache a `Set-Cookie` response (one that tells the browser to store a cookie, often a login session), it's even worse: private content gets served to other people.
5. **Add an origin shield.** Without one, a popular file missing at 200 edges at once triggers 200 origin requests. That's a **cache stampede** on a global scale. The shield **coalesces** (merges) those identical misses into a single fetch from the origin.
6. **Use the edge for more than caching.** TLS ends at the nearby edge, so a 3-round-trip handshake runs over 15 ms instead of across the ocean — ~600 ms drops to **45 ms**. Long-lived, already-warm **edge-to-origin connections** reuse finished handshakes, which speeds up even *uncacheable* dynamic requests. The provider's enormous network soaks up **DDoS** attacks (floods of malicious traffic meant to overwhelm a site). And `stale-if-error` gives you **graceful degradation**: during an origin outage, users get older content instead of an error. (Providers expose this as options like Cloudflare "Always Online".)

Walkthrough with numbers (Sydney, edge 15 ms away)

- **First byte.** TLS takes 3 x 15 ms = **45 ms** instead of **~600 ms**. That's true for cacheable *and* dynamic requests.
- **The 60 assets.** Once caches are warm, **~95%** are served from the edge in about **20 ms** each, mostly in parallel. The page loads in roughly **1.2 s** rather than 6 s. Dynamic content still makes the ocean trip, but the site no longer feels broken.
- **Load on the origin.** **70%+** of requests stop at the edge and never cross the ocean (the scaling capstone's rule that "most traffic shouldn't reach app servers"). Origin bandwidth falls. **CDN egress** (the bytes the CDN sends out) is usually cheaper per GB than origin traffic.
- **Flash-sale drill.** A hero image goes from **cold** (not in any cache) to popular worldwide in one minute. **~200** edges miss at the same time. The shield turns that into **one** fetch from the origin. The edges then serve the next **10 million** requests.
- **Deploy drill.** New JavaScript is shipped as `app.8c1d4e.js`, and newly served HTML (with a short `s-maxage`) points to the new name. There's no purge and no coordination across cities. The old file just sits unused until it's evicted.

Trade-offs

You gain:

- Latency that works with physics — content close to readers, handshakes close to users.
- A huge drop in origin load (**70–95%** for content-heavy sites) and cheaper bandwidth.
- DDoS absorption, plus `stale-if-error` keeping you available during origin outages.

You pay:

- **Staleness spread over 200 cities.** A purge doesn't happen instantly; it spreads. Fingerprinting removes staleness for assets, but HTML and API responses still need explicit TTL budgets.
- **The header rules affect security.** One overly loose header can land private content in a shared cache. Review cache directives as carefully as you review authentication code.
- **Debugging gets harder.** Which edge answered? Was it a hit or a miss? When was it stored? Look at headers like `cf-cache-status` and `x-cache` first.
- **You pay per GB, and a third party sits on your critical path.** Very large sites use several CDNs so they don't depend on just one provider.

When not to use it

- **Purely internal apps** — staff in two offices don't need copies in 200 cities.
- **Products used in one region with an origin nearby** — the latency argument gets weaker (though DDoS protection and offload might still make it worthwhile).
- **Every response is unique** (heavily personalized APIs) — shared caching doesn't help. Edge TLS and warm connections still do, though, so API-only products sometimes use CDN *acceleration* without caching response bodies.
- **As a cover-up for a slow dynamic origin** — the edge can't erase a **2-second** server render. Fix the query first (Module 4's rule).

Dangerous misuse: caching private data. Suppose an API response includes `Set-Cookie`, or an account page is missing `private`. A shared cache may then hand one user's response to someone else. The CDN did exactly what the headers said — the headers were just wrong.

Fingerprinted URLs vs. purging

	Fingerprinted URLs (`app.3f9a2c.js`)	Purge ("delete /banner.jpg everywhere")
Propagation	Instant — it's a new URL, so nothing needs invalidating	Seconds to minutes, depending on the provider, and you have to trust it
Can stale content exist?	Not possible by design	Yes, while the purge spreads
Cache lifetime	Forever + immutable — the best possible hit ratio	Whatever TTL you were brave enough to set
Works for	Anything that something else links to (assets referenced from HTML)	The documents doing the linking, and emergencies
Cost	Build-tool setup (standard in bundlers — tools that combine code files for the browser)	An API call — plus trust

Standard setup: fingerprint the assets that get referenced and cache them forever. Give the HTML that references them a short TTL using `s-maxage` plus `stale-while-revalidate`. Save purges for emergencies and legal takedowns.

Real-world examples

- **Akamai, Cloudflare, Fastly, CloudFront** — the pattern turned into an industry. Cloudflare alone carries a large share of web traffic.
- **Netflix Open Connect** — puts cache servers *inside* internet service providers. Video makes up a huge share of traffic, so skipping the ocean crossing entirely beats making it faster.
- **RFC 5861** (RFCs are the numbered documents that define internet standards) — the standard for `stale-while-revalidate` and `stale-if-error` (Module 4's stale serving and Module 5's fallbacks, written as HTTP headers).
- **Edge compute** (running small pieces of your own code on CDN servers) — Cloudflare Workers and Lambda@Edge run authentication, redirects, and experiment selection close to users.
- **For AI engineers:** model weights are enormous static files, so model hubs deliver them through CDNs. GPU inference **cold starts** (starting a model server from scratch) also pull weights through the same edge network. Token streams (the model's output arriving word-piece by word-piece) are unique to each request and can't be cached. They still benefit from edge TLS and warm origin connections, which cuts **time to first token** (how long until the first piece of the answer appears) for far-away users.

In the interview

A classic prompt: "Users in Australia see slow pages, but the Virginia servers look healthy." Spot distance before blaming capacity. Then explain how you'd handle invalidation and personalization.

> **30-second answer:** "This is a distance problem. Each round trip is 200 ms, and a TLS handshake costs three of them. No origin tuning fixes that. I'd put a CDN in front. Fingerprinted static assets get cached forever at the edge. HTML gets a short shared-cache TTL with stale-while-revalidate. TLS ends at the edge, so the handshake runs over 15 ms instead of an ocean. An origin shield merges global misses into one fetch. Dynamic requests travel over warm edge-to-origin connections. And I'd review Cache-Control like auth code, because one wrong header serves private data from a shared cache. Seventy-plus percent of requests never cross the ocean, and the rest cross once, on a warm connection."

Likely follow-ups (with answers):

- *"Marketing needs the homepage banner changed right now, but your assets are cached forever."* The new image has a new fingerprinted URL, so it never clashes with the old cached entry. Only the HTML that references it has to update — through its short TTL, or with one emergency purge. That's precisely why the policy has two layers.
- *"Can API responses go on the CDN?"* Public responses that are the same for every user: yes, using `s-maxage` and a correct cache key. Personalized bodies: never in a shared cache. They do still gain from edge TLS and warm connections across the ocean. Cache based on what makes a response unique.

Mistake that fails candidates: tuning backend code to fix a delay caused by geography; relying only on purges for invalidation; or caching a personalized API without defining its cache key.

Self-check

- **Q1: Why can't tuning the origin fix Sydney, and which two latency costs does the edge remove?** → The delay is ~200 ms of travel per round trip, not 20 ms of computing. TLS alone needs 2–3 trips, and then come the dependent asset fetches. The edge removes the **content journey** (cached bytes are 15 ms away) and the **handshake journey** (TLS goes from ~600 ms → 45 ms).
- **Q2: Give header policies for (a) a product image, (b) product-page HTML, (c) cart JSON.** → (a) A fingerprinted URL with `Cache-Control: public, max-age=31536000, immutable`. It's kept forever and only replaced by a new URL. (b) `s-maxage=60, stale-while-revalidate=300`. Shared caches keep it for one minute and serve the stale copy during a refresh. (c) `Cache-Control: private, no-store`. No shared cache should ever hold personal data.
- **Q3: After a deploy that only touched caching headers, users start seeing other customers' account dashboards. What's the mechanism, the repair, and the prevention?** → The deploy made personalized responses cacheable in shared caches. It either added `public`/`s-maxage` or removed `private`/`no-store` (caching a `Set-Cookie` response has the same effect). The CDN saved one user's dashboard under a shared URL and served it to others. Right now: purge every affected entry and put `private, no-store` back on authenticated responses. For prevention, treat cache directives as security-sensitive. Make `private, no-store` the default for authenticated routes, add automated checks, and require an auth-level review for anything that loosens them.

Related patterns

- **Reverse Proxy** — the CDN spreads this primitive across cities and adds caching.
- **Cache-Aside** — browser → CDN → app cache → database form a chain, where each layer handles the misses from the one before.
- **Cache Stampede Prevention** — the origin shield merges misses from across the globe; `stale-while-revalidate` makes stale serving a standard.
- **Graceful Degradation** — `stale-if-error` is a cached fallback while the origin is failing.

TL;DR

Problem	Distance — ~200 ms per ocean round trip, tripled by TLS and piled up by asset fetches — makes healthy origins feel broken to far-away users. No backend tuning touches travel time.
Mechanism	The reverse proxy, placed in 200 cities with caches: fingerprinted assets cached forever, short-TTL HTML with stale-while-revalidate, TLS at the edge, an origin shield merging misses, and warm connections for what can't be cached.
Costs	Managing staleness worldwide, header rules with security consequences, harder debugging, per-GB fees, and a third party on the critical path.
Skip it when	Apps are internal, users are in one region, or every response is unique — and never use it to hide a slow dynamic origin.

API Gateway

Original lesson: API Gateway on DesignGurus ↗

What it is. An API gateway is the one entrance for all **north-south** traffic (from the outside world into your services). It looks at each request's path and routes it to the right service. It enforces shared rules a single time — authentication, rate limits, TLS, validation. Then it passes the verified identity inward. No request reaches a service without being checked in, and no client has to know how your internals are laid out.

> **Analogy.** Think of a hotel front desk. Every guest checks in once, shows ID once, and receives a key card. From then on, room service and the gym trust the key card rather than checking passports again. Guests don't need to know which staff member works where; they just ask the desk. But the desk doesn't cook meals or run the spa. It checks people in, and each department does its own work.

The problem: eleven front doors

The store runs **11 services** (catalog, search, checkout, orders, accounts, reviews, …). Web, iOS, and Android clients call each one directly. Having eleven public entrances turns out to be costly:

- **Authentication is built 11 times.** Every service checks tokens (signed strings a client sends to prove who it is, issued at login) using whatever library version it last shipped. An audit turns up **3 implementations in 2 languages**. The reviews service's version is **14 months old** and ignores **token revocation** (cancelling a token before it would naturally expire). When a stolen account had all its tokens revoked, those tokens could *still post reviews*. That's a security incident, not an everyday bug.
- **The mobile app hardcodes 9 hostnames** (server names like `orders.shop.com`). Each public address is now a promise to clients, and a surface attackers can target. None of them can change without releasing a new app through the app store.
- **Internal refactors (restructuring code or services without changing what they do) break customers.** Splitting the orders service in two makes sense, but it breaks partner integrations. The internal **topology** (how the services are arranged) has turned into part of the public API.
- **Rate limits, log formats, and TLS settings** vary a little across all 11. None of them is reliably "the standard".

The shared root cause: **cross-cutting policy** (rules that apply to every request) is built separately by each service. It changes at 11 different deploy speeds, in 3 languages. It *will* drift apart. Better discipline can't fix something built into the structure — but a single shared entrance can.

Why the obvious fixes fail

- **"Use a shared auth library."** It's only consistent if all 11 services adopt it and keep upgrading. The reviews service fell 14 months behind precisely because upgrading was optional. Several languages need several parallel versions of the library. And the hostnames and topology are still exposed.
- **"Just put nginx in front for routing."** A plain reverse proxy gives you one address and hides the topology. But simple forwarding still leaves authentication in 11 places.
- **"Put it in the engineering standards doc."** A document can't overrule independent release schedules. Drift follows structure, not good intentions.

The reverse proxy has to turn into a real **enforcement point**, where the shared rules actually run.

How it works — five responsibilities

The course figure below shows Web, Mobile, and Partners all coming in through a single API Gateway (authN · rate limits · routing · TLS · validation · request IDs). The gateway routes `/catalog/*` (with verified identity), `/orders/*`, and `/reviews/*` to their services. The other services stay invisible to clients.

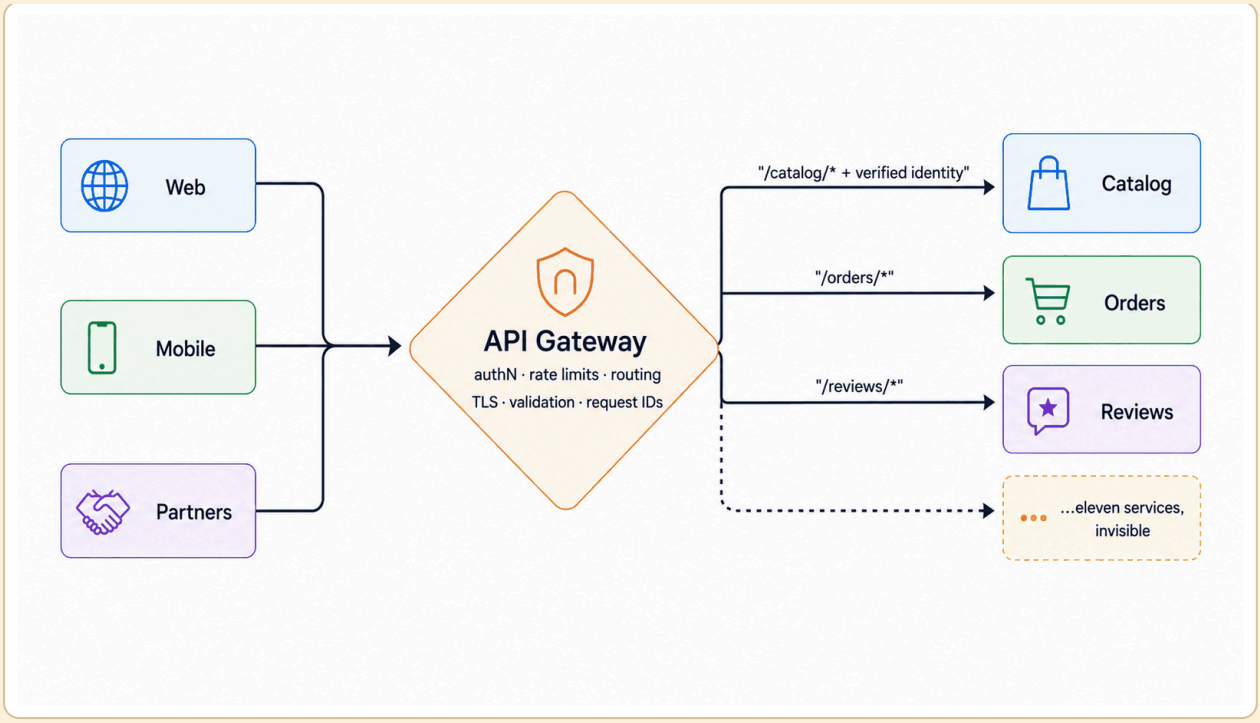


Figure — API Gateway (course diagram)

1. **Routing — the topology is private again.** Paths are mapped to services (`/orders/*` → orders), and clients keep using a single hostname. Splitting orders into order + fulfillment becomes just a routing change that nobody outside notices.
2. **Authentication, done once.** At the boundary, the gateway checks the token's signature (a cryptographic seal proving who issued it and that it wasn't altered), expiry, and revocation. Then it passes the verified identity inward as **signed headers or trusted claims** (claims are the facts inside a token, such as user ID and roles). Services never have to parse raw external tokens. *Why:* one revocation fix now protects every service, in a single deploy.
3. **AuthN at the gateway, AuthZ in the services.** **Authentication (AuthN)** answers "who are you?" — a fact that applies everywhere. **Authorization (AuthZ)** answers "are you allowed to do *this* particular thing?" (e.g., edit this review) — which is domain logic. Keep AuthN at the entrance, and keep AuthZ with each service's business rules.
4. **The policy set.** Rate limits and quotas, request-size limits, **schema validation** (checking that a request has the expected shape), and TLS. They're applied the same way at the one layer that sees every external request.
5. **The root of observability.** The gateway generates the **request ID** that distributed tracing (following one request across every service it touches) is built around. Every later **span** (a timed step in a trace) then links back to the moment the request first arrived. It also writes one uniform audit record for each external request.

The rule that keeps it stable: policy yes, product no. Since everything passes through it, teams will want to add response transformations, multi-service orchestration, discount checks… Every one of these pushes the gateway toward a hidden monolith (one big application that everything depends on and every team must change together). That monolith would need one team's sign-off for every feature, and would put *all* incoming traffic at risk on every deploy. Older architectures called this central integration layer an **ESB (enterprise service bus)**, and it was painful. **Test:** can you describe the rule without using any domain words? If not, it belongs in a service. A thin gateway that rarely changes is a stable gateway.

North-south only. Internal calls between services (**east-west**) must never leave the network through the gateway and come back in. That adds latency and a coupling that shouldn't exist. Internal traffic has its own tools (sidecar, mesh).

Walkthrough with numbers (one gateway for 11 services)

- **Auth:** a single implementation checks each token in about **1 ms**, using cached signing keys. The revocation fix goes out in **1 deploy instead of 11**.

- **Mobile app:** keeps **1** hostname instead of 9 — changing internal addresses no longer requires an app-store release.
- **Refactor drill:** orders is split into order + fulfillment through two gateway routes. Clients notice nothing.
- **The bill:** the gateway adds **1–2 ms** to each request. It also becomes one component whose failure hits all incoming traffic. So it's run redundantly, using load-balancer machinery (health checks, paired instances, anycast at scale). It's also kept thin, so it's rarely deployed.
- **Audit:** there's one consistent audit record per external request — which identity came in, through which route, with which request ID. It no longer relies on 11 services agreeing on a log format.

Trade-offs

You gain:

- **Policy enforced in one place** — auth with revocation that works, limits, and validation. Drift becomes impossible by design.
- **A private topology** — the internal architecture can change without breaking clients.
- **One root for observability** — request IDs, consistent logs, and the first span of every trace.
- **Simpler clients** — one hostname, one auth flow, and one set of error messages.

You pay:

- **A single point of failure, on purpose** — softened by redundancy and by keeping it thin (fewer changes = fewer outages).
- **An extra hop on everything** — 1–2 ms per external request, plus capacity for 100% of incoming traffic.
- **A risk of becoming an organizational bottleneck** — if one central team must approve every route, everyone slows down. Use routing-as-code, config files owned by each team, and automated review so everyday changes are self-service.
- **The risk of a disguised monolith, forever** — the urge to add domain logic never goes away. Keep enforcing the boundary.

When not to use it

- **A single service.** A monolith already has one entrance, and a reverse proxy handles the boundary work.
- **East-west traffic.** Sending internal calls through the north-south gateway adds latency, eats gateway capacity, and creates dependencies that shouldn't exist. That's the mesh's job.
- **Very small scale.** Two services and one client may not be worth it yet. That said, the moment a *second* independent auth implementation appears is where the argument for a gateway begins.

Watch out for the ESB coming back. Transformations, orchestration, and business rules may each seem reasonable alone. Together they add up to a distributed monolith with the gateway at its center. If product features regularly need gateway changes, the boundary has already broken down. Shaping for specific clients belongs in a BFF (next lesson).

API Gateway vs. Service Mesh

	API Gateway	Service Mesh
Direction	North-south: the outside world → your services	East-west: your services → one another
Identity question	"Which user or client is this?" (tokens, API keys — secret strings that identify a calling program)	"Which service is this?" (mutual TLS, workload identity)
Policy flavor	Auth, quotas, versioning — rules that face the product	Retries, timeouts, mutual TLS, traffic splits — infrastructure rules
Topology	One (redundant) entrance	A sidecar for each service — hundreds of small entrances
Failure blast radius	All incoming traffic	One service at a time

They work together. The gateway checks external users at a single entrance, and the mesh manages internal communication. Using either one in the wrong direction leads to exposure or needless detours.

Real-world examples

- **Kong, Apigee, AWS API Gateway** — the open-source, commercial, and managed versions of the pattern.
- **Netflix Zuul** — a well-known large-scale gateway. Its history shows how gateways pile up product logic until BFFs are needed to pull client-specific shaping back out.
- **Envoy-based gateways (Emissary, Gloo)** — the same proxy code runs at the public edge and inside meshes. One codebase plays different roles depending on where it's placed and how it's configured.
- **For AI engineers:** an **LLM gateway** is this pattern pointed at model providers. Authentication becomes safekeeping of API keys, rate limits become token budgets, and path routing becomes model selection. On top of that come usage metering (recording who used how much, for billing), fallback between providers, and semantic caching (reusing an answer for a question that means the same thing, even if worded differently).

In the interview

Once you've drawn microservices, expect "how do outside clients reach them?" Follow-ups then probe what the gateway is responsible for, how it stays available, and how you avoid a central monolith.

> **30-second answer:** "All north-south traffic comes in through an API gateway. It routes by path, so the internal topology stays private. It enforces the cross-cutting rules once — token validation with revocation, rate limits, size caps, TLS. And it passes a verified identity inward, so services never parse raw tokens. The line I hold is: the gateway authenticates, and services authorize. Identity is cross-cutting; permissions are domain logic. It stays thin — policy yes, product no — because a gateway full of business logic turns into a monolith with an approval queue. It runs redundantly like any entrance, and it creates the trace's request ID. Internal calls never take a detour through it; east-west belongs to the mesh."

Likely follow-ups (with answers):

- *"Isn't it a single point of failure?"* It's a dependency everyone shares *on purpose*, so run it the way you run the load balancer: redundant instances, health checks, and anycast at scale. Staying thin is the second layer of protection. Fewer changes mean fewer chances to break every client at once.
- *"What goes in the gateway, and what goes in the services?"* Ask if the rule can be stated without domain words. "Reject expired tokens" and "100 requests per minute" → gateway. "Only the review's author can edit it" or fraud rules → services. It's the same line as AuthN vs. AuthZ.

Mistake that fails candidates: putting business logic in the gateway; ignoring the slow creep of logic into it; sending internal calls through it; or claiming it can't fail without explaining the redundancy.

Self-check

- **Q1: Name the gateway's five responsibilities, and state the AuthN/AuthZ split in one sentence.** → Routing that keeps topology private; authentication with trusted identity passed inward; shared limits, validation, and TLS; the root of observability; and a simpler contract for clients. The gateway works out *who the caller is* (AuthN). Services decide *whether that identity may carry out a domain action* (AuthZ).
- **Q2: Gateway or service? (a) token signature validation (b) "may this user edit this document?" (c) per-API-key quotas (d) computing an order total (e) rejecting requests over 10 MB (f) applying a loyalty discount.** → Gateway: (a), (c), (e). Identity, quotas, and request hygiene need no domain words. Services: (b), (d), (f). Document ownership, order totals, and loyalty are product ideas. Any rule that contains domain nouns stays with its domain.
- **Q3: Two years in, almost every feature needs a change from the gateway team. Deploys happen daily and everyone dreads them. A trace also shows checkout's call to inventory leaving the datacenter and coming back in. Diagnose and repair.** → (1) ESB creep. The gateway has soaked up domain behavior, which forces constant product changes and a central approval queue. Move business rules back into services and client-specific shaping into BFFs. Add a review rule that bans domain vocabulary. Make routine routing self-service and owned by teams. (2) Checkout is pushing east-west traffic through the north-south gateway, which adds edge latency and shared load. Route internal calls directly or through the mesh. Both problems come from treating the entrance as the center of the system.

Related patterns

- **Reverse Proxy** — the gateway adds API policy to this primitive.
- **Rate Limiting** — one of the gateway's most important policies.
- **BFF** — takes client-specific response shaping out of the shared gateway.
- **Service Mesh** — the east-west counterpart, built on the same primitive.
- **Load Balancing** — its health checks and redundancy make sure one logical entrance isn't just one machine.

TL;DR

Problem	Eleven services with eleven entrances: auth built eleven times across eleven versions (one with no revocation), topology acting as public API, and policy drift baked into the structure.
Mechanism	A single north-south entrance. It routes by path, authenticates once and passes verified identity inward, enforces limits/validation/TLS, and creates the trace's request ID. It stays thin: policy yes, product no; AuthN at the entrance, AuthZ in services.
Costs	An intentional SPOF (keep it redundant and thin), 1–2 ms on every request, an organizational bottleneck unless config is self-serve, and a never-ending temptation to add logic.
Skip it when	There's one service (a reverse proxy is enough), the traffic is east-west (use the mesh), or the "gateway" is turning into an ESB.

Backend for Frontend

Original lesson: Backend for Frontend on DesignGurus ↗

What it is. A **Backend for Frontend (BFF)** is a thin backend made for *one* client experience — web, mobile, or TV — and owned by that client's team. It sits behind the gateway. It calls the domain

services it needs, in parallel, and hands back exactly the data that client's screens display. The services underneath stay the same; each client just gets its own API shape.

> **Analogy.** Think of a personal shopper. You could drive to six shops yourself (six slow trips). Instead, your shopper — who knows your size and taste exactly — visits them all quickly and brings back one bag holding only what you'll actually wear. The shopper doesn't set prices or make store policy. They only gather and tailor.

The problem: one API fits every client equally badly

The gateway gives every client one consistent way in. But a year later, the mobile team is having a hard time:

- The mobile **product screen needs data from 6 services**. That means **6 cellular round trips (request out, reply back, over the phone network) of ~300 ms** each.
- Each response sends back **40 fields**, but the phone displays only **6**. That wastes bytes, battery, and parsing time.
- Every change to a screen has to wait on a ticket to a backend team.

One API that belongs to no particular client can't suit a detailed web page, a bandwidth-hungry phone, *and* a bare-bones TV screen all at once. The gateway can't solve this either. Shaping responses for a specific client is **product behavior**, and the gateway's rule is "policy yes, product no". The BFF gives that shaping its own home, and the gateway stays thin.

How it works — four rules

In the course figure below, the mobile app and the web app each make **1 call** through the gateway's policy layer. The Mobile BFF (owned by the mobile team) fans out **6 parallel calls, ~5 ms each** to Catalog, Reviews, Pricing, Inventory, Recs, and Shipping. The Web BFF (owned by the web team) gathers what the web UI needs.

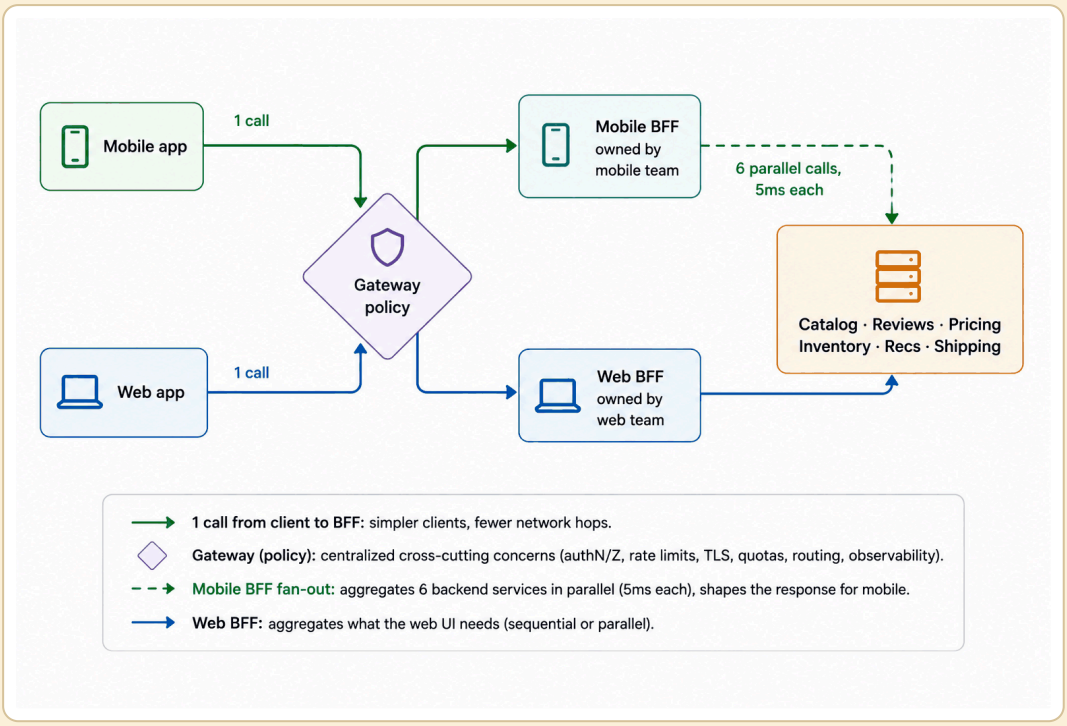


Figure — Backend for Frontend (course diagram)

1. **A BFF gathers and shapes data — and does nothing else.** It **fans out**, meaning it calls several services in parallel, over fast datacenter links. Then it returns exactly the fields one screen needs. Pricing, permissions, order totals, and other domain rules stay in the services. *Why:* a BFF that works out discounts turns into a chunk of monolith hiding in the presentation layer (the part of a system that formats data for display).

2. **The client team owns it.** The mobile team changes its own BFF on the mobile release schedule, with no backend ticket. *Why:* this change in ownership is the real point of the pattern.
3. **One BFF per experience, not per screen.** Web, mobile, and TV are different experiences, so each may get its own. **iOS and Android usually share a single mobile BFF** until their behavior truly splits. A **partner API** sits in a similar place in the diagram but is a different thing: a stable, documented, versioned public product.
4. **BFFs never call one another.** Requests travel in one direction: client → gateway → BFF → domain services. *Why:* calls from one BFF to another create an accidental service layer full of hidden dependencies.

Walkthrough with numbers (the mobile product screen)

- **Before:** 6 cellular round trips at ~300 ms (mostly one after another), bringing back a general-purpose payload of **~45 KB**.
- **After:** **1** cellular round trip to the BFF, which saves at least **1.8 s** of network delay. Inside the datacenter, the BFF makes **6 parallel calls of ~5 ms** and finishes gathering in about **20 ms**. It returns a **6 KB** payload with only what's shown on screen.
- **Why it works (the latency model):** move the *chatty* back-and-forth onto the fast network, and cross the slow network just once.

Trade-offs

You gain:

- **Latency and payload tuned for each client** — the chatty gathering happens over fast datacenter links.
- **Team independence** — client teams ship API changes at their own pace, and the ticket bottleneck goes away.
- **A gateway that stays thin for good** — response shaping has a home that isn't the entrance.

You pay:

- **N more things to deploy** — every BFF needs its own build pipeline (the automated steps that test and ship code), monitoring, and on-call owner (the engineer paged when it breaks).
- **Intentional duplication** — the web and mobile BFFs may put product data together differently, because their screens differ. Trying to erase all duplication with a shared "BFF framework" can bring back the very coupling you got away from.
- **A leak risk** — BFFs sit close to features, so domain rules tend to drift into them. If two BFFs apply the same rule differently, the product behaves inconsistently.

When not to use it

- **Only one client experience.** A web-only product's API already serves its one presentation. Adding another backend adds latency and operations work for nothing.
- **Small teams.** Three engineers can't usefully own four separate backends. The independence benefit only appears when separate teams own them.
- **When flexibility belongs on the client** — see GraphQL below.

BFF vs. GraphQL

Both solve **over-fetching** (receiving fields you don't use) and **under-fetching** (needing several calls to draw one screen). **GraphQL** lets the *client* write a query that describes exactly the shape it wants.

	BFF	GraphQL
Who shapes the response	The client's server team, per experience	The client's query, per request
Flexibility	Fixed shapes — any change needs a BFF deploy	Whatever the schema (the typed description of all available data) permits, no deploy needed
Server-side complexity	N thin services	One graph, with resolver (the function that fetches one field) efficiency to manage (the **N+1 query** problem — one query per item instead of one batched query), query cost limits, and caching complexity
HTTP caching	Easy (fixed endpoints)	Harder (queries are usually sent as POST, the HTTP method for sending data, which caches skip)

You can also mix them — some teams build each BFF *using* GraphQL (one graph per experience). A rule of thumb: use BFFs when you have a few stable client experiences. Use GraphQL when many clients' data needs change faster than you can deploy — and budget for GraphQL's server-side defenses against expensive queries.

In the interview

A typical prompt: a slow mobile screen makes six calls and gets big payloads — fix it without touching web. Or: you've already drawn a gateway; where does mobile-specific shaping go?

> **30-second answer:** "Every client experience gets a backend-for-frontend. It's a thin aggregation layer behind the gateway, owned by that client's team. Mobile's six-call screen turns into one call. The BFF fans out to the six services in parallel over datacenter links and returns exactly the fields the screen shows — crossing the slow network once instead of six times. The rules: BFFs aggregate and shape but contain no domain logic; they never call each other; and iOS and Android share one until they really diverge. GraphQL is the alternative when data needs change faster than BFF deploys — the same problem, fixed on the client side, with a cost on the server side."

Likely follow-ups (with answers):

- *"Can the BFF handle checkout? It already talks to every service."* It can gather reads and *kick off* the operation. But checkout is a write with rules about money and inventory, so a **domain-owned saga** (a multi-step workflow with an undo step for each action) has to coordinate it. The BFF may start the saga and format its status for display, but it doesn't own the business transaction.
- *"One BFF or two for iOS and Android?"* Start with one — they're usually the same mobile experience. Split only when a real difference (a feature only one has, a tablet redesign) justifies another deployable and another owner.

Mistake that fails candidates: putting business logic in a BFF; letting BFFs call each other; or making one backend per screen with no ownership model that can last.

Self-check

- **Q1: The gateway, BFF, and services now all sit between client and data. Give each layer's one-line job and its owner.** → Gateway (platform team): shared authentication, limits, and routing. BFF (each client team): gathering data and shaping responses. Services (domain teams): authorization, business rules, writes, and the authoritative data. Policy at the entrance, presentation in the middle, domain truth in the services.

- **Q2: You have web, iOS, Android, TV, and partner integrations. How many BFFs, and what's different about partners?** → Three BFFs plus one API product. The BFFs are: web; one shared mobile BFF for iOS + Android (until they diverge); and TV (very different screens). Partners get a **public API product** shaped like a BFF. It must stay stable, documented, and versioned, because partners can't update in lockstep with your releases.
- **Q3: A customer sees different order totals on their phone and laptop. The mobile BFF applies a loyalty discount locally to save time; the web BFF doesn't. What failed, what's the fix, and what's the rule?** → A domain rule leaked into a presentation layer, so the same order is calculated differently on each device. Move loyalty discounts into the pricing or orders service, and have both BFFs show the authoritative total. The rule: BFFs gather and shape; anything about discounts, totals, or permissions stays in a service. *If two BFFs could disagree about a value, it was never presentation logic.*

Related patterns

- **API Gateway** — BFFs sit behind it and keep response shaping out of it.
- **Request-Response** — the latency lesson: run calls in parallel over fast internal links, and cross the slow client network once.
- **Graceful Degradation** — a BFF can build a partial screen when one of the services it relies on is down.

TL;DR

Problem	One uniform API fits every client equally badly. Mobile pays for six cellular round trips and 45 KB of unused data per screen, and every client change waits in line behind a backend team.
Mechanism	A thin backend for each client experience, owned by the client team and placed behind the gateway. It fans out in parallel over fast links and returns one shaped response per screen. No domain logic, no BFF-to-BFF calls, and one per experience rather than per screen.
Costs	N deployables with real operational weight, intentional duplication, and an ongoing risk of business rules leaking into presentation.
Skip it when	There's only one client experience, teams are too small to own the layers, or GraphQL's flexibility fits better (with its server cost budgeted).

Rate Limiting

Original lesson: Rate Limiting on DesignGurus ↗

What it is. Rate limiting counts requests **per identity** at the entrance. When an identity goes over its budget, further requests are refused right away with a polite "slow down" that machines can read. That means HTTP **429 Too Many Requests**, a `Retry-After` header, and headers showing how much budget is left. It turns fairness into simple arithmetic.

> **Analogy.** Picture a turnstile with a personal pass. Everyone gets a certain number of rides per hour, and the pass refills over time. When yours runs out, the gate politely shows "try again in 30 seconds" instead of letting you clog the line. The course diagram even calls the gateway a "turnstile".

The problem: one partner's bug takes everyone down

The public API serves the mobile apps plus several hundred partner integrations. One partner ships a bug that makes it retry in a loop **without backoff**. (Backoff means waiting a bit longer between each retry.) Their traffic jumps from **40 requests/s to 4,000** — 100x their normal level. Each request has valid credentials and is properly formed, so the gateway authenticates every one. Shared services get overloaded. One customer's bug turns into an outage for every partner and every mobile user.

The logs also reveal two quieter abusers. One is a **scraper** pulling the whole catalog through search. The other is a **credential-stuffing** bot hitting login — it tries huge numbers of leaked username/password pairs. Accidental, greedy, and hostile callers all slip through the same structural gap: **the entrance checks *who* you are, but nobody counts *how often* you come in.** The bulkhead lesson (separate resource pools so one overload can't sink everything) kept damage contained inside the system, by resource. At the public boundary, though, callers have identities and pricing tiers. So fairness has to be enforced there, before the work gets in.

Why the obvious fixes fail

- **"Block the partner's IP."** The partner is still a paying customer. Their broken endpoint should be *slowed down*, not cut off completely. Bots also hop across thousands of addresses anyway. Worse, **NAT (network address translation)** places many users behind one public IP. Blocking a university's address can lock out **5,000** innocent students.
- **"Scale up and soak it up."** Buying servers to handle someone else's endless retry loop turns their bug into your bill. An autoscaler (a control loop that adds or removes servers as load changes) will gladly automate that waste for you. Capacity should serve real demand, not a malfunction with no limit.
- **"Email the partner."** Their fix might take days and depends on their deploy schedule, while your outage keeps going. As the circuit-breaker lesson put it: when minutes count, the right human decision has to become an automatic one.

How it works — four decisions

In the course figure below, every request reaches the **gateway turnstile** first. If there's budget left, it uses up a token (an atomic operation — one that happens completely or not at all, with no other request slipping in halfway) and forwards the request to the services. If the bucket is empty, it refuses at the door with **429 + Retry-After in ~0.1 ms**. The gateway instances share their counters in **Redis** (a fast in-memory key-value store) using an atomic check-and-decrement, so all the instances agree.

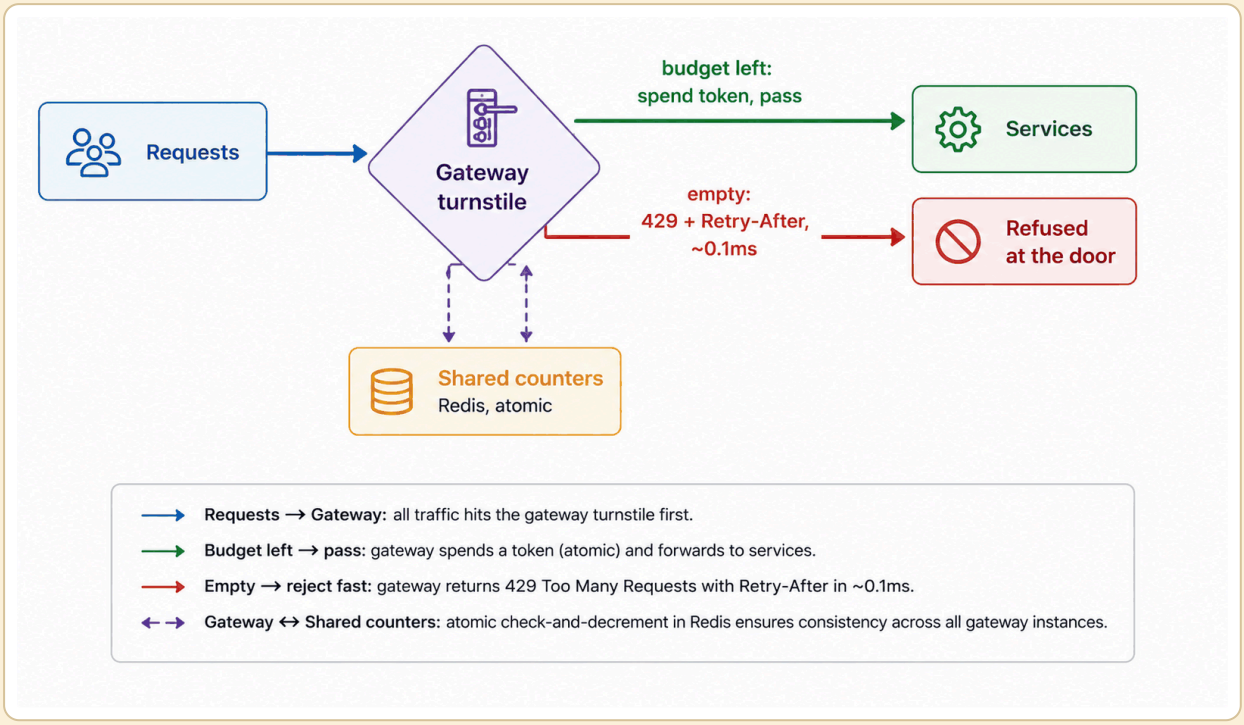


Figure — Rate Limiting (course diagram)

Decision 1 — a refusal is a contract, not just a "no". Send back `429` with `Retry-After: 30`, along with budget headers such as `X-RateLimit-Remaining`. *Why:* well-behaved SDKs (software development kits:

ready-made client libraries for calling an API), like the polite client from the retry lesson, read these and back off on their own. A 4,000/s loop then shrinks itself within seconds.

Decision 2 — the algorithm.

- **Fixed window** ("100 per minute; the counter resets at :00"). Very simple, but it suffers a **boundary burst**. 100 requests at 11:59:59 plus 100 at 12:00:01 = **200 in 2 seconds**, and all are allowed. It's only good enough for rough protection.
- **Sliding window** (keep counting the last 60 seconds continuously). There's no boundary burst, but there's more bookkeeping. The practical **sliding-window counter** approximates it by blending two neighboring fixed-window counts.
- **Token bucket — the default.** Every identity has a bucket that holds at most **b** tokens and refills at **r** tokens per second. Each request uses one token, and an empty bucket means 429. **r** is the rate you can keep up forever; **b** is the burst you're willing to forgive. *Why it's the default:* an ordinary page load fires **~10** requests at once. The token bucket lets those honest bursts through while still capping the long-term average.

Decision 3 — the key: what exactly are you counting per? Use a trusted **API key or account** as the main identity. Use IP only as a *generous* backstop for anonymous callers, since NAT can make one IP stand for thousands of people. Two refinements: **tiers** (e.g., free 10/min, paid 1,000/min) and **cost weighting** (an expensive search uses 10 tokens; a health check uses 1).

Decision 4 — where the counters live. A gateway normally runs N instances. If each one enforces 100/s by itself, the true limit becomes N x 100. Your options:

- **Shared Redis**, with an atomic `INCR` + `EXPIRE` (Redis commands to add one to a counter and make it vanish after a set time): about **0.5 ms** per request, and exact.
- **Local buckets that sync now and then:** faster, but approximate. A **tenant** (one customer in a shared system) briefly getting 110 instead of 100 is fine for *protection*. *Billing*, however, needs exact counts.

Walkthrough with numbers (Tuesday's incident, replayed: r=100/s, b=200 for the partner tier)

1. **The buggy partner** sends 4,000/s. The first **~200** requests use up the burst allowance. After that, **100/s** get through and **3,900/s** receive a 429 in about **0.1 ms** each. None of them reach an app service. Compared with **~50 ms** of real work, refusing is **~500x** cheaper.
2. **Nobody else notices anything.** Every partner and every mobile group has its own bucket, so one tenant can't drain another's. It's the bulkhead idea, applied by *identity* at the boundary.
3. **The partner's SDK** respects `Retry-After` and slows itself down. The 100/s refill keeps its normal traffic working. Later, their engineers get an incident graph instead of a shared outage.
4. **Login has a rule of its own: 5 attempts per minute per account**, plus an IP backstop. Credential stuffing falls from thousands of attempts to a few dozen.
5. **Search costs 10 tokens**, so a catalog scraper burns through its budget **10x** faster than cheap callers do.
6. **Operations watches the pattern, not just the total.** Dashboards show accepted vs. rejected requests by identity, endpoint, and tier. That helps tell one broken integration apart from a global capacity problem, or from a burst setting that's too small. Alert on *lasting changes* in each tenant's 429 rate — not on single 429s, since a working limiter refuses some traffic by design.

Trade-offs

You gain:

- **Fairness built into the structure** — no identity can starve the others, guaranteed by arithmetic rather than hope.
- **Abuse economics flipped around** — attacks and bugs cost ~0.1 ms each instead of full service work.

- **Sensible capacity planning** — known ceilings per tenant combine with the autoscaler's maximum, so you can finally work out the worst case.
- **A way to make money** — tiers turn the rate limiter into your pricing page.

You pay:

- **Real bursts get refused** if **b** is too small. Users see "random" failures during normal page loads. Measure actual client bursts before you choose b.
- **A counter hop on every request** (~0.5 ms for Redis) — or you accept approximate local counting. Pick between exactness and latency on purpose.
- **False positives from shared IPs** whenever IP is the main key.
- **Lots of knobs to tune** — endpoint costs, tiers, refill rates, and burst sizes all need owners and monitoring.
- **Risky config changes** — a mistaken global cut can deny *every* client just as fast as the intended rule blocks one abuser. So roll out limit changes through reviewed, staged releases that can be rolled back quickly.

When not to use it

- **As a replacement for capacity.** If ordinary, valid traffic goes over the limit, your users are paying for your under-provisioning. Limits should rein in *outliers*, not the typical workload.
- **Internal east-west overload, mostly.** Overload between services usually calls for **backpressure** (slowing down the sender) and bulkheads. Meshes *can* rate-limit internally, but their identities and goals aren't the same as customer fairness.
- **Small private APIs with three known callers** — just talking to them may be simpler than running limiter infrastructure.

Common misuse: sending a bare 429 with no `Retry-After`, no budget headers, and no documentation. A well-meaning client can't adjust, and may retry *harder*, which turns your protection into yet another burst.

The algorithms compared

	Fixed window	Sliding window (counter)	Token bucket
Boundary burst flaw	Yes — 2x at the edges of a window	No	No
Allows natural bursts	Only by accident	Not really — it's smooth by design	Yes, by setting b (that's the feature)
Memory per key	One counter	Two counters	Token count + timestamp
Complexity	Trivial	Low	Low
Reach for it when	Rough protection is enough and simplicity matters most	Smoothness matters (close to billing)	By default — it matches real traffic

Three ways to turn work away: rate limiting = ongoing fairness per *identity*, at the entrance. **Bulkhead** = ongoing containment per *resource*, inside the system. **Load shedding** = a global *emergency* drop of low-priority work, no matter who sent it. Large systems use all three, each in its own place.

Real-world examples

- **Stripe and GitHub** — publish their limits, document their tiers, and use a full set of headers (`X-RateLimit-*`, `Retry-After`), so SDKs behave predictably.
- **Cloudflare** — applies limits at edge locations before abusive traffic ever heads toward your origin.
- **Redis atomic counters** (`INCR` + `EXPIRE`) — the usual shared-state implementation. **Envoy and nginx** come with limiter modules.
- **For AI engineers:** LLM APIs limit **tokens per minute**, not requests, because one call can cost 100x more than another — that's cost-based limiting. An internal LLM gateway enforces per-tenant token

budgets and model quotas. It also acts as a polite client, handling the provider's 429s through the retry protocol. (Module 5 covered receiving `Retry-After`; this lesson covers sending it.)

In the interview

"Design a rate limiter" is often asked as a question by itself (algorithms, keys, distributed counters). Inside a broader API design, it's what keeps one client from using up shared capacity. Expect to be asked about the fixed-window boundary.

> **30-second answer:** "At the gateway, I'd use a token bucket per API key. The refill rate r is the sustained budget, and the bucket size b forgives natural bursts. I'd add tiers per plan, and cost weights so expensive endpoints use more tokens. Refusals are a contract — a 429 with Retry-After and remaining-budget headers — so well-behaved SDKs back off automatically. Counters live in Redis with atomic increments. That's exact enough for protection; I'd only pay for more precision if the limits drove billing. Per-IP limits are only a generous backstop for anonymous traffic, because NAT can make one IP five thousand people. Refusing costs a tenth of a millisecond, versus fifty to serve. The rate limiter is the cheapest capacity I own."

Likely follow-ups (with answers):

- *"What's wrong with fixed windows?"* The boundary burst. Under a "100 per minute" rule, 100 calls at 11:59:59 and 100 more at 12:00:01 add up to 200 in two seconds. Sliding windows get rid of the boundary; token buckets also model legitimate bursts.
- *"Your gateway runs 20 instances — where does the count live?"* Shared Redis counters give exact enforcement for the price of one sub-millisecond hop. Local buckets that sync periodically are faster but approximate. Protection can live with briefly allowing 110 against a limit of 100; billing can't live with that kind of error.

Mistake that fails candidates: banning customers instead of slowing them down; using IP as the main identity for authenticated traffic; sending silent 429s without `Retry-After`; or hiding a lack of capacity behind tight limits.

Self-check

- **Q1: In a token bucket, what do r and b control, and which normal behavior depends on b?** → r is the average rate that can be kept up forever. b is how far above that a short burst may go. Normal users do burst — page loads fire many requests at once, and mobile apps sync after waking up. Without b, the limiter makes normal behavior look broken.
- **Q2: Choose keys and limits for (a) public catalog search, (b) login, (c) a partner's nightly bulk export, (d) free vs. paid tiers on one endpoint.** → (a) A bucket per API key, with a high token cost for search, so scraping drains the budget quickly. (b) A strict per-account limit (e.g., 5/min), since credential stuffing targets accounts, plus a generous anonymous IP backstop. (c) A separate, documented schedule or bulk endpoint, so real batch work doesn't fight with interactive traffic. (d) r and b set by plan, with the upgrade path explained in the 429 response.
- **Q3: At a university launch, thousands of students install the app, and all of them get 429s within an hour. Overall traffic is modest and the services are healthy. Diagnose it, fix it, and state the rule.** → The whole campus shares one NAT IP, and IP was the main key. So thousands of people were counted as a single identity — the limiter itself caused the outage. Key authenticated requests by account or API key, and keep IP only as a generous anonymous backstop. The rule: **key on the identity the policy is actually meant to limit.** An IP is a network location, not a person.

Related patterns

- **API Gateway** — the shared place where this policy is enforced.

- **Retry with Exponential Backoff** — the client side that respects `Retry-After`.
- **Bulkhead** — contains internal resources, while rate limits restrict identities at the entrance.
- **Auto-Scaling** — known tenant ceilings make it possible to calculate the most capacity you'll need.
- **Graceful Degradation** — when the whole system is under pressure, load shedding can give priority to paid or critical work.

TL;DR

Problem	The entrance checks identity but not frequency. One buggy partner's 4,000/s loop, a scraper, or a bot — and everyone shares the outage. Blocking IPs bans whole campuses; scaling up buys servers for a malfunction.
Mechanism	Token buckets per trusted identity at the gateway (r = sustained rate, b = forgiven burst), weighted by endpoint cost, tiered by plan, and counted in shared atomic storage. Refusal takes ~0.1 ms and comes with a contract: 429, Retry-After, and budget headers.
Costs	Bursts set too small punish honest clients, a counter hop (or imprecision), NAT false positives when IP is the key, and many knobs that need owners and 429 dashboards.
Skip it when	It would just ration your own under-provisioning, the problem is internal east-west overload (use backpressure/bulkheads), or three known consumers could simply talk to each other.

Cursor-Based Pagination

Original lesson: Cursor-Based Pagination on DesignGurus ↗

What it is. Rather than asking the API for "page 2", the client tells it **where it stopped**. A **cursor** is an opaque (the client can't see inside or rely on its format) bookmark — something like "everything after order 8412" — that comes back with each page. It's tied to a *row*, not a *count*. So it stays fast no matter how deep you go, and it doesn't skip or repeat items when the list changes.

> **Analogy.** Compare a bookmark with "page 212". If someone adds new pages at the front of your book, "page 212" now shows different text. Your bookmark, though, stays exactly where you stopped reading.

The problem: offset pagination fails two ways

Offset pagination requests a numbered page and makes the database skip over a number of rows: `?page=2` → `OFFSET 20 LIMIT 20` (SQL for "skip 20 rows, then return 20").

1. **Drift.** A user reads page 1 of the mobile order feed. Then three new orders show up at the top, which shoves every row down by three. Page 2 now begins with three items the user has already seen — **duplicates**. Deletions do the reverse and **skip** rows the user never saw. Page numbers are positions in a list that keeps shifting.
2. **The cost of deep pages.** A partner exports the catalog using `?page=5000`. `OFFSET 100000` can't leap straight to row 100,000 — instead, the database has to scan and throw away 100,000 rows first. The work grows in a straight line with depth: page 1 takes **2 ms**, while page 5,000 takes **800 ms**. The further the export goes, the slower it gets.

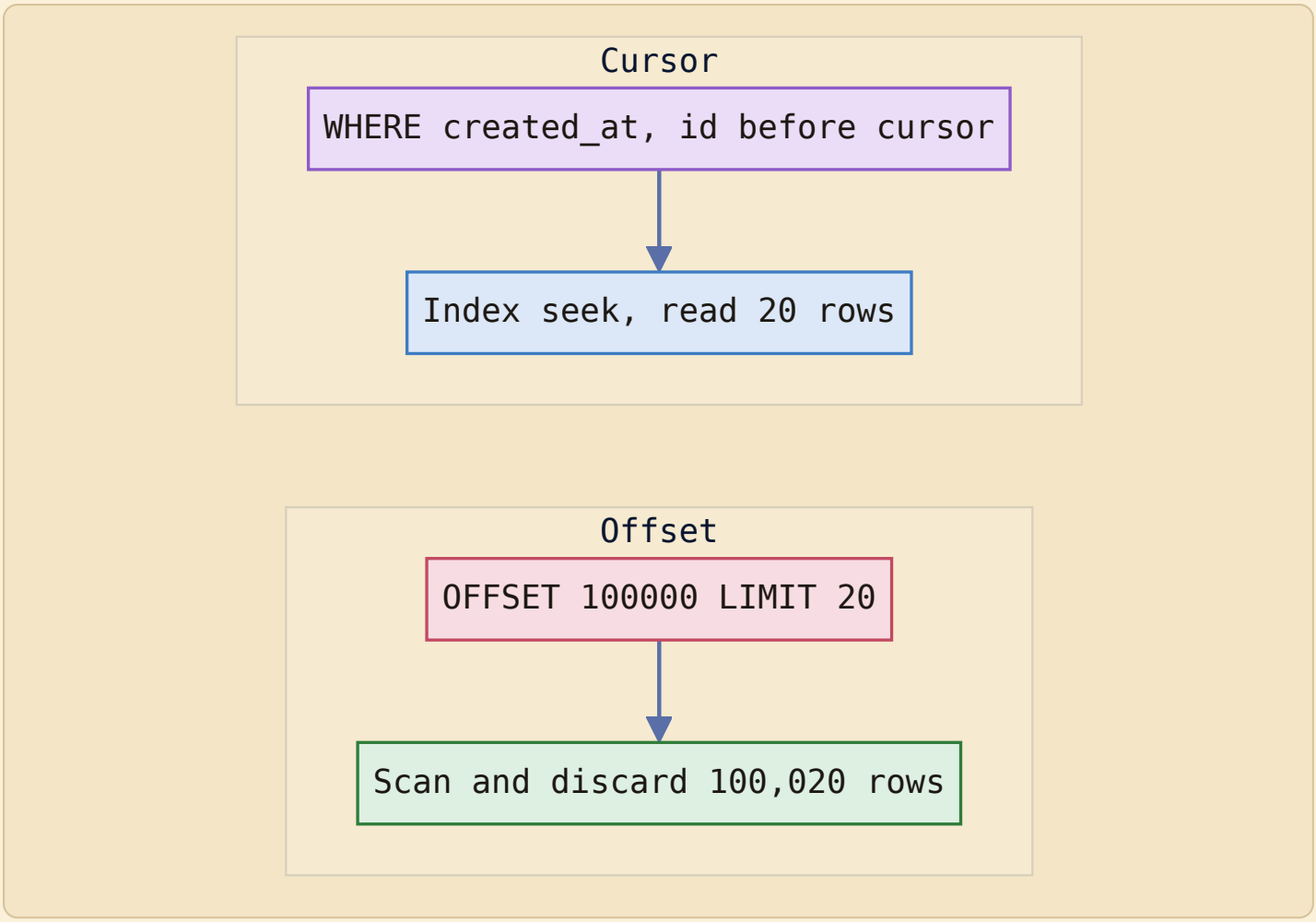
How it works

```
Offset:  SELECT ... ORDER BY created_at DESC, id DESC
         OFFSET 100000 LIMIT 20                -- scan & discard 100,020 rows

Cursor:  SELECT ... WHERE (created_at, id) < (:cursor_ts, :cursor_id)
         ORDER BY created_at DESC, id DESC LIMIT 20
                                               -- index seek: reads exactly 20 rows
```

1. **Send a cursor back with every page.** The response contains `next_cursor`, an opaque token. It's usually the last row's sort values, encoded in **base64** (a way of turning data into text). *Why:* it records "where I stopped" as values rather than as a count.
2. **Use an index to jump from the cursor.** The next request uses `WHERE (created_at, id) < (cursor values)`. That lets the database jump directly to that spot through an index (a sorted lookup structure, like a book's index, that finds rows without reading the whole table), taking the same time at any depth. *Why:* no rows get scanned and thrown away.
3. **Tie progress to a row.** New rows added before that row can't move it, so there's no drift.

The diagram below compares the two. Offset scans and throws away 100,020 rows. The cursor does an index seek and reads just 20.

Offset scans and discards; cursor seeks



Three correctness rules:

- **You must have a tiebreaker column.** Sorting by `created_at` alone breaks when bulk imports or busy seconds create identical timestamps. "After timestamp T" then repeats or skips the other rows that share T. Add a unique tiebreaker — `(created_at, id)` — and store both values in the cursor.
- **Opaque really means opaque.** Encode the cursor and document it only as "a token". If clients start reading or building it themselves, your internal sort key turns into a permanent public contract (which is a versioning problem).
- **A stable sort backed by an index.** The order must always come out the same way and must be supported by an index. If not, the database still scans, even though you're using cursor syntax.

Trade-offs

You gain:

- **The same cost at any depth** — page 5,000 costs as much as page 1.

- **No drift** — no duplicates or skips while the list changes. For infinite scroll, this is a correctness requirement.
- **Works well with replicas** — cheap index-seek reads are a good fit for **read replicas** (read-only copies of the database).

You pay:

- **No random access.** You get next and previous, but not "jump to page 47". You can still jump *by value*, though (e.g., "start from last Tuesday").
- **Totals are awkward.** "Page 3 of 812" needs a separate `COUNT(*)`, which can be expensive. Big products often use infinite scroll or approximate counts instead.
- **Cursor lifecycle.** Changing the sort breaks existing tokens, and every sort order you support needs its own indexed cursor design.

When not to use it

- **Small, shallow UIs where people jump between pages** — a **200-row** admin table where users click "page 4" works fine with offsets. Drift and depth are too small to matter.
- **When the product really needs numbered pages** over a small dataset — use offsets, with an explicit **depth cap** (even large search engines cap how deep you can page).

Common misuse: giving partners offset APIs. They later build deep exports that lock you into linear database work forever.

Offset vs. cursor

	Offset (`?page=47`)	**Cursor (`?after=eyJ0cy...`)**
Cost at depth	Linear — scan and throw away	Constant — index seek
List changes while scrolling	Duplicates and skips	Unaffected — tied to a row
Random page jumps	Yes — its single advantage	No — next/prev only (plus seeking by value)
"Page 3 of 812"	Comes naturally	Awkward (needs a separate count)
Fits	Small, shallow admin UIs where users jump between pages	Feeds, APIs, exports, infinite scroll — the default

Real-world examples

Cursor pagination is the standard choice for feeds, public and partner APIs, exports, bank transaction histories, and infinite scroll. (The course uses these as its use cases rather than naming specific products.)

In the interview

Questions about infinite scroll check whether you spot drift. "A partner's export times out on page 5,000" checks whether you know the cost of deep pages. Name both failures first.

> **30-second answer:** "Offset pagination breaks in two ways at scale. OFFSET N forces the database to count off and throw away N rows, so deep pages get slower in a straight line. And page numbers are positions in a moving list, so inserts during scrolling cause duplicates and skips. Cursors fix both. The response carries an opaque token holding the last row's sort values. The next request seeks with WHERE (created_at, id) < cursor over an index — constant time at any depth, and tied to a row the moving list can't shift. Three details matter: a unique tiebreaker, because timestamps collide; opaque tokens, so my sort key doesn't become a

public contract; and accepting that there's no jump-to-page-47 — which feeds never needed anyway."

Likely follow-ups (with answers):

- *"Why isn't `created_at` enough by itself?"* Two rows can have the same timestamp. So seeking strictly after T can repeat or skip the other rows tied at T. The composite `(created_at, id)` gives every row its own unique position.
- *"The client wants to start from last week, not from the top."* That's a seek by value: build the starting cursor from the date. Only page jumps *based on counts* go away.

Mistake that fails candidates: using offsets for a large feed or partner API; building cursors from timestamps only, with no tiebreaker; or publishing a cursor format that clients start picking apart.

Self-check

- **Q1: Name the two ways offsets fail and the one change that fixes both.** → **Drift** (inserts and deletes move rows across page boundaries → duplicates and skips) and **deep-page cost** (OFFSET N scans and discards N rows → linear work). A cursor ties progress to the last row's sort values. That allows an index seek to a position that earlier inserts can't disturb.
- **Q2: Which pagination for (a) a bank transaction-history API, (b) a 200-row internal config table, (c) a social feed that's updated constantly?** → (a) Cursor — partners walk through a permanent, growing list where correctness matters. (b) Offset — 200 rows and real page navigation don't justify anything more. (c) Cursor — the constant writes would cause visible offset drift.
- **Q3: A partner's nightly cursor export misses ~2% of orders, bunched up in busy seconds. The cursor holds only `created_at`. Diagnose it, and fix both the query and the token.** → Identical timestamps collide, and "strictly after this timestamp" can never reach the other rows that share it. The bunching in busy seconds is the giveaway. Query with the composite tuple, `WHERE (created_at, id) > (:ts, :id)`, and put both values in the cursor. Since the token was opaque, its format can change without breaking partners. They just rerun the export to pick up the missing rows.

Related patterns

- **API Gateway** — the public pagination contract can be documented and applied the same way everywhere, at the entrance.
- **API Versioning** — opaque tokens let the internal sort key change without a change to the public format.
- **Primary-Replica** — cheap indexed cursor reads are a good match for read replicas.

TL;DR

Problem	Offset pagination scans and throws away rows to reach deep pages (getting linearly slower), and it points at positions in a moving list (causing duplicates and skips while scrolling).
Mechanism	An opaque token holding the last row's sort values. The next page is an index seek, `WHERE (sort_key, tiebreaker) < cursor` — the same cost at any depth, tied to a row that inserts can't move.
Costs	No jump-to-page-N (seeking by value takes its place), awkward totals, and a cursor lifecycle tied to a stable, indexed, unique sort key.
Skip it when	You have small, shallow UIs where users jump between pages — offsets with a depth cap are fine there.

API Versioning

Original lesson: API Versioning on DesignGurus ↗

What it is. API versioning is the practice of changing a published API without breaking the clients that rely on it. It follows three rules. **Make additive changes whenever you can. When you can't, create an explicit, coarse-grained version. Retire old versions through a managed lifecycle.**

> **Analogy.** Think of a city renaming a street. It can't just swap the sign overnight, because every map, GPS unit, and delivery driver still uses the old name. So the city puts the new name up next to the old one. It announces a date and mails out letters. Now and then it puts up "old name ending soon" detour signs. Only after all that does it take the old sign down.

The problem: a rename that broke 200 partners

A team renames `order.state` to `order.status`, because "state" confused people. Within an hour, **200 partner integrations** report that they can't parse responses. On top of that, an **18-month-old mobile build** shows empty order screens. That build is still used by **one-third** of users, and they can't be forced to update.

Once an API is published, you can't take it back. Clients you can't see, can't update, and can't schedule have built on every field you ever released. Each of them keeps running your old design decisions on its own release schedule. (It's the same idea as the pub/sub lesson's rule about public schemas and the BFF lesson's "partner APIs are products".)

Why the obvious responses fail

- **"Never change anything."** Every early mistake becomes permanent. The confusing `state` name sticks around forever.
- **"Announce a migration deadline."** This assumes you control partners' engineering schedules and which app version users have installed. You don't.
- **"Break it and say sorry."** This burns the platform trust that integrations rely on. Twitter's v1→v2 move showed what that costs an ecosystem.

How it works — three rules

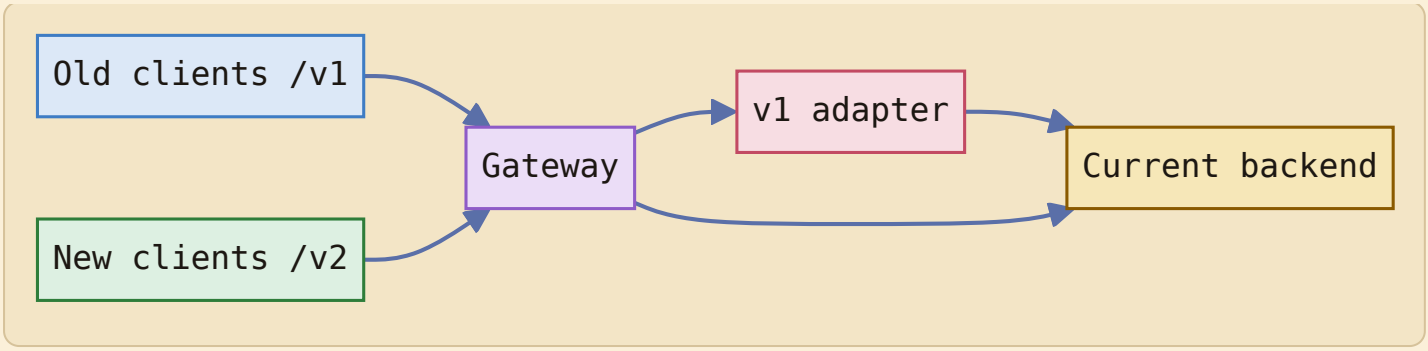
Rule 1 — additive first: most changes need no version at all.

- Add fields, endpoints, and optional parameters that have safe defaults. **Never remove, rename, or change the type** of anything that already exists.
- Clients should be **tolerant readers**, meaning they ignore fields they don't recognize.
- *Why:* additive servers plus tolerant clients make most changes invisible to existing integrations. (It's pub/sub's additive-only rule, applied to REST — the common style of HTTP APIs built around URLs for resources and verbs like GET and POST.)
- The honest cost is clutter. `status` sits next to `state`, both documented, with one marked as preferred, until the next major version allows a cleanup.

Rule 2 — when a breaking change can't be avoided, version explicitly and coarsely.

- The rename goes out in `/v2/`, while `/v1/` keeps returning the old shape.
- Version the **whole API or resource**, never single endpoints. Mixes like `/orders/v3` calling `/items/v5` create a testing matrix no one can manage.
- The gateway sends `/v1` through a thin **adapter** that translates old requests and responses onto today's internals. That gives you **one backend, two interfaces, and no forked implementation.** The diagram below shows old `/v1` clients passing through the v1 adapter, while new `/v2` clients go straight to the same current backend.

One backend, two interfaces



Where the version goes:

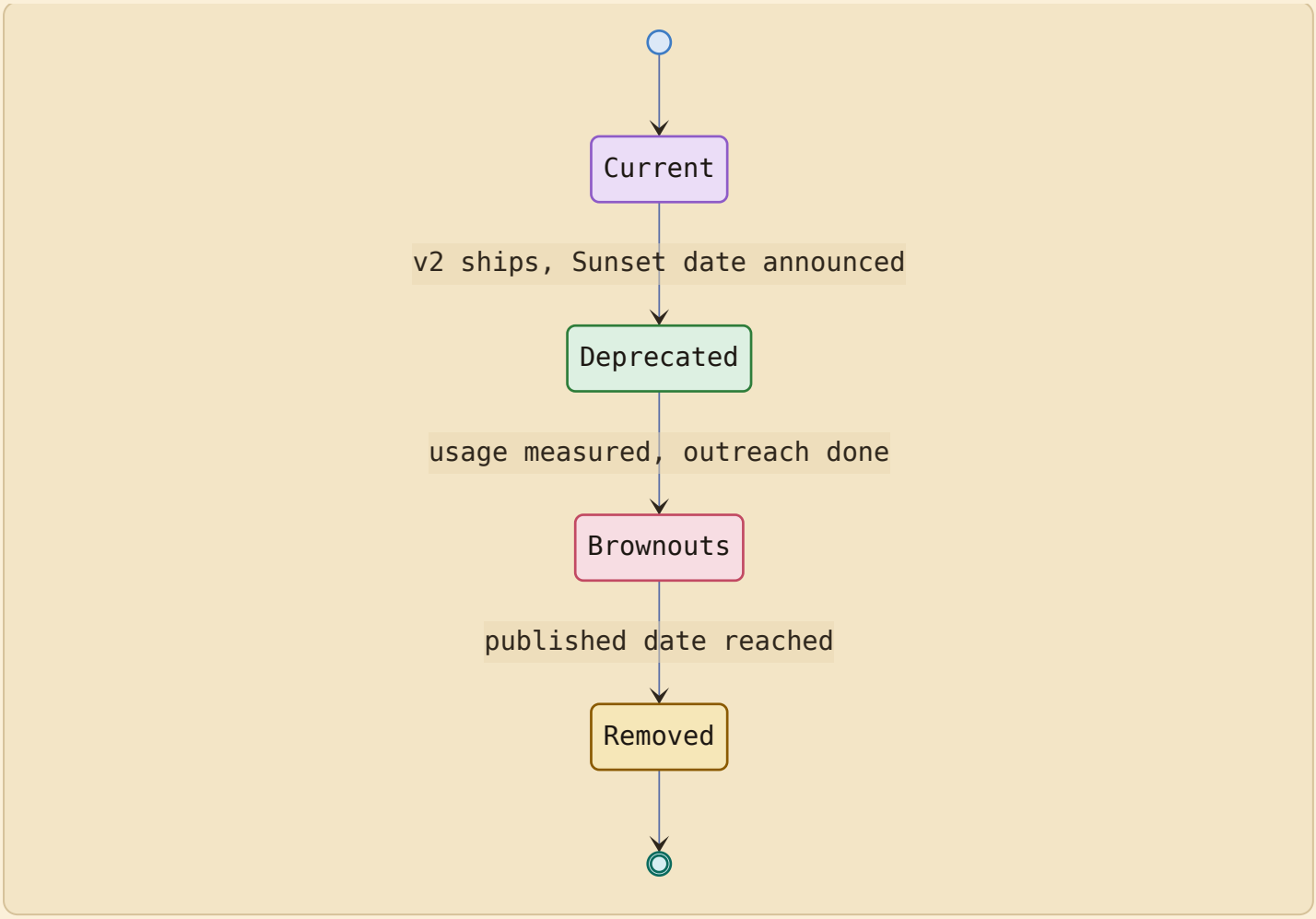
	URL path (`/v2/orders`)	**Header (`Accept: ...v2+json`)**
Visibility	Shows up in every log, link, and curl (command-line HTTP tool) command	Hidden — cleaner in theory, harder to debug
Caching/routing	Easy (the URLs are different)	Needs caches and routes that understand headers
Adoption reality	The practical default	Favored by a minority of REST purists

Putting the version in the path is the practical default. But **staying consistent matters more than where the version goes**.

Rule 3 — versions retire on a lifecycle, or they live forever. Each live version multiplies testing, documentation, security work, and support. The steps to retire one:

1. Announce a deadline with a **specific date**.
2. Send machine-readable `Deprecation` and `Sunset` headers (the Sunset header carries the removal date).
3. Track usage **per client and per version** on a dashboard.
4. Reach out directly to whoever is still using it.
5. Schedule **brownouts**. These are announced, temporary outages of the old version (e.g., **15 minutes of HTTP 410 Gone**). They flush out integrations whose owners ignore email.
6. Remove it on the published date. Keep **no more than two live versions**: the current one and the one being retired. The state diagram below shows this lifecycle.

Version lifecycle



Worked classification (additive vs. breaking)

- Add `order.estimated_delivery` → **additive** (tolerant readers just ignore it).
- Rename `state` → `status` → **breaking** (the old name goes away). The safe way: add `status` next to `state`, and remove `state` only in a major version.
- Make a request field that used to be optional required → **breaking** (requests that used to be valid now fail).
- Add an endpoint → **additive** (old clients never call it).
- Change `amount` from integer cents to a decimal string → **breaking** (the type and meaning change under the same name).

Trade-offs

You gain:

- Change without breaking the ecosystem — the rename ships, old apps keep working, and trust builds up over time.
- Mistakes you can fix — design errors get a planned way out instead of living forever.
- Retirement that machines can read — Sunset headers and brownouts push migrations along without endless meetings.

You pay:

- **Maintaining N versions** — each live version multiplies tests, docs, security, and support. The two-version limit is a budget.
- **Adapter decay** — translation code is real code, and it quietly rots.
- **Additive clutter** — duplicate fields pile up between major versions.

- **Sunsetting is organizational work** — dashboards, partner communication, and brownout scheduling reach beyond engineering.

When not to use it

- **Internal APIs with coordinated deploys** — when you own both the caller and the service, update them together and use **contract tests**. (These are automated tests that check both sides stick to the shared interface.)
- **Pre-launch and beta surfaces** — label instability clearly (e.g., `/beta/`) and keep the freedom to change. Long-term promises start at **GA** (general availability).
- **Event schemas** — same law, different transport. Use pub/sub's schema registries (central stores of message formats), additive rules, and versioned topics (named channels that messages are published to).

Common misuse: creating `/v3` because editing `/v2` feels risky, while `/v1` never gets retired. You end up with three permanent, half-maintained versions that slow down every change.

Real-world examples

- **Twitter's v1→v2** migration — the warning story of breaking changes eating up ecosystem trust.
- The techniques themselves (Sunset/Deprecation headers, brownouts with 410 Gone, gateway adapters) are the industry-standard retirement toolkit the course describes.

In the interview

A direct question: "How do you change an API without breaking a mobile app you can't update?" A tougher follow-up: "How do you remove v1 two years after v2?" That one checks whether your design includes retiring versions, not just creating them.

> **30-second answer:** "I go additive first: new fields and endpoints, never renames or removals, and clients act as tolerant readers. That handles most changes without a version bump. When a breaking change is truly needed, I version coarsely: /v2 at the gateway, routed through a thin adapter onto the current internals. Never a forked backend, and never a jumble of per-endpoint versions. And versions retire on a lifecycle, or they live forever. That means Sunset and Deprecation headers, a per-client usage dashboard, direct outreach, then scheduled brownouts — fifteen announced minutes of failures that wake up integrations that don't read email — and then removal. At most two live versions, because each extra one multiplies tests, docs, and support."

Likely follow-ups (with answers):

- *"Version in the URL or in a header?"* In the path, for practical reasons: it shows up in links and logs, and it's easy to cache and route. Headers are cleaner in REST theory but harder to debug. Being consistent matters more than which one you pick.
- *"How do you know v1 can really be removed?"* Measure traffic per client and per version instead of guessing. Use machine-readable headers, direct outreach, and brownouts to move the remaining consumers or bring them to light. Call the last few directly. Then remove it based on what the data shows, not on hope.

Mistake that fails candidates: promising "we'll be careful" instead of offering a real compatibility mechanism; versioning endpoints separately; or creating versions with no dated process for retiring them.

Self-check

- **Q1: Additive or breaking? (a) add `order.estimated_delivery` (b) rename `state`→`status` (c) require a previously optional request field (d) add an endpoint (e) change `amount` from integer cents to a decimal string.** → (a) additive. (b) breaking — first add `status` next to `state`, then remove `state` in a major version. (c) breaking — old requests become invalid. (d) additive. (e) breaking — the type and meaning change under an existing name.
- **Q2: What versioning policy fits an internal service API, a public partner API, and the mobile BFF — and why are they different?** → Internal: coordinated deploys plus contract tests, with no public versioning. Partner: coarse versions, Sunset headers, measured usage, and brownouts, because partners run on their own schedules. Mobile BFF: quick additive changes by the client team, with major versions tied to a **minimum-supported app build**, because old installed builds are consumers you can't control. The less control you have over deploys, the stronger the compatibility process has to be.
- **Q3: `/v1` was deprecated three years ago, still gets 30% of traffic, and nobody knows who's using it. List the retirement steps in order, and name the original mistake.** → Instrument usage per client (turning the unknown into a named list) → add `Deprecation` and `Sunset` headers with a real date → contact consumers, biggest first → schedule announced 15-minute 410 brownouts, repeating and escalating them so dormant integrations show up → call the final few → remove it on the published date. The original mistake was declaring "deprecated" with no measurement and no sunset date. That turned "deprecated" into "permanent".

Related patterns

- **API Gateway** — where version routes and compatibility adapters live.
- **BFF** — a BFF facing partners is a public API product, governed by these lifecycle rules.
- **Publish-Subscribe** — event schemas follow the same permanent-contract rule, using registries and additive changes.
- **Cursor Pagination** — opaque cursors reveal less of your internal structure, so they need fewer version changes.

TL;DR

Problem	Published APIs are used by clients you can't see, update, or schedule. A well-intended rename breaks 200 partners and every old mobile build all at once.
Mechanism	Additive-first changes (add, but never rename/remove/retype; tolerant readers). Coarse, explicit versions through gateway adapters when a break can't be avoided. Lifecycle retirement (Sunset headers, usage dashboards, outreach, brownouts, removal) — two live versions at most.
Costs	Maintaining N versions, adapters that rot, additive clutter, and sunset work that's organizational.
Skip it when	You control both sides (coordinated deploys + contract tests), the surface is labeled beta, or it's an event schema (use pub/sub's playbook).

Sidecar

Original lesson: Sidecar on DesignGurus ↗

What it is. A sidecar is a small helper process that runs **next to every service instance**, on the same machine. The service talks to it in plain text over **localhost** (the machine's own internal loopback network). The sidecar handles the networking work toward the outside world — TLS, retries, timeouts, metrics, and tracing. That leaves the service to do only its own job.

Analogy. Picture a motorcycle sidecar. Or picture a personal assistant seated beside each executive. The executive just says "call inventory". The assistant takes care of dialing, security

checks, retrying busy lines, and logging the call. Every assistant does it the same way for every executive, no matter what language the executive speaks.

The problem: east-west traffic has hundreds of entrances

The module now switches to **east-west traffic** — direct calls between internal services. These calls need the same consistent handling as public requests:

- **Mutual TLS (mTLS)** — encrypted connections where *both* sides prove who they are. (Normal TLS only proves the server's identity.)
- Timeouts, retries, standard metrics, and **trace-header propagation** — passing the trace ID along so distributed tracing can link the calls together.

Doing all this with application libraries across **11 services in 3 languages** gives you 11 versions of retry behavior. It's the policy drift from the gateway lesson, now showing up east-west. A gateway can't help here. It centralizes public policy because all external traffic passes through one entrance. Internal calls, rightly, go straight between hundreds of instances.

Why the obvious fixes fail

- **"Use a shared library."** You need a separate version for each language. Upgrades have to wait on 11 independent deploy schedules. The copies are bound to drift apart.
- **"Use one central internal proxy."** That gives one place for policy. But it forces every internal call through an extra hop and a single point of failure. And the gateway's own rules forbid east-west detours like that.

The sidecar turns the placement around: **if you can't bring all the traffic to one proxy, bring a proxy to every instance.**

How it works

In the course figure below, a Go Checkout instance and a Python Inventory instance each run their business logic. Each one talks in plain text over localhost to its own sidecar ("same binary" — the identical compiled program). The only hop that leaves the machine is sidecar-to-sidecar mutual TLS. That's where timeouts, retries, circuit breaking, metrics, and traces happen. The note at the bottom works out the fleet math: 11 services x 40 instances = 440 sidecars, ~50 MB each, so ~22 GB of memory goes to networking.

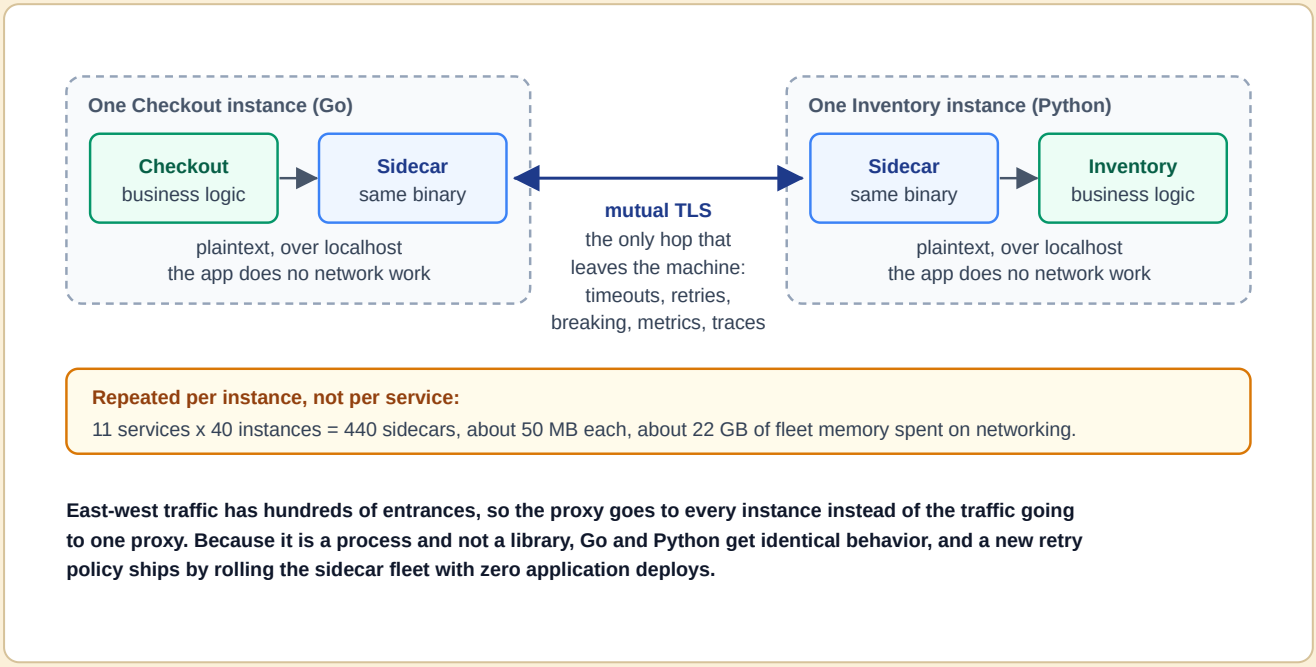


Figure — Sidecar Pattern (course diagram)

1. **Run one helper process next to each service instance.** The app sends plain requests to localhost. The sidecar adds TLS, timeouts, retries, and telemetry (automatically collected metrics, logs and traces) on the network side. *Why:* the networking logic lives outside the app.

2. **It works with any language by design.** The sidecar is a separate process, not a library linked into the app. So Python, Go, and Java services all run the exact same binary. There are no parallel implementations to keep in step.
3. **Policy upgrades move at infrastructure speed.** A new retry policy, timeout, or TLS certificate reaches every service just by rolling out the sidecar fleet — with **zero application deploys**. Networking gets its own lifecycle, instead of waiting on 11 release schedules.
4. **Application code gets smaller.** The mechanics of timeouts, retries, and **circuit breakers** (from the reliability module) move into infrastructure. Services keep only the *domain* decisions — for example, which reduced response to send back when a dependency fails.
5. **The wider family.** "Sidecar" means any helper that runs alongside a service — log collectors, secret fetchers (helpers that load passwords and keys from a secure store), config reloaders. But *proxy* sidecars became the most influential kind, because they standardized how services talk over the network. A proxy sidecar is the reverse-proxy primitive, deployed once per instance.
6. **A sidecar is a building block, not the whole system.** Hundreds of proxies still need certificates, routes, and policies from somewhere. The service mesh (next lesson) supplies that coordination. The sidecar does the work; the mesh decides its configuration and hands it out.

Roll it out bit by bit. Start with one service that isn't critical. Measure memory and latency, and check that startup and shutdown happen in the right order. Then expand one group at a time. That way you separate proxy bugs from application bugs before the whole fleet relies on it.

Walkthrough with numbers

- **Memory:** **~50 MB** plus a bit of CPU per sidecar seems cheap — until you multiply it out. **11 services x 40 instances = 440 sidecars ≈ 22 GB**. Always multiply per-instance costs by the size of the fleet.
- **Latency:** each sidecar adds **~0.5 ms**, and every internal call goes through two of them (the sender's outbound one and the receiver's inbound one). That's ≈ **1 ms** per call. Measured p99 might go up **~2 ms** once TLS is included. That's fine for most services, but not for microsecond budgets.
- **Lifecycle bug:** if the app is ready before its sidecar, its first outgoing calls fail after every deploy. The fix is a **readiness gate** / "hold application until proxy started", so the app waits for the sidecar. On shutdown, stop the app first and keep the sidecar running so it can finish in-flight requests.
- **Crash blast radius:** if one sidecar crashes, only that instance loses networking. It fails its health checks, is taken out of load-balancer rotation, and restarts. Just one instance is affected — compared with a central proxy that would take everything down.

Trade-offs

You gain:

- The same networking behavior in every language and framework, upgraded across the fleet without app deploys.
- mTLS everywhere in a single rollout — service-to-service encryption without changing any codebase.
- Slimmer services — shared mechanics move out of the application layer.

You pay:

- **A per-instance cost multiplied by everything** (the 22 GB above).
- **Latency on each hop** (~1 ms per call).
- **A new lifecycle to get right** — startup order, sidecars crashing on their own, and **version skew** (different sidecar versions across the fleet partway through an upgrade).
- **One more suspect when debugging** — in every network incident, you have to tell app behavior apart from proxy behavior.

When not to use it

- **Small fleets using one language.** Five Go services owned by a single team can use one well-kept library and skip paying 50 MB per instance.
- **Latency budgets measured in microseconds** (e.g., trading) — two extra process hops may be too much.
- **Serverless** — there's no long-running instance for a sidecar to sit next to, so the platform has to provide equivalent controls.
- **No platform team** — hundreds of injected processes, upgrades, and new ways to fail all need an owner. Newer **sidecar-less** options do the same jobs without a process per pod (a pod is Kubernetes' smallest deployable unit: one or more containers run together). They use **eBPF** (programs that run inside the operating-system kernel) or **ambient**, node-level (one per machine) proxies. The jobs still exist even when the separate process goes away.

Common misuse: adopting sidecars for ten services just because big companies do — even though there's one team, two languages, and little drift. You pay all the cost and get none of the benefit that comes with organizational scale. Revisit the choice as the number of teams and languages changes.

Sidecar vs. shared library

	Shared library	Sidecar
Multi-language coverage	One implementation for each language, drifting apart	One binary, the same everywhere
Upgrade path	Has to wait on N teams' deploy schedules	Across the fleet in minutes, with no app deploys
Latency	Zero — runs in-process	~1 ms per call (two hops)
Memory	Shared with the app	~50 MB x every instance
Fits	One language, few services, tight budgets	Many languages, many services, drift problems

The deciding factor is organizational. Libraries fall apart when many languages and teams must keep behavior matched on different schedules. Below that point, a library is lighter, faster, and good enough.

Real-world examples

Here the course names members of the family rather than products. Proxy sidecars are the Envoy-style proxies that meshes use (covered next). Log collectors, secret fetchers, and config reloaders are non-proxy sidecars. eBPF and ambient node-level modes are the sidecar-less alternatives.

In the interview

Sidecars usually come up as the first half of a service-mesh question. Or as: "How do you roll out mTLS across 11 services in 3 languages without touching every codebase?"

> **30-second answer:** "A sidecar is a proxy process that runs next to every service instance. The app talks plain text over localhost, and the sidecar handles mutual TLS, timeouts, retries, metrics, and trace propagation. Since it's a process rather than a library, every language gets exactly the same behavior. Policy changes roll out by upgrading the sidecar fleet — no app deploys and no drift. The costs I'd size first: roughly 50 MB and half a millisecond per instance, two hops per call, multiplied across the fleet. Startup ordering and sidecar crashes are real failure modes too. And a sidecar by itself is just the building block. Coordinating hundreds of them is the control plane's job — that's the service mesh."

Likely follow-ups (with answers):

- *"Why not just use a shared library?"* Several languages need several implementations, and separate team schedules cause version drift. A TLS or security change can reach every sidecar in minutes rather than waiting for app releases. Below that organizational tipping point, though, a library is still better.
- *"What happens when the sidecar crashes?"* That instance loses network communication. It fails health checks, and the load balancer pulls it from rotation until it restarts. Only one instance is affected — unlike a central proxy failure, which disrupts every service.

Mistake that fails candidates: treating one sidecar and a full service mesh as the same thing; or listing benefits without multiplying resource costs across the fleet.

Self-check

- **Q1: What moves out of app code into a sidecar, and which two abilities does a separate process have that a library doesn't?** → mTLS certificates and handshakes, timeouts, retries with backoff, circuit-breaking mechanics, metrics, and trace-header propagation. A process is **language-agnostic** (one binary for every stack) and can be **upgraded independently** of app deploys.
- **Q2: Fleet A has 5 Go services and one team. Fleet B has 30 services, 4 languages, and 9 teams. Library or sidecar?** → A: a shared library. One language and one team mean no implementation drift and no deploy drift, so sidecars would add memory, latency, and lifecycle complexity for nothing. B: sidecars. Four implementations across nine release schedules would drift apart. The deciding factor is how many independent language x team combinations have to stay consistent.
- **Q3: After adopting sidecars, internal p99 rises ~2 ms, and some pods fail their first requests after every deploy before recovering. Explain both. Which is a cost and which is a defect?** → The p99 increase is the **documented cost** of passing through two sidecars plus TLS — include it in capacity estimates. The first-request failures are a **defect**: a startup race where the app becomes ready before its proxy. Make app readiness wait on the sidecar (readiness gates / hold-until-proxy-started). On shutdown, stop the app first and let the sidecar drain.

Related patterns

- **Reverse Proxy** — the sidecar is this primitive at its smallest useful size, one per instance.
- **Service Mesh** — adds a control plane to configure hundreds of sidecars (next lesson).
- **API Gateway** — the north-south counterpart: one shared entrance rather than many local proxies.
- **Timeout / Retry** — the reliability mechanics that move out of each codebase and into the helper.

TL;DR

Problem	East-west traffic needs mTLS, retries, timeouts, and tracing. But it has hundreds of entrances and no single choke point, so libraries written per language drift apart at each team's deploy pace.
Mechanism	A proxy process next to every instance, reached over localhost. Apps speak plain text locally; sidecars do the networking the same way in every language, upgraded fleet-wide with no app deploys.
Costs	~50 MB and ~1 ms per call, multiplied across the fleet; startup/shutdown ordering; crashes of their own; and one more suspect when debugging.
Skip it when	Small fleets in one language (the library wins), microsecond budgets, serverless, or no one can own hundreds of new processes.

Service Mesh

Original lesson: Service Mesh on DesignGurus ↗

What it is. A service mesh gives a fleet of sidecars a central brain. It's divided into two **planes**. The **data plane** is the sidecars themselves: they carry and police the traffic on every hop. The **control plane** is a central coordinator. It keeps track of what's running, works out each sidecar's configuration (routes, certificates, policies), and keeps pushing updates to them over long-lived streams.

> **Analogy.** Think of air traffic control. The planes (sidecars) do the actual flying and follow the rules on every flight. The control tower (control plane) carries no passengers. But it knows where every plane is, gives out clearances and routes, and changes instructions in real time. If the tower's radio goes silent for a moment, planes keep flying on their last clearance. They just can't get new ones.

The problem: 440 sidecars, no shared brain

The sidecar rollout was a success: **440 proxies** next to 440 instances, all with consistent networking. Six months on, the trouble is keeping their configuration coordinated:

- **Certificates.** mTLS means every sidecar holds a certificate, and certificates expire. Rotating **440 certificates every 90 days** by hand is a chore that never ends. Miss one quarter, and at **2 AM** checkout can't call inventory — `certificate expired`. That's an outage caused by routine upkeep.
- **Policy drift, just moved somewhere else.** One engineer adjusts a service's retries during an incident and never undoes it. Another team is still on an old timeout. Eleven drifting libraries have turned into **440 drifting config files**.
- **Routing keeps changing.** Deploys and autoscaling change instance IP addresses every few seconds, and static files can't keep up.
- **Security's question has no answer.** There's no way to say "Only checkout and refunds may call payments". Right now, being reachable on the network *is* the permission.

Sidecars standardized *how* traffic moves. They didn't standardize the *knowledge* they need: which workloads exist, who is allowed to talk to whom, routes, policies, and certificates.

Why the obvious fixes fail

- **"Use config management — template the files in CI."** (CI, continuous integration, is the automated system that builds and tests every code change.) Templates handle static settings. But deploy pipelines take minutes, while IP addresses change in seconds. Routing is live state, and it needs to be delivered continuously.
- **"Use DNS for discovery."** DNS answers are cached for their **TTL**, so callers might keep hitting removed instances for minutes. (It's the load-balancer lesson's caching problem, now east-west.) DNS also only tells you location. It carries no health info, no identity certificates, and no authorization policy.
- **"Write a cert-rotation runbook."** (A runbook is a written step-by-step procedure for operators.) 440 certificates every 90 days has to be done perfectly, forever. One slip means an overnight outage. Routine decisions like this need automation that runs at machine speed.

How it works

In the course figure below, the control plane sits at the top. It watches the registry, issues and rotates certificates, and holds the policy. It pushes configuration and certificates down over long-lived **xDS** streams to the sidecar proxies next to Checkout, Inventory, and Payments (x 440). Those proxies talk to each other over mTLS. If the control plane goes silent, the sidecars keep serving with their last configuration — changes freeze, but traffic doesn't.

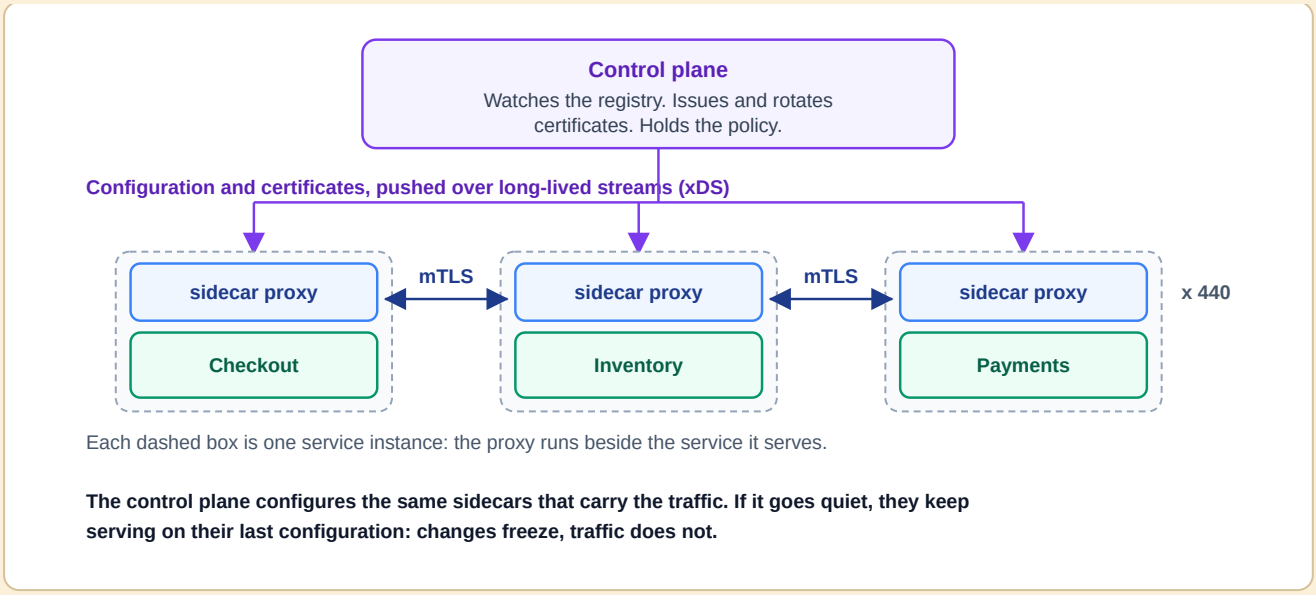


Figure — Service mesh: data plane and control plane (course diagram)

1. **Certificates take care of themselves.** The control plane acts as a **certificate authority (CA)** — the trusted issuer of identity certificates. It hands every workload a cryptographic **workload identity** (e.g., "checkout, in prod"), using **short-lived certificates that rotate automatically** (e.g., hourly). *Why:* outages from expired certificates become very unlikely by design. Also, services now trust each other based on *identity*, not *network location*. That's the foundation of **zero-trust networking**, where being inside the network doesn't earn you any automatic trust.
2. **Routing at platform speed.** The control plane watches the orchestrator's (the system that schedules containers onto machines) live registry (e.g., the **Kubernetes API server**). When a deploy changes IPs, it **streams** new routes to the affected sidecars within seconds. It uses **Envoy's xDS** protocol, which provides long-lived, typed streams for machine configuration. (This resolves the forward reference from the bidirectional-streaming lesson.)
3. **Policy becomes declarative and central.** **Declarative** means you describe the result you want, not the steps to get there. Retry budgets, timeouts, and circuit breaking (which many meshes call **outlier detection**) become versioned config objects. They can apply globally or to one service. Sidecars no longer keep their own local settings; the control plane keeps feeding them.
4. **Authorization finally has somewhere to be written.** A single policy object — "only checkout and refunds may call payments" — is enforced by the *receiving* payments sidecar. It checks the caller's cryptographic identity and logs every allow and deny. Access control between internal services becomes explicit and auditable.
5. **Observability comes free.** Every proxy sees every request. So it reports uniform rates, errors, latency, and trace spans for each pair of services — this baseline needs no app instrumentation. The **dependency graph** becomes visible right away.
6. **Traffic control is just configuration.** A small block of config can send **5%** of inventory calls to inventory-v2 (a **canary** — a small real-world test of a new version). It can **mirror** real traffic to a test service, or **inject faults** during planned drills. None of this needs code changes.
7. **The data plane fails static.** If the control plane goes down, sidecars carry on with their last configuration. Existing traffic keeps flowing, but new routes, certificates, and policies stop being updated. *Still*, the control plane isn't optional. New instances (from deploys or autoscaling) may get no routes or identity. Existing certificates also keep counting down toward expiry. The system stays stable in the short term while slowly losing its ability to change. So set control-plane recovery targets that fit inside that window.

Walkthrough with numbers (the four incident problems, under a mesh)

- **Certs:** identities rotate **hourly** across all 440 proxies, automatically. A risky manual chore turns into background automation, and you get zero-trust identity on top.
- **Canary:** one policy sends **5%** of traffic to inventory-v2 and can be undone **within seconds**. Before, every caller needed a code change or a special balancer. (The next module, on operations, depends on this.)

- **Security request:** one authorization policy at the receiving sidecar, logging every allowed and denied call by workload identity.
- **Dependency map:** dashboards instantly show traffic, errors, and **p99** (the latency that 99% of requests beat) for every pair of services. They often uncover call paths nobody documented.
- **The bill:** you still pay the sidecar lesson's **~22 GB** of fleet memory and **~1 ms** per call. On top of that, you have a distributed control plane that must be **HA** (highly available — still usable when parts of it fail). A platform team has to run it as an internal product. That means safe defaults, migration help, documentation, and a way to request exceptions. App teams then don't need to know every proxy setting, but they do understand the policies that affect them.

Trade-offs

You gain:

- **Zero-trust mTLS with workload identity, rotated automatically** — this security upgrade alone usually justifies the whole project.
- **Observability across the fleet without instrumentation** — the dependency graph, out in the open.
- **Traffic control through settings** — canaries, mirroring, and fault injection with no code changes.
- **Central policy that can't drift** — the incident's problems fixed at the layer where they ended up.

You pay:

- **A distributed system to manage your distributed system** — it needs high availability, careful upgrades, and specialist owners. A mesh is a product run by a platform team, not a config switch.
- **All the sidecar costs you inherit** — memory, CPU, latency, and lifecycle complexity.
- **More layers to dig through when debugging** — app code, the sidecar, *and* central policy are all suspects.
- **A global blast radius** — one bad global policy reaches the entire fleet as fast as config spreads.
- **Config sprawl** — authorization, routing, and reliability objects multiply until governing policy becomes a discipline of its own.

Failure example — the global retry rule. Someone adds a mesh-wide rule, "retry every failed request 3x", hoping to boost reliability. Then finance reports **duplicate orders**. Failed order `POST`s (which are not **idempotent** — repeating them causes extra effects) were retried and ran several times. The mesh didn't create the unsafe write. But it spread a bad policy everywhere instantly. Guardrails: treat policy like code (review it, roll it out in stages, and canary it on one service first). By default, retries should skip non-idempotent methods. Turning on retries for writes should require audited **idempotency keys**.

When not to use it

- **Fewer than a few dozen services** — with 8 services in one language, a library plus a gateway is the better choice.
- **No platform team** — a mesh nobody owns means hundreds of identical proxies no one can explain. That's arguably worse than local drift.
- **Latency-critical paths** — central config doesn't take away the delay on each hop.
- **As the public front door** — meshes govern east-west traffic. External traffic still goes through an API gateway.

Typical misuse: adopting a mesh because big companies publish success stories, when you really have two services and one slow query. **The bar for adopting one (you need all four):** a fleet spanning several languages, several dozen services, an explicit zero-trust requirement, and a staffed platform team.

Start with **one measurable goal** (usually workload identity or consistent observability). Judge success by certificate reliability, how long policy rollouts take, how much the proxies consume, and incident response. That tells you whether coordination work is actually shrinking or just moving elsewhere.

The adoption steps

Step	What it is	What it buys	Right when
1. Shared libraries	A reliability kit inside the code	No infrastructure; in-process speed	One language, few services, one team
2. Ad-hoc sidecars	Proxies next to instances, configured by hand	The same mechanics everywhere, across languages	Several languages, drift starting, config still manageable by hand
3. Full mesh	Sidecars + control plane	Identity, policy, observability, and traffic settings — shared knowledge made uniform	Dozens of services, a zero-trust requirement, a staffed platform team

Move up a step once coordinating the current step becomes the bottleneck. Libraries break down across languages and team schedules. Hand-configured sidecars break down once there are too many certificates and policies. Sidecar-less meshes (eBPF, ambient node-level modes) keep the same control-plane duties while changing how it's built underneath.

Real-world examples

- **Istio** — Envoy proxies plus a full-featured control plane, with a sizable cost to operate. **Linkerd** — intentionally has fewer settings and is simpler to run.
- **Envoy's xDS** — the streamed-config protocol underneath. The same proxy can serve as gateway, sidecar, or load balancer (one primitive, several placements).
- **Istio ambient and Cilium (eBPF)** — sidecar-less approaches that keep the control plane while cutting the cost of a proxy per pod.
- **For AI engineers:** model canaries (send 5% of inference requests to model-v2 while measuring quality), **mirroring** real production questions to a candidate model without affecting users, and workload identity to limit who can call expensive GPU services.

In the interview

A direct prompt: "fifty services in four languages need consistent mTLS, retries, and canary routing." Follow-ups check control-plane failure, how it differs from the gateway, and whether your scale justifies a mesh.

> **30-second answer:** "A service mesh is split into a data plane and a control plane. The data plane is the sidecars that move traffic. The control plane watches the platform, issues workload-identity certificates and rotates them hourly, and pushes routing and policy to every sidecar over long-lived streams — Envoy's xDS. What you get: zero-trust mTLS without touching app code, metrics and traces across the whole fleet for free, and traffic control as configuration — canaries and mirroring are just settings. The property I'd mention first: the data plane fails static, so a control-plane outage freezes changes, not traffic. And the honest bar for adoption: dozens of services in several languages, a real zero-trust requirement, and a platform team to own it. Below that, a gateway plus libraries is the better engineering choice."

Likely follow-ups (with answers):

- *"The control plane goes down at peak — what breaks?"* Existing sidecars keep routing on their last known config, so current traffic stays up. New workloads can't get routes or certificates,

so a long outage blocks deploys and autoscaling. Changes freeze; established traffic keeps going.

- *"Why not run all of this through the API gateway?"* The gateway is a single north-south entrance that applies *user* identity and product quotas. A mesh spreads east-west *workload* identity, retries, and authorization across hundreds of service-to-service relationships. Pushing internal calls through the gateway would re-centralize traffic that the sidecars intentionally keep local.

Mistake that fails candidates: recommending a mesh for five services; misunderstanding fail-static; or treating the public gateway and the internal mesh as if they were interchangeable.

Self-check

- **Q1: Name the two planes and their jobs, and define fail-static precisely.** → The **data plane** is the sidecars that carry traffic and enforce mTLS, timeouts, retries, and policy on every hop. The **control plane** watches the workload registry, hands out identities, and streams the computed config to the proxies. **Fail static** means: if the control plane can't be reached, sidecars keep working with their last configuration. Traffic continues; configuration changes stop.
- **Q2: Fleet A has 8 services, one language, and no compliance need. Fleet B has 60 services, 5 languages, a zero-trust requirement, and a 6-person platform team. Where does each sit on the ladder, and what would move A up?** → A: step 1 (shared libraries). One language removes implementation drift, and nothing justifies mesh identity. It would move up if more languages, several dozen services, or a security mandate made coordination the bottleneck. B: step 3 (full mesh) — it has multi-language drift, scale, a zero-trust need, and a team to own it.
- **Q3: A global mesh rule retries every failed request 3x, and finance reports duplicate orders. Reconstruct what happened, assign responsibility, and design guardrails.** → Failed order `POST`s with no idempotency were retried, so a single purchase ran several times. The unsafe write was already there; the mesh *extended the reach* of a wrong policy. Process guardrails: policy changes go through review and staged rollout, and are canaried on one service first. Technical guardrails: default retries skip non-idempotent methods, and retrying writes requires audited idempotency keys. Central policy fixes drift quickly — and spreads mistakes just as quickly.

Related patterns

- **Sidecar** — supplies the data plane that this pattern configures.
- **API Gateway** — the north-south counterpart.
- **Circuit Breaker** — mesh outlier detection applies breaker behavior through central policy.
- **Distributed Tracing** — spans produced by the proxies are its raw material.
- **Bidirectional Streaming** — xDS uses long-lived machine-to-machine streams to deliver configuration.

TL;DR

Problem	Hundreds of sidecars gave uniform mechanics but not uniform knowledge. Certificates expire on human schedules, configs drift pod by pod, routing can't keep up with autoscaling, and "who may call payments" has nowhere to be written down.
Mechanism	Control plane + data plane. The coordinator watches the registry, issues workload identities rotated hourly, holds declarative policy, and streams each sidecar's config over xDS. Sidecars enforce it on every hop and fail static if the control plane goes quiet.
Costs	A distributed system to manage your distributed system (owned by a platform team), the inherited sidecar costs, harder debugging, and a global blast radius for bad policy (so govern policy like code).
Skip it when	You have fewer than a few dozen services, one language, no zero-trust mandate, or no team to own it — you need all four to clear the bar.

Authorization Service

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. **Authentication** (AuthN) answers *who is calling?* **Authorization** (AuthZ) answers *may this caller do this action on this particular thing?* An authorization service moves the second question out of every service's private code into one shared system. Each service still *asks* at the moment of access ("may user 42 view invoice 8812?"), but the rules and the relationship data they depend on live in one place, are evaluated the same way everywhere, and are logged. The best-known design is Google's **Zanzibar**, which stores permissions as a graph of relationships.

> **Analogy.** A hotel key-card system. The front desk checks your passport once (authentication); every door reader then asks the central system "may card 7731 open room 512 now?" (authorization). The rules — guests open their own room, housekeeping opens floor 5 until 4 PM — live in one system, not 400 separately programmed locks. Cancel a card at the desk and every door knows.

The problem: thirty services, thirty permission systems

The store has opened a **seller portal**: 200,000 seller organisations, 2 million staff accounts, teams inside each seller, and 500 million orders and invoices. Each of ~30 services wrote its own checks, usually a SQL join (a query combining rows from several tables) in the handler (the function that processes one kind of request). Four findings from one quarter:

- **The data leak.** `GET /invoices/8812` checked that the caller was *logged in*, not that the invoice belonged to the caller's organisation. A curious seller changed the number in the URL and read a competitor's invoices. This is the most common API vulnerability there is — **broken object-level authorization** (BOLA), number one on the OWASP API Security Top 10 (OWASP is a non-profit that publishes widely used lists of common security flaws).
- **The half-removed employee.** A seller removed a fired employee from their "finance" team. The payout service stopped showing him reports. The search service, which cached team membership for a day, kept showing him invoice titles.
- **The slow "what can I see?"** Sharing works through folders inside folders and teams inside teams. The documents page runs recursive queries per request and takes **2 seconds** for large sellers.
- **The unanswerable audit.** A large customer asks, "list everyone who can read our payout data, and why." No system can answer, because the answer is spread across thirty codebases.

The gateway authenticated every request (the capstone's Stage 2), and deliberately refused to know about invoices or teams. So object-level decisions fell to each service — and thirty separate implementations drifted apart.

Why the obvious fixes fail

- **"Put roles in the JWT."** (A JWT, JSON Web Token, is a signed token that carries claims about the user.) A token can carry coarse roles ("seller-admin"), but not "can view these 40,000 invoices". Tokens also stay valid until they expire, so a revoked permission lives on for the token's lifetime (the Spring book's token revocation section covers this trade-off).
- **"Let the API gateway decide."** The gateway would need to know what an invoice is, who owns it and how folders nest — exactly the domain knowledge a thin gateway must not have. It stays the place for coarse checks: authenticated, token scope (the list of APIs a token is allowed to call), rate limits.
- **"A shared permissions library" / "copy the roles table everywhere."** The *logic* becomes consistent, but each service reads its own copy of the *data*, going stale at thirty different speeds — the half-removed employee still happens, and a rule change means thirty redeploys.

How it works

1. Split enforcement from decision. Two roles, named in the NIST (the US standards institute) and XACML (a standard language for access-control policies) vocabulary:

- The **policy enforcement point (PEP)** is the code at the moment of access — a handler, a gateway filter, a sidecar. It builds the question, asks, and obeys the answer.
- The **policy decision point (PDP)** is the authorization service. It holds the rules and the facts, and answers `allow` or `deny` (plus a reason, for the logs).

Services keep enforcement (only they know that `/invoices/8812` means "invoice 8812"); the decision moves to one place.

2. Pick a permission model — usually more than one.

Model	Question it answers	Example rule	Breaks down when
ACL (access-control list)	Who is on this object's list?	invoice 8812: {alice: read}	Millions of objects each need their own list; nobody can review them
RBAC (role-based)	Does the user have a role that grants this permission?	"finance-viewer" may read invoices	Roles must be *per organisation, per team, per project* → **role explosion** (thousands of roles)
ABAC (attribute-based)	Do attributes of user, resource and context satisfy a policy?	allow if `user.org == invoice.org` and `user.mfa == true` and time is 08:00–20:00	Answering "who can see X?" means evaluating the policy against every user
ReBAC (relationship-based)	Is there a path of relationships from the user to the object?	user → member of team → viewer of folder → parent of invoice	Pure attribute rules (time of day, IP range) don't fit a graph

Real systems combine them: **RBAC** for coarse staff roles ("support agent"), **ReBAC** for ownership and sharing (organisations, teams, folders), and **ABAC conditions** on top (require MFA — multi-factor authentication, such as a password plus a phone code — for payouts, block access from unusual countries). The Spring book shows the in-process RBAC end of this spectrum: authorizing requests with roles and authorities and method security.

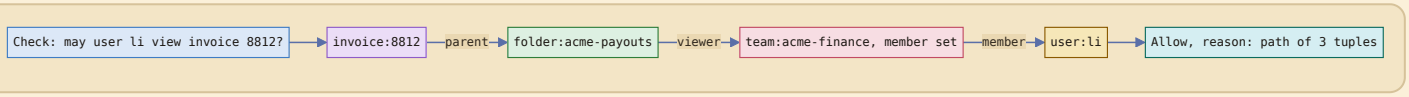
3. Zanzibar-style relationship tuples. Zanzibar stores every permission fact as a **relation tuple**: `object#relation@subject`. The subject can be a user, or a *set* of users described by another tuple:

```
org:acme#member@user:ravi
team:acme-finance#member@user:li
folder:acme-payouts#viewer@team:acme-finance#member   (every member of the team)
invoice:8812#parent@folder:acme-payouts
```

A per-type **schema** says how relations combine. For example: an invoice's *viewer* is anyone who is its direct viewer, *or* its editor (editors can view), *or* a viewer of its parent folder. The PDP then answers:

- **Check** — "is `user:li` a `viewer` of `invoice:8812`?" It walks the graph: invoice → parent folder → the folder's viewers include team acme-finance's members → li is a member → **allow**. Three hops.
- **Expand** — "show the full tree of who is a viewer of this folder, and why" — the audit answer.
- **Read / Write** tuples — the product writes a tuple when a user is added to a team or a folder is shared, and deletes it on removal. There is exactly one place where "li left the finance team" is recorded.
- **List objects** ("which invoices can li see?") — the reverse question. It is the expensive one; systems answer it with precomputed indexes (Zanzibar's *Leopard* index flattens deeply nested group membership) or by filtering search results (step 5).

One check, walked through the relationship graph



The check starts at the invoice and follows stored tuples: its parent folder grants *viewer* to the finance team's members, and li is a member, so the answer is allow — with the path itself as the audit reason.

4. Consistency: the "new enemy" problem. Authorization data changes, and a stale answer can be a security bug. The Zanzibar paper's example, translated: a seller removes Bob from the finance team at 10:00:00, then at 10:00:05 uploads a payout report to the finance folder. If the check reads a replica 10 seconds behind, Bob — the *new enemy* — reads content created *after* his removal. Zanzibar's fix is a **consistency token** (a *zookie*): saved with the content, it encodes "the permission data as of now", and checks on that content must use a snapshot at least that fresh. Old content can use cached, slightly stale snapshots; new content is guaranteed to see the removal. Open-source systems offer the same per-request choice ("fully consistent" vs "at least as fresh as this token").

5. Make it fast enough to sit on every request. Authorization is on the critical path of nearly every call, so:

- **Cache decisions and sub-results** close to the caller — "li is a member of acme-finance" is reused by thousands of checks. Keep cache lifetimes short, or tie them to consistency tokens.
- **Batch** — a page listing 50 invoices sends one bulk check, not 50 network calls.
- **Filter lists at the source.** *Pre-filter* (index each document with the principals — users, groups or services that can hold permissions — allowed to read it, updated from tuple changes) or *post-filter* (fetch a page, bulk-check, fetch more if items were dropped). Post-filtering suits small pages; pre-filtering is needed when most results would be dropped.
- **Run a local decision point** for pure policy rules: an ABAC engine such as Open Policy Agent (OPA) can run as a sidecar, with policies pushed to it, and answer in microseconds without a network hop.

6. Decide failure behaviour per action. If the authorization service is unreachable, most requests should **fail closed** (deny) — a leak is worse than an error. A few low-risk reads may **fail open** from cache for a short time. Write this down per endpoint, like the fail-open/fail-closed choices in Graceful Degradation.

7. Log every decision. Record the question, the answer, the policy version and the reason (the tuple path). That turns "who could read our payout data, and why?" into a query.

Where each check lives, end to end

Layer	Checks	Why there
API Gateway	Is the token valid? Does its scope allow this API?	Cheap, once per request, no domain knowledge
Service mesh (east-west)	May the *checkout service* call *payments*?	Workload identity, not user identity (the Service Mesh lesson)
Service handler + authorization service	May *this user* do *this action* on *this object*?	Only the service knows which object; only the PDP knows the relationships

Walkthrough with numbers (the seller portal)

- **Check volume.** 16,000 req/s reach the gateway (the capstone's figure); about a quarter hit the seller portal, and each portal request makes ~3 object checks → **~12,000 checks/s**. A decision cache with an 80% hit rate leaves **~2,400 checks/s** for the authorization service — a handful of instances.
- **Latency budget.** Most checks are 2–4 graph hops. With membership sub-results cached, a check is well under **10 ms at p99** — small next to a 200 ms page budget. List pages use one bulk check per page.
- **Tuple storage.** Memberships and shares: ~2M staff × ~10 tuples = **20M tuples**. Should all 500M invoices get a `parent` tuple? That's ~**50 GB** at ~100 bytes each — feasible, but a second copy of ownership data to keep in sync (by Change Data Capture — streaming every database change out to other systems — from the orders database). The alternative is to pass the invoice's folder in the check request as a **contextual fact** and store tuples only for things users actually share. Choose per object type.
- **The half-removed employee, replayed.** Removal deletes one tuple. The payout report saved after the removal carries a consistency token, so no service can show it to him. Search's pre-filter index updates from the tuple change within seconds.
- **The data leak, replayed.** `GET /invoices/8812` now calls `check(user, "view", invoice:8812)`. Changing the URL gets a **403** (HTTP "forbidden"). A shared test suite asserts, for every object endpoint, that a user from another organisation is denied.

Trade-offs

You gain:

- **One source of truth** for permissions — removals take effect everywhere, and rules change without redeploying thirty services.
- **Auditability** — every decision logged with its reason; "who can see X" becomes a query.
- **Expressive sharing** — nested teams, folders and organisations without hand-written recursive SQL.
- **Explicit freshness** — consistency tokens make the "removed but still sees new content" case impossible by design.

You pay:

- **A new critical dependency on nearly every request.** It needs high availability, caching and a deliberate fail-closed policy.
- **Data duplication.** Relationship tuples mirror facts that also live in domain databases, and the two must be kept in sync (dual-writes — the app writing to two stores separately — drift; CDC or the outbox pattern, which saves the event in the same database transaction and publishes it afterwards, is safer).
- **"What can I see?" remains hard.** Checks are cheap; list filtering needs extra indexes.
- **A policy language to learn and test.** Wrong schemas grant access silently — test policies like code, including "must deny" cases.

When not to use it

- **A monolith or a handful of services with a few roles.** Framework-level RBAC plus ownership checks in queries (`WHERE org_id = :callerOrg`) is simpler and entirely adequate.
- **As a feature-flag or entitlement system.** "Is the premium feature on for this plan?" is product configuration (the Feature Flags lesson's *permission flag*), not per-object authorization.
- **As a substitute for authentication.** The PDP trusts the identity it is given; verifying tokens stays at the gateway and service boundary.

Typical misuse: building a Zanzibar clone for five roles — or the opposite, thirty services each growing their own permission tables until an audit or a BOLA report forces the change.

Real-world examples

- **Google Zanzibar** — one system for the permissions of Drive, YouTube, Calendar, Cloud and more. The 2019 paper reports more than two trillion stored ACL tuples, millions of checks per second, and — over three years of production — 95th-percentile latency under 10 ms and availability above 99.999%.
- **Open-source Zanzibar-style systems:** SpiceDB (AuthZed), OpenFGA (a CNCF — Cloud Native Computing Foundation — project started at Auth0/Okta), Ory Keto. **Airbnb's Himeji** is an in-house system in the same family.
- **Policy engines for ABAC:** Open Policy Agent (OPA) with its Rego language, often run as a sidecar; AWS's **Cedar** language and the Amazon Verified Permissions service.
- **For AI engineers:** a RAG (retrieval-augmented generation: fetch relevant documents, then have the model answer using them) system must filter retrieved passages by the *asking user's* permissions before they reach the prompt — otherwise the model summarises documents the user may not open (RAG Pipeline). Agents that call tools on a user's behalf should be checked as *that user*, not with the agent's own broad service credentials.

In the interview

Prompts: "design the sharing model for a Google-Drive-like product", "how do 30 microservices enforce permissions consistently?", or a follow-up inside any multi-tenant (many customers sharing one system) design: "how do you stop one tenant reading another's data?"

> **30-second answer:** "I separate enforcement from decision. The gateway authenticates and checks coarse token scopes; each service enforces at the object — `check(user, action, object)` — against a central authorization service. For the model I'd use RBAC for staff roles, relationship-based rules for ownership and sharing — Zanzibar-style tuples like `folder#viewer@team#member`, so a check is a short graph walk — and attribute conditions like MFA on top. Removals delete one tuple, so they take effect everywhere; consistency tokens stop a removed user from seeing content created after the removal. It's on the hot path, so I cache membership sub-results, bulk-check list pages, pre-filter search, fail closed, and log every decision with its reason for audit."

Likely follow-ups (with answers):

- *"How do you show a user only the documents they may see in search?"* — Pre-filter: index each document with the principals allowed to read it, updated from tuple changes, and add "readable by any of my principals" to the query. Post-filter with bulk checks only works when few results are removed.
- *"The authorization service is down — what happens?"* — Fail closed for writes and sensitive reads; short-lived cached decisions for low-risk reads only if the product explicitly accepted that. It is tier-0 (the most critical class of service, which everything else depends on): replicated and cached, with availability at least that of its callers.

Mistake that fails candidates: checking only "is logged in" (BOLA); putting per-object permissions in JWTs; making the gateway understand the domain; ignoring revocation delay.

Self-check

- **Q1: Define PEP and PDP and where each lives.** → The PEP is where the request touches the object (service handler, or a gateway filter for coarse checks) and obeys the answer; the PDP is the authorization service holding rules and relationships. Enforcement stays local because only the service knows the object; the decision is central so it is consistent and auditable.
- **Q2: A product has organisations, nested teams and folders shared with teams. RBAC, ABAC or ReBAC?** → ReBAC for the sharing (paths from user through teams and folders to objects), RBAC for a few staff roles, ABAC conditions for context rules such as MFA. Pure RBAC would need a role per organisation per folder — role explosion; pure ABAC can't answer "who can see this folder?" without evaluating every user.
- **Q3: After removing Bob from a team, he opens a document uploaded 5 seconds later, because checks read a replica 10 seconds behind. Name the problem and the fix.** → The "new enemy" problem: stale permission data applied to newer content. Fix: store a consistency token with each content version and evaluate checks on that content at a snapshot at least that fresh; older content can still use cached snapshots.

Related patterns

- **API Gateway** — authenticates and checks coarse scopes; object-level decisions stay behind it.
- **Service Mesh** — authorizes *service-to-service* calls by workload identity; this lesson authorizes *users* on objects.
- **Sidecar** — a local policy engine (OPA) can run as a sidecar decision point.
- **Change Data Capture** — keeps relationship tuples in sync with domain databases.
- **Feature Flags** — gate features for cohorts; never a substitute for authorization.

TL;DR

Problem	Every service wrote its own permission checks: object-level holes (BOLA), removals that don't reach every service, slow recursive sharing queries, and no answer to "who can see X?"
Mechanism	Services enforce `check(user, action, object)`; a central decision service holds RBAC roles, ReBAC relationship tuples (Zanzibar-style `object#relation@subject`, checked by a short graph walk) and ABAC conditions; consistency tokens for freshness; caches, bulk checks and search pre-filters for speed; every decision logged.
Costs	A critical dependency on nearly every request (replicate, cache, fail closed), tuples to keep in sync with domain data, list filtering needing extra indexes, and a policy language to test.
Skip it when	A few services with a few roles — framework RBAC plus ownership conditions in queries is enough.

Sources: R. Pang et al., "Zanzibar: Google's Consistent, Global Authorization System", USENIX ATC 2019; D. Ferraiolo, R. Sandhu et al., "Proposed NIST Standard for Role-Based Access Control", *ACM TISSEC* 2001; NIST SP 800-162, *Guide to Attribute Based Access Control (ABAC) Definition and Considerations* (2014); OASIS XACML 3.0 (PEP/PDP terms); OWASP API Security Top 10 (2023), API1 "Broken Object Level Authorization"; Open Policy Agent documentation; J. Cutler et al., "Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization", OOPSLA 2024; OpenFGA and SpiceDB documentation; A. Yao et al., "Himeji: A Scalable Centralized System for Authorization at Airbnb", Airbnb Tech Blog (2021).

Capstone - The Entry Point, Assembled

Original lesson: Capstone - The Entry Point, Assembled on DesignGurus ↗

What this capstone does. It designs the full public API layer for web, mobile, TV, and 200 partner integrations. It does this by tracing **one request from a customer's phone in Sydney** inward, layer by layer, until it reaches the service that answers it. At each layer, it decides which responsibilities belong there and why. Whenever you hit a "pause and decide" point, settle on your answer before reading further.

Two rules come with you the whole way:

1. **Every layer is the same proxy primitive playing a different role.**
2. **Handle each request at the earliest layer that can handle it.** The cheapest request is the one that never goes any deeper.

> **Analogy.** Picture a well-run airport. Most questions get answered at the information kiosk by the entrance (the CDN). Everyone goes through a single security checkpoint (the gateway) that checks ID but doesn't care where you're flying. Frequent flyers and economy passengers get differently shaped service in their own lounges (BFFs). A turnstile limits how many times one badge can pass through per minute (rate limits). And behind the scenes, staff areas have their own badge readers between rooms (sidecars/mesh).

The brief

- **Clients:** web, iOS and Android, a TV app, and **200 partner integrations** (the kind whose schedules you can't control).
- **Traffic: 40,000 requests/s** at peak, spread across the globe. About **60%** is static or can be cached publicly.
- **Backend:** about **30 services in 3 languages**, running on the scaled fleet from earlier modules, with the reliability machinery inside.
- **Standing requirements:** partners must **never break** (versioning creates promises you can't take back). **No tenant** (a customer sharing the system) may starve the others (rate limiting for fairness). **East-west encryption** is on the security roadmap.

Stage 1 — The visitor's city (Edge / CDN)

Pause and decide: 60% of traffic is static or publicly cacheable, and Sydney is 200 ms from the origin. What stops at the edge, what only gets sped up there, and what must never be cached there?

A CDN edge in Sydney deals with three categories:

- **Stops at the edge.** Fingerprint every static asset (put a content hash in the filename) and cache it forever. API responses that are identical for everyone, such as catalog browsing, use `s-maxage=60, stale-while-revalidate`. *Why:* bytes that are the same for everyone should never cross the ocean.
- **Sped up at the edge.** TLS terminates over Sydney's **15 ms** path rather than the **200 ms** ocean route. Dynamic requests still cross the ocean, but on connections that are already warm. *Why this beats a Sydney datacenter:* it fixes the handshake and the bulk of static content without replicating the database.
- **Never cached.** Authenticated responses use `private, no-store`, set through **reviewed** configuration. *Why:* the CDN follows headers on every edge, so a mistake with private data would reach strangers all over the world.

Result: out of **40,000 req/s**, around **24,000** stop at this first layer and never use any application capacity.

Stage 2 — The one entrance (Gateway)

Pause and decide: 30 services and four client types. What does the entrance check, what does it refuse to know about, and where does its reliability come from?

The remaining **16,000 req/s** arrive at a single origin hostname.

- **A redundant API gateway** — it gets its reliability from load-balancer machinery (12 instances behind the load balancer, with health checks). *Why:* it's a deliberate single dependency, so it has to be redundant.
- **What it checks, once: AuthN** (who's calling, including token revocation). It passes trusted identity inward. It applies rate limiting, request checks (size caps, validation), TLS, and path routing (so topology stays private). And it creates the trace **request ID**.
- **What it refuses to know about:** discounts, order calculations, and shaping responses for clients. Services handle **AuthZ** (is this identity allowed to do this domain action?) — enforced in each service against a shared Authorization Service, and shaping goes to the BFFs. *Why:* thin = rarely changed = stable, which matters for a component every request depends on.

Stage 3 — One entrance, four shapes (BFF)

Pause and decide: the mobile product screen needs six services, TV shows much less, and partners care more about stability than quick change. How many BFFs, who owns each one, and which "BFF" isn't really a BFF?

- **Three BFFs + one API product:** a **web BFF**, one **shared mobile BFF** for iOS + Android, and a **TV BFF**, each owned by its client team. Each fans out in parallel over datacenter links (mobile: six calls x ~5 ms, and one ocean crossing) and returns a response shaped for the screen, with **no business rules**. Any number that two BFFs might disagree on belongs in a domain service.
- **Partners get a public API product**, not a BFF. It sits in a similar spot on the diagram, but it must be **versioned, documented, and stable** for consumers you can't update. *Why not a partner BFF owned by a client team:* partners can't keep up with your release speed.

Stage 4 — The rate limits

Pause and decide: Tuesday's partner bug fires 4,000 authenticated, valid retries per second. Which keys, which algorithm, which tiers — and where do the counters live, given 12 gateway instances?

- **Token buckets keyed by API key or account**, not by IP (one IP might be an entire campus). **b** is sized to let normal page-load bursts through.
- **Tiers:** free, paid, and partner, each with documented rates. Plus **cost weighting** — a search costs **10x** a health check.

- **The refusal contract:** 429 + `Retry-After` + remaining-budget headers, so partner SDKs back off on their own.
- **Shared counters:** Redis atomic increments across all **12** gateway instances. *Why:* limits kept locally would allow 12x the budget.
- **Login** gets its own per-account limit, since **credential stuffing** goes after accounts, not endpoints.

Failure walk — the 4,000 req/s partner bug: the burst allowance gets used up, 100/s keep going, and **3,900/s are refused in ~0.1 ms each** — roughly **1/500th** of what it would cost to serve them. Every other tenant is unaffected.

Stage 5 — The contracts that outlive you (Pagination and Versioning)

Pause and decide: partners run full exports, and old mobile builds stay installed for two years. Which pagination style, where do versions live, and — the senior-level question — what retirement plan is written down at the moment a version is announced?

- **Pagination:** opaque cursor tokens over composite `(created_at, id)` positions, across the entire public API. *Why not offsets:* partners may get as far as page **5,000**, where offsets mean ever-growing database work plus drift. **Opaque** means callers can't read or build the token, so the sort key can change later.
- **Versioning:** additive changes and tolerant readers by default. Breaks that can't be avoided become one coarse `/v2`, routed through a **gateway adapter** onto a single backend. Every version is *born with* its retirement plan: `Sunset` headers, per-client usage reports, and a brownout plan. **Two live versions at most.**
- **The middle ground for the mobile BFF:** quick additive changes, with major versions tied to the **oldest supported app build**.

Stage 6 — The turn inward (Sidecar / Mesh)

Pause and decide: 30 services, 3 languages, a security mandate — and a platform team of two people. Which step of the adoption ladder, being honest?

- The fleet has enough languages, services, and security need to justify a mesh. But **two engineers can't safely run a control plane that every team relies on**.
- **Decision: step 2 now** — sidecars that provide mutual TLS, uniform timeouts, and retries through one binary that works with any language, using **templated static config**. **Step 3 (the mesh control plane) comes later**, once a platform team is staffed. *Why not a mesh right away:* adoption thresholds include organizational ownership. The mesh lesson's incident shows that hand-configured sidecars will eventually need central coordination — so plan the hiring now.
- The gateway stays the only public entrance, and **east-west traffic never takes a detour through it**.

The final architecture

The course figure below puts every stage together. Clients (web, iOS/Android, TV, 200 partners) send 40,000 req/s to the Sydney CDN edge, where 24,000/s stop. The other 16,000/s go on to the 12-instance API gateway (AuthN with revocation, token buckets per API key with Redis counters, size caps, request IDs, private topology, "no domain words"). Below that are the Web, Mobile, and TV BFFs (shaping, one per experience, zero domain logic) and the Partner API (a product: cursors, /v2, Sunset). At the bottom sit ~30 services in 3 languages, each with a sidecar handling mTLS, timeouts, and retries from templated config, while the mesh control plane waits for a platform team. The buggy partner's 3,900/s are turned away at the entrance, 0.1 ms each.

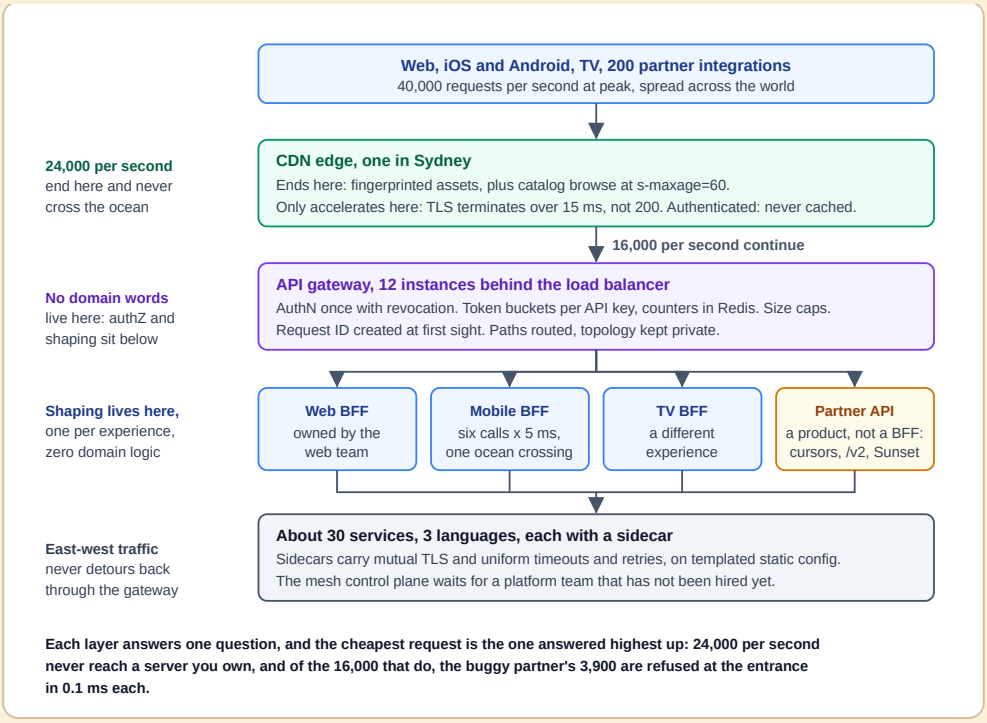


Figure — The entry point, assembled (course diagram)

Debrief — one question per layer

Layer	The question it answers	The decision that mattered
CDN	What never has to arrive at all?	60% stops at the edge; `private, no-store` reviewed like auth code
Gateway	What gets checked exactly once?	AuthN with revocation; no domain words at all; thin = stable
BFF	Who shapes responses, for each experience?	Three BFFs + one API product, owned by client teams
Rate limits	Who gets counted, and how?	Token buckets per real identity; refusal as a contract
Pagination/versioning	What outlasts your intentions?	Cursors with tiebreakers; versions created with retirement dates
Sidecar/mesh	What does the inside need right now?	Step 2 now, honestly; step 3 with a hiring plan
Authorization *(added lesson)*	May *this* user touch *this* object?	Services enforce at the object; one shared decision service holds roles and relationships

The middle column shows the same proxy primitive answering six different questions. Together they make up one boundary, where each request stops at the earliest layer able to deal with it: **served from the edge cache, turned away by the rate limiter, checked at the gateway, tailored by a BFF, or authenticated service-to-service.**

The only thing deliberately put off is the full mesh. That's because adoption thresholds include *organizational ownership*, not just technical scale. Good design chooses infrastructure that the team you actually have can run safely. Every layer that gets added has a clear owner, a benefit you can measure, and a boundary that keeps it from swallowing other layers' jobs.

Extend it yourself

- **The partner brownout.** A year after `/v2`, `/v1` still carries **30%** of partner traffic. Design the full retirement campaign: dashboards, outreach tiers, the brownout schedule, and what the `Retry-After` on a brownout **410** should say.

- **An entry point in multiple regions.** Add a second region. Which layers replicate easily (the CDN already does; the gateway mostly does)? Where do rate-limit counters live now? It's **global fairness vs. regional latency** — choose one and defend it. Which BFF assumptions stop holding?
- **For AI engineers.** Add an AI assistant endpoint. Streaming responses use **SSE** (server-sent events), so check which layers they pass through and how each one buffers. Add **token-based** rate limits next to the request-based ones. Place the internal **LLM gateway** as a second entrance — behind the BFFs or beside the services? Explain why.

Interview summary

> The public API layer is one proxy primitive playing six roles, and each request stops at the earliest layer that can deal with it. The CDN answers ~60% at the edge. The thin gateway authenticates once and rate-limits per identity with a 429 contract. BFFs owned by client teams shape the responses. Cursors, plus additive versioning with sunset dates, keep partner contracts safe. And sidecars secure east-west traffic today, with a full mesh put off until a platform team is in place to own it. Behind all of it, each service enforces object-level permissions against one shared authorization service, so "logged in" is never mistaken for "allowed".

Module 9 — Operating in Production (Observability and Delivery)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Hardware failures, traffic spikes and attackers cause some outages. But most production outages come from somewhere closer to home: *changes we make ourselves*. A deploy (putting a new version of code onto the servers), a config tweak or a database migration (a scripted change to a database's tables or stored data) touches a system that was healthy a minute ago, and suddenly it isn't. The reliability module earlier covered failures that arrive from the outside world. Here the focus shifts inward. You will learn to ship your own changes safely, and to *watch* production closely while each change goes out.

Analogy. Picture a hospital operating room. The surgery is the deploy. Before the surgeon touches anything, the patient is wired to monitors — that is observation. A skilled surgeon also knows how to pause or reverse any step the moment a monitor turns red — that is rollback. This module hands you the monitors first, then teaches the careful surgery.

The five patterns (plus two added lessons) and the job each does

The situation	The pattern
Is this one server fit to take traffic at this moment?	Health Check
A single slow request passed through thirty services — which one ate the time?	Distributed Tracing
Replace the old version with the new one in a single move, and keep an instant way back	Blue-Green Deployment
Make the new version earn trust on a small slice of real traffic before everyone gets it	Canary Deployment
Put the code in production "dark" (there, but switched off), then turn the feature on later — step by step, and with an off switch	Feature Flags
Package every service the same way and keep a fleet of them running in the declared state	Containers and Kubernetes *(added lesson)*
Get every change from commit (one saved, recorded code change) to production through one automated, trusted path, with the right rollout strategy	CI/CD Pipelines and Deployment Strategies *(added lesson)*

Two of these patterns **observe**. They answer "is this server okay?" and "where did this request spend its time?". The other three **change** production, but without betting every user on each deploy. We cover observation first for a simple reason. Every safe delivery technique relies on spotting failure fast and accurately. "Roll back if the metrics look bad" means nothing if you can't see the metrics.

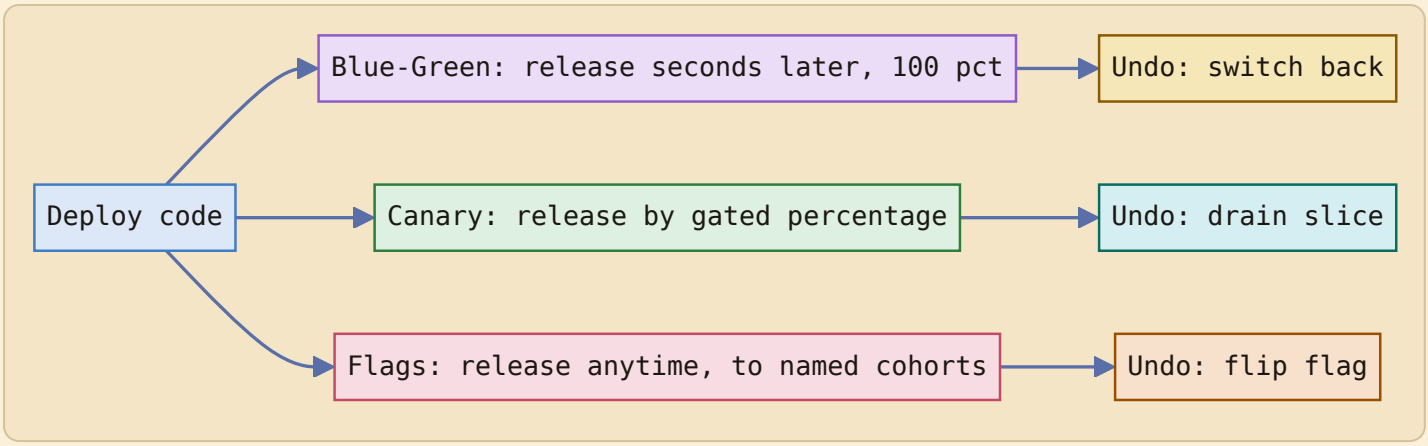
Two facts that organize the module

1. **Deploying and releasing are two separate events.** To *deploy* is to put new code on servers. To *release* is to let users experience what that code does. The traditional approach fuses the two: code lands, and instantly every user has it. The three delivery patterns pry them apart, each by a different amount:
   - Blue-green gets the new code ready in an idle environment, then releases it a few seconds later by moving traffic over.
   - Canary takes code that is already deployed and releases it to a small percentage of users.

    - Feature flags split the two events fully. You might deploy on Tuesday, release to a beta group the following week, and turn it off again in an instant. So for any change, the design question is: *how far apart should deploy and release be for this particular change?*

2. **Rolling back should be an everyday move, not a big project.** If a change is hurting production, you need to reverse it *right now*. The patterns differ in how cheap that reversal is. Blue-green moves traffic back in seconds. Canary drains the new version in seconds. A flag turns the behavior off instantly. Compare that with the slow path: building a code revert and pushing it through CI (continuous integration — the automated pipeline that builds and tests code). That can keep the outage going for tens of minutes. The rule, then: **every delivery design needs an undo design built in.** The capstone shows this in practice, keeping a safe way back open at every stage of a live migration.

Delivery patterns placed on the deploy-to-release separation spectrum



The diagram shows all three patterns sharing one starting step, "deploy code". They differ only in *when, and for whom,* the release happens. Each branch also finishes with its own cheap undo.

Four terms used throughout (load balancers, latency, replicas and other basics are explained in the System Design Fundamentals primer)

- **Deploy vs. release** — putting code on servers vs. letting users feel it (Fact 1). No distinction in this module matters more.
- **Exposure scope (blast radius)** — the number of users a bad change can hurt before operators manage to stop it. A naive release hits 100%. Blue-green also hits 100%, but you can undo it fast. A canary hits about 1%. A flag can shrink it to a single named team.
- **Roll back vs. roll forward** — to *roll back* is to reverse the change. To *roll forward* is to push a fix on top of it. When rolling back is an option, it is usually the quicker one. The capstone shows how to reshape a migration so that rollback stays available even where the naive plan would rule it out.
- **Golden signals** — the four numbers to keep your eyes on during any change: **traffic** rate, **error** rate, **latency** (how long each request takes) and **saturation** (how close resources are to full). A canary's verdict comes from these numbers — the very reason observation is taught before change.

How to read the module. Health Check extends the endpoints from the load-balancer lesson. It contrasts shallow and deep checks and includes a case where one dependency failure knocked out a whole fleet. Distributed Tracing tracks a single request as it crosses services; every later "watch the canary" step leans on that per-request evidence. The delivery lessons then move from the simplest reversal to the most flexible one: Blue-Green (two environments and one switch) → Canary (widen exposure as measurements come back good) → Feature Flags (split deploy from release entirely, and meet "flag debt"). Two added lessons (not in the original course) then zoom out to the platform these patterns run on: **Containers and Kubernetes** (images, pods, Services, probes, rolling updates and the control loops that keep a fleet in its declared state) and **CI/CD Pipelines and Deployment Strategies** (build once and promote, infrastructure as code, GitOps, and a side-by-side table of recreate, rolling,

blue-green, canary and shadow rollouts). Finally, the capstone uses every one of them in a zero-downtime database migration.

> Short on time? Start with **Canary Deployment** — interviewers dig into it more than any other delivery pattern. Health Check is brief, so don't skip it.

Before you start — three questions about a system you know

- Think of your most recent bad deploy. How long until someone *noticed*, and how long until it was *undone*? This module is about shrinking exactly those two numbers.
- When a request is slow, can you tell which service used up the time? Or do you grep (search text with a command-line tool) thirty log files and make a guess?
- Could your newest feature be switched off this minute, with no deploy? And who would be able to do it?

Health Check Endpoint

Original lesson: Health Check Endpoint on DesignGurus ↗

What it is. A health check is a very small HTTP endpoint (a URL the server answers, such as `/healthz`) that lives on each server instance. It exists to answer a single question: *"should this instance be getting traffic at this moment?"* Three callers hit it every couple of seconds and act on what it says: the load balancer (which spreads requests across servers), the container orchestrator (the system that starts, stops and places containerised apps on machines — Kubernetes, for example) and the auto-scaler (which adds or removes servers as load changes). Each call is cheap and they never stop. Yet the answer matters a lot, because the callers can only respond in drastic ways. They either stop sending traffic to the instance, or they kill it and start it again.

> **Analogy.** Imagine a manager asking a shop assistant two quite different questions. First: "Are you conscious?" A "no" means calling an ambulance — the equivalent of a restart. Second: "Can you serve customers this minute?" A "no" here is often harmless. Maybe the till is still being set up, or the counter is closing. The fix is just to point customers to the next counter for a bit. Confuse the two questions and you end up calling an ambulance whenever the till is slow.

The problem: naive health checks lie in both directions

The load-balancer lesson showed two mistakes that pull in opposite directions:

- A **shallow** check asks only whether the process answers ("I'm alive, HTTP 200" — the standard "OK" status code). The check can stay green even when the instance can't reach any of the databases or services it needs to do real work. The load balancer then keeps routing users to a server that can't help them.
- A **deep** check also probes dependencies — for instance, it pings the database. That fixes the first mistake but introduces a worse one. Suppose the *shared* database has a short hiccup. **Every** instance fails its check at the same instant, and the load balancer pulls *the whole fleet* out of service. What was a partial slowdown turns into a full outage.

The way out is to ask a better question. Not "is the entire architecture healthy?", but "should *this particular instance* get traffic?" Modern platforms go further and break that into two separate questions, each with its own consequence.

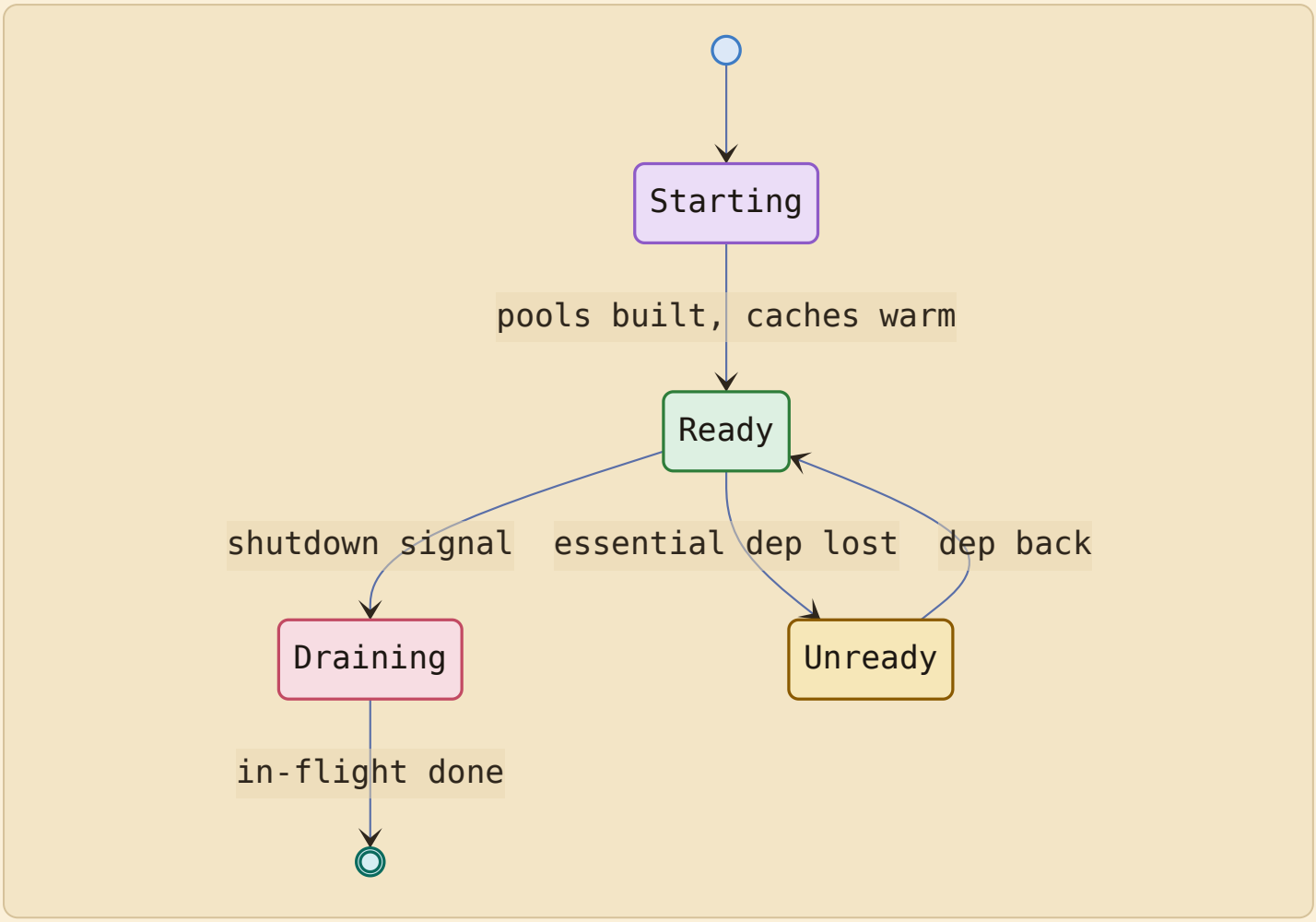
How it works: two verdicts, two consequences

1. **Liveness — "am I broken at a basic level?"** A failed liveness check makes the platform **restart** the process. That means liveness must cover *only* problems a restart can really cure: the process has stopped responding, it is deadlocked (threads waiting on one another forever), or it is stuck

internally for good. **Dependencies never go in this check.** There are two reasons. A restart of your app can't fix a broken database. And when a shared dependency goes down, *every* instance fails liveness, so the platform keeps restarting processes that were actually fine — a *restart storm*.

2. **Readiness — "should traffic come to me right now?"** A failed readiness check makes the platform **de-route** the instance for a while, meaning it stops sending requests there. This is a mild action and easy to reverse. An instance can be in perfect health and still honestly be *not ready*: during startup, while filling its caches, while setting up its connection pools (database connections opened ahead of time and reused), or while draining before it shuts down. De-routing can be undone; a restart wipes things out. So each check has to fit the consequence it sets off.
3. **Lifecycle honesty.** At each stage of its life, the instance should report the truth:
   - At **startup**, readiness begins as false.
   - The instance sets up its pools and warms its caches. Only after that does it flip readiness to true, so no user lands on a cold, sluggish instance.
   - At **shutdown**, readiness goes to false first. The instance keeps working on requests already in progress while the load balancer steers traffic away. Only then does it exit. During a rolling deploy, this ordering stops each new instance from being slow for its first ~30 seconds of serving.

Instance lifecycle as seen by readiness



This state diagram follows readiness across an instance's life. The instance reaches Ready only once warm-up is done. It may drop to Unready for a while if a dependency it truly cannot live without goes missing. And it always passes through Draining on its way out.

Three rules that keep health checks safe

- **Check yourself, not the architecture.** Handle dependency failures *inside* the service. Use circuit breakers (they stop calls to a dependency that is failing) and fallbacks (a simpler substitute answer used when the real one isn't available), so the service can keep reporting "ready" while

running in a reduced mode. There is one exception. Some services are completely pointless without one dependency — a pure database proxy is the classic case. There, readiness *may* include that dependency. The team just has to accept, with eyes open, that when it fails every instance gets de-routed.

- **Cost almost nothing.** Run the numbers. 30 services × 40 instances, each probed every 2 seconds, gives **600 checks per second** — without end. Put a database query in each probe and you have 600 useless queries per second. It gets worse: a slow probe can *time out*, making healthy capacity look failed. You have built an outage yourself. Aim for an in-memory check that finishes in roughly **1 ms**.
- **Tell the truth at every lifecycle stage.** This is step 3 from above: not ready until warmed up, and not ready while draining.

Routing checks vs. monitoring checks

Routing checks run per instance, run very often and must be cheap. **Monitoring checks** are something else. They exercise the whole system using **synthetic transactions** — scripted pretend requests, like a test checkout — along with deeper probes of dependencies. They run from a central place, less frequently, and their output is *alerts for people*, never routing decisions. Keep them on separate endpoints, since the consequences are different. Merging the two is precisely how deep dependency tests creep into routing and restart logic for the whole fleet.

Handle every probe condition like a policy that gets reviewed. When someone adds a new dependency, it's tempting to tack "one more network call" onto readiness. Don't do it casually. For each condition, write out three things: the condition itself → what action fires when it fails → how that action helps the system recover. If neither a restart nor de-routing can fix the problem, the condition has no place in that probe.

Response format. All the platform needs is pass or fail. Extra diagnostic fields that name the failing local condition are fine. But never put secrets or sensitive config in them, and don't do costly work just to produce them.

Liveness vs. readiness (comparison)

	Liveness	Readiness
Question	Am I broken at a basic level?	Should traffic come to me right now?
Consequence	Restart (drastic)	De-route (mild, reversible)
May include dependencies?	Never	Only the essential ones, picked on purpose
Can honestly say "no" while healthy?	No	Yes — while warming up or draining
What goes wrong if designed badly	Restart storms whenever a dependency fails	The whole fleet removed whenever a shared dependency fails

Trade-offs

You gain:

- Failing instances get pulled out at machine speed — load balancers react within seconds.
- Deploys happen in a safe order: readiness lets in only warmed-up instances and drains each one before it is terminated.
- Self-healing: auto-scaling platforms swap out any instance that fails liveness.

You pay:

- **A mistake here hits the whole fleet.** The code is only ~10 lines, but an error spreads everywhere. Dependencies in liveness → restart loops. Dependencies in readiness → the entire fleet pulled out. Slow probes → timeouts that invent failures.

- **Probe load.** Each probe is tiny, but it never stops and it runs on every instance. As the number of services grows, this needs discipline.
- **False confidence.** A green probe only shows the instance answered one narrow question. It does not show a customer can actually finish a checkout. That system-level proof comes from central monitoring.

When not to use it (or how to scope it)

Any service that receives routed traffic needs health checks. Just keep what they check narrow:

- **Don't use a single endpoint for routing, restarting and alerting.** Three different consequences call for three different designs.
- **Don't make readiness depend on optional dependencies.** If the recommendations service is down, the page should simply hide that widget (degrade) — not knock your whole fleet offline.
- **Batch workers and queue consumers** (processes that pull jobs from a message queue rather than receiving requests) receive no routed traffic. For them, "health" means liveness plus **backlog age** (the age of the oldest message not yet processed). The queue's own metrics serve as the readiness signal.

In the interview

Expect this after almost any load-balancer or Kubernetes design: "what does your health endpoint check?" The interviewer is listening for the liveness/readiness split, and for whether you see the danger of putting a shared dependency in either one.

> **30-second answer:** "Two endpoints, each giving its own verdict. Liveness asks: am I broken in a way a restart would fix? That means the process responds and isn't deadlocked. It never includes dependencies — restarting me won't repair the database, and a shared outage would turn into a restart storm. Readiness asks: should I be getting traffic right now? It's false while warming up and while draining, true the rest of the time. Dependencies stay out of it unless the service is genuinely useless without one, because serving in a degraded way is better than being pulled out. Both checks are cheap. Probes hit the whole fleet every two seconds, so a DB query in the check means hundreds of wasted queries a second — and one timeout could create an outage. Deep system checks do exist, but as central monitoring that feeds alerts. They never drive routing."

Likely follow-ups (with answers):

- *"How does an instance avoid serving while still cold after startup?"* — Readiness stays false while it fills its connection pools and warms its caches. It turns true only once the instance can serve at full speed. Shutdown runs in reverse: go unready, let in-flight requests finish while the load balancer drains traffic away, then exit.
- *"Your database-proxy service has no degraded mode to fall back to. Should readiness include the DB or not?"* — Include it, on purpose, since the proxy literally can't work without the database. Accept that a DB failure will de-route every instance, and make sure the *callers* degrade gracefully. The rule comes from the consequence, not from a blanket "never include dependencies".

Mistake that fails candidates: putting dependencies in liveness, using one endpoint for both routing and alerting, or suggesting "ping the database" without explaining how it knocks out the whole fleet at once.

Self-check:

- **Q1.** Name the two questions and their consequences. Why are dependencies forbidden in one and rationed in the other? → Liveness ("am I broken?") leads to a restart. No dependencies, because restarting me can't repair some other system. Readiness ("should I get traffic?") leads to de-routing. Include only the dependencies without which the service can't do anything useful. When an optional dependency fails, the service should degrade, not lose capacity.
- **Q2.** A service relies on a database (essential), Redis as a cache (on a miss it falls back to the DB) and a recommendations service (an optional widget). Where does each check go? → Liveness: just an in-memory check of the process. Readiness: the database plus the warm-up/draining state. Redis: in neither (a cache miss still works through the DB). Recommendations: in neither (just hide the widget). A synthetic checkout test: in central monitoring.
- **Q3.** At 3:07 AM the database goes through a 90-second **failover** (a replica gets promoted to be the new primary). Every instance of six services restarts, some more than once. The outage lasts 20 minutes rather than 90 seconds. Why? → Liveness included a DB call. So every instance appeared "broken" and got restarted. Each restart threw away in-flight work and came back cold. The instances then failed the same check again. And every restart rebuilt its connection pools, pounding the database while it was trying to recover. Fix: liveness checks only the process. The DB goes in readiness only if it is truly essential; otherwise circuit breakers handle degradation. Deployment review blocks any liveness condition that a restart can't fix.

Related patterns:

- **Load Balancing** — the biggest user of health checks; readiness drives both removal and draining.
- **Graceful Degradation** — the right home for dependency problems: degrade and stay ready, instead of failing the probe and disappearing.
- **Sidecar** — startup order: the app reports not-ready until its sidecar proxy is ready, and the sidecar is the last thing to drain.
- **Auto-Scaling** — failed liveness leads to the instance being replaced; this endpoint powers the fleet's self-healing.

TL;DR

- **Problem:** the routing and restart machinery wants a verdict for each instance roughly every ~2 s. A naive endpoint misleads in both directions — shallow stays green while the instance is useless; deep turns the whole fleet red when a shared dependency blips.
- **Mechanism:** two endpoints. Liveness covers only brokenness a restart can fix, and never dependencies. Readiness covers whether the instance deserves traffic: false while warming up or draining, and including only truly essential dependencies. Both run in ~1 ms. Deep checks belong to central monitoring.
- **Costs:** a tiny piece of code whose mistakes hit the whole fleet (restart storms, everyone removed at once, failures created by timeouts), plus probe load that never ends.
- **Skip it when:** never — but don't let one endpoint handle routing, restarts and alerting together, and let queue workers report backlog age instead.

Distributed Tracing

Original lesson: Distributed Tracing on DesignGurus ↗

What it is. Distributed tracing captures the whole trip one request takes as it moves from service to service. Where the request enters the system, it gets stamped with a **trace ID**. Then every service and database call it touches writes down a timed record, called a **span**. Each span carries the trace ID

and notes which call triggered it. A collector gathers the spans and rebuilds them into a single timeline, known as a **waterfall**. Looking at it, you can see precisely where the request's time went.

> **Analogy.** Think of a parcel with a tracking number. Every depot scans it on arrival and departure, recording the time and the previous stop. One depot's log alone says little — just "the parcel spent 5 minutes here". The tracking page joins all those scans into one story. Right away you spot that the parcel sat in a single sorting centre for three days.

The problem: the time hides *between* the logs

Support says checkout is "slow sometimes". The **p99** — the latency that only the slowest 1% of requests go beyond — sits at **4.2 seconds**. The budget is **800 ms**. And it only hits some requests, at times nobody can predict. One checkout request passes through **12 services through 40+ calls**: the gateway, the **BFF** (backend-for-frontend, a layer built to serve the mobile app), checkout, inventory, payment, fraud, pricing and others.

Twelve engineers each dig through one of the twelve services' logs at the same time. Every service reports "I was fast". Checkout logged just 8 ms of its own work. Payment seemed responsive. Every log is *correct*, yet none of them answers the question. That's because **the 4.2 seconds wasn't spent inside a single service; it was spent between the records**, along one specific sequence of calls that no individual log captures.

Next, three engineers put in two days aligning timestamps from different logging systems. The trouble is that those timestamps come from machines whose clocks disagree by milliseconds. (The vector-clocks lesson covers why physical timestamps can't reliably put events from different machines in order.) The team forms a theory and ships a fix — and p99 stays exactly where it was. The underlying issue: logs answer *"what did this component do?"*. The real question was *"which path did this request take, and where did its time go?"* Nothing had recorded that.

Why the obvious fixes fail

- **"Add more logging."** You end up with more records, but they still aren't *linked* to one another. The volume rises; the ability to connect them does not. What was missing was relationships, not more detail.
- **"Put a correlation ID in every log line."** (The gateway makes an ID and passes it on, and each service logs it.) This helps partly. Searching for the ID pulls up every log line for that request. But the lines still have no **parent/child relationships** (which call triggered which one). Their timings are hard to line up because the clocks are skewed. And nothing draws the path for you. You've gathered the records, but they're in no particular order.
- **"Send synthetic test requests."** Scripted test traffic can only recreate conditions you already know how to set off. This incident is hard exactly because no one knows what sets it off.

What we need is a record that carries one identifier through every service, notes when the request arrives and leaves, and lets a system piece the full path back together.

How it works

1. **The entry point creates a trace ID.** The first component the request reaches (the API gateway) generates a unique trace ID. This is why the gateway lesson gave the gateway the job of creating request IDs. It is the first to see every external request, so it naturally becomes the *root* of every trace tree.
2. **Each operation records a span.** A span is a single timed operation: handling the request, making an outbound call, or running a database query. It holds a name, a start time and an end time, its **parent span** (the operation that triggered it), and attributes like URL, status and error. Spans nest inside one another to form a parent/child tree. Draw that tree along a timeline and you have the waterfall.
3. **Context propagation carries the IDs forward.** The trace ID and the current span ID ride along with the request. On synchronous HTTP calls (the caller waits for the reply) they go in a standard header (W3C `traceparent`). On asynchronous work (handed off through a queue and processed later) they go in **queue message metadata** (extra labelled fields that travel with a message, separate from

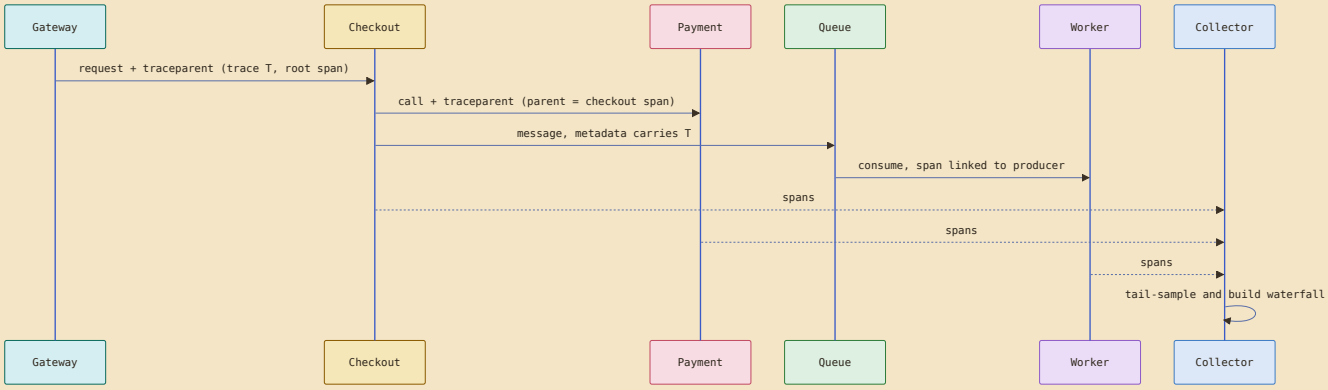
its body). The consumer's span *links* back to the producer's span, and the time between them shows how long the message sat in the queue. **If even one service fails to pass the header on, every span after it becomes an orphan** — it has no parent, so it drops out of the trace.

4. **Spans are sent to a collector**, which puts each trace back together as a waterfall you can view and search.

Here is a trace of the slow checkout (indentation shows parent/child):

```
Trace 7f3a...  (checkout, 4,214 ms total)
├── gateway                    2 ms
├── mobile BFF                 11 ms
│   ├── checkout svc           4,190 ms   ← there
│   │   ├── inventory          12 ms
│   │   ├── payment svc        4,110 ms   ← in here
│   │   │   ├── fraud check    1,210 ms  (timeout)
│   │   │   ├── fraud check    1,240 ms  (retry 1)
│   │   │   ├── fraud check    1,190 ms  (retry 2)   ← found it
│   │   │   └── charge         180 ms
│   │   └── pricing            9 ms
```

Context propagation through sync and async hops



This sequence diagram shows the trace context travelling two ways: on an HTTP call (checkout → payment) and across a queue hop (checkout → queue → worker). Every service reports its spans to the collector. The collector picks which traces to keep and assembles the waterfall.

Instrumentation — the three ways spans get created, from least to most effort:

- **Service-mesh sidecars** (proxies that run beside each service) capture every hop between services on their own. You get this visibility without changing the application at all.
- **Auto-instrumentation libraries**, such as **OpenTelemetry**, wrap the HTTP clients, database drivers (the libraries an app uses to talk to a database) and queue clients inside each process.
- **Manual domain spans** cover steps no library can guess — "apply discount rules", for example. Add these only where the same investigations keep coming back. The two automatic layers already provide a helpful skeleton, and that alone cracks many latency puzzles.

Sampling — keeping everything isn't possible. 40,000 requests/s × ~40 spans per request = **1.6 million spans per second**. Saving all of that isn't practical, so you keep only some:

- **Head sampling** makes the call when a request *begins* (say, keep 1%). It is cheap and easy. But it decides before the outcome is known, so there's a **99% chance of discarding** the slow or failing trace you actually care about.
- **Tail sampling** makes the call *once the request is done*. It can hold on to every error, every request that missed its latency objective, and a small random baseline. You keep exactly the traces you need. The price: the sampling system has to **buffer every trace** until it can decide. That is real infrastructure, and interviewers like to discuss it.
- What teams usually do: tail rules for errors and outliers, plus a head-sampled baseline of ordinary traffic.

Walkthrough with numbers: the same incident, with tracing

- **Minute 1:** an engineer filters on `duration > 4s` and gets dozens of traces straight away. Tail sampling kept them because they are latency outliers.
- **Minute 2:** opening one reveals the waterfall above. Inside payment's 4,110 ms span sit **three sequential fraud-check spans of ~1.2 s each**: one call that timed out, then two retries. The retry logic did just what it was configured to do. What changed is that fraud's p99 climbed past payment's timeout. So each affected checkout spends timeout × 3 attempts ≈ **3.9 s** before it succeeds.
- **Minute 4:** root cause located. Fraud's slower latency is the real problem; payment's timeout/retry settings shape how that problem shows up. The earlier wrong theory had burned six engineer-days. Tracing compresses **days of reconstruction into minutes of reading**.
- **Bonus:** combining many traces yields a live **dependency map** — which services call which, how often and how fast. Teams often find call paths nobody had documented this way.
- **The bill:** keeping errors + outliers + a 0.1% baseline means storing about **2,000 spans/s**, which is manageable. But the buffering tier that tail decisions require is genuine infrastructure and belongs in capacity planning.

The three pillars of observability

	Metrics	Traces	Logs
Question	"How much, how fast — overall?"	"Which path did this request take?"	"What exactly happened at this spot?"
Shape	Numbers over time; cheap enough to keep forever	Trees of timed spans; sampled	Text events; very high volume
Alerting	*The* layer that alerts (golden signals)	Seldom alerts; explains the alerts	Sometimes (on error patterns)
Cost	Small	Medium, kept in check by sampling	The largest

These aren't rivals; you use them in order. **Metrics ring the alarm** (p99 is over its objective). **Traces localize it** (fraud-check retries inside payment). **Logs explain it** (the exception and its parameters). Debug from left to right. Include the trace ID in log lines, so you can hop from a waterfall directly to the matching logs. Teams without traces jump straight from an overall alarm to searching every log — which is the incident this lesson opened with.

Trade-offs

You gain:

- Evidence about a single request across any number of services — something separate logs simply can't express, by their nature.
- Quicker latency diagnosis. Retry storms (waves of repeated retries that pile extra load onto a struggling service), lock contention (requests queuing to change the same locked data) and **N+1 queries** (one query per item, rather than one query for all the items) show up as visible sequences in the waterfall.
- Knowing which service really caused an error across service boundaries — which component actually returned the 500 (HTTP "internal server error").
- A live dependency map, as a free side effect.

You pay:

- **Propagation discipline, with no end date.** Every service, client library and queue consumer has to pass the context on. A single missing header orphans everything after it. So your traces are only as good as the *weakest* client's behavior.

- **Sampling infrastructure** — the tail-sampling buffer and the layer that makes the decisions, plus ongoing tweaking of what counts as "interesting".
- **Instrumentation work** beyond the automatic skeleton, added only where puzzles keep recurring.
- **Privacy exposure.** Spans can hold URLs, parameters and any attributes at all. So **PII scrubbing** (stripping out personal data) must happen inside the collection pipeline — the same rule as in the event-driven-architecture lesson.

When not to use it

- **A monolith** (one application deployed as a single unit). A profiler (a tool that measures where a program spends its CPU time) does a much better job of explaining time spent inside one process. Tracing earns its keep when latency is scattered *between* processes.
- **Three services and a queue.** Structured logs (log lines written as named fields, such as JSON, so tools can filter them) with correlation IDs might be all you need. Tracing becomes more valuable the more hops there are.
- **In place of logs.** Spans give you the skeleton (timing and structure). Logs give you the story (the details).
- Two frequent misuses: storing 100% of traces forever (costs blow up as traffic grows), and hand-instrumenting every internal function from day one (effort wasted on code that never puzzled anyone).

Real-world examples

- **Google's Dapper paper** — brought distributed tracing to planet-sized scale using heavy sampling. Most of today's ideas trace back to it.
- **OpenTelemetry** — vendor-neutral instrumentation, `traceparent` propagation and exporting. It is *the* standard to mention in interviews.
- **Jaeger, Zipkin, Grafana Tempo** — store and show traces. **Honeycomb, Datadog** — all-in-one observability platforms.
- **Service meshes** — generate the service-level trace skeleton without any extra work (the observability benefit from the mesh lesson, fed into a trace backend).
- **For AI engineers:** observing an agent (an LLM-driven program that plans steps and calls tools on its own) *is* tracing. One agent run is a trace. Every model call, tool call and retrieval is a span, with attributes like token count, cost and model version. A waterfall could reveal that a 40-second, $0.30 run made three model calls one after another that could have been merged — or it could point straight at the tool call that failed. LLM tracing tools such as **LangSmith** are descendants of Dapper. And context has to be passed through an LLM gateway just as it is through HTTP headers.

In the interview

A common prompt: "a slow request passes through thirty services — how do you find where the delay is?" "Search the logs" won't satisfy the interviewer. Follow-up questions dig into the cost of sampling and propagation across async hops.

> **30-second answer:** "Distributed tracing. The gateway mints a trace ID, and every operation records a span that is timed, linked to its parent and tagged. The context travels in headers across every hop, queue messages included, so async work stays part of the picture. Any slow request can be drawn as a waterfall, and you see where the time hid within minutes; retry storms and lock waits become visible. There are two design choices. First, sampling: tail-based, keeping errors and latency outliers plus a small baseline, because head sampling throws out the interesting 1% before it knows it's interesting. Second, propagation discipline, because a single service dropping the header orphans everything after it. The mesh gives me the service-level skeleton for free, and OpenTelemetry adds in-process detail where the mysteries keep coming back."

Likely follow-ups (with answers):

- *"Head or tail sampling, and why?"* — Tail for errors and latency outliers, since you can't know the outcome until the request is finished. Add a head-sampled baseline for normal traffic. Tail sampling means buffering every trace until a decision is made, which is extra infrastructure. Using head sampling alone tends to throw away exactly the rare p99 examples you need.
- *"How does a trace make it across a queue hop?"* — Store the trace context in the message metadata. The consumer starts a span that is *linked* to the producer's span. The time gap between them is the queue wait, so backlog delay shows up in the same waterfall as the synchronous work. Queues that don't propagate context are where traces break most often.

Mistake that fails candidates: suggesting more logs, keeping 100% of traces without any cost model, or letting propagation die at the first queue.

Self-check:

- **Q1.** Define a trace, a span and context propagation. Where does the trace ID get created, and why there? → A trace is the full path of one request, identified by a trace ID. A span is a single timed operation with a start and end, a parent and attributes. Propagation carries the trace and span IDs across every HTTP call and queue message so that new spans attach to the correct trace. The gateway creates the ID because it is the first component to see every external request, making it the natural root.
- **Q2.** Team A cares about cost and has ordinary traffic. Team B has rare p99 spikes that are gone before anyone can look. Design sampling for each. → Team A: a modest head sample (1%, say) plus always keeping errors — the cost stays predictable. Team B: tail sampling that keeps every error and every trace over the latency limit plus a small baseline, and accepts the buffer cost on purpose, because rare and unpredictable outliers are precisely what head sampling loses.
- **Q3.** Since Tuesday, every checkout trace stops at payment. The fraud and charge spans are gone, even though those calls clearly still happen. → Payment has stopped forwarding `traceparent`. Most likely a dependency update swapped in a different HTTP client or framework, so fraud's spans are now orphans. The fix is to apply OpenTelemetry auto-instrumentation to the new client. Protect against this whole category of bug with **trace-completeness monitoring** (alert when the orphan-span rate rises or expected child spans go missing) and with propagation tests in the service template.

Related patterns:

- **API Gateway** — mints the trace ID and the root span.
- **Service Mesh** — its sidecars supply the free service-level skeleton.
- **Message Queue** — message metadata carries the context, so async work and queue delay remain in the waterfall.
- **Retry with Exponential Backoff** — tracing makes retry storms visible (exactly what the walkthrough uncovered).
- **Canary Deployment** — canary verdicts rely on this telemetry.

TL;DR

- **Problem:** when a slow request crosses many services, its time hides between the logs of each service. Every service claims "I was fast", and piecing timestamps together takes days and ends in a guess.
- **Mechanism:** the gateway creates the trace ID; every operation records a timed span with a parent; context travels in headers and queue metadata; a collector rebuilds the waterfalls; tail sampling picks errors and outliers out of the 1.6M spans/s stream.

- **Costs:** propagation discipline forever, sampling infrastructure, instrumentation beyond the mesh skeleton, and PII scrubbing.
- **Skip it when:** there's a single process (use a profiler), about 3 services (correlation IDs may be enough), or you hoped it would replace logs.

Blue-Green Deployment

Original lesson: Blue-Green Deployment on DesignGurus ↗

What it is. You keep two production environments that are exact copies of each other. **Blue** is live and handling users. **Green** sits idle. The new version goes onto green. You test it there, on genuine production infrastructure, while no users touch it. Then you flip the router, and within seconds all traffic is on green. To roll back, you make the same flip in the other direction.

> **Analogy.** Picture a theatre that has two matching stages. The audience is watching stage Blue. Meanwhile, the crew builds and rehearses the next scene in full on stage Green. When Green is ready, the seats swivel to face it. If anything goes wrong, swivel them back — Blue's set is still up.

The problems it solves

- **Rollback is slow.** Rolling back in place means rebuilding the old version and redeploying it through CI, artifact storage (where built, ready-to-deploy packages are kept) and rollout. Even when engineers know exactly what to do, the system may limp along for **15–25 minutes**. Rollback ends up being one more deployment.
- **The verification gap.** Staging is not production: its data, config, integrations and load all differ. With no second *production* environment, a release gets its first full test only once users are already on it.

How it works

1. **Deploy to green** while blue handles everyone. Green *is* production: same infrastructure, same config, same database. **Infrastructure-as-code** (describing environments in version-controlled files) keeps the two the same. That matters, because quiet **environment drift** would break the whole idea.
2. **Check green before any user reaches it.** Run **smoke tests** (fast checks that the basics work) against the real infrastructure. Wait for health/readiness to turn green. Warm up the caches and connection pools. This is deploy and release split apart by just a few seconds: the code is completely deployed, yet no user has been released onto it.
3. **Switch.** The router starts sending new traffic to green. You do this by changing the load balancer's target, or in Kubernetes by flipping a service's selector (the label rule that decides which pods a Service sends traffic to). Blue **drains**, meaning requests already underway finish on the environment where they began. User sessions carry over because they're kept in a shared store rather than on the servers (the earlier statelessness lessons set this up).
4. **Keep blue on standby** for an undo window, usually a few hours. Switching back *is* the rollback. It takes about **10 seconds**, with no CI and no build. Once everyone is confident, blue turns into the green for the next release.

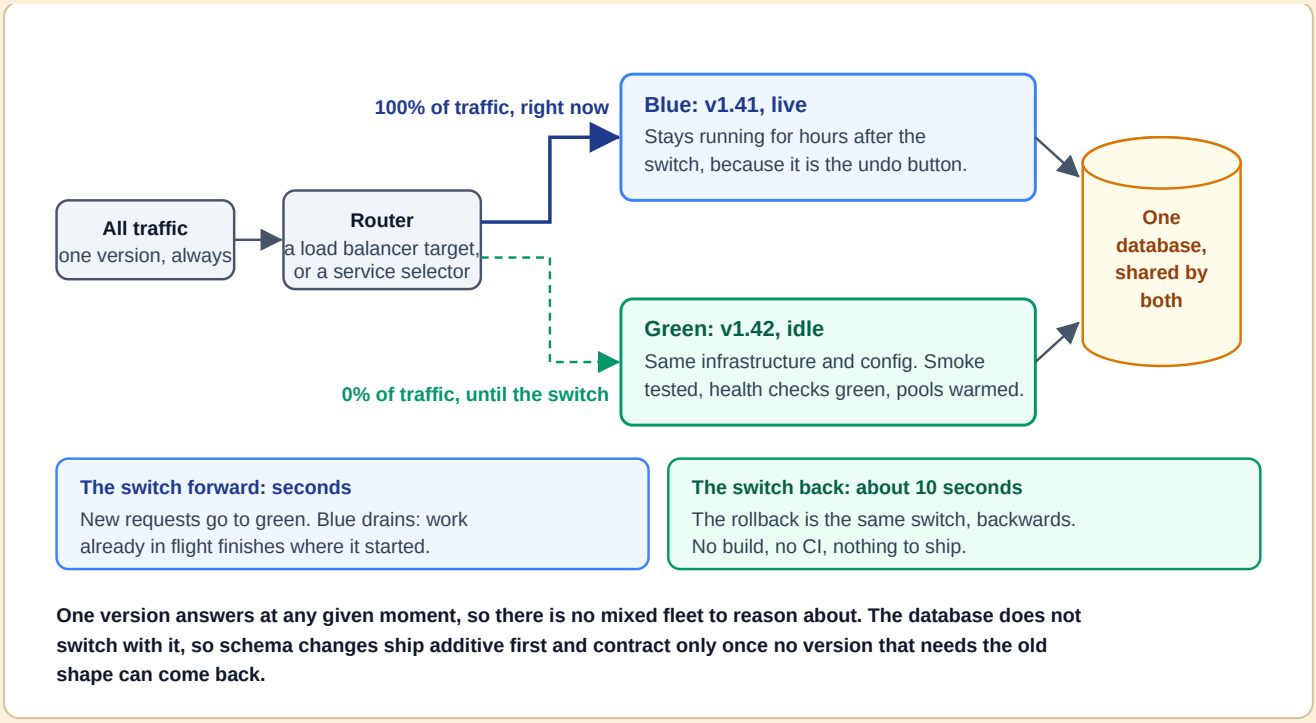


Figure — Blue-Green Deployment (course diagram)

In this figure, the router sends 100% of traffic to blue (v1.41, live) and 0% to green (v1.42, smoke-tested and warmed up) until the switch happens. After the switch, blue stays running for hours, because it *is* the undo button. Look at the one database in the middle that both sides share. It is the single component that does not switch.

The database does not blue-green

The two versions talk to the **same database**. For the whole rollback window, the old code and the new code both have to read and write the shared schema (the database's table and column layout) and data without breaking anything. That's why schema changes follow **expand-contract**:

- **Expand:** add the new structures in a way that doesn't break anything, write data in both the old and new forms, and make sure either version can read safely.
- **Contract:** delete the old structure only when *no* version that relies on it could ever return — that is, once the rollback window has closed.

This is the "additive first" rule from the API-versioning lesson, applied to data. Blue-green swaps application environments that hold no state. *State* needs its own compatibility discipline. The entire capstone is about this.

Blue-green vs. rolling deploy

A **rolling deploy** swaps out instances where they stand, a handful at a time.

	Rolling (replace in place)	Blue-Green
Extra capacity	About 1 instance's worth	2× for the duration of the window
Rollback	Redeploy: minutes	Switch back: seconds
Version mixing	v1 and v2 both serve for the whole rollout	Never — the switch is atomic
Blast radius at cutover	Grows gradually, instance by instance	100% at the moment of the switch
Fits	A cheap default; versions that are compatible with each other	When you need instant undo; changes that can't cope with mixed versions

Rolling deploys are cheap and every orchestrator supports them, which makes them a reasonable default. Blue-green justifies paying for double capacity when reversal speed matters or when the two versions

can't run side by side. Both decide who gets the new code by *where the infrastructure is*; the flags lesson will decide it by *who the user is*.

Trade-offs

You gain:

- **Rollback in seconds.** It's a routing change rather than a build, so reversing becomes an everyday move.
- **Checking the whole environment** before a single user touches the release.
- **No fleet running mixed versions.** Exactly one version serves at a time. (With a rolling deploy, v1.41 and v1.42 responses are interleaved, so the two must be able to live side by side safely.)

You pay:

- **2× capacity** while the release window lasts. In the cloud, you can spin up green for a few hours and then delete it. On-premises (in your own datacenter rather than the cloud), you may need hardware that sits idle all the time. Compare that against the cost of the outages it prevents.
- **All-or-nothing exposure.** The switch moves **100% of users at once**. If a bug slipped past verification, everyone hits it right away. (Canary solves this.)
- **Schema discipline as an ongoing cost.** Every migration has to remain expand-contract compatible, or the rollback button stops working.
- **Draining is tricky.** Long-lived connections (server-sent events, websockets — connections the server keeps open to push data to the client) keep blue busy until they close or move over.

When not to use it

- **When you need to validate step by step.** Blue-green checks the environment, not the variety of user behavior. A bug that only 3% of users trigger still reaches 100% once you switch — use a canary for that.
- **When 2× costs serious money.** Very large fleets often favor canaries, which reuse the capacity they already have. On-prem teams might keep blue-green for risky releases only.
- **When state is the main concern.** In a database-heavy system, swapping a thin app layer doesn't buy much, since the migration is where the real risk lives.
- **Common misuse:** flawless blue-green on the app servers, paired with a destructive schema rename or a dropped column. The router can send traffic back to blue, but blue can't read the altered data — so the "rollback" makes things worse.

In the interview

The prompt: "zero-downtime deployment with instant rollback". Be ready for a follow-up on the shared database — that is what determines whether your rollback really works.

> **30-second answer:** "Two identical environments. I deploy to the idle one and verify it against real infrastructure with zero users on it — gated on health and warmed up. Then I switch the load balancer. All traffic moves within seconds, and rollback is just switching back, so undo becomes a reflex instead of a 20-minute redeploy. Blue drains its in-flight requests and stays ready through the undo window. The database is what doesn't switch. Both versions share it, so schema changes follow expand-contract: additive first, write both forms, and contract only once rollback can no longer need the old shape. Skip that and the rollback button doesn't work. The honest limits: 2× capacity during the window, and the switch exposes 100% of users at once — gradual exposure is the canary's job."

Likely follow-ups (with answers):

- *"Walk me through the database during one blue-green cycle."* — Expand with an additive migration that works for both versions. Deploy and switch to green. Keep things compatible for the whole observation/rollback window. Contract only once blue can never come back. The rollback window and the schema-compatibility window have to last exactly as long as each other.
- *"What happens to sessions and in-flight requests at the switch?"* — Blue drains the requests it already has while green picks up the new ones. Sessions live in a shared store (thanks to the statelessness work), so nobody gets logged out.

Mistake that fails candidates: forgetting about the database, saying rollback is safe after green has written a format blue can't read, or picking blue-green when what's needed is gradual validation of behavior.

Self-check:

- **Q1.** What switches, what drains, what never switches, and what keeps it safe? → The router target switches (the stateless environments trade roles). Blue drains its in-flight requests while green takes the new ones. The shared database never switches; expand-contract protects it (additive changes, compatibility with both versions, and removal only after the rollback window).
- **Q2.** Team A runs in the cloud and deploys twice a day. Team B is on-prem at 60% utilization. What does blue-green cost each of them? → Team A spins up green for a few hours per release and then deletes it — a small cost next to even one avoided 20-minute outage. Team B would need hardware sitting idle permanently. So B might save blue-green for high-risk releases and use rolling deploys or feature flags for the rest, accepting a different kind of release risk.
- **Q3.** Green (v1.42) goes live at 14:00. At 14:30 you switch back to blue, and blue begins failing to read recent orders. → v1.42's migration renamed, removed or changed the type of a column, or it wrote data in an incompatible format. Blue can't read green's data, so the rollback itself caused an outage. Immediate fix: repair v1.42 and **roll forward** (blue is no longer compatible), then examine the data written 14:00–14:30. Prevention: expand-contract, with the contract phase held back until the rollback window has closed.

Related patterns:

- **Load Balancing** — the switch is a routing change, and draining is how it's carried out.
- **Health Check** — readiness tells you when green is genuinely warm.
- **Canary Deployment** — the gradual upgrade that avoids exposing 100% at once.
- **API Versioning** — expand-contract is its additive-first rule, applied to schemas.

TL;DR

- **Problem:** with in-place deploys, rollback becomes a ~20-minute redeploy, and the new version first meets real infrastructure only after users are on it.
- **Mechanism:** two identical environments. Deploy to the idle one and verify it; flip the router within seconds; drain the old one; keep it ready through the undo window; protect the shared DB with expand-contract.
- **Costs:** 2× capacity (all the time, if on-prem), 100% exposure at the switch, tricky draining, and schema discipline forever.
- **Skip it when:** you need gradual validation across varied traffic, 2× is serious money at your scale, or the risk sits in the data layer.

Canary Deployment

Original lesson: Canary Deployment on DesignGurus ↗

What it is. Start by releasing the new version to a small group of real users — 1%, for example — and compare its vital signs with the old version's *as they happen*. If the new version looks just as healthy, grow the group in steps (5%, 25%, 100%). A measured verdict must approve every step. If it looks worse, drain it within seconds. Only that small group was ever exposed.

> **Analogy.** The name comes straight from coal mines. Miners took a canary underground because the bird reacted to bad air before people did. Having the bird wasn't the point; *watching* it was. In the same way, a canary release is worthless without observation and an automatic response.

The problem: "it runs" is not "it behaves"

Version 1.43 runs through the complete blue-green process. Green passes health checks, production smoke tests and warm-up, and traffic switches at 14:00. Every environmental check is green.

At 14:08, support reports that checkout is failing for users with **a saved payment method *and* a non-US locale — 3% of traffic**. No test covered that pairing. Tests that had just one of the two conditions all passed. Because the switch moved everyone at once, every affected user hit the bug right away, and thousands of checkouts failed before the rollback.

The takeaway: green proved the version could *run*. It didn't prove the version *behaves* properly for real users. Production is full of combinations — payment state, locale, account age, device, usage — and they multiply faster than any test library can keep up. **Real user traffic is the only complete test suite that exists.** The goal is to use it without exposing everyone before a verdict is in.

Why the obvious fixes fail

- **"Improve the staging data."** That helps. But staging is always a partial copy of user behavior that keeps changing. It can't recreate every long-lived account state, device, payment-provider response and dependency condition all at once. Better data lowers the risks you know about; it can't prove there are no unknown interactions.
- **"Mirror (shadow) production traffic to green first."** Shadowing sends copies of live requests to the new version and discards whatever it answers. That's fine for read-only paths. It's unsafe for writes with side effects, like payments: replaying a charge might bill a customer twice, and blocking it skips the exact path you wanted to test. Thrown-away responses also produce no business outcomes (conversion, complaints). v1.43 broke in a write flow, which this technique can't reach.
- **"Deploy at 3 AM."** You get fewer failures per minute. But the same users hit the bug in the end, the team is exhausted, and light traffic gives *weaker* evidence — so regressions that affect a percentage of requests are spotted later. Picking a quiet time doesn't replace limiting exposure and measuring.

How it works

1. **The split.** Load-balancer weights, or a service mesh's traffic policy, route a controlled percentage (say 1%) to the new version. (Routing by cohort with feature flags is a close alternative.)
2. **Sticky slicing.** Every user stays on *one* version for their entire workflow. Bouncing between v1.42 and v1.43 on each request could break a multi-step checkout and muddy the comparison. Hashing the **user ID** (turning it into a number with a fixed formula, so the same ID always lands in the same place) onto a hash ring (as in consistent hashing) produces a stable 1% cohort. When the canary grows, new users are added and earlier ones don't move. Stable assignment also lets an engineer work out which version served a session someone reported.
3. **Compare against the baseline, not a fixed threshold.** The question isn't "is the canary's error rate under X?". It's "is the canary *worse than the baseline at this moment*?" The remaining 99% — the **concurrent baseline** — sees the same traffic, the same dependency conditions and the same cache state at the same instant. Yesterday's figures or fixed thresholds, by contrast, confuse normal daily swings with regressions. **Netflix Kayenta** runs this canary-vs-baseline analysis automatically over many metrics.

4. **Compare the right things.** Golden signals (errors, latency, saturation, traffic) **plus business metrics** (conversion — the share of visitors who complete a purchase — and add-to-cart rate) **plus traces**. A broken button might raise no errors and add no latency, yet stop people buying. Comparing traces can expose a new 200 ms span even while overall p99 still looks healthy. A gate can only be as good as the telemetry behind it. With no conversion metric, a release that is technically fine but commercially broken sails straight through.
5. **Gates and rollback run automatically.** A timed analysis moves each stage forward (1 → 5 → 25 → 100%) once its evidence window is over — or rejects it. No person stares at dashboards (people lose focus and don't apply statistics consistently). A failing score **drains the canary automatically in seconds**.
6. **Respect the statistics of small groups.** At 40,000 req/s, a 1% canary receives **400 req/s**. A bug affecting 0.1% of requests creates roughly **24 bad events per minute** — you'll catch it fast. Something affecting 0.01% of users could take hours, or need a bigger group. **Detection time depends on cohort size × effect frequency.** So work out how many events you expect *before* you pick percentages and windows. If you don't, a "clean" gate might only mean you had too few samples.
7. **The database rule still holds, and always will.** Canary and baseline share a single schema, so expand-contract is required. And unlike blue-green's atomic switch, the two versions **coexist during every rollout**. Old and new code must both read and write the same in-between data safely. Compatibility across mixed versions becomes a permanent trait of your delivery system.

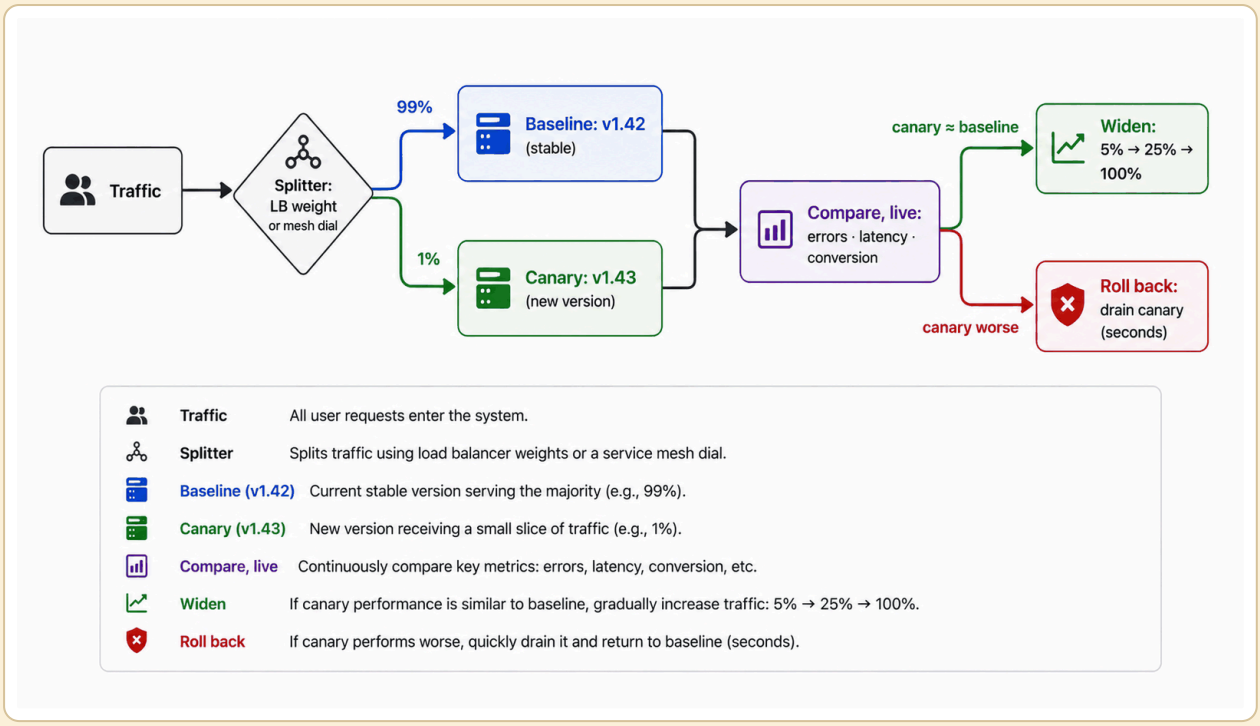


Figure — Canary Deployment (course diagram)

In this figure, traffic flows into a splitter (an LB weight or a mesh dial). The splitter sends 99% to the stable baseline (v1.42) and 1% to the canary (v1.43). Both sides feed a live comparison of errors, latency and conversion. If the canary ≈ baseline, the rollout grows 5% → 25% → 100%. If the canary is worse, it's drained within seconds.

Walkthrough with numbers

- **v1.43 run again, this time as a canary.** 14:00: a stable 1% cohort receives v1.43. 14:06: the automated analysis notices a **2.1%** error rate on the canary against **0.3%** on the baseline, and drains it. Roughly 1% of users were affected for six minutes, and idempotent clients (safe to retry without side effects) retried their failed checkouts without harm. Once fixed, a fresh canary begins at 15:30 and clears 1%, 5%, 25% and 100%.
- **A subtler rescue, two weeks on.** v1.44 shows no change in errors or latency. But in the canary cohort, checkout **conversion drops 8%**. A CSS (the stylesheet language that controls page layout) bug had hidden the payment button on small screens, and it raised no exceptions at all. The gate on the

business metric caught it and drained the canary. That's why technical metrics by themselves don't tell the whole story.

- **The bill.** A rollout now stretches over **hours** of observation windows, rather than a blue-green switch that takes seconds. What those hours buy is evidence — worth it for risky changes.

Canary vs. blue-green

	Blue-Green	Canary
Exposure	100% at the switch	1% → 5% → 25% → 100%, each step gated
Verifies	The environment (it runs, connects and serves)	How it behaves across the variety of real traffic
Rollback	Switch back: seconds	Drain the slice: seconds, with fewer people hurt
Version mixing	Not allowed (atomic)	Required (the mix *is* how it works)
Capacity	2× for the window	About 1% extra
Speed	Seconds	Hours

Experienced teams use both together. They build a green environment, then move traffic onto it through a canary ladder rather than in one atomic cut. That gives them environmental verification *and* evidence about behavior. **A/B testing** relies on similar routing but asks something else. A canary asks "does this release work?". An A/B test asks "which product option does better?", over a longer experiment.

Trade-offs

You gain:

- **Blast radius becomes a dial you set.** A bad release hits only the current percentage, for minutes — not everyone, for as long as a rollback takes.
- **Verification by the only complete test suite** — varied real traffic, with explicit verdicts.
- **Confidence that builds on itself.** Once teams trust canaries, they ship smaller deploys more often, and those are easier to diagnose and reverse.

You pay:

- **Time.** Each rollout takes hours (in an emergency, skipping the ladder can be justified).
- **Ongoing mixed-version compatibility.** Schemas, APIs and messages must work with the old and new versions at all times.
- **Discipline in statistics and telemetry.** Weak baselines, missing business metrics or unstable cohorts end up approving bad releases or blocking good ones.
- **Harder diagnosis.** Two versions run deliberately, so every metric, log and trace needs a label saying which version served it. Otherwise the canary's regression gets lost inside the far larger baseline totals.
- **Infrastructure.** Splitting traffic, keeping users sticky and running automated analysis require tools like a service mesh, **Argo Rollouts** or **Flagger**.

When not to use it

- **Low traffic.** 1% of 300 req/s is **3 req/s** — barely any signal. Go with blue-green, solid tests, or flags aimed at hand-picked users.
- **Versions that can't run side by side.** Use blue-green's atomic switch rather than creating a mix that isn't allowed.
- **Emergencies.** If production is already down, get it back up first; canary the *next* release.

- **Bugs that are too rare, or limited to one cohort,** for the slice's statistics to see (see self-check Q3). A canary is one layer of protection, not a guarantee.
- **"Canary theater":** a cron job (a task run on a fixed schedule) bumps the percentage in steps, with no automated analysis and no rollback. It resembles a canary but never produces a verdict — it's only a staged rollout.

Real-world examples

- **Netflix Kayenta** (used with Spinnaker) — scores the concurrent canary against the baseline across whole sets of metrics, automatically.
- **Argo Rollouts and Flagger** — provide Kubernetes rollout stages, analysis runs and automatic rollback, while the mesh handles the traffic split.
- **Google's percent rollouts** — the same idea at huge scale.
- **For AI engineers:** a *model* canary routes 1% of live requests to the new model and compares it with the current model over the same period. It looks at answer-quality evals (automated tests that score a model's answers), negative user feedback, refusal rate, cost per request and per-token latency. Routing through a mesh or LLM gateway does the split; automated evals deliver the verdict.

In the interview

Any question about a "risky release" expects a canary at the heart of the answer. Follow-ups dig into the gate metrics, the concurrent baseline, sticky routing and statistical limits.

> **30-second answer:** "I release to a sticky 1% cohort, hashed by user, so no one changes version halfway through a flow. I compare the canary with the concurrent baseline, never with yesterday — errors, latency and business metrics such as conversion, since the worst bugs don't throw errors. Automated analysis gates every step, 1-5-25-100, and a failing score drains the canary within seconds, so the blast radius is just the current percentage. It adds a permanent requirement: the two versions coexist during every rollout, so schemas and APIs have to stay compatible with the mix — expand-contract, always. And the limit: small slices can't statistically see very rare effects, so canaries sit alongside flags and tests; they don't replace them."

Likely follow-ups (with answers):

- *"Why compare with the baseline instead of a threshold?"* — Error and latency numbers rise and fall with the time of day and the mix of traffic. The concurrent 99% is living through the same conditions right now. Historical values and fixed thresholds aren't.
- *"How long do you stay at 1%?"* — Until the size of effect you care about would have shown up, given that cohort's request volume. That's minutes for big failures at high traffic, and hours (or a bigger slice) for subtle regressions. Detection time grows with cohort size × effect frequency.

Mistake that fails candidates: comparing with historical data, watching dashboards by hand, routing that isn't sticky, or running a canary on versions that can't coexist.

Self-check:

- **Q1.** Describe the split, the stickiness, what the verdict is measured against, and what happens when a gate fails. → LB or mesh weights send 1%. Hashing the user ID keeps cohort membership stable. Golden signals, business metrics and trace shapes are compared with the concurrent baseline. A failed gate drains the canary automatically, so only the current slice was ever exposed.

- **Q2.** What can a canary catch that blue-green, by design, can't — and the other way round? → A canary catches behavior bugs that only real traffic reveals (locale combinations, conversion drops). Blue-green gives an atomic switch with exactly one version serving, which you need when versions can't coexist. Pairing a green environment with staged traffic gets you both.
- **Q3.** v1.45 clears every gate all the way to 100%. Two days later, screen-reader users (~0.4% of traffic) can't check out. Why wasn't it caught, and what would catch it? → Inside a 1% canary they make up 0.4% × 1% = **0.004%** of traffic — far too few sessions. The failure raises no server error, and it shifts overall conversion by less than normal noise. To catch it: accessibility tests in CI (deterministic), a flag-targeted beta that includes that cohort on purpose, and conversion monitoring broken out for users of assistive technology.

Related patterns:

- **Blue-Green Deployment** — the environmental half; combine them as "stand up green, then shift traffic canary-style".
- **Feature Flags** — exposure decided by *picking* a cohort rather than by statistics; finer control within a release.
- **Service Mesh** — turns the traffic split into a one-line config.
- **Distributed Tracing** — supplies raw material for the verdict ("observe before changing").
- **Consistent Hashing** — cohort stickiness relies on the same ring.

TL;DR

- **Problem:** environment checks show a version runs, not that it behaves. Real traffic is the only complete test suite, and blue-green runs that suite on 100% of users at once.
- **Mechanism:** a sticky, hashed 1% cohort; verdicts from a live canary-vs-baseline comparison of golden signals and business metrics; automated 1-5-25-100 gates that drain on failure; permanent compatibility across mixed versions.
- **Costs:** hours for each rollout, expand-contract forever, rigor in statistics and telemetry, and rare cohorts too small for the slice to detect.
- **Skip it when:** traffic is too low, the versions can't coexist, or it's an emergency — and never put on canary theater.

Feature Flags

Original lesson: Feature Flags on DesignGurus ↗

What it is. A feature flag is a switch in the running code — `if flags.enabled("wishlist-sharing", user)` — and you can change its on/off rules *without a deploy*. The code goes out "dark" (it's there, but switched off). Later you **release** it by editing the rule: all at once, bit by bit by percentage, or only for groups of users you pick. To **kill** it, you flip it back. That takes seconds and touches only that one feature.

Analogy. A new wing is wired up while a building is being constructed, but its light switches are left off. On opening day, someone flips them on — for staff first, then for everyone. If a circuit sparks, you cut power to that wing alone, and the rest of the building stays lit.

The problem: deployment is doing release's job

Three teams share a single deployment pipeline, and marketing has already promised a launch date:

- **Timing.** A new *wishlist-sharing* feature has to go live **Tuesday 9:00 AM** on the web and on both mobile apps; the press release and influencer posts are already scheduled. The feature is risky, so it should go through a canary — but canaries widen over hours. Also, app-store review takes days, so

the mobile code has to ship early. There's no way deploy time and release time can be *the same* moment.

- **Integration.** The checkout refactor and the search fixes travel on the same **release trains** (scheduled deployments that bundle every team's changes). Half-built wishlist code can't be merged, because users would see it. So it sits on a **six-week feature branch** (a separate line of code changes kept apart from the shared main line until merged) while main changes every day. Merging it at the end takes **two days** of resolving conflicts.
- **Reversal.** At **11 AM Tuesday**, at peak traffic, wishlist swamps the database with an **N+1 query** (a separate query per item, rather than one query for all of them). The canary's low volume never surfaced it. The only way to turn it off is a rollback — and that would also strip out the checkout refactor and search fixes that shipped after it. **Killing one feature kills three.**

What ties these together: timing, audience and reversal all hinge on which binary (the compiled, deployable program) is running. Fact 1 says they can be pulled apart; flags finish that job.

Why the obvious fixes fail

- **"Keep half-finished work on branches."** Branches that live a long time drift away from main and produce enormous, painful merges (like the six-week branch above). **Trunk-based development** — everyone merges into main every day — prevents that. But then unfinished code lands in production, so it has to be deployed dark.
- **"Just deploy at 9 AM on Tuesday."** That turns deploys back into coordinated events (the opposite of what this module aims for). It also can't work across platforms: the iOS build went into app-store review the Thursday before, so no single deploy time can make web and the app stores go live together.
- **"Roll back to kill the feature."** A rollback undoes the entire release train, not only the broken feature. What's needed is an off switch for each *feature*, not for each *binary*.

How it works

1. **Evaluate locally and cheaply.** Flag checks live on **hot paths** (code that runs on every request). So the **SDK** — the flag library built into each service — checks *cached* rules inside the process in microseconds, with no network call per check. Rule changes **stream** out to instances over long-lived connections, much like the way a service mesh pushes out config (xDS).
2. **Fail static.** Settle ahead of time what happens if the flag service goes down. Instances carry on using the **last known rules**, and every flag has a **safe default**. Users notice nothing; operators just can't edit rules until the service is back. An outage of the flag service must never turn features off or bring the site down.
3. **Targeting gives each release a clearly defined audience.** Rules look at user attributes, so a release can move: employees → opted-in beta → 1% → 10% → 100%. A **cohort** is a named group of users. Targeting by name lets you include rare groups on purpose — like the screen-reader users who got lost inside the canary's 1% statistical sample. The split of duties: a canary asks "*is the binary safe?*"; flags ask "*who should get this feature, and when?*"
4. **Be clear about which type of flag you're making — each one needs its own lifecycle:**

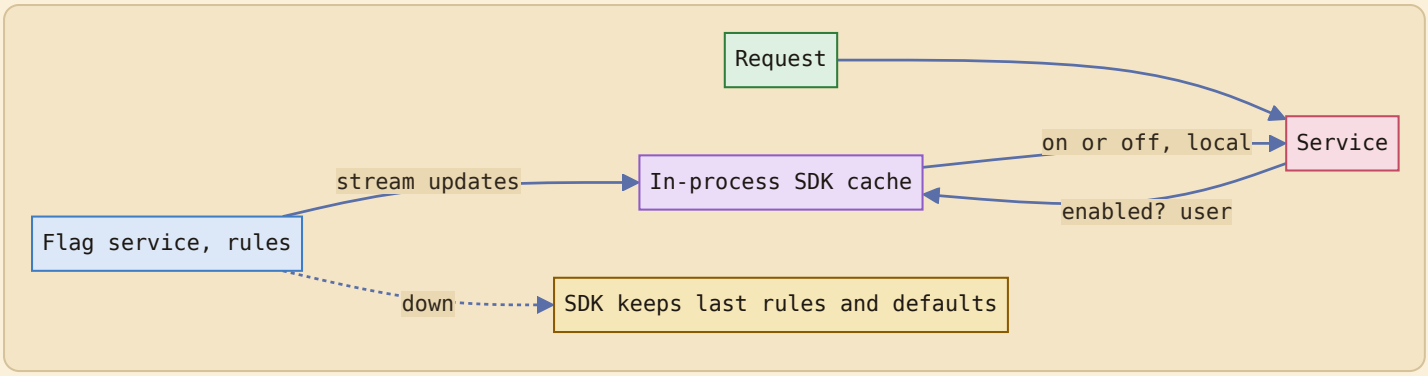
Type	Purpose	Lifecycle
Release flag	Ship dark, ramp up, launch	Temporary (weeks); has to be removed once at 100%
Ops kill-switch	Switch off costly or risky behavior when the system is under stress	Permanent, documented and tested on a regular schedule
Experiment flag	A/B test: which variant wins?	Lasts for the experiment's statistical window, then gets removed
Permission flag	Entitlements: plan tiers, enterprise features	Permanent — really product configuration rather than a flag

5. **Flag debt is the largest danger.** Each release flag that gets forgotten leaves behind a dead `else` branch that no one tests or owns. **Knight Capital** is the cautionary tale the whole industry cites. A

deployment reused an old flag name. The dead code path tied to that name still existed on **one server that missed the rollout**. That revived code traded out of control and lost **$460 million in 45 minutes**. How to prevent it: give every release flag an **owner and expiry date** when it's created. Plan cleanup as real, scheduled work — hitting 100% doesn't finish a flag; *deleting the* `if` does. And **never reuse a flag name**.

6. **The limit:** flags switch **code paths, not data shapes**. Schema changes still require expand-contract. Putting a flag in front of a destructive migration doesn't make that migration reversible.

Flag evaluation architecture with fail-static



In this diagram, the service asks its *local* SDK cache whether a flag is on for the current user. The flag service's only job is streaming rule updates into that cache. The dotted path shows fail-static: if the flag service goes down, the SDK holds on to the last rules and defaults, and requests carry on untouched.

Walkthrough with numbers: wishlist with flags

- **Weeks before launch:** small pieces of work merge into main every day, hidden behind the dark `wishlist-sharing` flag. This spreads the integration effort thin. Mobile builds clear app-store review with the feature switched off inside them.
- **Wednesday:** the flag is turned on for **500 employees**.
- **Thursday:** the flag is turned on for the opted-in beta, and the accessibility cohort is included on purpose. Now a screen-reader bug affects dozens of beta users rather than triggering thousands of support tickets.
- **Tuesday 9:00:** exposure climbs to 100% over **ten minutes** on web, iOS and Android at the same time. It's a rule change, not a deploy — every binary has carried the code for weeks.
- **11:04 AM:** the N+1 bug strikes at peak, and database load rises. An operator flips the kill-switch within **30 seconds**. Wishlist vanishes, checkout and search keep running, and the DB recovers. The fix is deployed Wednesday and ramped up again Thursday, and the flag is deleted the following sprint (a team's fixed work period, often two weeks).
- **Side by side:** without flags — a cross-platform 9 AM deploy that can't actually happen, an 11 AM rollback that pulls three features, and a six-week merge. With flags — three rule updates and one deleted `if`.

Trade-offs

You gain:

- **Deploy and release completely separated.** Marketing decides the exact timing, named cohorts decide the audience, and a single rule change reaches every platform.
- **A kill switch for each feature.** What you have to reverse shrinks from an entire release train to one feature.
- **Trunk-based development.** Half-finished work keeps integrating continuously while it stays dark.
- **The most precise control in progressive delivery**, working alongside canaries rather than replacing them.

You pay:

- **Flag debt.** Dead branches pile up and edge you toward another Knight Capital. The defense: owners, expiry dates, and "done" meaning deleted.
- **State combinations.** N flags produce **2^N** possible code paths. Keep N small. Test the combinations that are really live in production, plus each flag's default.
- **A new critical dependency.** Local evaluation and fail-static soften the impact, but a *bad rule* still spreads everywhere at the speed of config. Treat rule changes with the same rigor as deployments (the mesh lesson's warning about centralized control).
- **Runtime indirection.** Each check adds a branch (flags on hot paths must take microseconds). And to know which code handled a request, you now need to know its flag state. Record the evaluated flags as trace attributes.

The progressive delivery stack

Progressive delivery means releasing in measured stages and observing each stage before moving on. The three delivery patterns work together rather than competing:

Layer	Question it answers	Mechanism	Undo
Blue-Green	Is the environment healthy?	Switch from one environment to the other	Switch back: seconds
Canary	Is the binary safe with real traffic?	A statistical slice with measured verdicts	Drain: seconds
Flags	Should *this user* see *this feature* right now?	Rules at runtime, named cohorts	Flip: instant

Mature pipelines use all three. They create green, move traffic onto it through a canary ladder, then switch on the features inside it, each on its own timetable. The one-line way to tell them apart: **canaries select traffic statistically; flags select people by name**. That's why flags can include a rare cohort that a statistical sample would overlook.

When not to use it

- **Not for every change.** Flagging trivial changes "just to be safe" builds debt without adding real protection — canaries already handle those. Save flags for features that need their own release control.
- **Not as authorization.** Putting an admin panel behind a flag only affects whether it's *visible*. **AuthZ** (authorization) is what controls *access*. A flag SDK is not a permission system.
- **Not for long-running business logic branches.** Permanent switches for plan tiers or regional rules are product configuration. Model and name them as such (the "permission" category); the cleanup rules for release flags don't suit them.
- **Not wrapped around schema changes.** Expand-contract is what governs data.
- **The classic misuse: the flag nobody deleted.** Imagine 10,000 flags, three years old, whose owners have left and whose defaults nobody knows. Reading the code no longer tells anyone what production really does. Runtime control is both the benefit *and* the cost, so keep the number of flags small.

Real-world examples

- **LaunchDarkly, Unleash, OpenFeature** — streamed rules, local evaluation and targeting. OpenFeature is a newer vendor-neutral standard for flags (what OpenTelemetry is for tracing).
- **Facebook's Gatekeeper** — ran dark launches and widened them cohort by cohort to billions of users, before commercial flag products existed.
- **GitHub ships dark** — a feature can sit in production for months, visible only to staff, before customers get it.
- **Knight Capital** — the $460M lesson in flag hygiene: a reused name + dead code left in place + a deployment that didn't finish, all within 45 minutes.

- **For AI engineers:** flags are turning into the operating controls of LLM products. Prompt versions change through config with no deploy. New models go to named cohorts first, then get canary verdicts. Ops kill-switches handle fallbacks when a provider fails, turn off costly features like AI summaries at peak, and limit agent tools to approved cohorts (internal ones, say). Module 5's graceful-degradation sequence says *what* to shed; flags hand on-call engineers (the people paged when production breaks) the switches to shed it.

In the interview

The prompt could be "launch at a precise time on web and mobile", "pull one bad feature without rolling back the others", or "merge unfinished work continuously". Flags solve all three. Follow-ups dig into flag debt and what happens when the flag service goes down.

> **30-second answer:** "Feature flags split deploy from release entirely. Code ships dark with every release, trunk-based, so the pain of integrating is spread thin. Releasing is a runtime rule change — instant for marketing's 9 AM, gradual by percentage, or aimed at named cohorts like employees and beta users, which catches what canary statistics miss. Killing is a flip: one feature, thirty seconds, and nothing else rolled back. Three disciplines keep it safe. Local cached evaluation with fail-static defaults, so the flag service can't take the site down. A clear taxonomy: release flags are temporary and deleted after launch, kill-switches are permanent and tested. And an owner plus expiry date on every flag, because dead flags are how Knight Capital lost $460 million in 45 minutes."

Likely follow-ups (with answers):

- *"The flag service goes down — what does your app do?"* — Nothing users can see. SDKs evaluate rules cached locally, and each flag has a declared default (fail static, the same property the mesh has). Operators just can't change rules for a while. Never make a network call on each evaluation — a flag service on the hot path would make outages bigger.
- *"How do you keep flags from piling up forever?"* — A lifecycle policy. Require an owner and expiry at creation. Make deletion part of the definition of done. Send automatic reminders for expired flags. Use **lint rules** (automated code checks) that block reusing names. Schedule cleanups. And monitor the total number of flags and how old they are — a growing heap of old release flags means growing debt.

Mistake that fails candidates: using flags for authorization, making a network call per evaluation, having no plan for deletion, or claiming a flag makes a destructive schema migration safe.

Self-check:

- **Q1.** What are the four flag types, how long does each live, and which practice prevents the Knight Capital kind of failure? → Release (temporary; delete once at 100%), ops kill-switch (permanent, documented, tested regularly), experiment (lasts for the statistical window), permission (permanent product config). Prevention: deletion is the definition of done — remove the `if` and its dead branch — along with a ban on reusing flag names.
- **Q2.** Place the three delivery patterns on the deploy/release split and show how they fit together. → Blue-green splits them by seconds ("is the environment healthy?"; the switch is the release). Canary splits them by percentage ("is the binary safe with real traffic?"). Flags split them fully — in time, audience and platform ("should this user get it now?"). The pipeline: create green → canary traffic onto it to verify the binary → features inside stay dark until rules release them to cohorts, each on its own schedule.

- **Q3.** A new discount engine uses the flag `new-pricing`. That name was last used two years ago by a dead experiment whose `else` branch was never fully cleaned out. The deploy lands on 19 of 20 servers (one fails without any warning), and then the flag is turned on. → 19 servers run the new engine. The single stale server runs the old code, where `new-pricing=on` wakes up the two-year-old, half-deleted experiment. The load balancer sends ~5% of pricing requests to that server → prices that are intermittent, inconsistent and possibly expensive — exactly Knight Capital's recipe. Rules: (1) never reuse flag names (a registry remembers retired ones); (2) dead branches die along with their flags (delete the code, don't just archive the flag); (3) flips wait for the deploy to converge (block or warn while instances report different versions). Flags are code paths steered by config, so they need code review *and* config governance.

Related patterns:

- **Canary Deployment** — canaries for binaries, flags for features: statistical safety plus named audiences.
- **Graceful Degradation** — ops kill-switches are the manual levers for Module 5's degradation sequence.
- **Circuit Breaker** — the automatic twin of a kill-switch. Breakers respond to measurements; switches respond to people. Production needs both.

TL;DR

- **Problem:** deploys end up doing release's work — launch timing is chained to release trains, half-done work lives on drifting branches, and killing one feature means rolling back everybody's.
- **Mechanism:** runtime switches checked locally against streamed, cached rules with fail-static defaults. Deploy dark; release by flip, percentage or named cohort; kill any one feature in seconds. Kept in order by a four-type taxonomy and "done means deleted".
- **Costs:** flag debt (Knight Capital), 2^N state combinations, a config surface whose mistakes reach everyone, and code that no longer fully tells you what production does.
- **Skip it when:** the change carries no release risk (a canary is enough), the "flag" is really auth or product configuration, or the risk lives in the data layer (expand-contract).

Containers and Kubernetes

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. A **container** is an ordinary process that the Linux kernel (the core of the operating system, which controls hardware and processes) isolates and limits: it sees its own filesystem, network and process list, and it may use only the CPU and memory it was given. It starts from an **image** — a packaged, read-only filesystem with the app and everything it needs. **Kubernetes** is a system that runs containers across a fleet of machines and keeps them in the state you declared: "20 copies of checkout, version 4.2, each with 1 CPU and 1 GiB (gibibyte, about 1.07 billion bytes), reachable as `checkout`". When reality drifts (a machine dies, a process crashes), Kubernetes notices and corrects it.

Analogy. Shipping containers: a standard box lets any crane, ship or truck move any cargo without knowing what is inside. Kubernetes is the port authority — you file a manifest, and its cranes keep making the port match it.

The problem: 30 services, 440 hand-kept servers

The store runs ~30 services as **440 instances on 440 virtual machines** (VMs — software-emulated computers, each with its own operating system, sharing one physical host). A month of operations shows the cost:

- **"It works on my machine."** A service passed every test on build servers with one version of a TLS library and crashed in production, which had another. Every VM is a slightly different snowflake built by scripts over years.
- **Waste.** Each instance got its own VM "for isolation". Average CPU use across the fleet: **15%**. Most of the compute bill pays for idle machines.
- **Slow change.** Baking a new VM image for a deploy takes **40 minutes**; starting a VM takes minutes more. Every team wrote its own deploy script.
- **3 AM restarts.** When a process dies, a human gets paged to restart it — or it stays dead until someone notices.

The underlying issue: the unit of deployment is a *machine*, set up by imperative scripts. The unit should be the *application*, described once and placed wherever there is room.

Why the obvious fixes fail

- **"Better configuration-management scripts"** (Ansible, Chef, Puppet) reduce drift, but dependencies are still installed *onto* a machine instead of travelling *with* the app.
- **"Several services per VM"** — without isolation they fight over library versions and ports, and one memory leak takes its neighbours down (**noisy neighbour**).
- **"Smaller VMs"** — still a whole OS per instance, still minutes to boot.

How it works

Part 1 — what a container really is. Three Linux features, no magic:

1. **Namespaces — what the process can see.** A PID namespace gives it its own process list (it thinks it is process 1); a network namespace, its own network interfaces and ports; a mount namespace, its own filesystem view; others isolate hostname, users and inter-process communication.
2. **Control groups (cgroups) — what it may use.** Limits and accounting for CPU time, memory and I/O (input/output: disk and network reads and writes). Exceed the memory limit and the kernel kills the process (**OOMKilled**); exceed the CPU limit and it is **throttled** (paused until the next scheduling period).
3. **Layered images.** An image is a stack of read-only layers (base OS files, then the runtime — e.g. the Java or Python environment that runs your code — then libraries, then your app), plus metadata such as the start command. A running container adds a thin writable layer on top (**copy-on-write**). Layers are shared and cached: 20 services on the same Java base image store that layer once per machine. Images follow the **OCI** (Open Container Initiative) format, so any compliant runtime (containerd, CRI-O) can run them.

Because containers share the host's kernel, they start in well under a second and cost little memory beyond the app itself — much lighter than a VM, which boots its own kernel. The price: a kernel bug is shared, so isolation is weaker than a VM's (which is why serverless platforms run tenants inside microVMs; see the Scaling module's multi-region lesson).

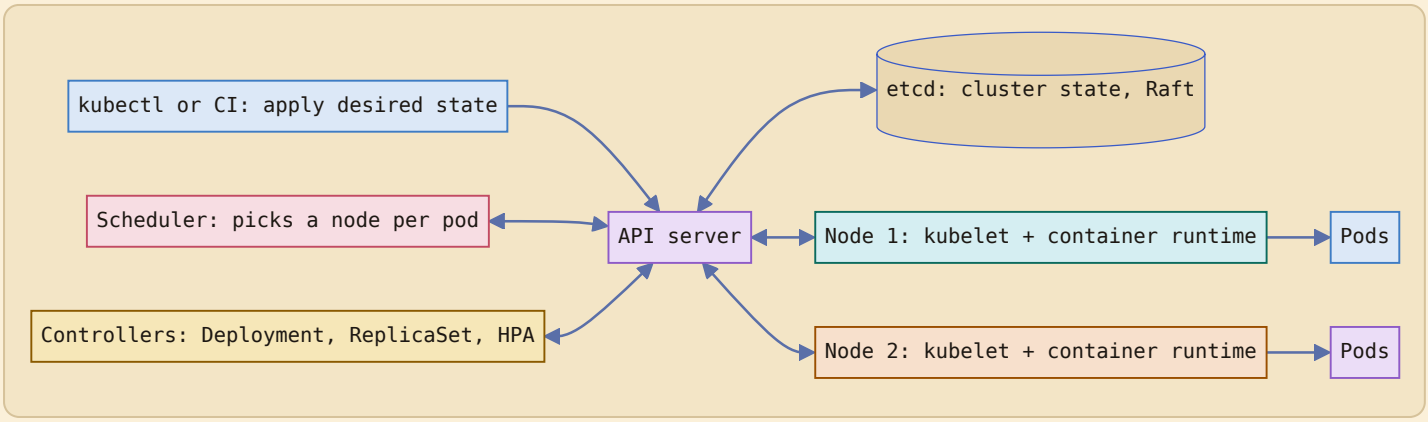
Image practices that matter:

- **Multi-stage builds** — compile in a large "builder" image, copy only the result into a small runtime image. Fewer packages mean a smaller attack surface (less code an attacker could exploit) and faster pulls.
- **Order layers for the cache** — dependencies first, source code last, so a code change rebuilds only the top layer.
- **Build once, deploy by digest** — the **digest** is the image's content hash. Tags like `latest` can be moved; a digest can't, so the image tested is the image deployed.
- **No secrets in images** (anyone who pulls the image can read every layer), run as a **non-root** user (not the all-powerful administrator account), one main process per container, logs to standard output (the default text stream a process prints to).
- The Spring book shows the Java side — layered jars and building images with buildpacks — in Faster startup and container images.

Part 2 — Kubernetes: desired state and control loops. You never tell Kubernetes "start a container on host 12". You submit a **desired state**, and **controllers** — control loops like the Auto-Scaling lesson's —

repeatedly compare it with the actual state and act to close the gap.

Kubernetes control plane and nodes



Everything goes through the **API server**, which stores the cluster's state in **etcd** (a small key-value store kept consistent with the Raft consensus algorithm). The **scheduler** assigns each new pod to a node with enough free resources. **Controllers** watch for differences between desired and actual state. On each node, the **kubelet** starts and stops containers through the container runtime and reports back.

The objects you use every day:

Object	What it is	Why it exists
Pod	One or more containers scheduled together, sharing a network address and volumes	The unit of scheduling; usually one app container, sometimes plus helpers
Deployment	"Keep N identical pods of this template running; roll out changes gradually"	Stateless services — the horizontal-scaling fleet
Service	A stable name and virtual IP in front of the pods matching a label	Pods come and go with new IPs; callers use the Service name (built-in discovery)
Ingress / Gateway API	HTTP routing from outside the cluster to Services	The cluster's L7 front door (behind your edge and API gateway)
ConfigMap / Secret	Configuration and credentials injected as files or environment variables (named settings a process reads at startup)	Same image in every environment; config varies
StatefulSet	Pods with stable names and their own persistent volumes (disk storage that survives pod restarts)	Databases, brokers — things that are not interchangeable
DaemonSet	One pod on every node	Log shippers, monitoring agents
Job / CronJob	Run to completion, once or on a schedule	Batch work, the nightly report
HorizontalPodAutoscaler	Changes a Deployment's replica count from a metric	The Auto-Scaling control loop, built in

Service types in one line: **ClusterIP** (reachable only inside the cluster — the default), **NodePort** (a port opened on every node), **LoadBalancer** (asks the cloud for an external load balancer), and **headless** (no virtual IP; DNS returns each pod, used by StatefulSets). The Spring book explains why a Kubernetes Service usually replaces a separate discovery server: Service discovery on Kubernetes.

Part 3 — the settings that decide whether it works in production.

- **Requests and limits.** A **request** is what the scheduler reserves for the container ("needs 0.5 CPU, 1 GiB"); a **limit** is the ceiling the kernel enforces. Memory limit exceeded → OOMKilled. CPU limit reached → throttled, which shows up as mysterious latency spikes. Many teams set memory limits equal to requests and leave CPU unlimited, or generous.

- **Probes.** **Readiness** decides whether the pod receives traffic; **liveness** decides whether it gets restarted; **startup** gives slow starters time before liveness begins. These are the Health Check Endpoint lesson's two verdicts, and the same rule applies: a liveness probe must never depend on a shared downstream, or one database blip restarts the whole fleet. Spring Boot's mapping is in Kubernetes probes.
- **Graceful shutdown.** On a rollout or scale-down, Kubernetes sends **SIGTERM** (the operating system's polite "please shut down" signal), removes the pod from the Service's endpoints, and waits up to `terminationGracePeriodSeconds` (default 30 s) before killing it. The endpoint removal and SIGTERM happen at nearly the same time, so an app should keep serving for a few seconds after SIGTERM (a short `preStop` sleep) and then drain in-flight work — the load balancer's *draining*, done per pod. See Graceful shutdown in Kubernetes.
- **Rolling updates.** A Deployment replaces pods in waves controlled by **maxSurge** (how many extra pods may exist during the rollout) and **maxUnavailable** (how many may be missing). A new pod counts only once its readiness probe passes, so a broken version stalls the rollout instead of replacing everything. (Blue-green and canary on Kubernetes are covered in the next lesson.)
- **PodDisruptionBudget** — "never take more than 2 of my 20 pods down at once for maintenance", respected when nodes are drained for upgrades.
- **Spread across failure domains** — topology spread constraints place replicas across nodes and availability zones, so one zone loss removes a third of the pods, not all of them.

Part 4 — design patterns for containers. Because a pod can hold several containers that share a network address and disk, some helper jobs become separate, reusable containers:

- **Sidecar** — a helper beside the app (proxy, log shipper, certificate refresher); the Sidecar lesson in the API module is this pattern.
- **Ambassador** — a local proxy that represents a remote service to the app (the app talks to `localhost`; the ambassador handles sharding, retries or TLS).
- **Adapter** — converts the app's output to a standard form (for example exposing its custom metrics in the format the monitoring system expects).
- **Init container** — runs to completion *before* the app starts (wait for a dependency, fetch config, run a check).
- Across pods: **leader election** (only one replica runs the scheduled job — the horizontal-scaling checklist's distributed lock), **work queue** (a Job's pods pull tasks), and **scatter-gather**.

Walkthrough with numbers: the 440 instances, containerised

- **Packing.** Each instance needs about **0.5 CPU and 1 GiB** at normal load. 440 × 0.5 = **220 CPUs**. A 16-CPU node has about 14 allocatable after system reservations → 220 / 14 ≈ **16 nodes**. Add room for a 25% rollout surge and one zone's worth of failure (N+1 at the zone level) → **~24 nodes** instead of 440 VMs. Utilisation rises from ~15% to ~60%.
- **Deploy time.** Image build with cached layers: **~3 minutes** (only the app layer changes). A rolling update of 20 replicas with maxSurge 5 and maxUnavailable 0 runs in 4 waves; each wave waits for readiness (~30 s) → **~2–3 minutes** to finish, with no capacity loss.
- **The 3 AM crash.** The kubelet restarts a crashed container within seconds. If a whole node dies, it is marked not-ready after ~40–50 s, and by default its pods are evicted and rescheduled once a **5-minute** toleration expires (tunable per pod — shorten it for stateless services). Nobody is paged unless restarts keep repeating (**CrashLoopBackOff** alerts).
- **The OOM trap.** A Java service with a 1 GiB memory limit is OOMKilled every few hours. Heap (the memory area where a Java program keeps its objects) was set to 1 GiB, but the JVM (Java Virtual Machine, the runtime that executes Java) also needs memory outside the heap (threads, metaspace, buffers). Fix: set the heap as a percentage of the container limit (for example `-XX:MaxRAMPercentage=75`) — modern JVMs read cgroup limits — and alert on restarts.

Trade-offs

You gain:

- **Identical artifacts from laptop to production** — the image carries its dependencies.

- **Density** — bin-packing many containers onto fewer machines (here ~440 VMs → ~24 nodes).
- **Self-healing and declarative operations** — restarts, rescheduling, rollouts and scaling handled by control loops, not scripts.
- **One platform vocabulary** for 30 teams: the same Deployment, Service, probe and rollout everywhere.

You pay:

- **A complex platform to run.** Upgrades every few months, networking plugins, certificates, etcd backups. Managed offerings (EKS, GKE, AKS) remove some of it, not all of it.
- **Many new ways to be wrong:** missing requests (overpacked nodes), tight CPU limits (throttling), deep liveness probes (restart storms), missing PodDisruptionBudgets (maintenance outages).
- **Weaker isolation than VMs** — the kernel is shared; untrusted multi-tenant code needs sandboxing (gVisor, microVMs) or separate nodes.
- **Stateful workloads are harder.** Databases on Kubernetes need StatefulSets, persistent volumes and usually an **operator** (a controller that encodes a DBA's — database administrator's — procedures). Many teams keep databases on managed services instead.

When not to use it

- **A few services and a small team.** A managed container service (AWS ECS or Fargate, Google Cloud Run, Azure Container Apps), a platform-as-a-service (a hosted platform where you hand over code and it runs it for you), or serverless functions (code the provider runs on demand, with no servers for you to manage) give most of the benefit without running a cluster.
- **"Kubernetes because everyone uses it."** Cloud-native anti-patterns include lifting old, stateful "pet" servers into containers unchanged (same fragility, more layers), a microservice per function so every request crosses ten network hops (a *distributed monolith*), and a cluster to serve a static site.
- **Workloads that need the whole machine** — very large databases or latency-critical systems tuned to specific hardware — often run better on dedicated VMs.

Real-world examples

- **Google Borg** ran nearly all of Google's work in containers for a decade before Kubernetes; Kubernetes was designed by some of the same engineers and applies Borg's lessons (labels, pods, declarative state), per "Borg, Omega, and Kubernetes" (2016).
- **Managed Kubernetes**: Amazon EKS, Google GKE, Azure AKS; the runtime underneath is usually **containerd**.
- **The Twelve-Factor App** — factors such as config in the environment, logs as event streams, and disposable processes are the habits that make an app container-friendly.
- **For AI engineers:** GPU nodes are scheduled as an extended resource (`nvidia.com/gpu: 1`), model weights are too large to bake into images comfortably (pull them from object storage or a cache volume at start), and slow model loading makes the **startup probe** and warm pools essential — the GPU Auto-Scaling lesson's boot-time problem, on Kubernetes.

In the interview

Prompts: "how would you deploy these services?", "what happens when a pod dies?", "explain requests vs limits", "why is my pod being OOMKilled?" Interviewers want the model (desired state and control loops), not a list of YAML (the indented text format Kubernetes configuration is written in) fields.

> **30-second answer:** "A container is a process isolated with namespaces and limited with cgroups, started from a layered, immutable image — so the artifact we test is the artifact we run. Kubernetes keeps a declared state true: a Deployment keeps N pods of an image running and rolls changes out in waves gated by readiness; a Service gives them a stable name; the HPA adjusts N; controllers reschedule pods when nodes die. The production details I'd call out are

> requests and limits — memory over limit is OOMKilled, CPU over limit is throttled — readiness vs liveness probes with liveness kept shallow, graceful SIGTERM handling with a short pre-stop delay, and spreading replicas across zones. For a handful of services I'd consider a managed container service before running a cluster."

Likely follow-ups (with answers):

- *"Container vs VM?"* — A VM runs its own kernel (strong isolation, slow boot, GBs of overhead); a container shares the host kernel (sub-second start, little overhead, weaker isolation). MicroVMs (Firecracker) sit in between.
- *"A pod restarts every few minutes with no error in the app log. What do you check?"* — `kubectl describe pod` for the last state: OOMKilled (raise the limit or cap the heap), liveness probe failures (probe too strict, or checking a dependency), or exit codes from a crash on start (CrashLoopBackOff).

Mistake that fails candidates: treating Kubernetes as a deploy script rather than a reconciliation system; liveness probes that call the database; no resource requests; saying containers are "lightweight VMs" without mentioning the shared kernel.

Self-check:

- **Q1.** Which kernel features make a container, and what does each do? → Namespaces (what the process sees: its own PIDs, network, mounts, hostname, users), cgroups (what it may use: CPU, memory, I/O, with OOM kills and throttling), and a layered copy-on-write image as its filesystem.
- **Q2.** A Deployment has 20 replicas, maxSurge 5, maxUnavailable 0, and the new version's readiness probe never passes. What happens? → Kubernetes starts 5 new pods; none become ready, so no old pods are removed and the rollout stalls with all 20 old pods still serving. The rollout reports it isn't progressing; roll back or fix. Readiness-gated rollouts turn a bad version into a stuck deploy, not an outage.
- **Q3.** After a database blip, all 60 checkout pods restart at once and the site is down for 4 minutes. Explain. → The liveness probe checked the database, so every pod failed it and restarted together, cold. Liveness tests only the process; dependency status belongs in readiness (or degraded responses).

Related patterns:

- **Health Check Endpoint** — readiness and liveness probes are its two verdicts.
- **Auto-Scaling** — the HPA and cluster autoscaler are the control loop, applied to pods and nodes.
- **Sidecar / Service Mesh** — pod-level helper containers, and the mesh that configures them.
- **Blue-Green and Canary Deployment** — built from Deployments, Services and traffic weights (next lesson).

TL;DR

- **Problem:** machines as the unit of deployment — snowflake servers, 15% utilisation, 40-minute image bakes and humans restarting crashed processes.
- **Mechanism:** containers (namespaces + cgroups + layered immutable images) as the artifact; Kubernetes control loops keep declared state true — Deployments, Services, probes, rolling updates, autoscaling, rescheduling.
- **Costs:** a platform to operate, new misconfiguration traps (limits, probes, disruption budgets), shared-kernel isolation, and harder stateful workloads.

- **Skip it when:** a handful of services fits a managed container service or serverless, or the workload needs a whole tuned machine.

Sources: Linux man pages `namespaces(7)` and `cgroups(7)`; Open Container Initiative image and runtime specifications; Kubernetes documentation (concepts: Pods, Deployments, Services, probes, resource management, disruptions); A. Verma et al., "Large-scale cluster management at Google with Borg", EuroSys 2015; B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes, "Borg, Omega, and Kubernetes", *ACM Queue* 2016; B. Burns and D. Oppenheimer, "Design Patterns for Container-based Distributed Systems", HotCloud 2016; Docker documentation, "Building best practices"; A. Wiggins, *The Twelve-Factor App*.

CI/CD Pipelines and Deployment Strategies

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. **Continuous integration (CI)** means every engineer merges small changes into the main branch at least daily, and every merge is automatically built and tested. **Continuous delivery (CD)** means every change that passes the pipeline *could* go to production at the push of a button; **continuous deployment** goes one step further and sends it there automatically. The **pipeline** is the automated path from a commit to production, written as code. A **deployment strategy** is how the new version replaces the old one on the servers — recreate, rolling, blue-green, canary or shadow. This lesson ties the module's delivery patterns into one path.

> **Analogy.** A car factory's assembly line. Each station does one check (frame, paint, brakes), a car that fails a station never reaches the next, and every car is built the same way. Nobody hand-builds a car for the showroom. The final road test happens on a quiet road before the car joins traffic — that's the canary.

The problem: the monthly release

The store's platform team of 40 engineers ships once a month:

- **Big batches.** Each release carries **~400 commits**. When it breaks, finding the guilty change means bisecting (repeatedly testing the halfway point to narrow down) 400 commits under incident pressure.
- **Merge week.** Long-lived branches take **two days** to merge before each release; bugs appear in the merge itself.
- **Different builds per environment.** Staging and production each build their own binary from the same tag, with slightly different dependency versions resolved on different days. "It passed in staging" proves little.
- **Hand-made infrastructure.** Load balancers and databases were created by clicking in the cloud console. Nobody can say what production's configuration is, and staging doesn't match it.
- **Slow fixes.** A one-line hotfix (an urgent small fix shipped outside the normal schedule) takes **6 hours**: manual approvals, a manual QA (quality assurance — people testing by hand) pass and a manual deploy runbook.

Each safeguard (more QA, fewer releases) made batches *bigger*, and bigger batches made each release riskier.

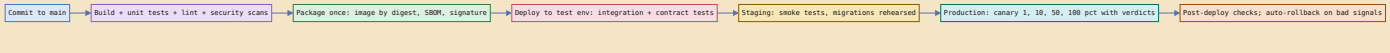
Why the obvious fixes fail

- **"Release less often to be safer."** Fewer releases mean more changes per release, harder diagnosis and longer rollbacks. Risk grows with batch size.
- **"More manual QA."** It slows every change and still misses what only real traffic reveals. Manual checks don't scale with 120 merges a day.
- **"A code freeze before big events."** Useful for a few days around Black Friday, but as a habit it just moves the batch problem to after the freeze.

How it works

1. The pipeline — build once, promote the same artifact.

A pipeline from commit to production, one artifact all the way



- **Commit stage (minutes):** compile, unit tests (fast tests of one small piece of code in isolation), linters (tools that flag suspicious or badly styled code), static security analysis, dependency vulnerability scans. Keep it **under ~10 minutes** so engineers wait for it instead of context-switching.
- **Package once:** the output is an immutable artifact — a container image identified by its **digest** — plus a **software bill of materials** (SBOM, the list of every component inside) and a signature. Every later stage deploys *this exact artifact*. Environments differ only in configuration (Twelve-Factor's "build, release, run" separation).
- **Test environments:** integration tests against real dependencies, and **contract tests** that check each service still honours the API its consumers expect (see the Spring book's contract testing section).
- **Production rollout:** a progressive strategy (below) with automatic verdicts from the golden signals, then post-deploy checks. A failed verdict triggers rollback without a human.
- **Test pyramid:** many fast unit tests, fewer integration tests, very few slow end-to-end tests. **Flaky tests** (tests that sometimes fail on unchanged code) are quarantined and fixed — a pipeline people learn to ignore protects nothing.

2. Choose the deployment strategy per change.

Strategy	How the new version replaces the old	Downtime	Rollback	Extra capacity	Users exposed to a bad version
Recreate	Stop all old, start all new	Yes	Redeploy old (minutes)	None	100%
Rolling	Replace instances in waves (Kubernetes default)	No	Roll back in waves (minutes)	A small surge	Grows wave by wave
Blue-green	Start a full new environment, switch traffic at once	No	Switch back (seconds)	2× during the switch	100%, briefly
Canary	Send a small slice of traffic to the new version, widen on good verdicts	No	Drain the slice (seconds)	Small	1–10% until trusted
Shadow (mirroring)	Copy real requests to the new version; discard its answers	No	Nothing to undo	Up to 2×	0% — users only see the old version
Feature flag	Code deployed dark; behaviour released by rule	No	Flip (instant)	None	Any chosen cohort

A/B testing is sometimes listed here, but it answers a product question ("which variant converts better?") rather than a safety question; it is usually run with experiment flags. Details of three rows live in this module: Blue-Green Deployment, Canary Deployment and Feature Flags. On Kubernetes, rolling is built in; blue-green is two Deployments with a Service switched between them; canary uses replica ratios or weighted routing in an ingress, gateway or mesh, and tools such as Argo Rollouts or Flagger automate the verdicts.

Every strategy has the same hidden requirement: **old and new versions must work with the same database and message formats at the same time**, because during any rollout both are running. Schema changes therefore follow expand-contract, as the capstone below shows. Shadow traffic must not cause side effects — a mirrored "place order" must never charge a card, so writes are stubbed or sent to a sandbox.

3. Infrastructure as code (IaC). Servers, networks, load balancers and databases are described in files kept in version control, reviewed like code and applied by the pipeline.

- **Terraform** (and similar tools) is *declarative*: you describe the resources you want, `plan` shows the difference from what exists, and `apply` makes the changes. It records what it manages in a **state file**, which must be stored remotely with locking so two runs can't collide.
- **Drift** — someone changed production by hand — shows up as an unexpected diff in the next plan.
- Reviewing the *plan* in the pull request (a proposed change submitted for review before it is merged) means reviewers see "this will delete a database", not just a code diff.

4. GitOps — the cluster pulls its desired state. GitOps applies the Kubernetes control-loop idea to deployment itself. The desired state of each environment (which image digest, which config) lives in a Git repository (a version-controlled folder that records every change). An agent inside the cluster (Argo CD, Flux) continuously compares the cluster with Git and fixes any difference. Deploying is a reviewed commit that changes a digest; rolling back is reverting that commit. The CI system never needs credentials to production — the cluster pulls instead of being pushed to, which removes a powerful secret from the pipeline.

5. Secrets in pipelines. Avoid long-lived cloud keys stored in the CI system. Modern CI can prove its identity to the cloud with short-lived **OIDC** (OpenID Connect, a standard way to prove identity with signed tokens) tokens (the cloud trusts "this workflow in this repository on this branch"), and apps read secrets from a secrets manager at runtime (see Storing keys and secrets).

6. Measure delivery with the four DORA metrics. The *Accelerate* research (Forsgren, Humble and Kim) found four measures that separate strong delivery teams from weak ones: **deployment frequency**, **lead time for changes** (commit to production), **change failure rate** (share of deploys causing a failure) and **time to restore service**. The key finding is that speed and stability rise *together*: small, frequent changes are easier to test, diagnose and undo.

Vocabulary interviewers mix in (one line each)

- **DevOps** — a culture: the team that builds a service also runs it, with shared tooling and automation. **SRE** (Site Reliability Engineering) — Google's practice of running operations as software engineering, with reliability targets (SLOs — service-level objectives, e.g. "99.9% of requests succeed") and error budgets (the amount of failure the SLO still allows; once spent, releases slow down) deciding how fast to ship. **Platform engineering** — a team that builds the internal "paved road" (pipelines, clusters, templates) other teams use as a self-service product.
- **Monorepo vs polyrepo** — one repository for many services (atomic cross-service changes, one toolchain, needs build tooling that only rebuilds what changed) versus a repository per service (clear ownership and independent pipelines, but cross-service changes need coordinated releases).
- **How Git stores history** — every file version (blob), directory (tree) and commit is stored under the hash of its content; a branch is just a movable pointer to a commit. **Merge** joins two histories with a merge commit and keeps them as they happened; **rebase** replays your commits on top of the target, giving a straight history but rewriting commit IDs — so never rebase commits others have already pulled.

Walkthrough with numbers: the monthly release, replaced

- **Flow.** 40 engineers merge ~**120 small changes a day** to main. Each merge runs an **8-minute** commit stage and a **12-minute** integration stage.
- **Production.** Changes deploy in small batches (often 1–3 changes per deploy). The canary runs at 1% → 10% → 50% → 100% with a **10-minute bake** at each step and automatic verdicts → **~45 minutes** from merge to full traffic, with no human in the loop for routine changes.
- **Hotfix.** The same path, with the bake times shortened: **~25 minutes** instead of 6 hours — because the path is always exercised, it is trusted.
- **A bad change.** It fails its canary verdict at 1%: about **1% of requests for ~10 minutes**, rolled back automatically, and the culprit is one of ~2 commits, not one of 400.

- **Infrastructure.** The console-made load balancers are imported into Terraform; the first `plan` reveals **17 differences** between staging and production, which is exactly the drift that made "passed in staging" meaningless.

Trade-offs

You gain:

- **Small batches** — each deploy is easy to understand, test and undo; failures point at a few commits.
- **One trusted path** for features, fixes and infrastructure alike — the emergency path is the everyday path.
- **Environments that match**, because they come from the same artifact and the same code.
- **Speed and stability together**, as the DORA research shows.

You pay:

- **Investment in automation** — test suites, pipelines and rollout tooling are real software to build and maintain.
- **Discipline** — trunk-based development, backward-compatible changes, expand-contract migrations and flag hygiene, on every change.
- **Pipeline security** — the pipeline can change production, so it is a prime attack target: protect its credentials, pin and verify dependencies, sign artifacts.
- **Flaky tests and slow stages** erode trust; they need owners and budgets.

When not to use it

- **Continuous *deployment* where releases can't be undone quickly** — mobile apps (store review, users on old versions), firmware, and some regulated systems use continuous *delivery* (always releasable, human-approved release) plus flags.
- **Shadow traffic for writes with real side effects** unless they are stubbed.
- **Canary for tiny traffic** — at a few requests per minute a 1% slice gives no statistical verdict; use blue-green with smoke tests, or flags for named cohorts.

Real-world examples

- **Amazon** reported (Velocity 2011) that in May 2011 the mean time between production deployments on a weekday was 11.6 seconds — many small, automated, independent deployments rather than one big release.
- **Google's SRE book** describes release engineering with hermetic builds (the same inputs always give the same output) and progressive rollouts.
- **Netflix's Spinnaker** (open source) with **Kayenta** automated canary analysis (built with Google) compares canary and baseline metrics statistically and decides automatically.
- **Pipelines:** GitHub Actions, GitLab CI, Jenkins; **GitOps:** Argo CD and Flux; **IaC:** Terraform/OpenTofu, Pulumi, AWS CloudFormation.
- **For AI engineers:** a model or prompt change goes through the same pipeline with an extra gate — an offline **evaluation suite** (golden questions, LLM-judge scores) must pass before the canary, and the canary's verdict includes quality and cost metrics, not only errors and latency.

In the interview

Prompts: "walk me through how code gets to production", "how would you deploy this with zero downtime?", "how do you roll back a bad release?", or "compare blue-green and canary".

> **30-second answer:** "Trunk-based development with CI on every merge: a commit stage under ten minutes, then one immutable, signed image built once and promoted unchanged through test,

> staging and production — only configuration differs. Infrastructure is Terraform, reviewed as plans in pull requests, and environments are reconciled GitOps-style so the cluster pulls its desired state. In production I'd use a canary with automatic verdicts on golden signals and auto-rollback, blue-green where I need an instant full switch, and feature flags to separate release from deploy. Every rollout assumes old and new versions run together, so schema changes are expand-contract. I'd track the four DORA metrics to see whether it's working."

Likely follow-ups (with answers):

- *"Why build once instead of per environment?"* — So the thing you tested is bit-for-bit the thing you run. Rebuilding can pull different dependency versions or compiler output; environment differences belong in configuration, not the binary.
- *"Rollback isn't possible because the new version already wrote data in a new format. What went wrong?"* — The change wasn't backward compatible. Old and new must read each other's data during any rollout: expand first (both formats readable), deploy, and contract only once the old version is gone — the capstone's sequence.

Mistake that fails candidates: rebuilding per environment; `latest` tags in production; a strategy with no rollback story; ignoring database compatibility during rollouts; "we test in staging" as the only safety net.

Self-check:

- **Q1.** Continuous integration, continuous delivery, continuous deployment — define each. → CI: merge small changes to main at least daily, each automatically built and tested. Continuous delivery: every passing change is releasable, and release is a button. Continuous deployment: every passing change goes to production automatically.
- **Q2.** Pick a strategy: (a) a payment service with high traffic, (b) an internal tool with 20 requests an hour, (c) a new search ranking you want to compare against the old one without affecting users. → (a) Canary with automatic verdicts (plus flags for risky features). (b) Blue-green or rolling with smoke tests — a 1% canary would see almost no traffic. (c) Shadow traffic: mirror real queries to the new ranker and compare offline, then canary or A/B test.
- **Q3.** Two engineers ran `terraform apply` at the same time and a load balancer was deleted. What protection was missing, and how does GitOps change who can deploy? → Remote state with locking (one apply at a time) and applies run only by the pipeline from reviewed plans. With GitOps, the cluster's agent applies what's in Git, so the path to production is a reviewed commit rather than someone's credentials.

Related patterns:

- **Blue-Green, Canary, Feature Flags** — the strategies this pipeline chooses between.
- **Health Check Endpoint** — readiness gates every wave of a rollout.
- **Distributed Tracing** — the evidence behind a canary verdict when something is slow.
- **The capstone (next)** — expand-contract keeps rollback possible when data changes.

TL;DR

- **Problem:** monthly releases of ~400 commits, per-environment builds, hand-made infrastructure and 6-hour hotfixes — each safeguard made batches bigger and riskier.
- **Mechanism:** trunk-based CI; one immutable signed artifact promoted through environments; infrastructure as code and GitOps reconciliation; a deployment strategy per change (rolling, blue-green, canary, shadow, flags) with automatic verdicts and rollback; DORA metrics to measure it.

- **Costs:** automation to build and maintain, backward-compatibility discipline on every change, a pipeline that must be secured, and flaky tests to police.
- **Skip it when:** continuous *deployment* can't be undone quickly (mobile, firmware, some regulated systems) — use continuous delivery plus flags instead.

Sources: J. Humble and D. Farley, *Continuous Delivery* (2010); N. Forsgren, J. Humble, G. Kim, *Accelerate* (2018) and the DORA *State of DevOps* reports; B. Beyer et al. (eds.), *Site Reliability Engineering*, O'Reilly 2016, ch. 8 "Release Engineering"; J. Jenkins, "Velocity Culture", O'Reilly Velocity 2011 (Amazon's deployment rate); Netflix Tech Blog and Google Cloud Blog, "Introducing Kayenta" (2018); HashiCorp Terraform documentation (plan/apply, state and locking); OpenGitOps principles (CNCF); Kubernetes documentation (Deployment strategies); *The Twelve-Factor App* (build, release, run); *Pro Git* (S. Chacon, B. Straub), ch. 10 "Git Internals".

Capstone - The Zero-Downtime Migration

Original lesson: Capstone - The Zero-Downtime Migration on DesignGurus ↗

What this capstone is. A step-by-step look at changing the way a busy production database stores money — with zero downtime and a safe way back at every stage. Migrations are frequently riskier than feature releases. They alter *stored data*, and a router can't just switch stored data back. This exercise puts the module's two rules to work: keep deploy apart from release, and keep rollback an everyday move at every stage. Where a stage at first seems impossible to roll back, you redesign that stage.

> **Analogy.** Think of relocating a busy restaurant's kitchen to a different room without ever stopping dinner service. First you build the new kitchen, and nobody cooks in it yet. Next you prepare every dish in both kitchens and taste them side by side. Then you slowly start serving from the new one. Only weeks later, once you're sure, do you tear out the old kitchen.

Each time you stop to make a decision, ask one question: **what is the undo button right now, and what does pressing it cost?**

The brief

- An audit discovers the orders table keeps money in `total FLOAT` (floating point — binary fractions that can't hold cents exactly). Rounding drift has pushed totals away from the payment provider's records. Finance wants `total_cents` **as an integer**.
- The table receives **400 writes/second**, nonstop, and there's **no maintenance window**.
- Every service that reads orders has to keep working. There are **12 consumers**, among them **CQRS projections** (read-side copies of the data shaped for queries) and reporting systems.
- This is money, so the new format needs **proof** before anything relies on it. Every stage has to stay reversible until that proof is in hand.
- The blue-green warning holds the whole way through: **the database does not blue-green**. The entire project is expand-contract, done with care.

Stage 1 — Expand: the change that changes nothing

Decision: add `total_cents BIGINT NULL` (a large-integer column that may be empty). It's nullable, nothing writes to it and nothing reads it. The application behaves exactly as before — being invisible is what makes this the expand phase. **Undo:** drop the column. It costs nothing, because no code refers to it.

Why give it the full delivery process anyway? Even this "trivial" migration goes through a canary and is watched on readiness and golden signals. An `ALTER TABLE` (the SQL command that changes a table's structure) can take **locks** (holds that block other reads or writes to the table) you didn't expect, or run past a migration tool's timeout. Schema changes that look harmless still cause 3 AM incidents. The acceptance test: signals stay healthy and no user notices anything.

Stage 2 — Dual-write, behind a flag, plus a backfill

Decision 1 — the dual-write code ships dark and a flag releases it. New code that writes both `total` and `total_cents` for every order goes out switched off. A canary checks the *binary*. Then a separate flag change switches the *behavior* on. Why a flag instead of deploying it already on? Dual-writing could add latency, clash with a database trigger (code the database runs automatically when a row changes), or expose an N+1 problem that only shows up at peak. With a flag, undoing it is a **30-second** flip rather than a full rollback. Users don't see this change, so no cohort ramp is needed — the goal is simply independent control.

Decision 2 — backfill (fill the new column in for rows that already exist) 200 million historical rows using `total_cents = round(total × 100)`. What makes it safe to run, and to run again, comes from the Module 5 reliability patterns:

- **Idempotent** (safe to repeat): every write sets a value that is always the same for the same row, so a failed batch can just run again without damaging data.
- **Rate-limited as a bulkhead**: the job is slowed down and kept separate, so production requests always come first when both want the database.
- **Checkpointed**: lasting progress markers (the write-ahead-log habit) let it pick up from where it stopped.

Undo: switch the flag off and halt the job. No data is lost, because the old `total` column is still the source of truth.

Stage 3 — Verify: the stage everyone skips

At this point both columns are filled in, and the code *could* begin reading `total_cents` tomorrow. It doesn't. First you need proof across 200 million rows plus 400 writes/s:

- **A comparison job** keeps counting rows where `total_cents ≠ round(total × 100)`. It turns up **several thousand** mismatches. They came from an old code path that never did the dual-write (legacy paths often linger unnoticed). Engineers fix that path and backfill its rows again. Then they wait until the mismatch count reaches **zero and stays there for days**.
- **Shadow reads**: a read path behind its own flag works out the value from *both* columns but hands back the old one. It logs each mismatch along with its **trace ID**, so every discrepancy can be looked into. Production traffic turns into a test that never stops — the canary idea, applied to data.

Undo: nothing to undo, since users see no change. This stage creates the *evidence* the next stage depends on. Skip it, and a money migration rests on an assumption nobody tested.

Stage 4 — Switch reads with a flag ramp and verdicts

Decision: shift reads from `total` to `total_cents` using a flag ramp — internal users → 1% → 10% → 100% — with canary-style verdicts. Compare flagged users with baseline users on golden signals **plus business metrics** like disputes over order totals, odd refund patterns and checkout conversion. (Customers may complain about a one-cent difference before any error rate budges.)

Why is rollback instant and safe here, when blue-green's rollback went so badly? Because dual-writing is still running, **both columns stay current**. Pointing reads back at `total` sends them to a source that's been kept up to date the whole time. In the blue-green failure, the old version couldn't read data written after the switch. Here, neither format ever falls behind. So the rollback window can stay open for as long as verification needs. **Undo:** flip the flag — seconds.

Stage 5 — Contract: the one-way door, entered deliberately

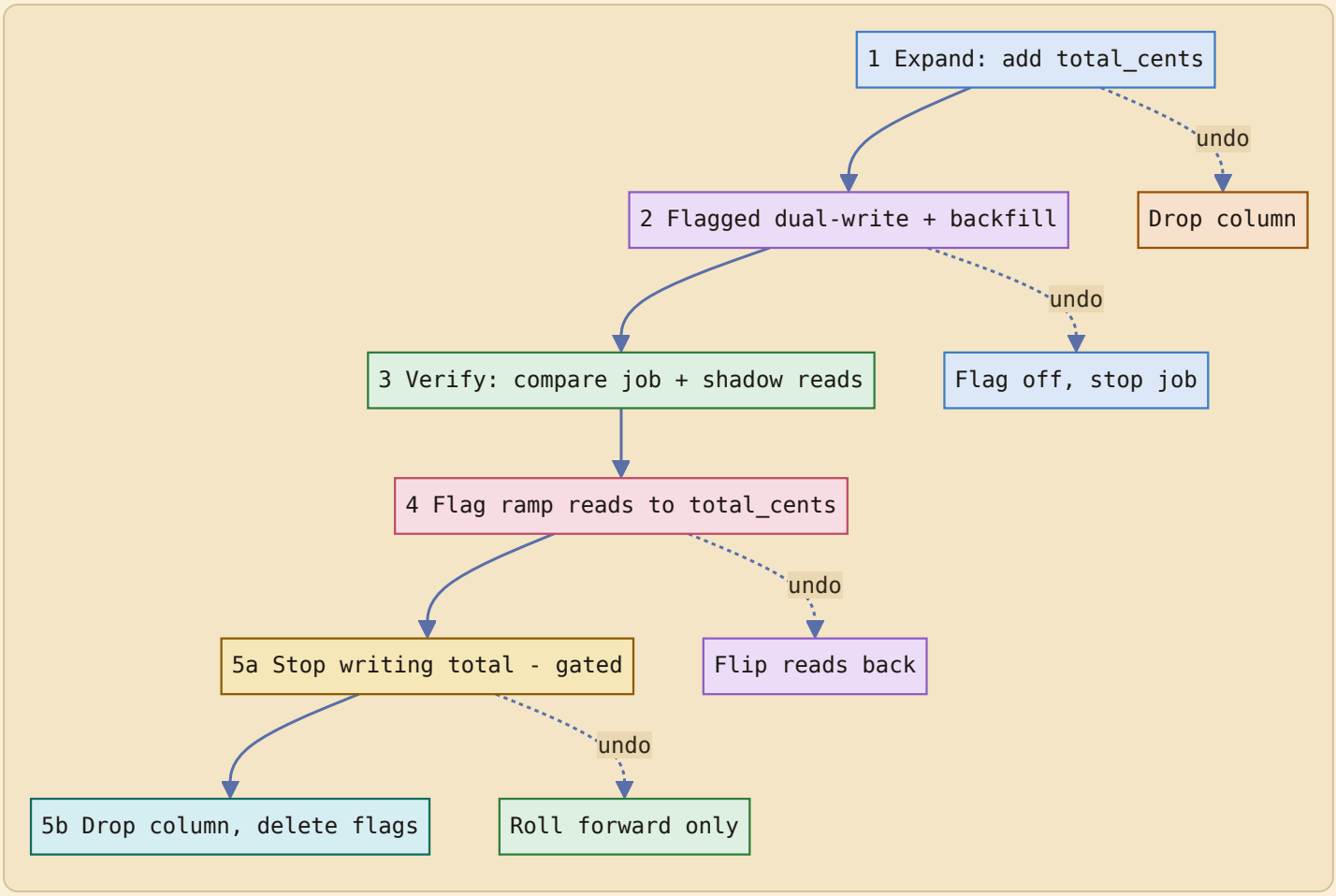
By now reads have used `total_cents` for weeks. What remains is two sub-steps, spaced weeks apart:

- **5a. Stop writing** `total` (flip the write flag to cents-only). From here on, `total` goes stale, and rolling reads back is no longer safe. So three **gates** have to pass first. The mismatch count has been zero **for weeks**. Every consumer is confirmed as migrated (the dependency map from the tracing lesson helps find any forgotten report still reading `total`). And there is written **sign-off from finance**. After this point, recovery is **roll forward only** (you fix things with a new change). Losing reversibility is the very reason the gates and the waiting period exist.

- **5b. Drop the column** after one more safe interval and one last safeguard: a documented **backup snapshot** of the old schema. Then clean up the flags — delete both migration flags and their dead branches. **Dropping the column doesn't finish the migration; deleting the temporary two-column code does.**

The step you can't reverse comes last, backed by all the evidence gathered, and only by a conscious decision. It's the saga lesson's **pivot** (the point of no return), applied to operations.

Expand-contract migration pipeline with undo at each stage



This flowchart lists the five stages from top to bottom. Each dotted arrow leads to that stage's undo button. Notice how undoing gets costlier the further down you go, until 5a is left with only "roll forward".

Debrief: the undo column

Stage	The change	The undo button	Its cost
1. Expand	Add a nullable column	Drop it	Free
2. Dual-write + backfill	Write to both, backfill old rows	Flag off, stop the job	Free
3. Verify	Comparison job, shadow reads	n/a — nothing changes	—
4. Switch reads	Flag ramp with canary verdicts	Flip back (safe because dual-write kept running)	Seconds
5a. Stop old writes	Cents only from now on	Roll forward only — which is why the gates exist	—
5b. Drop + cleanup	Column and flags removed	Backup snapshot (for disasters only)	—

Scan the undo column from top to bottom and you're watching Fact 2 at work: free, free, not needed, seconds — and then, on purpose, gone, but only after gates and approval. **Sequencing the undo options is the migration design.** Cheap reversals come first; the one-way door comes last, backed by evidence. Every pattern in this module plays a part. Canaries check binaries. Flags release behavior. Health checks gate deploys. Traces tie mismatches back to requests. Expand-contract governs the schema. And underneath, the pipeline builds each image once and promotes it unchanged, while Kubernetes' readiness-gated rolling updates carry every stage's deploy. Add Module 5's reliability patterns, and a high-risk change becomes routine and predictable (the same aim as the scaling capstone).

Interview one-liner

Move state over in stages. Expand with a column nobody uses yet. Turn on a flagged dual-write, plus a backfill that is idempotent, throttled and checkpointed. Verify using a comparison job and shadow reads. Ramp the reads with a flag and canary verdicts. Then contract in two gated steps. Each stage keeps a cheap undo, right up to the last, deliberate one-way door.

Extend it yourself (practice questions)

- **Queue-format version:** run the same process for a breaking change to a message format that five teams consume. The matching steps are publishing both formats at once, shadow-consuming the new format, and contracting later. Work out what a **schema registry** (a central service that stores message formats and rejects incompatible changes) ought to reject.
- **Multi-region:** with primary-replica replication across regions, backfill traffic can raise **replication lag** (how long it takes a primary's write to show up on a replica). Design the comparison job so that stale replicas don't set off false mismatch alerts.
- **For AI engineers:** migrate prompt-v1 → prompt-v2 or model A → model B. Here, "dual-write" means running both and comparing their outputs. The comparison job runs **LLM-judge evals** (using a language model to grade answers) on sampled pairs. Define the flag ramp, and the evidence you'd need before the contract step retires prompt-v1 for good.

Module 10 — Data-Intensive Systems (Batch and Streaming)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. So far, most modules looked at one request at a time, answered within milliseconds. This module changes the question. Now we ask things about *lots* of data at once: revenue per hour, patterns that point to fraud, or features built from a long history. The input never stops coming. It is a **continuous flow** of clicks, order events and changes to database tables. Data engineers are usually the people who build these systems. Still, every engineer should know how the numbers on a dashboard are actually produced.

> **Analogy.** Think about counting votes in an election. One option is to wait until the polls close and then count every ballot carefully. That is batch: the result is complete, but it comes late. The other option is to keep a running tally as each ballot box arrives. That is streaming: the result is fresh, but only provisional, because some boxes turn up late. The question is the same. What changes is the balance between completeness and freshness.

New constraints this kind of work brings: the input may be too big for one machine. Records may show up late, or show up more than once. Different consumers may read at different speeds. Because of this, every data design has to answer four things. How is the work **partitioned** (divided up across machines)? Where does the **intermediate state** (the half-finished results) live? How does processing **resume after a failure**? And which delays can we live with, versus which data losses we cannot accept?

Two facts that organize the module

1. **Batch and streaming are the same computation with different staleness budgets.** A *staleness budget* is how old an answer is allowed to be. Take "revenue per product". You could compute it every night from yesterday's full data. That is batch: simpler and thorough, but the answer is hours stale. Or you could update it continuously as each order comes in. That is streaming: the answer is seconds fresh, but it costs more to operate. Just like in the caching module, the allowed age decides which mechanism you pick. Batch works on **bounded** data, meaning a finished dataset that has a known end. Streaming works on **unbounded** data, which never ends. So streaming cannot wait for "all the input". Instead, it must decide when a partial answer is complete enough to publish.
2. **The log is the foundation.** In Module 3 we met the append-only log that lives inside databases. That was the **write-ahead log, WAL**: every change is first written to a sequential log, and only then applied. In this module, the log grows into a whole data platform. Kafka-style logs carry the flows of events (**Kafka** is a widely used open-source system that stores streams of events as a durable, replayable log). **CDC** (Change Data Capture) takes a database's WAL and turns it into a public stream. **Replay**, meaning reading the log again from an earlier position, makes reprocessing possible. And exactly-once processing works by combining log positions with **idempotency** (the property that doing the same operation twice has the same effect as doing it once). For a refresher on logs, queues, replication and other basics used here, see System Design Fundamentals.

The seven patterns (plus one added lesson)

The situation	The pattern
One big question asked over a huge dataset	MapReduce
Getting answers while data is still coming in	Stream Processing
Bringing a slow-but-thorough result and a fast-but-rough result into agreement	Lambda & Kappa
Broadcasting every change the database makes	Change Data Capture
Counting each thing once, even with retries and replays	Exactly-Once Semantics
A producer that sends faster than its consumer can keep up	Backpressure
Running consumers in parallel while keeping the order for each key	Partitioned Consumption
Knowing what the log underneath really guarantees, and which settings decide it	Inside Kafka *(added lesson)*

How to read it. MapReduce shows the basic shape of batch work: split the input, compute on each piece locally, then merge the results. Its answers are hours old on purpose, and that gap is the reason Stream Processing exists. Stream Processing covers windows, event time versus processing time, and answering questions over input that isn't finished. Lambda & Kappa weighs two options: running a separate batch pipeline and stream pipeline, or running one stream pipeline you can replay. The remaining patterns are supporting infrastructure. CDC creates streams out of database changes. Exactly-Once builds on the idempotency lesson. Backpressure deals with traffic that exceeds what a consumer can handle. Partitioned Consumption keeps per-key order while several consumers work in parallel. **Inside Kafka** is an added lesson (not in the original course) that opens the log itself: segments and retention, why it is fast, replication with the ISR and high watermark (the set of copies that are fully caught up, and the marker for how far all of them have got), every way messages can be lost and the setting that prevents it, consumer-group internals (a **consumer group** is a set of readers that share the work of reading one stream), and how idempotent producers and transactions implement exactly-once. The capstone puts everything together in a clickstream dashboard. Treat each pattern as the answer to a requirement someone has stated. Pick batch when completeness matters more than freshness. Pick streaming only when answers have short deadlines.

> Short on time? If you are a data engineer, begin with Stream Processing and CDC, because data-platform interviews test those the most. Everyone else should begin with CDC, since it fixes many everyday integration problems.

Four terms used throughout

- **Bounded vs. unbounded** — bounded data has a known end, like yesterday's orders, and suits batch. Unbounded data is a flow that never ends, and suits streaming, which has to answer without seeing all the input.
- **Event time vs. processing time** — *event time* is the moment something really happened. *Processing time* is the moment the system got it. For example, a mobile order placed inside a tunnel at 9:02 might only arrive at 9:14.
- **Watermark** — the processor's claim that "I think I have now seen every event up to time T". This claim lets a computation over endless data publish answers tied to time.
- **Throughput vs. latency** — batch tries to maximize rows processed per hour. Streaming tries to minimize the delay for each result. Always say which one matters, because it shapes both the design and the cost.

Before you start

- Pick a number on your main dashboard. How stale is it? And does anyone actually *know* that, or do people just assume it is live?

- When the analytics team needs data from the database, how do they get it: a nightly dump, scraping an API, or a read replica? What does that route fail to capture?
- Suppose event volume doubled for one day. Would your pipeline slow down, fall behind, or break? And did anyone ever decide which of those should happen?

MapReduce

Original lesson: MapReduce on DesignGurus ↗

What it is. MapReduce answers one big question over a huge dataset by spreading the work over hundreds of machines. You only write two small functions. **Map** turns each record into key/value pairs. **Reduce** combines all the values that share one key. A framework does the rest for you: it splits the data, runs your code close to where the data sits, groups results by key, and re-runs any task that fails.

Analogy. Picture counting the votes in a national election. Every polling station counts its own ballots per candidate (that's map plus a local tally). The stations then send these subtotals to regional centres, sorted by candidate (that's the shuffle). Each centre adds up the subtotals for its candidate (that's reduce). If the counter at one station gets sick, someone else recounts just that station. Nobody restarts the count for the entire country.

The problem

Ten years in, finance wants **"total revenue by product category, all time, by region"**. The data is **5 billion order rows, ~2 TB**, stored in **object storage** (cheap cloud storage, such as Amazon S3, that keeps data as whole files called objects).

An analyst writes a script. It reads every row and keeps running totals. Do the math: one machine reads roughly **100 MB/s**. So 2 TB ÷ 100 MB/s = **20,000 seconds ≈ 5.5 hours just to read**. The script is started Monday 9 AM. At 12:40 PM the cloud provider takes back the **spot instance** (cheap cloud capacity that the provider can reclaim whenever it wants). Three hours of progress held in memory are gone, and the script starts again at row one. On Wednesday, the retry fails after four hours because of a network failure. By Thursday the analyst has built a checkpoint file and a resume flag by hand, and there is still no answer.

There are two different limits here. First, **one machine is too slow**. Second, **at multi-hour runtimes, failure is likely**: even a 1% chance of hardware failure per hour makes a five-hour job risky. Long jobs therefore need two things: parallelism *and* automatic recovery.

Why the obvious fixes fail

- **"A bigger machine."** Its read bandwidth still has a ceiling. And it is still a single **failure domain** (one unit whose failure takes everything inside it down), so one crash wipes out everything held in memory.
- **"Split it by hand"** over 20 machines, then merge the pieces with a script. This works once. But problems pile up. If machine 14 dies at minute 50, a person has to spot it and rerun that share. If the chunks are uneven, 19 machines sit idle while one grinds away for hours. The merge script breaks easily. And the next question needs a whole new manual procedure. This is not a system you can reuse.
- **"Run it on the production database shards."** Why not **scatter-gather** the query (send it to every shard at once and combine the answers) over the orders DB, which is already **sharded** (split across several database servers, each holding part of the rows)? Because then the analytics scans fight checkout for production **I/O** (disk and network reads and writes). The storage module split those two workloads on purpose. That is exactly why analytics keeps its own copy in object storage, and it is why analytics needs its own compute too.

The hand-split approach actually had the right *algorithm*: divide, count locally, merge. What was missing was everything around it: scheduling, handling failures, handling **stragglers** (a straggler is a

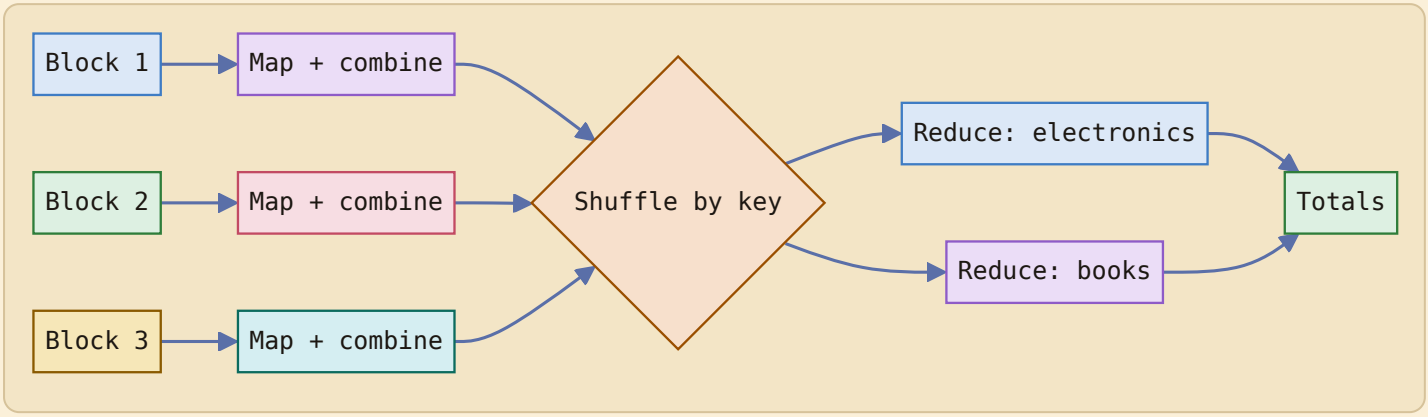
task that runs far slower than the others), and merging reliably. MapReduce provides that execution framework.

How it works

```
map(order)             -> emit (order.category, order.amount)
       ... the shuffle groups: all "electronics" amounts to one place ...
reduce(category, [amounts]) -> emit (category, sum(amounts))
```

1. **Split and map, where the data lives.** The framework cuts the input into blocks. It runs the map tasks on the machines that store those blocks, or close to them. So rather than shipping 2 TB to one program, it ships a small program to the data. This is called **data locality**. It is what makes distributed batch affordable, because the network is the resource in shortest supply.
2. **Combine (optional but important).** A **combiner** does a first round of aggregation on each mapper's output, right there on the machine. For example, each mapper sends "electronics: $4.2M" one time, instead of a million separate amounts. For aggregations, this makes the next step smaller by orders of magnitude.
3. **Shuffle — the expensive middle.** The framework regroups the output of every mapper *by key* across the whole cluster. This is an all-to-all sort plus network transfer. The result is that every value for "electronics" lands at one reducer.
4. **Reduce.** Each reducer merges the values for the keys it owns (for example, by summing them) and writes out the results.
5. **Fault tolerance falls out of the model's constraints.** Map and reduce have to be **pure functions**: they have no side effects, and the same input always gives the same output. They also run over **immutable** input, meaning files that never change. Because of that, any task can be run again on any machine and produce the same result. You get idempotency for free. If a machine dies, its tasks are rescheduled on other machines, and you lose minutes rather than the whole job. If a task straggles, the framework starts a **speculative duplicate** and keeps whichever copy finishes first. In short, the framework automates the very checkpoint-and-resume logic the analyst was building by hand.

Map-combine-shuffle-reduce for revenue by category



The diagram above shows each input block being mapped and combined on its own machine. The shuffle then sends each category's partial sums to a single reducer. Finally, the reducers produce the finished totals.

Walkthrough with numbers (200-machine cluster)

- **Read:** 2 TB ÷ 200 = 10 GB per machine, which takes about **100 seconds** when all machines read in parallel. The 5.5-hour read shrinks to less than two minutes.
- **Map + combine:** every machine sends out one partial sum per category. That is dozens of rows, not millions, so the shuffle only moves **kilobytes** per machine.
- **Reduce:** adding up 200 partial sums for each category takes a few seconds.
- **End to end: ~10 minutes**, spent mostly on startup and reading. Compare that with a week of restarts someone had to babysit.

- **Failure drill:** machine 141 dies at minute 4. Its ~30 map tasks run again on other machines. This adds **90 seconds**, and the failure goes into the log.
- **Straggler drill:** machine 77 has a disk that is failing, and it slows down at minute 8. Speculative copies of its tasks run on other machines and finish first. Machine 77 is flagged for the ops team. Nobody had to wait.

Trade-offs

You gain:

- **Scale by addition** — 200 machines give roughly 200× the read bandwidth, and the framework takes care of the coordination.
- **Fault tolerance at no extra cost** — because the functions are pure and the input is immutable, re-running is safe. Long jobs survive failures by design.
- **A tiny programming model** — just two functions. The question you are asking *is* the program.

You pay:

- **A latency floor by construction** — you always pay for cluster startup, a full pass over the data, and the shuffle. That means minutes at best and hours at scale. The answer also reflects the data as it was when the job began. Fresh answers are out of reach (that is the next lesson's problem).
- **A rigid shape** — every job must fit map → shuffle → reduce. Real analyses often need many stages, and classic MapReduce wrote each stage's output to disk, which multiplied the I/O.
- **Shuffle-dominated cost**, and **skew** makes it worse. Skew means one key has many more records than the others (see self-check Q3).
- **It's a scanner, not an index** — reading all 2 TB to find "revenue for order #8412" is the wrong tool for the job.

When not to use it

- **Interactive questions** — analysts trying idea after idea need answers in seconds. That calls for a **data warehouse** (indexed and columnar). A data warehouse is a database built for analytics queries over huge tables; *columnar* means it stores each column's values together, so reading one column across billions of rows is fast. Under their SQL, warehouses still run map-shuffle-reduce shapes. The idea lives on; only the way you use it changed.
- **Small data — and "small" has grown.** 2 TB fits on a laptop SSD. One powerful machine running **DuckDB** or **Polars** (single-machine analytics tools: an embedded SQL engine and a fast DataFrame library) can scan it in minutes, with no cluster at all. The modern rule: **exhaust one machine before renting two hundred** (the same restraint as the vertical-scaling lesson, where *vertical scaling* means buying a bigger machine instead of more machines). Setting up cluster infrastructure for data that fits on a laptop is a common mistake today.
- **Low-latency needs** — for unbounded streams, see the next lesson.
- **Iterative algorithms** — ML (machine learning) training reads the same data hundreds of times. Classic MapReduce does full disk I/O on every pass, which is too slow. That pushed people to build in-memory successors, and modern ML now uses specialized training systems.

MapReduce vs. Spark (the idea vs. the implementation)

Apache Spark is an open-source engine for processing large datasets across a cluster (a group of machines working together). It keeps the split-locally-then-merge idea but runs whole pipelines in memory.

	Classic MapReduce	Spark (and successors)
Between stages	Saved to disk after every stage	Held in memory through the whole pipeline
Program shape	Strictly map, then shuffle, then reduce	Any **DAGs** (graphs of steps) made of transforms: map, filter, join…
Multi-stage / iterative work	A chain of complete jobs, so the I/O multiplies	A single job with cached intermediate results: 10–100× faster
Fault tolerance	Run the tasks again from the input	Same idea: rebuild lost partitions from their **lineage** (the recorded chain of transforms)
You write	Java boilerplate, two functions at a time	SQL / DataFrames (tables you manipulate in code), and the shape is generated for you

The takeaway for interviews: **the execution model outlived the original implementation.** No one hand-writes MapReduce anymore. Yet Spark plans, the BigQuery and Snowflake engines (two popular cloud data warehouses), and distributed ML data prep all do the same three things: split the work locally, shuffle by key, and merge.

Real-world examples

- **Google's 2004 paper** — where it all started. Indexing the web needed exactly this shape, and the paper kicked off the "big data" era.
- **Hadoop** — the open-source implementation of MapReduce (plus a distributed file system); it made the model available to everyone for about a decade. **Spark** — began at Berkeley to skip the disk writes between stages and speed up iterative work. It took over from Hadoop for many workloads and, through **Databricks** (the company that sells a hosted Spark platform), runs a large share of production batch.
- **BigQuery, Snowflake and peers** — SQL on the surface, map-shuffle-reduce underneath.
- **dbt** — declarative data pipelines that run on those same engines.
- **For AI engineers:** preparing training data is the largest MapReduce workload today. Take deduplicating, filtering and **tokenizing** (splitting text into the small pieces, called tokens, that a language model reads) a crawl of the whole web. Map handles each document on its own, fully in parallel. Shuffle groups near-duplicates together using a hash key. Reduce chooses which copies survive. Every serious lab runs this on Spark. **Embedding** a corpus (turning each document into a list of numbers that captures its meaning) is just a map with no shuffle. And the **all-reduce** step in distributed training merges **gradients** (the per-step numbers that say how to adjust a model's weights) across **GPUs** (graphics chips used for the heavy maths of ML) on every step, which belongs to the same compute-locally-then-merge family.

In the interview

The classic prompt is "count word frequencies across 10 TB". Newer versions sound like "compute X over everything" or "the analytics job takes 9 hours and fails at hour 7 — fix it". Follow-up questions dig into the shuffle and into what happens on failure.

> **30-second answer:** "This is a batch aggregation with a map-shuffle-reduce shape. The map step runs on the machines that hold the data, so the code goes to the terabytes instead of the other way round, and it outputs key-value pairs. Combiners do a first aggregation on each machine, so the shuffle carries partial sums rather than raw records. Reducers then merge everything for each key. Fault tolerance comes from the structure itself: the functions are pure and the input never changes, so any task can be rerun on any machine. A dead machine costs a few minutes, and slow tasks get speculative duplicates. In practice I'd write it in Spark or SQL, since nobody hand-writes MapReduce anymore, but this is the shape running underneath. The shuffle is always the costly part. And the answer is stale by design, which is exactly where streaming comes in."

Likely follow-ups (with answers):

- *"Where does the job actually spend its time?"* — Mostly in the shuffle (regrouping everything by key, all-to-all, over the network) and in scanning the input. So two levers really matter. Combiners shrink how much gets shuffled. Partitioning and skew management spread out what gets shuffled.
- *"A machine dies at minute 40 of 60 — what happens?"* — Its tasks run again on other machines, reading the same immutable inputs. You lose a few minutes, but nothing is corrupted and nothing starts from zero. This is a direct benefit of the model banning side effects and mutation, so state that cause-and-effect link out loud.

Mistake that fails candidates: suggesting batch jobs for needs that are interactive or low-latency; ignoring the shuffle; or building distributed counting from scratch instead of naming the shape and the modern tools that implement it.

Self-check:

- **Q1.** Write "average order value per country" as map and reduce, and watch out for the averaging trap. → You cannot take an average of averages. Map emits `(country, (amount, 1))`. Reduce (and the combiner too) adds both parts pair by pair to get `(sum_amounts, sum_counts)`. Divide only at the very end. Carrying the pair keeps the operation **associative**, meaning partial results can be merged in any grouping. That property is what lets combiners and parallel reducers work. The general rule: build aggregations out of pieces that can be merged.
- **Q2.** Name three failures of the hand-split version that the framework avoids, plus the property that makes each fix work. → (1) A machine dies: the framework reruns its tasks. This is safe because tasks are pure and read immutable input. (2) Stragglers: speculative duplicates. These are harmless because running a task twice gives the same answer. (3) Uneven chunks and custom merge scripts: the framework uses fixed-size blocks, and the shuffle *is* a standard merge. All three come from limiting the programming model so that running things again costs nothing.
- **Q3.** On 200 machines, 199 reducers finish in 2 minutes, but one runs for 6 hours. All machines are healthy. → This is **skew**. One hot key (say Electronics = 40% of orders) sends 40% of the work to a single reducer. It is the sharding lesson's hot-key problem, in batch form: hashing spreads keys, not how heavy each key is. Fixes: combiners, so mappers send 200 partial sums instead of 2 billion rows. For keys that are still too heavy, use **salting**: split the key into `electronics-1..16` across 16 reducers, then merge those in a second stage. You can also process extreme keys separately.

Related patterns:

- **Stream Processing** — the answer to the latency floor: the same computations, but over unbounded input.
- **Sharding** — the same partitioning math and the same skew problem, one layer lower down.
- **Lambda & Kappa** — how batch and streaming split responsibility for the truth.

TL;DR

- **Problem:** one machine needs hours just to read the data, and at multi-hour runtimes failure is routine. Splitting by hand means reinventing scheduling, straggler handling, merging and recovery.
- **Mechanism:** two pure functions — map (per record, run where the data lives) and reduce (merge per key). The framework owns splitting, the shuffle (made smaller by combiners), speculative copies for stragglers, and re-execution.
- **Costs:** latency of minutes to hours, a rigid shape (classically with disk I/O between stages), cost dominated by the shuffle, and skew.

- **Skip it when:** the work is interactive (use a warehouse), fits on one machine, needs fresh answers (use streaming), or is iterative (ML).

Stream Processing

Original lesson: Stream Processing on DesignGurus ↗

What it is. A stream processor is a job that **never ends**. It reads an event log while the log keeps growing. For each thing it tracks, it holds running **state** such as counts, sums or windows. And it sends out answers all the time. The hard question at its centre is this: when the input never ends, at what point is an answer "final"?

> **Analogy.** Imagine a bank teller watching a live feed of card swipes. The teller keeps a tally sheet for each card covering the last 10 minutes. The moment a card reaches six swipes, the teller flags it. Compare that with reading yesterday's statements the following morning.

The problem: correct, but eight hours too late

The fraud team runs a nightly batch job that spots coordinated use of stolen cards with **99% precision**. On Tuesday, the run flags **30 fraudulent orders** from one ring. All of them were placed between **3:40–3:55 PM**, and all had shipped by 6 PM. The job runs at **2 AM**, which is eight hours after the goods left the warehouse. **The answer is right but useless.** For "revenue by category, all time", being hours stale is fine. But "is this card being abused *right now*" needs an answer within seconds, and no amount of tuning a batch job will get you there. The same goes for an ops dashboard that shows yesterday's "orders per minute". Fraud detection, live metrics and fresh ML features all need ongoing answers about the last few seconds of an input that never ends.

Why the obvious fixes fail

- **"Run the batch every five minutes"** (this is called micro-batching). It is a real approach (it inspired Spark Streaming), but it has two limits. First, the overhead of starting each job puts a floor under the interval: a cluster job launched every 30 s might spend most of its time just starting up. Second, a question like "orders on this card in the last hour" covers many runs, so every run needs **running state from previous runs**. The true difference between batch and streaming is *persistent state*, not only speed.
- **"Query the database on every order"** — running `SELECT COUNT(*) WHERE card=? AND time > now()-1h` means **400 expensive queries/s** against production. That is the analytics-on-production problem once more, and the indexes were not designed for it. A database is good at storing history; it is not designed to *watch* events as they arrive.
- **"Count in application memory."** The checkout service is scaled out horizontally (run as many identical copies behind a load balancer). **40 servers each see ~1/40** of a given card's orders, so no single server's counter is complete. This is the per-server-counter problem. Every event for a card has to reach *one* place that holds that card's state, which means you need **key-based routing**.

How it works

1. **State is the core.** The operator (one processing step) keeps its running answer locally, either in memory or on local disk, for example per-card counts or per-minute sums. The state is **keyed**. All events for card X go to the operator instance that owns card X. This is the shard-key idea applied to compute, and Partitioned Consumption formalizes it. Most of what a streaming framework does is manage this state.
2. **Recovery = checkpoint + rewind.** Every so often, the framework saves a durable (stored so it survives a crash) **snapshot of all state together with the log position** that the state matches. After a crash, the operator loads the snapshot, **rewinds the log** to that position, and processes again from there. This only works because the source is a log that can be *replayed*. One side effect: events close to the crash get processed **twice**, so the outputs must be able to handle duplicates (→ Exactly-Once lesson).

3. **Windows slice the endless input into finite pieces**, because an infinite stream has no "final total":
   - **Tumbling** — fixed intervals placed back to back. Each event falls into exactly one of them (e.g. orders per minute).
   - **Sliding** — intervals that overlap (e.g. orders in the last 10 minutes, checked again every 30 s).
   - **Session** — a window that stays open until an **inactivity gap** appears (e.g. a browsing session is over after 30 idle minutes). The data decides how long it lasts.
4. **Event time vs. processing time — the hardest, most consequential idea.** Suppose you group by *processing time*, meaning arrival time. Then the 9:02 tunnel order gets counted at 9:14. That is simple, but wrong: retries, mobile batching and network delays shuffle the arrival order and can produce fake spikes. Suppose instead you group by *event time*. Now the order is counted at 9:02 as it should be. But at 9:03 the system has no way of knowing the event exists yet, so it has to keep windows open for late arrivals.
5. **Watermarks decide when a window may close.** The framework keeps an estimate such as "I've probably seen everything up to 9:05". That estimate is the watermark. Once it moves past the end of a window, the window closes and emits its result. The delay of the watermark *is* the staleness budget. A longer delay gives more complete but later answers. A shorter one gives faster answers, but more events count as "late". That is why every consumer needs a **late-data policy**. The choices are: drop late events, emit corrected results that consumers agree to accept, or send late events to a side channel for reconciliation.

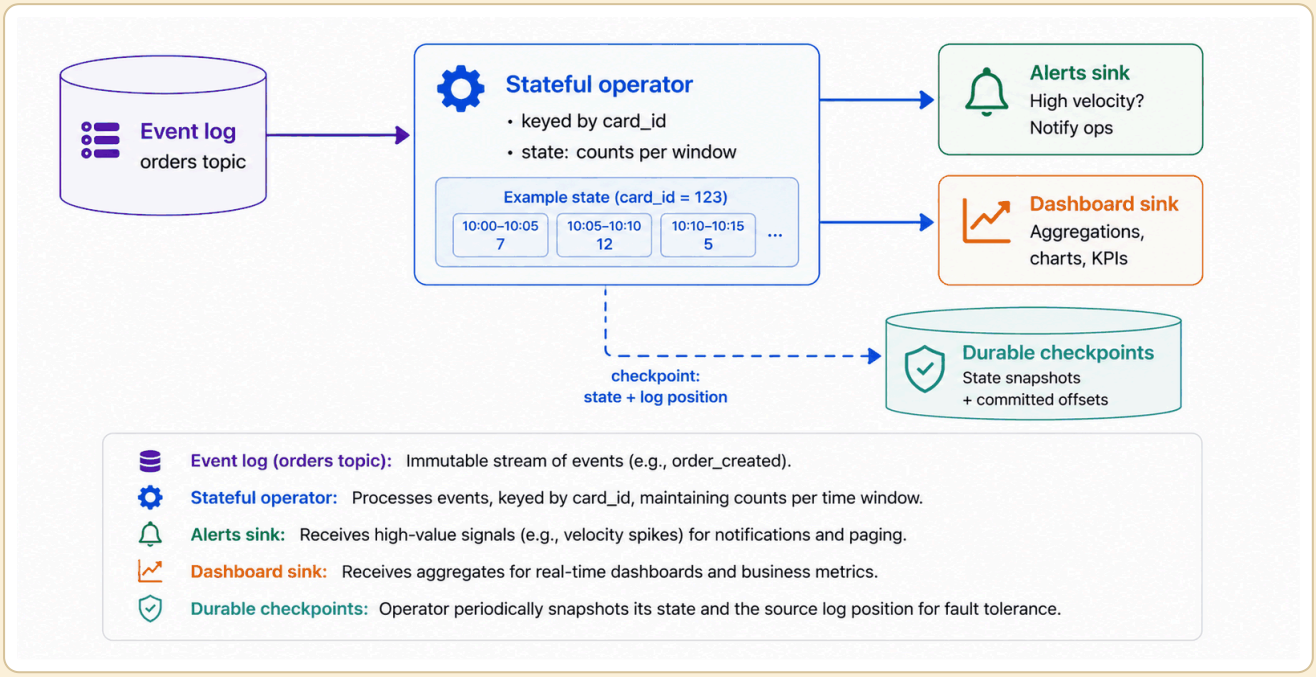


Figure — Stream Processing (course diagram)

The figure above shows an orders **topic** (a named stream of events in the log) feeding a stateful operator keyed by `card_id`. Its state stores counts for each time window (e.g. card 123: 7, 12, 5 across three 5-minute windows). The operator sends results to an alerts **sink** and a dashboard sink (a sink is wherever a pipeline writes its output). It also regularly writes durable checkpoints of its state + log position so it can recover.

Walkthrough with numbers (rule: more than 5 orders on one card within 10 minutes)

- **The path:** order event → log → the operator that holds this card's state → the count is updated → the threshold is checked → alert. The whole trip takes **~200 ms**. Tuesday's ring gets caught at **order #6 at 3:44 PM**, while the goods are still on the shelf.
- **State size by arithmetic:** about **200,000** cards are active in any 10-minute window, and each needs a few bytes, so the total is **megabytes**. Work this out before reaching for heavy state storage.
- **Crash drill:** load the **30-second-old** checkpoint, rewind the log by 30 s, and reprocess. Recovery takes **under a minute** and no orders are missed. Alerts from those replayed 30 s go out twice, so the

alert sink **dedupes on** `(card, window)` (that's idempotency).

- **Tunnel drill:** fraud cares about speed, so its watermark lags by only **30 s**. The 9:02 order that arrives at 9:14 misses its window, which makes it late data. Fraud's policy: process it anyway, update the count, and send a late alert if the threshold is crossed (a late alert is better than none). The dashboard reading the same stream picked **drop**, because a per-minute chart won't redraw itself for a single straggler. **Same stream, two policies — the policy belongs to the question, not the pipeline.**

True streaming vs. micro-batch

	True streaming (per event)	Micro-batch (tiny frequent jobs)
Latency floor	Milliseconds	The length of the batch interval (seconds up to a minute)
State between "runs"	Built in — the whole operator is designed around it	Carried from run to run by the framework
Complexity	Everything described above	Clearly easier to reason about
Fits	Fraud, alerting, features needed in under a second	Dashboards, feeds, pipelines with a one-minute budget

The budget decides: with a one-minute budget, micro-batch is the simpler and correct choice (**Spark Structured Streaming**). With a one-second budget, only true streaming will do (**Flink**, i.e. Apache Flink, an open-source engine for stateful stream processing). Both run enormous production workloads. Saying "the budget chooses" is a better answer than being loyal to either tool.

Trade-offs

You gain: answers that are seconds fresh over input that never ends, so fraud checks, live metrics and fresh features become possible in the first place; incremental economics, since each event updates state once rather than rescanning everything; and solid machinery for correctness (checkpoints plus a replayable log).

You pay:

- **The steepest concept stack in the course** — windows, two kinds of time, watermarks, late policies and state. Teams tend to underestimate it.
- **Stateful operations** — state has to be checkpointed, restored, monitored and **rescaled**. Rescaling means re-partitioning the state when the job grows or shrinks, and it is hard.
- **Provisional answers** — results based on event time can be changed when stragglers show up. So downstream consumers must accept corrections (**eventual consistency** in analytics: the numbers agree with the truth eventually, not instantly).
- **Operational weight** — a stateful distributed job that runs forever is really a *service*. It needs **on-call** (an engineer ready to respond to alerts at any hour), upgrades and capacity planning.

When not to use it

- **No deadline** — nightly revenue rollups and weekly cohorts are simpler and cheaper in batch, and easy to reprocess.
- **Small scale** — a **cron job** (a task the operating system runs on a fixed schedule) with a SQL query can count a startup's daily active users without any Flink cluster.
- **Ad-hoc exploration** — analysts need a warehouse, not a new streaming job for every idea they want to test.
- **No team to run it** — a stateful platform with no owner turns into the main cause of outages between the log and the dashboard.
- Misuse: rebuilding stable nightly jobs as streams when the business has no need for fresher data.

Real-world examples

- **Apache Flink** — implements the full model (event time, watermarks, exactly-once state). Most of this lesson's vocabulary comes from Flink.
- **Google's Dataflow model (2015 paper)** — brought windows, watermarks and late-data triggers together into one model. Most frameworks implement its ideas.
- **Kafka Streams** — the pattern packaged as a library inside your own service, with no separate cluster.
- **Spark Structured Streaming** — micro-batch at industrial scale.
- **For AI engineers:** how fresh your features are affects how good your model is. A fraud model using hour-old velocity features misses coordinated attacks that fresh streaming features would catch. A **feature store** is a system that computes and serves the input values (*features*) that ML models use, both for training and for live predictions. Its **online path** uses jobs like these to produce keyed aggregates that are fresh to the millisecond. Token-usage metering, prediction monitoring and **evals** (automated checks of model quality) over sampled traffic rely on it too. Use **event time**: if you train on processing-time features, the model learns *network lag* instead of user behavior.

In the interview

Typical direct prompts are "design real-time fraud detection" or "a live orders-per-minute dashboard". The follow-ups are predictable: late events, recovering from a crash, and counting across a fleet of servers.

> **30-second answer:** "I'd run a job that never stops, reading the order log with its state keyed by card. That way every event for a card goes to the one instance holding that card's counts. For each event it checks a sliding window and can alert within a couple hundred milliseconds. The windows use event time rather than processing time, because retries and mobile lag scramble the arrival order, and I want to count when things actually happened. A watermark running thirty seconds behind tells each window when to close. Late stragglers still update the count, since a late fraud alert is better than none. Recovery is checkpoint plus rewind: snapshots of state tied to log positions, and the replayable log reprocesses the gap. Because some events get processed twice, the sink dedupes on card-and-window. If the budget were a minute instead of a second, I'd use micro-batching and keep things simple."

Likely follow-ups (with answers):

- *"An event arrives 12 minutes late — walk me through it."* — The watermark has already passed, so its event-time window is closed. Now the late policy kicks in, and it is chosen *per consumer*. Fraud updates the count and alerts late. The dashboard drops the event. A reconciliation stream might send it to a side channel. Pointing out that the policy is set per consumer, not globally, is the senior-level detail.
- *"The operator crashes mid-afternoon — what's lost?"* — Nothing durable. Load the last checkpoint, rewind the log to the matching position, and reprocess the gap. The sinks see a few seconds get replayed, which is why you need idempotent sinks and the exactly-once lesson. What a crash really costs you is *duplicate emissions*.

Mistake that fails candidates: using processing-time windows for money or fraud; keeping state in application memory spread over a fleet; or running a database query for every event. Each of these is one of the failed "obvious fixes" dressed up as a design.

Self-check:

- **Q1.** Choose windows for: (a) revenue per calendar minute for a chart; (b) "more than 5 orders in any 10-minute span"; (c) average items per shopping session. → (a) Tumbling, which gives clean

buckets that don't overlap. (b) Sliding, because it has to catch *any* 10-minute span, not only ones lined up with the clock. (c) Session, since the boundary is the gap in user activity; fixed sizes would cut real sessions apart or glue them together.

- **Q2.** A carrier outage holds back 20 minutes of app events, and then they all arrive in one burst. Compare a processing-time pipeline with an event-time pipeline that uses a 2-minute watermark. → With processing time, you first see a fake quiet dip during the outage. Then you see a huge fake spike "now", because 20 minutes get squeezed into one. That sets off rate alerts and permanently records traffic that never happened. With event time, the delayed events keep their real timestamps. The windows covering the outage close ~2 minutes after they end, with provisional *undercounts*. Then the late events trigger corrections according to the late policy, and the numbers settle on what really happened. Processing time measures the network; event time measures users.
- **Q3.** After a routine deploy, fraud receives duplicate alerts (same card, same window) for roughly a minute of traffic, with no real spike in fraud. → The deploy restarted the operator through the normal recovery path: restore the checkpoint, rewind, and reprocess ~1 minute. The computation was correct both times, but the alert sink sent its output twice. Fix it at the sink: make alerts idempotent, keyed on `(card_id, window_id)` (or use a transactional exactly-once sink). **Every deploy rehearses the crash**, so sinks have to cope with replays.

Related patterns:

- **Message Queue** — the replayable log is the source, and it is what makes rewind-based recovery possible.
- **Partitioned Consumption** — the keyed routing that gets each card's events to that card's state.
- **Exactly-Once Semantics** — dealing with what replays do to sinks.
- **Write-Ahead Log** — checkpoints borrow its idea: durable state plus a position, and recovery by replay.
- **Lambda & Kappa** — what to do when batch and streaming both claim to be the truth.

TL;DR

- **Problem:** deadline questions (fraud now, metrics now, features now) are about the last few seconds of an endless input, and batch answers show up hours too late.
- **Mechanism:** a job that runs forever, with keyed state over a replayable log; windows (tumbling, sliding, session); event-time processing with watermarks; late policies chosen per consumer; and recovery through checkpoint + rewind.
- **Costs:** the steepest concept stack, running a stateful service, provisional answers that get amended, and replays that require idempotent or transactional sinks.
- **Skip it when:** the staleness budget is generous, the scale fits a cron job, the work is ad-hoc exploration, or nobody is available to own a service that runs forever.

Lambda and Kappa

Original lesson: Lambda and Kappa on DesignGurus ↗

What it is. These are two architectures for when you need *both* fresh streaming answers and complete, correct batch answers. **Lambda** runs a batch pipeline and a streaming pipeline next to each other. Every so often, the batch results **overwrite** the streaming results. **Kappa** runs just **one** streaming pipeline over a log that keeps its history. When something needs correcting, it fixes the code and **replays** the log through it.

Analogy. Lambda is like a newspaper that posts quick online updates all day, then prints a carefully fact-checked edition overnight that replaces them. Kappa is like a single newsroom that,

on finding a mistake, reruns its stories from the archive using the corrected facts.

The problem: two pipelines, two truths

The store runs a nightly batch pipeline for its official reports, and a streaming pipeline for the live dashboard. Both calculate revenue. On Wednesday morning, finance spots a mismatch. The dashboard says Tuesday was **$412,880**, while the batch report says **$413,205**.

By their own rules, both numbers are "right". The streaming job dropped some late events because of its watermark policy (the point after which it stops waiting for stragglers). It also had a **one-minute dedupe gap** during Tuesday's deploy. (*Dedupe* means removing duplicate events; this is the stream lesson's Q3 happening for real.) The batch job read everything and is correct as of 2 AM. The trouble is that nobody ever decided which one is *the truth*.

Three weeks go by. A change to a discount rule gets shipped, but only in the batch code; no one updates the streaming copy. Now the two differ because of a **coding error**, and the gap widens every day. A monthly "reconciliation meeting" shows up on the calendar, where people referee between their own computers.

Named precisely: two codebases implement the same business logic, and over time they drift apart. Anyone running batch and streaming separately has to handle this on purpose.

Why the obvious fixes fail

- **"Perfect the streaming pipeline and retire batch."** Exactly-once cuts down processing errors. But the main strength of batch is **reprocessing history with current code**: when a fraud rule changes or a bug is found, you recompute past periods. A stream handles events as they arrive. It can only go back to them if a replayable log keeps the history, and that becomes Kappa's key idea.
- **"Run batch hourly."** It is fresher, but you still have two codebases changing independently. The window of inconsistency gets shorter, but its cause is still there.
- **"Label the dashboard 'approximate' and declare batch the truth."** This is a valid policy if stakeholders agree to it. But it does nothing to stop forgotten code changes from drifting, and you are still maintaining two codebases.

How it works — Lambda

Nathan Marz proposed it (during the Twitter / Storm era; **Storm** was an early open-source stream processor). It has three layers:

1. **Speed layer** — streaming; quick but provisional.
2. **Batch layer** — slow but authoritative; recomputed on a schedule from the raw history.
3. **Serving layer** — joins the two: recent hours come from the speed layer, settled history comes from the batch layer.

The central mechanism is **the overwrite**. Each batch run *replaces* the speed layer's results for the period that run covers. So any bug in streaming gets wiped out automatically at the next batch run. That includes the deploy gap, dropped stragglers, and even the discount rule someone forgot to add to the speed copy. Lambda doesn't stop speed-layer errors from happening; it **bounds their lifetime to one batch cycle**. The cost never goes away, though: **every metric is implemented twice**, so logic drift is baked into the architecture.

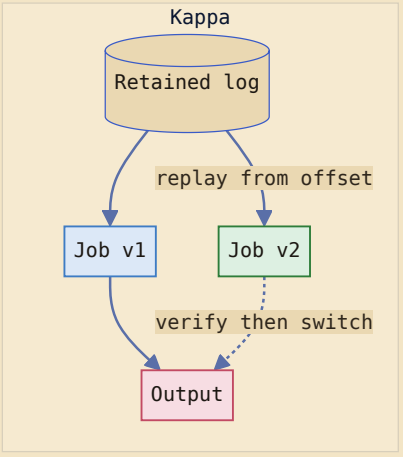
How it works — Kappa

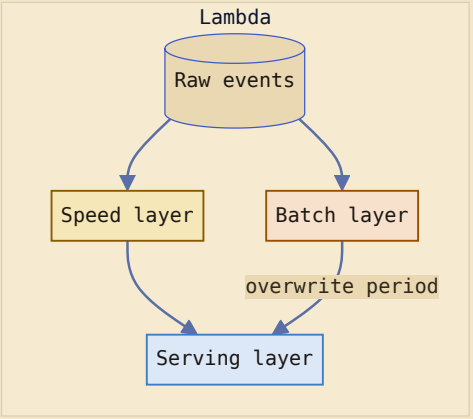
Jay Kreps proposed it in his essay "Questioning the Lambda Architecture". His insight: suppose the event log keeps its history for weeks, months, or practically forever using **tiered storage** (older log segments are moved to cheap object storage). Then batch is simply *streaming a bounded prefix of the log, quickly*. Reprocessing turns into a streaming operation:

1. Deploy job **v2**, containing the new logic, reading from log offset zero (or, for example, 90 days back). An **offset** is a position in the log.
2. v2 races through the history at full read speed. This takes hours, not weeks.
3. v2 catches up to "now" and runs alongside v1, and the two outputs are compared.
4. Point consumers at v2's output and retire v1.

This is the same playbook as the zero-downtime migration capstone (run both, check, switch, retire), applied to pipelines. With only one codebase, the discount-rule drift cannot happen, because there is no second copy to forget about. The costs: keeping the log for a long time costs money; replay needs far more compute than the live rate (to reprocess 90 days in one afternoon); and the streaming engine has to be mature enough (exactly-once, event time) to serve as the *only* source of truth.

Lambda vs Kappa data flow





In the diagram above, Lambda sends the same raw events into two layers, and the batch layer overwrites its period in the serving layer. Kappa has only one kind of job. A new version replays the retained log, and it takes over the output only once it has been verified.

The pragmatic synthesis

Most organizations run a practical **dual path**: streaming for operational freshness, and a warehouse for deep analysis. They often leave out Lambda's formal overwrite. They should still keep **scheduled backfills**, though. (A backfill recomputes a past period and replaces the results that already exist.) The engines have also grown closer together. **Flink** can run bounded inputs in batch mode. **Spark** can process streams. And **lakehouse tables (Delta, Iceberg)** (open table formats that put database-like tables on top of plain files in object storage) take streaming writes while also serving batch queries. More and more, you end up with **one logic, two execution tempos**, which is exactly what Kappa aims for.

Walkthrough with numbers: the deploy-gap bug under each design

- **Lambda:** Tuesday's streaming deploy quietly loses **~300 orders** from the dashboard's revenue. At 2 AM on Wednesday, the batch job recomputes Tuesday from the raw data and overwrites the speed layer. The dashboard fixes itself. This is what the second codebase buys you.
- **Kappa:** developers fix the bug in the one codebase and replay from **Monday's offset**. The log delivers history at **~50× the live rate**, so **two days reprocess in under an hour**. When v2 has caught up, operators check it against finance's **$413,205**, move consumers over, and retire v1. Recovery is a repeatable operation, but a person has to kick it off.

- **The discount-rule drift:** under Lambda, only one of the two implementations received the update. It came to light when Wednesday's overwrite changed Tuesday's dashboard numbers. Kappa rules this failure out completely, since there is only one implementation. In summary: Lambda fixes operational errors automatically but creates the risk of two implementations disagreeing. Kappa gets rid of the duplication but needs someone to run a replay.

Comparison

	Lambda	Kappa	Pragmatic dual path
Codebases per metric	Two, on purpose	One	Two, by accident (fix: shared definitions)
Streaming-bug recovery	Wiped out automatically by the next batch overwrite	Replay from an offset — a routine operation	A manual backfill (if someone notices)
Logic drift	Part of the design	Impossible by structure	What happens by default
History economics	Raw data kept in cheap storage	Log retention (tiered)	Warehouse tables
Demands	Stitching in the serving layer	Mature streaming plus capacity to replay	Discipline that nobody enforces

A verdict ready for interviews: Lambda's overwrite lives on as the habit of scheduled backfills. Engines increasingly support Kappa's single codebase. And many teams are running **accidental Lambda** without realizing it; they should fix it with automatic overwrites or unified logic.

Trade-offs

- **Lambda gains:** scheduled, automatic correction limits streaming errors to one batch cycle; each layer is simpler by itself; and you can always recompute the truth from the raw history. **Lambda costs:** every metric exists twice (a constant risk of drift, plus duplicate infrastructure); and the serving layer has to define where "recent" stops and "settled" starts.
- **Kappa gains:** one codebase, so no drift between implementations; reprocessing becomes a tested migration routine; and the log is the only history. **Kappa costs:** the cost of retention (lowered by tiered storage), capacity for bursts of replay, and complete dependence on the streaming engine. If the pipeline is unreliable, your only source of truth is compromised.

When not to use each

- **Don't Lambda with a small team** — you won't be able to keep two pipelines in agreement. Running two pipelines *without* the automatic overwrite is **accidental Lambda**: you get all of the duplication and none of the recovery. That's the opening incident, and it is very common.
- **Don't Kappa when history is enormous and reprocessing rare** — keeping years of log around for a replay once a year can cost more than rerunning a warehouse job. Also, work that is batch-shaped (large multi-way joins, ML training passes) fits batch better.
- **Don't be dogmatic** — for most teams, a dual path with disciplined backfills and **one shared metric-definition library** (definitions written one time and run by both engines) works better than either pure form.

Real-world examples

- **The founding documents:** Marz's Lambda (from the Storm era) and Kreps' "Questioning the Lambda Architecture" (from the Kafka side). Read together as a debate, they *are* this lesson.
- **Uber, Netflix, LinkedIn** have written about their journeys along this spectrum. The usual story: Lambda first (because of growth), then a move toward Kappa where a log-centred culture developed, and a dual path where warehouses were dominant.
- **Tiered storage** in Kafka took away Kappa's objection about retention. Flink's batch mode, Spark's streaming, and Delta/Iceberg lakehouse tables are where the tools are converging.
- **For AI engineers:** this is **training/serving skew** showing up in infrastructure. Batch code over the warehouse computes the features for *offline training*. Streaming code computes the features for

online serving. If the two define "user's 7-day order velocity" differently, the model gets worse. **Feature stores** solve this by defining each feature once and running that shared logic in both modes.

In the interview

The prompt: "design analytics with a live dashboard *and* accurate finance reports". The trap is building two pipelines without realizing it. A typical follow-up: "your streaming pipeline had a bug for three hours — walk me through recovery."

> **30-second answer:** "There are two needs, and I want one truth. The dashboard needs numbers that are seconds fresh, and the reports need complete ones. The risk is building the metrics twice and watching them drift apart, which is the accidental Lambda most companies end up running. I'd build around the replayable log and lean toward Kappa: one streaming codebase, where recovery means reprocessing. Fix the logic, replay from an earlier offset at about 50× real time, check the result against the old output, and switch over. That's the migration playbook, applied to pipelines. Work that is truly batch-shaped, like huge joins or training passes, reads from the same log and uses the same shared metric definitions. I'd also take Lambda's best idea in the form of scheduled backfills, so any streaming error has a limited lifetime. Whichever way, each metric gets exactly one definition, one owner and one source of logic."

Likely follow-ups (with answers):

- *"The streaming job computed wrong numbers for three hours — recovery?"* — Under Kappa: replay from before the bug using the fixed code. Either the sinks accept the restated numbers (the stream lesson's amended answers, at the scale of the whole architecture), or you swap the output over in one go after verifying it. Under Lambda: there's nothing to do, since the next batch overwrite wipes out that period. Under the pragmatic path: run a backfill job, if the team has that habit. Your answer shows which architecture you really have.
- *"Why not always run both to be safe?"* — If there is no automatic overwrite, running both costs twice as much and produces drift. Even with the overwrite, every metric needs two implementations. The ability to recover comes from *replayable history*. A second pipeline is only one way of using that history.

Mistakes that fail candidates: two pipelines with no plan for reconciling them (accidental Lambda that nobody noticed); suggesting Kappa without doing the math on retention or replay capacity; and treating the choice as a matter of taste instead of budget and team.

Self-check:

- **Q1.** What are Lambda's three layers, and what mechanism justifies building everything twice? → Speed (streaming, fast, provisional), batch (slow, authoritative, recomputed from raw history), and serving (stitches recent + settled together). The justification is the **overwrite**. Each batch run replaces the speed layer's results for its period, so any streaming error (gaps, drops, even logic mistakes) lasts no longer than one batch cycle. Take the overwrite away and Lambda is just two pipelines that disagree.
- **Q2.** Case A: an organization built around logs, metrics that change monthly, and 90 days of history is enough for all reprocessing. Case B: 5 years of history, reprocessed roughly once a year, with heavy multi-way joins. → A: **Kappa**. Retention covers the needed time range. Frequent changes make a single codebase cheaper than keeping two in agreement. And experience with logs supplies the maturity. B: a **batch-anchored dual path**. Keeping 5 years of log for one replay a year isn't worth it, and big joins suit batch. Keep the warehouse as the authority, stream only the operational views, and share the metric definitions.

- **Q3.** You take over the opening incident (numbers that disagree, discount logic that drifted, no overwrite, a monthly meeting). Name the problem, give both ways out with their first steps, and state the invariant. → It is **Accidental Lambda.** Path 1 — finish the Lambda: add the overwrite. First step: write the nightly batch results into the dashboard store, keyed by period. Path 2 — Kappa: merge into the streaming codebase. First step: extend retention beyond the reprocessing horizon and rehearse one replay-verify-switch for a single metric. The invariant for either path: **one definition, one logic source, one owner per metric**.

Related patterns:

- **MapReduce** and **Stream Processing** — the two layers whose disagreement this pattern settles.
- **Message Queue** — the replayable, retained log; the condition that makes Kappa possible.
- **Exactly-Once Semantics** — the correctness bar a single pipeline has to meet before it can be the only truth.
- **Feature Store** — training/serving skew is this same lesson in ML form.

TL;DR

- **Problem:** batch and streaming compute the same metrics in two codebases. These drift apart (through policy differences and forgotten changes) until people have to reconcile them every month.
- **Mechanism:** Lambda — run both, and let a scheduled batch overwrite the speed layer's period (so errors last at most one cycle). Kappa — one streaming codebase over a retained log, reprocessing through replay-verify-switch.
- **Costs:** Lambda — every metric twice, drift built in, seams where results are stitched. Kappa — retention cost, capacity for burst replays, and total dependence on a mature streaming engine.
- **Skip / adapt when:** the team is small (don't run two of anything), history is huge and rarely reprocessed (the warehouse wins), or the compute is batch-shaped — and never run accidental Lambda.

Change Data Capture

Original lesson: Change Data Capture on DesignGurus ↗

What it is. Change Data Capture (CDC) takes the change log a database keeps for itself and turns it into a public stream of events. A **connector** reads the database's **write-ahead log (WAL)**. The WAL is the ordered record of every committed change, which the database keeps so it doesn't lose data. The connector reads it just like a **replica** (a copy of the database kept in sync from that log) would. The difference is that, instead of applying the changes, it *publishes* them. Every committed insert, update and delete becomes an event on a topic, no matter which code path made it, and in commit order. Other systems (search, caches, warehouses, streams) then consume those events.

Analogy. Picture an office. You could ask every clerk to remember to call the other departments whenever they change a file, but they will forget. Instead, you tap into the office's official logbook, where *every* change has to be written down before it counts, and you broadcast each entry.

The problem: four incidents, one cause

Four incidents from earlier in the course have the same root cause:

- **Search (CQRS lesson):** the app wrote to the database, and then wrote separately to **Elasticsearch** (a popular search engine). During a deploy, the Elasticsearch calls timed out. Orders existed in the DB but never showed up in search.
- **Cache (cache-aside lesson):** a bulk import wrote directly to the DB and never **invalidated** the cache (deleted the stale cached copy so the next read fetches fresh data). Products stayed stale for an hour.

- **Analytics (pub/sub lesson):** analytics was not allowed to read the orders tables directly, yet it still needs that data.
- **Warehouse:** loading the warehouse means a full dump every night. That means a 2 AM window, data that is a day stale, and reading *everything* just to find the 1% that changed.

Derived systems need a *complete* record of what changed in the database. But announcing changes ad hoc always leaks. App code can forget to send a notice, and bulk imports or hand-written SQL skip the app altogether. Meanwhile, the database *already* writes down every committed change from every path, in order, in its WAL (this is Module 3's durability mechanism). **What if that log stopped being internal?**

Why the obvious fixes fail

- **Dual-write** — the app updates the DB and then notifies the other systems. No single **transaction** (a group of writes that all succeed or all fail together) covers both writes. So if the app crashes between them, the systems end up **permanently and silently inconsistent**. This is *loss*, not *lag*. (You can see lag and recover from it; a missed write leaves no error behind.)
- **Polling** — running `SELECT * WHERE updated_at > :last_check` on a timer. It has three built-in gaps. (1) **Deletes are invisible**: a row that was deleted can't show up in any query result, so the search index keeps a **ghost row** (an entry whose source row is gone). (2) **Intermediate states collapse**: three updates between two polls look like one final snapshot. That may be fine for a cache, but not for auditing or event processing. (3) It relies on **every writer** keeping `updated_at` correct.
- **Database triggers writing to a change table.** (A **trigger** is code the database runs automatically on every insert, update or delete.) Triggers really do see every write, including the ones that bypass the app. That is the right idea (let the database capture the changes). But triggers add synchronous work to every transaction and need upkeep for each table. And the database already keeps an ordered record of changes *below* the trigger layer.

How it works

A CDC connector **tails** the replication log, meaning it keeps reading the log as it grows. Each of its properties fixes one of the problems above:

1. **It cannot miss.** Every committed change has to pass through the WAL; that is what "committed" means. So edits from admin tools, bulk imports and manual `UPDATE`s are all captured, and the application doesn't need any discipline for this to work.
2. **Deletes arrive as events.** The log records a delete like any other change. CDC publishes it as a **tombstone**, an event that says "this key no longer exists". Deletion becomes a recorded fact (the rule from the vector-clocks lesson), so there are no ghost rows.
3. **Ordered per key, for free** — events come out in the order they were committed.
4. **Zero application changes** — you get a full history of changes without rewriting the app or switching to **event sourcing** (storing every change as an event and deriving the current state from those events).
5. **Delivery is at-least-once** — an event may show up more than once, but it is never lost. That's because after a crash, the connector picks up again from the log position it saved. This means consumers must be **idempotent**: applying an event twice must give the same result as applying it once. Row-image events such as "set name to X" are idempotent by nature.
6. **Snapshot bootstrap.** The first time it connects, the connector takes a consistent snapshot of the table. It then switches to tailing the log with no gap in between. New consumers therefore get the history plus the live stream, in order.

What CDC events are, and aren't. CDC tells you *what* changed ("row 8412: status active → cancelled"). It does not tell you *why* ("customer cancelled due to delay"). These are **row diffs, not business intent**. For data infrastructure such as indexes, caches and warehouses, diffs are exactly what you want. For **domain events** (business-level facts like "OrderCancelled"), use the outbox.

The outbox pattern — the standard replacement for dual-writes. The app writes its business event ("OrderCancelled, reason: delay") into an **outbox table in the same transaction** as the data change. So either both happen, or neither does. CDC then reads the outbox table and publishes the event, business meaning included. **The transaction provides atomicity (all-or-nothing), the log provides delivery, the app provides intent.** This is the "reliable publisher" that the pub/sub lesson promised.

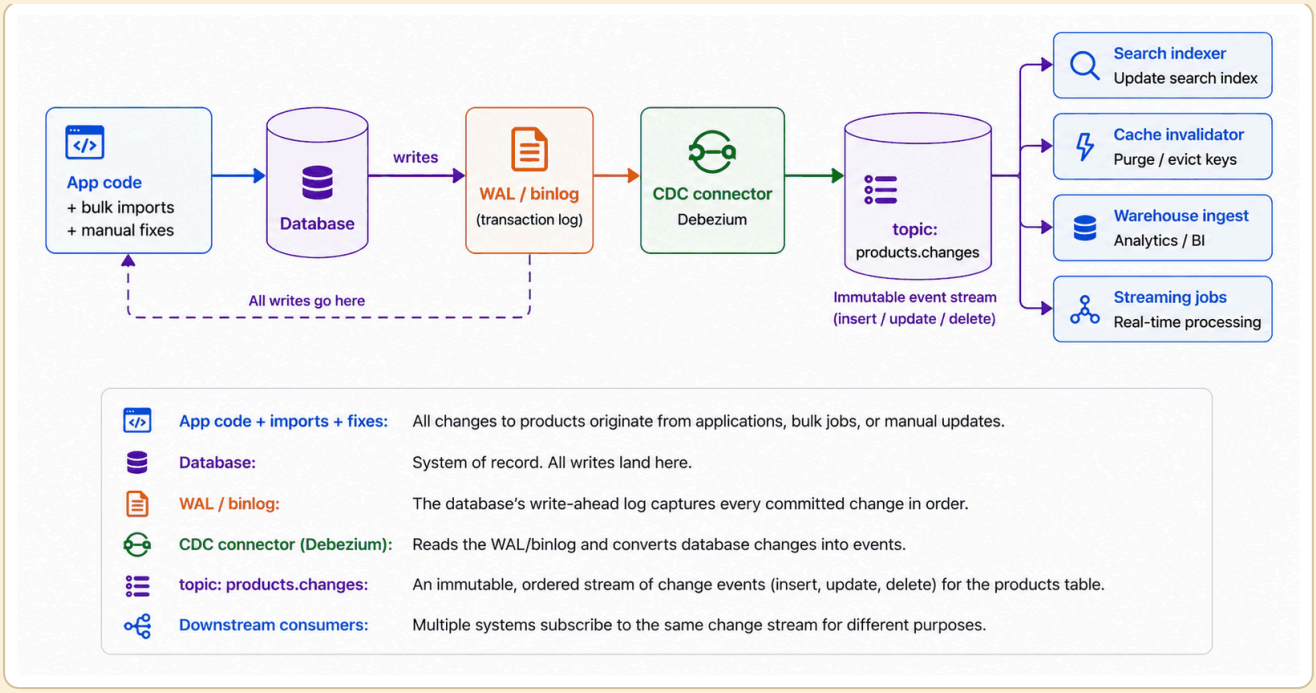


Figure — Change Data Capture (course diagram)

In the figure above, every kind of write — app code, bulk imports and manual fixes — lands in the database, and so ends up in its WAL/binlog (*binlog* is MySQL's name for its change log). The **Debezium** CDC connector (a popular open-source CDC tool) reads that log. It publishes an ordered stream of insert/update/delete events to a `products.changes` topic. Several consumers read that topic independently: a search indexer, a cache invalidator, warehouse ingest, and streaming jobs.

Walkthrough with numbers: the four problems, fixed

- **Search:** Debezium reads the products table, and the indexer applies the changes idempotently. Timeouts during a deploy can no longer cause a permanent mismatch. If the connector is interrupted, the result is **lag** that you can measure and recover from. Freshness stays **under a second**, and bulk imports update the index without any extra import code.
- **Cache:** a consumer deletes cache keys whenever a change event arrives. Forgotten invalidations go away, because every write path that commits produces an event.
- **Warehouse:** the nightly dump read **2 TB to find 20 GB** of changes. Streaming just the changes instead works out to **400 writes/s × ~500-byte events ≈ 200 KB/s**. "Yesterday's data" becomes "seconds ago".
- **Failure drill:** the connector stops on Friday at **6 PM**, and nobody notices until **8 PM**. The database holds on to the unread WAL segments, because the connector's **replication slot** (a bookmark that stores where one log reader has got to) hasn't confirmed them yet. The connector restarts and catches up within minutes — lag, not loss. But if a connector *never* comes back, the retained log grows without limit (self-check Q3).

Three ways to publish a change

	Naked dual-write	Raw CDC (table streams)	Outbox + CDC
Atomicity	None — this is the torn-write bug	Perfect (the log *is* the commit)	Perfect (a single transaction)
Carries intent	It could, but writes can tear	No — only row diffs	Yes — whatever event you choose
App changes	Yes, and every path needs discipline	None	One outbox write for each event
Consumer coupling	Tied to your API	Tied to your table schemas (column names and types)	Tied to a contract you designed
Fits	Nothing — retire it	Your own infrastructure: caches, indexes, warehouse	Domain events that other teams consume

Rule to memorize: use raw CDC for derived infrastructure that you own; use the outbox for events other teams consume; never use dual-writes. Together, these solve the reliable-publishing problem from Module 2.

Trade-offs

You gain:

- **Cannot-miss completeness** — every code path is covered, including bypass writes and deletes. No discipline at the app level ever managed that.
- **One integration, N consumers** — search, cache, warehouse and streams all read the same feed (the economics of pub/sub).
- **Seconds of freshness** rather than nightly updates, and failures that lead to **lag, not loss**.

You pay:

- **Schema coupling — the big one.** Raw CDC events copy the table schemas. So renaming a column affects every consumer, and your internal schema effectively turns into a public API. You can soften this with an outbox for domain events, a **schema registry** (a service that stores event schemas, checks them, and rejects changes that would break compatibility), or a transformation layer between the table schema and the event schema.
- **WAL retention pressure.** A slow or dead connector stops the log from being trimmed (the failure mode named in the WAL lesson). Lag on the replication slot must be a top-priority alert, or it turns into a full disk on the **primary** (the main database server, the one that accepts writes).
- **Diffs, not intent** — consumers that need to know "why" need the outbox.
- **Another critical component** — connector **high availability** (keeping it running through failures, usually with a standby), managing snapshots, keeping topics tidy. It's small, but it sits on the path of everything derived.

When not to use it

- **When other teams want meaning** — don't send them the internal structure of your tables. Use the outbox instead (it's still CDC underneath, but it publishes a contract you designed). Raw table streams are for infrastructure you own.
- **Tiny scale** — a nightly dump may be good enough. CDC's operating cost needs a real payoff in freshness or completeness.
- **As a transaction mechanism** — CDC is **asynchronous** (it happens after the original write, not as part of it), and consumers lag by design. It spreads facts around *eventually*. It does not make two systems atomically consistent (that belongs to Module 7).

- **No log access** on a managed database (one a cloud provider runs for you) — this is getting rare (Postgres logical replication, MySQL binlog and the cloud equivalents are almost everywhere), but check before you commit.
- **Common misuse:** treating raw CDC topics as the company-wide bus for domain events. Then every consumer depends on table schemas, and every migration turns into a negotiation across many teams (the governance problem from the EDA, or event-driven architecture, lesson).

Real-world examples

- **Debezium** — popular open-source connectors that read Postgres, MySQL, MongoDB and others into Kafka. Interviewers often expect you to know it.
- **Postgres logical replication and replication slots** — the mechanism underneath (and the cause of the Q3 disk failure). **MySQL's binlog** is the matching change log.
- **Managed ingest: Fivetran, AWS DMS, Snowpipe Streaming** — CDC and ingestion offered as services someone else runs.
- **Netflix's DBLog** and similar in-house systems — the pattern at the scale of a streaming giant, with snapshot and tail combined.
- **For AI engineers:** CDC keeps feature stores and retrieval indexes fresh. Feature stores read changes from operational tables, so values like "user's current plan" or "orders this week" update within seconds, without app teams having to publish anything. For **RAG** (retrieval-augmented generation), changes to content tables trigger **re-embedding only the changed rows** (the vector-index read model from the CQRS lesson; a **vector index** finds items whose embeddings are close to a query's embedding). That way, stale embeddings don't hurt relevance.

In the interview

The direct prompt: "keep the search index / cache / warehouse in sync with the database". Dual-writes fail because they aren't atomic. Polling is incomplete because it misses deletes and intermediate states. The complete answer is log-based CDC, plus an outbox for domain events.

> **30-second answer:** "I'd attach a CDC connector, Debezium-style, that tails the database's own write-ahead log, and publish every committed insert, update and delete as ordered events on a topic. It can't miss anything. Bulk imports and manual fixes go through the same log as app writes do. Deletes show up as tombstones, which polling can never see by design. The consumers — the search indexer, the cache invalidator, the warehouse — apply events idempotently, because delivery is at-least-once. If the connector goes down, we get lag, not loss. I'd follow two rules. Raw table streams are only for my own infrastructure, since their schema is just my table's schema. Domain events meant for other teams go through an outbox table, written in the same transaction as the change, which gets rid of the dual-write problem. Finally, I'd alert on replication-slot lag, because a dead connector silently stops WAL trimming until the primary's disk is full."

Likely follow-ups (with answers):

- *"Why not just have the app write to both places?"* — Because of the torn write. No transaction covers both the database and the search index, so a crash between the two writes splits them apart silently and forever. That is loss, not lag, and no error is left behind. CDC reads *after* the commit, from the commit's own record. So a split is impossible by structure, and the only thing left is lag you can see.
- *"A row is updated then deleted between two polls — what does polling see vs. CDC?"* — Polling sees nothing. The deleted row can't match the next query, its in-between state was never sampled, and the index is left with a ghost row. CDC sends two events in order: first the update, then the delete tombstone, because the log recorded both when they were committed.

Mistakes that fail candidates: proposing dual-writes with confidence; presenting polling as complete with no answer for deletes; using raw CDC topics as the company's event bus; or not mentioning idempotency alongside an at-least-once stream.

Self-check:

- **Q1.** Why can log-tailing never miss a change, and what two things does timestamp polling miss by design? → Every committed change has to pass through the WAL (that is the meaning of commit). So a WAL reader sees every change from every path, in commit order. Polling misses (1) deletes, because a deleted row appears in no result set, and (2) intermediate states, because N updates between polls collapse into one. On top of that, it depends on every writer keeping `updated_at` up to date.
- **Q2.** Three consumers use order data: (a) your team's search index, (b) an "order shipped" notification sent to partners, (c) the warehouse. Raw CDC or outbox for each? → (a) Raw CDC: your own index wants row diffs, and no app changes are needed. (b) Outbox: partners need a versioned "OrderShipped" contract that carries intent. Published contracts are hard to take back, so never expose internal tables. Write the event in the same transaction as the status change. (c) Raw CDC: you are copying your own data internally. Add a transformation layer downstream for analytics models, plus a schema registry to catch migrations that break compatibility.
- **Q3.** Saturday, 4 AM: the primary's disk is at **94%** and rising, write traffic is normal, and WAL segments are not being trimmed. A replication slot shows a consumer **30 hours** behind — a connector that crashed on Thursday. → The slot prevents the database from deleting WAL segments the connector hasn't confirmed. So a dead connector for a derived system causes endless disk growth, which endangers the *authoritative* database. Triage: if the connector can catch up before the disk fills, restart it (30 hours of log might take tens of minutes). If not, drop the slot to free space, then take a new snapshot and rebuild the consumers. Guards: **alert on replication-slot lag** measured in bytes or time (after a few hours, not when the disk hits 94%); make the connector highly available; and write a maximum-retention policy that says when operators are allowed to drop a slot to protect the primary.

Related patterns:

- **Write-Ahead Log** — CDC publishes the internal log from Module 3.
- **Publish-Subscribe** — one feed, N consumers, with the schema-contract rules fully in force.
- **CQRS** — read models fed correctly, now that dual-writes have failed.
- **Event Sourcing** — the contrast: use CDC when you need a history of changes (diffs), event sourcing when the events *are* the product (intent), and the outbox in between.
- **Cache-Aside** and **Idempotency** — CDC provides complete invalidation events, and consumers have to dedupe.

TL;DR

- **Problem:** derived systems drift away from the database, because every app-level way of announcing changes leaks: dual-writes tear, polling can't see deletes, and bypass writes tell no one.
- **Mechanism:** tail the WAL like a replica and publish every committed change (inserts, updates, delete tombstones) in commit order. Bootstrap with a snapshot, and deliver at-least-once to idempotent consumers. Use an outbox for domain events that carry intent.
- **Costs:** table schemas turn into contracts; replication-slot retention can fill the primary's disk; diffs carry no intent; and there is one more critical connector.
- **Skip it when:** consumers deserve a designed contract (use the outbox), the scale fits a nightly dump, or someone expects it to make systems atomic.

Exactly-Once Semantics

Original lesson: Exactly-Once Semantics on DesignGurus ↗

What it is. This is a group of techniques that make a streaming pipeline's *results* count each event exactly once, even if the messages underneath get delivered and processed more than once. The key insight: **exactly-once *delivery* is impossible** over a network that can fail. But an **exactly-once *effect*** can be built. You keep at-least-once delivery underneath, and you design things so duplicates can't do any harm.

Analogy. Say you are counting people entering a stadium with a hand clicker. A fire alarm goes off and you lose track. You have a photo of the crowd from earlier, so you must recount from that point. You set the clicker back to the number in the photo *and* you count only the people who came in after that moment. If you reset only the clicker, or only redo the counting, the total comes out wrong. The count and your "where I was" bookmark have to be restored together.

The problem: wrong in both directions

A streaming job (a job that reads events from a log as they come in and keeps running results) counts every partner's API calls per day so the company can send invoices. It is the most money-sensitive pipeline in the company.

- **Month one — overbilling.** A routine **crash-and-rewind** happened: the job restarted and read the log again from its last saved point. It reprocessed **40 minutes** of events, and the running counts added those 40 minutes twice. Partner A got a bill for **1,041 calls**, but their own logs said **987**. They ran an audit. The refund cost little; the message "your metering can't be trusted" cost a lot.
- **Month two — underbilling.** To stop the duplicates, operations began committing the **offset** (the job's reading position in the log) *before* processing. Now a crash throws away events that were in flight. That is an undercount, and it can go unnoticed for months.

The two classic failure modes:

- **At-least-once:** an event is never lost, but sometimes one is counted twice.
- **At-most-once:** nothing is ever counted twice, but sometimes an event is lost.

The team wants "exactly once". The idempotency lesson said exactly-once delivery is impossible, and yet Kafka's docs advertise "exactly-once semantics". Both statements are true. Here, exactly-once describes an **effect**, not a promise about delivery.

Why the obvious fixes fail

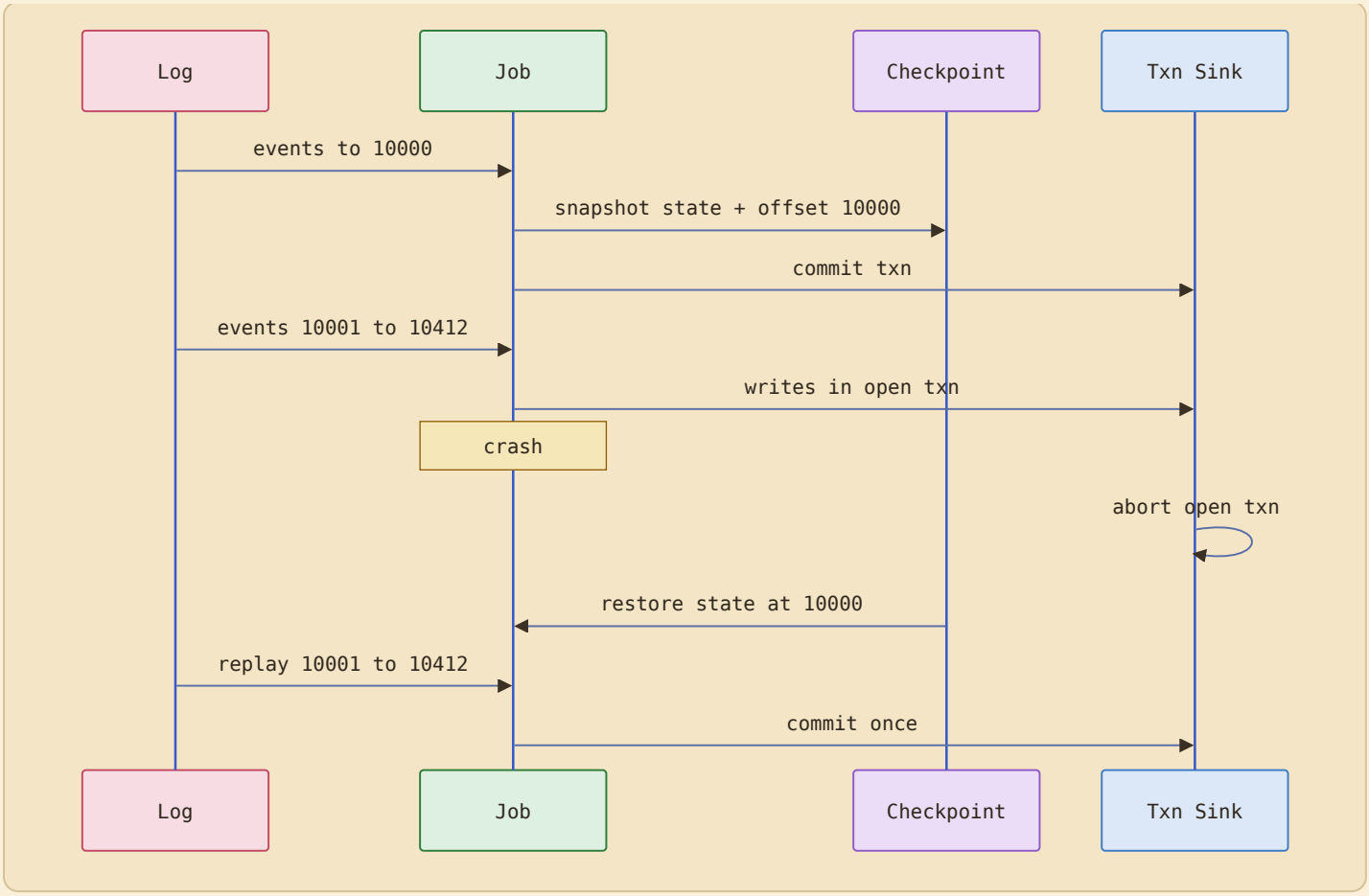
- **"Commit the offset after processing."** That gives at-least-once. If the job crashes after processing but before committing, the batch gets replayed — month one's overbilling. It's the right default, but it isn't enough on its own.
- **"Commit before processing."** That gives at-most-once. If the job crashes after committing but before processing, the batch disappears — month two's silent leak. For money this is strictly worse, because loss happens silently, while duplication at least sets off audits.
- **"Dedupe in the sink with idempotency keys."** (The **sink** is where the pipeline writes its results. An **idempotency key** uniquely names a write, so a repeated write can be ignored.) This technique from Module 5 handles ~80% of cases, where writes are keyed and replace the whole value. But it breaks for **aggregates** (counts and sums built from many events). On a replay, the sink can drop the repeated *output*, but the operator's *internal* running count has already gone up a second time. To be correct, you need state, reading position and outputs all tied together. Deduping outputs doesn't stop state from double-counting, and snapshotting state doesn't stop duplicate outputs.

How it works

Exactly-once effect = at-least-once delivery + atomic coupling at two points. It is the idempotency-key idea stretched over an entire pipeline: set up the bookkeeping so that counting again can't change the total.

1. **Coupling point 1 — state with position.** The stream lesson's **checkpoint** saves the running counts *atomically with* the log offset they belong to. So restore-and-rewind puts the count and the offset back at the same logical moment. The replayed events then update a state that doesn't include them yet, and the total comes out right. **Flink** takes a consistent snapshot of a whole pipeline with many operators using **checkpoint barriers**. These are markers that travel through the stream along with the data. Each operator takes its snapshot when the barrier reaches it, so every snapshot describes the same logical point.
2. **Coupling point 2 — outputs with position.** Replays show up at the sink, and there are two ways to make them harmless:
   - **Transactional sink** (e.g. Kafka). A **transaction** bundles writes so that they all become visible together, or none do. The job writes its outputs *and* commits the offsets it consumed, all in one transaction. Downstream readers configured with `read_committed` see both or neither. If the job crashes partway through a window, the open transaction **aborts**. The partial outputs stay hidden and the offsets stay put. The replay then reprocesses the events and commits once. Under the hood this is a **two-phase commit** (a protocol where a coordinator first asks every participant to *prepare*, and only when all agree tells them to *commit*) inside the log system, where the participants are replicated and the coordinators can recover.
   - **Idempotent sink.** Writes that are deterministic and keyed: `UPSERT (partner, day) = count` (an **upsert** inserts the row if it's missing, or overwrites it if it exists). A replay writes the very same values, so you don't need transactions or the read-committed delay. For most production pipelines this is enough, so try it first.
3. **Kafka's two enablers underneath:** the **idempotent producer** tags every append with a sequence number per producer and per **partition** (one ordered slice of a topic), so the **broker** (a Kafka server) throws away duplicate appends (it's an idempotency key by another name). And **transactions** make writes across many partitions, together with the offset commit, atomic.
4. **The boundary.** The guarantee applies **only inside the transactional log ecosystem**. Outside side effects — sending an email, calling a partner's API, firing a **webhook** (an HTTP call one system makes to notify another) — can't be rolled back or overwritten by a Kafka transaction. For those, Module 5's rules still apply: idempotency keys and deduplication on the receiving side.

Crash recovery with checkpoint and transactional sink



The sequence above is the same as the crash drill below. State and offset are saved together at 10,000. When the crash happens, the outputs for 10,001–10,412 are sitting in an open transaction. That transaction aborts, state goes back to 10,000, and the 412 events are replayed and committed exactly once.

Walkthrough with numbers: billing rebuilt with a transactional sink

- **Normal flow:** counts build up in checkpointed state. Every **2 seconds**, a checkpoint finishes and the open transaction (outputs + offsets) commits. The invoice tables only read committed data, so they move forward in 2-second steps.
- **Crash drill:** the job dies at event **10,412**, and the most recent checkpoint covers up to **10,000**. The open transaction that holds the outputs for 10,001–10,412 aborts, so readers of committed data never see them. State is restored, the log rewinds to 10,000, the same 412 events are processed again from the immutable log, and they commit once. Partner A's count comes out at exactly **987**. At-least-once delivery prevents loss. Recovering state and offset as a pair, plus transactional outputs, prevents double-counting.
- **The bill:** transactions cost about **3–5% throughput** (events handled per second), plus latency from the commit interval. Outputs can only be as fresh as the checkpoint cadence (2 s here). For invoices, that is worth paying. The live dashboard reading the *same* stream picked plain at-least-once with idempotent upserts, and avoids the lag. **Two consumers, two guarantees, one stream** — the guarantee belongs to the question (just like the stream lesson's late-data policy).

Idempotent sink vs. transactional sink

	Idempotent sink (keyed upserts)	Transactional sink (Kafka txns / 2PC)
Mechanism	A deterministic key; a replay overwrites with identical values	Outputs + offsets commit together atomically; readers only see committed data
Running aggregates	Yes, when the output is "current total per key"	Yes, including outputs that aren't keyed or that span several topics
Freshness	Right away	Moves in steps set by the checkpoint/commit cadence
External systems	Any store that can upsert	Only inside the transactional ecosystem
Complexity	Picking a good key	All of the machinery and its fine points
Reach for it	First — the ~80% default	When being atomic with offsets is a must: money, ledgers

Senior summary: you get an exactly-once effect from idempotent upserts where outputs are keyed, and from transactions where outputs have to move atomically with offsets. Neither one reaches into external side effects.

Trade-offs

You gain: correctness good enough for money, even through routine crashes, rewinds and deploys, so billing, ledgers and audited numbers can run on a stream; and no more flipping between at-least-once and at-most-once.

You pay:

- **Throughput and freshness tax** — the overhead of transactions, and read-committed outputs that are only as fresh as the checkpoint cadence.
- **A bounded guarantee** — it ends at the edge of the log system. Assuming external calls are covered is a common misuse.
- **Operational subtlety** — debugging aborted transactions, tuning checkpoints, and **barrier alignment stalls under backpressure** (when one slow stage forces the stages before it to slow down, the barriers slow down too).
- **No protection from wrong logic** — exactly-once runs *incorrect* code exactly once. Fixing the logic still needs the Lambda/Kappa kind of recovery: fix the code, then replay.

When not to use it

- **When approximately-once is honestly fine** — dashboards, trend lines and ops metrics that can live with ±0.1%. Use at-least-once with idempotent upserts (or nothing at all). Keep the heavy machinery for numbers that auditors check.
- **Side-effect-heavy pipelines** — if the main output is emails or calls to partner APIs, transactions can't cover the part that matters. Design the external effects to be idempotent and dedupe on the receiver side instead.
- **When a deterministic keyed upsert is enough** — which is most of the time. Transactions are for outputs that aren't keyed, go to several destinations, or have to move atomically with offsets.
- **Common misuse:** flipping on "exactly-once" as a config setting for a pipeline whose sink calls several external APIs. Duplicates still reach those systems.

Real-world examples

- **Kafka EOS** — idempotent producer + transactions + read-committed consumers. It turned "exactly-once semantics" into a product phrase and a permanent interview topic.
- **Flink checkpoint barriers + two-phase-commit sinks** — prepare when the checkpoint happens, commit when the checkpoint completes (the real-world example the 2PC lesson promised).

- **Spark Structured Streaming** — checkpoints plus idempotent sinks stamped with transaction IDs, running at micro-batch cadence (small batches every few seconds).
- **Google Dataflow** — exactly-once processing as the default on a managed platform.
- **For AI engineers:** token-usage metering (billing per million tokens) is exactly the pipeline from this incident. It is counter-shaped, high-volume and about money, so it runs on this machinery. Usage events go on the log, aggregation is checkpointed, and transactional or keyed-upsert sinks feed billing, while a live dashboard reads the same stream more loosely. Counting eval pass rates and attributing cost per **tenant** (one customer account in a shared system) deserve the same care. This also closes out the caveat from the write-behind lesson: anything an invoice or leaderboard depends on needs state, position and output tied together.

In the interview

The trap version: "Can you guarantee exactly-once delivery?" Start by explaining the distinction (delivery: no; effect: yes, if you build for it), and only then describe the machinery. The applied version: "design usage-based billing on a stream".

> **30-second answer:** "Exactly-once delivery can't be done. Acknowledgments (the receiver's "got it" replies) can get lost, so a sender always has to pick between resending and risking loss. What you can get is an exactly-once *effect*: at-least-once delivery, with duplicates made harmless by tying three things together. First, operator state is snapshotted along with the log offset, so a rewind puts the counts and the bookmark back to the same moment. Second, outputs move together with offsets. That's either one transaction, Kafka-style, with read-committed consumers, or deterministic keyed upserts that a replay overwrites with identical values. I try the upsert first. The boundary I always call out: the guarantee lives inside the log ecosystem. An email sent from the pipeline escapes every transaction, so external effects need idempotency keys and deduplication at the receiver. And for a dashboard I'd skip all of this, because the machinery is for numbers that auditors check."

Likely follow-ups (with answers):

- *"Crash mid-window — walk me through recovery."* — The open transaction aborts, so readers of committed data never see what was in it. State is restored from the checkpoint along with its offset. The log replays the gap, and the reprocessed outputs are committed once. The count stays exact because the restored state and the replay offset describe the same logical moment.
- *"The pipeline also emails a usage alert at 80% of quota — covered?"* — No. The email is an external effect that a Kafka transaction can't undo. Give it its own idempotency key, e.g. `(partner, quota-threshold, period)`. Then replays can retry, and the sending layer throws away duplicates.

Mistakes that fail candidates: saying exactly-once delivery exists; proposing sink dedupe for a running aggregate (the state still double-counts); putting transactions on a dashboard; and having no answer for external side effects.

Self-check:

- **Q1.** What are the two coupling points, and what breaks if you have only one? → State ↔ position: without this, a rewind replays events into a state that already contains them. That is internal double-counting, and no sink can repair it. Outputs ↔ position (transactional commit or deterministic upserts): without this, a replay sends out again results that downstream has already consumed. You need both.
- **Q2.** Choose the correctness machinery for (a) partner billing, (b) an ops dashboard of orders per minute, (c) a job whose output is webhook calls to partners. → (a) The full treatment:

checkpointed state, a transactional sink (or keyed upserts if the shape of the output allows it), and read-committed consumers. Money justifies the cost. (b) At-least-once with idempotent upserts keyed on `(metric, minute)`, or simply accepting approximation. No auditor, so no tax. (c) The machinery can't cover external calls. Use at-least-once processing and make each webhook idempotent (event IDs that the receiver dedupes, and retries following Module 5). The guarantee depends on where the output ends up.

- **Q3.** The full exactly-once machinery is in place, yet partner B's count comes out **~2% high** every month. Their mobile SDK re-sends usage events on timeout, each time with a **freshly generated event ID**. → The duplicate was created *upstream* of the log. The pipeline counted correctly, once each, two events with different IDs that happen to describe one API call. Nothing downstream can tell them apart. Fix it where it starts (the first rule of the idempotency lesson): the client creates the event ID *before* the first attempt and reuses it on every retry. Then ingestion (or the first keyed stage) dedupes on that ID. **End-to-end exactly-once is a chain.** Every hop from the SDK to the invoice must either avoid creating duplicates or carry the key that exposes them.

Related patterns:

- **Idempotency** — the foundation. Its rule about client-generated keys covers everything upstream of the log.
- **Stream Processing** — checkpoints provide the state-position coupling; this lesson seals up the sinks.
- **Two-Phase Commit** — the protocol behind the transactional sink, run in a controlled setting.
- **Message Queue** — the immutable, replayable log that makes recounting possible.
- **Write-Behind** — the billing caveat from Module 4, now resolved: dashboards may approximate, invoices use this.

TL;DR

- **Problem:** streaming aggregates that touch money swing between failure modes. At-least-once double-counts on every rewind, at-most-once silently loses events, and sink dedupe can't repair state that gets incremented again.
- **Mechanism:** at-least-once underneath, plus two atomic couplings. State is snapshotted together with the log position. Outputs are committed together with offsets (Kafka transactions + read-committed) or written as deterministic keyed upserts. Try upserts first.
- **Costs:** a few percent of throughput, freshness tied to the checkpoint cadence, subtle operations, and a firm boundary at external side effects.
- **Skip it when:** there is no auditor (dashboards), the outputs are external calls (make those effects idempotent instead), or a keyed upsert already does the job.

Backpressure

Original lesson: Backpressure on DesignGurus ↗

What it is. Sometimes a producer sends data faster than its consumer can handle it. When that happens, something has to give. A buffer fills up, data gets thrown away, or the producer slows down. **Backpressure** means *choosing on purpose* which of these happens. When slowing down is the choice, it also means carrying the consumer's "slow down" signal back upstream. If nobody makes a deliberate choice, the default is a buffer with no limit, and that eventually crashes with out-of-memory (the process has used up its RAM and is killed).

Analogy. Picture a kitchen where the chef plates food faster than the waiters can carry it out. You could use a heated shelf that only holds so many plates (a bounded buffer). You could throw away dishes that go cold, because only fresh ones count (drop). Or you could tell the chef to wait

until a waiter is free (slow the producer). What you can't do is let plates pile up on the floor forever.

The problem: rate mismatch is normal

You have already seen this three times. In the bidirectional-streaming lesson, a slow phone filled up a gateway's memory. In the queue lesson, a growing queue depth revealed a mismatch. And the rate limiter's **429** ("Too Many Requests") put a limit at the edge of the system. The rule behind all three: **producers outrunning consumers is a normal operating condition, not an exception.** Consumers get redeployed, pause for **garbage collection** (the language runtime briefly stopping to reclaim unused memory), run into slow **sinks** (the place where a stage writes its results), and change speed over time. So every link between a faster producer and a slower consumer has to decide where the extra data goes.

Here's the math of the default. A producer at **5,000 events/s** feeding a consumer at **3,000/s** piles up **2,000 buffered events every second**. With a buffer that has no limit, the process *will* run out of memory unless the mismatch stops. It's a crash with a timetable, and the size of the mismatch sets the timetable.

How it works: exactly three responses

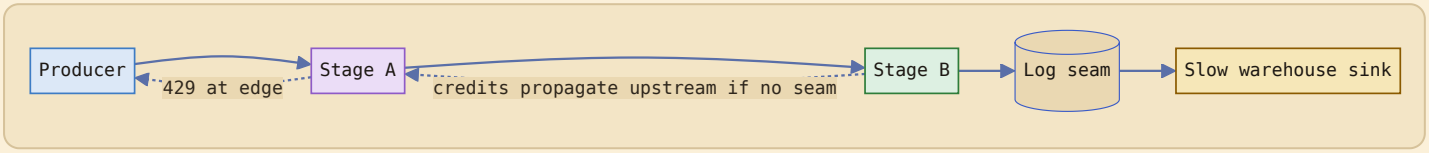
Every system chooses at least one of these for each stream:

1. **Buffer it (bounded).** Hold on to the extra data when you expect the mismatch to be *temporary*. This is only correct if the buffer has a fixed maximum size, and the mismatch is a burst in front of a consumer that can catch up. A buffer with no limit isn't a strategy; it's a failure you can predict.
2. **Drop it (by explicit policy).** Throw the extra data away when old data is worthless. A video call has no use for frames from three seconds ago. A metrics stream can sample. A "current location" update replaces the last one, so **drop-oldest, keep-latest** acts like a buffer with just one slot. Dropping is only a failure when it happens *by accident*.
3. **Slow the producer.** Send the consumer's available capacity back upstream, so the source produces at a rate that can be sustained. Examples:
   - **TCP flow control** — **TCP** is the internet protocol that delivers bytes reliably and in order; the receiver's *window* tells the sender how much it is allowed to send.
   - **Kafka's pull model** — consumers ask for each batch only when they are ready, so the consumer's pace *is* the flow rate. You get backpressure for free, with no return signal needed.
   - **Reactive streams** (a standard API for asynchronous streams with built-in backpressure) `request(n)` — the consumer gives permission for *n* more items.
   - **Flink credit-based channels** — when an operator slows down, its reduced capacity (its credits) passes back stage by stage until it reaches the source.

A log between stages. A common design answer is to put a Kafka topic between the producer and the consumer. It works as a huge buffer bounded by *disk*, with **retention** (how long each event is kept available). The two sides then run independently, and any mismatch shows up as **consumer lag** (how far behind the consumer has fallen). Lag can be measured, alerted on, and used to **autoscale** (automatically add or remove consumer instances) based on how old the backlog is. It doesn't remove the limit: retention is finite, and lag that goes past retention means lost data. What it does is turn a sudden full-buffer crash into a backlog you can see and recover from.

Inside a streaming topology (the chain of processing stages that a stream passes through), backpressure makes the *whole pipeline* run at the speed of its slowest stage. And since checkpoint barriers travel along the same channels as the data, they slow down too. So **checkpoint timeouts are often the first visible symptom, even when the cause is several stages away** (see self-check Q3).

Backpressure propagation and the log seam



In the diagram above, imagine there were no log between stage B and the slow warehouse sink. The slowdown would then travel upstream as shrinking credits (the dotted arrows), until it hit the edge as a 429 sent to the producer. With the log seam in place, the sink can fall behind by itself without dragging the rest of the pipeline down with it.

The three responses side by side

	Bounded buffer	Drop (by policy)	Slow the producer
Where the excess goes	Into memory or disk, for a while	Thrown away, on purpose	It is never created
Fits	Bursts in front of a consumer that can keep up	Data where only freshness matters	A mismatch that lasts
Failure mode	A full buffer still needs a backup plan	Dropping the wrong data	Pressure reaches the edge (users notice)
The log's version	The retention window	Lag beyond retention	Pull-based consumption

Real pipelines mix all three: bounded channels inside, logs between stages, drop policies for streams where only freshness matters, and 429s at the boundary. Each one is an explicit policy.

Trade-offs

You gain: predictable memory, since every buffer has a limit; mismatches you can see, since lag and buffer fill become signals you can operate on; and controlled bursts, since buffers soak up short ones and propagation deals with long ones.

You pay:

- **Someone upstream feels it.** The slowdown spreads until it hits the edge of the system — an upload gets throttled, or an API returns 429. At that boundary, backpressure *turns into* rate limiting, so decide on purpose where clients will run into the signal.
- **Latency under pressure** — when bounded buffers are full, every event waits. The pipeline is alive, but slow.
- **Whole-pipeline coupling** — one slow sink sets the pace for everything and holds up checkpoints, unless logs between stages isolate it.

When dropping is right (and when it never is)

- **Freshness-only data** — live positions, video frames, sensor readings where the newest value replaces the rest. Here drop-oldest is better than slowing the producer. Nobody needs the full history of where a driver has been.
- **Sampled observability** — metrics and **traces** (records of one request's path through many services) are built to shed data. **Tail sampling** (deciding which traces to keep only after seeing how each request ended) is dropping done on principle.
- **Never for money or facts** — billing events, orders, anything covered by exactly-once guarantees has to stay in a log or slow the source. Dropping them is data loss.
- **Common misuse: hidden unbounded buffers** — in-memory queues, **thread-pool** queues with no limit (a thread pool is a fixed set of worker threads that take tasks from a queue), default async channels. Each is an out-of-memory crash waiting to happen at some unknown time. Check every "unbounded" setting, because each one is a capacity decision that nobody actually made.

In the interview

The direct prompt: "your producer emits 5,000/s but the consumer handles 3,000 — what happens?" This tests whether you know the three responses and the unbounded-buffer failure. A debugging version: "why have our checkpoints started timing out?"

30-second answer: "Something has to give: you buffer, drop, or slow down. The one wrong answer is a buffer with no limit, because it turns a mismatch you could see into an out-of-memory crash on a timer. For bursts, I'd use bounded buffers. For data where only freshness matters, I'd drop on purpose and keep the latest. For a mismatch that lasts, I'd push the pressure upstream, and pull-based consumers give you that for free. That's one reason a log between stages is the go-to design: the mismatch turns into lag that you can see, alert on, and use to drive autoscaling. Inside a topology, a slow sink sets the pace for the whole pipeline and stalls checkpoints. At the very edge, backpressure becomes a 429 with Retry-After (an HTTP header telling the client how long to wait before trying again). The signal has to end up somewhere; the design choice is where."

Likely follow-ups (with answers):

- *"Why do pull-based systems get backpressure for free?"* — Because the consumer starts each exchange. It asks for the next batch only when it's ready, so its speed is the flow rate. When no request comes, that *is* the signal. Push systems have to build a separate return channel (credits, windows, 429s).
- *"If every stage slows, where does the pressure finally go?"* — Back to the original producer at the edge of the system — a user, partner or device. It shows up as a throttled upload, a 429, or "retry later". Backpressure never just vanishes. So good designs pick that boundary on purpose, and when capacity runs out they shed load according to priority tier.

Mistakes that fail candidates: saying "we'd buffer it" without a limit; dropping billing events; and not linking checkpoint stalls to a slow sink.

Self-check:

- **Q1.** Name the three responses, where each one fits, and the classic mistake. → Bounded buffering for bursts in front of a consumer that can keep up. Deliberate dropping for data where only freshness matters, and for sampled observability. Slowing the producer for a mismatch that lasts (through credits, windows, pull, or 429s). The classic mistake is an unbounded buffer, which quietly commits you to running out of memory.
- **Q2.** Pick responses for: (a) order events that feed billing; (b) live driver locations shown on a map; (c) a 10× burst lasting 90 s, hitting a consumer sized for normal steady traffic. → (a) Buffer in the log and never drop (this is exactly-once territory). If the mismatch lasts, scale the consumer using backlog age as the signal. (b) Drop-oldest, keep-latest — a buffer with one slot per driver. (c) A bounded buffer big enough to hold 90 seconds of 10× traffic, with the log's retention as the deeper version of the same idea.
- **Q3.** Checkpoints begin timing out at **2 PM**. By **2:20**, end-to-end lag is rising and alerts are going off. There were no deploys. At **1:55** the warehouse sink slowed down sharply because of a lock on its target table. Work out what happened and how to contain it. → At 1:55 the sink slowed, and its bounded input channels filled up. Credit-based flow control then cut capacity one operator at a time, moving upstream. By 2:00 the entire topology was running at the sink's speed. The checkpoint barriers, which travel in the same channels, slowed down and timed out. That was the first alarm, several stages away from the cause; the lag alerts followed ~25 minutes after the cause. Backpressure protected memory exactly as designed; the symptoms just showed up far from the source. To diagnose it, use Flink's per-operator backpressure metrics and look for the busy stage whose downstream is idle. To contain it, put a **log between the pipeline and the warehouse sink**. The sink then falls behind on its own topic (bounded by retention and monitored by age), while the main pipeline and its checkpoints run at full speed.

Related patterns:

- **Message Queue** — a log separates the stages and turns a mismatch into lag you can measure.
- **Stream Processing** — propagation inside a topology, and the checkpoint stalls that give it away.
- **Rate Limiting** — backpressure at the boundary: 429 + Retry-After is "slow the producer" said to callers outside the system.
- **Bidirectional Streaming** — the version for a single connection: bounded send buffers, and drop-and-resync for each socket.

TL;DR

- **Problem:** producers outrunning consumers is the normal state of things. The default response (a buffer with no limit) schedules an out-of-memory crash instead of making the mismatch visible.
- **Mechanism:** choose for each stream. Bounded buffers for bursts, deliberate drops for data where only freshness matters, and slow-down signals passed upstream (credits, windows, pull, 429s) for mismatches that last. Logs between stages turn crises into lag you can see.
- **Costs:** someone upstream always feels it (so design where), latency goes up under pressure, and the slowest stage sets the whole pipeline's pace unless seams isolate it.
- **Drop when:** the newest value replaces older ones (positions, frames) or the stream is designed to be sampled — never for facts that auditors check.

Partitioned Consumption

Original lesson: Partitioned Consumption on DesignGurus ↗

What it is. This is how you process a stream in parallel *without* losing the order of events for each entity. The stream is divided into **partitions** using a key (e.g. customer ID). Every event with a given key goes to the same partition, and each partition is read by exactly one consumer at a time. The contract: **order per key, parallelism across keys.**

Analogy. Think of a supermarket with many checkout lanes, where each family is always sent to the same lane. Inside a lane, items are scanned strictly in order. Different families are served at the same time in different lanes. Hiring more cashiers than there are lanes doesn't help. And one family with twenty trolleys holds up its lane, no matter how many other cashiers are standing around.

The problem

- **Parallel consumers break ordering.** In the queue lesson's Q3, a refund got processed before its charge. Two workers had picked up events for the same customer independently.
- **Global order kills parallelism** — a single consumer handling everything in order can't scale.
- **Keyed streaming state** (like the per-card counts from the stream lesson) *depends on* every event for a key reaching the same operator. Kafka-style **consumer groups** are the one mechanism behind all three.

How it works

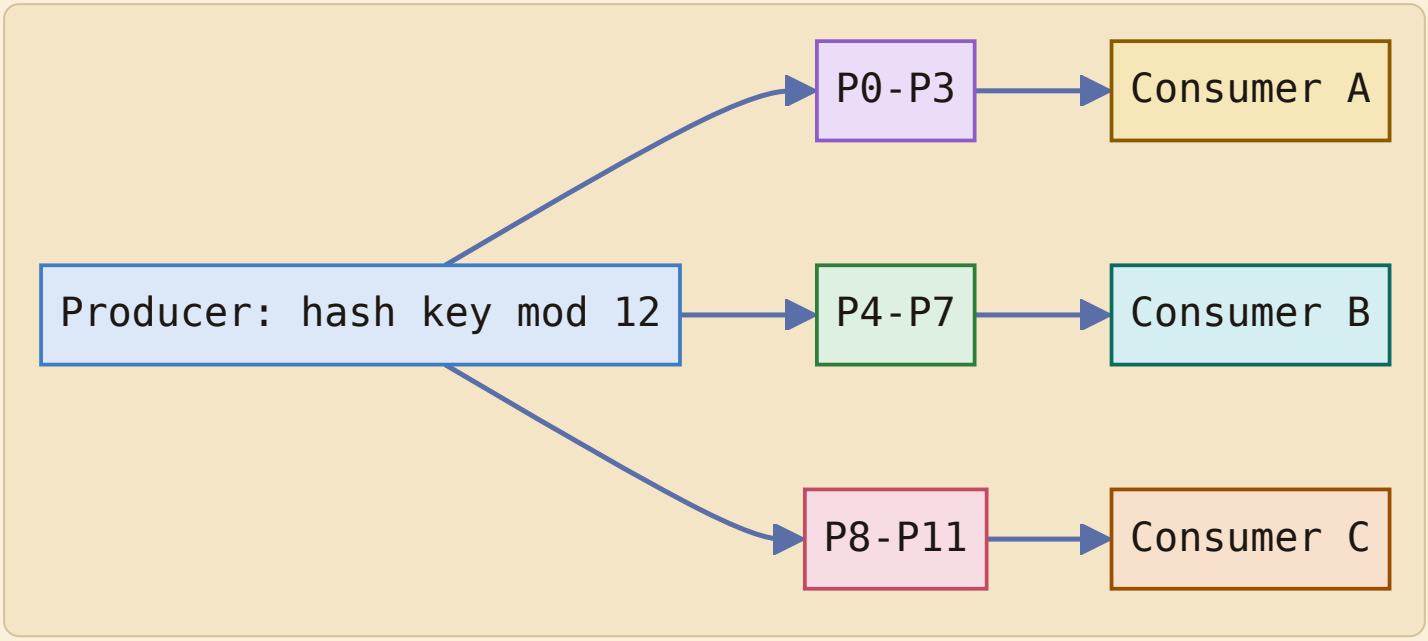
```
topic: orders (12 partitions, keyed by customer_id)

producer:  partition = hash(customer_id) % 12
           → customer 8412's events ALWAYS land in partition 7, in order

consumer group (3 consumers):
   consumer A ← partitions 0-3      each partition has exactly ONE
   consumer B ← partitions 4-7      consumer; each consumer owns its
   consumer C ← partitions 8-11     partitions' offsets (bookmarks)
```

1. **The producer routes by key hash.** A **hash** function turns a key into a number, always the same number for the same key, and `% N` (modulo) keeps the remainder after dividing by N. So `partition = hash(key) % N` always gives the same answer for the same key. So every event for one key ends up in one ordered partition, in the order it was produced. Pick the key the way you'd pick a shard key: it has to keep related events together *and* spread the load evenly.
2. **The exclusive lease.** Inside one consumer group, exactly one consumer owns each partition at any moment. That exclusivity is what keeps things in order: only one reader processes that partition and moves its position forward. *Different* groups each receive all the partitions on their own (pub/sub fan-out).
3. **Offsets per partition.** An **offset** records how far a consumer has read in a single partition. If you commit the offset *after* processing, you get at-least-once delivery (a message may be seen twice). So handlers need to be **idempotent**.
4. **Rebalancing.** When a consumer joins or dies, the group hands out the partitions again. Processing pauses briefly while ownership, offsets and any local state move over. For stateful consumers, the state has to move too. Frequent **rebalance storms** usually come from unstable consumers, or from processing that takes longer than the session timeout (so the group decides the consumer is dead).
5. **Partition count = maximum parallelism.** 12 partitions can support at most 12 active consumers; a 13th has nothing to do. Raising the count later is disruptive. Changing `% 12` to `% 24` sends keys to new partitions in the middle of the stream. That can break per-key order and split existing keyed state. This is the streaming version of the **modulo trap**. So pick plenty of partitions from the start; partitions you don't use yet are cheap.

Two problems you've seen before, in a new form:

- **The hot partition.** A **hot key** (one with much more traffic than the rest) sends all its volume to a single partition. That could be 40% of all the work landing on one consumer. Hashing spreads *keys*, not *weight* (the same skew you saw with shards, cache keys and reducers). Remedies: cache reads, split or **salt** the heavy key (a composite key, but only if its ordering needs allow it), pre-aggregate, or handle the extreme keys separately (e.g. give them their own topic).
- **Per-lane head-of-line blocking** (one stuck item holding up everything queued behind it). A **poison message** (one that fails every time it's processed) only blocks its own partition, not the whole topic. But in lanes that must stay strictly in order, moving it to a **dead-letter queue** (DLQ: a side queue where messages that keep failing are parked for later) changes the order. So those consumers must **pause and fix** instead of using a DLQ.

Key-hashed partitions with exclusive consumer ownership



The diagram above matches the code block. The producer hashes each key into one of 12 partitions, and each group of partitions belongs to exactly one consumer.

Partitioned log vs. competing-consumer queue

	Partitioned log (Kafka-style)	Competing consumers (SQS-style; Amazon's simple cloud queue)
Ordering	Strict for each key (each lane)	None — any worker can take any message
Scaling granularity	By partition; the count is fixed at the start	By worker; add one whenever you like
Replay / history	Built in — rewind the offsets	Gone as soon as it's acknowledged
Poison message	Blocks its lane (pause or DLQ, by policy)	Blocks nothing (DLQ it freely)
Fits	Keyed facts: orders, events, state feeds	Independent jobs: renders, emails, resizes

The ruling: **independent jobs → competing consumers; ordered facts → partitioned logs.** Within one key, you can't have strict order and unlimited parallelism at once. The partition count sets how much parallelism is available.

Trade-offs

You gain: strict order per key *and* horizontal parallelism, which plain competing consumers can't give you together; locality for free, since every event for a key reaches one consumer (this is what makes keyed streaming state possible); and a limited blast radius, since a poison message blocks only one lane.

You pay:

- **A parallelism ceiling chosen in advance** — repartitioning is disruptive.
- **Rebalance pauses** and storms — tuning liveness settings, moving state, and hiccups at deploy time.
- **Skew exposure** — the lane with a hot key runs hot while other lanes sit idle.
- **Order only per key** — there is no order across keys. It's rarely needed, but people often assume it wrongly.

When not to use it

- **Order doesn't matter** — independent jobs (like image resizes) fit a plain competing-consumer queue: easier scaling, no lanes, no rebalances.
- **You need global order** — that means a single partition and a single consumer, which puts a cap on throughput. First ask what *really* needs to be in order. Usually it's per entity, which this pattern already handles.
- **Tiny throughput** — one consumer and one lane are enough.
- **Common misuse:** using random or **round-robin** keys (assigned in turn, in a fixed rotation) while assuming each entity's events stay in order. That brings back refund-before-charge. The key *defines* the scope of the ordering.

In the interview

The prompt: "process events in parallel but keep per-user order". Follow-ups: partition count, rebalancing and hot keys.

> **30-second answer:** "I'd partition the topic by the entity key, such as customer ID, so each key's events form one ordered lane. Then each partition goes to exactly one consumer in the group. That exclusivity is what guarantees the order, and the parallelism comes from the number of lanes. I'd choose the partition count up front with room to grow, because it's the ceiling on parallelism, and resizing moves keys around mid-stream. I'd expect two classic

> pains. One is rebalance pauses when consumers join or leave. The other is hot partitions when one key dominates; hashing can't fix that, because it spreads keys, not weight. A poison message blocks only its own lane. If the lane has to stay strictly ordered, the answer is pause-and-fix rather than dead-lettering. Jobs that don't care about order skip all of this and use plain competing consumers."

Likely follow-ups (with answers):

- *"How many partitions?"* — Work backward from consumer throughput: 50,000 events/s ÷ 2,000 per consumer = **25 minimum**. Then add a generous margin (e.g. 48–96, or more if growth is fast), because partitions are cheap and resizing is not. The count should cover 3–5 years of growth in parallelism.
- *"You added consumers and throughput stopped improving — why?"* — There are two possible limits. (1) There are already as many consumers as partitions, so the extra ones have no work. More partitions would help, but that means disruptive repartitioning. (2) One partition holds a hot key. Extra consumers can't split a partition, so use techniques for managing skew.

Mistakes that fail candidates: assuming order across partitions; using random keys when order is required; or casually suggesting you'll add partitions later without dealing with keys being rerouted.

Self-check:

- **Q1.** State the contract, and say which component enforces each half. → "Order per key, parallelism across keys." Routing by a deterministic key hash puts each key's events in one lane (the first half of ordering). The group's exclusive lease of one consumer per partition prevents reads from being interleaved (the second half). The number of lanes × one consumer each gives you the parallelism.
- **Q2.** 50,000 events/s, 2,000/s per consumer, per-customer order required, and volume doubling every 18 months. Choose the partition count. → The minimum is 25. With growth: ~50 consumers within 2 years, ~100 within 4. Choose **96–128** partitions. Partitions are cheap. Resizing, on the other hand, is surgery that reroutes keys and breaks per-key order during the switch, and the count limits *future* parallelism. Run 25 consumers today, each holding 4–5 partitions, and for years scale by adding consumers, not partitions.
- **Q3.** Payments are keyed by `customer_id`. Throughput stopped rising after going from 12 to 20 consumers, and partition 7's lag is **40×** that of every other partition. → Partition 7 is a hot partition. One very large enterprise customer's entire volume lands there, which holds up completion while other consumers sit idle. **Extra consumers could never fix this**, since each partition has only one reader. Handle that tenant separately (its own topic), or use a composite key like `customer_id + region` *if* its ordering needs allow it. Or pre-aggregate / batch reads upstream. (The plateau also happened because consumers beyond 12 had no partitions to read.) The lesson: look at lag per partition before looking at total lag.

Related patterns:

- **Message Queue** — this adds per-key ordering on top of the queue's delivery guarantees.
- **Stream Processing** — keyed state is possible because each key is delivered to one place.
- **Sharding** — the same discipline around keys and the same skew problem, applied to computing over streams.
- **Dead Letter Queue** — the poison-message policy per lane: DLQ where order is loose, pause-and-fix where it is strict.

TL;DR

- **Problem:** parallel consumers wreck ordering, global ordering wrecks parallelism, and keyed state needs each key's events in a single place.
- **Mechanism:** hash keys to partitions (one ordered lane per key), give each partition an exclusive single consumer inside a group, and keep offsets per lane. The partition count is a ceiling you choose ahead of time.
- **Costs:** repartitioning is surgery (so choose with years of room to grow), rebalance pauses, skew from hot partitions, and order only within a key.
- **Skip it when:** order doesn't matter (use competing consumers) or throughput is tiny — and never assume an order that your keys don't promise.

Inside Kafka

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. A look inside the log that Stream Processing, CDC, Exactly-Once, Backpressure and Partitioned Consumption all assume: how a broker stores a partition, why that is fast, how replicas agree on what is committed, how consumer groups share work, and which settings each guarantee depends on. (Designing such a system from scratch, with capacity numbers, is Case 4 in Module 13. Client code in Spring is in the Spring book's Kafka in five minutes and Producer guarantees.)

Analogy. A paper ledger copied by clerks: the head clerk only ever writes at the end, two assistants copy new lines, and an entry is "on the record" once every clerk who is keeping up has copied it. Readers keep their own bookmarks; old pages are archived after a set time.

The problem: three beliefs, three incidents

The team's Kafka knowledge was three slogans. Each one cost an incident:

- **"Kafka never loses messages."** Producers used `acks=1` (the leader confirms as soon as *it* has the write). A broker's disk controller failed before followers copied the last ~40 ms: **~800 confirmed orders vanished** (≈20,000 orders/s × 0.04 s).
- **"Replication factor 3 means three copies."** (The **replication factor** is how many copies of each partition Kafka keeps: one **leader** that takes writes and **followers** that copy from it.) Both followers of a partition fell out of sync; the topic accepted writes with one in-sync copy, then that broker died — and an out-of-sync follower was allowed to become leader, so offsets **went backwards** with a gap of missing events.
- **"Just add consumers."** A consumer taking 6 minutes per batch was declared dead (limit 5 minutes), its partitions moved, it came back, the group rebalanced again — a **rebalance storm** that stalled processing for an hour.

None of these was a Kafka bug. Each was a setting whose meaning nobody had checked.

Why the obvious fixes fail

- **"Set replication factor 3 and relax."** Replication factor is how many copies *can* exist. What protects a write is how many copies exist *before the producer is told "ok"* — that's `acks` together with `min.insync.replicas`.
- **"Force every write to disk (fsync)."** (**fsync** forces the operating system to push data from memory onto the physical disk.) Kafka deliberately relies on copies on *other machines* (ideally other **zones**, meaning separate data centres within one cloud region) instead; per-message fsync would cut throughput heavily. Durability comes from replication, not one disk.
- **"Turn on exactly-once."** Kafka's exactly-once covers *reading from Kafka, processing and writing back to Kafka*. It does not stop your consumer from sending the same email twice. External effects still need the Idempotency and Exactly-Once Semantics lessons' machinery.

How it works

1. Storage: a partition is a folder of segment files. A **topic** is split into **partitions**; each partition is an ordered, append-only log with its own offsets (0, 1, 2, …). On a broker's disk a partition is a directory of **segments**:

- `00000000000000500000.log` — the records themselves, grouped into **record batches** (a batch carries its base offset, timestamps, a **checksum** (a small value computed from the bytes to detect corruption), and producer ID and sequence numbers used by idempotence). The file name is the first offset in the segment.
- `.index` — a **sparse offset index**: an entry roughly every 4 KB of log, mapping offset → byte position. To find offset 500,123, Kafka **binary-searches** the index (repeatedly halves the range it is looking in), jumps near it and scans a few KB.
- `.timeindex` — timestamp → offset, so a consumer can "rewind to 09:00".
- Only the newest (**active**) segment is written to. When it reaches its size limit (1 GiB by default) or age, it is closed and a new one starts.

Retention works on whole segments. With `cleanup.policy=delete`, closed segments older than the retention time (7 days by default) or beyond a size cap are deleted — deleting a file is cheap, no per-message work. With `cleanup.policy=compact`, a background cleaner keeps only the **latest record per key**, and a record with an empty value (a **tombstone**) eventually removes the key — this is how Kafka stores consumer offsets and how CDC reference tables stay small (Log compaction in DDIA). **Tiered storage** (production-ready since Kafka 3.9) moves closed segments to object storage, so long retention no longer needs large broker disks.

2. Why it is fast. Five choices, all about avoiding work:

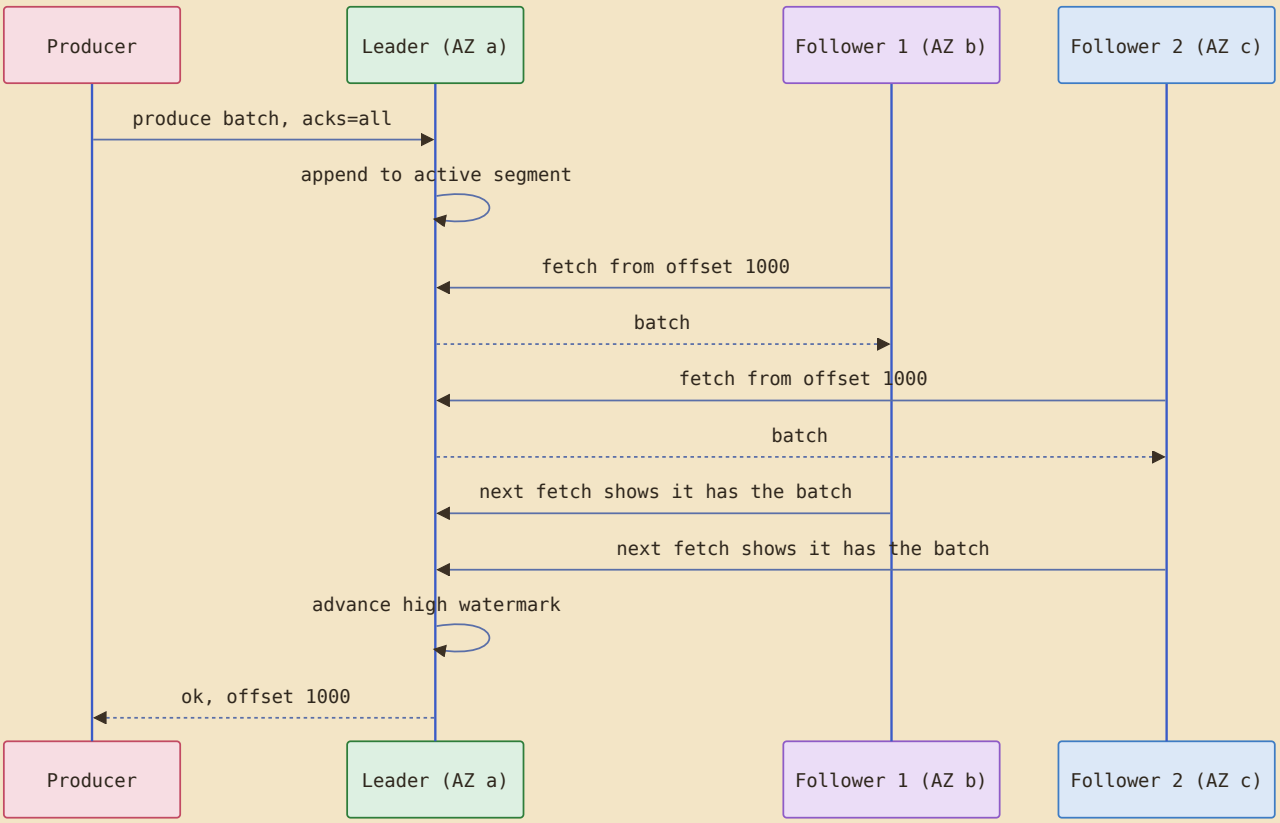
- **Sequential I/O** — appends and forward reads are the fastest thing a disk does (hundreds of MB/s even on spinning disks).
- **The OS page cache** (memory the operating system uses to hold recently read or written file data) — no big in-process cache; consumers that keep up read recent data from kernel memory.
- **Zero-copy** — bytes are never modified, so `sendfile` moves them from page cache to **socket** (the operating system's endpoint for a network connection) without passing through the application (**TLS**, the encryption behind HTTPS, needs them in user space, so encrypted clusters lose this and need more CPU).
- **Batching and compression end to end** — producers fill per-partition batches (waiting `linger.ms`), compress them; brokers store them as is; consumers decompress. Fewer requests, 3–5× fewer bytes for text-like data.
- **Pull with long polling** (the server holds a request open until data arrives or a time limit passes) — "everything after offset X, up to 1 MB, wait up to 500 ms"; a consumer's progress is one number per partition.

3. Replication: leader, followers, ISR and the high watermark. Each partition has one **leader** replica (it serves all writes and, by default, reads) and followers on other brokers — placed in different **availability zones** (AZs) when brokers are labelled with their rack/zone. Followers *fetch* from the leader just like consumers.

- **LEO (log end offset)** — the next offset a replica will write; each replica has its own.
- **ISR (in-sync replica set)** — the leader plus followers that have caught up within `replica.lag.time.max.ms` (30 s by default). A slow follower drops out of the ISR, so it can't hold back commits, and rejoins after catching up.
- **High watermark (HW)** — the highest offset that every ISR member has. Records below it are **committed**. Consumers can read only up to the HW, so they never see a record that a leader **failover** (switching to a new leader after the old one fails) could erase.
- **Leader epochs** — each new leader increments an epoch number. A returning old leader uses it to find where its log diverged and **truncates** its uncommitted tail, rather than keeping records nobody else has.
- **The controller** — a small **quorum** (a group that decides by majority) of controller nodes (KRaft mode, using the **Raft consensus algorithm**, a protocol that lets several nodes agree on the same data even

if some fail; **ZooKeeper**, the separate coordination service Kafka used to depend on, was removed in Kafka 4.0) stores metadata, detects dead brokers and elects new partition leaders from the ISR.

A produce with acks=all: the producer is answered only after every in-sync replica has the batch



Each follower's *next* fetch tells the leader how far it got; once every in-sync replica has the batch, the high watermark moves past it — committed, visible to consumers, and acknowledged.

4. "Can Kafka lose messages?" — every path, and its setting.

Where	How data is lost	The setting that prevents it
Producer	`acks=0` or `acks=1`: "ok" before followers have it; leader dies	`acks=all` (the default since Kafka 3.0)
Producer	Send failed after retries, but the app ignored the error callback	Handle send failures: retry, alert, or stop
Producer	Retries create duplicates or reorder batches	`enable.idempotence=true` (default since 3.0)
Broker	ISR shrinks to 1; that one copy is lost	`min.insync.replicas=2` with replication factor 3 — writes are *refused* instead of accepted with one copy
Broker	An out-of-sync follower becomes leader, discarding committed records	`unclean.leader.election.enable=false` (the default)
Broker	All replicas in one zone; a power loss hits before the page cache is flushed	Spread replicas across zones (rack awareness)
Consumer	Offset committed (automatically or by hand) *before* processing; crash in between	Commit *after* processing; idempotent handlers
Retention	A consumer falls behind by more than the retention time; old segments are deleted unread	Alert on **lag age** well before retention (e.g. at 50%)

The safe default set: **replication factor 3 across 3 zones, `min.insync.replicas=2`, `acks=all`, idempotence on, unclean election off, commit after processing, alert on lag age.** It tolerates losing one broker with no loss *and* no downtime; losing two makes writes fail loudly rather than silently.

5. Consumer groups, inside. Each group has a **group coordinator** — the broker leading the partition of the internal `__consumer_offsets` topic that the group's name hashes to. Members send **heartbeats** (small regular "I'm alive" messages) to it; committed offsets are written to that compacted topic.

- **Two timers decide "dead".** `session.timeout.ms` (45 s by default) — no heartbeat for this long means the process is gone. `max.poll.interval.ms` (5 minutes by default) — no call to `poll()` for this long means the consumer is stuck, so it is removed even though its heartbeat thread is alive. That second timer caused the rebalance storm above: process in smaller batches (`max.poll.records`) or hand slow work to a thread pool.
- **Rebalancing styles.** The original *eager* protocol made every member give up every partition on each change — a stop-the-world pause. **Cooperative (incremental) rebalancing** moves only the partitions that must move. **Static membership** (`group.instance.id`) lets a restarting **pod** (one running container instance in Kubernetes) keep its partitions if it returns within the session timeout, so **rolling deploys** (updating instances a few at a time) don't trigger rebalances. The **new consumer group protocol** (KIP-848; a **KIP** is a Kafka Improvement Proposal, generally available in Kafka 4.0; opt in with `group.protocol=consumer`) moves assignment to the broker and makes every rebalance incremental.
- Everything else — ordering per key, partitions as the parallelism ceiling, hot partitions — is the Partitioned Consumption lesson.

6. Exactly-once, inside. Two mechanisms, layered:

- **Idempotent producer.** Each producer gets a **producer ID**, and numbers its batches per partition (**sequence numbers**). The leader remembers the last sequence numbers per producer and drops a retried batch it has already written, or rejects one that arrives out of order. That's why idempotence allows at most 5 in-flight requests per connection — the broker tracks that many recent batches.
- **Transactions.** A producer with a `transactional.id` (a stable name that survives restarts) can write to several partitions *and* commit a consumer group's offsets as one atomic unit. A **transaction coordinator** records the transaction's progress in the internal `__transaction_state` topic; on commit it writes **commit markers** into every partition touched. Consumers with `isolation.level=read_committed` read only up to the **last stable offset** — they never see records from aborted or still-open transactions. When a restarted instance registers the same `transactional.id`, its **epoch** is bumped and the old, possibly still-running "zombie" instance is **fenced**: its writes are rejected.

Together these give **read-process-write exactly-once within Kafka** — what Kafka Streams enables with `processing.guarantee=exactly_once_v2`. The boundary is the one the Exactly-Once Semantics lesson draws: writing to a database or sending an email is outside the transaction, so those sinks must be idempotent or transactional themselves.

7. How message brokers evolved (why Kafka looks the way it does).

Generation	Examples	Model	Strength	Weak spot
Enterprise queues (1990s)	IBM MQ (MQSeries, 1993), JMS (Java Message Service) brokers	Queue manager; per-message acknowledgment; transactions	Reliable delivery of individual business messages	Vertical scaling; messages gone once consumed
Smart brokers (2007 →)	RabbitMQ (AMQP, an open messaging protocol)	Exchanges route messages to queues; per-message acks; priorities, delays	Flexible routing, task queues	Throughput per queue; no replay (streams added later)
The distributed log (2011 →)	Apache Kafka (from LinkedIn)	Partitioned, replicated append-only log; consumer offsets	Huge throughput, replay, many independent readers	Per-message routing, priorities and delays need extra topics
Separated storage (2016 →)	Apache Pulsar (from Yahoo)	Stateless brokers over a separate storage layer (BookKeeper)	Independent scaling of serving and storage; multi-tenancy; built-in geo-replication	More components to run

Newer designs keep Kafka's protocol but change the engine or storage (Redpanda in C++; systems writing segments straight to object storage for cheap retention at some latency cost). Underneath all of them: **the log is the foundation.**

Walkthrough with numbers: the order topic, hardened

- **Setup.** `orders`: 48 partitions, replication factor 3, one replica per zone, `min.insync.replicas=2`, `acks=all`, idempotence on, 7-day retention. Peak 20,000 orders/s × 1 KB = **20 MB/s** in, **60 MB/s** written cluster-wide with replication.
- **Latency cost of `acks=all`.** Followers are ~1–2 ms away across zones, and producers linger 5 ms to batch. **p99** produce latency (the time that 99% of requests beat) ≈ **10–15 ms** instead of ~5 ms with `acks=1` — the price of not losing the 800 orders.
- **A broker dies.** The controller elects new leaders from the ISR within **seconds**; producers retry (idempotence drops the duplicates); nothing committed is lost. The ISR of affected partitions is now 2 — still at `min.insync.replicas`, so writes continue.
- **A second broker dies.** Partitions with ISR 1 **reject writes** (`NotEnoughReplicas`) — a visible outage instead of silent single-copy data.
- **The rebalance storm, replayed.** `max.poll.records` 500 → 50 makes a poll loop ~40 s instead of 6 minutes; static membership keeps partitions in place during deploys. Rebalances per deploy: **~20 → 0**.
- **The retention trap.** Alert when the oldest unconsumed record is older than **3.5 days** (half of retention), long before unread data is deleted.

Trade-offs

You gain: very high throughput from sequential I/O, batching and zero-copy; durability with no single-disk dependence; replay and many independent consumer groups; and precise, documented guarantees — once you set them.

You pay: guarantees are **opt-in combinations** (`acks`, min ISR, zone placement *and* consumer commits — one wrong setting silently weakens them); **latency for safety** (`acks=all` across zones adds ms, transactions more); **operational weight** (partitions, rebalances, retention, upgrades — or a managed bill); and it is **a log, not a smart queue** (per-message routing, priorities, delays and acks are awkward).

When not to use it

- **A modest task queue** that needs priorities, delays or per-message retries — RabbitMQ or a cloud queue (SQS) is simpler.

- **Request-reply** between services — use a direct call (Request-Response); a log adds latency and bookkeeping for no gain.
- **Tiny volume and no replay need** — a table plus polling, or a managed queue, avoids running a cluster.
- **Common misuse:** treating a topic as a database to query by field ("find the order with ID 123") — logs are read sequentially; build a keyed store from the log instead.

Real-world examples

- **LinkedIn** built Kafka for activity data and metrics; by 2019 it reported handling about **7 trillion messages a day**.
- **Managed services:** Confluent Cloud, Amazon MSK, Azure Event Hubs (Kafka-compatible endpoint), Aiven; **Kafka-compatible engines:** Redpanda.
- **Kafka Streams and Apache Flink** build keyed state and exactly-once pipelines on top of these internals.
- **For AI engineers:** Kafka carries feature-update streams into online feature stores and logs every model request and response for evaluation. Compacted topics hold "latest embedding version per document", so a vector index can be rebuilt by replaying one topic.

In the interview

Prompts: "can Kafka lose messages?", "why is Kafka fast?", "what is the ISR?", "how does Kafka do exactly-once?" Answer with mechanisms and settings, not slogans.

> **30-second answer:** "A partition is an append-only log split into segment files with a sparse offset index; retention deletes whole segments or compacts to the latest value per key. It's fast because of sequential I/O, the page cache, zero-copy sends, and compressed batches end to end. Each partition has a leader and followers that fetch from it; the ISR is the set that's caught up, and the high watermark — what every ISR member has — is what consumers can see. Kafka can lose data if you let it: `acks=1`, `min.insync.replicas=1`, unclear leader election, replicas in one zone, or committing offsets before processing. The safe set is RF 3 across zones, min ISR 2, `acks=all`, idempotence on, commit after processing. Exactly-once is idempotent producers plus transactions with read-committed consumers — inside Kafka only; external side effects still need idempotency."

Likely follow-ups (with answers):

- *"Why doesn't Kafka fsync every message?"* — Durability comes from copies in different zones before the "ok"; a zone power loss loses only unflushed data on that zone's replicas. Per-message fsync would cost throughput for little extra safety (it can be forced per topic).
- *"Consumers keep rebalancing — how do you debug it?"* — Find which timer fires: session timeout (crashes, long GC) or `max.poll.interval.ms` (slow processing per poll). Smaller batches or off-thread work, cooperative rebalancing, static membership.

Mistakes that fail candidates: "RF 3 means no data loss" without `acks` and min ISR; claiming Kafka exactly-once covers database writes; assuming order across partitions; not knowing that consumers only read up to the high watermark.

Self-check:

- **Q1.** Define LEO, ISR and high watermark; why may consumers read only up to the HW? → LEO: a replica's next offset to write. ISR: leader plus followers caught up within the lag limit. HW: the highest offset every ISR member has (committed). Anything above it could be truncated after a leader change, so reading it could show events that later disappear.

- **Q2.** RF 3, `min.insync.replicas=2`, `acks=all`: one broker down, then a second. → One down: ISR 2, writes continue, nothing lost. Two down: ISR 1 < 2, writes are rejected — nothing is ever acknowledged with one copy, and all acknowledged data is on the survivor. Availability traded for durability, on purpose.
- **Q3.** A billing consumer auto-commits every 5 s and writes to a database; after a crash some invoices are missing and some duplicated. → Offsets committed before the DB write → skipped (missing); rows written whose offsets weren't committed → re-read (duplicates). Fix: commit after the write and make it idempotent (upsert by event ID), or store offsets in the same DB transaction.

Related patterns:

- **Partitioned Consumption** — the consumer-side contract that these internals implement.
- **Exactly-Once Semantics** — the end-to-end boundary around Kafka's idempotence and transactions.
- **Write-Ahead Log** — the same append-first idea, turned into a distributed product.
- **Change Data Capture** — compacted topics carry table snapshots; CDC streams are Kafka topics.
- **Case 4 in Module 13** — the same system designed from scratch, with capacity numbers.

TL;DR

- **Problem:** teams run Kafka on slogans — "never loses messages", "RF 3 is safe", "exactly-once" — and each hides a setting that decides the real guarantee.
- **Mechanism:** partitions as segmented append-only logs with sparse indexes; speed from sequential I/O, page cache, zero-copy and compressed batches; leader/follower replication with ISR, high watermark and leader epochs; consumer groups with a coordinator and two liveness timers; idempotent producers and transactions for exactly-once inside Kafka.
- **Costs:** safety is a combination of settings (RF 3 across zones, min ISR 2, `acks=all`, idempotence, commit after processing, lag-age alerts), extra latency for durability, and real operational work.
- **Skip it when:** you need a small task queue with priorities and delays, request-reply, or you have tiny volume and no replay need.

Sources: J. Kreps, N. Narkhede, J. Rao, "Kafka: a Distributed Messaging System for Log Processing", NetDB 2011; J. Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction" (2013); Apache Kafka documentation — Design (persistence, efficiency, replication), Configuration, and KIP-98 (exactly-once delivery and transactions), KIP-101 (leader epochs), KIP-345 (static membership), KIP-405 (tiered storage), KIP-429 (cooperative rebalancing), KIP-500 (KRaft), KIP-848 (new consumer group protocol); Apache Pulsar documentation (architecture overview); RabbitMQ documentation (AMQP 0-9-1 model); LinkedIn Engineering, "How LinkedIn customizes Apache Kafka for 7 trillion messages per day" (2019); Kleppmann, *DDIA* ch. 11 (Partitioned Logs).

Capstone - Clickstream to Dashboard

Original lesson: Capstone - Clickstream to Dashboard on DesignGurus ↗

What this capstone is. A step-by-step build of the store's analytics platform. It starts with taking in clickstream and order events, and ends with a live dashboard, fresh fraud features and audited finance reports. It uses all seven patterns from the module, and covers recovering from failures and behaving well at peak load.

Analogy. Imagine designing a city's water system for three very different customers. A fire hydrant needs water *now*, and a bit of dirt is fine. A hospital needs water certified as pure, and can wait for it to be treated. A garden sprinkler needs water quickly and roughly clean. There is one source but three delivery standards, and you don't pay for hospital-grade treatment for the sprinkler.

The design rule for every stage: match each implementation to the **staleness budget** (the maximum acceptable age of the data) of a named consumer. Match each correctness guarantee to its question. And match each partition key to the ordering scope that is really needed.

The brief

- **In:** the **clickstream** (the running record of what users view and click) — page views, searches and cart events — at about **30,000 events/s at peak** from web and mobile, plus order events (which we've had since Module 2).
- **Out — three customers, three budgets:**
  - **Live ops dashboard** — orders per minute, conversion, top products. Staleness: *seconds*. Accuracy: honest, but forgiving.
  - **Daily business reports** — revenue by category, good enough for finance. Staleness: *daily*. Accuracy: *audited*.
  - **Fresh fraud features** — card velocity (the example from the stream lesson). Staleness: *sub-second*.
- **Incidents it must prevent** (all from earlier lessons): the dashboard disagreeing with finance; double-billing after a rewind; a refund processed before its charge; and one slow warehouse sink stalling the entire pipeline.

Stage 1 — Ingestion: keys and event IDs

Decision: two topics, each keyed by its own ordering scope.

- **Clickstream keyed by session ID** — funnel and journey analysis needs the events of one session in order, so all events from a session stay in one partition.
- **Orders keyed by customer ID** — so a customer's charge is always handled before their refund.
- **Overprovision partitions** (e.g. **96** for clickstream), because resizing later reroutes keys and causes disruption. *Why not fewer?* The partition count is the ceiling on future parallelism (and Stage 6 depends on it).

Decision: event IDs generated by the client, created before the first send and reused on every retry. Mobile **SDKs** (client libraries built into the apps) resend when the network is flaky. If every retry got a new ID, duplicates would look exactly like genuine separate events, and nothing downstream could catch them (Exactly-Once Q3). So preventing duplicates starts in the SDK.

Stage 2 — Enrichment: getting reference data to the stream

Every click carries a `product_id`, but the dashboard wants product names and categories. Those live in the catalog database. So the job has to join the stream with **reference data** (lookup data that changes slowly). *How does the catalog get to the stream, and how does it stay fresh?*

Decision: CDC into local keyed state. **Debezium** reads the catalog tables and publishes changes to a **compacted topic** (a topic that only keeps the latest event for each key). The streaming job loads this into a lookup map keyed by product ID, held locally in each partition. Enrichment becomes a lookup in local state.

- *Why not query the catalog DB per event?* That is the per-event database query the stream lesson already rejected (hundreds of expensive queries/s against production).
- *Why not a nightly dump?* It would be a day stale, and it's another copy that can drift. With CDC, when merchandising renames "Gadgets" to "Electronics", the change reaches the stream **within seconds**. The same feed also serves cache invalidation and the search index — one integration, many readers.

Stage 3 — Processing: time, windows, and three late policies

Mobile events can arrive minutes late (tunnels exist). The dashboard counts per minute, and fraud counts per card per 10 minutes.

Decision: event time with watermarks. Counting by arrival time (processing time) measures the network, not the users. After a carrier outage, the delayed events all land at once and create a fake spike that

sets off the dashboard's rate alerts. So the analytics jobs use event time, with a watermark running about **60 seconds** behind.

Decision: a late-data policy per consumer (one stream, three policies):

- **Dashboard: drop** late events — its per-minute chart doesn't go back and change past minutes, and its budget of seconds can live with small inaccuracies.
- **Reports: accept amendments** — finance's numbers become complete in the end, and readers accept restatements on the same day.
- **Fraud: process late events** — a late alert is still worth having.

Stage 4 — Correctness tiers: the guarantee belongs to the question

Dashboard counts and finance revenue are built from the same events. *Which sink machinery does each one get?*

- **Dashboard → idempotent keyed upserts** `(metric, minute) = value`. A replay (for example, the restore-and-rewind that happens on every deploy — "every deploy rehearses the crash") sends out identical values, which overwrite harmlessly. There are no transactions and no read-committed delay, which suits its forgiving accuracy budget.
- **Finance revenue → full exactly-once:** checkpointed state paired with offsets, a transactional sink, and readers set to read-committed. Audited numbers get the strongest guarantee.
- **Fraud features → idempotent upserts**, and the fraud decision still double-checks against the authoritative source.
- *Why not transactions everywhere?* That would add cost and latency without making the two consumers that don't need it any better.

Stage 5 — The reports: refusing accidental Lambda

The usual move would be a separate nightly batch job over the warehouse, just for finance. *Why not?* Because that means two implementations of the same metrics. It's the accidental Lambda from the Lambda & Kappa lesson, and it is sure to drift.

Decision: one logic source. Metric definitions (how revenue is calculated, how returns are adjusted) live in **one shared library**. The streaming job computes them continuously. The "daily report" is then just a view of those aggregates **finalized at day close**.

Where does the warehouse fit? It is still there, fed by CDC and the same event topics. It serves ad-hoc analysis, **ML training**, and batch-shaped work such as joins over many years and **cohort studies** (comparing groups of users who, for example, signed up in the same month). It stores the stream's outputs and the raw history, but it **never re-implements** the dashboard metrics.

When a metric definition changes: use the Kappa move. Replay from an earlier offset with the new logic, verify, and switch consumers over (the migration playbook). **Scheduled backfills**, borrowed from Lambda, put a limit on how long any streaming error can last. Dashboard and finance agree because each metric has a single implementation.

Stage 6 — Black Friday: backpressure by design

Peak traffic triples the input to **90,000 events/s**. Mid-morning, the warehouse sink (the one from the table-lock incident) slows down again. *Follow the path of the pressure.*

1. **The log absorbs the burst.** Producers keep writing at 90k/s, and consumers process at rates they can sustain. The difference shows up as **lag**, and the age of that lag is monitored.
2. **Autoscaling on backlog age** adds more consumer instances within minutes, up to the partition count picked in Stage 1. This is exactly why those extra partitions were bought.
3. **The warehouse sink reads from its own topic** — a **log seam**. When its table lock strikes at **10:40**, only the warehouse consumer falls behind. The dashboard, fraud and finance pipeline keeps going at full speed, and checkpoints aren't held up.
4. **Who feels what:** one consumer's backlog grows on a dashboard, and nobody else notices anything. No facts are dropped, and bounded buffers stop memory from running out. On-call looks into one lagging sink, not a dead pipeline.

The final architecture

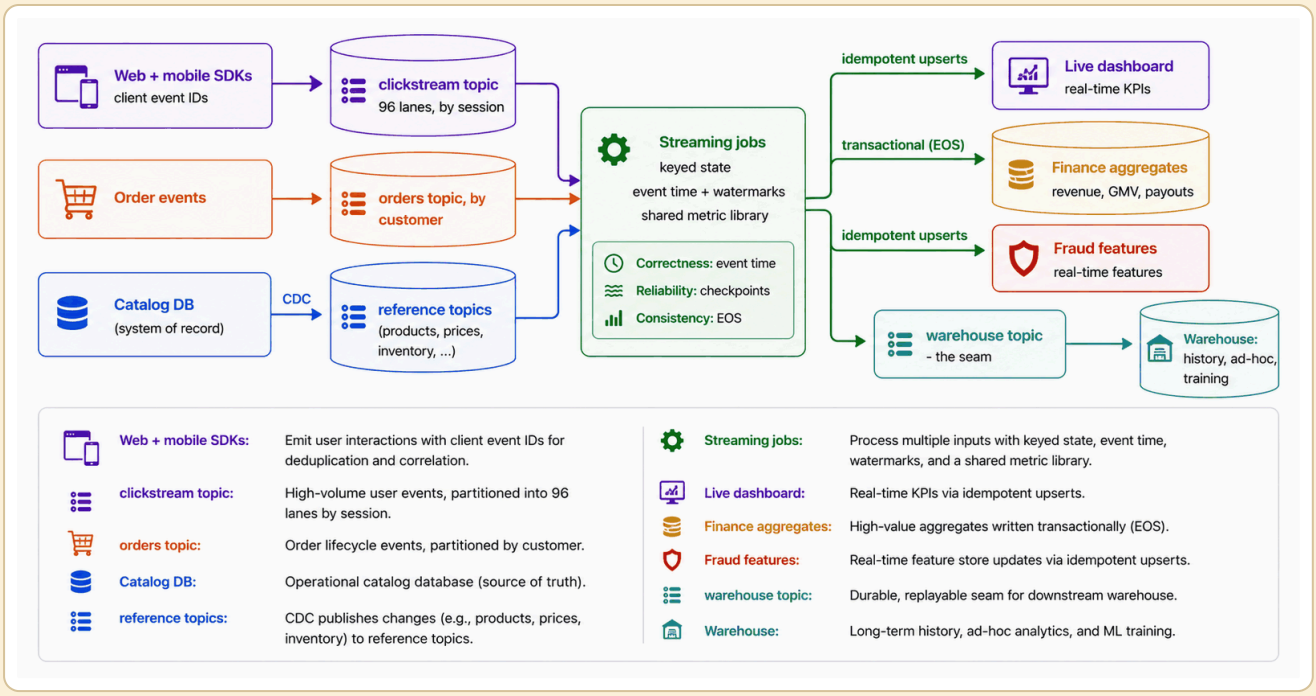


Figure — Clickstream: final architecture (course diagram)

In the figure above, web and mobile SDKs (which stamp client event IDs) write to a clickstream topic that has 96 lanes keyed by session. Order events go to an orders topic keyed by customer. The catalog DB feeds reference topics through CDC. The streaming jobs (keyed state, event time + watermarks, the shared metric library, checkpoints, exactly-once) write idempotent upserts to the live dashboard and the fraud features, and transactional (EOS) writes to the finance aggregates. A separate warehouse topic — the seam — feeds the warehouse, which handles history, ad-hoc queries and training.

Debrief: the module, applied

Decision	Lesson used	Incident it prevents
A key for each topic, and extra partitions	Partitioned Consumption	Refund-before-charge; resize surgery
RF 3 across zones, min ISR 2, `acks=all`, commit after processing, lag-age alerts	Inside Kafka *(added lesson)*	Confirmed events lost on a broker failure; unread data deleted by retention
Client event IDs created before the first send	Exactly-Once (Q3)	Duplicates created upstream of every guarantee
Reference data brought in via CDC into keyed state	CDC	Dumps, drift, a DB query per event
Event time; three late policies	Stream Processing	Counting the network instead of the users
Upserts for forgiving consumers, transactions for audited ones	Exactly-Once	Over-billing and over-engineering
One logic source; the warehouse stores but never recomputes	Lambda & Kappa	The same question answered by two authors
Log seams; lag instead of loss; scaling on backlog age	Backpressure	One slow sink stalling everything

What the design deliberately **did not buy** matters just as much: no transactions on the dashboard, no streaming for cohort studies, and no global ordering anywhere. That is MapReduce's "restraint" rule, applied more broadly. **Every guarantee, freshness tier and ordering promise was purchased for a named customer** — the economics of the whole module in one sentence.

Interview one-liner

Give each topic a key that matches its ordering scope and plenty of partitions, stamp event IDs on the client, and enrich from local state fed by CDC. Count in event time with late policies per consumer, and spend exactly-once only on outputs that get audited. Keep one metric definition that the stream and the warehouse share, and put log seams in front of slow sinks so a burst turns into lag rather than an outage.

Extend it yourself (practice questions)

- **Sessionization:** product wants funnel analysis (browse → cart → checkout). Design the session windows. What defines the inactivity gap? What state grows for each session? And what happens to a session that lasts across a deploy?
- **Multi-region:** clickstream comes from two continents, with regional logs and one global dashboard. Where do you merge the data, and what happens to event-time watermarks when one region's stream falls far behind the other?
- **For AI engineers:** the Stage 4 fraud features now also feed training (the feature store's offline path). Using the skew diagnosis from Lambda & Kappa: what guarantees that "card velocity" at training time matches "card velocity" at serving time? (Answer hint: Stage 5's single shared library of metric definitions already built it. One definition, run in both modes, prevents training/serving skew.)

Module 11 — AI-Era Systems (LLM and ML Infrastructure)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Here we study the plumbing that sits under AI products. That includes feature stores for classic machine-learning models, fleets of **GPUs** (graphics processors, the chips that do the heavy maths of running models) that serve large language models (LLMs), gateways placed in front of model vendors, caches that match on meaning, vector indexes, and retrieval-augmented generation (RAG). The big message is a comforting one: **nothing here is architecturally new**. You have already seen every one of these patterns in earlier modules. What changed is the "physics" they operate under.

- Take Module 8's **API gateway** (the single front door that handles login checks, routing and rate limits for your services) and face it outward, toward model vendors rather than your own services. That is the **LLM gateway**.
- **Semantic caching** is the caching from Module 4. The only twist is that "the keys are equal" turns into "the meanings are similar."
- **Vector database sharding** reuses Module 3's sharding, now applied to a different kind of index.
- **GPU auto-scaling** is Module 6's auto-scaling (automatically adding or removing servers as load changes), just under far tougher limits.
- The **feature store** combines CQRS (one model for writes, a separate one for reads) with a lesson from the data module, "one question, two implementations," and applies both to ML.

According to the course, roughly **60 recurring shapes** sit underneath all of system design. If that is true, even the newest, most advanced stack should break down into those same shapes. This module tries to prove it.

The three constants that changed. Every lesson below is driven by at least one of these:

1. **Work is counted in tokens, and requests differ hugely in size.** A *token* is a piece of text the model reads or writes; it is roughly 3/4 of an English word. On the web, requests cost about the same as each other. With an LLM, one request might hold 10 tokens and the next might hold 2,000. So a single request can cost **100x** as much as the one beside it. If your **load balancer** (the component that spreads requests across servers), **rate limiter** (the component that caps how much each caller may send), or capacity plan counts *requests*, it is silently wrong. (These building blocks are introduced in System Design Fundamentals.) The true unit of work is the token.
2. **Workers are costly and slow to boot.** A GPU machine costs roughly **100x** what a web server costs. Before a new **replica** (one more running copy of the model server) can serve anything, it must load tens of gigabytes of model weights into memory. (Weights are the numbers that hold what the model learned.) That makes a cold start last *minutes*, not seconds. Reactive scaling stops working, and choosing between "leave it idle" and "let it go cold" becomes a genuine money decision. In-progress work is heavy as well. The **KV cache** holds what the model has already worked out about the conversation, and it sits in GPU memory. Because of it, later turns of a conversation prefer to land on the *same* replica. The **stateless** web (where any server can handle any request, because no server keeps per-user memory) spent ten years getting rid of "session affinity" (sticky routing), and now it is back.
3. **Equality turned into similarity.** Classic web caches and indexes only answer when the key matches exactly. AI systems instead ask "what is *near* this?" When lookups become fuzzy, a cache hit means something different, indexes act differently, and even what counts as a "correct" answer shifts. This is the one truly new area in the module. Even so, once you redefine the key, the old rules still apply.

Four terms used throughout (we define them just once, here):

- **Token:** the unit of text a model works in (~3/4 of a word). Cost, compute, and rate limits are all measured in tokens. A request is simply a bag of tokens.
- **KV cache:** the working memory a model builds for each conversation as it reads and writes. It sits in GPU memory and is costly to rebuild. It can use as much memory as the weights do. However, it only survives for the length of the request or conversation.
- **Embedding:** a list of numbers (a vector) that captures the *meaning* of some text as a position in space. Points near each other have similar meanings. Semantic caching, vector search, and RAG all rest on embeddings, since embeddings turn "how similar are these?" into a math problem.
- **TTFT (time to first token):** the wait before the first word of the reply shows up. In a streaming interface, this is the delay users really notice. So it is the main SLO for the **inference** tier (inference means running a trained model to produce an answer or prediction). (An SLO, or service level objective, is the latency target the service commits to.)

The seven patterns

The situation	The pattern
Training and live predictions must get exactly the same inputs	Feature Store
Workers are expensive, but each request is small	Model Serving & Batching
A new worker needs minutes before it can serve	GPU Auto-Scaling
A single entry point for many model vendors	LLM Gateway
The same question asked in many different wordings	Semantic Caching
An index built on meaning grows too big for one server	Vector DB Sharding
Answers should come from documents, not from memory	RAG Pipeline

How to read the module. We begin with Feature Store, which continues from where the data module's capstone stopped (training/serving skew). Next, Model Serving & Batching lays out the money side of the inference tier, which the later lessons take for granted. GPU Auto-Scaling then tackles the difficult GPU case that Module 6 put off. LLM Gateway is the place where calls to providers are routed, metered, and failed over; apart from CDC, readers requested this lesson more than any other. Then come two "fuzzy" lessons: Semantic Caching (the similarity version of Module 4) and Vector DB Sharding (the similarity version of Module 3). RAG Pipeline ties everything together. Finally, the capstone builds a production RAG chatbot that draws on every module in the course.

Short on time? Focus on LLM Gateway and RAG Pipeline first, because AI-infrastructure interviews probe those two hardest. After those, Semantic Caching gives the best return.

Before you start, choose an AI product you have built or used, and work through three questions:

1. What does one request cost? Do your rate limits, capacity plans, and prices allow for the next request being 100x more expensive?
2. When traffic surges, how many minutes go by before newly added capacity is truly serving users, and who pays the price during that window?
3. If two people ask the same question in different words, does your system notice they overlap, or does the second person pay full cost all over again?

Feature Store

Original lesson: Feature Store on DesignGurus ↗

What it is. In a feature store, each input to a model is **written once, as code**. Such an input is called a "feature," for example "the number of orders this card made in the past hour." The platform

then runs that one definition in two places. It runs it over historical data to produce training sets. It also runs it over live data to feed the model in production. Since only one definition exists, the values the model gets in production equal the values it learned from.

> **Analogy.** Picture a restaurant chain whose test kitchen and whose branch cooks each write their own version of a recipe. What gets tasted in testing is never exactly what customers are served. A feature store is one master recipe card, and the test kitchen and every branch all cook from it.

The problem (the incident). A fraud team ships a new model. When tested offline against several months of historical data, it catches **40% more fraud** than the rules it replaces. Three weeks into production, it is performing *worse* than those rules. Yet nobody touched the model or its weights. The cause is a single feature, `card_velocity_1h`, which counts a card's orders over the last hour:

- On the **training** side, it was written in warehouse SQL. It used *event-time* windows, meaning orders were grouped by when they actually happened. It ran over the full history, counted late-arriving events, and converted NULLs (SQL's marker for a missing value) to 0.
- On the **serving** side, it was a hand-coded Java service running on **Redis** (a very fast in-memory key-value store). It used something near *processing-time* windows, meaning events were grouped by when they reached the service. It dropped late events and let NULLs pass through unchanged.

Two languages and two teams produced two slightly different takes on what should have been a single number. The model trained on one take and then received the other in production. We call this gap **training/serving skew**. It is dangerous because it fails *silently*: nothing errors out, the decisions just get worse. It is the "one question, two implementations" problem from the Lambda architecture lesson, and here it hurts model quality. Two more problems surfaced along the way. First, each new feature took **3 weeks** to ship, because a data scientist wrote SQL and engineers then rewrote it for the serving stack. Second, the growth team was coding a *third* copy of the velocity feature, since no one could locate the second one.

Why the obvious fixes fail

- *"Have one team own both implementations and review them closely."* Two codebases in two languages, shipped on two release schedules, will drift apart simply because of how they are set up. Reviews can slow that drift. Over many quarters, though, discipline never beats structure.
- *"Serve features directly from the warehouse, so only one implementation exists."* That gives you one implementation, but at the wrong speed. A warehouse query takes from hundreds of milliseconds up to seconds. The inference path has only about **10 ms** to spend fetching features.
- *"Record what serving computed and train on those records" (log-and-wait).* This truly stops skew for any period you have logs for. The catch: a new feature has no logs yet, so you spend months gathering training data. Also, the logs faithfully capture any bugs in the serving code. Hold on to this idea, but use it to *verify*, not as the core fix.

How it works

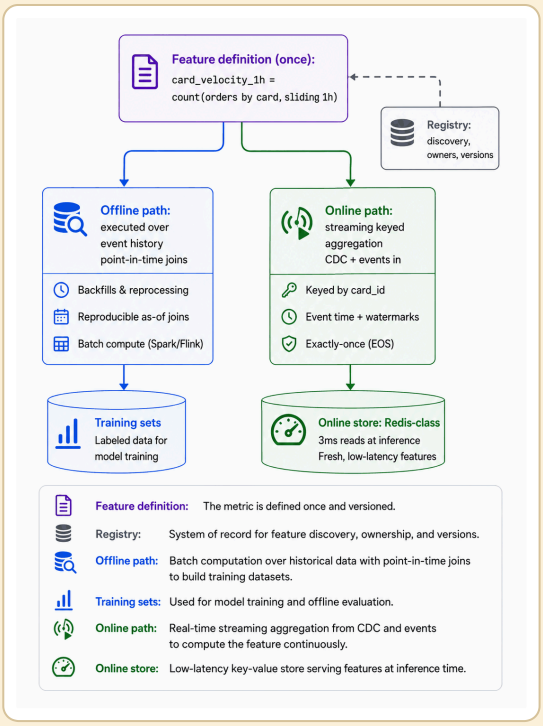


Figure — Feature Store (course diagram)

In the figure above, a single feature definition feeds two execution paths: offline for training and online for serving. A registry sits between them. Here are the steps:

1. **Write every feature once, as a definition.** The data scientist writes `card_velocity_1h` in about a dozen lines of the platform's feature language. (That language is a DSL, a domain-specific language.) The platform turns it into both batch SQL and a streaming job. *Why:* with a single author, the two paths can't disagree on what the feature means. This is the Kappa-style fix: get rid of the duplicate definitions rather than trying to keep them in sync.
2. **Run the online path (for live predictions).** Event topics, together with change data capture (CDC, which streams the database's change log), feed a keyed sliding-window aggregation. That aggregation writes into a Redis-class key-value store using idempotent upserts. (An upsert inserts or updates; idempotent means retrying it is safe.) The stored values trail reality by only about **200 ms**, and a read takes about **3 ms**. *Why:* that sits comfortably within the ~10 ms inference budget. It is precisely the Module 10 streaming machinery, wrapped up as a product.
3. **Run the offline path with point-in-time correctness (for training).** Suppose a training example is **labeled** (tagged with the known correct answer the model should learn) "fraud on Tuesday at 3 PM." It must be paired with the feature values *as they stood* at 3 PM that Tuesday, not with today's values. *Why:* if you attach today's velocity to last month's fraud labels, the model learns from facts that didn't yet exist when the decision was made. That is called **leakage**. It causes the well-known pattern of "amazing offline, useless live." A point-in-time join is simply the "as-of" query from **event sourcing** (storing every change as an event so you can rebuild the state at any past moment), applied to training data.
4. **Maintain a registry.** Each feature can be found, has an owner, and is **versioned**. *Why:* because features are discoverable, the growth team can reuse the existing velocity feature instead of writing another copy (one producer, many consumers). Versions matter because a model was trained on the meaning of v1. A definition is therefore a contract, just like a public API.
5. **Log and verify.** The serving side keeps logging the values it really computed. A background job recomputes the same moments using the offline path and confirms they match. The rule to hold is **zero difference**. *Why:* even with one definition, the two compiled paths might still drift apart (for example, from a compiler bug). This check catches that.

Walkthrough with numbers (the fraud feature, rebuilt)

- *Time to ship:* the definition is a dozen lines, and both paths get built automatically. **3 weeks becomes about 2 days**, and most of those 2 days go to validation.
- *Serving:* CDC events flow through the keyed window. The online store answers in **3 ms** with values roughly **200 ms** old, well within budget.
- *Training:* the same definition runs across **6 months** of history with point-in-time joins. So each training row sees only the values that existed at its own timestamp. After retraining, the model's live results land *within noise* of its offline scores. That agreement between offline and online is the real product the platform delivers.
- *Reuse:* the growth team's ranking model takes the very same feature from the registry. One definition now feeds three consumers, and nobody wrote another implementation.
- *The bill:* **10M cards × 40 features** comes to a few gigabytes of RAM, which is fine. Each new feature needs a **backfill**, meaning history is replayed so the feature exists for training. That is an ordinary Kappa replay.

Trade-offs

- *You gain:* skew is removed by design, so offline evaluation now predicts production (which is the entire reason to evaluate offline). Preventing leakage becomes something the platform guarantees, not something each scientist must watch for. Features are written once and shared by every model, and shipping drops from weeks to days.
- *You pay:*
  - **A platform to run:** streaming jobs, an online store, a registry, and backfill tooling, all operated by a team. Run it like a product, with the ML teams as its customers.
  - **The DSL's limits:** whatever the definition language can't express needs an escape hatch. Each escape hatch is a spot where skew can sneak back in.
  - **Freshness tiers:** some features are streamed (velocity, refreshed in seconds). Others are computed in a nightly batch (90-day aggregates, at roughly **1/10 the cost**). Consumers need to know each feature's tier, because a model is only as fresh as the stalest feature it uses.
  - **Backfill compute** each time you add a feature.

When not to use it

- *One team, one model, and around a dozen features:* a single shared code module that both paths call, plus log-and-verify, gets you the principle without building a platform.
- *Features computed only from the incoming request* (like text length or time of day): these are stateless transforms. Compute them inside the model server with one function; no store is needed.
- *Inference that only runs in batch* (such as nightly scoring): there's no online path, so careful point-in-time joins in the warehouse are enough.
- *Classic misuse:* rolling out the store but allowing teams to hand-code "just this one" serving path. That is the third implementation all over again, now hidden outside the registry.

Comparison: three consistency strategies

	Two implementations + review	Log-and-wait	Feature store (define once)
Skew	Built into the setup; review only slows it down	Removed for the time range you have logs for	Removed by design
Adding a feature	Written twice; takes weeks	Months spent waiting for logs to build up	Write the definition + backfill; a few days
Leakage protection	Depends on each scientist being careful	Good (logs are as-of by nature)	Point-in-time joins that the platform enforces
Cost	Seems cheap; grows more costly each quarter	Cheap	A whole platform
Verdict	Avoid	Keep, but only as a verification tool	Use this, with log-and-verify included

Real-world examples

- **Uber's Michelangelo** is where it started: its machinery for keeping features consistent gave the category its name. **Airbnb's Zipline** appeared around the same time.
- **Feast** (open source), **Tecton**, and the cloud offerings (**SageMaker, Vertex, Databricks**) package the pattern as a product, with definitions, dual materialization (computing and storing each feature's values in both the offline and online stores), and registries.
- Big ranking, fraud, pricing, and recommendation systems follow this design, even when they don't use the name "feature store."
- *For LLM engineers:* the "user context" you inject into **prompts** (the text sent to an LLM), such as subscription plan, recent activity and preferences, is online feature serving by another name. If a prompt template pulls from three ad-hoc services, that's three chances to drift from what you evaluated: the same incident, just with tokens.

In the interview. Questions such as "design a fraud detector" or "design a recommender" eventually boil down to one thing: how do you keep features at serving time identical to features at training time? Answering "same logic, careful testing" states the goal but offers no structural fix.

> **30-second answer:** "Each feature is defined once, as code, in a registry, and the platform executes that one definition along two paths. The offline path runs over event history using point-in-time joins, so every training row only sees what was known at its timestamp; joining today's values onto old labels would leak the future. The live path uses a streaming aggregation to keep the same feature current in a Redis-class store that responds in milliseconds. There's one author, so skew can't happen; log-and-verify catches any compiler drift; and offline evaluation finally predicts production."

Likely follow-ups:

- **"Why are point-in-time joins so important?"** Leakage. When training rows get *current* feature values, they include facts that only appeared after the labeled event. The model learns to rely on those facts, so offline scores jump. Then live performance collapses, since production never gets to see the future. This is the priciest silent bug in ML, and teams frequently mistake it for "drift."
- **"Which features should stream, and which should batch?"** Decide feature by feature, based on how stale each one may be. Things like velocity or session activity need refreshing within seconds, so they stream. 90-day aggregates can handle a nightly batch at roughly a tenth of the cost. In a tiered store, every feature declares its own freshness level, instead of pushing everything through the priciest pipeline.

Mistakes that fail candidates: waving off two implementations as acceptable; building training sets from current-state tables (leakage that goes unnoticed); hitting the warehouse from the inference path.

Self-check:

- **Q1:** Define training/serving skew, the wider engineering problem behind it, and the structural fix. → Skew means the features at serving time differ from those at training time, whether in definition, windowing, null handling, or freshness. So live inputs come from a different distribution, and quality quietly falls. Underneath, it's one question answered by two separate implementations. The fix: a single definition compiled into both paths, plus log-and-verify.
- **Q2:** A teammate joins March fraud labels onto the *current* features table and gets an offline AUC of 0.97 (AUC scores ranking quality; 1.0 is perfect). What will production look like? → Much worse. The 0.97 comes from leakage: a card's velocity today may be high *precisely because* of the March fraud the label records. The fix is to rebuild using point-in-time joins.
- **Q3:** After six months, a model slowly gets worse. There were no deploys, and log-and-verify is clean. An upstream service changed `order_total` so it no longer includes shipping. Why didn't the store notice? → The store promises that the two paths agree. It doesn't promise that the *meaning* stays the same. Both paths computed the new meaning identically (a temporal skew). Add upstream schema/semantic contracts (outbox, schema registry). Add monitoring of feature distributions against the training profile (mean, **percentiles** (the value below which a given share of observations fall, e.g. the 95th percentile), null rate). And use versioned features (`order_value_v2`) that models pin to.

Related patterns: CQRS (the offline path writes, the online path reads; features are the read models); Lambda & Kappa (skew is their duplicate-implementation problem, and define-once is the ML fix); Stream Processing and CDC (the machinery behind the online path); Event Sourcing (point-in-time joins are as-of queries).

TL;DR

- *Problem:* code a feature twice (SQL for training, a service for serving) and the copies drift. The model learns from one version and runs on the other, so it looks great offline and quietly fails live. Each feature takes weeks, and third copies appear.
- *Mechanism:* define it once in a registry. Compile it into an offline path (history plus point-in-time joins) and an online path (streaming into a millisecond store). Use log-and-verify as the drift alarm, and backfill by replay.
- *Costs:* a platform to run, DSL limits, freshness tiers per feature, and replay compute for each new feature.
- *Skip it when:* one team owns only about a dozen features, the transforms can be computed from the request, or inference is batch-only.

Model Serving (and Batching)

Original lesson: Model Serving on DesignGurus ↗

What it is. Model serving is the layer that executes a model to answer live requests. Its most important trick is **batching**. Rather than pushing requests through the GPU one by one, you hold them for a few milliseconds and run a group together. The LLM version of this is **continuous batching**: requests can enter and exit the batch at every token the model generates.

Analogy. A bus that runs its whole route for a single rider costs nearly as much as one carrying 32. The fuel goes into moving the bus, not the people inside. Batching means filling the bus. Continuous batching means riders hop off at their own stop and new riders hop on at every stop, instead of everyone staying aboard until the last stop.

The problem (the incident). The store runs its own 7-billion-parameter ("7B"; parameters are the model's learned weights) support model (so data stays private). Like a normal web service, it processes requests one at a time. Latency looks fine. The capacity math does not:

- An A100-class GPU (a popular NVIDIA data-centre GPU) costs roughly **$35K/year**. It produces about **50 tokens/second** when handling one request.
- A typical reply is **400 tokens**. So each request uses **8 seconds** of GPU time, and a single GPU can handle **0.125 requests/second**.
- At peak the service needs **20 req/s**. That works out to **160 GPUs ≈ $5.6M/year**, while the feature's business plan had set aside only about a tenth of that.

Profiling (measuring where time and resources actually go) reveals each GPU uses just about **8%** of its compute. Here's why. With a batch of one, every token step has to stream tens of gigabytes of weights out of GPU memory and through the chip, while the real math for that single request is tiny. The work is **memory-bandwidth-bound**. In other words, the limit is how quickly data moves out of memory, not how quickly the chip can calculate.

Why the obvious fixes fail

- *"Buy more GPUs."* Now you have 160 GPUs, each still only 8% busy. The bottleneck is how much work happens per pass over the weights, not how many chips you own.
- *"Switch to a smaller model."* Distillation (teaching a small model to copy a big one) and quantization (storing each weight in fewer bits) reduce how much data moves per step. But a small model served one request at a time still sits mostly idle. You change the quality/cost balance without fixing utilization. So think about it *after* batching, and only if quality permits.
- *"Put everything in a queue and run batch jobs every hour."* Great for offline jobs (providers offer batch processing at about **50% off**). But in chat, a customer is sitting there waiting. You need batches assembled in *milliseconds*, not hours.

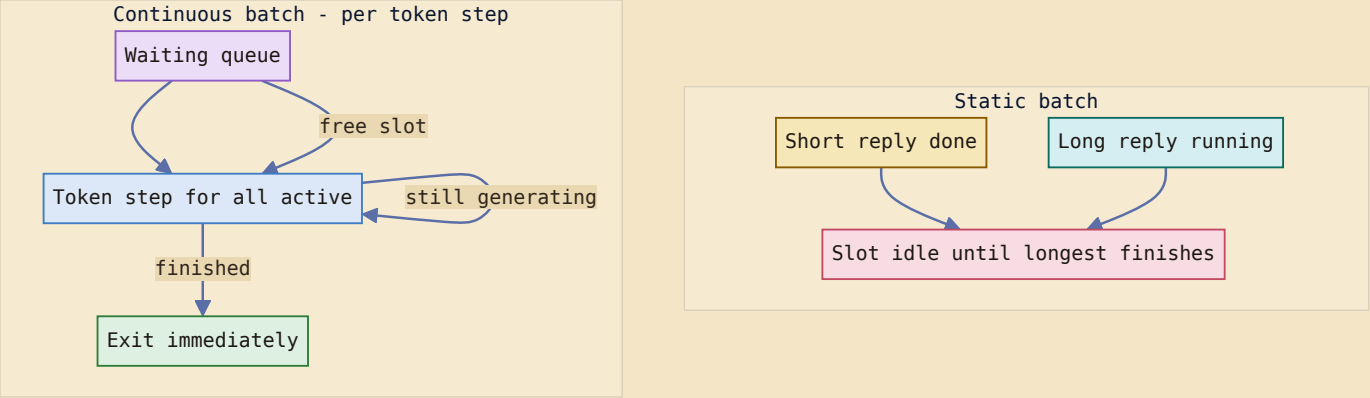
How it works

1. **Dynamic batching (the general version).** The server waits a very brief moment (about **5–10 ms**) or until a maximum batch size is hit, whichever happens first. Then it runs the collected requests together. *Why it works:* each step streams the weights through the chip just once, however many requests share that step. So **32** requests together take barely longer than one alone, yet deliver about **20–30x** the **throughput** (work completed per second). The price is the few-millisecond wait. When every job is the same shape and cost (embeddings, image classification), this alone is the full solution.
2. **The LLM twist: variance (Constant 1).** In a *static* batch, no one leaves until the slowest generation finishes. A 10-token reply stuck alongside a 2,000-token essay has to wait for the essay, and finished requests leave empty slots behind. This is the **convoy effect**: short jobs trapped behind long ones.
3. **Continuous (in-flight) batching.** The scheduler thinks in *token steps*, not whole requests. On each step it produces one token for every active request. Requests that are done leave right away, and waiting ones join in the middle. *Why:* capacity is recovered on every token, so utilization stays high even when output lengths vary by 100x.
4. **Paged KV-cache memory (PagedAttention).** Every active request keeps its KV cache in GPU memory. Simple allocation breaks that memory into fragments, which caps how many requests can fit. Paging is an idea taken from operating-system **virtual memory**, where memory is handed out in small fixed-size pages that need not sit next to each other. It eliminates the **fragmentation** (free memory scattered in pieces too small to use) and lets **2–4x more concurrent requests** share the same GPU.
5. **The stack around it.** Place replicas behind a load balancer that chooses a replica based on **queue depth and tokens still pending**, not connection counts. (Connection counts mislead when work

varies 100x.) Keep each conversation **sticky** to the replica that already holds its KV cache. At big scale, **disaggregate prefill from decode**. *Prefill* (reading the prompt) is **compute-bound** (limited by how fast the chip can calculate), while *decode* (writing the reply) is bandwidth-bound, so separate pools can each be tuned for their own job.

The diagram below compares the two batching styles. In the static batch, a short reply that has finished sits idle until the longest reply is done. In the continuous batch, each token step cycles through the active requests; finished ones leave immediately, and queued ones fill the freed slots.

Static batching vs continuous batching



Walkthrough with numbers (same assistant, vLLM-class engine with continuous batching) — **vLLM** is a popular open-source engine for serving LLMs.

- *Per GPU:* with a batch of about 32, total generation climbs to **2,500–3,000 tokens/s**, compared with 50 for a single stream. Any one 400-token reply still needs a few seconds, but the GPU is now serving **6–7 requests/s** in parallel.
- *Fleet:* 20 req/s ÷ ~6 per GPU ≈ 3.3 → **4 GPUs + 1 for headroom = 5 GPUs instead of 160**. Yearly cost falls from **$5.6M to about $175K**. That's a **30x** cut from scheduling alone, with the same model and the same quality. Batching is what makes the product financially possible.
- *Latency cost:* TTFT rises by about **20–30 ms**, which nobody notices when the reply streams. The pace per token barely changes.
- *Utilization:* **8%** → **about 70%**.
- *Bulk work moved elsewhere:* the nightly job that writes **10M product descriptions** goes to the provider's discounted (half-price) batch API. It is **bulkheaded** (kept separate) from the interactive fleet.
- Batching stays almost free until the batch's combined compute takes longer than streaming the weights, or until KV memory fills up. Past that point, you need another GPU.

Trade-offs

- *You gain:* **10–30x throughput per dollar**, which is what makes hosting your own models affordable at all. Variance is under control. Autoscaling has fewer, fuller GPUs to manage.
- *You pay:*
  - **A small latency floor:** the batch window plus scheduler overhead add a few milliseconds to TTFT. That matters for strict real-time uses.
  - **Shared fate:** requests in one batch fight over the same memory and compute. One enormous prompt can squeeze out the rest, and if the engine crashes, every request in the batch dies.
  - **A truly complicated engine** (token schedulers, paged memory, **preemption**, i.e. pausing or evicting a running request to make room). Don't write it yourself; **buy it** (vLLM, TGI, Triton). These days, building your own scheduler is a mistake, not an accomplishment.
  - **Tuning for each workload:** batch caps, windows, and memory budgets.

- *Giant-prompt spikes:* when someone pastes an entire contract, **p99** TTFT (the value 99% of requests come in under) for everyone can leap from **200 ms to 4+ s**. The culprit is an enormous, compute-heavy **prefill** that grabs the token steps. Fixes: **chunked prefill** (break the reading of the huge prompt into parts spread across many steps); **input tiering** (send huge inputs to a long-context pool or an async flow, which is a bulkhead); **product caps with honest UX** (user experience) ("large document: may take longer"); and **prefill/decode disaggregation** at scale.

When not to use it

- *Inference that has to complete in a few milliseconds* (voice frames or some ranking calls, for example): you may not have room for any batching window, so use small-batch or specialized runtimes.
- *Very little traffic* (dev environments, an internal tool at 0.1 req/s): if only one request ever arrives, a batch of one is correct.
- *Offline bulk jobs:* keep them off the interactive fleet. Batch APIs cost half as much and don't mind waiting.
- *Classic misuse:* interactive and bulk traffic on the same fleet, so the 10M-item nightly job starves customer chats. That's a bulkhead failure, GPU edition.

Comparison: dynamic vs. continuous batching

	Dynamic (request-level)	Continuous (token-level)
Batch membership	Set when the batch starts; all wait for the last one	Changes constantly; leave when done, join midway
100x output variance	Convoy effect	Handled; capacity recovered on every token
Good for	Same-sized jobs like embeddings, vision models, rankers	Generating LLM text
Implementation	A simple window + max size (Triton)	A complete scheduler (buy vLLM/TGI/TensorRT-LLM/SGLang)

Rule of thumb: **when jobs are all the same size, batch by request; when sizes differ, batch by token.** The embedding endpoint (which the semantic cache will soon call) uses dynamic batching, running next to the continuous-batching LLM fleet.

Real-world examples

- **vLLM:** open-source continuous batching plus paged KV memory. It made running your own LLM serving practical for many workloads.
- **NVIDIA Triton:** dynamic batching offered as general infrastructure for any model type.
- **TGI, TensorRT-LLM, SGLang:** other serving engines, each a variant of the ideas above.
- **Provider batch APIs (Anthropic, OpenAI):** about 50% off for work that can wait. The providers slot it into spare fleet capacity, following the same economics.

In the interview. Questions on designing LLM inference serving are becoming more common. Asking a candidate to explain *why batching helps GPUs so much* quickly separates people who have run inference from people who have only called a hosted API.

> **30-second answer:** "Served one request at a time, LLM inference is capped by memory bandwidth. Every token step pushes all the weights through the chip for barely any math, so GPUs idle around 8% and the fleet costs millions. Batching shares that weight stream across ~32 requests, giving 20–30x throughput for a few ms of TTFT, which streaming hides. LLMs need continuous, token-level batching because output lengths vary 100x; request-level batching

would make short answers wait behind long ones. Paging the KV cache fits 2–4x more requests at once. Balance load by queue length and pending tokens, keep conversations sticky, buy a vLLM-class engine, and route bulk work to half-price batch APIs."

Likely follow-ups:

- **"How can 32 requests take barely longer than one?"** Most of each step's cost is streaming the weights out of GPU memory, and that's the same at any batch size. The compute for each request comes along almost for free, until the batch grows big enough to max out the compute units.
- **"Someone pastes a 50-page file and everyone's latency spikes. What's going on, and how do you fix it?"** The huge prefill is a burst of compute that hogs the token steps (shared fate). Use chunked prefill, cap inputs and tier them by size, and at large scale move prefill and decode onto separate pools so the biggest requests get a pool to themselves.

Mistakes that fail candidates: pricing one-request-per-pass serving without spotting the waste; suggesting request-level batching for chat (missing the convoy effect); writing the token scheduler by hand.

Self-check:

- **Q1:** Why does a batch of 32 take roughly as long per token step as one request, and when does that stop being true? → Every step has to stream all the weights through the chip, a fixed cost that dominates; each request's math is small. It stops when total compute outlasts the streaming time, or when KV memory runs out. At that point the GPU is compute-bound and you need another GPU (the signal for autoscaling).
- **Q2:** Match strategies to: (a) embeddings (each job ~50 ms), (b) customer chat (outputs vary up to 100x), (c) nightly summaries of 10M pages. → (a) dynamic batching (Triton-style); (b) continuous batching + paged KV (vLLM-class); (c) neither fleet: a provider batch API or an off-peak offline run, with a bulkhead rule so bulk work never touches the interactive pool.
- **Q3:** p99 TTFT, usually 200 ms, jumps to 4+ s for everyone when users paste contracts. → A giant, compute-bound prefill is competing with every decode (Constant 1 applies to inputs as well). Stack up chunked prefill, input tiering, product caps with honest UX, and prefill/decode disaggregation.

Related patterns: Load Balancing (routing by queue depth / outstanding tokens); GPU Auto-Scaling (adding GPUs once they're full); Message Queue (sending bulk work through batch queues at half price); Semantic Caching (the cheapest request is the one the GPU never sees).

TL;DR

- *Problem:* serving LLM requests one by one leaves each GPU ~8% busy (bandwidth-bound), so 20 req/s costs $5.6M/year.
- *Mechanism:* batch. Use continuous, token-level batching to handle 100x variance, plus paged KV (2–4x). Buy a vLLM-class engine.
- *Costs:* a small TTFT floor, shared fate (giant prompts), a complex engine, and tuning per workload.
- *Skip it when:* you need hard real-time milliseconds, traffic is trivial, or the work is offline bulk (batch APIs, bulkheaded).

GPU Auto-Scaling

Original lesson: GPU Auto-Scaling on DesignGurus ↗

What it is. GPU auto-scaling adds GPU model-serving replicas when demand rises and removes them when it falls. The idea matches web auto-scaling. The difference: an idle GPU costs ~100x more, and a new one takes minutes (not seconds) to start. So you don't rely on a single reactive loop. Instead you

layer defenses: the correct signal, a pool of spares that are already warm, pre-warming on a schedule, quicker boots, a plan for degrading during the gap, and careful scale-down.

> **Analogy.** A restaurant can't hire and train a chef in the ten minutes after a tour bus pulls up. So it keeps a couple of chefs on call (warm pool). It reads the bookings calendar and adds staff before known rushes (scheduled scaling). It trains new hires faster (boot engineering). It switches to a simpler menu when overwhelmed (smaller model). And it never sends a chef home in the middle of cooking a dish (drain before scale-down).

The problem (the incident). The five-GPU support fleet is running at 70% utilization. A mention on a morning show then quadruples traffic within **15 minutes**. The web-style autoscaler fails in three ways, one after another:

1. **The signal is wrong, in two ways.** The scaler watches CPU (the server's general-purpose processor), but the real work runs on the GPU. So host CPU shows only **12%** while requests pile up. GPU utilization misleads too: under continuous batching the chip always looks busy, even while the waiting line keeps growing. Only **queue depth** (the number of requests waiting for a GPU) showed reality, and nothing was monitoring it.
2. **Booting takes minutes.** Getting a GPU node takes **3–8 minutes**, and that's if the **region** (a cloud provider's group of data centres in one geographic area) even has one free. Then the node pulls a **15 GB container** (a packaged app plus everything it needs to run), copies **14 GB** of weights onto the GPU, and warms up the runtime. All together, **8–12 minutes** pass between the decision and the first token. The spike ends before the new capacity shows up.
3. **The maximum isn't under your control.** At minute 20, asking for four more A100s returns "insufficient capacity in this region." The maximum you configured was really the cloud provider's **quota** (the most resources the provider will let your account use).

At night the reverse happens. The five GPUs are only **10%** busy but still cost around **$20/hour** while serving almost nobody. Yet scaling to zero means the next customer waits ~10 minutes. Next to web servers, idle capacity costs **100x** more, and a cold start lasts **60x** longer.

Why the obvious fixes fail

- *"Tune the reactive loop more aggressively."* Roughly 1 minute to notice plus ~10 minutes to boot, against a 15-minute spike, is math no setting can beat. Reactive-only GPU scaling is a carefully tuned way to show up late.
- *"Just buy enough for the peak."* A GPU costs roughly **$35K a year**, so holding 4x extra capacity wastes the 30x savings that batching just earned.
- *"Use serverless GPUs (Modal, Baseten, RunPod)."* (*Serverless* means you pay per use and the platform manages the machines for you.) This is a genuine option and is often the right one. These services hide cold starts behind warm pools shared by many customers. Essentially, you pay another company to solve this lesson on your behalf. Once you get big or face strict privacy rules, the problem lands back with you, so it's worth knowing how their approach works.

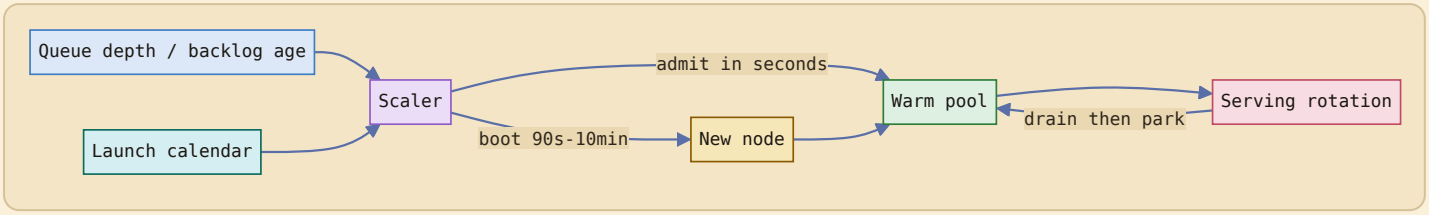
How it works: layered defenses

1. **Track the right metrics: queue depth and backlog age for each replica.** *Backlog age* is how long the oldest queued request has been waiting, and it's what should trigger the alarm. Replicas needed ≈ token demand ÷ token throughput per GPU. *Why:* CPU usage and connection counts don't reflect GPU work, which is measured in tokens.
2. **Hold a warm pool.** Keep *k* instances fully started, with weights loaded and runtime ready, but *outside* the serving rotation. Scaling up moves one of them into rotation within **seconds**. Meanwhile, the slow boot happens in the background to top the pool back up. Size it with math: **pool ≥ spike arrival rate × boot time**. *Why:* this removes the 10-minute boot from the critical path. The idle pool is the stated cost of instant capacity (Module 6's headroom, at GPU prices).

3. **Scale on a schedule and ahead of time.** Pre-warm before the predictable daily curve (for example, before the 9 AM ramp). Require marketing to submit pre-warm requests for launches. *Why:* the predictable share of demand then sees zero delay, and reactive scaling only has to handle the surprises that remain.
4. **Speed up the boot.** Stream the container image so it can start while it's still downloading. Keep weights on fast local NVMe disks (very fast solid-state drives attached directly to the machine), or on snapshot volumes attached ahead of time, instead of pulling them from remote storage. Restore from memory snapshots where that's supported. Co-locate models on nodes that keep their weights loaded. *Why:* boot time drops from **8–12 minutes to 60–90 seconds**. A shorter boot makes every other layer cheaper and simpler to size.
5. **Degrade gracefully during the gap.** While extra capacity is on the way: queue requests and give users an honest wait time → reject free-tier traffic before paying users (rate-limit tiers) → **switch to a smaller model, and say so** (for instance, a quantized 7B when the 70B pool is full). *Why:* users would rather get a somewhat weaker answer, clearly labeled, than stare at a spinner. Answer quality becomes one more thing you can shed.
6. **Scale down without cutting anyone off.** Stop sending new requests to a replica, let generations already underway finish, put the replica back in the warm pool, and only then release it. Keep a minimum floor overnight. Allow scale-to-zero only if the breakeven math works: idle hours × **~$4/GPU-hour** against a multi-minute wait for the next user. (Dev environments usually pass this test; production rarely does.) *Why:* killing a replica immediately cuts off a paying user's answer mid-sentence.
7. **Treat quota as capacity you procure.** Secure capacity through reservations and committed-use contracts (discounts in exchange for promising to pay for capacity over a year or more). Also set up a place for extra traffic to overflow to (another region, another cloud, or a hosted model API), with the LLM gateway doing the routing (next lesson).

The diagram below shows the warm-pool cycle. Queue depth drives the scaler. The scaler moves pooled instances into service in seconds, while new nodes boot in the background to refill the pool. The launch calendar also feeds the scaler, and scale-down drains replicas back into the pool.

Warm pool: fast admission, slow refill off the critical path



Walkthrough with numbers (the morning show, replayed with layers)

- *T−1 day:* the launch calendar kicks off pre-warming, and the pool grows from **2 to 5**. The cheapest fix in this whole lesson is a meeting.
- *T+0 to 60 s:* traffic goes 4x. Queue-depth alarms go off right away. The pool's 5 instances join the rotation within seconds, about doubling capacity in the first minute.
- *T+90 s onward:* the faster boots bring new capacity every **90 s** and refill the pool.
- *Worst minute:* p95 TTFT (the value 95% of requests come in under) stretches to **3.8 s**. Free-tier requests wait briefly behind paid ones. **12%** of traffic is served by the smaller model, with a label. No error page appears.
- *T+25 min:* demand peaks and starts dropping. Scale-down stops new admissions, lets generations finish, parks replicas in the pool, and holds the new baseline.
- *3 AM:* the fleet drops to its floor of **2**. **Preemptible** nightly summarization jobs, which get kicked off instantly if real traffic shows up, fill the quiet hours. Result: **70% utilization by day, 60% at night**. Since GPUs cost about 100x more than web servers, making use of that idle time saves real money.

Trade-offs

- *You gain:* you survive spikes despite 10-minute boot physics; the bill tracks demand; conversations complete even during scale-down.
- *You pay:* the ongoing cost of the warm pool (insurance that looks like waste to anyone who wasn't around for a spike, so it has to be priced and defended every quarter); layered operational complexity (calendars, pools, boots, ladders, preemption); preemption machinery (priorities, instant eviction, batch jobs that checkpoint so they can resume); and continuing quota work (booking reservations, planning overflow to other regions, managing provider relationships).

When not to use it

- *Steady traffic:* if demand hardly moves, run a fixed set of GPUs and fill spare time with preemptible bulk jobs. You don't need a scaler.
- *Tiny scale:* with just one or two GPUs, plain queueing (with honest wait times) beats building warm-pool machinery.
- *Pure batch workloads:* there's no interactive deadline, so schedule the fleet the way you schedule batch work.
- *When you shouldn't own GPUs in the first place:* begin with a hosted model API. Let the provider own the GPUs, and their cold-start and quota problems too, until your traffic, latency needs, or privacy rules push you to host your own. Plenty of companies that are proud of their GPU autoscaler would be better off owning no GPUs.
- *Classic misuse:* scaling GPUs the web way: CPU as the signal, reactive only, scale-to-zero in production, and termination on scale-down. Each choice is right for the web and wrong for GPUs.

Comparison: own the fleet vs. rent the problem

	Provider APIs	Serverless GPU	Own fleet + this lesson
Cold starts	The provider's problem; you never notice them	Eased by the platform's warm pools	Yours to solve: everything in this lesson
Cost at high volume	Highest per token	Middle	Lowest, provided the GPUs stay busy
Privacy / control / custom models	Least	Middle	Full
Ops burden	Almost none	Low	A whole team
Fits	The starting point for most companies	Bursty, specialized workloads and fast experiments	Big steady volume, strict privacy rules, or custom models

Shift one column to the right only when your traffic or your constraints force it. The cost of each shift is everything this lesson covers.

Real-world examples

- **KEDA** (Kubernetes event-driven autoscaling; **Kubernetes** is the standard system for running containers across a cluster of machines) is an open-source tool that can scale based on queue depth, which handles the signal layer. **Ray Serve** and vLLM's built-in autoscaling handle the parts that have to understand model serving.
- **Cloud providers' warm-pool features** (like ASG (AWS Auto Scaling Group) warm pools and GPU reservations) and **capacity reservations**: the readiness layer and quota negotiation, sold as products.
- **Modal, Baseten, RunPod:** serverless GPU, which is to say warm pools you rent.

- **SkyPilot-class multi-cloud spillover:** hunts for capacity across regions and providers once quota runs out.

In the interview. A common prompt: "Traffic to your LLM service goes up 5x. How do you handle capacity?" Every follow-up question lines up with one of the layers (signal, cold start, scale-to-zero, quota). Each one has an appealing one-line wrong answer that the interviewer is hoping to hear.

> **30-second answer:** "GPU scaling is web scaling where idle cost and cold starts are each ~100x worse, so purely reactive scaling is late by design. I scale on queue length plus the age of the oldest waiting request, never on CPU. A warm pool sized at spike rate × boot time lets admission happen in seconds while new nodes refill it; a calendar pre-warms for known events; local weights and streamed images bring boots down to ~90 s. During the gap, a degradation ladder queues, drops lower tiers, and falls back to a labeled smaller model. Scale-down drains generations in flight and keeps an overnight floor. Quota gets negotiated, with spillover routed through the gateway. And I'd begin API-first until volume or privacy makes owning GPUs worth it."

Likely follow-ups:

- **"Why is CPU a poor scaling signal?"** The model runs on the GPU, so the host CPU can look idle while users sit waiting. GPU utilization is misleading too under continuous batching: the chip looks busy while the queue gets longer. Queue length and backlog age tell you directly whether requests are coming in faster than they're being completed.
- **"Should production ever scale to zero?"** Do the breakeven math: overnight idle at ~$4/GPU-hour versus the first morning user (and everyone after them) sitting through a multi-minute cold start. For dev and internal tools: zero. For production: a small floor nearly always wins, since paying for a couple of idle GPU-hours costs less than losing even one customer. Show the math, not a rule of faith.

Mistakes that fail candidates: triggering GPU scaling off CPU; relying on reaction alone when boots take minutes; scale-to-zero in prod without doing the breakeven math; scale-down that kills generations mid-stream.

Self-check:

- **Q1:** Which three delays make purely reactive GPU scaling show up too late, and which layer handles each? → Detection lag (~1 min while metrics aggregate) → queue-depth/backlog-age signals. Boot lag (minutes) → warm pools + boot engineering. Spike duration (often 10–20 min, about the same as the first two added together) → scheduled/predictive pre-warming. The degradation ladder covers any gap left over.
- **Q2:** Spikes bring up to 2 GPUs' worth of demand per minute, and boots take 90 s. How big should the pool be? → ≥ 2 × 1.5 = **3 GPUs** (+1 if spikes overlap). That's about 3 × $35K ≈ **$105K/year** of intentional idleness, which turns a 10-minute outage into 10-second admission. Redo the math whenever boot time or spike shape changes.
- **Q3:** Around 2 AM, answers get cut off mid-sentence, and the first users every morning wait "forever." The config is: min=0, scale down after 15 min of low utilization, immediate termination. → The cutoffs happen because termination kills live generations. Fix it by draining: stop admission, finish up to the max output length, park in the pool, then release. The morning waits happen because min=0 means full cold starts. Keep a floor of 1–2 based on breakeven, and pre-warm ahead of the morning ramp. The principle: **with GPUs, scaling down hits users directly**, so treat it as carefully as anything user-facing.

Related patterns: Auto-Scaling (the parent pattern, with idle cost and boot time both ~100x worse); Model Serving (what's being scaled; fuller batches mean fewer GPUs); Graceful Degradation (the gap

ladder, including the smaller-model step); LLM Gateway (where spillover and capacity federation get routed); Message Queue (preemptible bulk work during quiet hours).

TL;DR

- *Problem:* cold starts lasting minutes make reactive scaling late; $4/h of idleness makes overprovisioning expensive; and the real ceiling is the cloud provider's quota.
- *Mechanism:* queue-depth signals; a warm pool = spike rate × boot time; calendar pre-warming; 90-second boots; a ladder ending in a labeled smaller model; drain-then-park scale-down; negotiated quota with spillover through the gateway.
- *Costs:* paid-for idleness, a layered ops stack, preemption machinery, and capacity procurement.
- *Skip it when:* traffic is flat, scale is tiny, work is pure batch, or you shouldn't own GPUs at all (go API-first).

LLM Gateway

Original lesson: LLM Gateway on DesignGurus ↗

What it is. An LLM gateway is a single internal service that *every* call to a model provider passes through. Services reach the gateway using their internal identity. The gateway then does several jobs. It keeps the provider **API keys** (secret strings that prove you are allowed to call a paid API). It counts every token. It translates "logical" model names into real providers. It applies timeouts, retries, and **circuit breakers** (switches that stop calling a dependency that keeps failing) for each provider. And when a provider fails, it follows a fallback sequence planned in advance. Think of it as Module 8's API gateway, applied to **outbound** (egress) traffic.

Analogy. Imagine a company where, instead of each department carrying its own corporate credit card and haggling with suppliers, there is a single purchasing office. That office holds the cards, logs who bought what, enforces budgets, and knows which backup supplier to phone when the regular one has run out.

The problem (the incident). After 18 months of excitement about AI, **14 services** call model providers directly. They include the assistant, search, summarization, three agent experiments, a fraud explainer, and eight that nobody has catalogued. Three bills show up at once:

- **A key leaks.** One team put a provider API key inside a client bundle (the code shipped to users' browsers or apps), and it shows up on a public paste site. To **rotate** it (issue a new key and revoke the old one), you must track down every copy across 14 services, 3 languages, and unknown config files, then redeploy every one.
- **Finance can't tell who spent what.** "Which team spent the **$80K** last month?" The invoices group spending by keys that teams pass around informally. Meanwhile, an experimental agent kept retrying in a loop for two days and burned **$40K** before anybody spotted it. There was no one place where a budget cap could be enforced.
- **A provider brownout** hits at peak. (A brownout is a partial outage: replies are slow or failing.) The fourteen services react in fourteen different ways: **6 fail with no fallback, 5 hang with no timeout, 3 run untested custom degradation logic.**

The underlying cause is a familiar one from Module 8: when every service implements a **cross-cutting** policy (a rule every service must apply, like auth, logging or budgets) on its own, the copies drift. What's different here is that each request carries a direct dollar cost, so that drift can rack up large unplanned charges within two days.

Why the obvious fixes fail

- *"Build a shared SDK"* (software development kit: a client library teams import). Rollout depends on 14 separate deploy schedules, 3 languages mean 3 SDKs, and the keys still sit in 14 configs.

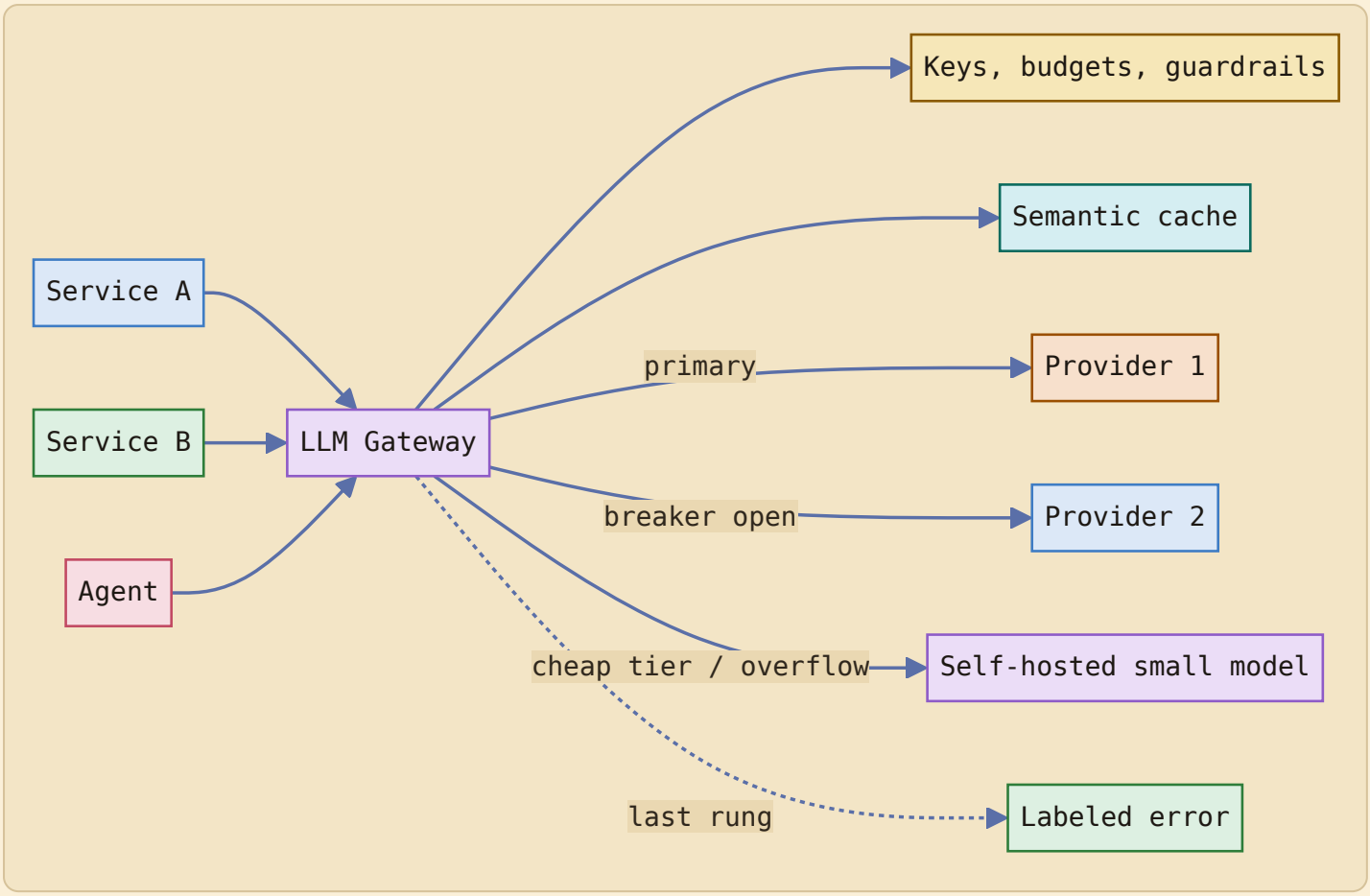
A library centralizes *code*. But key custody, accounting, and live policy need one *runtime* enforcement point that every call goes through.

- *"Keep a key registry and write a spending policy document."* Structure beats discipline. The runaway agent never read the document, and a document can't cap spending at 2 AM on a Saturday.
- *"Let each team harden its own integration."* That gives you 14 half-finished gateways that drift apart, which is just the incident made official.

How it works: each job is a Module 8 job, pointed outward

The diagram below shows services and agents all calling a single gateway. The gateway applies keys, budgets, and guardrails, checks the semantic cache, and sends traffic to the main provider. If that provider's breaker is open, traffic goes to a second provider. Cheap tiers and overflow go to a self-hosted small model. The final rung is a labeled error.

One egress gateway in front of all model providers



1. **Key custody (formerly: authentication).** Services prove who they are to the gateway with the **service mesh**'s workload identity. (A service mesh is infrastructure that carries traffic between services and gives each one a verifiable identity.) Only the gateway keeps provider keys. *Why:* a leak means rotating **one** key with **zero** redeploys, and each call is automatically tied to whoever made it.
2. **Token accounting and budgets (formerly: rate limiting).** Count tokens per team, per feature, and per user. Apply soft alerts and **hard caps**. On the inside, this is rate limiting by tokens per minute. On the outside, the gateway handles the provider's `429 Too Many Requests` replies, retrying in a way that respects the `Retry-After` header. The metering also feeds billing, which needs exactly-once processing when invoices rely on it. *Why:* a single component manages both your own budgets and the provider's limits, and the runaway **agent** (an LLM program that takes actions in a loop on its own) runs into a cap rather than a **postmortem** (a written review after an incident).
3. **Logical model routing (formerly: path routing).** Services ask for a capability, such as `summarize-fast` or `chat-premium`. The gateway maps it to a provider, a model, and a version. *Why:* upgrading a model, reacting to a price change, or switching providers becomes one routing change, not 14 deploys. Roll it out as a **canary behind a feature flag** (say, 5% of `chat-premium` traffic to the new

model), and judge it by evaluation scores and negative-feedback rates. (A **canary** sends a small share of traffic to the new version first; a **feature flag** is a switch that changes behavior without a deploy.) Picking a model becomes an operational setting.

4. **Reliability kit for each provider.** Timeouts are tuned to each provider's p99 (the latency that 99% of requests finish under). Retries respect `Retry-After` and have a retry budget per provider. There's a **circuit breaker for each provider-model pair**. (A circuit breaker stops sending calls to a dependency that keeps failing.) In a brownout, the breaker opens within seconds and traffic steps down the **fallback ladder**: primary provider → secondary provider → smaller self-hosted model → semantic cache → honest error. *Why:* this is Module 5's graceful degradation, set up and tested a single time.
5. **Streaming passthrough.** LLM replies stream over SSE (server-sent events). The gateway has to pass them along *without buffering*. *Why:* a gateway that waits for the whole reply before forwarding it wrecks TTFT. This is an engineering obligation, so test it in **CI** (continuous integration: automated tests run on every code change) as you would a feature.
6. **Safety point.** PII (personally identifiable information) scrubbing (stripping out personal information), **prompt-injection** screening (catching hidden instructions in input that try to hijack the model), and audit logging with **trace IDs** (one ID carried by a request across every service it touches) each happen once, right here.
7. **Capacity federation.** When the self-hosted GPU pool is full, the overflow goes to provider APIs via the same chains. (This is where the GPU lesson's spillover ends up.)
8. **Keep it thin.** The gateway routes, meters, and protects. It does **not** own prompts, templates, or business logic. Building prompts is product code and belongs in the services. *Why:* if it doesn't, every prompt tweak requires a ticket to the platform team (the "monolith in disguise"). Where it sits: the API gateway handles inbound **north-south** traffic (traffic between outside clients and your system), while the LLM gateway handles outbound calls. It usually lives behind the API gateway, alongside the services.

Walkthrough with numbers (the incident's three bills, replayed)

- *The leak:* rotate one key in one **vault** (a secure store for secrets). It takes minutes and zero deploys, and the dashboards already list every caller.
- *The brownout:* the primary's error rate passes the breaker threshold in about **20 s**. **80%** of traffic shifts to the secondary provider (**+300 ms**, quality within tolerance). The `classify-fast` tier shifts to the cheaper self-hosted **7B** model. **15%** of chat gets answered from the semantic cache. The UI shows a labeled "responses may be slower" banner. **Zero services down**, compared with six before, because the response was configuration prepared ahead of time.
- *The weekend agent:* stopped at **$500** rather than $40K.
- *What the gateway itself costs:* **+3–8 ms** of proxy overhead per call, which is noise beside generations that take several seconds. It is one more component that has to be highly available, so run it redundant and thin.

Trade-offs

- *You gain:* policy (custody, budgets, safety) runs once, at the only spot that sees every call. Failover is built and tested once, and model swaps, canaries, and choosing providers by price all become routing changes. You can see costs down to each feature, and runaway spending is capped at machine speed.
- *You pay:* **a dependency on the critical path** of every AI feature (run it redundant; the component that changes least is the most stable); **streaming passthrough as an ongoing obligation** (a single buffering regression and TTFT silently triples); **adapter churn** as provider APIs evolve (one reason to buy the core); and **a constant pull toward scope creep** as prompts and "just one transformation" try to sneak in.

When not to use it

- *One service, one provider:* a well-set-up client library with timeout, retry, and breaker is plenty. There's nothing to coordinate.
- *Fully self-hosted with a single model:* the serving load balancer already does the routing. An extra hop adds latency without adding any new policy.
- *Tiny local models where latency is critical* (in-process rankers): no door needed at all.
- *Build vs. buy:* start with an existing product: **LiteLLM** (open source), **OpenRouter** (gateway as a service), **Portkey**, or cloud AI gateways. Build your own only where key custody, safety, or routing policy are truly specific to you, and keep an open-source core even then. Writing every provider adapter yourself repeats the "hand-built scheduler" mistake.
- *Classic misuse:* using the gateway as a **prompt CMS** (content management system). Templates, **few-shot examples** (sample question-and-answer pairs placed in the prompt), and product logic accumulate in the gateway's config until every feature change requires a platform ticket (the ESB, or enterprise service bus, centralization problem).

Comparison: API gateway vs. LLM gateway, job by job

Job	API Gateway (inbound)	LLM Gateway (egress)
Identity	AuthN (authentication): who sent this request?	Key custody: our identity toward providers; each caller attributed
Fairness	Rate limits for each client	Token budgets for each team/feature, with hard caps
Routing	Paths → services	Logical models → providers/versions
Change safety	API versioning	Model flags + canaries judged by evals
Reliability	Timeout/retry for each upstream	The full kit for each provider + a fallback ladder
Observability	Request IDs, trace root	Token/cost accounting, trace propagation (passing the trace ID along)
Discipline	Thin: no domain logic	Thin: no prompts

It's the same pattern facing the opposite way. Mature stacks run both, one behind the other.

Real-world examples

- **LiteLLM:** the open-source standard (a single interface to every provider, with budgets and fallbacks). It's the default core to build on.
- **OpenRouter:** the pattern turned into a business: a public gateway over dozens of providers, charged per token.
- **Portkey, Kong AI Gateway, Cloudflare AI Gateway:** enterprise and edge variants. Guardrails and caching plug into the same chokepoint.
- Every serious AI platform team ends up building this within a year of the "14 services" incident, usually right after it happens.

In the interview. "A dozen teams call providers directly. What breaks, and what do you build?" The follow-ups dig into the outage chain, safe model swaps, and streaming.

> **30-second answer:** "A single internal egress gateway for all model traffic, which is Module 8's gateway turned outward. Services authenticate with their internal identity, and the gateway holds the provider keys, so a leak means one rotation instead of 14 deploys. It meters tokens per team and per feature, with hard caps. Logical model names map to providers, so upgrades become canaried routing changes judged by evals. Every provider gets tuned timeouts, retries that respect Retry-After, and a breaker per provider-model, all feeding a fallback ladder:

> secondary provider, smaller self-hosted model, semantic cache, labeled error. Streams pass through unbuffered to protect TTFT, and the gateway stays thin, holding no prompts. Start from a LiteLLM-class core."

Likely follow-ups:

- **"Walk through the provider outage."** The breaker for that provider-model trips on its error rate within seconds. Traffic falls down the configured chain: the secondary provider for tiers where quality matters, the self-hosted small model for cheap tiers, the semantic cache where hits exist, and an honest error at the end. Budgets stop failover from doubling spend. Key line: "the response was configuration, not a war room."
- **"How do you move chat to a new model safely?"** Logical names separate callers from models. So send 5% through a flag and compare it with the current model's concurrent baseline on live evals, negative-feedback rate, cost, and per-token latency. Ramp up or roll back through config, using the same release controls you use for code.

Mistakes that fail candidates: keeping prompts in the gateway; having no hard budget caps (so the $40K weekend can still happen); buffering streams; hand-writing provider adapters.

Self-check:

- **Q1:** Match five API-gateway jobs to their LLM-gateway counterparts, and name the two disciplines. → AuthN → key custody + caller attribution; rate limits → token budgets with hard caps; path routing → logical-model routing; API versioning → model flags + canaries judged by evals; request-ID observability → token/cost accounting + trace propagation. The disciplines: **unbuffered streaming passthrough** and **thinness** (no prompts or product logic).
- **Q2:** Design fallback chains for (a) customer chat and (b) internal ticket classification. → (a) premium model → a comparable model from the secondary provider → semantic cache for high-similarity hits → **an honest, labeled error with retry guidance**. It stops short of a much smaller model, because badly shaped answers in open conversation are worse than waiting (**fail closed** on quality: when in doubt, refuse rather than risk a bad result). (b) cheap model → self-hosted 7B → keyword rules → an error almost never, because the quality bar is low and availability matters most. How long a chain should be is a product decision for each tier, priced by what a wrong answer costs.
- **Q3:** The gateway team has become a bottleneck. The config holds 400 prompt templates, and every prompt change needs a ticket. → It's turned into a monolith in disguise (a prompt CMS, the ESB mistake). Move prompts, templates, and output formats into the services that own them, or into a product-owned prompt registry (similar to the BFF split; a **BFF**, backend-for-frontend, is an API layer built for one client app). The gateway keeps custody, metering/caps, routing, the reliability kit, the fallback ladder, safety, and passthrough. Rule: if something uses product vocabulary, it doesn't belong in the gateway.

Related patterns: API Gateway (the parent, inbound; mature stacks run both); Circuit Breaker and Retry (the per-provider kit); Graceful Degradation (the fallback ladder, with quality as one step); Semantic Caching (a shared response cache at the gateway); Canary Deployment + Feature Flags (how model swaps get shipped).

TL;DR

- *Problem:* 14 services call providers directly: keys scattered everywhere (one leak = 14 deploys), spending nobody can attribute ($40K runaway), and 14 homemade responses to an outage.
- *Mechanism:* one thin egress door: key custody, token metering with hard caps, logical-model routing (canaried config), a per-provider reliability kit feeding a fallback ladder, guardrails applied once, unbuffered streaming, and no prompts.
- *Costs:* a highly available component on the critical path, adapter churn (buy the core), passthrough testing, and a permanent fight against prompt creep.

- *Skip it when:* there's one service + one provider, single-model self-hosting, or in-process models; and never write adapters from scratch.

Semantic Caching

Original lesson: Semantic Caching on DesignGurus ↗

What it is. A semantic cache saves LLM answers under the *meaning* of the question instead of its exact wording. Every incoming question gets converted into an embedding. The cache then searches for a stored question whose embedding is close enough, meaning above a similarity **threshold**. If it finds one, it returns the saved answer in milliseconds. If not, it generates a fresh answer and saves it for the next person who asks the same thing in other words.

> **Analogy.** A good librarian doesn't make you repeat your question word for word. "Books on WWII tanks" and "armored vehicles in the Second World War" lead to the same shelf. A careful librarian, though, also double-checks before handing you something that merely *sounds* similar.

The problem (the incident). The support assistant's inference bill runs about **$2,000/day**. Looking at a week of queries reveals that **40% of all questions are rephrasings of about 200 questions** ("order status pls," "when will my package arrive," "has my stuff shipped"). Every one pays full price, roughly **$0.002** and **2.5 seconds**. The team uses Module 4's cache-aside pattern (look in the cache first; on a miss, compute and save), keyed on the query string. The **hit ratio** (the share of lookups the cache can answer) comes out at **4%**. Nobody words things exactly the same way, so exact keys only catch copy-pastes. A 4% hit ratio on a workload that is 40% repetitive captures just a tenth of the opportunity, and about **$800/day** is spent re-answering questions already answered. The answers could be cached; the *keys* just can't recognize the questions. Module 4's laws still apply. What broke is the equality operator (Constant 3).

Why the obvious fixes fail

- *"Normalize the text"* (lowercase it, strip punctuation, **stem** the words, i.e. cut them down to their root, such as "shipped" → "ship"). The hit ratio only rises to **7%**. Rephrasings aren't typos: "where is my package" and "order status?" have no stems in common.
- *"Match keywords against a canonical FAQ."* That recreates the fragile chatbot from before LLMs, an endless maintenance chore that misses any paraphrase nobody predicted.
- *"Cache each user's session."* Useful for users who repeat themselves. But the 40% is repetition *across* different users, and per-user caches never see that.

The cache needs a key that two different sentences share whenever they mean the same thing. That's a job for geometry, not string manipulation.

How it works

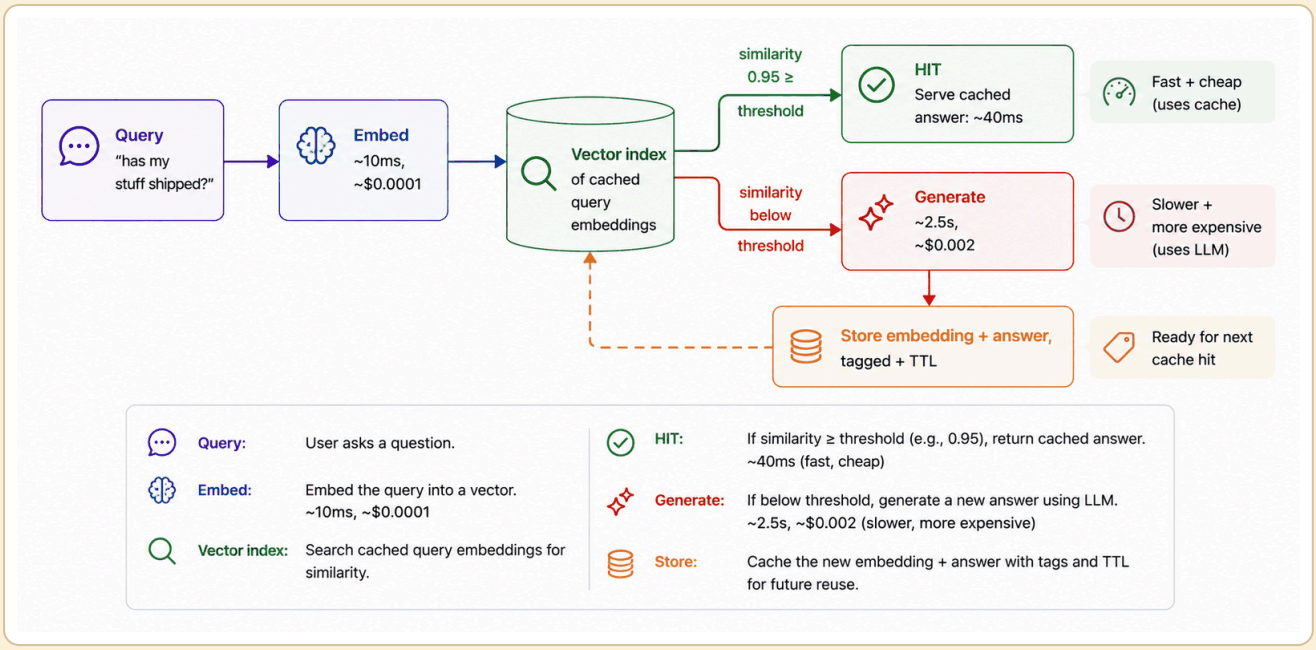


Figure — Semantic Caching (course diagram)

The figure above shows the read path. The query gets embedded and compared with cached entries. Then it's either answered from the cache (if it's above the threshold and passes the guards) or sent on to generation, with the new answer written back. Step by step:

1. **Embed the query.** A small embedding model turns the question into a vector in roughly **10 ms**, for a tiny fraction of what a generation costs. *Why:* the embedding is the new key, and similar meanings end up close to each other.
2. **Nearest-neighbor search** (finding the stored vectors closest to the query's vector) across the embeddings of the cached questions.
3. **Check against the threshold.** If similarity beats the threshold and the guards pass, return the cached answer. If not, generate and save `(embedding, query, answer, tags)`.
4. **Set the threshold on the cautious side.** Exact caches have two possible results: hit or miss. Semantic caches add a third, the **false hit**. That's when a cached answer gets served for a question that *seems* similar but actually means something else ("cancel my order" vs. "cancel my subscription"). *Why cautious:* the costs are lopsided. A false miss costs $0.002. A false hit hands the user wrong instructions and hurts trust.
5. **Guard every hit.** (a) **Entity check:** when the query names order #8412, never return an answer that names #7311. (b) **Never put personalized content** in a shared cache (the CDN (content delivery network) lesson's `private, no-store` rule, now in embedding space). (c) **TTLs** (time-to-live, how long an entry remains valid): for example, a return-policy answer expires because policy can change. (d) **Sampled grading** of real hits, since you have to *measure* the false-hit rate rather than assume it from the config.
6. **Invalidate by tag.** You can't delete fuzzy keys by name ("which entries are about shipping?"). So when writing each entry, tag it with topic metadata, and invalidate with "purge tag: shipping-policy." TTL is the safety net.
7. **Put it at the LLM gateway,** so all teams share one cache. The read path goes **exact cache → semantic cache → generation**.
8. **Decide what can be cached.** Answers that are informational, impersonal, and stable: yes. Account state: no (it depends on who's asking). Creative requests: no (getting the same poem twice is a bug). Time-sensitive answers: only with short TTLs.

Walkthrough with numbers

- *Tuning the threshold:* an offline evaluation set marks pairs of queries as same-meaning or different-meaning, and several thresholds get tested. At **0.92** similarity, the hit ratio is **31%** (versus 4% for exact) with a **0.3%** false-hit rate in audits. The team bumps it to **0.94** and adds

the entity guard, getting **27% hits, 0.05% false hits**. They trade 4 points of hit ratio for **6x fewer wrong answers**, and they document the threshold as a product decision.

- *Money:* 27% of $2,000/day ≈ **$540/day saved, about $200K/year**, all for an embedding model, an index, and a threshold.
- *Latency:* hits come back in about **40 ms** versus 2.5 s for generation, so cached answers feel instant.
- *Viral questions:* during an incident, many versions of "is the site down?" can be served from a single generation **10,000 times an hour**. That's **stampede** protection (**single-flight**: when many identical requests arrive together, only one does the work and the rest share its result), matched by meaning.

Trade-offs

- *You gain:* **25–40%** lower inference cost on workloads full of repetition (often the most valuable single change in an LLM stack); hits that land in a whole faster latency class; and stampede protection for viral questions at no extra cost.
- *You pay:*
  - **A new kind of failure, "confidently wrong":** exact caches serve stale data, but semantic caches can serve *incorrect* data. Cautious thresholds and entity guards limit it, and sampling monitors it, but it's never zero, and someone has to own it.
  - **Infrastructure on the hot path** (the code that every request runs): an embedding model and a vector index. If they break, **fail open**: bypass the cache and generate, because the miss path is always correct.
  - **Fuzzy invalidation:** tags and TTLs stand in for precise deletes, so correctness depends more heavily on expiry.
  - **Geometry upkeep:** the keys live in one embedding model's coordinate space. Upgrade the embedder and every saved key is suddenly in a foreign coordinate system, so migrating needs the dual-run playbook (see Q3).

When not to use it

- *Personal or stateful queries* ("where's my order?", balances): these never go in the shared cache. Entity checks are a fallback, not the main way of keeping them out.
- *When variety is the product* (creative writing, brainstorming).
- *Long-tail workloads:* if analysis shows only ~**5%** repetition, the infrastructure costs more than it saves. **Measure the repetition rate first**; that one number is the pattern's whole business case.
- *Domains that allow zero false hits* (medical, legal, financial advice): the threshold creeps toward exact match, so just use exact matching.
- *Classic misuse:* announcing cost savings without sampled correctness audits, so false hits stay hidden until a customer complains.

Comparison: the caching stack

	Exact cache	Semantic cache	Provider prompt caching
Key	A byte-for-byte identical string	Meaning (how close the embeddings are)	An identical prefix (system prompt, i.e. fixed instructions; docs)
What is reused	The entire answer	The entire answer, across different wordings	The model's internal KV state for the shared prefix
Layer	In front of the gateway	At the gateway	Inside the provider/model
False hits	Impossible	The risk you manage	Impossible (the prefix is exact)
Wins	Free and safe; catches copy-pastes	The 40% repetition	50–90% off input tokens when long prompts are shared

Two takeaways. First, use a three-tier read path: exact first, guarded semantic second, generation last. Second, **provider prompt caching is a separate mechanism**: it reuses model state for identical prompt prefixes and carries no false-hit risk. It runs next to response caching, not in place of it.

Real-world examples

- **GPTCache** (open source): the reference design (embed, index, threshold, serve).
- **Portkey, Cloudflare AI Gateway:** semantic caching built in as a tier of the gateway.
- **Anthropic / OpenAI prompt caching:** the *other* kind of caching, reusing prefixes/KV state for 50–90% input discounts.
- **Support-deflection platforms:** recognizing FAQs was semantic caching before anyone had the words for it.

In the interview. "Your LLM bill is huge; bring it down" leads straight here. The key follow-ups are about the threshold, false hits, and invalidation.

> **30-second answer:** "40% of queries repeat a meaning, but only 4% repeat the exact string, so I key the cache by meaning: embed the query, run a nearest-neighbor search over cached entries, and serve anything above a similarity threshold. The costs are lopsided (a false miss costs a fifth of a cent; a false hit costs trust), so I tune conservatively on labeled pairs, check entities on hits, never cache personal answers, and audit a sample of hits. I invalidate with topic tags plus TTL. It sits at the gateway behind an exact-match tier. And it's different from provider prompt caching, which reuses KV state for identical prefixes; I'd use both."

Likely follow-ups:

- **"How do you pick the threshold?"** Offline, test a range of thresholds on a labeled set of same/different pairs and plot hit ratio against false-hit rate. Online, keep grading a sample of real hits. Pick a tight value, since wrong answers cost trust while misses cost fractions of a cent, and write the decision down.
- **"What about 'where's my order?', your most common question?"** It's the most common *and* can't be cached: the answer is per-user state, and the entity guard blocks it. The right design is a tool/API call that looks up the real status, with the LLM formatting what comes back. Cache the answer's template, never its content.

Mistakes that fail candidates: caching personalized answers ("a privacy incident with a savings dashboard"); not monitoring false hits; mixing up semantic caching with prompt/prefix caching; skipping the measurement of the repetition rate.

Self-check:

- **Q1:** Why does exact-match caching fall apart on natural language, what does the key turn into, and what new outcome joins hit and miss? → People word things differently, so byte keys only match copy-pastes. The key turns into the query's embedding, matched by nearest-neighbor similarity. The new outcome is the **false hit** (confidently wrong), and that's why the threshold is cautious and every hit has guards.
- **Q2:** Classify these: (a) "what's your return policy?" (b) "where's my order?" (c) "write a haiku about autumn" (d) "how do I reset my password?" → (a) cache it, with a `returns-policy` tag and a TTL; (b) never: it's personal state, so use a tool call; (c) no: variety is the product; (d) cache the shared instructions, with the entity guard making sure nobody else's reset link or account data leaks. Instructions can be cached; artifacts can't.
- **Q3:** After upgrading the embedding model, the hit ratio falls from 27% to 2%, and a few strange false hits show up (a shipping question gets a password-reset answer). → Different models create **different coordinate systems**, so comparing them with cosine similarity is meaningless. (Cosine similarity scores how closely two vectors point in the same direction.) New queries land far from old keys, which causes misses. Now and then a nonsense score clears the threshold, which causes false hits. Fix: tag every embedding with its version and **never compare across versions**. Re-embed stored queries in a batch job, or run two indexes (the old one serves, the new one fills). Then switch over and retire the old one: expand → dual-run → verify → contract. **An embedding model is a schema**, so upgrading it is a migration.

Related patterns: Cache-Aside (every law carries over; only equality is swapped out); Cache Stampede Prevention (single-flight by meaning); LLM Gateway (where it plugs in); Vector Database Sharding (the index once it outgrows one node); CDN (the private/no-store rule in embedding space).

TL;DR

- *Problem:* 40% of questions are rewordings of known ones, but exact keys catch only 4%, so money is wasted generating answers you already have.
- *Mechanism:* embed, search nearest neighbors, and serve above a cautious threshold. Add entity guards, topic tags, TTLs, and sampled audits. Place it at the gateway, behind an exact-match tier.
- *Costs:* false hits (measured, not set by config), embedding + index on the hot path (fail open), fuzzy invalidation, and keys tied to one embedder.
- *Skip it when:* answers are personal, variety is the product, there's little repetition, or no false hits can be tolerated.

Vector Database Sharding

Original lesson: Vector Database Sharding on DesignGurus ↗

What it is. A vector database holds embeddings and answers one kind of question: "which vectors are closest to this one?" Once the index grows past the RAM of a single machine, it has to be split (sharded) across several machines. Here's how it differs from ordinary sharding: a nearest-neighbor query comes with **no shard key**. So by default it has to ask *every* shard and combine what they return (scatter-gather). The exception is when the domain provides a filter, such as tenant ID (the customer account the data belongs to), that can act as a key.

Analogy. Suppose a library spreads its books across 12 buildings, sorted by author name. A request for "books by Tolkien" goes to one building. But "books *similar to* The Hobbit" might be in any building. So you have to ask all 12 librarians for their best picks and merge the lists. The exception: if you only search *your own company's* private library, it all sits in one building.

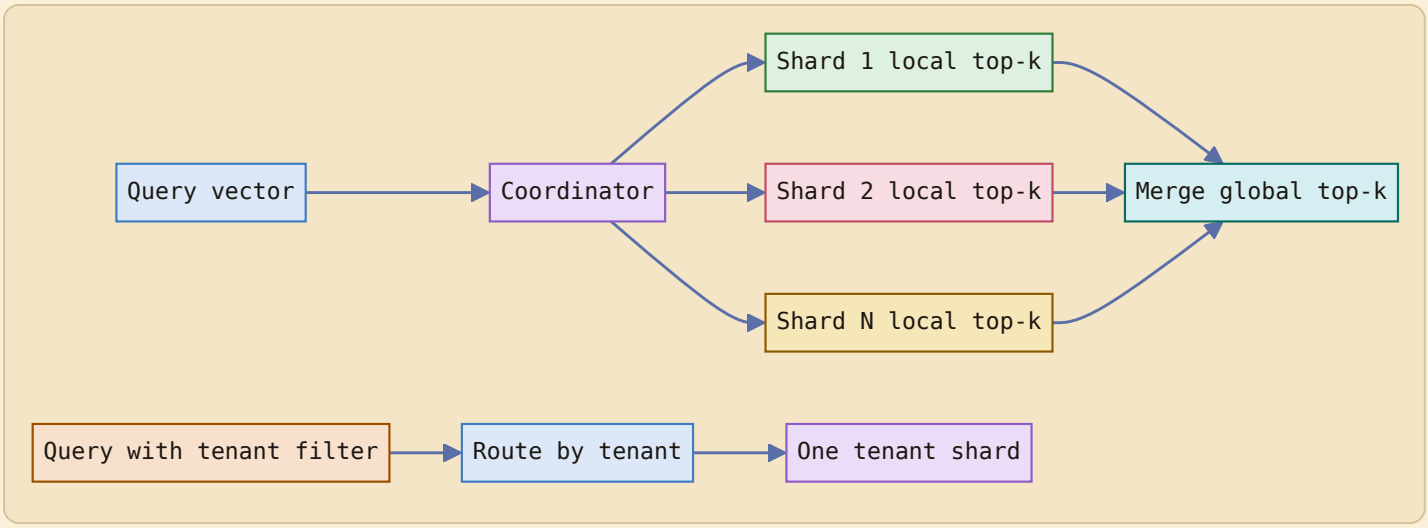
The problem. An embedding is a list of numbers, and each number is one *dimension*. **500M embeddings × 1,536 dimensions × 4 bytes ≈ 3 TB** of raw vectors. On top of that, ANN indexes need RAM and bring extra

overhead. (*ANN* stands for approximate nearest neighbor. It's an index, like **HNSW**, that finds nearby vectors fast without checking each one.) No one machine can hold all of that. In Module 3, every query came with a key (such as `customer_id`) that sent it to one shard. "Find the 10 nearest vectors" doesn't name any key. **Proximity is a result, not an address**, so the neighbors might be on any shard.

How it works: three ways to split

The diagram below shows the default and the main way out. On top, a coordinator sends the query vector to all N shards. Each shard returns its own local top-k, and a merge step chooses the global top-k. On the bottom, a query that carries a tenant filter goes straight to that tenant's one shard.

Default scatter-gather vs tenant-keyed routing



1. **Even shards + full scatter-gather (the default).** Distribute vectors randomly and evenly over N shards. Each query goes to all N. Every shard returns its local *top-k*, meaning the k nearest vectors it holds. A coordinator merges those N×k candidates into the global top-k. *Why:* it's correct and predictable. *Cost:* each query costs N shard queries, and the slowest shard sets the latency. So use **hedged requests**: send the same query to a slow shard's replica too, and keep whichever reply comes back first.
2. **Partition by metadata / tenant.** In a multi-tenant product, one system serves many customers, and each customer is a *tenant*. There, "search your documents" is always limited to one tenant. *Why:* **a filter that appears in every query can serve as the shard key**. So each query hits a single shard, and all of Module 3 applies again, including **dedicated shards for giant tenants**. This is the most common design in production.
3. **Partition by vector geometry (centroid routing, IVF-style).** Group the embedding space into clusters around center points (*centroids*). Send each query only to the clusters closest to it, probing **2–3 shards instead of 12**. *Costs:* **recall** (the fraction of the true nearest neighbors you actually get back) becomes a matter of probability, because a true neighbor might sit in a cluster you didn't probe. Also, popular topics form **hot clusters** (hot shards).

Three more Module 3 ideas carry straight over:

- **Replication:** every shard runs as **primary-replica** (one main copy takes writes; read-only copies serve reads), because RAG retrieval is overwhelmingly reads. Replicas give each shard more read capacity.
- **The index is a read model:** a copy of the data shaped for reading, which you can rebuild from its source. The source of truth is the documents plus the embedding-model version. So upgrading the embedder means a **dual-index rebuild and cutover**, never an edit in place (an embedding model is a schema).
- **Filtered search** ("similar to this, AND in stock, AND under $50") is clumsy for ANN structures. *Pre-filtering* narrows the candidate set before the geometric search. *Post-filtering* removes results afterward and may give back fewer than k. Engines mix the two.

Trade-offs

- *You gain:* indexes bigger than one machine's RAM (the only route to 500M+ vectors); tenant isolation with single-shard routing wherever the domain permits; and the freedom of a read model (rebuilds, re-embeddings, and experiments all run on a copy you can throw away).
- *You pay:* scatter-gather as the default cost model (query cost × number of shards; p99, the latency 99% of requests come in under, is set by the slowest shard; hedging and replicas as permanent mitigations); a recall trade-off that has to become a *measured* SLO; merge machinery (scores from different shards need normalizing so they can be compared fairly) and rebalancing as shards fill up (**consistent hashing**: a way of placing keys on servers so that adding a server moves only a small share of keys); and skew in the form of hot clusters or giant tenants.

When not to use it

- *Under one machine's RAM:* tens of millions of vectors fit on one big node with replicas. Scale up (a bigger machine) before you scale out (more machines).
- *When quantization fixes it:* storing vectors in fewer bytes (int8, meaning 1-byte integers; binary; or product quantization/PQ, which stores compact codes for groups of dimensions) reduces memory **4–32x** for a small recall loss, and can delay sharding for years. **Compress before you add machines.**
- *When the real issue is a single giant tenant:* give that tenant its own shard instead of redesigning everything.
- *Classic misuse:* hash-sharding by document ID the way you would an orders table. Every similarity query still touches all 12 shards.

Comparison: the three splits

	Even + scatter-gather	Tenant/metadata-keyed	Centroid-routed (IVF)
Query fan-out	All N shards	A single shard	The 2–3 closest clusters
Recall	Exact (setting aside per-shard ANN)	Exact within the tenant	Probabilistic: an adjustable dial
Skew risk	None (random spread)	Giant tenants	Hot clusters
Fits	Global search with no natural filter	Multi-tenant products (the usual case)	One enormous corpus where latency is critical

Routing rule: if every query includes a filter, shard on it. If not, scatter-gather with hedging. Route by geometry only when you have one enormous corpus, and make the recall trade-off explicit.

In the interview. "Your RAG index has outgrown one machine" comes up after every RAG design. What they're checking: do you realize similarity queries have no shard key?

> **30-second answer:** "Half a billion embeddings means terabytes of RAM, so the index has to be split. But a nearest-neighbor query has no key, so the default is scatter-gather: even shards each return a local top-k, a coordinator merges them, hedged requests handle slow shards, and replicas carry the read-heavy load. First, though, I check whether every query is scoped to a tenant. If it is, I shard by tenant, which brings back single-shard routing, and give the biggest tenants dedicated shards. For one enormous corpus, centroid routing probes only the nearest clusters, giving up exact recall to cut fan-out. The index is a read model, so re-embedding means building a second index and switching over."

Likely follow-ups:

- **"Why can't you route it like the orders table?"** An orders query says `customer_id = 8412`, which is a key. A vector query says "nearest to this point." That's a property you can only work out by comparing candidates, and the candidates are spread everywhere. With no key in the query, there's no routing, unless the domain supplies a filter.
- **"How do you upgrade the embedding model on a 12-shard index?"** Build the v2 index next to the old one by re-embedding from the source documents. Run both, and compare retrieval quality on live queries. Switch over tenant by tenant or all at once, then retire v1. Version tags make sure no query ever compares across coordinate systems.

Mistakes that fail candidates: hash-sharding while assuming keyed routing; leaving out the merge step; ignoring the RAM math; re-embedding in place.

Self-check:

- **Q1:** Why does similarity search break keyed routing, and what are the three strategies and their fan-outs? → A nearest-neighbor query doesn't name a key. Proximity is a result, so neighbors could be on any shard. Even + scatter-gather: fan-out N, exact, the default. Tenant/metadata-keyed: fan-out 1 when every query includes the filter. Centroid-routed: fan-out 2–3, with probabilistic recall.
- **Q2:** A B2B SaaS (software sold as an online service to other businesses) runs RAG over each customer's private documents: 4,000 tenants, 3 of them enormous. → Shard by tenant (every query is scoped to one tenant). Pack small tenants many to a shard, sized by volume, and give the 3 giants **dedicated shards**, heading off the giant-tenant hot shard. Add replicas per shard. Indexes per tenant also turn deletion/**compliance** (meeting legal and contractual data rules) into one clean purge.
- **Q3:** A centroid-routed index (probing 2 of 16 clusters) has poor recall on niche technical topics, and two of its shards carry 6x the load. → Both problems come from one property: clusters follow *semantic density*, not load. Niche topics make small, sparse clusters whose true neighbors lie just outside the probed set. Mainstream topics make huge, hot clusters. The dials: raise the probe count (per query, for low-confidence matches); split hot centroids and merge sparse ones (re-clustering as routine maintenance); and give hot shards more replicas than cold ones.

Related patterns: Sharding (the parent; keyless queries turn it upside down, tenant filters set it right again); Primary-Replica (read scaling per shard); CQRS (the index as a disposable read model); Consistent Hashing (placement and rebalancing as shards fill).

TL;DR

- *Problem:* 500M embeddings take terabytes of RAM, and nearest-neighbor queries have no shard key.
- *Mechanism:* even shards with scatter-gather, a global top-k merge, hedging, and replicas per shard. The way out is tenant/metadata keys (the common case) or centroid routing (a recall dial). The index is a read model you can rebuild.
- *Costs:* query cost × shards, a slowest-shard p99, probabilistic recall, merge machinery, and hot clusters and giant tenants.
- *Skip it when:* it fits on one machine (quantize first), or the real issue is one giant tenant.

RAG Pipeline

Original lesson: RAG Pipeline on DesignGurus ↗

What it is. RAG (retrieval-augmented generation) answers questions using *your current documents*, rather than whatever the model memorized when it was trained. When a question comes in, the system fetches the relevant pieces of documents that the user is allowed to see. It hands those pieces to the model and asks for an answer based on them, with citations. Under the hood it's **two pipelines**. One is

ingestion, which runs all the time and keeps an index of document chunks current. The other is the *query* pipeline, which runs once per request.

> **Analogy.** Think of an open-book exam. Rather than trusting what a student memorized a year ago (the model's weights), you give them the current textbook pages that relate to the question. You ask them to answer from those pages and cite them. And you only give them pages they're cleared to read.

The problem (the incident). The assistant tells a customer they have **60 days** to return an item. But last quarter the policy changed to **30 days**. The customer returns the item on day **45**, the dispute escalates, and the company ends up paying. The model did its job correctly; its knowledge was just out of date. The 60-day fact is stored in the model's **weights**. Knowledge stored in weights has three built-in limits that no prompt can fix:

- **No way to update:** the policy changed, but the weights stayed the same.
- **No citations:** the model can make a claim, but it can't point to where it came from.
- **No access control:** anything it knows, it knows for everyone who asks.

The fix: **look up current documents when the question is asked, rather than depending on the weights.**

Why the obvious fixes fail

- *"Fine-tune the model on our documents."* Fine-tuning means training the model further on your own data, and it happens on a training schedule. So the new policy would wait weeks. Fine-tuning also gives no citations, weights can't check each user's permissions, and training doesn't store facts reliably. Use **fine-tuning for behavior** (tone, format, domain reasoning style), and use retrieval for knowledge.
- *"Paste every document into the context."* The context is everything you send the model in one request. A **500K-token** knowledge base costs dollars per question, even with discounts. It also adds latency, and models overlook facts buried in the middle of very long contexts. That said, a small, stable corpus (like a 50-page handbook) *can* use long context together with prompt caching.
- *"Let the model search the wiki or the web on its own"* (agentic retrieval, where the model chooses what to search and when). It's more flexible but harder to control. Each extra search adds latency, quality hinges on the search queries the model writes, and governance has to decide what it may read, cite, or reveal. Agents should call the controlled pipeline described below.

How it works: two pipelines

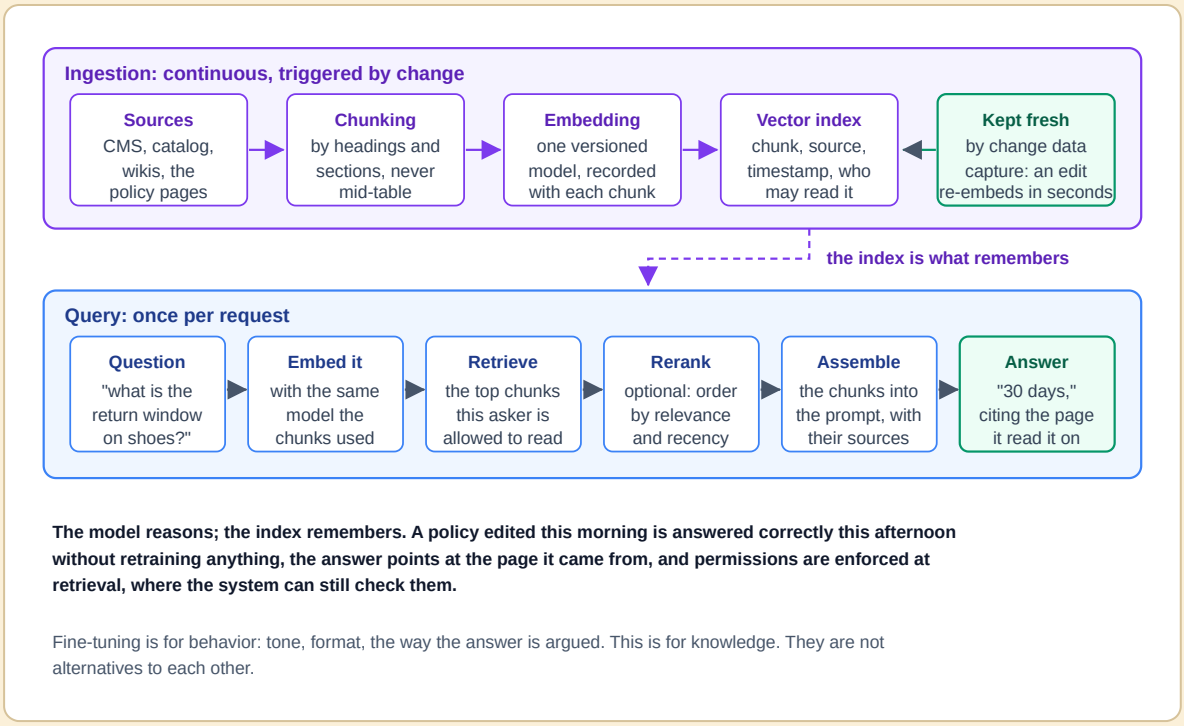


Figure — RAG: two pipelines (course diagram)

The figure above shows both halves. On the ingestion side, sources pass through chunking and embedding into the ACL-tagged vector index, which CDC keeps up to date. On the query side, a question goes through the cache, gets embedded, pulls back the chunks it's permitted to see, is reranked, and reaches generation.

Ingestion (always running):

1. **Chunk the documents.** Break them into pieces that can be retrieved (*chunks*). If chunks are too big, retrieval drags unrelated text into the prompt. If they're too small, answers lose their context. If you split in the middle of a table, no chunk makes sense. Use **structure-aware splitting**: split along headers, sections, and semantic boundaries, with some overlap. *Why it matters:* chunking **sets the quality ceiling** for everything that follows. No reranker can turn bad chunks into good answers.
2. **Embed every chunk** with an embedding model.
3. **Index** the vectors in the sharded vector database, together with metadata and **ACL tags**. (An ACL, or access control list, records who may read each document.)
4. **Stay fresh with CDC.** A change-data-capture connector watches the CMS and the catalog tables. When something is edited, *only the pieces that changed* get re-chunked and re-embedded, within seconds. *Why:* there's no nightly recrawl, staleness doesn't pile up, and bulk imports update the index as well. By design, this stops another 60-day incident from happening.

Query (for each request):

1. **Look in the semantic cache** first.
2. **Embed the question** using the *same* embedding model that embedded the chunks (different models live in different coordinate systems).
3. **Retrieve with access control.** Search the sharded index (routed by tenant where that applies) for the top-k candidates (say, top-50), **filtered by the requester's ACL tags at the moment of retrieval**. *Why:* the model must never be given content the user isn't allowed to read. Telling the model "don't reveal confidential info" is **not** access control. Authorization has to happen before any content goes into the context.
4. **Rerank.** Vector search is a quick, broad stage for recall. A **cross-encoder reranker** reads the query and each chunk *side by side*. It's slower but more accurate, and it chooses the best **5**, for about **40 ms**. *Why:* a cheap filter followed by a costly judge beats a single stage of search.
5. **Build the prompt** out of the chunks, their sources, and the question. Instruct the model to answer only from that context, cite its sources, and say "I don't know" when the context doesn't cover it.

The citations are the **audit trail** that lets people review answers when there's a dispute.

6. **Generate** via the LLM gateway (fallbacks, budgets) and stream the reply back over SSE.

The eval discipline that makes RAG manageable: measure each stage on its own. **Retrieval quality** asks, "did we fetch the right chunk?" It's measured with **recall@k**, the fraction of the correct chunks that show up in the top k results. **Generation faithfulness** asks, "did the answer really come from those chunks?" These are separate failures with separate fixes. Most tickets saying "the bot gave a wrong answer" are really **retrieval failures dressed up as generation failures**.

The whole design is built from earlier patterns. Ingestion uses CDC + stream processing + the index as a read model. The query path uses the semantic cache, sharded retrieval, the gateway, batched serving on autoscaled GPUs, and SSE.

Walkthrough with numbers

- *The incident, replayed:* the policy edit hits the CMS at **14:02:11**. CDC emits the change at **14:02:15**. Three chunks are re-embedded and re-indexed by **14:02:19**. Freshness: **about 8 seconds** from the edit to a servable truth, versus weeks for fine-tuning.
- *Query latency budget:* the cache check takes **10 ms** (and answers **27%** of questions). On a miss: embed **10 ms** + sharded retrieval **25 ms** + rerank **40 ms** + generation about **2 s**. Retrieval adds roughly **75 ms to a 2,000 ms answer (<4%)**.
- *Economics:* retrieval costs about **1/10** of what generation costs. Also, supplying outside knowledge can let a *smaller* model match the quality of a bigger one, because it uses its capacity for reasoning instead of for memorized facts.
- *Eval dashboard:* **recall@5 = 91%** (retrieval) and **faithfulness = 96%** (generation), each measured and alarmed on its own. So when quality drops, you can ask "which stage?"

Trade-offs

- *You gain:* knowledge you can update (edit the doc and the assistant knows within seconds, no retraining); citations (for trust, audits, and settling disputes: "section 3, updated March 4th"); access control that really works (ACLs applied at retrieval, per asker); and the option of smaller models.
- *You pay:* **retrieval caps quality** (if you fetch garbage, the model faithfully generates garbage; chunking, embedding, and reranking are skills you'll practice forever); **two pipelines to run** (CDC health, re-embedding lag, and replication-slot disk on ingestion; a four-stage latency budget on query); **hallucination reduced, not removed** (a *hallucination* is a claim the context doesn't support; the model can still ignore its context); and **a permanent two-stage eval workload**.

When not to use it

- *Behavior problems* (tone, format, style, reasoning patterns): that's what fine-tuning is for.
- *Small, stable corpora:* a 50-page handbook fits in the context, and prompt caching lowers the cost of repeating the prefix.
- *General knowledge* ("capital of France?"): parametric memory (knowledge stored in the weights) handles stable public facts fine.
- *Latency budgets below ~100 ms:* there isn't enough time to embed, retrieve, rerank, and generate.
- *Classic misuse:* reaching for RAG by habit when 12 PDFs would fit in one prompt. Rule: **RAG for knowledge, fine-tuning for behavior, long context for small stable corpora**, combined when the needs overlap. (A fine-tuned model that receives RAG context is a common enterprise design.)

Comparison: RAG vs. fine-tuning vs. long context

	RAG	Fine-tuning	Long context (paste it all)
Knowledge freshness	Seconds (fed by CDC)	Tied to the training schedule: weeks	Fresh (paste again on each change)
Citations	Built in: chunks carry their sources	None	Possible, but unstructured
Access control	ACLs at retrieval time: real	Impossible inside weights	All or nothing for each prompt
Cost profile	The index + ~10% per query	Training runs + hosting	Tokens × corpus size × every query
What it's for	Knowledge: large, changing, access-controlled	Behavior: tone, format, reasoning style	Small, stable corpora (with prompt caching)

These are layers, not competitors. Fine-tuning shapes the voice, RAG provides current facts, and prompt caching makes a stable preamble cheap.

Real-world examples

- **Every enterprise assistant** (support bots, internal knowledge tools, **Glean**-class products) uses this architecture. RAG is the standard shape of "LLM + your data."
- **Code assistants** run it with the repository (the project's version-controlled code) as the **corpus** (the body of documents being searched), chunking along function and file structure.
- **Agentic retrieval** layers on query rewriting, multi-hop retrieval (find a policy, then fetch its exceptions), and letting the model decide when to retrieve, calling the controlled pipeline through a **planner** (the agent component that decides the next step).

In the interview. "Design a chatbot over your company's documents" is *the* signature AI system-design question. Its follow-ups are predictable: freshness, permissions, and debugging bad answers.

> **30-second answer:** "RAG is two pipelines. Ingestion watches the content sources with CDC, so an edit re-chunks and re-embeds just the changed pieces within seconds. It uses structure-aware chunking, because bad chunks cap everything downstream, and writes into a sharded vector index with metadata and ACL tags. The query path checks the semantic cache, embeds the question, retrieves the top-k filtered by the requester's permissions (so the model never sees what the user can't), reranks with a cross-encoder, and generates a cited answer through the gateway. I evaluate recall@k and faithfulness separately and debug retrieval first. RAG for knowledge, fine-tuning for behavior, long context for small stable corpora."

Likely follow-ups:

- **"A user asks about a document they can't access. What happens?"** The retrieval filter leaves it out. Its chunks never make it into the candidate set, the model never sees them, and the answer is an honest "I don't have information on that." Candidates who enforce this with a prompt instruction fail: instructions to a text generator aren't access control.
- **"Answers got worse last week. Debug it."** Split the stages apart and check **retrieval first**: recall@k on a fixed evaluation set where the correct chunks are known. If retrieval looks fine, check faithfulness (is the model ignoring its context?). Then look at chunking changes, embedder version drift, and ACL configuration that might be leaving candidate sets empty.

Mistakes that fail candidates: fine-tuning for freshness; ACLs enforced by prompt; no plan for citations; brushing off chunking as a detail; a single quality metric covering both stages.

Self-check:

- **Q1:** Name the two pipelines and their stages, and explain what keeps ingestion fresh without recrawling. → Ingestion: sources → structure-aware chunking → embedding → sharded vector index with metadata + ACL tags, kept fresh by CDC, which reprocesses only changed chunks within seconds. Query: semantic cache → embed the question → ACL-filtered top-k retrieval → cross-encoder rerank → prompt with citation rules → generation through the gateway, streamed.
- **Q2:** Choose the tool: (a) a 50-page handbook updated once a year; (b) a changing knowledge base of 500K documents with permissions per team; (c) "answer in our brand voice." → (a) long context + prompt caching; (b) full RAG (CDC freshness, ACLs at retrieval time, citations); (c) fine-tuning (voice is a behavior). They can be combined: a fine-tuned model that gets its facts from RAG.
- **Q3:** Users report confidently wrong answers about warranties, yet faithfulness shows 96%. → The model is faithfully using the *wrong* chunks. It's a retrieval failure that looks like a generation failure. Run recall@k on a warranty eval set. Then check: chunking that's shredding warranty tables; an embedder upgrade without re-indexing (retrieval across coordinate systems); CDC connector lag or a stall (a stale index); and misconfigured ACL tags filtering out the right chunks. Principle: **generation sits downstream of retrieval, so grade retrieval first.**

Related patterns: Change Data Capture (keeping ingestion fresh); Vector Database Sharding (the index at scale, keyed by tenant); Semantic Caching (the front tier; 27% of questions never get to retrieval); LLM Gateway (generation's door: budgets, fallbacks, streaming).

TL;DR

- *Problem:* knowledge in the weights can't be updated, can't be cited, and can't be access-controlled, and fine-tuning fixes none of the three quickly.
- *Mechanism:* CDC-fed ingestion (structure-aware chunking → embedding → a sharded, ACL-tagged index) plus a query funnel (cache → ACL-filtered retrieval → rerank → cited generation through the gateway), evaluated stage by stage (recall@k, then faithfulness).
- *Costs:* retrieval sets the ceiling, two pipelines to run, hallucination reduced but not removed, and two-stage evals.
- *Skip it when:* the problem is behavior, the corpus is small and stable, or the knowledge is public and general.

Capstone — A Production RAG Chatbot at Scale

Original lesson: Capstone - A Production RAG Chatbot at Scale on DesignGurus ↗

What this capstone is. You'll design the store's production assistant for roughly **5 million users** plus B2B partners, where every request costs actual money. The capstone also puts the module's thesis to the test. At each stage, identify **which earlier module provides each mechanism**. The finished design should consist almost entirely of patterns you already know. (The course asks you to stop at each "diagnose" prompt and make the decisions yourself before you keep reading.)

The brief (requirements)

- **Scale: 2,000** concurrent conversations at peak and **30 requests/second** to the models. It's multi-tenant: the store's own customers plus **200 B2B partners** who embed the assistant under their own brands.
- **Knowledge:** the catalog + policies (RAG), plus live order status (personal state, which is a different problem).
- **Experience:** answers that stream and include citations, and conversations that survive reconnects.
- **Money:** partners are billed by usage, internal features get budgets, and the **$40K** runaway loop must never recur.

- **Compound requirement:** remain useful during a provider brownout *at the same time* as a traffic spike.

Stage 1 — Knowledge plane. *Challenge:* catalog and policy content changes all the time, and each partner's knowledge base is private.

- **Decision:** CDC watches the CMS and catalog tables. Edits are re-chunked (structure-aware) and re-embedded within seconds. *Why:* freshness in seconds, with no recrawls (Module 10).
- **Decision:** the index is **sharded and keyed by tenant**. Every partner query includes a tenant filter, so retrieval touches a single shard, and the **2 largest partners get dedicated shards**. *Why not uniform scatter-gather:* the filter is a natural shard key. It brings back single-shard routing and isolates giant tenants (Module 3 sharding).
- **Decision:** every chunk has **ACL tags**, which makes filtering at retrieval time the security boundary. *Why:* prompts aren't access control.
- **Decision:** the index is a read model you can rebuild (CQRS, Module 3). The embedder is versioned like a schema, with **dual-index migrations** (the zero-downtime playbook, Module 9). That's four modules in use before a single question gets answered.

Stage 2 — Conversation plane. *Challenge:* streaming token by token, users who reconnect after their network drops, and follow-up turns that need the history.

- **Decision:** stream using **SSE**. (Server-sent events flow one way, from server to client, over plain HTTP, and reconnect automatically using `Last-Event-ID`.) *Why not WebSocket* (a two-way connection that stays open): the client never sends anything mid-answer, so a two-way transport gains nothing (Module 2).
- **Decision: sticky routing by conversation ID on the consistent-hash ring**, so later turns reach the replica that holds the warm KV cache. *Why:* this is Constant 2's stickiness, built with Module 3's ring (and only ~1/N of conversations move when replicas change).
- **Decision: the stream is only transport; the history store is the source of truth.** Finished messages are saved in a database. After a disconnect, the server replays from saved history, not from whatever stream state was left over (Module 2's resume design). This plane adds no new patterns.

Stage 3 — Serving plane. *Challenge:* 30 req/s, 100x variation in output length, the $40K incident, and a marketing calendar. Following one request from front to back:

1. The **semantic cache** answers **more than 25%** of questions in **40 ms**, using a cautious threshold, entity guards, and sampled audits (Module 4, with fuzzy keys).
2. Misses go to the **LLM gateway**: provider keys, **token budgets per tenant with hard caps** (so there's no second $40K), logical model routing, and the fallback ladder (Module 8 gateway + rate limiting; Module 5 degradation).
3. The **self-hosted fleet** uses continuous batching (the measured **30x** cost saving). When it's full, overflow goes to provider APIs through the gateway.
4. The fleet scales on **queue depth**, with **warm pools sized at spike rate × boot time**, pre-warmed using marketing's calendar (Module 6, at 100x prices; scheduled scaling works as before). That's six modules in a single request path.

Stage 4 — Money plane. *Challenge:* partners pay per token. Where should metering live, and what guarantee does each consumer need?

- **Decision:** the gateway meters every call (tokens in/out, model, tenant, feature) and publishes usage events to a **log partitioned by tenant** (Module 2). *Why the gateway:* it's the one spot every call passes through.

- **Decision:** live spend **dashboards** aggregate using **idempotent upserts** (insert-or-update writes where repeating one changes nothing). They're forgiving and fresh.
- **Decision: invoices** use **exactly-once** machinery (checkpointed aggregation, transactional sinks), because partners audit their bills (Module 10: choose the guarantee for each consumer).
- **Decision:** event IDs are created **on the client before the first send**, which gives an unbroken idempotency chain from the SDK all the way to the invoice (Module 5).

Stage 5 — Personal-state boundary. *Challenge:* "Where's MY order?" is the question asked most often. It can't be cached, and it must never be in the RAG index. And what should happen when someone says "cancel that order"?

- **Decision:** personal state is handled by a **tool call**, such as `get_order_status(order_id)`. An ordinary request-response API pulls the authoritative state from the orders system, and the model just formats it. *Why:* the index holds shared knowledge, while the source systems hold current personal state (Module 2).
- **Decision:** "cancel it" is a write with consequences. So the tool call carries an **idempotency key derived from the tool-call ID** (frameworks retry, and agents change their minds). The cancel then serves as the entry point into the orders domain's own **saga** (a multi-step flow with compensations). The assistant starts it; the domain orchestrates it. Rule (the BFF rule, restated for agents): **presentation composes reads; domains own writes** (Modules 5, 7).

Stage 6 — The bad day (failure walk). It's Tuesday, 11 AM. Marketing's segment airs and traffic triples (**3x**). At 11:04 the primary provider browns out.

1. **11:00:** the calendar had already pre-warmed the pool, so the surge hits GPUs that are ready, while queue-depth scaling boots replacements in the background.
2. **11:04:** the breaker for each provider-model pair opens after about **20 s** of errors. The configured chain sends overflow to the **secondary provider** and `classify`-tier work to the **self-hosted small model**.
3. **The semantic cache carries the heaviest load:** it's already serving ~25%, and it soaks up the trending questions that every spike brings ("is the sale still on?"). One generation serves **10,000** askers.
4. **Budgets per tenant** keep failover from doubling anyone's spending.
5. **What users see:** slightly slower answers and a single labeled banner.
6. **No human decisions needed:** every branch was configuration written on a quiet afternoon (degradation tiers, gateway chains, the autoscaler calendar). The on-call engineer watches dashboards confirm the plan.

Final architecture

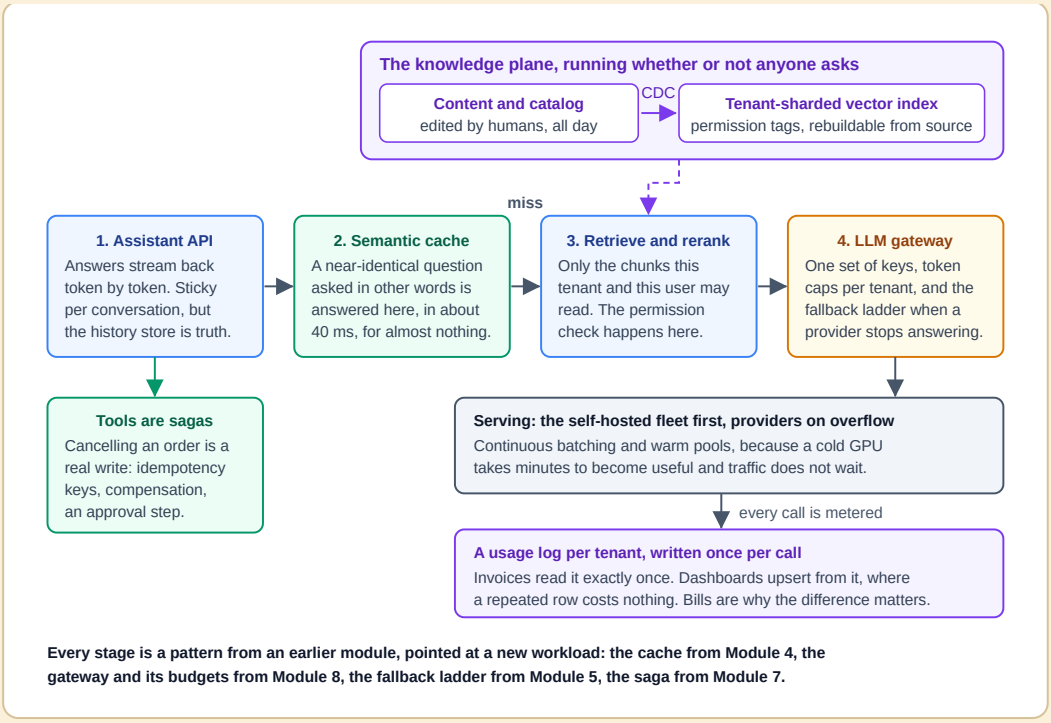


Figure — RAG chatbot at scale: final architecture (course diagram)

The figure above brings the planes together. On one side is the knowledge index, fed by CDC and sharded by tenant. Then there's the SSE conversation layer, with ring stickiness and a history store. The serving path runs semantic cache → gateway → batched GPU fleet, with overflow to APIs. The metering log feeds dashboards and invoices. And tool calls reach into the orders domain.

Debrief: the tally

Plane	Decisions	Modules doing the work
Knowledge	CDC for freshness, tenant shards, ACL-filtered retrieval, dual-index migrations	10, 3, 9
Conversation	SSE + resume, stickiness on the ring, history as the truth	2, 3
Serving	Cache → gateway → ladder → batching → warm pools	4, 8, 5, 6, 11
Money	Metering on a partitioned log; upserts or exactly-once depending on the audience	2, 10, 5
Personal state	Tool calls, idempotency keys, saga boundaries	2, 5, 7
The bad day	Breakers, tiers, calendars: configuration instead of war rooms	5, 6, 8, 9

The right-hand column covers **every module in the course apart from the capstones**. The truly AI-specific pieces are few: embeddings, similarity thresholds, and token meters. That backs up the module's thesis: AI systems reuse well-known patterns under three changed constants.

Extend it yourself (follow-up questions)

- **Agents:** the assistant takes on multi-step tasks ("find me a cheaper alternative and set a price alert"). Which boundaries do the agent's tool calls strain the most (saga pivots? budget caps?), and what would a **durable-execution** answer look like (a workflow engine that saves each step so a long task can resume after a crash)?
- **Voice:** add speech. What transport takes over from SSE (bidirectional streaming)? What happens to the latency budget when TTFT turns into time-to-first-*syllable*? Which plane stays exactly the same?

- **Multi-region:** partners demand EU **data residency** (EU users' data must be stored and processed inside the EU). Which planes can replicate without coordination? Where does the compliance design of the tenant-sharded index help most? Which component (the billing log) needs the strongest guarantees from the consistency module?

Interview one-liner: "Knowledge comes in through CDC into a tenant-sharded, ACL-tagged index. Conversations stream over SSE, with history as the truth and ring stickiness for KV caches. Requests flow semantic cache → gateway with hard budgets and a fallback ladder → a batched, warm-pooled GPU fleet that overflows to APIs. Metering travels on a log, exactly-once for invoices. Personal state and actions go through idempotent tool calls into domain sagas. Every bad-day response is configuration written ahead of time."